面向21世纪计算机专业本科系列教材

数字逻辑

Digital logic

（第二版）

欧阳星明　主编



华中科技大学出版社
http://press.hust.edu.cn

《数字逻辑》教学资源

★教辅教材《数字逻辑学习与解题指南》

★互动型数字逻辑虚拟实验平台(IVDLEP)

★多媒体CAI课件(教师版)

★多媒体网络课件

★多媒体视频课件

★数字逻辑试题库













ISBN 7-5609-3344-0

9 787560 933443 >

定价：29.80元(附赠光盘一片)

面向21世纪计算机专业本科系列教

数字逻辑

（第二版）

主编　欧阳星明
编者　欧阳星明　陈传波　章　勤　李　昕

华中科技大学出版社

图书在版编目(CIP)数据

数字逻辑(第二版)/欧阳星明 主编
武汉:华中科技大学出版社,2005年3月
ISBN 7-5609-3344-0

Ⅰ.数 …
Ⅱ.欧阳…
Ⅲ.数字逻辑-高等学校-教材
Ⅳ.TP302.2

数字逻辑(第二版) 欧阳星明 主编

责任编辑:沈旭日 封面设计:潘 群
责任校对:刘 飞 责任监印:熊庆玉

出版发行:华中科技大学出版社
武昌喻家山 邮编:430074 电话:(027)87557437

录 排:华中科技大学惠友文印中心
印 刷:湖北省新华印务有限公司

开本:787×960 1/16 印张:23.25 字数:400 000
版次:2005年3月第2版 印次:2005年3月第13次印刷 定价:29.80元
ISBN 7-5609-3344-0/TP·558 (附赠光盘一片)

(本书若有印装质量问题,请向出版社发行部调换)

内 容 简 介

本书以飞速发展的数字集成电路为背景，将数字电子技术和数字逻辑电路的有关知识融为一体，系统地介绍了数字系统逻辑电路分析与设计的基本知识、理论和方法，并阐述了采用各种不同规模的逻辑器件进行分析与设计的详细过程。全书内容包括基本知识、逻辑代数基础、集成门电路与触发器、组合逻辑电路、同步时序逻辑电路、异步时序逻辑电路、中规模通用集成电路、可编程逻辑器件和综合设计举例等内容。

本书体系新颖、结构合理、取材先进、文句精练、题例丰富。随书配有光盘一片，内含实验演示和学习自评，并编写了与教材配套的教辅教材《数字逻辑学习与解题指南》。此外，研究开发了相应的数字逻辑虚拟实验软件、多媒体 CAI 课件、远程网络课件、视频课件和试题库等，构成了一个集理论教学、实践教学以及自学、自测等环节为一体的立体化教学系统。

本书可作为高等学校计算机、电子工程、自动化、通信等专业"数字逻辑"课程的教材，也可作为成人教育相关课程的教材，并可供相关专业科技人员参考。

面向21世纪计算机教材出版指导委员会

主任　陈火旺　沈绪榜　邹寿彬

委员　（以姓氏笔画为序）

王长胤　韦　敏　卢开澄　卢正鼎

齐　欢　刘太林　何炎祥　张　峰

杨传谱　苏锦祥

秘书　沈旭日

面向21世纪计算机专业本科系列教材编委会

主任　何炎祥　卢正鼎

委员　（以姓氏笔画为序）

卢炎生　肖德宝　陈　珉　张　峰

贺贵明　姜新祺　熊前兴

秘书　沈旭日



总 序

自 1946 年世界上第一台电子数字计算机 ENIAC 诞生以来，计算机硬件系统经过了电子管、晶体管、小规模集成电路和大规模集成电路等几个阶段，正遵循着摩尔定律高速地发展：1998 年，速度最快的个人 PC 微处理器是 Intel 450MHz 的 Xeon，1999 年速度最快的已达 800MHz；1997 年 2.1GB 的磁盘容量已经很不错了，1999 年则已突破 10GB……软件方面，无论是操作系统、数据库系统，还是编程语言、应用软件，更是频繁地更新换代，令人眼花缭乱。

与此同时，作为计算机与通信技术结合的产物——计算机网络得到了迅速发展，特别是 Internet 技术的广泛应用，使得计算机网络的规模越来越大，网上主机数目一直保持每 3 年增长 10 倍的速率，Internet 上的数据流量则保持着平均每半年就翻一番的增长速率，信息网络已交叉纵横整个世界，将偌大的世界连成了一个“地球村”。

计算机技术日新月异的进步，对现有的计算机专业的教学模式提出了挑战，同时也带来了前所未有的机遇。深化面向 21 世纪的教学改革，寻求一条行之有效的途径，培养跨世纪的高素质的科技人才，已是当务之急。如果说教学内容、课程体系的改革是教学改革的重点和难点，那么，教材建设则是其不可或缺的重要组成部分。华中理工大学出版社敏锐地抓住了这一点，在其倡议和组织下，我们经过研究、讨论和对教学经验进行总结，规划了这套“面向 21 世纪计算机教材”。为了满足各级各类学校人才培养的需要，这套教材计划包括计算机专业类教材和非计算

机专业类教材，从层次上则可划分为研究生层次、本科生层次、高职高专层次、中职中专层次、中小学层次等若干个子系列，将陆续分批出版。

当今世界，信息革命方兴未艾，知识经济已见端倪，教育观念正面临从注重以知识为主体向以能力为主体的转变。我们在对教材进行规划和评审时，尤其注重把提高学生素质、培养学生的应用能力和创新能力作为首要的评价标准，同时注意教材的特色和教学的实用性，反映最新的教学和科研成果，体现时代特征。

限于水平和经验，这批教材的编写、出版还存在不足，希望使用教材的学校、教师和学生以及其他读者积极提出批评意见，以便我们及时更新、修订，以满足读者要求。

面向21世纪计算机教材出版指导委员会主任

陈火旺（中国科学院院士，国防科技大学教授）

沈绪榜（中国科学院院士，华中理工大学教授）

邹寿彬（华中理工大学教授，副校长）

2000年2月10日

面向21世纪计算机专业本科系列教材

序

人们已普遍认识到:21世纪是信息时代,以计算机为核心的信息技术是21世纪科技发展的大趋势。那么,作为计算机专业人才培养基地的大学计算机专业,如何适应这种发展,培养出符合时代要求、具有创新能力的人才呢?这是近年来计算机教育界讨论的热门话题,也是我们长期思考并努力探索的课题。

教材是人才培养的基础。在华中理工大学出版社的倡议和委托下,我们自1998年下半年起就开始讨论、筹划编写一套适应21世纪人才培养需要的计算机本科专业系列教材。在此基础上,我们组织了武汉大学、华中理工大学、华中师范大学、武汉测绘科技大学、武汉水利电力大学、武汉交通科技大学等院校的部分教师共同编写了这套"面向21世纪计算机系列教材",以期总结我们在教学内容和课程体系改革方面的体会和做法,在适应21世纪的教材建设方面作出自己的努力。

值得欣慰的是,在教材的编写过程中,全国计算机专业教学指导委员会、中国计算机学会教育委员会联合推出了"计算机学科教学计划2000"(简称"2000教程"),这就更增强了我们编好这套教材的信心。在编写过程中,我们吸收了其中与我们内容相异的新内容。因此,也完全可以说,这套教材是与"2000教程"完全配套的教材。

我们这套系列教材的编写计划分为两个阶段:第一阶段,在2000年内出版"2000教程"中所涉及的所有专业课和部分专业基础课教材;第二阶段,在2000年以后出版与这套教材相配套的实践课和实验课教材,以及教学辅导书。

我们希冀这套教材具有以下特点:

1. 基础性和先进性相结合。与其他学科相比,计算机学科的一个显著特点就是知识内容更新更快,这对教学内容的选取、课程知识结构的构建提出了挑战。基于大学教育应努力实现知识、能力、素质三者辨证统一的目标,我们把编写的重点放在基础知识、基本技能和基本方法上,希望提高学生的理论素养和分析问题、解决问题的能力;与此同时,注重介绍最新的技术和方法,以拓展学生的知识面,激发他们学习的积极性和创新意识。

2. 理论性与应用性相结合。理论是规律的表现形式,良好的理论素养是应用的前提,而掌握理论的目的就是应用。在教材的编写过程中,我们注意了理论的系

统性，在讲深讲透主要知识的基础上，各门课程知识点的选取做到尽量广一些；融理论性和应用性于一体，在阐述理论的同时，尤其注意理论方法的讲授，以培养学生应用理论和技术的能力。此外，精心设计了比较多的习题，以加强应用能力和创造能力的培养。

3. 时代性和实用性相结合。力求精简旧的知识点，增加新的知识点，使整个知识建立在“高”、“新”平台上，体现教材的时代特征。但是，并不片面追求“高”、“新”，而是实事求是地充分考虑一般高校目前所拥有的教学设备、师资条件，注重教材的实用性。我们以为，教材建设不可能毕其功于一役，而必须根据学科的发展和客观环境以及条件的变化不断努力和改进。需要说明的是，与“2000 教程”相比，我们根据人的认识规律和教学安排的需要，将有些课程进行了划分或合并，以便于教师根据需要灵活安排。

4. 科学性与通俗性相结合。概念原理、新技术的阐述力求准确、精练；写作风格上尽量通俗易懂、深入浅出、图文并茂，增加可读性，便于学生自学。

如果说科学技术快速发展是 21 世纪的一个重要特征的话，那么，教学改革将是 21 世纪教育工作永恒的主题，是需要不断探索的课题。我们要达到以上目标，还需要不断地努力实践和完善。欢迎使用这套教材的教师、学生和其他读者提出宝贵意见。

最后，衷心感谢参加这套教材编写的所有作者所贡献的成果和辛勤的汗水，对为这套教材的编写提供支持的有关学校、院系的领导和老师表示诚挚的谢忱！感谢华中理工大学出版社为本系列教材的出版所付出的艰辛和努力！

面向 21 世纪计算机专业本科系列教材编委会主任

何炎祥（武汉大学教授）

卢正鼎（华中理工大学教授）

1999 年 11 月 20 日

第二版前言

时代的发展和技术的进步，对传统的教育体系和人才培养模式提出了新的挑战。面向21世纪的高等教育正在对专业结构、课程体系、教学内容和教学方法进行系统的、整体的改革，教材建设是改革的重要内容之一。我们正处在一个信息的时代，各行各业对信息学科人才的需求越来越大。如何为社会培养更多的具有创新能力、解决实际问题能力和高素质的信息学科人才，是目前高等教育的首要任务之一。

“数字逻辑”是信息学科各专业本、专科学生必修的一门专业技术基础课。设置本课程的主要目的是：使学生掌握数字系统分析与设计的基本知识与理论，熟悉各种不同规模的逻辑器件，掌握各类逻辑电路分析与设计的基本方法，为数字计算机和其他数字系统的硬件分析与设计奠定坚实的基础。为满足教学的需求，国内外出版了大量相关的教科书，这些教科书各具特色，其中有许多被公认是十分优秀的作品。然而，该领域的教科书一般都受摩尔定律影响，即便是最优秀的教科书也必须不断更新、完善。本教材第一版是参照全国高校计算机专业教学指导委员会、中国计算机学会教育委员会与全国高等学校计算机教育研究会联合推荐的《计算机学科教学计划2000》的指导思想，从传授知识和培养能力的目标出发，结合作者长期从事教学与科研的经验及本课程教学的特点、要点和难点编写的；2000年出版发行之后受到了广大读者的关爱，在4年多的时间里先后12次印刷。然而，随着信息技术的飞速发展和教学改革的不断深入，在教材使用过程中我们深感其仍存在许多不尽人意的地方，常常不免有愧对读者之感，希望不断加以改进和完善。本次修订就是在第一版的基础上进行的。

数字集成电路是数字计算机和各类数字系统功能实现的物质基础，本教材以高速发展的数字集成电路为纽带，将数字电子技术和数字逻辑的有关知识融为一体，较完整地阐述了各种不同规模的数字集成电路及其在数字系统逻辑设计中的应用，力图使学生在掌握逻辑设计基本理论和方法的基础上，了解数字器件的更新换代对数字系统设计方法产生的重要影响以及新的发展趋势、不断掌握新的技术，以适应数字技术快速发展的需要。全书共分9章，内容可归纳为四大部分。第一部分主要介绍数字系统逻辑设计的基本知识、基本理论和基本逻辑器件，由1～3章组成；第二部分以小规模集成电路为基础，详细讨论了组合逻辑电路和时序逻辑电路分析与设计的经典方法，由4～6章组成；第三部分重点介绍常用中规模通用

集成电路、大规模可编程逻辑器件及其在逻辑设计中的应用，包括常用中规模组合逻辑电路、中规模时序逻辑电路、信号产生与变换电路、常用可编程逻辑器件以及20世纪90年代出现的ISP技术等内容，由7、8两章组成；第四部分综合运用该课程所学知识，进行了实际问题设计举例，意在进一步将理论知识与实际应用紧密结合，达到学以致用的目的，在第9章完成。本教材的本科教学参考学时数为80学时(含16学时左右实验)，不同专业和不同层次可按课程学时数的多少和实际需求，由任课教师根据具体情况对教材内容，尤其是对标题前注有星号(*)的部分进行适当取舍。

为了满足教学与自学的需要，随书配有光盘一片，内含实验演示和学习自评，并编写了与教材配套的辅导教材《数字逻辑学习与解题指南》。此外，还研究开发了相应的数字逻辑虚拟实验软件、多媒体CAI课件、远程网络课件、视频课件和试题库等丰富的教学资源，构成了一个集理论教学、实践教学以及自学、自测等环节为一体的立体化教学体系。使用者可根据需要对各项教学资源灵活选用。

本书由欧阳星明主编，陈传波、章勤和李昕参与了部分编写工作。在本书的编写过程中，得到了华中科技大学计算机学院领导和同事的许多关心，“数字逻辑”课程教学小组的秦磊华、孙百勇、刘景宁、唐九飞等老师，以及外校的雷晓平、王槐斌、喻成等老师对该书内容的组织提出了许多宝贵的意见，华中科技大学出版社为本书的出版给予了大力支持，在此一并表示衷心感谢。

由于编者水平有限，书中缺点、错误难免，殷切希望广大读者批评指正。

编　者

2004年12月于华中科技大学

目　录

第 1 章　基本知识 …… (1)

1.1　概　述 …… (1)

1.1.1　数字系统 …… (1)

1.1.2　数字逻辑电路的类型和研究方法 …… (4)

1.2　数制及其转换 …… (6)

1.2.1　进位计数制 …… (6)

1.2.2　数制转换 …… (9)

1.3　带符号二进制数的代码表示 …… (12)

1.3.1　原码 …… (13)

1.3.2　反码 …… (14)

1.3.3　补码 …… (15)

1.4　几种常用的编码 …… (16)

1.4.1　十进制数的二进制编码 …… (16)

1.4.2　可靠性编码 …… (18)

* 1.4.3　字符编码 …… (21)

习题一 …… (22)

第 2 章　逻辑代数基础 …… (23)

2.1　逻辑代数的基本概念 …… (23)

2.1.1　逻辑变量及基本逻辑运算 …… (24)

2.1.2　逻辑函数及逻辑函数间的相等 …… (27)

2.1.3　逻辑函数的表示法 …… (28)

2.2　逻辑代数的基本定理和规则 …… (29)

2.2.1　基本定理 …… (29)

2.2.2　重要规则 …… (31)

2.2.3　复合逻辑 …… (33)

2.3　逻辑函数表达式的形式与变换 …… (35)

2.3.1　逻辑函数表达式的基本形式 …… (35)

2.3.2 逻辑函数表达式的标准形式 …… (36)
2.3.3 逻辑函数表达式的转换 …… (39)
2.4 逻辑函数化简 …… (42)
2.4.1 代数化简法 …… (42)
2.4.2 卡诺图化简法 …… (44)
* 2.4.3 列表化简法 …… (55)
习题二 …… (59)

第 3 章 集成门电路与触发器 …… (62)

3.1 数字集成电路的分类 …… (62)
3.2 半导体器件的开关特性 …… (64)
3.2.1 晶体二极管的开关特性 …… (64)
3.2.2 晶体三极管的开关特性 …… (68)
3.3 逻辑门电路 …… (71)
3.3.1 简单逻辑门电路 …… (71)
3.3.2 TTL 集成逻辑门电路 …… (74)
3.3.3 CMOS 集成逻辑门电路 …… (87)
3.3.4 正逻辑和负逻辑 …… (92)
3.4 触 发 器 …… (94)
3.4.1 基本 R-S 触发器 …… (95)
3.4.2 常用的时钟控制触发器 …… (100)
* 3.4.3 不同类型时钟控制触发器的相互转换 …… (112)
习题三 …… (115)

第 4 章 组合逻辑电路 …… (118)

4.1 组合逻辑电路分析 …… (118)
4.1.1 分析方法概述 …… (119)
4.1.2 分析举例 …… (119)
4.2 组合逻辑电路设计 …… (122)
4.2.1 设计方法概述 …… (122)
4.2.2 设计举例 …… (123)
4.2.3 设计中几个实际问题的处理 …… (127)
4.3 组合逻辑电路的险象 …… (136)
4.3.1 险象的产生 …… (136)

4.3.2　险象的判断 …… (138)
4.3.3　险象的消除 …… (140)
习题四 …… (142)

第5章　同步时序逻辑电路 …… (144)

5.1　时序逻辑电路概述 …… (144)
5.1.1　时序逻辑电路的结构 …… (144)
5.1.2　时序逻辑电路的分类 …… (145)
5.1.3　同步时序逻辑电路的描述方法 …… (147)
5.2　同步时序逻辑电路分析 …… (149)
5.2.1　分析方法和步骤 …… (149)
5.2.2　分析举例 …… (150)
5.3　同步时序逻辑电路设计 …… (157)
5.3.1　设计的一般步骤 …… (157)
5.3.2　完全确定同步时序逻辑电路设计 …… (159)
* 5.3.3　不完全确定同步时序逻辑电路设计 …… (174)
5.3.4　同步时序逻辑电路设计举例 …… (180)
习题五 …… (187)

第6章　异步时序逻辑电路 …… (191)

6.1　异步时序逻辑电路的特点与分类 …… (191)
6.2　脉冲异步时序逻辑电路 …… (192)
6.2.1　脉冲异步时序逻辑电路的结构模型 …… (192)
6.2.2　脉冲异步时序逻辑电路的分析 …… (192)
6.2.3　脉冲异步时序逻辑电路的设计 …… (196)
6.3　电平异步时序逻辑电路 …… (202)
6.3.1　电平异步时序逻辑电路的结构模型与描述方法 …… (202)
6.3.2　电平异步时序逻辑电路的分析 …… (205)
6.3.3　电平异步时序逻辑电路的竞争 …… (207)
* 6.3.4　电平异步时序逻辑电路的设计 …… (209)
习题六 …… (222)

第7章　中规模通用集成电路及其应用 …… (226)

7.1　常用中规模组合逻辑电路 …… (226)

7.1.1 二进制并行加法器 …… (226)
7.1.2 译码器和编码器 …… (231)
7.1.3 多路选择器和多路分配器 …… (239)
7.2 常用中规模时序逻辑电路 …… (246)
7.2.1 集成计数器 …… (246)
7.2.2 集成寄存器 …… (253)
7.3 常用中规模信号产生与变换电路 …… (256)
7.3.1 集成定时器 555 及其应用 …… (256)
7.3.2 集成 D/A 转换器 …… (265)
7.3.3 集成 A/D 转换器 …… (270)
习题七 …… (273)

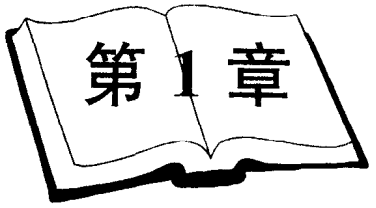
第 8 章 可编程逻辑器件 …… (276)

8.1 PLD 概述 …… (276)
8.1.1 PLD 的发展 …… (276)
8.1.2 PLD 的基本结构 …… (277)
8.1.3 PLD 的电路表示法 …… (277)
8.2 常用的可编程逻辑器件 …… (279)
8.2.1 可编程只读存储器 …… (279)
8.2.2 可编程逻辑阵列 PLA …… (285)
* 8.2.3 可编程阵列逻辑 PAL …… (290)
8.2.4 通用阵列逻辑 GAL …… (298)
8.3 在系统编程技术简介 …… (310)
8.3.1 ISP 技术的主要特点 …… (310)
8.3.2 ISP 逻辑器件 …… (311)
8.3.3 ISP 器件的开发软件与设计流程 …… (317)
习题八 …… (320)

*** 第 9 章 综合应用举例** …… (322)

9.1 简单运算器设计 …… (322)
9.1.1 设计要求 …… (322)
9.1.2 功能描述 …… (322)
9.1.3 电路设计 …… (323)
9.2 时序信号发生器设计 …… (325)

9.2.1 设计要求 …… (325)
9.2.2 功能描述 …… (326)
9.2.3 电路设计 …… (327)
9.3 地址译码电路设计 …… (330)
9.3.1 设计要求 …… (330)
9.3.2 功能描述 …… (330)
9.3.3 电路设计 …… (331)
9.4 弹道计时器设计 …… (335)
9.4.1 设计要求 …… (335)
9.4.2 功能描述 …… (335)
9.4.3 电路设计 …… (336)
9.5 汽车尾灯控制器设计 …… (339)
9.5.1 设计要求 …… (339)
9.5.2 功能描述 …… (339)
9.5.3 电路设计 …… (341)
9.6 数字钟设计 …… (344)
9.6.1 设计要求 …… (344)
9.6.2 功能描述 …… (344)
9.6.3 电路设计 …… (345)
习题九 …… (349)
参考文献 …… (350)
《数字逻辑》教学资源简介 …… (351)



基本知识

欢迎来到数字系统设计领域！众所周知，21 世纪是信息的时代，如何对各种各样的信息进行描述、传递、处理和存储呢？应该说，人类迄今为止找到的最佳信息表达形式是“数字”！数字系统已经成为各个领域，乃至人们日常生活的重要组成部分。在我们去研究计算机以及其他数字系统中那些神奇的硬件是如何工作的之前，必须首先了解有关数字系统设计的基本知识。本章在对数字系统基本概念作简单介绍的基础上，重点讨论数字系统中数据的表示形式。

1.1 概　　述

1.1.1 数字系统

什么是数字系统？简单地说，数字系统是一个能够对数字信号进行加工、传递和存储的实体，它由实现各种功能的数字逻辑电路相互连接而成。例如，广泛应用于科学计算、数据处理和过程控制等领域的数字计算机就是一种典型的数字系统。

1. 数字信号

在客观世界中，存在各种不同的物理量。按其变化规律可以分为两种类型：一类是连续量，另一类是数字量。所谓连续量是指在时间上和数值上均作连续变化的物理量，例如，温度、压力等。在工程应用中，为了方便处理和传送这种物理量，通常用某一种连续量去模拟另一种连续量，例如，用电压的变化模拟温度的变化等。因此，人们习惯将连续量称为**模拟量**。表示模拟量的信号称为**模拟信号**。对模拟信号进行处理的电路称为模拟电路。反之，另一类物理量的变化在时间上和数值上都是离散的，或者说断续的。例如，学生成绩记录，工厂产品统计，电路开关的状态等。这类物理量的变化可以用不同的数字反映，所以称为**数字量**。表示数

字量的信号称为**数字信号**。

数字系统中处理的是数字信号，当数字系统要与模拟信号发生联系时，必须经过模/数(A/D)转换电路和数/模(D/A)转换电路对信号类型进行变换。例如，某控制系统的框图如图 1.1 所示。

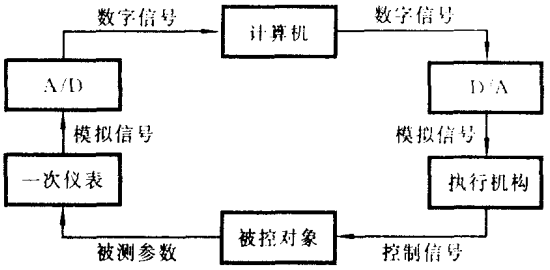


图 1.1 某控制系统的框图

2. 数字逻辑电路

用来处理数字信号的电子线路称为**数字电路**。由于数字电路的各种功能是通过逻辑运算和逻辑判断来实现的，所以，又将数字电路称为**数字逻辑电路或者逻辑电路**。数字逻辑电路与模拟电路相比，具有如下特点：

① 电路的基本工作信号是二值信号。它表现为电路中电压的“高”或“低”、开关的“接通”或“断开”、晶体管的“导通”或“截止”等两种稳定的物理状态。

② 电路中的半导体器件一般都工作在开、关状态，对电路进行研究时，主要关心输出和输入之间的逻辑关系。

③ 电路结构简单、功耗低、便于集成制造和系列化生产。产品价格低廉、使用方便、通用性好。

④ 由数字逻辑电路构成的数字系统工作速度快、精度高、功能强、可靠性好。

由于具有上述特点，所以数字逻辑电路的应用十分广泛。

随着半导体技术和工艺的发展，出现了数字集成电路，人们已不再用分立元件去构造实现各种逻辑功能的部件，而是采用标准集成电路进行逻辑设计。因此，数字集成电路是数字系统实现各种功能的物质基础。

数字集成电路的基本逻辑单元是逻辑门，任何一个复杂的数字部件均可由逻辑门构成。一块集成电路芯片所容纳的逻辑门数量反映了芯片的集成度，集成度越高，单个芯片所实现的逻辑功能就越强。通常，按照单片芯片所集成的逻辑门数量将数字集成电路分为小规模(SSI)、中规模(MSI)、大规模(LSI)和超大规模(VLSI)等几种类型。

3. 数字系统的层次结构

任何复杂的数字系统都是由最底层的基本电路开始逐步向上构建起来的。从底向上，复杂度逐层增加，功能不断增强。图 1.2 所示为数字系统的层次结构。

如上所述，集成电路是构成数字系统的物质基础，数字系统设计时考虑的基本逻辑单元为逻辑门，一旦理解了基本逻辑门的工作原理，便不必过于关心门电路内部电子线路的细节，而是更多地关注它们的外部特性及用途，以便实现更高一级的逻辑功能。

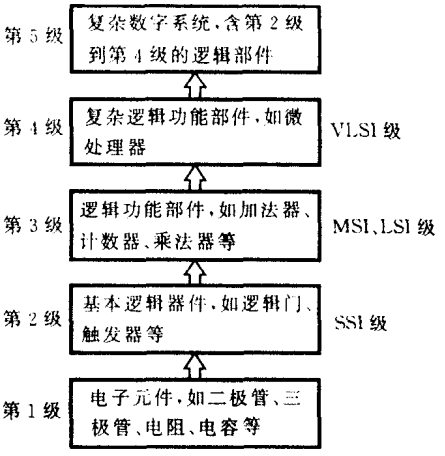


图 1.2　数字系统的层次结构

4. 典型的数字系统——数字计算机

数字计算机是一种能够自动、高速、精确地完成数值计算、数据加工和控制、管理等功能的数字系统。

(1) 数字计算机的组成

数字计算机由存储器、运算器、控制器、输入设备、输出设备以及适配器等主要部分组成，各部分通过总线连成一个整体。图 1.3 所示为数字计算机的一般结构。

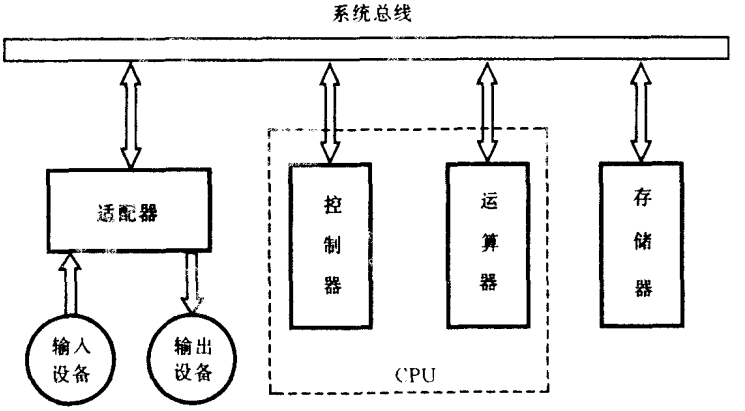


图 1.3　数字计算机的一般结构

(2) 计算机的发展历程

1946 年，在美国宾夕法尼亚大学诞生了世界上第一台数字计算机——

ENIAC，人们将其誉为数字计算机的始祖。该台机器共用了 18 000 多个电子管，占用空间的长度超过 30 m，重量达 30 t，而运算速度仅 5 000 次/s。显然，用现代的眼光来看这是一台体积庞大、耗费多而又十分低效的计算机，但正是这台机器的研制成功奠定了数字计算机的基础，成为科学史上一次划时代的创新。

自从第一台数字计算机问世以来，根据组成计算机的主要元器件的不同，计算机的发展历程如表 1.1 所示。

表 1.1 数字计算机的发展历程

主要器件	开始时间/年	运算速度
电子管	1946	几千次/秒～几万次/秒
晶体管	1958	几万次/秒～几十万次/秒
小、中规模集成电路	1965	几十万次/秒～几百万次/秒
大规模、超大规模集成电路	1971	几百万次/秒～几千万次/秒
巨大规模集成电路	1986	几亿次/秒～几百亿次/秒

计算机发展的历程表明，从 1946 年以来，总的发展规律是大约每隔 5 年运算速度提高 10 倍，可靠性提高 10 倍，成本降低 10 倍，体积缩小 10 倍。自 1970 年以来，计算机的生产数量每年递增 25%。可见，计算机的发展速度是十分惊人的。据专家预测，这种发展趋势还将继续维持至少 10 年以上。

1.1.2 数字逻辑电路的类型和研究方法

1. 数字逻辑电路的类型

根据一个电路有无记忆功能，可将数字逻辑电路分为组合逻辑电路和时序逻辑电路两种类型。

如果一个逻辑电路在任何时刻的稳定输出仅取决于该时刻的输入，而与电路过去的输入无关，则称之为**组合逻辑电路**(Combinational Logic Circuit)。由于这类电路的输出与过去的输入信号无关，所以不需要有记忆功能。例如，一个“多数表决器”，由于表决的结果仅取决于各个参与表决成员当时的态度是“赞成”还是“反对”，因此，它属于组合电路。

如果一个逻辑电路在任何时刻的稳定输出不仅取决于该时刻的输入，而且与过去的输入相关，则称之为**时序逻辑电路**(Sequential Logic Circuit)。由于这类电路的输出与过去的输入相关，所以需要有记忆功能，通常采用电路中记忆元件的状态来反映过去的输入信号。例如，一个统计输入脉冲信号个数的计数器，它的输出

结果不仅与当时的输入脉冲相关，还与前面收到的脉冲个数相关，因此，计数器是一个时序逻辑电路。时序逻辑电路按照是否有统一的时钟信号进行同步，又可进一步分为**同步时序逻辑电路**和**异步时序逻辑电路**。

2. 数字逻辑电路的研究方法

对数字系统中逻辑电路的研究有两个主要任务：一是分析，二是设计。对一个给定的数字逻辑电路，研究它所实现的逻辑功能和电路的工作性能称为**逻辑电路分析**；根据客观提出的功能要求，在给定条件下构造出实现预定功能的逻辑电路称为**逻辑电路设计**，简称**逻辑设计**或者**逻辑综合**。

随着集成电路技术的飞跃发展，数字逻辑电路的分析和设计方法在不断发生变化。但不管怎样变化，用逻辑代数作为基本理论的传统方法仍不失为逻辑电路分析和设计的基本方法。传统方法详细讨论了从问题的逻辑抽象到功能实现的全过程，可以说是至今为止最成熟、最基本的方法。该方法是建立在小规模集成电路基础之上的，它以技术经济指标作为评价一个设计方案优劣的主要性能指标，设计时追求的是如何使一个电路达到最简。因此，在组合逻辑电路设计时，通过逻辑函数化简，尽可能使电路中的逻辑门和连线数目达到最少。而在时序逻辑电路设计时，则通过状态化简和逻辑函数化简，尽可能使电路中的记忆元件、逻辑门和连线数目达到最少。但值得指出的是，一个**最简**的方案并不等于一个**最佳**的方案，最佳方案应满足全面的性能指标和实际应用要求。所以，在用传统方法求出一个实现预定功能的最简方案之后，往往要根据实际情况进行相应调整。

随着中、大规模集成电路的出现和集成电路规模的迅速发展，单个芯片内部容纳的逻辑器件越来越多，因而，实现某种逻辑功能所需要的门和触发器数量已不再成为影响经济指标的突出问题。如何用各种廉价的中、大规模集成组件去构造满足各种功能的、经济合理的电路，这无疑给设计人员提出了新的更高的要求。要适应这种要求就必须充分了解各种器件的逻辑结构和外部特性，做到合理选择器件，充分利用每一个已选器件的功能，用灵活多变的方法完成各类逻辑电路或功能模块的设计。此外，**可编程逻辑器件**(Programmable Logic Devices，简称 PLD)的出现，给逻辑设计带来了一种全新的方法。人们不再用常规硬线连接的方法去构造电路，而是借助丰富的计算机软件对器件进行编程来实现各种逻辑功能，无疑这给逻辑设计者带来了极大的方便。

其次，面对日益复杂的集成电路芯片设计和数字系统设计，人们不得不越来越多地借助计算机辅助设计(Computer Aided Design，简称 CAD)。目前，已有各种电子设计自动化(Electronic Design Automatic，简称 EDA)软件在市场上出售。计算机辅助逻辑设计方法正在不断推广和应用。不少人认为计算机设计自动化已

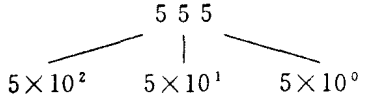
形成计算机科学中的一个独立的学科。

1.2 数制及其转换

1.2.1 进位计数制

数制是人们对数量计数的一种统计规律。日常生活中广泛使用的是十进制，而数字系统中使用的是二进制。

十进制中采用了 0,1,…,9 共 10 个基本数字符号，进位规律是“逢十进一”。当用若干个数字符号并在一起表示一个数时，处在不同位置的数字符号，其值的含意不同。如



同一个字符 5 从左到右所代表的值依次为 500、50、5。该数又可表示成

$$555=5\times10^{2}+5\times10^{1}+5\times10^{0}$$

广义地说，一种进位计数制包含着基数和位权两个基本因素：

● **基数**是指计数制中所用到的数字符号的个数。在基数为 R 的计数制中，包含 0、1、…、$R-1$ 共 R 个数字符号，进位规律是“逢 R 进一”，称为 R 进位计数制，简称 R 进制。

● **位权**是指在一种进位计数制表示的数中，用来表明不同数位上数值大小的一个固定常数。不同数位有不同的位权，某一个数位的数值等于这一位的数字符号乘上与该位对应的位权。R 进制数的位权是 R 的整数次幂。例如，十进制数的位权是 10 的整数次幂，其个位的位权是 10^0，十位的位权是 10^1……

一般来说，一个 R 进制数 N 可以有以下两种表示方法：

● **并列表示法**，又称位置计数法，其表达式为

$$(N)_R=(K_{n-1}K_{n-2}\cdots K_1K_0.K_{-1}K_{-2}\cdots K_{-m})_R$$

● **多项式表示法**，又称按权展开法，其表达式为

$$\begin{aligned}(N)_R&=K_{n-1}\times R^{n-1}+K_{n-2}\times R^{n-2}+\cdots+K_1\times R^1+K_0\times R^0\\&\quad+K_{-1}\times R^{-1}+K_{-2}\times R^{-2}+\cdots+K_{-m}\times R^{-m}\\&=\sum_{i=-m}^{n-1}K_iR^i\end{aligned}$$

其中，R 表示基数；n 为整数部分的位数；m 为小数部分的位数；K_i 为 R 进制中的一个数字符号，其取值范围为

$$0 \leqslant K_i \leqslant R-1 \qquad -m \leqslant i \leqslant n-1$$

例如，十进制数 2 005.18 可以表示成

$$(2005.18)_{10}=2\times10^3+0\times10^2+0\times10^1+5\times10^0+1\times10^{-1}+8\times10^{-2}$$

1. 二进制

基数 $R=2$ 的进位计数制称为二进制。二进制数中只有 0 和 1 两个基本数字符号，进位规律是“逢二进一”。二进制数的位权是 2 的整数次幂。

任意一个二进制数 N 可以表示成

$$\begin{aligned}(N)_2&=(K_{n-1}K_{n-2}\cdots K_1K_0.K_{-1}K_{-2}\cdots K_{-m})_2\\&=K_{n-1}\times2^{n-1}+K_{n-2}\times2^{n-2}+\cdots+K_1\times2^1+K_0\times2^0\\&\quad+K_{-1}\times2^{-1}+K_{-2}\times2^{-2}+\cdots+K_{-m}\times2^{-m}\\&=\sum_{i=-m}^{n-1}K_i\times2^i\end{aligned}$$

其中，n 为整数位数；m 为小数位数；K_i 为 0 或者 1，$-m\leqslant i\leqslant n-1$。

例如，一个二进制数 1011.01 可以表示成

$$(1011.01)_2=1\times2^3+0\times2^2+1\times2^1+1\times2^0+0\times2^{-1}+1\times2^{-2}$$

二进制数的运算十分简便，其运算规则如下：

加法规则	0+0=0	0+1=1	
	1+0=1	1+1=0	（进位为 1）
减法规则	0−0=0	0−1=1	（借位为 1）
	1−0=1	1−1=0	
乘法规则	0×0=0	0×1=0	
	1×0=0	1×1=1	
除法规则	0÷1=0	1÷1=1	

例如，二进制数 $A=11001$，$B=101$，则 $A+B$，$A-B$，$A\times B$，$A\div B$ 的运算为

$$\begin{array}{cccccc} & 1 & 1 & 0 & 0 & 1 \\ + & & & 1 & 0 & 1 \\ \hline & 1 & 1 & 1 & 1 & 0 \end{array} \qquad \begin{array}{cccccc} & 1 & 1 & 0 & 0 & 1 \\ - & & & 1 & 0 & 1 \\ \hline & 1 & 0 & 1 & 0 & 0 \end{array}$$

$$
\begin{array}{r}
\quad 1\ 1\ 0\ 0\ 1 \\
\times \qquad\ 1\ 0\ 1 \\
\hline
\quad 1\ 1\ 0\ 0\ 1 \\
0\ 0\ 0\ 0\ 0 \\
+\ 1\ 1\ 0\ 0\ 1 \\
\hline
1\ 1\ 1\ 1\ 1\ 0\ 1
\end{array}
\qquad
\begin{array}{r}
\qquad\quad 1\ 0\ 1 \\
\hline
101\ )\ 1\ 1\ 0\ 0\ 1 \\
-\ 1\ 0\ 1 \\
\hline
\qquad 1\ 0\ 1 \\
-\ 1\ 0\ 1 \\
\hline
\qquad\quad 0
\end{array}
$$

二进制除了运算简单之外，还具有物理实现容易，存储和传送方便、可靠等优点。因为二进制中只有 0 和 1 两个数字符号，可以用电子器件的两种不同状态来表示一位二进制数，例如，可以用晶体管的截止和导通表示 1 和 0，也可以用电平的高和低表示 1 和 0 等，所以在数字系统中普遍采用二进制。

二进制的缺点是数的位数太长且字符单调，使得书写、记忆和阅读不方便。因此，人们在进行指令书写、程序输入和输出等工作时，通常采用八进制数和十六进制数作为二进制数的缩写。

2. 八进制

基数 $R=8$ 的进位计数制称为八进制。八进制数中有 0,1,…,7 共 8 个基本数字符号，进位规律是“逢八进一”。八进制数的位权是 8 的整数次幂。

任意一个八进制数 N 可以表示成

$$
\begin{aligned}
(N)_8 &= (K_{n-1}K_{n-2}\cdots K_1K_0.K_{-1}K_{-2}\cdots K_{-m})_8 \\
&= K_{n-1}\times 8^{n-1}+K_{n-2}\times 8^{n-2}+\cdots+K_1\times 8^1+K_0\times 8^0 \\
&\quad +K_{-1}\times 8^{-1}+K_{-2}\times 8^{-2}+\cdots+K_{-m}\times 8^{-m} \\
&= \sum_{i=-m}^{n-1} K_i\times 8^i
\end{aligned}
$$

其中，n 为整数位数，m 为小数位数，K_i 表示 0～7 中的任何一个字符，$-m\leqslant i\leqslant n-1$。

3. 十六进制

基数 $R=16$ 的进位计数制称为十六进制。十六进制数中有 0,1,…,9,A,B,C,D,E,F 共 16 个数字符号，其中，A～F 分别表示十进制数的 10～15。进位规律为“逢十六进一”，十六进制数的位权是 16 的整数次幂。

任意一个十六进制数 N 可以表示成

$$
\begin{aligned}
(N)_{16} &= (K_{n-1}K_{n-2}\cdots K_1K_0.K_{-1}K_{-2}\cdots K_{-m})_{16} \\
&= K_{n-1}\times 16^{n-1}+K_{n-2}\times 16^{n-2}+\cdots+K_1\times 16^1+K_0\times 16^0 \\
&\quad +K_{-1}\times 16^{-1}+K_{-2}\times 16^{-2}+\cdots+K_{-m}\times 16^{-m} \\
&= \sum_{i=-m}^{n-1} K_i\times 16^i
\end{aligned}
$$

其中，n 为整数位数，m 为小数位数，K_i 表示 0～9 及 A～F 中的任何一个字符，$-m \leqslant i \leqslant n-1$。

表 1.2 列出了与十进制数 0～16 对应的二进制数、八进制数、十六进制数。

表 1.2 十进制数与二、八、十六进制数对照表

十进制数	二进制数	八进制数	十六进制数
0	0000	00	0
1	0001	01	1
2	0010	02	2
3	0011	03	3
4	0100	04	4
5	0101	05	5
6	0110	06	6
7	0111	07	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

1.2.2 数制转换

人们习惯使用的是十进制数，数字系统中普遍采用的是二进制数。人们在与二进制数打交道时，为了书写和阅读的方便，又通常使用八进制数或十六进制数，因此，产生了不同进位计数制之间的转换问题。下面讨论二进制数与十进制数、八进制数和十六进制数之间的相互转换。

1. 二进制数与十进制数之间的转换

(1) 二进制数转换为十进制数

二进制数转换成十进制数非常简单，只需将二进制数表示成按权展开式，并按十进制数的运算法则进行计算，所得结果即为该数对应的十进制数。例如，

$$\begin{aligned}(10110.101)_2 &= 1\times 2^4+1\times 2^2+1\times 2^1+1\times 2^{-1}+1\times 2^{-3}\\ &= 16+4+2+0.5+0.125\\ &= (22.625)_{10}\end{aligned}$$

(2) 十进制数转换为二进制数

十进制数转换成二进制数时，应对整数和小数分别进行处理。整数转换采用**除 2 取余**的方法，小数转换采用**乘 2 取整**的方法。

1) 整数转换

整数转换采用**除 2 取余**法。将十进制整数 N 除以 2，取余数记为 K_0；再将所得商除以 2，取余数记为 K_1……依此类推，直至商为 0，取余数记作 K_{n-1} 为止，即可得到与 N 对应的 n 位二进制整数 $K_{n-1}\cdots K_1K_0$。

例如，将十进制整数 45 转换成二进制整数：

2	45	余数	
2	22	……1(K_0)	低位
2	11	……0(K_1)	
2	5	……1(K_2)	
2	2	……1(K_3)	
2	1	……0(K_4)	
	0	……1(K_5)	高位

即 $(45)_{10}=(101101)_2$

2) 小数转换

小数转换采用**乘 2 取整**法。将十进制小数 N 乘以 2，取整数部分记为 K_{-1}；再将其小数部分乘以 2，取整数部分记为 K_{-2}……依此类推，直至其小数部分为 0 或达到规定精度要求，取整数部分记作 K_{-m} 为止，即可得到与 N 对应的 m 位二进制小数 $0.K_{-1}K_{-2}\cdots K_{-m}$。

例如，将十进制小数 0.6875 转换成二进制小数：

			0.6875
	整数部分	×	2
高位	1(K_{-1})……		1.3750
		×	2
	0(K_{-2})……		0.7500
		×	2
	1(K_{-3})……		1.5000
		×	2
低位	1(K_{-4})……		1.0000

即 $(0.6875)_{10}=(0.1011)_2$

值得注意的是，有的十进制小数不能用有限位二进制小数精确表示。这时只

能根据精度要求，求出相应的二进制位数近似地表示。一般当要求二进制数取 m 位小数时，可求出 $m+1$ 位，然后对最低位作 **0 舍 1 入**处理。

例如，将十进制小数 0.323 转换成二进制小数（保留 4 位小数）：

$$
\begin{array}{rr}
 & 0.323 \\
\times & \quad 2 \\
\hline
 & \boxed{0}.646 \\
\times & \quad 2 \\
\hline
 & \boxed{1}.292 \\
\times & \quad 2 \\
\hline
 & \boxed{0}.584 \\
\times & \quad 2 \\
\hline
 & \boxed{1}.168 \\
\times & \quad 2 \\
\hline
 & \boxed{0}.336
\end{array}
$$

即　$(0.323)_{10}=(0.0101)_2$

此外，当一个十进制数既包含整数部分，又包含小数部分时，只需将整数部分和小数部分分别转换为二进制数，然后用小数点将两部分结果连起来即可。

例如，将十进制数 35.625 转换成二进制数：

$$
\begin{array}{r|l}
2 & 35 \\
2 & 17 \quad \cdots\cdots 1 \\
2 & 8 \quad \cdots\cdots 1 \\
2 & 4 \quad \cdots\cdots 0 \\
2 & 2 \quad \cdots\cdots 0 \\
2 & 1 \quad \cdots\cdots 0 \\
 & 0 \quad \cdots\cdots 1
\end{array}
$$

$$
\begin{array}{rr}
 & 0.625 \\
\times & \quad 2 \\
\hline
 & \boxed{1}.250 \\
\times & \quad 2 \\
\hline
 & \boxed{0}.500 \\
\times & \quad 2 \\
\hline
 & \boxed{1}.000
\end{array}
$$

即　$(35.625)_{10}=(100011.101)_2$

2. 二进制数与八进制数、十六进制数之间的转换

(1) 二进制数与八进制数之间的转换

由于 $2^3=8$，所以 1 位八进制数所能表示的数值恰好等于 3 位二进制数所能表示的数值，即八进制中的基本数字符号 0～7 正好和 3 位二进制数的 8 种取值 000～111 对应。因此，二进制数与八进制数之间的转换可以按位进行。

二进制数转换成八进制数时，以小数点为界，分别往高、往低每 3 位为一组，最

后不足 3 位时用 0 补充，然后写出每组对应的八进制字符，即为相应八进制数。

例如，将二进制数 11100101.01 转换成八进制数：

$$\underbrace{0\ 1\ 1}_{3}\ \underbrace{1\ 0\ 0}_{4}\ \underbrace{1\ 0\ 1}_{5}\ .\ \underbrace{0\ 1\ 0}_{2}$$

即 $(11100101.01)_2=(345.2)_8$

八进制数转换成二进制数时，只需将每位八进制数用 3 位二进制数表示。

例如，将八进制数 56.7 转换成二进制数：

$$\overbrace{1\ 0\ 1}^{5}\ \overbrace{1\ 1\ 0}^{6}\ .\ \overbrace{1\ 1\ 1}^{7}$$

即 $(56.7)_8=(101110.111)_2$

(2) 二进制数与十六进制数之间的转换

二进制数与十六进制数之间的转换类似于二进制数与八进制数之间的转换，只不过是 4 位二进制数对应 1 位十六进制数，即 4 位二进制数的取值0000～1111分别对应十六进制字符 0～F。

二进制数转换成十六进制数时，以小数点为界，分别往高、往低每 4 位为一组，最后不足 4 位时用 0 补充，然后写出每组对应的十六进制字符即可。

例如，将二进制数 101110.011 转换成十六进制数：

$$\underbrace{0\ 0\ 1\ 0}_{2}\ \underbrace{1\ 1\ 1\ 0}_{E}\ .\ \underbrace{0\ 1\ 1\ 0}_{6}$$

即 $(101110.011)_2=(2E.6)_{16}$

十六进制数转换成二进制数时，只需将每位十六进制数用 4 位二进制数表示。

例如，将十六进制数 5A.B 转换成二进制数：

$$\overbrace{0\ 1\ 0\ 1}^{5}\ \overbrace{1\ 0\ 1\ 0}^{A}\ .\ \overbrace{1\ 0\ 1\ 1}^{B}$$

即 $(5A.B)_{16}=(1011010.1011)_2$

由于八进制数、十六进制数与二进制数的转换很方便，因此，常用来作二进制数的缩写。

1.3 带符号二进制数的代码表示

在对带符号数进行算术运算时，必然涉及数的符号问题。人们通常在一个数的前面用“+”号表示正数，用“−”号表示负数。而在数字系统中，符号和数值一样是用 0 和 1 来表示的，一般将数的最高位作为符号位，用 0 表示正，用 1 表示负。其格式为

符号位	数值位

为了区分一般书写表示的带符号二进制数和数字系统中的带符号二进制数，通常将用“+”、“−”表示正、负的二进制数称为符号数的**真值**，而把将符号和数值一起编码表示的二进制数称为**机器数**或**机器码**。常用的机器码有原码、反码和补码三种。

1.3.1 原码

用原码表示带符号二进制数时，符号位用0表示正，1表示负；数值位保持不变。原码表示法又称为符号－数值表示法。

1. 小数原码的定义

设二进制小数 $X=\pm 0.x_{-1}x_{-2}\cdots x_{-m}$，则其原码定义为

$$[X]_{原}=\begin{cases} X & 0\leqslant X<1 \\ 1-X & -1<X\leqslant 0 \end{cases}$$

例如，若 $X_1=+0.1011$，$X_2=-0.1011$，则 X_1 和 X_2 的原码为

$$[X_1]_{原}=0.1011$$

$$[X_2]_{原}=1-(-0.1011)=1.1011$$

根据定义，小数“0”的原码可以表示成 0.0…0 或 1.0…0。

2. 整数原码的定义

设二进制整数 $X=\pm x_{n-1}x_{n-2}\cdots x_0$，则其原码定义为

$$[X]_{原}=\begin{cases} X & 0\leqslant X<2^n \\ 2^n-X & -2^n<X\leqslant 0 \end{cases}$$

例如，若 $X_1=+1101$，$X_2=-1101$，则 X_1 和 X_2 的原码为

$$[X_1]_{原}=01101$$

$$[X_2]_{原}=2^4-(-1101)$$

$$=10000+1101$$

$$=11101$$

同样，整数“0”的原码也有两种形式，即 00…0 和 10…0。

采用原码表示带符号的二进制数简单易懂，但实现加、减运算不方便。当进行两数加、减运算时，要根据运算及参加运算的两个数的符号来确定是加还是减。如果是做减法，则还需根据两数的大小确定被减数和减数，以及运算结果的符号。显

然，这将增加运算的复杂性。

1.3.2 反码

用反码表示带符号的二进制数时，符号位与原码相同，即用 0 表示正，用 1 表示负；数值位与符号位相关，正数反码的数值位和真值的数值位相同，而负数反码的数值位是真值的数值位按位变反。

1. 小数反码的定义

设二进制小数 $X=\pm 0.x_{-1}x_{-2}\cdots x_{-m}$，则其反码定义为

$$[X]_{反}=\begin{cases}X & 0\leqslant X<1\\ (2-2^{-m})+X & -1<X\leqslant 0\end{cases}$$

例如，若 $X_1=+0.1011$，$X_2=-0.1011$，则 X_1 和 X_2 的反码为

$$[X_1]_{反}=0.1011$$

$$\begin{aligned}[X_2]_{反}&=2-2^{-4}+X_2\\&=10.0000-0.0001-0.1011\\&=1.0100\end{aligned}$$

根据定义，小数“0”的反码有两种表示形式，即 0.0…0 和 1.1…1。

2. 整数反码的定义

设二进制整数 $X=\pm x_{n-1}x_{n-2}\cdots x_0$，则其反码定义为

$$[X]_{反}=\begin{cases}X & 0\leqslant X<2^n\\ (2^{n+1}-1)+X & -2^n<X\leqslant 0\end{cases}$$

例如，若 $X_1=+1001$，$X_2=-1001$，则 X_1 和 X_2 的反码为

$$[X_1]_{反}=01001$$

$$\begin{aligned}[X_2]_{反}&=(2^5-1)+X\\&=(100000-1)+(-1001)\\&=11111-1001\\&=10110\end{aligned}$$

同样，整数“0”的反码也有两种形式，即 00…0 和 11…1。

采用反码进行加、减运算时，无论两数相加还是两数相减，均可通过加法实现。其加、减运算规则如下：

$$[X_1+X_2]_{反}=[X_1]_{反}+[X_2]_{反}$$

$$[X_1-X_2]_{反}=[X_1]_{反}+[-X_2]_{反}$$

运算时，符号位和数值位一样参加运算。当符号位有进位产生时，应将进位加到运算结果的最低位，才能得到最后结果。

例如，若 $X_1=+0.1110$，$X_2=+0.0101$，则求 X_1-X_2 可通过反码相加实现。运算如下：

$$[X_1-X_2]_{反}=[X_1]_{反}+[-X_2]_{反}=0.1110+1.1010$$

$$\begin{array}{rr} & 0.1\ 1\ 1\ 0 \\ + & 1.1\ 0\ 1\ 0 \\ \hline & \boxed{1}\ 0.1\ 0\ 0\ 0 \\ + & \llcorner\!\!\!

$$[X]_{补}=\begin{cases}X & 0\leqslant X<2^n\\ 2^{n+1}+X & -2^n\leqslant X<0\end{cases}$$

例如，若 $X_1=+1010$，$X_2=-1010$，则 X_1 和 X_2 的补码为

$$[X_1]_{补}=01010$$

$$[X_2]_{补}=2^5+X$$

$$=100000-1010$$

$$=10110$$

同样，整数“0”的补码也只有一种表示形式，即 00…0。

采用补码进行加、减运算时，其加、减运算均可以通过加法实现，运算规则如下：

$$[X_1+X_2]_{补}=[X_1]_{补}+[X_2]_{补}$$

$$[X_1-X_2]_{补}=[X_1]_{补}+[-X_2]_{补}$$

运算时，符号位和数值位一样参加运算，若符号位有进位产生，则应将进位丢掉才能得到正确结果。

例如，若 $X_1=-1001$，$X_2=+0011$，则采用补码求 X_1-X_2 的运算如下：

$$[X_1-X_2]_{补}=[X_1]_{补}+[-X_2]_{补}=10111+11101$$

$$\begin{array}{rrrrrrr} & & 1 & 0 & 1 & 1 & 1\\ & + & 1 & 1 & 1 & 0 & 1\\ \hline \text{丢掉} & \boxed{1} & 1 & 0 & 1 & 0 & 0\end{array}$$

即 $[X_1-X_2]_{补}=10100$

由于结果的符号位为 1，表示是负数，故

$$X_1-X_2=-1100$$

显然，采用补码进行加、减运算最方便。

1.4 几种常用的编码

1.4.1 十进制数的二进制编码

在数字系统的输入/输出过程中，为了既满足系统中使用二进制数的要求，又适应人们使用十进制数的习惯，通常使用 4 位二进制代码对十进制数字符号进行编码，简称为二-十进制代码，或称 BCD(Binary Coded Decimal)码。它既有二进制数的形式，又有十进制数的特点，便于传递、处理。

十进制数中有 0～9 共 10 个数字符号，4 位二进制代码可以组成 16 种不同状

态，从 16 种状态中取出 10 种状态来表示 10 个数字符号的编码方案很多，但不管哪种编码方案都有 6 种状态不允许出现。根据代码中每一位是否有固定的权，通常将 BCD 码分为有权码和无权码两种类型。常用的 BCD 码有 8421 码、2421 码和余 3 码 3 种，它们与十进制数字符号对应的编码如表 1.3 所示。

表 1.3　常用的 3 种 BCD 码

十进制字符	8421 码	2421 码	余 3 码
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

1. 8421 码

8421 码是最常用的一种有权码，其 4 位二进制码从高位至低位的权依次为 2^3、2^2、2^1、2^0，即为 8、4、2、1。显然，这与普通的 4 位二进制数的权是一样的。因此，按 8421 码编码的 0～9 与用 4 位二进制数表示的 0～9 完全一样。值得注意的是 4 位二进制数中的 1010～1111 不允许在 8421 码中出现，因为没有十进制数字符号与其对应。此外，十进制数字符号的 8421 码与相应 ASCII 码的低 4 位相同，这一特点有利于简化输入/输出过程中 BCD 码与字符代码的转换。所以 8421 码是一种人机联系时广泛使用的中间形式。

8421 码与十进制数之间的转换是按位进行的，即十进制数的每一位与 4 位二进制编码对应。例如，

$$(258)_{10} = (0010\ 0101\ 1000)_{8421码}$$

$$(0001\ 0010\ 0000\ 1000)_{8421码} = (1208)_{10}$$

2. 2421 码

2421 码是另一种有权码，其 4 位二进制码从高位至低位的权依次为 2、4、2、1。若一个十进制字符 X 的 2421 码为 $a_3a_2a_1a_0$，则该字符的值为

$$X = 2a_3 + 4a_2 + 2a_1 + a_0$$

例如，　　$(1101)_{2421码} = (7)_{10}$

2421 码不具备单值性，为了与十进制字符一一对应，2421 码不允许出现 0101

～1010 的 6 种状态。

在 2421 码中，十进制字符 0 和 9、1 和 8、2 和 7、3 和 6、4 和 5 的各码位互为相反。具有这种特性的代码称为**对 9 的自补代码**，即一个数的 2421 码只要自身按位变反，便可得到该数对 9 的补数的 2421 码。例如，4 对 9 的补数是 5，将 4 的 2421 码 0100 按位变反，便可得到 5 的 2421 码 1011。具有这一特征的 BCD 码可给运算带来方便，因为直接对 BCD 码进行运算时，可利用其对 9 的补数将减法运算转化为加法运算。

2421 码与十进制数之间的转换同样是按位进行的，例如，

$$(258)_{10}=(0010\ 1011\ 1110)_{2421码}$$

$$(0010\ 0001\ 1110\ 1011)_{2421码}=(2185)_{10}$$

3. 余 3 码

余 3 码是由 8421 码加上 0011 形成的一种无权码，由于它的每个字符编码比相应 8421 码多 3，故称为余 3 码。例如，十进制字符 5 的余 3 码等于 5 的 8421 码 0101 加上 0011，即为 1000。同样，余 3 码中也有 6 种状态 0000、0001、0010、1101、1110 和 1111 是不允许出现的。

余 3 码也是一种对 9 的自补代码，因而可给运算带来方便。其次，在将两个余 3 码表示的十进制数相加时，能正确产生进位信号，但对“和”必须修正。修正的方法是：如果有进位，则结果加 3；如果无进位，则结果减 3。

余 3 码与十进制数之间的转换也是按位进行的，值得注意的是每位十进制数的编码都应余 3。例如，

$$(256)_{10}=(0101\ 1000\ 1001)_{余3码}$$

$$(1000\ 1001\ 1001\ 1011)_{余3码}=(5668)_{10}$$

1.4.2 可靠性编码

顾名思义，可靠性编码的作用是为了提高系统的可靠性。代码在形成和传送过程中都可能发生错误。为了使代码本身具有某种特征或能力，尽可能减少错误的发生，或者出错后容易被发现，甚至检查出错误的码位后能予以纠正，因而形成了各种编码方法。下面介绍两种简单、常用的可靠性编码。

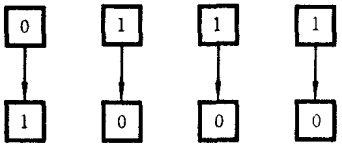
1. 格雷码

格雷码(Gray Code)的特点是任意两个相邻的数，其格雷码仅有一位不同。表 1.4 给出了与 4 位二进制码对应的典型格雷码。

表 1.4　与 4 位二进制码对应的典型格雷码

十进制数	4 位二进制码	典型格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

在数字系统中，数字 0 或 1 是用电子器件的不同状态来表示的。当数据按升序或降序变化时，若采用普通二进制数，则每次增 1 或者减 1 时，可能引起若干位发生变化。例如，用 4 位二进制数表示的十进制数由 7 变为 8 时，要求 4 位同时发生变化，即由 0111 变为 1000。与 4 位二进制数对应的 4 个电子器件的状态变化如下：



显然，当电子器件的变化速度不一致时，便会产生错误代码，如产生 1111（假定最高位变化比低 3 位快）、1001（假定最低位变化比高 3 位慢）等错误代码。尽管这种错误代码时间是短暂的，但有时是不允许的，因为它将形成干扰，影响数字系统的正常工作。格雷码从编码上杜绝了这类错误的发生。

格雷码可以从普通二进制码转换得到。设二进制码为 $B=B_{n-1}B_{n-2}\cdots B_{i+1}B_i\cdots B_1B_0$，对应格雷码为 $G=G_{n-1}G_{n-2}\cdots G_{i+1}G_i\cdots G_1G_0$，则有

$$G_{n-1}=B_{n-1}$$

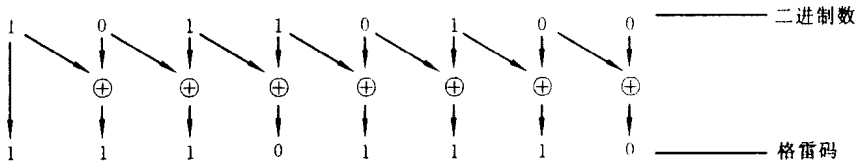
$$G_i=B_{i+1}\oplus B_i \qquad 0\leqslant i\leqslant n-2$$

其中，运算“⊕”称为“异或”运算，运算规则是

$$0\oplus 0=0 \qquad 0\oplus 1=1 \qquad 1\oplus 0=1 \qquad 1\oplus 1=0$$

异或运算又称为“模 2 加”，即不计进位的二进制加法。

例如，二进制码转换 10110100 对应的格雷码为 11101110，转换过程如下：



2. 奇偶检验码

二进制信息在传送时，可能由于外界干扰或其他原因而发生错误，即可能有的 1 错为 0 或者有的 0 错为 1。奇偶检验码(Parity Check Code)是一种能够检验出这类错误的代码。这种代码由两部分组成：一是信息位，即需要传递的信息本身，可以是位数不限的一组二进制代码；二是奇偶检验位，它仅有一位。奇偶检验位的编码方式有两种：一种是使信息位和检验位中“1”的个数为奇数，称为奇检验；另一种是使信息位和检验位中“1”的个数为偶数，称为偶检验。例如，二进制代码 1001101 奇偶检验码表示时的两种编码方式为

信息位(7 位)	采用奇检验的检验位(1 位)	采用偶检验的检验位(1 位)
1001101	1	0

表 1.5 列出了与 8421 码对应的奇偶检验码。

表 1.5 8421 码的奇偶检验码

十进制数码	采用奇检验的 8421 码		采用偶检验的 8421 码	
	信息位	检验位	信息位	检验位
0	0000	1	0000	0
1	0001	0	0001	1
2	0010	0	0010	1
3	0011	1	0011	0
4	0100	0	0100	1
5	0101	1	0101	0
6	0110	1	0110	0
7	0111	0	0111	1
8	1000	0	1000	1
9	1001	1	1001	0

奇偶检验码的工作原理如图 1.4 所示。

采用奇偶检验码进行错误检测时，在发送端由编码器根据信息位编码产生奇偶检验位，形成奇偶检验码发往接收端；接收端通过检测器检查代码中含“1”个数的奇偶，判断信息是否出错。例如，当采用偶检验时，若收到的代码中含奇数个“1”，则说明发生了错误。但判断出错后，并不能确定是哪一位出错，也就无法纠

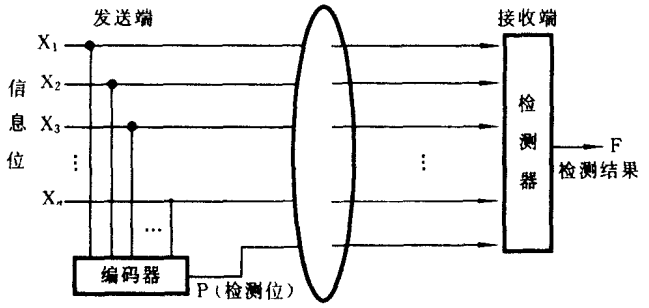


图 1.4 奇偶检验码的工作原理图

正。因此，奇偶检验码只有检错能力，没有纠错能力。其次，这种码只能发现单错，不能发现双错，但由于单错的概率远远大于双错，所以，这种编码还是很有实用价值的。加之它编码简单、容易实现，因而，在数字系统中被广泛采用。

*1.4.3 字符编码

数字系统中处理的数据除了数字之外，还有字母、运算符号、标点符号以及其他特殊符号，人们将这些符号统称为字符。所有字符在数字系统中必须用二进制编码表示，通常将其称为字符编码(Alphanumeric Code)。

最常用的字符编码是美国信息交换标准码，简称 ASCII 码(American Standard Code for Information Interchange)。ASCII 码用 7 位二进制码表示 128 种字符，编码规则如表 1.6 所示。由于数字系统中实际是用一个字节表示一个字符，所以使用 ASCII 码时，通常在最左边增加一位奇偶检验位。

表 1.6 7 位 ASCII 码编码表

低 4 位代码	高 3 位代码($a_7a_6a_5$)							
($a_4a_3a_2a_1$)	000	001	010	011	100	101	110	111
0000	NUL	DEL	SP	0	@	P	、	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{

续表

低4位代码 $(a_4a_3a_2a_1)$	高3位代码($a_7a_6a_5$)							
	000	001	010	011	100	101	110	111
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	−	=	M	]	m	}
1110	SO	RS	.	>	N	∧	n	~
1111	SI	US	/	?	O	−	o	DEL

注：NUL 空白 SOH 序始 STX 文始 ETX 文终 EOT 送毕 ENQ 询问
ACK 承认 BEL 告警 BS 退格 HT 横表 LF 换行 VT 纵表
FF 换页 CR 回车 SO 移出 SI 移入 DEL 转义 DC1 机控1
DC2 机控2 DC3 机控3 DC4 机控4 NAK 否认 SYN 同步 ETB 组终
CAN 作废 EM 载终 SUB 取代 ESC 扩展 FS 卷隙 GS 群隙
RS 录隙 US 元隙 SP 间隔 DEL 抹掉

习 题 一

1.1 什么是模拟信号？什么是数字信号？试各举一例。

1.2 数字逻辑电路具有哪些主要特点？

1.3 数字逻辑电路可分为哪两种类型？主要区别是什么？

1.4 最简电路是否一定最佳？为什么？

1.5 把下列不同进制数写成按权展开形式：

(1) $(4517.239)_{10}$ (2) $(10110.0101)_2$ (3) $(325.744)_8$ (4) $(785.4AF)_{16}$

1.6 将下列二进制数转换成十进制数、八进制数和十六进制数：

(1) 1110101 (2) 0.110101 (3) 10111.01

1.7 将下列十进制数转换成二进制数、八进制数和十六进制数(精确到小数点后4位)：

(1) 29 (2) 0.27 (3) 33.33

1.8 如何判断一个二进制正整数 $B=b_6b_5b_4b_3b_2b_1b_0$ 能否被$(4)_{10}$整除？

1.9 写出下列各数的原码、反码和补码：

(1) 0.1011 (2) −10110

1.10 已知$[N]_{补}=1.0110$，求 $[N]_{原}$，$[N]_{反}$和 N。

1.11 将下列余3码转换成十进制数和2421码：

(1) 011010000011 (2) 01000101.1001

1.12 试用8421码和格雷码分别表示下列各数：

(1) $(111110)_2$ (2) $(1100110)_2$



逻辑代数基础

逻辑代数是从哲学领域中的逻辑学发展而来的。1847年，英国数学家乔治·布尔(G. Boole)提出了用数学分析方法表示命题陈述的逻辑结构，并成功地将形式逻辑归结为一种代数演算，从而诞生了有名的“布尔代数”。1938年，克劳德·向农(C. E. Shannon)将布尔代数应用于电话继电器的开关电路，提出了“开关代数”。随着电子技术的发展，集成电路逻辑门已经取代了机械触点开关，故“开关代数”这个术语已很少使用。为了与“数字系统逻辑设计”这一术语相适应，人们更习惯于把开关代数叫做逻辑代数。

逻辑代数是数字系统逻辑设计的数学工具。无论何种形式的数字系统，都是由一些基本的逻辑电路所组成的。为了解决数字系统分析和设计中的各种具体问题，必须掌握逻辑代数这一重要数学工具。

本章将从实用的角度介绍逻辑代数的基本概念、基本定理和规则、逻辑函数的表示形式以及逻辑函数的化简。

2.1 逻辑代数的基本概念

逻辑代数L是一个封闭的代数系统，它由一个逻辑变量集K，常量0和1以及“或”、“与”、“非”3种基本运算所构成，记为$L=\{K,+,\cdot,-,0,1\}$。该系统应满足下列公理。

公理1 交换律

对于任意逻辑变量A、B，有

$$A+B=B+A \qquad A\cdot B=B\cdot A$$

公理2 结合律

对于任意的逻辑变量A、B、C，有

$$(A+B)+C=A+(B+C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

公理 3 分配律

对于任意的逻辑变量 A、B、C，有

$$A+(B \cdot C)=(A+B) \cdot (A+C)$$

$$A \cdot (B+C)=A \cdot B+A \cdot C$$

公理 4 0-1 律

对于任意逻辑变量 A，有

$$A+0=A \qquad A \cdot 1=A$$

$$A+1=1 \qquad A \cdot 0=0$$

公理 5 互补律

对于任意逻辑变量 A，存在惟一的 $\overline{A}$，使得

$$A+\overline{A}=1 \qquad A \cdot \overline{A}=0$$

公理是一个代数系统的基本出发点，无需加以证明。

显而易见，逻辑代数是一种比普通代数更为简单的代数。为了加深对这个代数系统的了解，更好地弄清它和普通代数的区别，下面将对逻辑代数中涉及的一些基本概念作进一步说明。

2.1.1 逻辑变量及基本逻辑运算

逻辑代数和普通代数一样，也是用字母表示变量。所不同的是，在普通代数中，变量的取值可以是任意实数，而逻辑代数是一种二值代数系统，即任何逻辑变量的取值只有两种可能性——取值 0 或取值 1。逻辑值 0 和 1 不再像普通代数中那样具有数量的概念，而是用来表征矛盾的双方和判断事件真伪的形式符号，无大小、正负之分。在数字系统中，开关的接通与断开，电压的高和低，信号的有和无，晶体管的导通与截止等两种稳定的物理状态，均可用 1 和 0 这两种不同的逻辑值来表征。然而，要描述一个数字系统，仅用逻辑变量的取值来反映单个开关元件的两种状态是不够的，还必须反映一个复杂系统中各开关元件之间的联系，这种相互联系反映到数学上就是几种运算关系。如前所述，逻辑代数中定义了“或”、“与”、“非”3 种基本运算。

1. 或运算

在逻辑问题的描述中，如果决定某一事件发生的多个条件中，只要有一个或一个以上条件成立，事件便可发生，则称这种因果关系为或逻辑关系。在逻辑代数中，或逻辑关系用或运算描述。或运算又称逻辑加(Logic Addition)，其运算符号

为“+”，有时也用“∨”表示。两变量或运算的关系可表示为

$$F=A+B \qquad \text{或者} \qquad F=A \vee B$$

读作“F 等于 A 或 B”。这里，A、B 是参加运算的两个逻辑变量，F 表示运算结果。意思是：A、B 中只要有一个为 1，则 F 为 1；仅当 A、B 均为 0 时，F 才为 0。该逻辑关系可用表 2.1 来描述。

表 2.1　或运算表

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

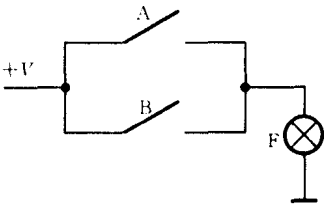


图 2.1　并联开关电路

例如，在图 2.1 所示电路中，开关 A 和 B 并联控制灯 F。可以看出，当开关 A、B 中有一个闭合或者两个均闭合时，灯 F 即亮。因此，灯 F 与开关 A、B 之间的关系是“或”逻辑关系。假定开关断开用 0 表示，开关闭合用 1 表示，灯灭用 0 表示，灯亮用 1 表示，则灯 F 与开关 A、B 的关系即为表 2.1 所示的或运算关系。

由表 2.1 可得出或运算的运算法则为

$$0+0=0 \qquad 1+0=1$$

$$0+1=1 \qquad 1+1=1$$

在数字系统中，实现或运算关系的逻辑电路称为或门。

2. 与运算

在逻辑问题中，如果决定某一事件发生的多个条件必须同时具备，事件才能发生，则称这种因果关系与逻辑。在逻辑代数中，与逻辑关系用与运算描述。与运算又称为逻辑乘(Logic Multiplication)，其运算符号为“·”，有时也用“∧”表示。两变量与运算关系可表示为

$$F=A \cdot B \qquad \text{或者} \qquad F=A \wedge B$$

读作“F 等于 A 与 B”。意思是：若 A、B 均为 1，则 F 为 1；否则，F 为 0。该逻辑关系可用表 2.2 来描述。

例如，在图 2.2 所示电路中，两个开关串联控制同一个灯。显然，仅当两个开关均闭合时，灯才能亮，否则，灯灭。假定开关闭合状态用 1 表示，开关断开状态用 0 表示，灯亮用 1 表示，灯灭用 0 表示，则电路中灯 F 和开关 A、B 之间的关系即为表 2.2 所示的与运算关系。

由表 2.2 可得出与运算的运算法则为

$$0 \cdot 0=0 \qquad 1 \cdot 0=0$$
$$0 \cdot 1=0 \qquad 1 \cdot 1=1$$

在数字系统中，实现与运算关系的逻辑电路称为与门。

表 2.2 与运算表

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

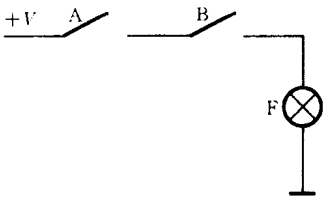


图 2.2 串联开关电路

3. 非运算

在逻辑问题中，如果某一事件的发生取决于条件的否定，即事件与事件发生的条件之间构成矛盾，则称这种因果关系为非逻辑。在逻辑代数中，非逻辑用非运算描述。非运算也叫求反运算或者逻辑否定(Logic Negation)。其运算符号为“—”，有时也用“¬”表示。非运算的逻辑关系可表示为

$$F=\overline{A} \qquad \text{或者} \qquad F=\neg A$$

读作“F 等于 A 非”。意思是：若 A 为 0，则 F 为 1；若 A 为 1，则 F 为 0。该逻辑关系可用表 2.3 描述。

表 2.3 非运算表

A	F
0	1
1	0

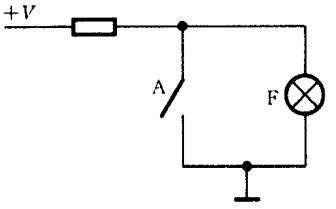


图 2.3 开关与灯并联电路

例如，在图 2.3 所示电路中，开关与灯并联。显然，仅当开关断开时，灯亮；一旦开关闭合，则灯灭。令开关断开用 0 表示，开关闭合用 1 表示，灯亮用 1 表示，灯灭用 0 表示，则电路中灯 F 与开关 A 的关系即为表 2.3 所示的非运算关系。

由表 2.3 可得出非运算的运算法则为

$$\overline{0}=1 \qquad \overline{1}=0$$

在数字系统中，实现非运算功能的逻辑电路称为非门，有时又称为反相器。

上面介绍的或、与、非 3 种逻辑运算是逻辑代数中最基本的逻辑运算，由这些基本逻辑运算可以组成各种复杂的逻辑关系。

2.1.2　逻辑函数及逻辑函数间的相等

在实际逻辑问题中，前面介绍的 3 种基本运算是很少单独出现的。通常，总是以这些基本运算构成各种复杂程度不一的逻辑关系来描述各种各样的逻辑问题。对各种复杂的逻辑关系必须进一步用数学方法加以描述，这就引出了逻辑函数的概念。

1. 逻辑函数的定义

逻辑代数中函数的定义与普通代数中函数的定义极为相似。但和普通代数中函数的概念相比，逻辑函数具有它自身的特点：

①逻辑变量和逻辑函数的取值只有 0 和 1 两种可能；

②函数和变量之间的关系是由或、与、非 3 种基本运算决定的。

从数字系统研究的角度看，逻辑函数的定义可叙述如下：

设某一逻辑电路的输入逻辑变量为 $A_1, A_2, \cdots, A_n$，输出逻辑变量为 F，如图 2.4 所示。如果当 $A_1, A_2, \cdots, A_n$ 的值确定后，F 的值就惟一地确定下来，则称 F 为 $A_1, A_2, \cdots, A_n$ 的逻辑函数，记为

$$F=f(A_1, A_2, \cdots, A_n)$$



图 2.4　广义的逻辑电路

图 2.4 所示为一广义的逻辑电路。相对某一具体的逻辑电路而言，其逻辑变量的取值是“自行”变化的，而逻辑函数的取值则是由逻辑变量的取值和电路本身的结构决定的。在逻辑电路和逻辑函数之间存在着严格的对应关系，任何一个逻辑电路的全部属性和功能都可由相应的逻辑函数完全描述，这便使我们能够将一个具体的逻辑电路转换为抽象的代数表达式，从而很方便地对它加以分析研究。

2. 逻辑函数的相等

逻辑函数和普通代数中的函数一样存在相等的问题。什么叫做两个逻辑函数相等呢？

设有两个逻辑函数

$$F_1=f_1(A_1, A_2, \cdots, A_n)$$

$$F_2=f_2(A_1, A_2, \cdots, A_n)$$

若对应于逻辑变量 $A_1, A_2, \cdots, A_n$ 的任何一组取值，F_1 和 F_2 的值都相同，则称函数 F_1 和 F_2 相等，记作 $F_1=F_2$。

判断两个逻辑函数是否相等，通常有两种方法：一种方法是列出输入变量所有可能的取值组合，并按逻辑运算法则计算出各种输入取值下两个函数的相应值，然

后进行比较；另一种方法是用逻辑代数的公理、定理和规则进行证明。

2.1.3 逻辑函数的表示法

描述逻辑函数的方法并不是惟一的，常用的方法有逻辑表达式、真值表、卡诺图 3 种。

1. 逻辑表达式

逻辑表达式是由逻辑变量和或、与、非 3 种运算符所构成的式子。例如，

$$F=f(A,B)=\overline{A}\cdot B+A\cdot\overline{B}$$

为一由两个变量(A 和 B)进行逻辑运算构成的逻辑表达式，它描述了一个两变量的逻辑函数 F。函数 F 和变量 A、B 的关系是：当变量 A 和 B 取值不同时，函数 F 的值为“1”；否则，函数 F 的值为“0”。

关于逻辑表达式的书写，为了简便起见，可按下述规则省略某些括号或运算符号：

① 进行非运算可不加括号，如 $\overline{A}$，$\overline{A+B}$等。

②与运算符一般可省略，如 A · B 可写成 AB。

③ 在一个表达式中，如果既有与运算又有或运算，则按先与后或的规则进行运算，如(A · B)+(C · D)可以省略括号写成 AB+CD。

④ 由于“与”运算和“或”运算均满足结合律，因此，(A+B)+C 或者 A+(B+C)可用 A+B+C 代替；(AB)C 或者 A(BC)可用 ABC 代替。

2. 真值表

用真值表描述逻辑函数的方法是一种表格表示法。由于一个逻辑变量只有 0 和 1 两种可能的取值，故 n 个逻辑变量一共只有 2^n种可能的取值组合。任何逻辑函数总是和若干个逻辑变量相关的，由于变量的个数是有限的，所以变量取值组合的总数也必然是有限的，因此，可以用穷举的方法来描述逻辑函数的功能。

为了清晰，常用的方法是对一个函数求出所有输入变量取值下的函数值并用表格形式记录下来，这种表格称为**真值表**。换而言之，真值表是一种由逻辑变量的所有可能取值组合及其对应的逻辑函数值所构成的表格。

真值表由两部分组成，左边一栏列出变量的所有取值组合，为了不发生遗漏，通常各变量取值组合按二进制数码顺序给出；右边一栏为逻辑函数值。例如，函数 $F=A\overline{B}+\overline{A}C$的真值表如表 2.4 所示。事实上，前面介绍 3 种基本运算时所列出的表 2.1、表 2.2、表 2.3 分别是或、与、非 3 种逻辑运算的真值表。

真值表是一种十分有用的逻辑工具。在逻辑问题的分析和设计中，将经常用到这一工具。

表 2.4　函数 $F=A\bar{B}+\bar{A}C$ 的真值表

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

3. 卡诺图

卡诺图是由表示逻辑变量所有取值组合的小方格所构成的平面图。它是一种用图形描述逻辑函数的方法，这种方法在逻辑函数化简中十分有用，将在后面结合函数化简问题进行详细介绍。

上述表示逻辑函数的 3 种方法各有特点，它们各适用于不同场合。但针对某个具体问题而言，仅仅是同一问题的不同描述形式，彼此之间可以很方便地相互变换。

2.2　逻辑代数的基本定理和规则

逻辑代数和普通代数一样，作为一个完整的代数系统，它具有用于运算的一些定理和规则。本节将对逻辑代数的一些基本定理和规则进行介绍。

2.2.1　基本定理

根据逻辑代数的公理，可以推导出逻辑代数的基本定理。由于每条公理的逻辑表达式是成对出现的，所以由公理推导出来的定理的逻辑表达式也是成对出现的。下面只对定理中的一个表达式加以证明，另一个留给读者自己证明。

定理 1　　$0+0=0$　　$1+0=1$　　$0+1=1$　　$1+1=1$

$0\cdot 0=0$　　$1\cdot 0=0$　　$0\cdot 1=0$　　$1\cdot 1=1$

证明　在公理 4 中，A 表示集合 K 中的任意元素，因而可以是 0 或 1。于是，以 0 和 1 代替公理 4 中的 A，就得到上述关系。

如果以 1 和 0 代替公理 5 中的 A，则可得到如下推论：

$\bar{1}=0$　　$\bar{0}=1$

定理 2　　$A+A=A$　　$A\cdot A=A$

证明

$$A+A=(A+A)\cdot 1 \qquad \text{公理 4}$$

$$=(A+A)\cdot(A+\bar{A}) \qquad \text{公理 5}$$

$$=A+(A\cdot\bar{A}) \qquad \text{公理 3}$$

$$=A+0 \qquad \text{公理 5}$$

$$=A \qquad \text{公理 4}$$

定理 3　　$A+A\cdot B=A$　　$A\cdot(A+B)=A$

证明　　$A+A\cdot B=A\cdot 1+A\cdot B$　　公理 4

$=A\cdot(1+B)$　　公理 3

$=A(B+1)$　　公理 1

$=A\cdot 1$　　公理 4

$=A$　　公理 4

定理 4　　$A+\overline{A}\cdot B=A+B$　　$A\cdot(\overline{A}+B)=A\cdot B$

证明　　$A+\overline{A}\cdot B=(A+\overline{A})\cdot(A+B)$　　公理 3

$=1\cdot(A+B)$　　公理 5

$=A+B$　　公理 4

定理 5　　$\overline{\overline{A}}=A$

证明　令　　$\overline{A}=X$

因而　　$\overline{A}\cdot X=0$　　$\overline{A}+X=1$　　公理 5

但是　　$A\cdot\overline{A}=0$　　$\overline{A}+A=1$　　公理 5

由于 X 和 A 都满足公理 5，因此，根据公理 5 的惟一性，得到

$A=X$

定理 6　　$\overline{A+B}=\overline{A}\cdot\overline{B}$　　$\overline{A\cdot B}=\overline{A}+\overline{B}$

证明　由于　　$(\overline{A}\cdot\overline{B})+(A+B)=(\overline{A}\cdot\overline{B}+A)+B$　　公理 2

$=(\overline{B}+A)+B$　　定理 4

$=A+(\overline{B}+B)$　　公理 1,2

$=A+1$　　公理 5

$=1$　　公理 4

而且　　$(\overline{A}\cdot\overline{B})\cdot(A+B)=\overline{A}\cdot\overline{B}\cdot A+\overline{A}\cdot\overline{B}\cdot B$　　公理 3

$=0+0$　　公理 1,5

$=0$　　定理 1

所以，根据公理 5 的惟一性可得到

$\overline{A+B}=\overline{A}\cdot\overline{B}$

定理 7　　$A\cdot B+A\cdot\overline{B}=A$　　$(A+B)\cdot(A+\overline{B})=A$

证明　　$A\cdot B+A\cdot\overline{B}=A\cdot(B+\overline{B})$　　公理 3

$=A\cdot 1$　　公理 5

$=A$　　公理 4

定理 8　　$A\cdot B+\overline{A}\cdot C+B\cdot C=A\cdot B+\overline{A}\cdot C$

$(A+B)\cdot(\overline{A}+C)\cdot(B+C)=(A+B)\cdot(\overline{A}+C)$

证明　　$A\cdot B+\overline{A}\cdot C+B\cdot C$

$$=A\cdot B+\overline{A}\cdot C+B\cdot C\cdot(A+\overline{A}) \qquad \text{公理 5}$$
$$=A\cdot B+\overline{A}\cdot C+B\cdot C\cdot A+B\cdot C\cdot\overline{A} \qquad \text{公理 3}$$
$$=A\cdot B+A\cdot B\cdot C+\overline{A}\cdot C+\overline{A}\cdot B\cdot C \qquad \text{公理 1}$$
$$=A\cdot B\cdot(1+C)+\overline{A}\cdot C\cdot(1+B) \qquad \text{公理 3}$$
$$=A\cdot B\cdot(C+1)+\overline{A}\cdot C\cdot(B+1) \qquad \text{公理 1}$$
$$=A\cdot B+\overline{A}\cdot C \qquad \text{公理 4}$$

2.2.2　重要规则

逻辑代数有3条重要规则，即代入规则、反演规则和对偶规则。这些规则在逻辑运算中十分有用，要求熟练地掌握。

1. 代入规则

任何一个含有变量A的逻辑等式，如果将所有出现A的位置都代之以同一个逻辑函数F，则等式仍然成立。这个规则称为代入规则。

例如，给定逻辑等式$A(B+C)=AB+AC$，若等式中的C都用$(C+D)$代替，则该等式仍然成立，即

$$A[B+(C+D)]=AB+A(C+D)$$

代入规则的正确性是显然的，因为任何逻辑函数都和逻辑变量一样，只有0和1两种可能的取值。

代入规则在推导公式中有重要意义。利用这条规则可以将逻辑代数公理、定理中的变量用任意函数代替，从而推导出更多的等式。这些等式可直接当作公式使用，无需另加证明。

例如，已知$A+\overline{A}=1$(公理5)，逻辑函数$F=f(A_1,A_2,\cdots,A_n)$，将等式中的变量A用函数F取代，便可得到等式

$$f(A_1,A_2,\cdots,A_n)+\overline{f}(A_1,A_2,\cdots,A_n)=1$$

即一个函数和其反函数进行“或”运算，其结果为1。

值得注意的是，使用代入规则时必须将等式中所有出现同一变量的地方均以同一函数代替，否则代入后的等式将不成立。

2. 反演规则

如果将逻辑函数表达式F中所有的“·”变成“+”，“+”变成“·”，“0”变成“1”，“1”变成“0”，原变量变成反变量，反变量变成原变量，并保持原函数中的运算顺序不变，则所得到的新的函数为原函数F的反函数$\overline{F}$。这一规则称为反演规则。

例如，已知函数 $F=\overline{A}B+C\overline{D}$，根据反演规则可得到

$$\overline{F}=(A+\overline{B})\cdot(\overline{C}+D)$$

反演规则实际上是定理 6 的推广，可通过定理 6 和代入规则得到证明。

显然，运用反演规则可以很方便地求出一个函数的反函数。使用反演规则时，应注意保持原函数式中运算的优先顺序不变。

例如，已知函数 $F=\overline{A}+\overline{B}\cdot(C+\overline{D}E)$，根据反演规则得到的反函数应该是

$$\overline{F}=A\cdot[B+\overline{C}\cdot(D+\overline{E})]$$

而不应该是 $\overline{F}=A\cdot B+\overline{C}\cdot D+\overline{E}$

3. 对偶规则

如果将逻辑函数表达式 F 中所有的“·”变成“+”，“+”变成“·”，“0”变成“1”，“1”变成“0”，并保持原函数中的运算顺序不变，则所得到的新的逻辑表达式称为函数 F 的对偶式，并记作 F'。例如，

$F_1=\overline{A}B+\overline{B}(C+0)$　　$F_1'=(\overline{A}+B)(\overline{B}+C\cdot 1)$

$F_2=(\overline{A}+B)(\overline{B}+C\cdot 1)$　　$F_2'=\overline{A}B+\overline{B}(C+0)$

$F_3=AB+\overline{A}C+C(D+E)$　　$F_3'=(A+B)(\overline{A}+C)(C+DE)$

$F_4=(A+B)(\overline{A}+C)(C+DE)$　　$F_4'=AB+\overline{A}C+C(D+E)$

从上面的例子可以看出，如果 F 的对偶式是 F'，则 F' 的对偶式就是 F。也就是说，F 和 F' 互为对偶式。

有些逻辑函数表达式的对偶式就是原函数表达式本身，即 $F'=F$。这时，称函数 F 为自对偶函数。

例如，函数 $F=(A+\overline{C})\overline{B}+A(\overline{B}+\overline{C})$ 是一自对偶函数，因为

$$\begin{aligned}F'&=(A\cdot\overline{C}+\overline{B})\cdot(A+\overline{B}\overline{C})\\&=(A+\overline{B})(\overline{C}+\overline{B})(A+\overline{B})(A+\overline{C})\\&=(A+\overline{B})(\overline{C}+\overline{B})(A+\overline{C})\\&=A(\overline{B}+\overline{C})(A+\overline{C})+\overline{B}(\overline{B}+\overline{C})(A+\overline{C})\\&=(\overline{B}+\overline{C})(A+A\overline{C})+(\overline{B}+\overline{B}\,\overline{C})(A+\overline{C})\\&=A(\overline{B}+\overline{C})+\overline{B}(A+\overline{C})\\&=F\end{aligned}$$

此外，求某一逻辑表达式的对偶式时，同样要注意保持原函数的运算顺序不变。

若两个逻辑函数表达式 F 和 G 相等，则其对偶式 F' 和 G' 也相等。这一规则称为对偶规则。

根据对偶规则，当已证明某两个逻辑表达式相等时，便可知道它们的对偶式也相等。

例如，已知 $AB+\overline{A}C+\overline{B}C=AB+C$，根据对偶规则可知等式两端表达式的对偶式也相等，即有

$$(A+B)\cdot(\overline{A}+C)\cdot(\overline{B}+C)=(A+B)\cdot C$$

显然，利用对偶规则可以使定理、公式的证明减少一半。

2.2.3　复合逻辑

前面提到，用来实现与、或、非3种基本运算的逻辑电路分别称为与门、或门、非门，它们是最基本的逻辑门。尽管由这3种基本逻辑门可以实现各种复杂的逻辑功能，但实际应用中更广泛采用的是与非门、或非门、与或非门、异或门等逻辑功能更强的逻辑门。这些逻辑门输出和输入之间的逻辑关系可由3种基本运算构成的复合运算来描述，故通常将这种逻辑关系称为复合逻辑关系，相应的逻辑门则称为复合门。

1. 与非逻辑及或非逻辑

(1) 与非逻辑

与非逻辑是由与、非两种逻辑复合形成的，可用逻辑函数表示为

$$F=\overline{A\cdot B\cdot C\cdots}$$

其逻辑功能是：只要变量A,B,C,…中有一个为0，则函数F为1；仅当变量A,B,C,…全部为1时，函数F为0。

实现与非逻辑的逻辑门称为与非门。

由定理$\overline{A\cdot B}=\overline{A}+\overline{B}$不难看出，“与”之“非”可以产生“或”的关系。因此，实际上只要有了与非逻辑便可实现与、或、非3种基本逻辑。以两变量与非逻辑为例：

与　　$F=\overline{AB\cdot 1}=\overline{AB}=A\cdot B$

或　　$F=\overline{\overline{A\cdot 1}\cdot\overline{B\cdot 1}}=\overline{\overline{A}\cdot\overline{B}}=A+B$

非　　$F=\overline{A\cdot 1}=\overline{A}$

由于任何一种逻辑关系均可由3种基本逻辑通过适当的组合来实现，而与非逻辑又可实现3种基本逻辑，所以，只要有了与非门便可组成实现各种逻辑功能的电路。由此可见，采用与非逻辑可以减少逻辑电路中门的种类，提高标准化程度。

(2) 或非逻辑

或非逻辑是由或、非两种逻辑复合形成的，可用逻辑函数表示为

$$F=\overline{A+B+C+\cdots}$$

其逻辑功能是：只要变量A,B,C,…中有一个为1，则函数F为0；仅当变量A,B,C,…全部为0时，函数F为1。

实现或非逻辑的逻辑门称为或非门。

同样，由定理$\overline{A+B}=\overline{A}\cdot\overline{B}$可知，“或”之“非”可以产生“与”的关系。因此，或非逻辑也可以实现与、或、非3种基本逻辑。下面以两变量或非逻辑为例进行说明：

与　　$F=\overline{\overline{A+0}+\overline{B+0}}=\overline{\overline{A}+\overline{B}}=A\cdot B$

或　　$F=\overline{\overline{A+B}+0}=\overline{\overline{A+B}}=A+B$

非　　$F=\overline{A+0}=\overline{A}$

可见，只要有了或非门，同样可组成实现各种逻辑功能的逻辑电路。

2. 与或非逻辑

与或非逻辑是由3种基本逻辑复合形成的，其逻辑函数表达式的形式为

$$F=\overline{AB+CD+\cdots}$$

其逻辑功能是：仅当每一个与项均为0时，才能使F为1，否则F为0。

实现与或非功能的逻辑门称为与或非门。

显然，与或非逻辑可以实现与、或、非的功能，这就意味着可以仅用与或非门去组成实现各种功能的逻辑电路。但实际应用中这样做一般会很不经济，所以，与或非门主要用来实现与或非形式的函数。必要时可将逻辑函数表达式的形式变换成与或非的形式，以便使用与或非门来实现其逻辑功能。例如，

$$\begin{aligned}F&=\overline{A}B+C=\overline{\overline{A}B\cdot\overline{C}}\\&=\overline{(A+\overline{B})\cdot\overline{C}}=\overline{A\overline{C}+\overline{B}\overline{C}}\end{aligned}$$

该函数经变换后可用与或非门来实现。

3. 异或逻辑及同或逻辑

(1) 异或逻辑

异或逻辑是一种两变量逻辑关系，可用逻辑函数表示为

$$F=A\oplus B=\overline{A}B+A\overline{B}$$

式中，$\oplus$是异或运算的运算符。

异或逻辑的功能是：变量A、B取值相同，F为0；变量A、B取值相异，F为1。

实现异或运算的逻辑门称为异或门。

根据异或逻辑的定义可知：

$$A\oplus 0=A\qquad A\oplus 1=\overline{A}$$

$$A\oplus A=0\qquad A\oplus\overline{A}=1$$

当多个变量进行异或运算时，可用两两运算的结果再运算，也可两两依次运算。例如，

$$F=A\oplus B\oplus C\oplus D=(A\oplus B)\oplus(C\oplus D)=[(A\oplus B)\oplus C]\oplus D$$

在进行异或运算的多个变量中，若有奇数个变量的值为1，则运算结果为1；若有偶数个变量的值为1，则运算结果为0。

(2) 同或逻辑

同或逻辑也是一种两变量逻辑关系，其逻辑函数表达式为

$$F=A\odot B=\overline{A}\,\overline{B}+AB$$

式中，“$\odot$”为同或运算的运算符。

同或逻辑的功能是：变量A、B取值相同，F为1；变量A、B取值相异，F为0。

实现同或运算的逻辑门称为同或门。

同或逻辑与异或逻辑的关系既互为相反，又互为对偶，即有

$$\overline{A\oplus B}=\overline{A\overline{B}+\overline{A}B}=(\overline{A}+B)(A+\overline{B})$$
$$=AB+\overline{A}\,\overline{B}=A\odot B$$
$$(A\oplus B)'=(A\overline{B}+\overline{A}B)'=(A+\overline{B})(\overline{A}+B)$$
$$=\overline{A}\,\overline{B}+AB=A\odot B$$

当多个变量进行同或运算时，若有奇数个变量的值为0，则运算结果为0；反之，若有偶数个变量的值为0，则运算结果为1。

由于同或实际上是异或之非，所以实际应用中通常用异或门加非门实现同或运算。

2.3　逻辑函数表达式的形式与变换

任何一个逻辑函数，其表达式的形式都不是惟一的。下面从分析与应用的角度出发，介绍逻辑函数表达式的基本形式、标准形式及其相互转换。

2.3.1　逻辑函数表达式的基本形式

逻辑函数表达式有“与-或”表达式和“或-与”表达式两种基本形式。

1. 与-或表达式

所谓与-或表达式是指由若干与项进行或运算构成的表达式。每个与项可以是单个变量的原变量或者反变量，也可以由多个原变量或者反变量相与组成。例如，$\overline{A}B$、$A\overline{B}C$、$\overline{C}$均为与项，将这3个与项相或便可构成一个3变量函数的与-或表达式。即

$$F(A,B,C)=\overline{A}B+A\overline{B}C+\overline{C}$$

与项有时又被称为积项，相应地与-或表达式又称为“积之和”表达式。

2. 或-与表达式

所谓或-与表达式是指由若干或项进行与运算构成的表达式。每个或项可以是单个变量的原变量或者反变量，也可以由多个原变量或者反变量相或组成。例如，$(\overline{A}+B)$、$(B+\overline{C})$、$(A+\overline{B}+C)$、$\overline{D}$ 均为或项，将这 4 个或项相与便可构成一个 4 变量函数的或-与表达式。即

$$F(A,B,C,D)=(\overline{A}+B)(B+\overline{C})(A+\overline{B}+C)\overline{D}$$

或项有时又被称为和项，相应地或-与表达式又称为“和之积”表达式。

通常逻辑函数表达式可以被表示成任意的混合形式，如函数 $F(A,B,C)=(A\overline{B}+C)(\overline{A}+\overline{B\overline{C}})+\overline{B}$ 既不是与-或式也不是或-与式，但不论什么形式都可以变换成上述两种基本形式。

2.3.2 逻辑函数表达式的标准形式

逻辑函数的两种基本形式都不是惟一的。为了在逻辑问题的研究中使逻辑函数能和惟一的表达式对应，引入了逻辑函数表达式的标准形式。

1. 最小项和最大项

逻辑函数表达式的标准形式是建立在最小项和最大项概念的基础之上的。所以，下面首先介绍最小项和最大项的定义和性质。

(1) 最小项的定义和性质

定义 如果一个具有 n 个变量的函数的与项包含全部 n 个变量，每个变量都以原变量或反变量形式出现，且仅出现一次，则该与项被称为最小项。有时又将最小项称为标准与项 。

由定义可知，n 个变量可以构成 2^n 个最小项。例如，3 个变量 A，B，C 可以构成 $\overline{A}\overline{B}\overline{C}$，$\overline{A}\overline{B}C$，…，ABC 共 8 个最小项。

为了书写方便，在变量个数和变量顺序确定之后，通常用 m_i 表示最小项。下标 i 的取值规则是：按照变量顺序将最小项中的原变量用 1 表示，反变量用 0 表示，由此得到一个二进制数，与该二进制数对应的十进制数即下标 i 的值。例如，3 变量 A，B，C 构成的最小项 $A\overline{B}C$ 可用 m_5 表示。

最小项具有如下性质：

性质 1 任意一个最小项，其相应变量有且仅有一种取值使这个最小项的值为 1。并且，最小项不同，使其值为 1 的变量取值也不同。

显然，在由 n 个变量构成的任意与项中，最小项是使其值为 1 的变量取值组合

数最少的一种与项，因而将其称为最小项。

性质 2　相同变量构成的两个不同最小项相与为 0。

因为任何一种变量取值都不可能使两个不同最小项同时为 1，故相与为 0。

性质 3　n 个变量的全部最小项相或为 1。通常借用数学中的累加符号“$\sum$”，将其记为

$$\sum_{i=0}^{2^n-1} m_i = 1$$

性质 4　n 个变量构成的最小项有 n 个相邻最小项。

相邻最小项是指除一个变量互为相反外，其余部分均相同的最小项。例如，$A\bar{B}C$ 和 $A\bar{B}\bar{C}$。

(2) 最大项的定义和性质

定义　如果一个具有 n 个变量的函数的或项包含全部 n 个变量，每个变量都以原变量或反变量形式出现，且仅出现一次，则该或项被称为最大项。有时又将最大项称为标准或项。

n 个变量可以构成 2^n 个最大项。例如，3 个变量 A，B，C 可构成 $A+B+C$，$A+B+\bar{C}$，…，$\bar{A}+\bar{B}+\bar{C}$ 共 8 个最大项。

为了书写方便，在变量个数和变量顺序确定之后，通常用 M_i 表示最大项。下标 i 的取值规则是：按照变量顺序将最大项中的原变量用 0 表示，反变量用 1 表示，由此得到一个二进制数，与该二进制数对应的十进制数即下标 i 的值。例如，3 变量 A，B，C 构成的最大项 $\bar{A}+B+\bar{C}$ 可用 M_5 表示。

最大项具有如下性质：

性质 1　任意一个最大项，其相应变量有且仅有一种取值使这个最大项的值为 0。并且，最大项不同，使其值为 0 的变量取值也不同。

由此可见，在由 n 个变量构成的任意或项中，最大项是使其值为 1 的变量取值组合数最多的一种或项，因而将其称为最大项。

性质 2　相同变量构成的两个不同最大项相或为 1。

因为任何一种变量取值都不可能使两个不同最大项同时为 0，故相或为 1。

性质 3　n 个变量的全部最大项相与为 0。通常借用数学中的累乘符号“$\prod$”将其记为

$$\prod_{i=0}^{2^n-1} M_i = 0$$

性质 4　n 个变量构成的最大项有 n 个相邻最大项。

相邻最大项是指除一个变量互为相反外，其余变量均相同的最大项。

2 变量最小项、最大项真值表如表 2.5 所示。真值表体现了最小项、最大项的

有关性质。

表 2.5 2 变量最小项、最大项真值表

变量	最小项				最大项			
A B	$\bar{A}\bar{B}$	$\bar{A}B$	$A\bar{B}$	AB	$A+B$	$A+\bar{B}$	$\bar{A}+B$	$\bar{A}+\bar{B}$
	m_0	m_1	m_2	m_3	M_0	M_1	M_2	M_3
0 0	1	0	0	0	0	1	1	1
0 1	0	1	0	0	1	0	1	1
1 0	0	0	1	0	1	1	0	1
1 1	0	0	0	1	1	1	1	0

(3) 最小项和最大项的关系

在前面讨论最小项、最大项的简写形式时，为什么在确定最小项 m_i 的下标 i 时，令标准与项中的反变量用 0 表示、原变量用 1 表示，而在确定最大项 M_i 的下标时，却令标准或项中的反变量用 1 表示，原变量用 0 表示呢？因为这样做可使在同一问题中下标相同的最小项和最大项互为反函数。或者说，相同变量构成的最小项 m_i 和最大项 M_i 之间存在互补关系。即

$$\overline{m}_i = M_i \quad \text{或者} \quad m_i = \overline{M}_i$$

例如，由 3 变量 A、B、C 构成的最小项 m_3 和最大项 M_3 之间有

$$\overline{m}_3 = \overline{\bar{A}BC} = A+\bar{B}+\bar{C} = M_3$$

$$\overline{M}_3 = \overline{A+\bar{B}+\bar{C}} = \bar{A}BC = m_3$$

2. 逻辑函数表达式的标准形式

逻辑函数表达式的标准形式有标准“与-或”表达式和标准“或-与”表达式两种类型。

(1) 标准与-或表达式

由若干最小项相或构成的逻辑表达式称为标准“与-或”表达式，也叫做最小项表达式。

例如，$\bar{A}\bar{B}C$，$\bar{A}B\bar{C}$，$A\bar{B}\bar{C}$，ABC 为 3 变量构成的 4 个最小项，对这 4 个最小项进行“或”运算，即可得到一个 3 变量函数的标准“与-或”表达式

$$F(A,B,C) = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

该函数表达式又可简写为

$$F(A,B,C) = m_1 + m_2 + m_4 + m_7 = \sum m(1,2,4,7)$$

(2) 标准或-与表达式

由若干最大项相与构成的逻辑表达式称为标准或-与表达式，也叫做最大项表达式。

例如，$A+B+C$，$\overline{A}+B+\overline{C}$，$\overline{A}+\overline{B}+\overline{C}$ 为3变量构成的3个最大项，对这3个最大项进行与运算，即可得到一个3变量函数的标准或-与表达式

$$F(A,B,C)=(A+B+C)(\overline{A}+B+\overline{C})(\overline{A}+\overline{B}+\overline{C})$$

该表达式又可简写为

$$F(A,B,C)=M_0M_5M_7=\prod M(0,5,7)$$

2.3.3　逻辑函数表达式的转换

将一个任意逻辑函数表达式转换成标准表达式有两种常用方法，一种是代数转换法，另一种是真值表转换法。

1. 代数转换法

所谓代数转换法，就是利用逻辑代数的公理、定理和规则进行逻辑变换，将函数表达式从一种形式变换为另一种形式。

用代数转换法求一个函数的标准与-或表达式，一般分为两步。

第一步：将函数表达式变换成一般与-或表达式。

第二步：反复使用 $X=X\cdot(Y+\overline{Y})$，将表达式中所有非最小项的与项扩展成最小项。

例2.1　将逻辑函数表达式 $F(A,B,C)=\overline{(A\overline{B}+B\overline{C})\cdot\overline{AB}}$ 转换成标准与-或表达式。

解　第一步：将函数表达式变换成与-或表达式。即

$$\begin{aligned}F(A,B,C)&=\overline{(A\overline{B}+B\overline{C})\cdot\overline{AB}}\\&=\overline{A\overline{B}+B\overline{C}}+AB\\&=(\overline{A}+B)(\overline{B}+C)+AB\\&=\overline{A}\overline{B}+\overline{A}C+BC+AB\end{aligned}$$

第二步：把所得与-或式中的与项扩展成最小项。具体地说，若某与项缺少函数变量 Y，则用 $(\overline{Y}+Y)$ 和这一项相与，并把它拆开成两项。即

$$\begin{aligned}F(A,B,C)&=\overline{A}\overline{B}(\overline{C}+C)+\overline{A}C(\overline{B}+B)+(\overline{A}+A)BC+AB(\overline{C}+C)\\&=\overline{A}\overline{B}\overline{C}+\overline{A}\overline{B}C+\overline{A}\overline{B}C+\overline{A}BC+\overline{A}BC+ABC+AB\overline{C}+ABC\\&=\overline{A}\overline{B}\overline{C}+\overline{A}\overline{B}C+\overline{A}BC+AB\overline{C}+ABC\end{aligned}$$

该标准与-或式的简写形式为

$$\begin{aligned}F(A,B,C)&=m_0+m_1+m_3+m_6+m_7\\&=\sum m(0,1,3,6,7)\end{aligned}$$

当给出函数表达式已经是与-或表达式时，可直接进行第二步。

类似地，用代数转换法求一个函数的标准或-与表达式同样分为两步。

第一步：将函数表达式转换成一般或-与表达式。

第二步：反复利用定理 $X=(X+\overline{Y})(X+Y)$ 把表达式中所有非最大项的或项扩展成最大项。

例 2.2 将逻辑函数表达式 $F(A,B,C)=\overline{A\overline{B}+\overline{A}C}+\overline{B}C$ 变换成标准或-与表达式。

解 第一步：将函数表达式变换成或-与表达式。即

$$\begin{aligned}F(A,B,C)&=\overline{A\overline{B}+\overline{A}C}+\overline{B}C\\&=\overline{A\overline{B}}\cdot\overline{\overline{A}C}+\overline{B}C\\&=(\overline{A}+\overline{B})(A+\overline{C})+\overline{B}C\\&=[(\overline{A}+\overline{B})(A+\overline{C})+\overline{B}]\cdot[(\overline{A}+\overline{B})(A+\overline{C})+C]\\&=(\overline{A}+\overline{B}+\overline{B})(A+\overline{C}+\overline{B})(\overline{A}+\overline{B}+C)(A+\overline{C}+C)\\&=(\overline{A}+\overline{B})(A+\overline{B}+\overline{C})(\overline{A}+\overline{B}+C)\end{aligned}$$

第二步：将所得或-与表达式中的非最大项扩展成最大项。即

$$\begin{aligned}F(A,B,C)&=(\overline{A}+\overline{B})(A+\overline{B}+\overline{C})(\overline{A}+\overline{B}+C)\\&=(\overline{A}+\overline{B}+\overline{C})(\overline{A}+\overline{B}+C)(A+\overline{B}+\overline{C})(\overline{A}+\overline{B}+C)\\&=(A+\overline{B}+\overline{C})(\overline{A}+\overline{B}+C)(\overline{A}+\overline{B}+\overline{C})\end{aligned}$$

该标准或-与表达式的简写形式为

$$F(A,B,C)=M_3\cdot M_6\cdot M_7=\prod M(3,6,7)$$

当给出函数已经是或-与表达式时，可直接进行第二步。

2. 真值表转换法

一个逻辑函数的真值表与它的最小项表达式具有一一对应的关系。假定在函数 F 的真值表中有 k 组变量取值使 F 的值为 1、其他变量取值下 F 的值为 0，那么，函数 F 的最小项表达式由这 k 组变量取值对应的 k 个最小项组成。因此，求一个函数的最小项表达式时，可以通过先列出该函数的真值表，然后根据真值表写出最小项表达式。

例 2.3 将函数表达式 $F(A,B,C)=A\overline{B}+B\overline{C}$ 表示成最小项表达式。

解 首先，列出 F 的真值表如表 2.6 所示，然后，根据真值表直接写出 F 的最小项表达式

$$F(A,B,C)=\sum m(2,4,5,6)$$

表 2.6　函数 $F(A,B,C)=A\bar{B}+B\bar{C}$ 的真值表

$F(A,B,C)=\sum m(2,4,5,6)$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

类似地，一个逻辑函数的真值表与它的最大项表达式之间同样具有一一对应的关系。假定在函数 F 的真值表中有 k 组变量取值使 F 的值为 0、其他变量取值下 F 的值为 1，那么，函数 F 的最大项表达式由这 k 组变量取值对应的 k 个最大项组成。因此，当求一个函数的最大项表达式时，可以先列出函数真值表，然后根据真值表直接写出最大项表达式。

例 2.4　将函数表达式 $F(A,B,C)=\bar{A}C+A\bar{B}\bar{C}$ 表示成最大项表达式的形式。

解　首先，列出 F 的真值表如表 2.7 所示。然后，根据真值表直接写出 F 的最大项表达式

$$F(A,B,C)=\prod M(0,2,5,6,7)$$

由于函数的真值表与函数的两种标准表达式之间存在一一对应的关系，而任何一个逻辑函数的真值表是惟一的，所以，任何一个逻辑函数的两种标准形式也是惟一的。这给我们分析和研究逻辑问题带来了很大的方便。

表 2.7　函数 $F(A,B,C)=\bar{A}C+A\bar{B}\,\bar{C}$ 的真值表

$F(A,B,C)=\prod M(0,2,5,6,7)$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

2.4 逻辑函数化简

逻辑函数表达式有各种不同的表示形式，即使同一类型的表达式也有繁有简。对于某一个逻辑函数来说，尽管函数表达式的形式不同，但所描述的逻辑功能是相同的。在数字系统中，实现某一逻辑功能的逻辑电路的复杂性与描述该功能的逻辑表达式的复杂性直接相关。一般说，逻辑函数表达式越简单，设计出来的相应逻辑电路也就越简单。然而，从逻辑问题概括出来的逻辑函数通常都不是最简的，因此，为了简化电路结构、降低系统成本、提高可靠性，必须对逻辑函数进行化简。通常，把逻辑函数化简成最简形式称为逻辑函数的最小化。

在各种各样的逻辑表达式中，与-或表达式和或-与表达式是最基本的形式，通过这两种基本形式可以很方便地将逻辑表达式转换成任何其他所要求的形式。因此，将从这两种基本形式出发讨论函数化简问题，并将重点放在与-或表达式的化简上。

下面，分别介绍逻辑函数化简的 3 种方法，即代数化简法、卡诺图化简法和列表化简法。

2.4.1 代数化简法

代数化简法是指运用逻辑代数的公理、定理和规则对逻辑函数进行化简的方法。这种方法没有固定的步骤可以遵循，主要取决于对逻辑代数中公理、定理和规则的熟练掌握及灵活运用的程度。尽管如此，还是可以总结出一些适用于大多数情况的常用方法。

1. 与-或表达式的化简

最简与-或表达式应满足两个条件：

① 表达式中的与项个数最少；

② 在满足条件①的前提下，每个与项中的变量个数最少。

满足上述两个条件可以使相应逻辑电路中所需门的数量、门的输入端个数以及相互连线均为最少，从而使电路最经济。

化简与-或表达式有以下几种常用方法。

(1) 并项法

利用定理 7 中的 $AB+A\overline{B}=A$，将两个与项合并成一个与项，合并后消去一个变量。例如，

$$\overline{A}BC+\overline{A}B\overline{C}+\overline{A}\overline{B}=\overline{A}B+\overline{A}\overline{B}=\overline{A}$$

(2) 吸收法

利用定理3中 $A+AB=A$，消去多余的项。例如，

$$\bar{A}B+\bar{A}B\bar{C}D(E+\bar{F})=\bar{A}B$$

(3) 消去法

利用定理4中 $A+\bar{A}B=A+B$，消去多余变量。例如，

$$\begin{aligned}AB+\bar{A}\bar{C}+B\bar{C}&=AB+(\bar{A}+B)\bar{C}\\&=AB+\overline{AB}\,\bar{C}\\&=AB+\bar{C}\end{aligned}$$

(4) 配项法

利用公理4和公理5中的 $A\cdot 1=A$ 及 $A+\bar{A}=1$，先从函数式中适当选择某些与项，并配上其所缺的一个合适的变量，然后再利用并项、吸收和消去等方法进行化简。例如，

$$\begin{aligned}A\bar{B}+B\bar{C}+\bar{B}C+\bar{A}B&=A\bar{B}+B\bar{C}+(A+\bar{A})\bar{B}C+\bar{A}B(C+\bar{C})\\&=A\bar{B}+B\bar{C}+A\bar{B}C+\bar{A}\bar{B}C+\bar{A}BC+\bar{A}B\bar{C}\\&=A\bar{B}+B\bar{C}+\bar{A}C\end{aligned}$$

上面介绍的是几种常用的方法，举出的例子都比较简单。而实际应用中遇到的逻辑函数往往比较复杂，化简时应灵活使用所学的公理、定理及规则，综合运用各种方法。

例 2.5　化简 $F=AD+A\bar{D}+AB+\bar{A}C+BD+\bar{B}E+DE$。

解

$$\begin{aligned}F&=AD+A\bar{D}+AB+\bar{A}C+BD+\bar{B}E+DE\\&=A+AB+\bar{A}C+BD+\bar{B}E+DE\\&=A+\bar{A}C+BD+\bar{B}E+DE\\&=A+C+BD+\bar{B}E\end{aligned}$$

例 2.6　化简　$F=\bar{A}\bar{C}(\bar{B}+BD)+A\bar{C}D$。

解

$$\begin{aligned}F&=\bar{A}\bar{C}(\bar{B}+BD)+A\bar{C}D\\&=\bar{A}\bar{C}[(\bar{B}+B)(\bar{B}+D)]+A\bar{C}D\\&=\bar{A}\bar{B}\bar{C}+\bar{A}\bar{C}D+A\bar{C}D\\&=\bar{A}\bar{B}\bar{C}+\bar{C}D(\bar{A}+A)\\&=\bar{A}\bar{B}\bar{C}+\bar{C}D\end{aligned}$$

例 2.7　化简　$F=AB+A\bar{C}+\bar{B}C+B\bar{C}+\bar{B}D+B\bar{D}+ADE$。

解

$$\begin{aligned}F&=AB+A\bar{C}+\bar{B}C+B\bar{C}+\bar{B}D+B\bar{D}+ADE\\&=A\,\overline{\bar{B}\bar{C}}+\bar{B}C+B\bar{C}+\bar{B}D+B\bar{D}+ADE\\&=A+\bar{B}C+B\bar{C}+\bar{B}D+B\bar{D}+ADE\\&=A+\bar{B}C+B\bar{C}+\bar{B}D+B\bar{D}\end{aligned}$$

$$=A+\bar{B}C(\bar{D}+D)+B\bar{C}+\bar{B}D+B\bar{D}(C+\bar{C})$$
$$=A+\bar{B}C\bar{D}+\bar{B}CD+B\bar{C}+\bar{B}D+BC\bar{D}+B\bar{C}\bar{D}$$
$$=A+(\bar{B}C\bar{D}+BC\bar{D})+(\bar{B}CD+\bar{B}D)+(B\bar{C}+B\bar{C}\bar{D})$$
$$=A+C\bar{D}+\bar{B}D+B\bar{C}$$

2. 或-与表达式的化简

同最简与-或表达式类似，最简或-与表达式也应满足两个条件：

① 表达式中的或项个数最少；

② 在满足条件①的前提下，每个或项的变量个数最少。

用代数化简法化简或-与表达式可直接运用公理、定理中的或-与形式，并综合运用前面介绍与-或表达式化简时提出的各种方法进行化简。

例 2.8 化简 $F=(A+B)(A+\bar{B})(B+C)(B+C+D)$。

解
$$F=(A+B)(A+\bar{B})(B+C)(B+C+D)$$
$$=(A+B)(A+\bar{B})(B+C)$$
$$=A(B+C)$$

如果对于公理、定理中的或-与形式不太熟悉，也可以采用两次对偶法：首先，对用或-与表达式表示的函数 F 求对偶，得到与-或表达式 F′，并按与-或表达式的化简方法求出 F′的最简表达式；然后，对 F′再次求对偶，即可得到 F 的最简或-与表达式。

例 2.9 化简 $F=(A+\bar{B})(\bar{A}+B)(B+C)(\bar{A}+C)$。

解 先求 F 的对偶式 F′并进行化简。

$$F'=A\bar{B}+\bar{A}B+BC+\bar{A}C$$
$$=A\bar{B}+\bar{A}B+(B+\bar{A})C$$
$$=A\bar{B}+\bar{A}B+\overline{A\bar{B}}C$$
$$=A\bar{B}+\bar{A}B+C$$

再对 F′求对偶，得到

$$F=(F')'=(A+\bar{B})(\bar{A}+B)C$$

从前面的介绍可以看出，代数化简法的优点是不受变量数目的约束，当对公理、定理和规则十分熟练时化简比较方便；缺点是没有一定的规律和步骤，技巧性很强，而且在很多情况下难以判断化简结果是否为最简。因此，这种方法有较大的局限性。

2.4.2 卡诺图化简法

卡诺图化简法又称为图形化简法。这种方法简单、直观、容易掌握，因而在逻辑设计中得到了广泛应用。

1. 卡诺图的构成

卡诺图是一种平面方格图，n个变量的卡诺图由2^n个小方格构成。可以把卡诺图看成是真值表图形化的结果，n个变量的真值表是用2^n行的纵列依次给出变量的2^n种取值，每行的取值与一个最小项对应；而n个变量的卡诺图是用二维图形中2^n个小方格的坐标值给出变量的2^n种取值，每个小方格与一个最小项对应。

卡诺图中最小项的排列方案不是惟一的，但任何一种排列方案都应保证能清楚地反映最小项的相邻关系。本教材中使用图2.5所示的画法。图2.5(a)、(b)、(c)、(d)分别为2变量、3变量、4变量、5变量卡诺图。图中，变量的坐标值0表示相应变量的反变量，1表示相应变量的原变量。各小方格依变量顺序取坐标值，得到的二进制数所对应的十进制数即为相应最小项的下标i。在5变量卡诺图中，为了方便省略了符号"m"，直接标出m的下标i。

从图2.5所示的各卡诺图可以看出，卡诺图上变量的排列是有一定规律的。假定把彼此只有一个变量不同，且这个不同变量互为反变量的两个最小项称为相

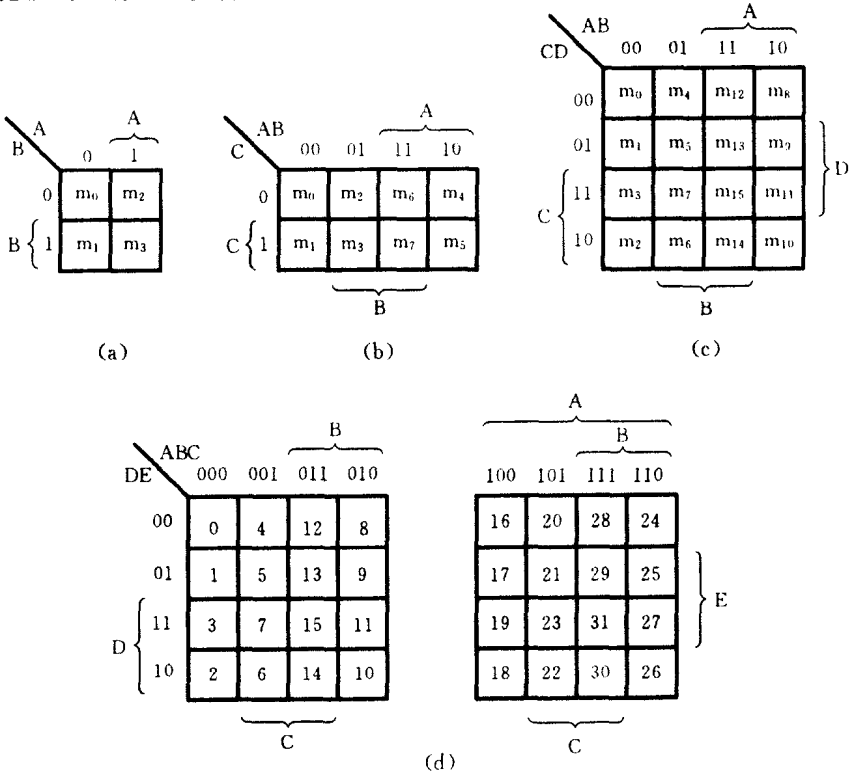


图2.5　2～5变量卡诺图

邻最小项，那么，卡诺图上变量的排列规律将使最小项的相邻关系能在图形上清晰地反映出来。具体地说，在 n 个变量的卡诺图中，能从图形上直观、方便地找到每个最小项的 n 个相邻最小项。例如，4 变量卡诺图中，每个最小项应有 4 个相邻最小项。如 m_5 的 4 个相邻最小项分别是 m_1，m_4，m_7，m_{13}，这 4 个最小项对应的小方格与 m_5 对应的小方格分别相连，也就是说在几何位置上是相邻的，这种相邻称为几何相邻。而 m_0 则不完全相同，它的 4 个相邻最小项除了与之几何相邻的 m_1 和 m_4 之外，另外两个是处在“相对”位置的 m_2（同一列的两端）和 m_8（同一行的两端）。这种相邻似乎不太直观，但只要按图 2.6(a)所示把这个图的上、下边缘连接，卷成圆筒状，便可看出 m_0 和 m_2 在几何位置上是相邻的。同样，按图 2.6(b)所示把图的左、右边缘连接，卷成圆筒状，便可使 m_0 和 m_8 相邻。通常把这种相邻称为相对相邻。

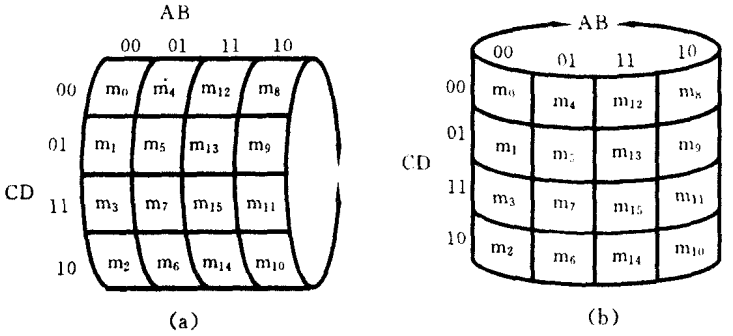


图 2.6　4 变量卡诺图卷成筒状的示意图

除此之外，还有“相重”位置的最小项相邻，如 5 变量卡诺图中的 m_3，除了几何相邻的 m_1，m_2，m_7 和相对相邻的 m_{11} 外，还与 m_{19} 相邻。对于这种情形，可以把卡诺图左边的矩形重叠到右边矩形之上来看，凡上下重叠的最小项相邻，称为重叠相邻。

归纳起来，卡诺图在构造上具有以下两个特点：

① n 个变量的卡诺图由 2^n 个小方格组成，每个小方格代表一个最小项；

②卡诺图上处在相邻、相对、相重位置的小方格所代表的最小项为相邻最小项。

2. 逻辑函数在卡诺图上的表示

当逻辑函数为标准与-或表达式时，只需在卡诺图上找出和表达式中最小项对应的小方格填上 1，其余小方格填上 0，即可得到该函数的卡诺图。

例如，3 变量函数

$$F(A,B,C)=\sum m(1,2,3,7)$$

其卡诺图如图 2.7 所示。

C \ AB	00	01	11	10
0	0	1	0	0
1	1	1	1	0

图 2.7　$F(A,B,C)=\sum m(1,2,3,7)$ 的卡诺图

CD \ AB	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	1	1	1	1
10	1	0	1	0

图 2.8　$F(A,B,C,D)=AB+CD+\bar{A}\bar{B}C$ 的卡诺图

当逻辑函数为一般与-或表达式时，可根据“与”的公共性和“或”的叠加性作出相应卡诺图。

例如，4 变量函数

$$F(A,B,C,D)=AB+CD+\bar{A}\bar{B}C$$

只需在 4 变量卡诺图上依次找出和与项 AB、CD、$\bar{A}\bar{B}C$ 对应的小方格填上 1，便可得到该函数的卡诺图，如图 2.8 所示。

当逻辑函数表达式为其他形式时，可将其变换成上述形式后再作卡诺图。

为了叙述的方便，通常将卡诺图上填 1 的小方格称为 1 方格，填 0 的小方格称为 0 方格。0 方格有时用空格表示。

3. 卡诺图上最小项的合并规律

根据定理 $AB+A\bar{B}=A$ 和相邻最小项的定义可知，两个相邻最小项可以合并为一项并消去一个变量。例如，4 变量最小项 $\bar{A}B\bar{C}D$ 和 $\bar{A}BCD$ 相邻，$\bar{A}B\bar{C}D+\bar{A}BCD$ 可以合并为 $\bar{A}BD$。如果 $\bar{A}BD$ 还与另一个与项相邻，如 ABD，那么，按同样道理可进一步将 $\bar{A}BD+ABD$ 合并为 BD。

卡诺图有一个重要特征，就是它从图形上直观、清晰地反映了最小项的相邻关系。用卡诺图化简逻辑函数的基本原理就是把上述逻辑依据和图形特征结合起来，通过把卡诺图上表征相邻最小项的相邻小方格圈在一起进行合并，达到用一个简单与项代替若干最小项的目的。通常把用来包围那些能由一个简单与项代替的若干最小项的圈称为**卡诺圈**。

究竟哪些最小项可以合并呢？下面以 2、3、4 变量卡诺图为例予以说明。

① 当两个(2^1个)小方格相邻，或处于某行(列)两端时，其所代表的最小项可以合并，合并后的与项可消去 1 个变量。

图 2.9 给出了 2、3 变量卡诺图上 2 个相邻最小项合并的典型情况。

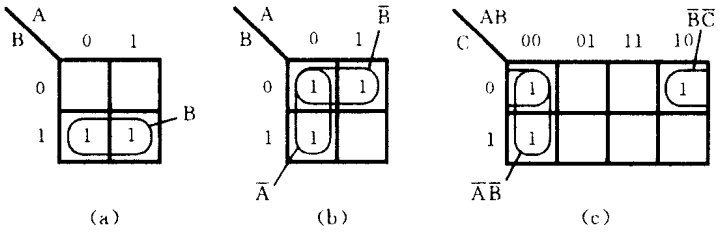


图 2.9　2、3 变量卡诺图上 2 个相邻最小项合并的典型情况

在图 2.9(a)所示 2 变量卡诺图中的 1 方格对应 2 个最小项 $m_1=\overline{A}B$ 和 $m_3=AB$，这 2 个最小项彼此相邻，故可以合并，合并后的与项可消去 1 个变量，结果为 B，即

$$\overline{A}B+AB=B$$

在图 2.9(b)所示 2 变量卡诺图中的 1 方格对应 3 个最小项，其中，$m_0=\overline{A}\overline{B}$ 既和 $m_1=\overline{A}B$ 相邻，又和 $m_2=A\overline{B}$ 相邻，合并时可对 m_0 重复使用。m_0 和 m_1 合并的结果为 $\overline{A}$，m_0 和 m_2 合并的结果为 $\overline{B}$，卡诺图中 1 方格合并的最终结果为 $\overline{A}+\overline{B}$，即

$$\overline{A}\overline{B}+\overline{A}B+A\overline{B}=\overline{A}\overline{B}+\overline{A}B+\overline{A}\overline{B}+A\overline{B}=\overline{A}+\overline{B}$$

在图 2.9(c)所示 3 变量卡诺图中的 1 方格对应 3 个最小项，其中，$m_0=\overline{A}\overline{B}\overline{C}$ 既和 $m_1=\overline{A}\overline{B}C$ 相邻，又和 $m_4=A\overline{B}\overline{C}$ 相邻，合并时可对 m_0 重复使用，m_0 和 m_1 合并的结果为 $\overline{A}\overline{B}$；m_0 和 m_4 合并的结果为 $\overline{B}\overline{C}$，卡诺图中 1 方格合并的最终结果为 $\overline{A}\overline{B}+\overline{B}\overline{C}$，即

$$\overline{A}\overline{B}\overline{C}+\overline{A}\overline{B}C+A\overline{B}\overline{C}=\overline{A}\overline{B}\overline{C}+\overline{A}\overline{B}C+\overline{A}\overline{B}\overline{C}+A\overline{B}\overline{C}=\overline{A}\overline{B}+\overline{B}\overline{C}$$

② 当 4 个(2^2个)小方格组成一个大方格，或组成一行(列)，或处于相邻两行(列)的两端，或处于四角时，其所代表的最小项可以合并，合并后的与项可消去 2 个变量。

图 2.10 给出了 3 变量卡诺图上 4 个相邻最小项合并的典型情况。

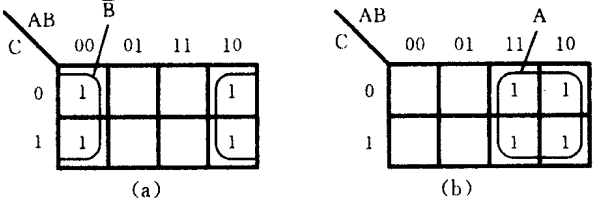


图 2.10　3 变量卡诺图上 4 个相邻最小项合并的典型情况

在图 2.10(a)所示 3 变量卡诺图中的 1 方格对应 4 个最小项 m_0，m_1，m_4，m_5，这 4 个最小项处于相邻两行的两端，故可以合并，合并后的与项可消去 2 个变量，结果为 $\overline{B}$，即

$$\overline{A}\overline{B}\overline{C}+\overline{A}\overline{B}C+A\overline{B}\overline{C}+A\overline{B}C=\overline{A}\overline{B}+A\overline{B}=\overline{B}$$

在图 2.10(b)所示 3 变量卡诺图中的 1 方格对应 4 个最小项 m_4，m_5，m_6，m_7，这 4 个最小项组成一个大方格，故可以合并，合并的结果为 A。

图 2.11 给出了 4 变量卡诺图上 4 个相邻最小项合并的典型情况。

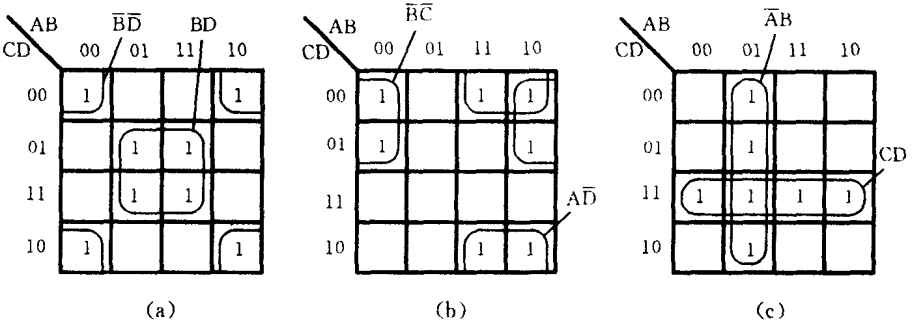


图 2.11　4 变量卡诺图上 4 个相邻最小项合并的典型情况

在图 2.11(a)所示 4 变量卡诺图中的 1 方格对应 8 个最小项，其中，最小项m_0，m_2，m_8，m_{10}处于卡诺图的四角，故这 4 个最小项可以合并，合并后的与项可消去 2 个变量，结果为 $\bar{B}\bar{D}$；最小项 m_5，m_7，m_{13}，m_{15}组成一个大方格，故这 4 个最小项可以合并，合并后的与项为 BD；卡诺图中所有 1 方格合并的最终结果为 $\bar{B}\bar{D}+BD$。

在图 2.11(b)所示 4 变量卡诺图中的 1 方格对应 7 个最小项，其中最小项m_0，m_1，m_8，m_9处于相邻两行的两端，故可以合并，合并后的与项可消去 2 个变量，结果为 $\bar{B}\bar{C}$；最小项 m_8，m_{10}，m_{12}，m_{14}处于相邻两列的两端，故这 4 个最小项可以合并，合并后的与项为 $A\bar{D}$；卡诺图中所有 1 方格合并的最终结果为 $\bar{B}\bar{C}+A\bar{D}$。

在图 2.11(c)所示 4 变量卡诺图中的 1 方格对应 7 个最小项，其中，最小项 m_4，m_5，m_6，m_7组成一列，故可以合并，合并后的与项可消去 2 个变量，结果为 $\bar{A}B$；最小项 m_3，m_7，m_{11}，m_{15}组成一行，故这 4 个最小项可以合并，合并后的与项为 CD；卡诺图中所有 1 方格合并的最终结果为 $\bar{A}B+CD$。

③ 当 8 个(2^3个)小方格连成一体，或为相邻的两行(列)，或处于两个边行(列)时，其所代表的最小项可以合并，合并后可消去 3 个变量。

图 2.12 给出了 3、4 变量卡诺图上 8 个相邻最小项合并的典型情况。

在图 2.12(a)所示 3 变量卡诺图中的 8 个小方格全部为 1，这 8 个最小项可以合并，合并后的与项可消去 3 个变量，结果为 1，即全部最小项相或为 1。

在图 2.12(b)所示 4 变量卡诺图中的 1 方格对应 12 个最小项，其中最小项 m_0，m_1，m_2，m_3，m_8，m_9，m_{10}，m_{11}处于两个边列，故这 8 个最小项可以合并，合并后的“与项”可消去 3 个变量，结果为 $\bar{B}$；最小项 m_1，m_3，m_5，m_7，m_9，m_{11}，m_{13}，m_{15}连成一体，故这 8 个最小项可以合并，合并后的与项为 D；卡诺图中所有 1 方格合

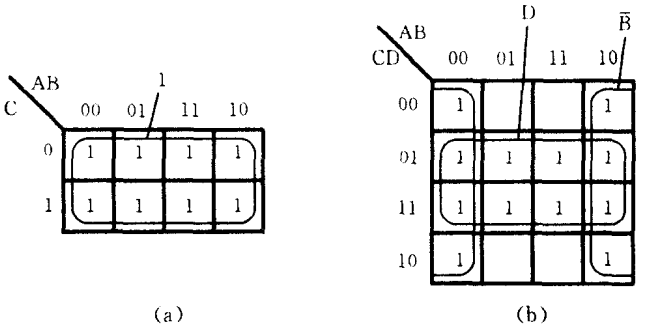


图 2.12 3、4 变量卡诺图上 8 个相邻最小项合并的典型情况

并的最终结果为 $\bar{B}+D$。

至此，以 2、3、4 变量卡诺图为例，讨论了 2、4、8 个最小项的合并方法。依此类推，不难得出 5 变量以上卡诺图中最小项的合并规律。归纳起来，n 个变量卡诺图中最小项的合并规律如下。

① 卡诺圈中小方格的个数必须为 2^m 个，m 为小于或等于 n 的整数。

② 卡诺圈中的 2^m 个小方格有一定的排列规律，具体地说，它们含有 m 个不同变量，$(n-m)$ 个相同变量。

③ 卡诺圈中的 2^m 个小方格对应的最小项可用 $(n-m)$ 个变量的与项表示，该与项由这些最小项中的相同变量构成。

④ 当 $m=0$ 时，卡诺圈中包含 1 个最小项；当 $m=n$ 时，卡诺圈包围了整个卡诺图，可用 1 表示，即 n 个变量的全部最小项相或为 1。

4. 卡诺图化简逻辑函数的步骤

(1) 几个术语

在讨论用卡诺图化简逻辑函数的具体步骤之前，先定义几个术语如下。

蕴涵项：在函数的与-或表达式中，每个与项被称为该函数的蕴涵项(Implicant)。显然，在函数卡诺图中，任何一个 1 方格所对应的最小项或者卡诺圈中的 2^m 个 1 方格所对应的与项都是函数的蕴涵项。

质蕴涵项：若函数的一个蕴涵项不是该函数中其他蕴涵项的子集，则此蕴涵项称为质蕴涵项(Prime Implicant)，简称为质项。显然，在函数卡诺图中，按照最小项合并规律，如果某个卡诺圈不可能被其他更大的卡诺圈包含，那么，该卡诺圈所对应的与项为质蕴涵项。

必要质蕴涵项：若函数的一个质蕴涵项包含有不被函数的其他任何质蕴涵项所包含的最小项，则此质蕴涵项被称为必要质蕴涵项(Essential Prime Implicant)，

简称为必要质项。在函数卡诺图中，若某个卡诺圈包含了不可能被任何其他卡诺圈包含的 1 方格，那么，该卡诺圈所对应的与项为必要质蕴涵项。

(2) 求逻辑函数的最简"与-或"表达式

在上述定义的基础上可归纳出用卡诺图求逻辑函数最简与-或表达式的一般步骤如下。

第一步：作出函数的卡诺图。

第二步：在卡诺图上圈出函数的全部质蕴涵项。

第三步：从全部质蕴涵项中找出所有必要质蕴涵项。

第四步：若函数的所有必要质蕴涵项尚不能覆盖卡诺图上的所有 1 方格，则从剩余质蕴涵项中找出最简的所需质蕴涵项，使它和必要质蕴涵项一起构成函数的最小覆盖(即最简的质蕴涵项集)。

下面举例说明用卡诺图化简逻辑函数的全过程。

例 2.10　用卡诺图化简逻辑函数 $F(A,B,C,D)=\sum m(0,3,5,6,7,10,11,13,15)$。

解　第一步：作出给定函数 F 的卡诺图，如图 2.13(a)所示。

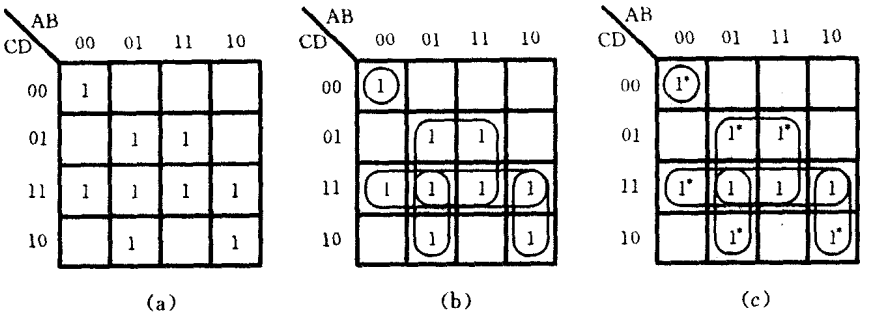


图 2.13　例 2.10 中函数 F 的卡诺图化简

第二步：在函数 F 的卡诺图上圈出函数的全部质蕴涵项。

按照卡诺图上最小项的合并规律，对函数 F 卡诺图中的 1 方格画卡诺圈。为了圈出全部质蕴涵项，画卡诺圈时先画大圈，再画小圈，且大圈中不再画小圈，按此约定，得到图 2.13(b)。在图 2.13(b)中的 5 个卡诺圈均不可能被更大的卡诺圈包围，故它们对应的"与"项均为质蕴涵项。

第三步：从全部质蕴涵项中找出所有必要质蕴涵项。

为了从卡诺图上找出所有必要质蕴涵项，必需在卡诺图上找出那些只被一个卡诺圈包围的最小项，这些最小项被称为必要最小项，而包含必要最小项的质蕴涵项就是必要质蕴涵项。图 2.13(c)中标有"*"号的最小项为必要最小项，它们分

别是最小项 m_0，m_3，m_5，m_6，m_{10}和 m_{13}。由此可见，卡诺图上圈出的 5 个质蕴涵项均为必要质蕴涵项。为了保证所得结果无一遗漏地覆盖函数的所有最小项，函数表达式中必须包含所有必要质蕴涵项。在此题中，5 个必要质蕴涵项已将函数的全部最小项覆盖，故函数 F 的最简表达式为

$$F(A,B,C,D)=\bar{A}\bar{B}\bar{C}\bar{D}+\bar{A}BC+A\bar{B}C+B\bar{D}+C\bar{D}$$

在这个例子中，函数的必要质蕴涵项集覆盖了所有最小项，所以，函数的最简与-或表达式可由全部必要质蕴涵项构成。但有些函数的全部最小项并不能由它的全部必要质蕴涵项覆盖。在这种情况下，还将进行第四步。

例 2.11 用卡诺图化简逻辑函数 $F(A,B,C,D)=\bar{A}CD+\bar{B}C\bar{D}+\bar{A}BC\bar{D}+A\bar{B}\bar{D}+AB\bar{C}\bar{D}$。

解 第一步：作出 F 的卡诺图，如图 2.14(a)所示。

第二步：在卡诺图上圈出函数的全部质蕴涵项，见图 2.14(b)。由图可知，该函数包含 4 个质蕴涵项，即 $\bar{A}C$，$\bar{B}C\bar{D}$，$A\bar{B}\bar{D}$ 和 $AB\bar{C}$。

第三步：找出所有必要质蕴涵项，如图 2.14(c)所示。由图可知，该函数包含两个必要质蕴涵项，即 $\bar{A}C$ 和 $AB\bar{C}$。

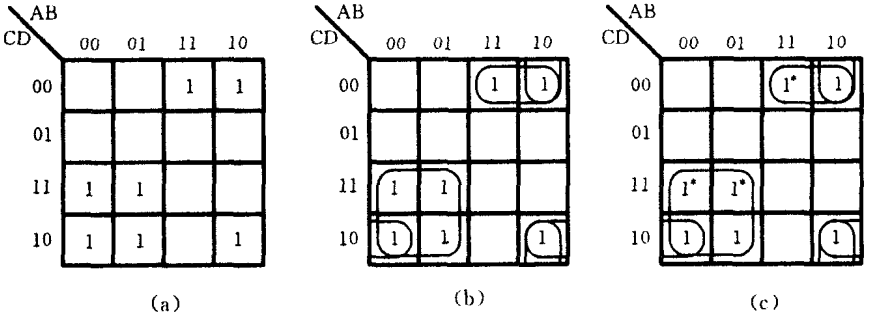


图 2.14 例 2.11 中函数 F 的卡诺图化简

由图可见，在选取必要质蕴涵项之后，尚有最小项 m_{10}未被覆盖。

第四步：确定除必要质蕴涵项外的最简质蕴涵项，求出最简质蕴涵项集。

为了覆盖最小项 m_{10}，可选质蕴涵项 $A\bar{B}\bar{D}$ 或 $\bar{B}C\bar{D}$，由于这两个质蕴涵项均由 3 个变量组成，故可任选其中之一作为所需质蕴涵项，即 F 的最简质蕴涵项集可为

$$\{\bar{A}C,A\bar{C}\bar{D},A\bar{B}\bar{D}\}\quad 或\quad \{\bar{A}C,A\bar{C}\bar{D},\bar{B}C\bar{D}\}$$

求得函数 F 的最简与-或表达式为

$$F(A,B,C,D)=\bar{A}C+A\bar{C}\bar{D}+A\bar{B}\bar{D}$$

或

$$F(A,B,C,D)=\bar{A}C+A\bar{C}\bar{D}+\bar{B}C\bar{D}$$

这里，函数 F 的最简式有两个，它们分别由两个相同的必要质蕴涵项和一个不同的所需质蕴涵项组成，其复杂程度相同。由此可见，函数的最简与-或表达式不一定是惟一的。

在掌握了卡诺图化简的基本方法和步骤后，不一定按步进行，即在熟练的情况下，完全可以一次求出最简结果。其化简的总原则是：

在覆盖函数中的所有最小项的前提下，卡诺圈的个数达到最少，每个卡诺圈达到最大。

(3) 求逻辑函数最简或-与表达式

上面介绍了用卡诺图求函数最简与-或表达式的方法和步骤。当需要求一个函数的最简或-与表达式时，在上述方法的基础上，通常采用“两次取反法”。具体可分如下两种情况处理。

① 当给定逻辑函数为与-或表达式时，首先作出函数 F 的卡诺图，合并卡诺图上的 0 方格，求出反函数 $\overline{F}$ 的最简与-或表达式；然后，对 $\overline{F}$ 的最简与-或表达式取反，得到函数 F 的最简或-与表达式。

例 2.12　用卡诺图化简法求逻辑函数 $F(A,B,C,D)=A\overline{C}+AD+\overline{B}\overline{C}+\overline{B}D$ 的最简或-与表达式。

解　首先作出函数 F 的卡诺图，如图 2.15所示。

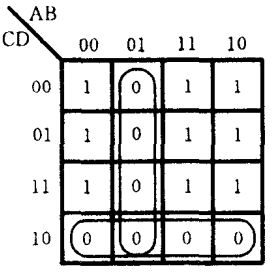


图 2.15　例 2.12 中函数 F 的卡诺图

图中，1 方格表示原函数 F 所包含的最小项，0 方格表示反函数 $\overline{F}$ 所包含的最小项（F 为 0 时，$\overline{F}$ 为 1）。合并卡诺图中的 0 方格，可得到反函数的 $\overline{F}$ 最简与-或表达式为

$$\overline{F}(A,B,C,D)=\overline{A}B+C\overline{D}$$

再对 $\overline{F}$ 的最简与-或表达式取反，即可得到函数 F 的最简或-与表达式

$$F(A,B,C,D)=\overline{\overline{A}B+C\overline{D}}=(A+\overline{B})(\overline{C}+D)$$

② 当给定逻辑函数为或-与表达式时，首先根据反演规则求出函数 F 的反函数 $\overline{F}$，并作出 $\overline{F}$ 的卡诺图，合并卡诺图上的 1 方格，求出反函数 $\overline{F}$ 的最简与-或表达式；然后，对 $\overline{F}$ 的最简与-或表达式取反，得到函数 F 的最简或-与表达式。

例 2.13　用卡诺图化简法求逻辑函数 $F(A,B,C,D)=(\overline{A}+D)(B+\overline{D})(A+B)$ 的最简或-与表达式。

解　首先根据反演规则求出函数 F 的反函数 $\overline{F}$ 为

$$\overline{F}(A,B,C,D)=A\overline{D}+\overline{B}D+\overline{A}\overline{B}$$

根据所得表达式可作出 $\overline{F}$ 的卡诺图，如图 2.16 所示。首先，合并卡诺图中的 1 方格可得到反函数的 $\overline{F}$ 最简与-或表达式

$$\overline{F}(A,B,C,D)=\overline{B}+A\overline{D}$$

然后，再对 $\overline{F}$ 的最简与-或表达式取反，即可得到函数 F 的最简或-与表达式

$$F(A,B,C,D)=B(\overline{A}+D)$$

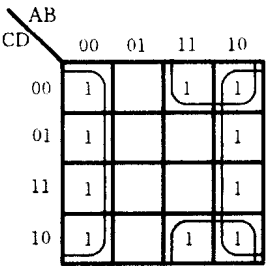


图 2.16 例 2.13 中反函数 $\overline{F}$ 的卡诺图

例 2.14 用卡诺图化简法求逻辑函数 $F(A,B,C,D)=\prod M(3,4,6,7,11,12,13,14,15)$ 的最简与-或表达式和最简或-与表达式。

解 根据给定函数的标准或-与表达式，可作出函数的卡诺图，如图 2.17(a)所示。

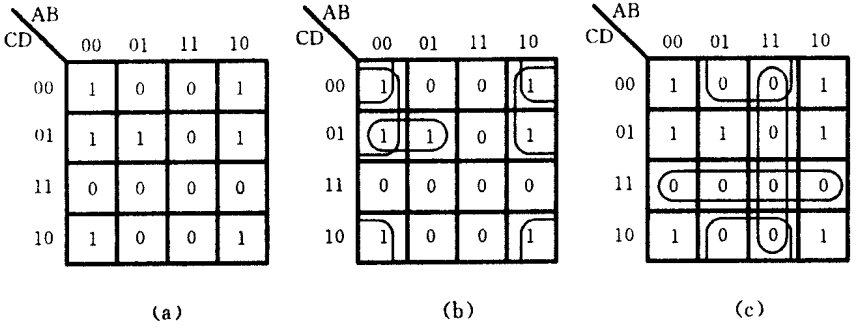


图 2.17 例 2.14 中函数 F 的卡诺图

求逻辑函数最简与-或表达式的过程如图 2.17(b)所示，即合并卡诺图上的 1 方格，便可得到函数 F 的最简与-或表达式为

$$F(A,B,C,D)=\overline{B}\overline{D}+\overline{B}\overline{C}+\overline{A}\overline{C}D$$

求逻辑函数最简或-与表达式的过程如图 2.17(c)所示，首先合并卡诺图上的 0 方格，得到函数 $\overline{F}$ 的最简与-或表达式为

$$\overline{F}(A,B,C,D)=AB+CD+B\overline{D}$$

然后，再对 $\overline{F}$ 的最简与-或表达式取反，即可得到函数 F 的最简或-与表达式为

$$F(A,B,C,D)=(\overline{A}+\overline{B})(\overline{C}+\overline{D})(\overline{B}+D)$$

从上面介绍可以看出，用卡诺图化简逻辑函数具有方便、直观、容易掌握等优点。但这种方法受到函数变量数目的制约，当变量个数大于 6 时，画图以及对图形的识别都变得相当复杂，从而失去了它的优越性。因此，当变量数目太大时，不能用卡诺图人工化简。为了进一步解决化简问题，下面介绍一种适合计算机“自动”化简的方法——列表化简法。

*2.4.3　列表化简法

列表化简法又称奎恩-麦克拉斯基(Quine-McCluskey)法，是一种系统化简法，简称为Q-M化简法。

这种化简法和卡诺图化简法的基本思想大致相同。它也是通过找出函数F的全部质蕴涵项、必要质蕴涵项以及最简质蕴涵项集来求得最简表达式。所不同的是，在列表化简法中上述结果都是通过约定形式的表格，按照一定规则求得的。

用列表化简法化简逻辑函数，一般可以概括为以下4个步骤。

第一步：将函数表示成"最小项之和"的形式，并用二进制码表示每一个最小项。

第二步：找出函数的全部质蕴涵项。

寻找函数全部质蕴涵项通常借助质蕴涵项产生表，具体方法是先将n个变量函数中的相邻最小项合并，消去相异的一个变量，得到$(n-1)$个变量的与项；再将相邻的$(n-1)$个变量的与项合并，消去相异的变量，得到$(n-2)$个变量的与项……依此类推，直到不能再合并为止。所得到的全部不能再合并的与项(包括不能合并的最小项)，即为所要求的全部质蕴涵项。

第三步：找出函数的必要质蕴涵项。

找出函数的全部质蕴涵项后，可借助必要质蕴涵项产生表，找出函数的必要质蕴涵项。

第四步：找出函数的最小覆盖。

当必要质蕴涵项不能覆盖函数的全部最小项时，可借助所需质蕴涵项产生表，求出函数的最小覆盖。

以上各步均是通过表格进行的，下面举例说明。

例2.15　用列表化简法化简逻辑函数$F(A,B,C,D)=\sum m(0,5,7,8,9,10,11,14,15)$。

解　第一步：用二进制代码表示函数中的每一个最小项，如表2.8所示。

表2.8　例2.15中函数F的部分真值表

项号	A	B	C	D	F	项号	A	B	C	D	F
0	0	0	0	0	1	10	1	0	1	0	1
5	0	1	0	1	1	11	1	0	1	1	1
7	0	1	1	1	1	14	1	1	1	0	1
8	1	0	0	0	1	15	1	1	1	1	1
9	1	0	0	1	1						

第二步：求函数的全部质蕴涵项。

考虑到相邻最小项的二进制码中 1 的个数只能相差 1，因此，将表 2.8 中最小项按二进制编码中含 1 的个数进行分组，且按含 1 个数的递增顺序排列在表 2.9（Ⅰ）栏中。这样，可以合并的最小项便只能处于相邻的两组内。因此，可将（Ⅰ）栏中相邻两组的二进制码逐个进行比较，找出那些只有一个变量不同的最小项合并，消去不同变量，组成$(n-1)$个变量的与项列于表的（Ⅱ）栏中。例如，首先将 0 组的最小项 m_0与 1 组的 m_8进行比较、合并，即

项号	A	B	C	D
0	0	0	0	0
8	1	0	0	0
0,8	—	0	0	0

消去相异的 A 变量，这里用“—”表示消去的变量，然后将合并后的与项列入（Ⅱ）栏中。由于该与项是由 m_0和 m_8合并产生的，故在（Ⅰ）栏中 m_0和 m_8的右边打上“√”标记，表示它们已经包含在（Ⅱ）栏的与项中了，并在（Ⅱ）栏中的第二列指出相应与项是由哪几个最小项合并产生的。第 0 组的最小项与第 1 组的最小项比较完后，接着比较第 1 组和第 2 组的最小项。即将 m_8与 m_5，m_9，m_{10}分别进行比较，显然 m_8与 m_5不能合并，因为它们之间有多个变量不同，而 m_8与 m_9可以合并消去变量 D，m_8与 m_{10}可以合并消去变量 C，将合并后得到的与项同样列入（Ⅱ）栏中。依次类推，将（Ⅰ）栏中全部最小项逐一进行比较、合并，得到表的（Ⅱ）栏。在（Ⅱ）栏中的与项均由$(n-1)$个变量组成，此例（Ⅱ）栏的与项由 3 个变量组成。

按上述同样的方法，再对表 2.9（Ⅱ）栏中的全部与项进行比较、合并，可形成表的第（Ⅲ）栏。由于第（Ⅲ）栏的与项不再相邻，故合并到此结束。

表 2.9 质蕴涵项产生表

（Ⅰ）最小项				（Ⅱ）$(n-1)$个变量的与项				（Ⅲ）$(n-2)$个变量的与项			
组号	m_i	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i	组号	$\sum m_i$	ABCD	p_i
0	0	0000	√	0	0,8	−000	p_5	0	8,9,10,11	10−−	p_2
1	8	1000	√	1	8,9	100−	√	1	10,11,14,15	1−1−	p_1
2	5	0101	√		8,10	10−0	√				
	9	1001	√	2	5,7	01−1	p_4				
	10	1010	√		9,11	10−1	√				
3	7	0111	√		10,11	101−	√				
	11	1011	√		10,14	1−10	√				
	14	1110	√	3	7,15	−111	p_3				
4	15	1111	√		11,15	1−11	√				
					14,15	111−	√				

表 2.9 中凡是没有打“√”标记的与项，即函数的质蕴涵项，用 p_i 表示，该函数的全部质蕴涵项为

$$p_1 = \sum m(10,11,14,15) = AC$$

$$p_2 = \sum m(8,9,10,11) = A\bar{B}$$

$$p_3 = \sum m(7,15) = BCD$$

$$p_4 = \sum m(5,7) = \bar{A}BD$$

$$p_5 = \sum m(0,8) = \bar{B}\,\bar{C}\,\bar{D}$$

第三步：求函数的全部必要质蕴涵项。

通过建立必要质蕴涵项产生表，可求出函数的全部必要质蕴涵项。

本例的必要质蕴涵项产生表如表 2.10 所示。表中，第一行为 F 的全部最小项，第一列为上一步求得的全部质蕴涵项。必要质蕴涵项可按下述步骤求得。

表 2.10　必要质蕴涵项产生表

p_i	m_i								
	0	5	7	8	9	10	11	14	15
p_1*						×	×	⊗	×
p_2*				×	⊗	×	×		
p_3			×						×
p_4*		⊗	×						
p_5*	⊗			×					
覆盖情况	√	√	√	√	√	√	√	√	√

① 逐行标上各质蕴涵项覆盖最小项的情况。例如，表中质蕴涵项 p_1 可覆盖最小项 m_{10}，m_{11}，m_{14} 和 m_{15}，故在 p_1 这一行与上述最小项相应列的交叉处打上“×”标记，其他各行依此类推。

② 逐列检查标有“×”的情况，凡只有一个“×”号的列的相应最小项即为必要最小项，在“×”外面加上一个圈（即⊗）。例如，表中最小项 m_0，m_5，m_9，m_{14} 各列均只有一个“×”，故都在“×”号外加上圈。

③ 找出包含⊗号的各行，这些行对应的质蕴涵项即为必要质蕴涵项，在这些质蕴涵项右上角加上“*”标记。例如，表中的 p_1，p_2，p_4 和 p_5 均为必要质蕴涵项。

④ 在表的最后一行覆盖情况一栏中，标上必要质蕴涵项覆盖最小项的情况。凡能被必要质蕴涵项覆盖的最小项，在最后一行的该列上打上标记“√”，供下一步找函数最小覆盖时参考。

第四步：找出函数的最小覆盖。

所谓最小覆盖即既要覆盖全部最小项，又要使质蕴涵项数目达到最少。

为了能覆盖全部最小项，必要质蕴涵项是首先必须选用的质蕴涵项。本例从表 2.10 的覆盖情况一行可知，选取必要质蕴涵项 p_1，p_2，p_4，p_5 后即可覆盖函数的全部最小项。因此，该函数化简的最终结果为

$$\begin{aligned} F(A,B,C,D) &= p_1 + p_2 + p_4 + p_5 \\ &= AC + A\bar{B} + \bar{A}BD + \bar{B}\bar{C}\bar{D} \end{aligned}$$

当给定函数的必要质蕴涵项集不能覆盖该函数的全部最小项时，还需进一步从剩余质蕴涵项集中找出所需质蕴涵项，以构成函数的最小质蕴涵项集。

进一步找出所需质蕴涵项的途径是，首先建立一个“所需质蕴涵项产生表”，该表是将“必要质蕴涵项产生表”中的必要质蕴涵项及其所覆盖的最小项去掉后形成的；然后按照一定的方法从“所需质蕴涵项产生表”中找出所需质蕴涵项。通常采用的方法是行列消去法，其规则如下：

行消去规则 对于所需质蕴涵项产生表中的任意质蕴涵项 p_i 和 p_j，若 p_i 行中的“×”号完全包含在 p_j 行中，即 $p_i \subset p_j$，则可消去 p_i 行。这是因为选取了质蕴涵项 p_j 后不仅可以覆盖质蕴涵项 p_i 所能覆盖的最小项，而且还可覆盖其他最小项。

列消去规则 对于所需质蕴涵项产生表中的任意最小项 m_i 和 m_j，若 m_i 列中的“×”号完全包含在 m_j 列中，即 $m_i \subset m_j$，则可消去 m_j 列。这是因为选取了覆盖 m_i 的质蕴涵项后一定能覆盖 m_j，反之则不一定。

按照上述规则消去多余的行和多余的列后，可再次选出必要质蕴涵项。该规则可重复使用，直至找出能覆盖函数全部最小项的最小质蕴涵项集为止。下面举例说明选取所需质蕴涵项的方法。

例 2.16 已知函数 $F(A,B,C,D)=\sum m(0,3,4,5,6,7,8,10,11)$ 的必要质蕴涵项产生表如表 2.11 所示，求出该函数的最小覆盖。

解 从表 2.11 可知，p_1 为必要质蕴涵项，故 p_1 是函数最小覆盖中必取的质蕴涵项。在选取了 p_1 作为 F 的最简质蕴涵项集的一个元素后，可覆盖最小项 m_4，m_5，m_6，m_7。尚有最小项 m_0，m_3，m_8，m_{10}，m_{11} 未被覆盖，故需进一步从剩余质蕴涵项 p_2，p_3，p_4，p_5，p_6，p_7 中找出所需质蕴涵项。为此，从表 2.11 中消去 p_1 和它所覆盖的最小项 m_4，m_5，m_6，m_7，得到所需质蕴涵项产生表，如表 2.12 所示。

表 2.11 必要质蕴涵项产生表

p_i	m_i								
	0	3	4	5	6	7	8	10	11
p_1*			×	⊗	⊗	×			
p_2								×	×
p_3		×							×
p_4		×				×			
p_5							×	×	
p_6	×						×		
p_7	×		×						
覆盖情况			✓	✓	✓	✓			

表 2.12 所需质蕴涵项产生表

p_i	m_i				
	0	3	8	10	11
p_2				×	×
p_3		×			×
p_4	—	×	—	—	—
p_5			×	×	
p_6	×		×		
p_7	-×-	—	—	—	—

在表 2.12 中，p_4 行中的“×”完全包含在 p_3 行中，p_7 行中的“×”完全包含在 p_6 行中。根据行消去规则，可消去 p_4 和 p_7 两行，得到表 2.13，它由剩下的质蕴涵项

p_2,p_3,p_5,p_6和最小项 m_0,m_3,m_8,m_{10},m_{11}组成。注意,在表 2.12 中没有可以消去的列。

表 2.13 中,m_0列中的"×"号完全包含在 m_8列中,m_3列中的"×"号完全包含在 m_{11}列中,根据列消去规则,可消去 m_8和 m_{11}所对应的列,得到表 2.14。

表 2.13　消去多余行后的所需质蕴涵项产生表

p_i	m_i				
	0	3	8	10	11
p_2			|	×	×
p_3		×	|		×
p_5			×	×	|
p_6	×		×		|

表 2.14　消去多余行和多余列后的所需质蕴涵项产生表

p_i	m_i		
	0	3	10
p_2			×
p_3**		⊗	
p_5			×
p_6**	⊗		

由表 2.14 可见,m_0和 m_3所对应的列均只有一个"×"号,故 p_3和 p_6是必须选取的质蕴涵项,通常称为二次必要质蕴涵项,并标以"* *"。

选取 p_3,p_6作为所需质蕴涵项后,还剩有最小项 m_{10}未被覆盖。为覆盖 m_{10}可以选取 p_2或 p_5,这里 p_2和 p_5的复杂度相等,可任选其中的一个,于是,求得所需质蕴涵项集为

$$\{p_2,p_3,p_6\} \quad 或 \quad \{p_3,p_5,p_6\}$$

再加上第一次得到的必要质蕴涵项 p_1,可得到函数最小覆盖的质蕴涵项集为

$$\{p_1,p_2,p_3,p_6\} \quad 或 \quad \{p_1,p_3,p_5,p_6\}$$

即函数 F 的最简表达式为

$$F(A,B,C,D)=p_1+p_2+p_3+p_6=\overline{A}B+A\overline{B}C+\overline{B}CD+\overline{B}\overline{C}\overline{D}$$

或

$$F(A,B,C,D)=p_1+p_3+p_5+p_6=\overline{A}B+\overline{B}CD+A\overline{B}\overline{D}+\overline{B}\overline{C}\overline{D}$$

列表化简法化简逻辑函数的优点是规律性强,对变量数较多的函数,尽管工作量很大,但总可经过反复比较、合并得到最简结果。该方法非常适用于计算机处理。

习　题　二

2.1　假定一个电路中,指示灯 F 和开关 A、B、C 的关系为

$$F=(A+B)C$$

试画出相应电路图。

2.2　用逻辑代数的公理、定理和规则证明下列表达式:

(1) $\overline{A\bar{B}+\bar{A}C}=A\bar{B}+\bar{A}C$

(2) $AB+A\bar{B}+\bar{A}B+\bar{A}\bar{B}=1$

(3) $A\,\overline{\bar{A}BC}=A\bar{B}\bar{C}+A\bar{B}C+AB\bar{C}$

(4) $ABC+\bar{A}B\bar{C}=\overline{A\bar{B}+B\bar{C}+\bar{A}C}$

2.3 用真值表验证下列表达式：

(1) $A\bar{B}+\bar{A}B=(\bar{A}+\bar{B})(A+B)$

(2) $(\bar{A}+\bar{B})(A+B)=\overline{AB+\bar{A}\bar{B}}$

2.4 利用反演规则和对偶规则求下列函数的反函数和对偶函数：

(1) $F=AB+\bar{A}\bar{B}$

(2) $F=(A+B)(\bar{A}+C)(C+DE)+\bar{E}$

(3) $F=(\bar{A}+B)(C+D\,\overline{AC})$

(4) $F=A[\bar{B}+(C\bar{D}+\bar{E})G]$

2.5 判断下列逻辑命题正误，并说明理由：

(1) 如果 X+Y 和 X+Z 的逻辑值相同，那么，Y 和 Z 的逻辑值一定相同。

(2) 如果 XY 和 XZ 的逻辑值相同，那么，Y 和 Z 的逻辑值一定相同。

(3) 如果 X+Y 和 X+Z 的逻辑值相同，且 XY 和 XZ 的逻辑值相同，那么，Y 和 Z 的逻辑值一定相同。

(4) 如果 X+Y 和 X·Y 的逻辑值相同，那么，X 和 Y 的逻辑值一定相同。

2.6 用代数化简法求下列逻辑函数的最简与-或表达式：

(1) $F=AB+\bar{A}\bar{B}C+BC$

(2) $F=A\bar{B}+B+BCD$

(3) $F=(A+B+C)(\bar{A}+B)(A+B+\bar{C})$

(4) $F=B\bar{C}+D+\bar{D}(\bar{B}+\bar{C})(AC+B)$

2.7 将下列逻辑函数表示成“最小项之和”及“最大项之积”的简写形式：

(1) $F(A,B,C,D)=B\bar{C}\bar{D}+\bar{A}B+AB\bar{C}D+BC$

(2) $F(A,B,C,D)=\overline{\bar{A}\bar{B}+AB\bar{D}}+B+CD$

2.8 用卡诺图化简法求出下列逻辑函数的最简与-或表达式和最简或-与表达式：

(1) $F(A,B,C,D)=\bar{A}\,\bar{B}+\bar{A}\,\bar{C}D+AC+B\bar{C}$

(2) $F(A,B,C,D)=BC+D+\bar{D}(\bar{B}+\bar{C})(AD+B)$

(3) $F(A,B,C,D)=\prod M(2,4,6,10,11,12,13,14,15)$

2.9 用卡诺图判断函数 F(A,B,C,D)和 G(A,B,C,D)之间的关系：

(1) $F(A,B,C,D)=\bar{B}\,\bar{D}+\bar{A}\,\bar{D}+\bar{C}\,\bar{D}+AC\,\bar{D}$

$G(A,B,C,D)=\bar{B}D+CD+\bar{A}\,\bar{C}D+ABD$

(2) $F(A,B,C,D)=(A\bar{B}+\bar{A}B)\bar{C}+\overline{(A\bar{B}+\bar{A}B)}C$

$G(A,B,C,D)=\overline{AB+BC+AC}(A+B+C)+ABC$

2.10　某函数的卡诺图如图 2.18 所示，请回答如下问题：

(1) 若 $b=\bar{a}$，则当 a 取何值时能得到最简的与-或表达式？

(2) 若 a、b 均任意，则 a 和 b 各取何值时能得到最简的与-或表达式？

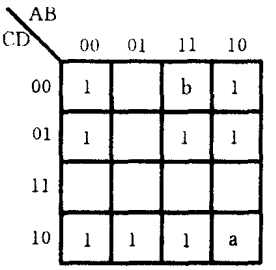
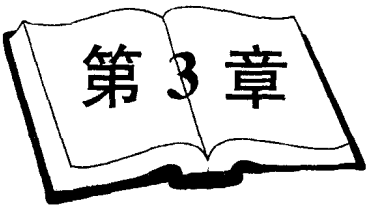


图 2.18　卡诺图

2.11　用列表法化简逻辑函数：

$F(A,B,C,D)=\sum m(0,2,3,5,7,8,10,11,13,15)$



集成门电路与触发器

逻辑门和触发器是组成各类数字逻辑电路的基本逻辑器件。随着微电子技术的发展，人们不再使用二极管、三极管、电阻、电容等分立元件设计各种逻辑器件，而是把实现各种逻辑功能的元器件及其连线都集中制造在同一块半导体材料基片上，并封装在一个壳体中，通过引线与外界联系，这就构成了所谓的集成电路，通常又称为集成电路芯片。采用集成电路进行数字系统设计，不仅可以大大简化设计和调试过程，而且可以使数字系统具有可靠性高、可维性好、功耗低、成本低等优点。

本章在简单介绍数字集成电路的类型以及半导体器件开关特性的基础上，主要讨论集成门电路与触发器的逻辑功能及其基本工作原理。学习本章，要求重点掌握两种基本逻辑器件的外部特性及功能应用。

3.1 数字集成电路的分类

数字集成电路是数字系统的物质基础。集成电路的种类很多，可以从不同的角度对其进行分类，通常有如下 3 种分类方法。

1. 根据采用的半导体器件分类

根据所采用的半导体器件不同，目前常用的数字集成电路可以分为两大类：一类是采用双极型半导体器件作为元件的双极型集成电路；另一类是采用金属-氧化物-半导体场效应管（Metal-Oxide-Semiconductor Field Effect Transistor，简写为 MOSFET）作为元件的单极型集成电路，简称为 MOS 集成电路。相对而言，双极型集成电路的特点是速度快、负载能力强，但功耗较大、结构较复杂，因而使集成规模受到一定限制；MOS 型集成电路的特点是结构简单、制造方便、集成度高、功耗低，但速度一般比双极型集成电路稍慢。

双极型集成电路又可分为 TTL(Transistor Transistor Logic,即三极管-三极管逻辑)电路、ECL(Emitter Coupled Logic,即射极耦合逻辑)电路和 I^2L(Integrated Injection Logic,即集成注入逻辑)电路等类型。TTL 电路于 20 世纪 60 年代问世,经过电路和工艺的不断改进,不仅具有速度快、逻辑电平摆幅大、抗干扰能力和负载能力强等优点,而且具有不同型号的系列产品,是至今应用最广泛的一类集成电路。ECL 电路的最大优点是速度特别快,平均传输延迟时间可降低到 1ns 以下;主要缺点是制造工艺复杂、功耗大、抗干扰能力较弱,常用于高速系统中。I^2L电路的主要优点是电路结构简单、功耗低,适合于构造大规模和超大规模集成电路;主要缺点是抗干扰能力较差,因而很少加工成中、小规模集成电路使用。

MOS 集成电路又可分为 PMOS(P-channel Metal-Oxide-Semiconductor,即 P 沟道 MOS)、NMOS(N-Channel Metal-Oxide-Semiconductor,即 N 沟道 MOS)和 CMOS(Complement Metal-Oxide-Semiconductor,即互补 MOS)等类型。PMOS 和 NMOS 按其工作特性又均可进一步分为耗尽型和增强型两种类型。PMOS 管是早期产品,不仅工作速度低,而且由于电源电压为负压,构成的逻辑器件兼容性差,因而很少单独使用。相对而言,NMOS 工作速度较高,且电源电压为正压,构成的逻辑器件兼容性较好,因而得到广泛应用。CMOS 电路是由 PMOS 增强型管和 NMOS 增强型管组成的互补 MOS 电路,这种电路是继 TTL 电路问世之后所开发的第二种广泛应用的电路,它以其优越的综合性能被应用于各种不同规模的集成逻辑器件中。

本章主要讨论 TTL 门电路和 CMOS 门电路。

2. 根据集成规模的大小分类

根据集成电路规模的大小,通常将其分为小规模集成电路(Small Scale Integration,简写为 SSI)、中规模集成电路(Medium Scale Integration,简写为 MSI)、大规模集成电路(Large Scale Integration,简写为 LSI)和超大规模集成电路(Very Large Scale Integration,简写为 VLSI)。分类的依据是一片集成电路芯片上所包含的元器件数目。一般来说,单片内含元器件数目小于 100 个的属于 SSI;单片内含元器件数目在 100～999 个之间的属于 MSI;单片内含元器件数目在 1000～99999 个之间的属于 LSI;单片内含元件数目大于 100000 个的属于 VLSI。值得指出的是,用来作为分类依据的元器件数目不是绝对精确的数量概念,而仅仅是一个大致范围。

本章讨论的逻辑门和触发器属于小规模集成电路。

3. 根据设计方法和功能定义分类

根据设计方法和功能定义,数字集成电路可分为非用户定制电路(Non-tom

design IC)、全用户定制电路(Full-custom design IC)和半用户定制电路(Semi-custom design IC)。非用户定制电路又称为标准集成电路,这类电路具有生产量大、使用广泛、价格便宜等优点,例如,各种小、中、大规模通用集成电路产品。全用户定制电路是为了满足用户特殊应用要求而专门生产的集成电路,通常又称为专用集成电路 ASIC(Application Specific Integrated Circuit)。专用集成电路具有可靠性高、保密性好等优点,但由于这类电路无论从性能、结构上讲都是专为满足某种用户要求而设计的,因而一般设计费用高、销售量小。半用户定制电路是由厂家生产出功能不确定的集成电路,再由用户根据要求进行适当处理,令其实现指定功能,即由用户通过对已有芯片进行功能定义将通用产品专用化。换而言之,这种电路从性能上讲是为满足用户的各种特殊要求而专门设计的,但从电路结构上讲则带有一定的通用性。例如,目前广泛使用的可编程逻辑器件(Programmable Logic Device,简称 PLD)便属于半用户定制电路。

本章讨论的逻辑门和触发器属于标准逻辑器件。

3.2 半导体器件的开关特性

半导体器件都有导通和截止的开关作用,数字电路中的晶体二极管、双极型晶体三极管(Bipolar Junction Transistor,简称 BJT)和 MOS 管等器件一般是以开关方式运用的。它们在脉冲信号作用下,时而饱和导通,时而截止,相当于开关的"接通"与"断开"。由于这些器件通常要运用在开关频率十分高的电路中(开关状态变化的速度可高达每秒百万次数量级甚至千万次数量级),因此,要求器件在导通与截止两种状态之间的转换必须在微秒甚至纳秒数量级的时间内完成。研究这些器件的开关特性时,除了要研究它们在导通与截止两种状态下的静止特性外,还要分析它们在导通和截止状态之间的转变过程,即动态特性。

3.2.1 晶体二极管的开关特性

1. 静态开关特性

晶体二极管由一个 PN 结构成。二极管的静态开关特性是指二极管处在导通和截止两种稳定状态下的特性。图 3.1(a)给出了一个硅二极管电路,与之对应的静态开关特性曲线(又称伏安特性曲线)如图 3.1(b)所示。

二极管的静态开关特性是由二极管的单向导电特性决定的,从伏安特性曲线

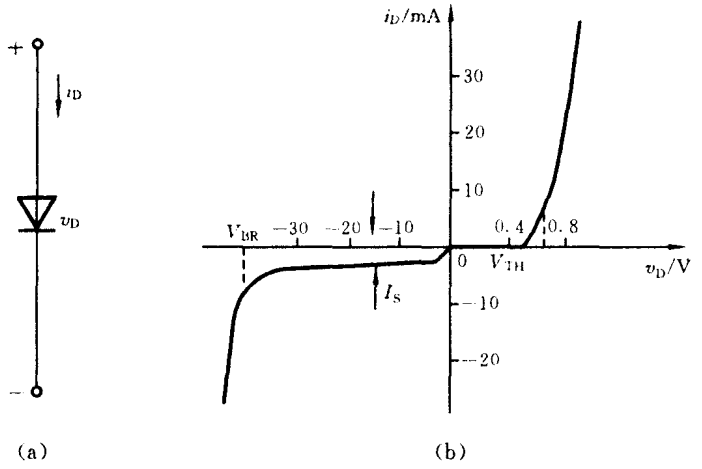


图 3.1　硅二极管电路与伏安特性曲线

可知，二极管的电压与电流关系是非线性的。其正、反向特性如下。

(1) 正向特性

二极管的正向特性表现为在外加正向电压作用下，二极管处于导通状态。但如图 3.1(b)所示，二极管的正向特性中存在一个门槛电压，或称阈值电压 V_{TH}(一般锗管约 0.1V，硅管约 0.5V)，当外加电压 v_D 小于 V_{TH} 时，管子处于截止状态，电阻很大、电流 i_D 接近于 0，此时二极管类似于开关的断开状态；当电压 v_D 达到导通电压 V_{TH} 时，管子开始导通，电流 i_D 开始上升；当 v_D 超过 V_{TH} 达到一定值(一般锗管为 0.3V，硅管为 0.7V)时，管子处于充分导通状态，电阻变得很小，电流 i_D 急剧增加，此时二极管类似于开关的接通状态。通常将使二极管达到充分导通状态的电压称为二极管的导通电压，用 V_F 表示。

(2) 反向特性

如图 3.1(b)所示，二极管的反向特性表现为当外加反向电压在一定数值范围内时，反向电阻很大，反向电流很小，而且反向电压的变化基本不引起反向电流的变化，二极管处于截止状态。截止状态下的反向电流被称为反向饱和电流，用 I_S 表示。在常温下硅二极管的反向饱和电流比锗管小，小功率硅二极管的 I_S 为纳安数量级，而小功率锗二极管的 I_S 为微安数量级。由于反向电流很小，通常可忽略不计，故此时二极管的状态类似于开关断开。当反向电压超过某个极限值时，将使反向电流突然猛增，致使二极管被击穿。通常将使二极管击穿的电压称为反向击穿电压，用 V_{BR} 表示，而将处于 0 和 V_{BR} 之间的电压称为反向截止电压，用 V_R 表示。

由于二极管具有上述的单向导电性，所以在数字电路中经常把它当作开关使用。使用二极管时应注意：由于正向导通时可能因流过的电流过大而导致二极管

烧坏，所以，组成实际电路时通常要串接一只电阻 R，以限制二极管的正向电流；在加反向电压时，一般反向电压应小于反向击穿电压 V_{BR}，以保证二极管正常工作。

图 3.2(a)给出了一个由二极管组成的开关电路，图 3.2(b)所示为二极管导通状态下的等效电路，图 3.2(c)所示为二极管在截止状态下的等效电路，图中忽略了二极管的正向压降。

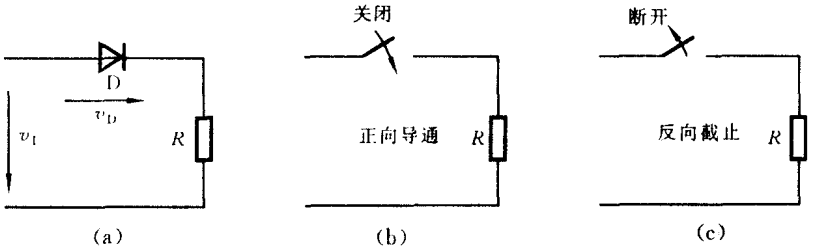


图 3.2 二极管开关电路及其等效电路

2. 动态开关特性

二极管的动态特性是指二极管在导通与截止两种状态转换过程中的特性，它表现为完成两种状态之间的转换需要一定的时间。通常把二极管从正向导通到反向截止所需要的时间称为**反向恢复时间**，而把二极管从反向截止到正向导通所需要的时间称为**开通时间**。相比之下，开通时间很短，一般可以忽略不计。因此，影响二极管开关速度的主要因素是反向恢复时间。

(1) 反向恢复时间

在理想情况下，当作用在二极管两端的电压由正向导通电压 V_F 转为反向截止电压 V_R 时，二极管应该立即由导通转为截止，电路中只存在极小的反向电流。但实际并非如此，如图 3.3 所示，当对图 3.3(a)所示二极管开关电路加入一个如图 3.3(b)所示的输入电压时，电路中电流变化过程如图 3.3(c)所示。

图中，在 $0\sim t_1$ 时间内输入正向导通电压 V_F，二极管导通，由于二极管导通时电阻很小，所以电路中的正向电流 I_F 基本上取决于输入电压和电阻 R，即 $I_F\approx V_F/R$。在 t_1 时刻，输入电压突然由正向电压 V_F 转为反向电压 V_R，在理想情况下二极管应该立即截止，电路中只有极小的反向电流。但实际情况是二极管并不立即截止，而是先由正向的 I_F 变到一个很大的反向电流 $I_R\approx V_R/R$，该电流维持一段时间 t_s 后才开始逐渐下降，经过一段时间 t_t 后下降到一个很小的数值 $0.1I_R$（接近反向饱和电流 I_S），这时二极管才进入反向截止状态。通常把二极管从正向导通转为反向截止的过程称为反向恢复过程，其中 t_s 称为存储时间，t_t 称为渡越时间，$t_{re}=t_s+t_t$ 称为反向恢复时间。

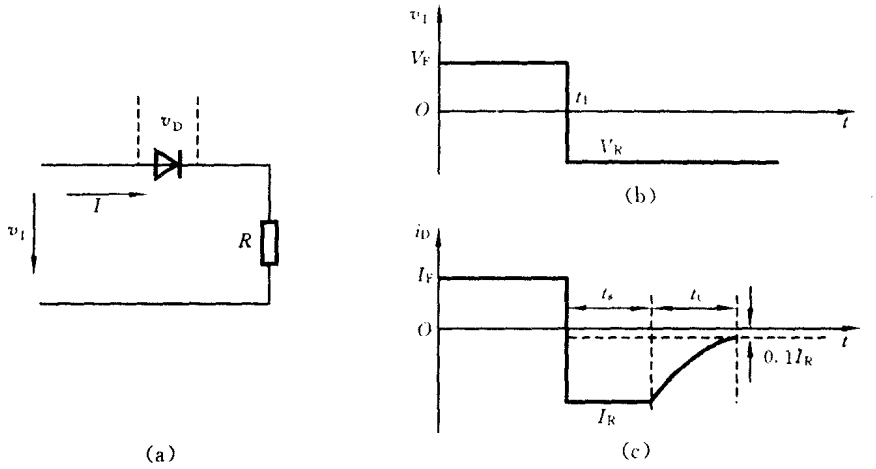


图 3.3　二极管的动态特性

产生反向恢复时间 t_{re} 的原因如下。当二极管外加正向电压 V_F 时，PN 结两边的多数载流子不断向对方区域扩散，这不仅使空间电荷区变窄，而且有相当数量的载流子存储在 PN 结的两侧。正向电流越大，P 区存储的电子和 N 区存储的空穴就越多，如图 3.4(a)所示。当输入电压突然由正向电压 V_F 变为反向电压 V_R 时，PN 结两边存储的载流子在反向电压作用下朝各自原来的方向运动，即 P 区中的电子被拉回 N 区，N 区中的空穴被拉回 P 区，形成反向漂移电流 I_R，由于开始时

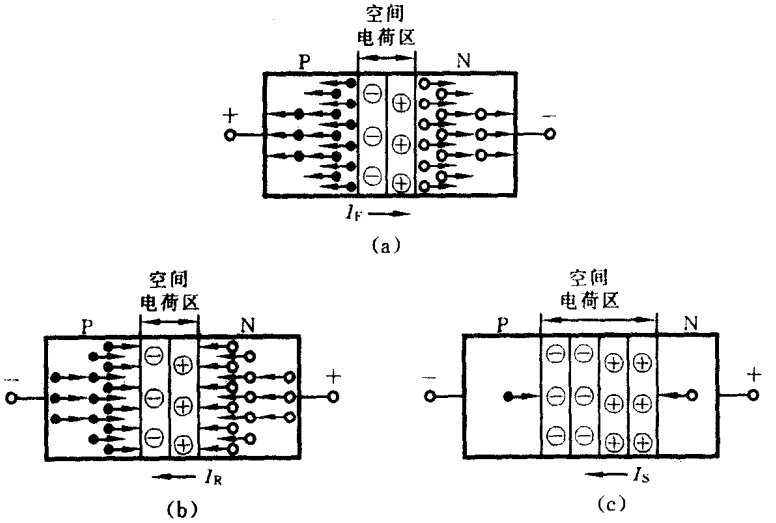


图 3.4　PN 结的反向恢复过程

空间电荷区依然很窄，二极管电阻很小，所以反向电流很大，$I_R \approx V_R / R$，如图 3.4(b)所示。经过时间 t_s 后，PN 结两侧存储的载流子显著减少，空间电荷区逐渐变宽，反向电流慢慢减小，直至经过时间 t_t 后，I_R 减小至反向饱和电流 I_S，二极管截止，如图 3.4(c)所示。

(2) 开通时间

二极管从截止转为正向导通所需要的时间称为开通时间。由于 PN 结在正向电压作用下空间电荷区迅速变窄，正向电阻很小，因而在导通过程中及导通以后，正向压降都很小，故电路中的正向电流 $I_F \approx V_F / R$。而且加入输入电压 V_F 后，回路电流几乎是立即达到 I_F 的最大值。这就是说，二极管的开通时间很短，对开关速度影响很小，相对反向恢复时间而言以致可以忽略不计。

3.2.2 晶体三极管的开关特性

1. 静态特性

双极型晶体三极管(BJT)由集电结和发射结两个 PN 结构成。根据两个 PN 结的偏置极性，三极管有截止、放大、饱和 3 种工作状态。图 3.5(a)给出了一个简单的 NPN 晶体三极管共发射极开关电路，其输出特性曲线如图 3.5 (b)所示。

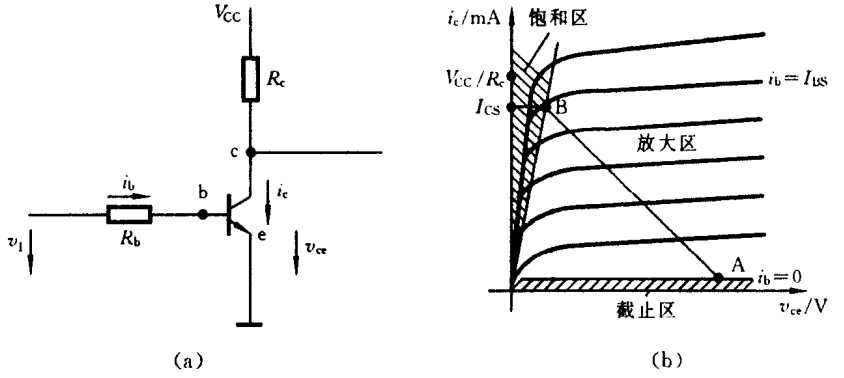


图 3.5 晶体三极管开关电路及其输出特性

在图 3.5(a)所示电路中，通过输入电压 v_I 对 b 点电压加以控制，可使三极管工作在截止、放大、饱和 3 种工作状态。

(1) 截止状态

当输入电压 v_I 小于晶体管阈值电压 V_{TH} 时，晶体管的发射结和集电结均处于反偏状态($v_b < v_e$，$v_b < v_c$)，晶体管工作在截止状态，对应于图 3.5 (b)中所示的截止区。此时，基极电流 $i_b \approx 0$，集电极电流 $i_c \approx 0$，输出电压 $v_{ce} \approx V_{CC}$，晶体三极管

类似于开关断开。

(2) 放大状态

当输入电压 v_I 大于晶体管阀值电压 V_{TH} 而小于某一数值，使得晶体管的发射结正偏而集电结反偏（$v_b > v_e$，$v_b < v_c$）时，晶体管工作在放大状态，对应于图3.5(b)中的放大区。此时，集电极电流 i_c 的大小受基极电流 i_b 的控制，i_c 的变化量是 i_b 变化量的 β 倍，即 $i_c = \beta i_b$。

(3) 饱和状态

当输入电压 v_I 大于某一数值，使得晶体管的发射结和集电结均处于正偏（$v_b > v_e$，$v_b > v_c$）时，晶体管工作在饱和状态，对应于图3.5(b)中所示的饱和区。此时，基极电流 $i_b \geqslant I_{BS}$（基极临界饱和电流）$\approx V_{CC}/\beta R_c$，集电极电流 $i_c = I_{CS}$（集电极饱和电流）$\approx V_{CC}/R_c$。输出电压 $v_{ce} \approx 0.3$V，类似于开关接通。

在数字逻辑电路中，三极管被作为开关元件工作在饱和与截止两种状态，相当于一个由基极信号控制的无触点开关，其作用对应于触点开关的“闭合”与“断开”。图3.6给出了图3.5所示电路在三极管截止（见图(a)）与饱和（见图(b)）状态下的等效电路。晶体三极管在截止与饱和这两种稳态下的特性称为三极管的静态开关特性。

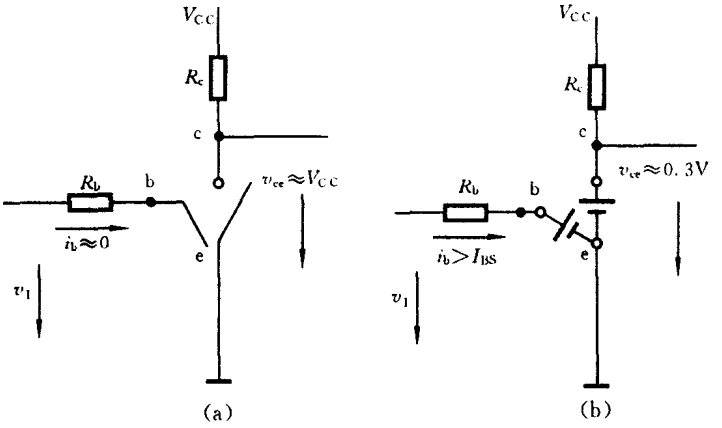


图3.6　晶体三极管开关等效电路

2. 动态特性

晶体三极管在饱和与截止两种状态转换过程中具有的特性称为三极管的动态特性。三极管的开关过程和二极管一样，管子内部也存在着电荷的建立与消失过程。因此，饱和与截止两种状态的转换也需要一定的时间才能完成。假如在图3.5(a)所示电路的输入端输入一个理想的矩形波电压，那么，在理想情况下，i_c 和 v_{ce} 的波形应该如图3.7(a)所示。但在实际转换过程中 i_c 和 v_{ce} 的波形如图3.7(b)

所示，无论从截止转向导通还是从导通转向截止都存在一个逐渐变化的过程。

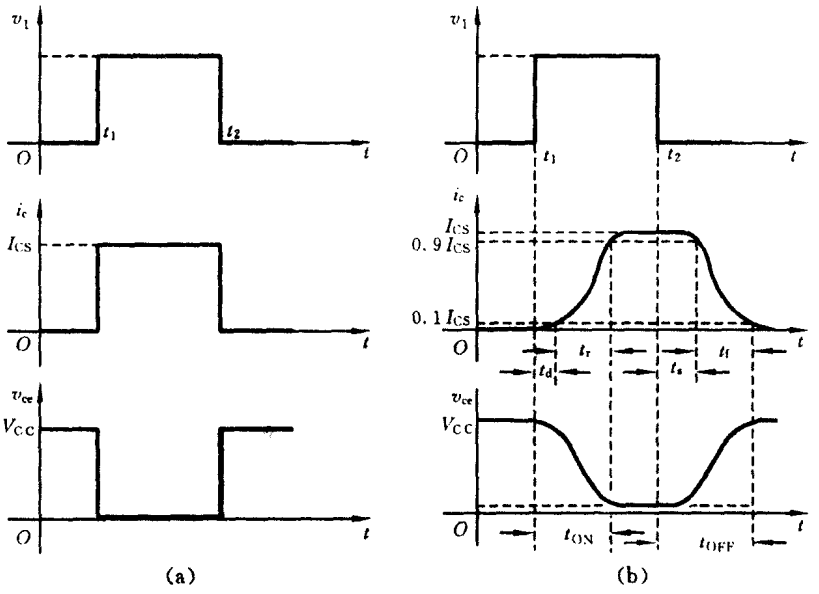


图 3.7 晶体三极管的动态特性

(1) 开通时间

开通时间是指三极管从截止到饱和导通所需要的时间，记为 t_{ON}。当三极管处于截止状态时，发射结反偏，空间电荷区比较宽。当输入信号 v_I 由低电平跳变到高电平时，由于发射结空间电荷区仍保持在截止时的宽度，故发射区的电子还不能立即穿过发射结到达基区。这时发射区的电子进入空间电荷区，使空间电荷区变窄，然后发射区开始向基区发射电子，晶体管开始导通，并开始形成集电极电流 i_c。从晶体管开始导通到集电极电流 i_c 上升到 $0.1I_{CS}$ 所需要的时间称为延迟时间 t_d。

经过延迟时间后，发射区不断向基区注入电子，电子在基区积累，并向集电区扩散。随着基区电子浓度的增加，i_c 不断增大。i_c 由 $0.1I_{CS}$ 上升到 $0.9I_{CS}$ 所需要的时间称为上升时间 t_r。

晶体三极管的开通时间 t_{ON} 等于延迟时间 t_d 和上升时间 t_r 之和，即

$$t_{ON}=t_d+t_r$$

开通时间的长短取决于晶体管的结构和电路工作条件。

(2) 关闭时间

关闭时间是指三极管从饱和导通到截止所需要的时间，记为 t_{OFF}。经过上升

时间以后，集电极电流继续增加到 I_{CS} 后，由于进入了饱和状态，集电极收集电子的能力减弱，过剩的电子在基区不断积累起来，称为超量存储电荷；同时，在集电区靠近边界处也积累起一定的空穴，故集电结处于正向偏置。

当输入信号 v_I 由高电平跳变到低电平时，上述存储电荷不能立即消失，而是在反向电压作用下产生漂移运动而形成反向基流，促使超量存储电荷泄放。在存储电荷完全消失前，集电极电流维持 I_{CS} 不变，直至存储电荷全部消散，晶体管才开始退出饱和状态，i_c 开始下降。从输入信号 v_I 下跳开始，到集电极电流 i_c 下降到 $0.9I_{CS}$ 所需要的时间称为存储时间 t_s。

在基区存储的多余电荷全部消失后，基区中的电子在反向电压作用下越来越少，集电极电流 i_c 也不断减小，并逐渐接近于零。集电极电流由 $0.9I_{CS}$ 降至 $0.1I_{CS}$ 所需的时间称为下降时间 t_f。

晶体管的关闭时间 t_{OFF} 等于存储时间 t_s 和下降时间 t_f 之和，即

$$t_{OFF}=t_s+t_f$$

同样，关闭时间的长短取决于三极管的结构和运用情况。

开通时间 t_{ON} 和关闭时间 t_{OFF} 的大小是影响电路工作速度的主要因素。

3.3　逻辑门电路

在数字系统中，各种逻辑运算是由基本逻辑电路来实现的。这些基本电路控制着系统中信息的流通，它们的作用和门的开关作用极为相似，故称为逻辑门电路，简称逻辑门或门电路。逻辑门是逻辑设计的最小单位，不论其内部结构如何，在数字电路逻辑设计中都仅作为基本元件出现。

了解逻辑门电路的内部结构、工作原理和外部特性，对数字逻辑电路的分析和设计是十分必要的，尤其是外部特性。本节将从实际应用的角度出发，主要介绍 TTL 集成逻辑门和 CMOS 集成逻辑门。学习时应重点掌握集成逻辑门电路的功能和外部特性，以及器件的使用方法。对其内部结构和工作原理只要求作一般了解。

3.3.1　简单逻辑门电路

实现与、或、非 3 种基本逻辑运算的逻辑电路分别称为与门、或门和非门，它们是 3 种最基本的逻辑门。为了使读者对门电路的工作原理有一个初步了解，在介绍 TTL 集成逻辑门和 CMOS 集成逻辑门之前，先对简单的晶体二极管与门、或门和晶体三极管非门（又称为反相器）进行简单介绍。

1. 与门

实现与逻辑功能的电路称为与门。与门有两个以上输入端和一个输出端。图3.8(a)给出了一个由二极管组成的两输入与门电路，与其对应的逻辑符号如图3.8(b)所示。

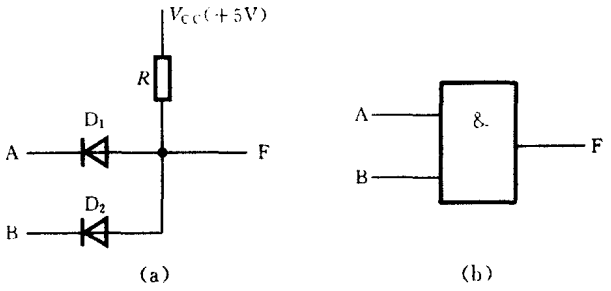


图 3.8 二极管与门电路及与门逻辑符号

图3.8(a)中，A、B为输入端，F为输出端。输入信号为低电平0V或者高电平+5V。假定二极管正向电阻为0，反向电阻无穷大，则根据输入信号取值的不同，可分为如下两种工作情况。

① 当两个输入端A、B的电压V_A、V_B均为低电平(0V)，或者其中的一个为低电平0V时，输入为低电平的二极管将处于导通状态，从而使得输出端F的电压被钳制在0V附近，即$V_F \approx 0V$。

② 当两个输入端A、B的电压V_A、V_B均为高电平+5V时，二极管D_1、D_2均截止，输出端F的电压等于电源电压V_{CC}，即$V_F = +5V$。

归纳上述两种情况，可得出该电路输入A、B和输出F的电压取值关系如表3.1所示。假定高电平+5V表示逻辑值1，低电平0V表示逻辑值0，则该电路输入/输出之间的逻辑取值关系如表3.2所示。

由表3.2可知，该电路实现了与运算的逻辑功能，其输出逻辑表达式为F=A·B。

表 3.1 与门输入/输出的电压关系

V_A/V	V_B/V	V_F/V
0	0	0
0	+5	0
+5	0	0
+5	+5	+5

表 3.2 与门真值表

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

2. 或门

实现或逻辑功能的电路称为或门。或门可以有两个或者两个以上输入端和一个输出端。由二极管构成的两输入或门电路如图 3.9(a)所示，与其对应的逻辑符号如图 3.9(b)所示。

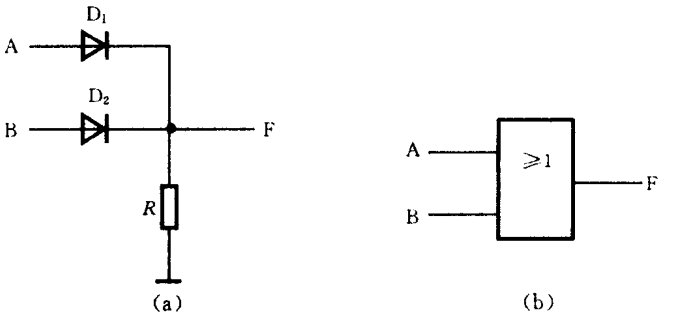


图 3.9　二极管或门电路及或门逻辑符号

图 3.9(a)中，A、B 为输入端，F 为输出端。按照前面与门中的假定，则该电路根据输入信号取值的不同，同样可分为如下两种工作情况。

① 当两个输入端 A、B 的电压 V_A、V_B均为低电平(0V)时，二极管 D_1、D_2均截止，输出端电压 $V_F=0V$。

② 当两个输入端 A、B 的电压 V_A、V_B均为+5V，或者其中的一个为+5V 时，输入为+5V 的二极管将处于导通状态，从而使得输出端 F 的电压为高电平，即$V_F \approx +5V$。

归纳上述两种情况，可得出该电路输入 A、B 和输出 F 的电压取值关系如表 3.3 所示。令高电平用逻辑值 1 表示，低电平用逻辑值 0 表示，可得出电路的逻辑取值关系如表 3.4 所示。

表 3.3　或门输入/输出的电压关系

V_A/V	V_B/V	V_F/V
0	0	0
0	+5	+5
+5	0	+5
+5	+5	+5

表 3.4　或门真值表

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

由表 3.4 可知，该电路实现或运算的逻辑功能，输出 F 和输入 A、B 之间的逻辑关系表达式为 F=A+B。

3. 非门

实现非逻辑功能的电路称为非门。有时又称为反门或反相器。它有一个输入端和一个输出端。用晶体三极管构成的非门电路如图 3.10(a)所示，与其对应的逻辑符号如图 3.10(b)所示。

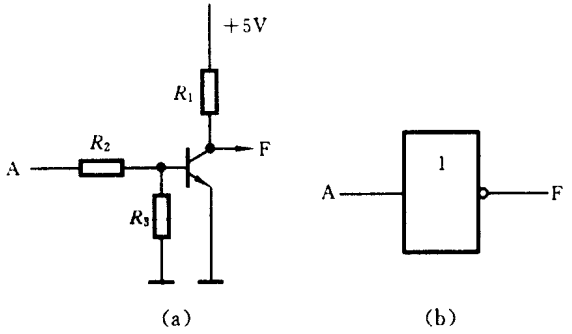


图 3.10 二极管或门电路及或门逻辑符号

图 3.10(a)所示电路中，A 为输入端，F 为输出端。假定三极管饱和导通时集电极输出电压近似为 0V，三极管截止时集电极输出电压近似为 +5V，根据三极管工作原理可知：当输入 A 为低电平时，三极管截止，输出电压 $V_F \approx +5V$；当输入 A 为高电平时，三极管饱和导通，输出电压 $V_F \approx 0V$。输入 A 和输出 F 之间的电压取值关系如表 3.5 所示，逻辑取值关系如表 3.6 所示。

表 3.5 非门输入/输出的电压关系

V_A/V	V_F/V
0	+5
+5	0

表 3.6 非门真值表

A	F
0	1
1	0

由表 3.6 可知，该电路实现“非”逻辑功能，输出 F 与输入 A 的逻辑关系表达式为 $F=\overline{A}$。

以上介绍了由二极管、三极管构成的 3 种简单门电路，虽然它们可以实现 3 种基本逻辑运算，但这种简单门电路的负载能力、开关特性等均不理想。实际应用中使用的是经过反复改进后的、性能更好的各种集成逻辑门电路。

3.3.2 TTL 集成逻辑门电路

TTL 是晶体管-晶体管逻辑(Transistor Transistor Logic)的简称。TTL 逻辑

门由若干晶体三极管、二极管和电阻组成。这种门电路于20世纪60年代即已问世，随后经过对电路结构和工艺的不断改进，性能得到不断改善，至今仍被广泛应用于各种逻辑电路和数字系统中。

TTL逻辑器件根据工作环境温度和电源电压工作范围的差别分为54系列和74系列两大类。相对而言，54系列比74系列的工作环境温度范围更宽，电源电压工作范围允许的偏差更大。54系列的工作环境温度为－55℃～＋125℃，电源电压工作范围为5V±10%；74系列的工作环境温度为0℃～＋70℃，电源电压工作范围为5V±5%。根据器件工作速度和功耗的不同，目前国产TTL集成电路主要分为4个系列：CT54/74系列(标准通用系列，相当于国际上SN54/74系列)；CT54H/74H系列(高速系列，相当于国际上SN54H/74H系列)；CT54S/74S系列(肖特基系列，相当于国际上SN54S/74S系列)；CT54LS/74LS系列(低功耗肖特基系列，相当于国际上SN54LS/74LS系列)。各个系列的详细性能参数可查阅集成电路手册。

1. 典型TTL与非门

由于与非逻辑可以实现任意逻辑运算，所以与非门是应用最广泛的逻辑门之一。下面就TTL与非门的工作原理和性能指标进行介绍。

图3.11(a)所示的是一个典型的CT54/74系列TTL与非门电路，与其对应的逻辑符号如图3.11(b)所示。

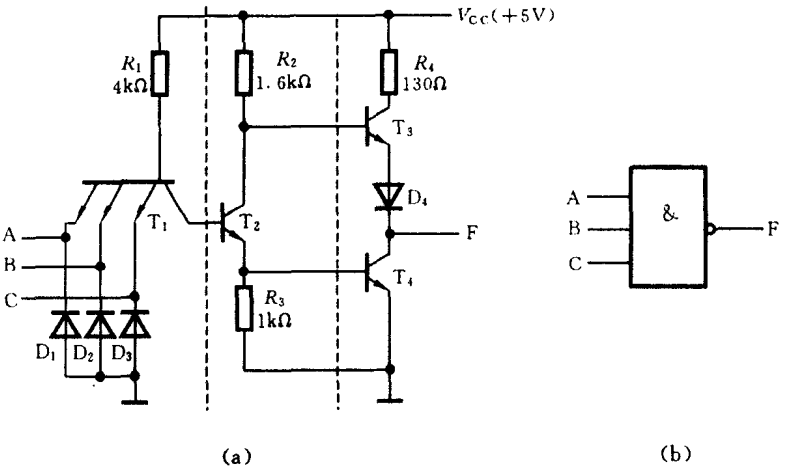


图3.11　典型的TTL与非门电路及逻辑符号

(1) 电路结构

图3.11(a)所示电路按图中虚线划分为3部分：第一部分由多发射极晶体管

T_1和电阻 R_1组成输入级，3 个输入信号通过多发射极晶体管 T_1的发射结实现与逻辑功能；第二部分由晶体管 T_2和电阻 R_2、R_3组成中间级，由 T_2的集电极和发射极同时输出两个相位相反的信号分别控制 T_3和 T_4的工作状态；第三部分由晶体管 T_3、T_4、二极管 D_4和电阻 R_4组成推拉式输出级，采用这种输出级的主要优点是既能提高开关速度，又能提高带负载能力。输入端的 D_1、D_2、D_3为输入端钳位二极管，用于限制输入端出现的负极性干扰信号，对晶体管 T_1起保护作用。

(2) 工作原理

当电路输入端 A、B、C 全部接高电平(3.6V)时，T_1的基极电位升高，使得T_1的集电结、T_2和 T_4的发射结导通，T_1的基极电压 $v_{b1}=v_{bc1}+v_{be2}+v_{be4}\approx 2.1V$。此时，$T_1$的集电极电压若为 1.4V，即 T_1处于发射结反向偏置，而集电结正向偏置的工作状态，称为“倒置”工作状态。另外，此时 T_2的集电极电压等于 T_2管 c、e 两点间的饱和压降与 T_4管的发射结压降之和，即 $v_{c2}=v_{ces2}+v_{be4}\approx 0.3V+0.7V\approx 1V$，该值不足以使 T_3、D_4导通，故 T_3、D_4处于截止状态。而 T_2的发射极向 T_4提供足够的基极电流，使 T_4处于饱和导通状态，故输出电压 $v_F\approx 0.3V$，即“输入全为高，输出为低”。通常将这种工作状态称为导通状态，该状态下的等效电路如图 3.12(a)所示。

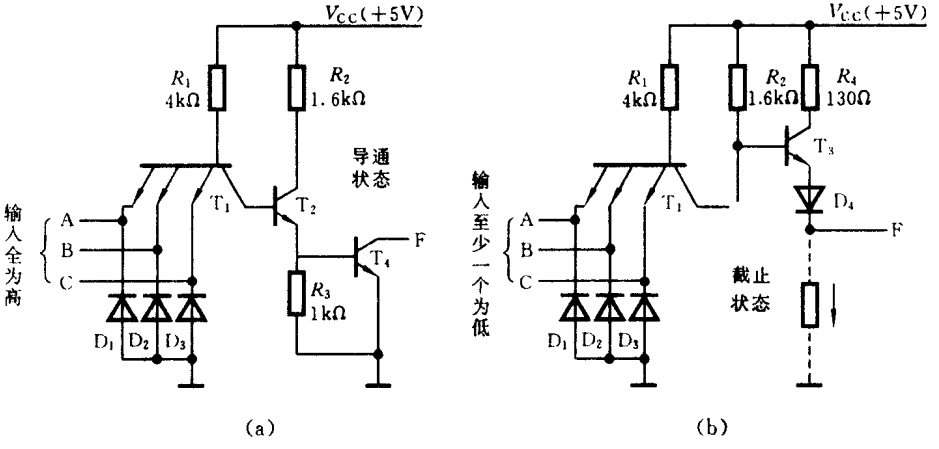


图 3.12 TTL 与非门导通与截止状态下的等效电路

当输入端 A、B、C 中至少有一个接低电平(0.3V)时，多射极晶体管 T_1对应于输入端接低电平的发射结导通，使 T_1的基极电位等于输入低电平加上发射结正向导通压降，即 $v_{b1}\approx 0.3V+0.7V=1V$。该电压作用于 T_1的集电结和 T_2、T_4的发射结上，显然不可能使 T_2和 T_4导通，所以 T_2、T_4均截止。由于 T_2截止，电源V_{CC}通过 R_2驱动 T_3和 D_4管，使之工作在导通状态，故 T_3发射结的导通压降和 D_4的导通压降均为 0.7V，电路输出电压 v_F为 $v_F\approx V_{CC}-i_{b3}R_2-v_{be3}-v_{D4}$；由于基流$i_{b3}$

很小，通常可以忽略不计，故 $v_F \approx V_{CC} - v_{be3} - v_{D4} = 5V - 0.7V - 0.7V = 3.6V$，即“输入有低，输出为高”。通常将电路的这种工作状态称为截止状态，该状态下的等效电路如图 3.12(b)所示。

综合上述，当输入 A、B、C 均为高电平时，输出为低电平；当 A、B、C 中至少有一个为低电平时，输出为高电平。假定高电平用字母 H 表示，低电平用字母 L 表示，电路输出/输入之间的电压取值关系如表 3.7 所示。令高电平对应逻辑值“1”，低电平对应逻辑值“0”，电路输出/输入之间的逻辑取值关系如表 3.8 所示。由表 3.8 可知，该电路实现了与非逻辑功能，即 $F = \overline{ABC}$。

表 3.7　TTL 与非门的电压取值关系

输入			输出
A	B	C	F
L	L	L	H
L	L	H	H
L	H	L	H
L	H	H	H
H	L	L	H
H	L	H	H
H	H	L	H
H	H	H	L

表 3.8　TTL 与非门的逻辑取值关系

输入			输出
A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

(3) TTL 与非门的主要外部特性参数

为了更好地使用各类集成门电路，必须了解它们的外部特性。TTL 与非门的主要外部特性参数有输出逻辑电平、开门电平、关门电平、扇入系数、扇出系数、平均传输时延和空载功耗等。

1) 输出高、低电平

输出高电平 V_{OH}：指与非门的输入至少有一个接低电平时的输出电平。输出高电平的典型值是 3.6V，产品规范值为 $V_{OH} \geqslant 2.4V$。

输出低电平 V_{OL}：指与非门输入全为高电平时的输出电平。V_{OL} 的典型值是 0.3V，产品规范值为 $V_{OL} \leqslant 0.4V$。

一般来说，希望输出高电平与低电平之间的差值愈大愈好，因为两者相差愈大，逻辑值 1 和 0 的区别便愈明显，电路工作也就愈可靠。

2) 开门电平与关门电平

开门电平 V_{ON}：指确保与非门输出为低电平时所允许的最小输入高电平，它表示使与非门开通的输入高电平最小值。V_{ON} 的典型值为 1.5V，产品规范值为 $V_{ON} \leqslant 1.8V$。

关门电平 V_{OFF}：指确保与非门输出为高电平时所允许的最大输入低电平，它

表示使与非门关断的输入低电平最大值。V_{OFF}的典型值为 1.3V，产品规范值 $V_{OFF} \geqslant 0.8V$。

开门电平和关门电平的大小反映了与非门的抗干扰能力。具体说，开门电平的大小反映了输入高电平时的抗干扰能力，V_{ON}愈小，在输入高电平时的抗干扰能力愈强。因为输入高电平和干扰信号叠加后不能低于 V_{ON}，显然，V_{ON}愈小，输入信号允许叠加的负向干扰愈大，即在输入高电平时抗干扰能力愈强。而关门电平的大小反映了输入低电平时的抗干扰能力，V_{OFF}愈大，在输入低电平时的抗干扰能力愈强。因为输入低电平和干扰信号叠加后不能高于 V_{OFF}，显然，V_{OFF}愈大，输入信号允许叠加的正向干扰愈大，即在输入低电平时抗干扰能力愈强。通常将输入高、低电平时所允许叠加的干扰信号大小分别称为高、低电平的噪声容限。

3) 扇入系数与扇出系数

扇入系数 N_I：指与非门允许的输入端数目，它是由电路制造厂家在电路生产时预先安排好的。一般 N_I为 2～5，最多不超过 8。实际应用中要求输入端数目超过 N_I时，可通过分级实现的方法减少对扇入系数的要求。

扇出系数 N_O：指与非门输出端连接同类门的最多个数，它反映了与非门的带负载能力。根据负载电流的流向，可以将负载分为“灌电流负载”和“拉电流负载”。所谓灌电流负载，是指负载电流从外接电路流入与非门，通常用 I_{IL}表示；所谓拉电流负载，是指负载电流从与非门流向外接电路，通常用 I_{IH}表示。下面对两种负载的工作情况分别作简单介绍。

① 灌电流负载工作情况。

TTL 与非门的灌电流负载工作情况如图 3.13(a)所示，图中虚线左边为驱动

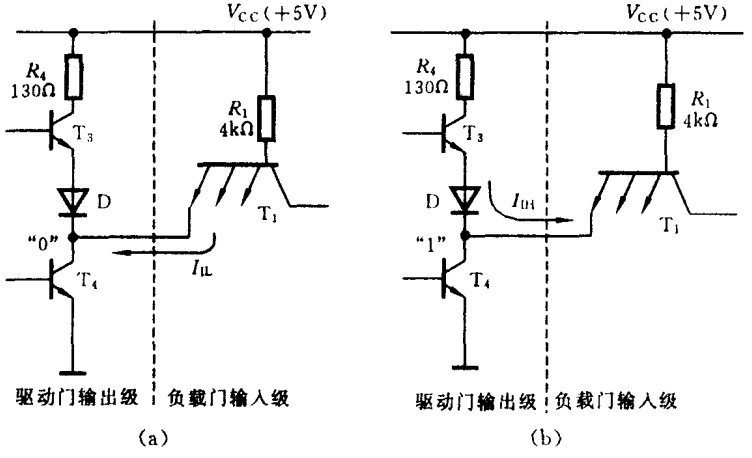


图 3.13 TTL 与非门的拉、灌电流负载工作情况

门输出级，右边为负载门输入级。当驱动门输出为低电平（逻辑值“0”）时，负载门由电源 V_{CC} 通过 R_1 和 T_1 的发射极向驱动门的集电极灌入电流 I_{IL}，即电流从负载门流入驱动门。显然，随着负载门个数的增加，总的灌电流 I_{IL} 将增加，其结果将引起输出低电平升高。然而，如前所述，输出低电平 V_{OL} 的产品规范值为 $V_{OL} \leqslant 0.4V$，因此灌电流 I_{IL} 增加的结果不能使输出低电平高于 0.4V，这就限制了与非门带灌电流负载的个数。

② 拉电流负载工作情况。

TTL 与非门的拉电流负载工作情况如图 3.13(b)所示。当驱动门输出为高电平（逻辑值“1”）时，将有电流 I_{1H} 从驱动门拉出而流至负载门。随着负载门个数的增加，总的拉电流 I_{1H} 将增加，其结果必然引起输出高电平降低。然而如前所述，输出高电平 V_{OH} 的产品规范值为 $V_{OH} \geqslant 2.4V$，因此拉电流 I_{1H} 增加的结果不能使输出高电平低于 2.4V，同样，这就限制了与非门带拉电流负载的个数。

一般情况下带灌电流负载的数目与带拉电流负载的数目是不相等的，扇出系数 N_O 常取二者中的最小值。典型 TTL 与非门的扇出系数约为 10，高性能门电路的扇出系数可高达 30～50。

4）输入短路电流和输入漏电流

输入短路电流 I_{IS}：指当与非门的一个或多个输入端接低电平，而其余输入接高电平或悬空时，流向低电平输入端的电流。在实际电路中，I_{IS} 是流入前级与非门的灌电流，它的大小将直接影响前级与非门的工作情况。因此，对输入短路电流要加以限制，产品规范值 $I_{IS} \leqslant 1.6mA$。

输入漏电流 I_{1H}：指某一输入端接高电平，而其他输入端接低电平时，流入高电平输入端的电流，又称为高电平输入电流。当与非门串联运用时，若前级门输出高电平，则后级门的 I_{1H} 就是前级门的拉电流负载，I_{1H} 过大将使前级门输出的高电平下降。所以，必须将 I_{1H} 限制在一定数值以下，一般 $I_{1H} \leqslant 50\mu A$。

5）平均传输延迟时间

平均传输延迟时间 t_{pd}：指一个矩形波信号从与非门输入端传到与非门输出端（反相输出）所延迟的时间。如图 3.14 所示，通常将从输入波上沿中点到输出波下

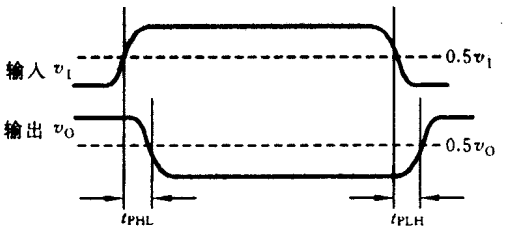


图 3.14　TTL 与非门的传输延迟时间

沿中点的时间延迟称为导通延迟时间 t_{PHL}；从输入波下沿中点到输出波上沿中点的时间延迟称为截止延迟时间 t_{PLH}。平均延迟时间定义为

$$t_{pd}=(t_{PHL}+t_{PLH})/2$$

平均延迟时间是反映与非门开关速度的一个重要参数。t_{pd}的典型值约 10ns，一般小于 40ns。

6）平均功耗

与非门的功耗：指在空载条件下工作时所消耗的电功率。通常将输出为低电平时的功耗称为空载导通功耗 P_{ON}，而输出为高电平时的功耗称为空载截止功耗 P_{OFF}，P_{ON}总比 P_{OFF}大。平均功耗 P 是取空载导通功耗 P_{ON}和空载截止功耗 P_{OFF}的平均值，即

$$P=(P_{ON}+P_{OFF})/2$$

TTL 与非门的平均功耗一般为 20mW 左右。

上面对 TTL 与非门的几个主要外部性能指标进行了介绍，目的在于使读者对逻辑门电路的性能指标有一个大致的了解。有关各种逻辑门的具体参数可在使用时查阅有关集成电路手册和产品说明书。

(4) TTL 与非门集成电路芯片

常用的 TTL 与非门集成电路芯片有 7400、7410 和 7420 等。7400 是一种内部有 4 个 2 输入与非门的芯片，其引脚分配图如图 3.15(a)所示；7410 是一种内部有 3 个 3 输入与非门的芯片，其引脚分配图如图 3.15(b)所示；7420 是一种内部有 2 个 4 输入与非门的芯片，其引脚分配图如图3.15(c)所示。图中，V_{CC}为电源引脚，GND 为接地脚，NC 为空脚。

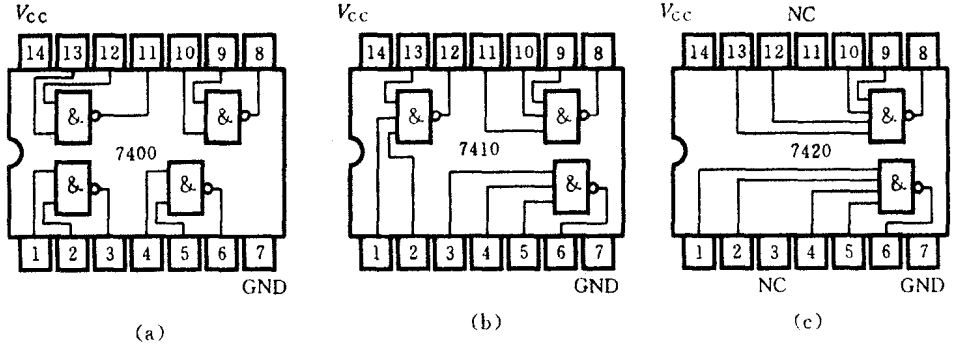


图 3.15 7400、7410、7420 引脚图

2. 常用的集成 TTL 门电路

常用的集成 TTL 门电路除了与非门外，还有非门、或非门、与或非门、异或

门、与门、或门等不同功能的产品。

(1) 非门

图 3.16(a)所示的是一个 TTL 非门电路。当输入 A 为低电平(0.3V)时，电路工作在截止状态，即 T_3 截止，T_4 和 D 导通，输出端 F 为高电平(3.6V)；当输入 A 为高电平(3.6V)时，电路工作在导通状态，即 T_3 饱和导通，T_4 和 D 截止，F 输出低电平(0.3V)，实现了逻辑"非"功能，即 $F=\overline{A}$。

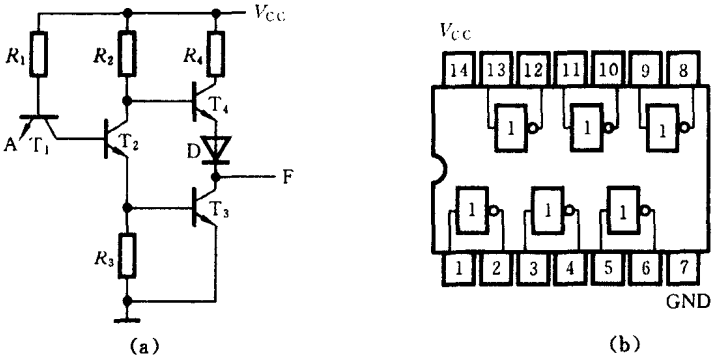


图 3.16　TTL 非门电路

常用的 TTL 非门集成电路芯片有六反相器 7404 等，图 3.16(b)所示为 7404 的引脚分配图。

(2) 或非门

图 3.17(a)所示的是一个 TTL 或非门电路，图中两个虚线框中部分完全相同。当输入 A、B 均为低电平时，T_2 和 T_2' 均截止，从而使 T_3 截止，T_4 和 D 导通，输出 F 为高电平；当 A 端输入高电平时，T_1 处于倒置放大状态，T_2 和 T_3 饱和导通，T_4 和 D 截止，输出 F 为低电平。同样，B 端输入高电平或 A、B 同时输入高电平，均使 T_3 饱和导通，T_4 和 D 截止，F 输出低电平。因此，该电路实现了或非逻辑功能，即 $F=\overline{A+B}$。两输入或非门的逻辑符号如图 3.17(b)所示。

常用的 TTL 或非门集成电路芯片有 2 输入 4 或非门 7402、3 输入 3 或非门 7427 等。图 3.17(c)给出了 7402 的引脚分配图。

(3) 与或非门

将图 3.17(a)所示或非门电路中的 T_1 和 T_1' 改成多射极晶体管，用以实现与的功能，即可构成与或非门。图 3.18(a)所示的是一个 TTL 与或非门电路图。该电路仅当 A_1、A_2 和 B_1、B_2 中均有低电平时，才使 T_2、T_2' 和 T_3 截止，T_4 和 D 导通，输出 F 为高电平。其他情况下，即 A_1、A_2 均为高，或者 B_1、B_2 均为高，或者 A_1、A_2 和 B_1、B_2 均为高，都将使 T_3 饱和导通，T_4 和 D 截止，输出 F 为低电平。因此，该电路

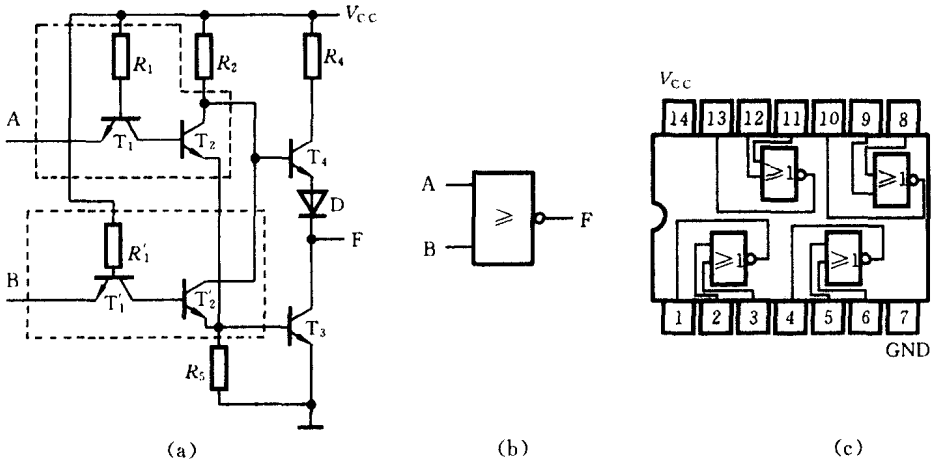


图 3.17 TTL 或非门电路

实现了与或非运算功能，输出和输入之间满足逻辑关系 $F=\overline{A_1A_2+B_1B_2}$。由于该输出函数表达式中包含两个与项，每个与项含两个变量，故通常将其称为 2-2 与或非门。2-2 与或非门的逻辑符号如图 3.18(b)所示。

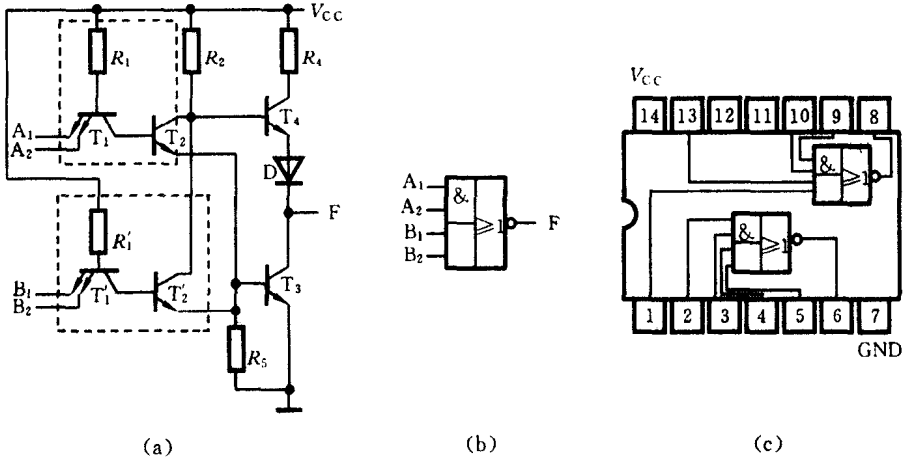


图 3.18 TTL 与或非门电路

常用的 TTL 与或非门集成电路芯片有双 2-2 与或非门 7451、3-2-2-3 与或非门 7454 等。7451 的引脚排列如图 3.18(c)所示。

(4) 异或门

图 3.19(a)所示的是异或门的逻辑符号，异或门只有两个输入端。常用的 TTL 异或门集成电路芯片有 7486 等，图 3.19(b)给出了 7486 的引脚分配图。

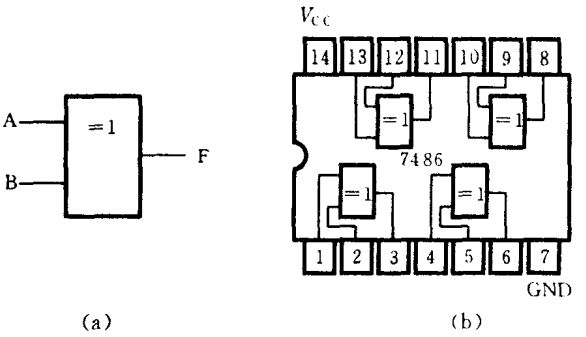


图 3.19　TTL 异或门的逻辑符号和 7486 的引脚分配图

(5) 与门

常用的 TTL 与门集成电路芯片有 2 输入 4 与门 7408、3 输入 3 与门 7411 等。7411 的引脚排列如图 3.20(a)所示。

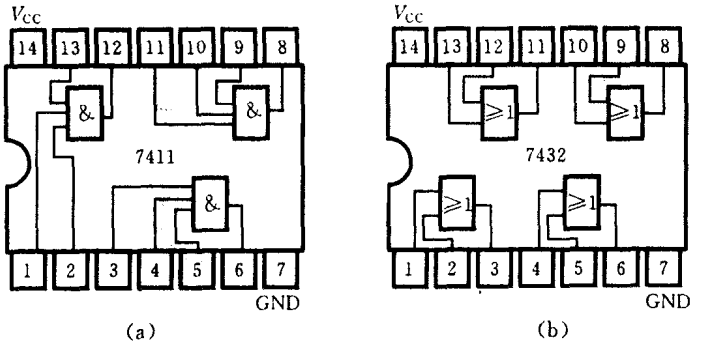


图 3.20　TTL 与门 7411 和或门 7432 的引脚分配图

(6) 或门

常用的 TTL 或门集成电路芯片有 2 输入 4 或门 7432 等。7432 的引脚排列如图 3.20(b)所示。

有关各种 TTL 集成门电路的详细资料可查阅集成电路芯片手册。

3. 两种特殊的门电路

除了前述的一些常用逻辑门之外，实际应用中还有两种广泛使用的特殊门电路——集电极开路门(OC 门)和三态门(TS 门)。

(1) 集电极开路与非门(OC 门)

在实际应用中，如果能将两个与非门的输出端直接对接，如图 3.21(a)所示，则可实现与逻辑功能。该电路中，只要 F_1 和 F_2 有一个为低电平，则 F 为低电平，

仅当 F_1 和 F_2 均为高电平时，才能使 F 为高电平，即 $F = F_1 \cdot F_2$。由于这种与逻辑功能的实现并没有使用与门，而是由门电路输出引线连接实现的，故称为“线与”逻辑。然而，前面介绍的推拉式输出结构的 TTL 逻辑门电路是不能将两个门的输出端直接并接使用的。如果将两个门的输出端直接相连，则电路的实际工作状况如图 3.21(b)所示。假定该电路中门 1 的输出 F_1 为高电平，门 2 的输出 F_2 为低电平，则由于推拉式输出级不论门电路处于导通状态还是截止状态，都呈现低阻抗，因而将会有一个很大的负载电流流过门 1 的 T_3 和 D_4 以及门 2 的 T_4，该电流远远超过电路的正常工作电流，有可能导致逻辑门损坏。

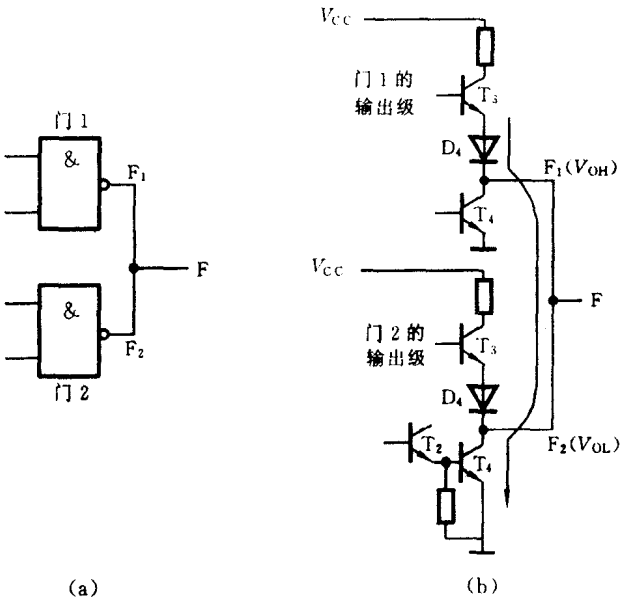


图 3.21　两个 TTL 与非门输出端直接并接使用的情况

为了使 TTL 逻辑门能够实现“线与”，以便解决实际应用的需要。TTL 系列产品中专门设计了一种输出端可以相互连接的特殊逻辑门，称为集电极开路门(Open Collector Gate，简称 OC 门)。

图 3.22 (a)和(b)分别给出了一个集电极开路与非门的电路结构和逻辑符号。

该电路把一般 TTL 与非门中的 T_3、D_4 去掉，令 T_4 的集电极悬空，从而把一般 TTL 与非门电路的推拉式输出级改为三极管集电极开路输出，使用时通过外接负载电阻 R_L 和电源 V'_{CC} 令其正常工作。只要电阻 R_L 和电源 V'_{CC} 选择恰当，就能既保证输出的高、低电平正常，又能使流过输出级的电流不致过大。

在数字系统中，使用集电极开路与非门可以很方便地实现“线与”逻辑、电平转换以及直接驱动发光二极管、干簧继电器等。

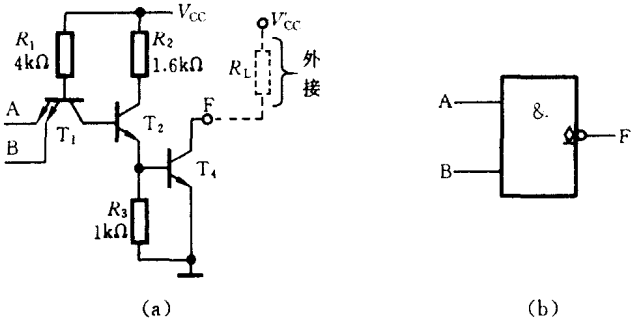


图 3.22　集电极开路与非门的电路结构和逻辑符号

例如，将两个 OC 与非门按图 3.23 所示连接，只要其中有一个输出为低电平，输出 F 便为低电平；仅当两个门的输出均为高电平时，输出 F 才为高电平。即 $F=F_1 \cdot F_2=\overline{A_1B_1C_1} \cdot \overline{A_2B_2C_2}$，从而实现了两个与非门输出相与的逻辑功能。

又如，将 OC 与非门按图 3.24 所示连接，即可实现电平转换，V_r 为转换后的电平值。

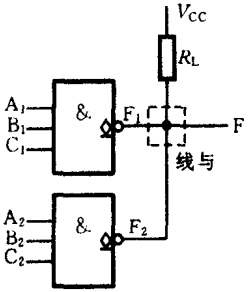


图 3.23　线与逻辑电路

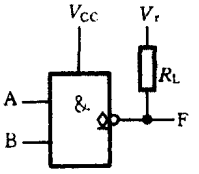


图 3.24　电平转换逻辑电路

(2) 三态输出门(TS 门)

三态输出门简称三态门，或称为 TS(Three state)门。三态门有 3 种输出状态：输出高电平、输出低电平和高阻状态。前两种状态为工作状态，后一种状态为禁止状态。值得注意的是，三态门不是指具有 3 种逻辑值。在工作状态下，三态门的输出可为逻辑“1”或者逻辑“0”；在禁止状态下，其输出呈现高阻抗，相当于开路。

图 3.25(a)给出了一个三态输出与非门的电路结构。

从图 3.25(a)可知，该电路在一般与非门的基础上，增加了一个二极管 D 和一个使能控制端 EN。当控制信号 EN=1 时，二极管 D 反偏，此时电路功能与一般与非门并无区别，输出 $F=\overline{AB}$；当控制信号 EN=0 时，一方面因为 T_1 有一个输入端为低，使 T_2、T_4 截止，另一方面由于二极管导通，迫使 T_3 的基极电位被钳制在 1V 左右，致使 T_3、D_4 也截止，这时输出 F 被悬空，即处于高阻状态。因为该电路是在 EN 为高电平时处于正常工作状态，所以称为使能控制端高电平有效的三态与

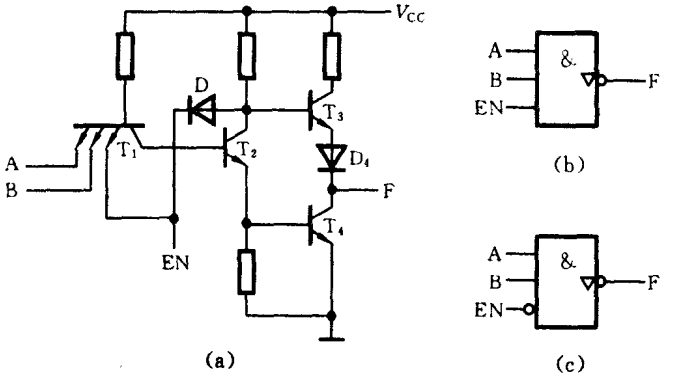


图 3.25　三态输出与非门电路结构和逻辑符号

非门，与其对应的逻辑符号如图 3.25 (b)所示。

三态门也有使能控制端为低电平有效的产品，为了与使能控制端为高电平有效的三态门相区别，通常在逻辑符号的控制端加一个小圆圈，如图 3.25(c)所示逻辑符号表示使能控制端为低电平有效的三态输出与非门。有时也将使能控制信号用$\overline{EN}$表示低电平有效的三态门。

利用三态门不仅可以实现线与，而且被广泛应用于总线传送，它既可用于单向数据传送，也可用于双向数据传送。

图 3.26 所示为用三态门构成的单向数据传输总线。当某个三态门的控制端为 1 时，该逻辑门处于工作状态，输入数据经反相后送至总线。为了保证数据传送的正确性，任意时刻，n 个三态门的控制端只能有一个为 1，其余均为 0，即只允许一个数据端与总线接通，其余均断开，以便实现 n 个数据的分时传送。

图 3.27 所示为用两种不同控制输入的三态门构成的双向数据传输总线。图

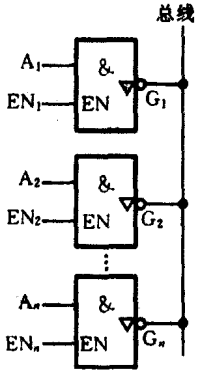


图 3.26　用三态门构成单向总线

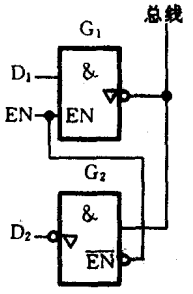


图 3.27　三态门构成的双向总线

中 EN=1 时，G_1工作，G_2处于高阻状态，数据 D_1被取反后送至总线；EN=0 时，G_2工作，G_1处于高阻状态，总线上的数据被取反后送到数据端 D_2，从而实现了数据的分时双向传送。

多路数据通过三态门共享总线，实现数据分时传送的方法，在计算机和其他数字系统中被广泛用于数据和各种信号的传送。

3.3.3　CMOS 集成逻辑门电路

以 MOS 管作为开关元件的门电路称为 MOS 门电路。由于 MOS 集成门电路具有制造工艺简单、集成度高、功耗小、抗干扰能力强等优点，因此，在数字集成电路产品中占据着相当大的比例。

MOS 门电路有使用 P 沟道 MOS 管的 PMOS 电路，使用 N 沟道 MOS 管的 NMOS 电路和同时使用 PMOS 管和 NMOS 管的 CMOS 电路 3 种类型，CMOS 电路是目前应用最广泛的一类集成电路。随着制造工艺的不断改进，CMOS 电路的工作速度已接近 TTL 电路，而在集成度、功耗、抗干扰能力等方面则远远优于 TTL 电路。目前，几乎所有的超大规模集成器件，如超大规模存储器件、可编程逻辑器件等都采用 CMOS 工艺制造。下面，在简单介绍 MOS 管静态开关特性的基础上，讨论几种常用的 CMOS 集成逻辑门电路。

1. MOS 晶体管的静态开关特性

MOS 集成电路的基本元件是 MOS 晶体管。MOS 晶体管是一种电压控制器件，它的三个电极分别称为栅极(G)、漏极(D)和源极(S)，由栅极电压控制漏源电流。

MOS 晶体管根据结构的不同可分为 P 型沟道 MOS 管和 N 型沟道 MOS 管两种，每种又可按其工作特性进一步分为增强型和耗尽型两类。现以 N 型沟道增强型 MOS 管为例对其静态工作特性作简单介绍。

N 沟道增强型 MOS 管的结构如图 3.28(a)所示。在一块 P 型硅片衬底上，利用扩散工艺形成两个高浓度掺杂的 N^+区，并用金属铝引出两个电极，分别作为漏极(D)和源极(S)。然后在半导体表面覆盖一层二氧化硅(SiO_2)绝缘层，再在漏、源之间的绝缘层上加上一层金属铝，引出一个电极作为栅极(G)，便构成了一个 N 沟道增强型 MOS 管。N 沟道增强型 MOS 管的逻辑符号如图 3.28(b)所示，符号中的箭头表示栅极电流的方向由 P 区指向 N 区。

由图 3.28(a)可知，在 N 沟道增强型 MOS 管的漏、源之间是两个背靠背的 PN 结。当栅源电压 $v_{GS}=0$ 时，不论漏源电压 v_{DS}的极性如何，总有一个 PN 结处于反偏状态，漏、源之间没有导电沟道，此时漏源电流 $i_{DS}=0$。如果在栅、源之间加

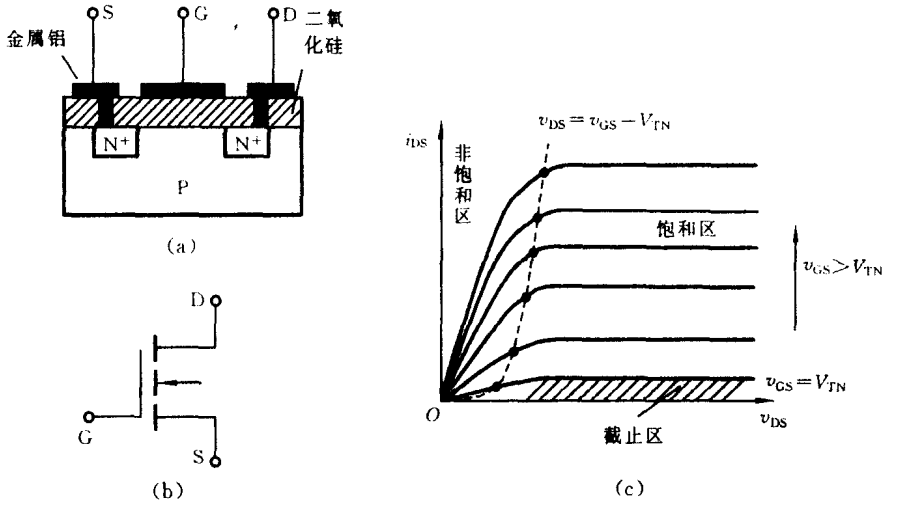


图 3.28　N 沟道增强型 MOS 管

上一定的正向电压 v_{GS}，则将形成一个由栅极指向衬底的电场，该电场一方面使栅极附近 P 型衬底中的空穴被排斥，同时又将 P 型衬底中的少数电子吸引到衬底表面。当 v_{GS}较小时，吸引电子的能力不够强，在漏、源之间不够形成导电沟道。随着 v_{GS}的增大，被吸引的电子浓度增大，当 v_{GS}达到某一数值时，足够多的电子在栅极附近 P 型衬底表面形成一个 N 型薄层，且与两个 N^+ 区相连通，从而在漏、源之间形成导电沟道。这时，若在漏、源之间加上电压 v_{DS}，就会产生漏源电流 i_{DS}。通常将开始形成导电沟道的栅源电压称为开启电压，用 V_{TN}表示，一般 V_{TN}大约在 1～2V之间。

形成导电沟道后，在漏源电压 v_{DS}的作用下，沟道厚度将发生一定变化，其结果使得靠近源极一端的最厚，而靠近漏极一端的最薄。因而，漏源电流 i_{DS}沿沟道产生的压降使沟道内各点与栅极间的电压不再相等，靠近源极一端的电压最大，而靠近漏极一端的电压最小。但当 v_{DS}比较小时，对沟道的影响不大。

图 3.28(c)所示是 N 沟道增强型 MOS 管的输出特性曲线。输出特性曲线表示在一定栅源电压 v_{GS}作用下，漏源电流 i_{DS}和漏源电压 v_{DS}之间的关系。

图 3.28(c)所示输出特性曲线被分为截止区、非饱和区和饱和区。由此可见，在一定栅源电压 v_{GS}控制下，N 沟道增强型 MOS 管有截止、非饱和、饱和 3 种工作状态。

(1) 截止状态

当 $v_{GS}<V_{TN}$时，对应图 3.28(c)中的截止区，此时没有导电沟道，$i_{DS}=0$，管子工作在截止状态。

(2) 非饱和状态

当漏源电压较小，满足 $v_{DS}<v_{GS}-V_{TN}$（对应图 3.28(c)中的非饱和区）时，漏源电流 i_{DS} 基本上随着漏源电压 v_{DS} 线性上升。而且 v_{GS} 愈大，形成的导电沟道愈宽，相应的等效电阻愈小，i_{DS} 愈大（输出特性曲线愈陡）。此时，管子工作在非饱和状态，有时又将其称为电阻可调状态。

(3) 饱和状态

当漏源电压加大到一定程度，满足 $v_{DS}\geqslant v_{GS}-V_{TN}$（对应图 3.28(c)中的饱和区）时，靠近漏极一端的导电沟道不断变薄的结果，致使在漏极附近被夹断。这时 i_{DS} 不再随 v_{DS} 线性上升，而是达到某一个值之后，v_{DS} 的增加只使 i_{DS} 产生微小的变化，几乎近似不变，即趋于饱和。此时，管子工作在饱和状态。

在输出特性曲线上，把满足 $v_{DS}=v_{GS}-V_{TN}$ 的临界点连接起来，便构成了饱和区与非饱和区的分界线（对应图 3.28(c)中的虚线），又称为临界线。

在数字系统中，MOS 管作为开关应用时，是在开关信号作用下交替工作在截止与饱和两种工作状态。

采用与 N 沟道增强型 MOS 管类似的方法，在一块 N 型硅片衬底上，利用扩散工艺形成两个高浓度掺杂的 P^+ 区，并用金属铝引出两个电极，分别作为漏极(D)和源极(S)。然后在半导体表面覆盖一层二氧化硅(SiO_2)绝缘层，再在漏、源之间的绝缘层上加上一层金属铝，引出一个电极作为栅极(G)，便构成了一个 P 沟道增强型 MOS 管。P 沟道增强型 MOS 管的结构和逻辑符号如图 3.29(a)、(b)所示。

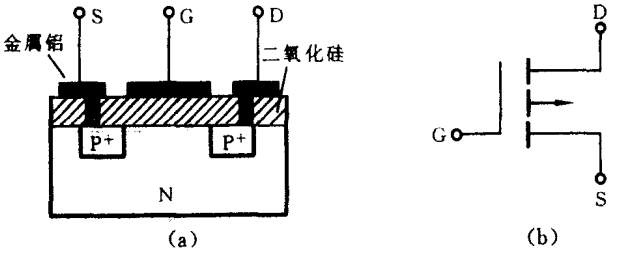


图 3.29　P 沟道增强型 MOS 管

P 沟道增强型 MOS 管的工作电压 v_{DS} 和 v_{GS} 均为负电压。在栅极加上一定的负电压，可使栅极附近 N 型衬底表面形成一个 P 型薄层，且与两个 P^+ 区相连通，从而在漏、源之间形成一个载流子为空穴的 P 型导电沟道。P 沟道增强型 MOS 管的开启电压用 V_{TP} 表示，一般 V_{TP} 大约在 $-2.5\sim-1.0$V 之间。

2. CMOS 集成逻辑门

(1) CMOS 反相器

CMOS 反相器由一个 P 沟道增强型 MOS 管和一个 N 沟道增强型 MOS 管串联组成，通常用 P 沟道增强型 MOS 管作为负载管，N 沟道增强型 MOS 管作为工作管。如图 3.30 所示，PMOS 管 T_P 为负载管(开启电压用 $V_{TP}<0$)，NMOS 管 T_N 为工作管(开启电压用 $V_{TN}>0$)，两管的栅极相连作为输入端，两管的漏极相连作为输出端。T_N 的源极接地，T_P 的源极接电源。为了保证电路正常工作，电源电压 V_{DD} 需大于 T_N 管开启电压 V_{TN} 和 T_P 管开启电压 V_{TP} 绝对值的和，即 $V_{DD}>V_{TN}+|V_{TP}|$，一般 $V_{DD}=5V$。

在该电路中，当 $v_I=0V$ 时，T_N 截止，T_P 导通，电路工作在截止状态，输出电压 $v_O\approx V_{DD}$ 为高电平；当 $v_I=V_{DD}$ 时，T_N 导通，T_P 截止，电路工作在导通状态，输出电压 $v_O\approx 0V$。即“若输入为低，则输出为高；若输入为高，则输出为低”。因此，该电路实现了反相器功能，即非的逻辑功能，电路输出与输入的关系为 $F=\overline{A}$。

由于 CMOS 反相器处在开关状态下总有一个管子处于截止状态，因而电流极小，电路静态功耗很低。此外，它还具有抗干扰性能好、负载能力强等优点。

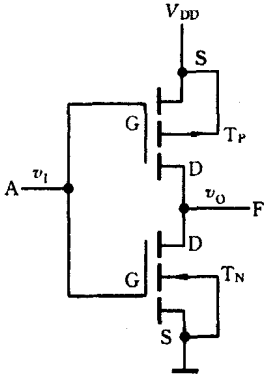


图 3.30 CMOS 反相器电路

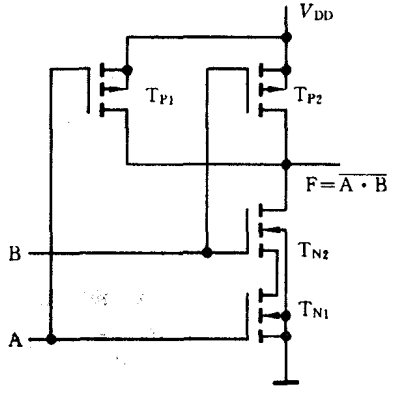


图 3.31 CMOS 与非门电路

(2) CMOS 与非门

图 3.31 所示的是一个两输入端的 CMOS 与非门电路，它由两个串联的 NMOS 管 T_{N1}、T_{N2} 和两个并联的 PMOS 管 T_{P1}、T_{P2} 构成。

每个输入端连到一个 PMOS 管和一个 NMOS 管的栅极。当输入 A、B 均为高电平时，T_{N1} 和 T_{N2} 导通，T_{P1} 和 T_{P2} 截止，输出端 F 为低电平；当输入 A、B 中至少有一个为低电平时，对应的 T_{N1} 和 T_{N2} 中至少有一个截止，T_{P1} 和 T_{P2} 中至少有一个导通，输出 F 为高电平。因此，该电路实现了与非逻辑功能，电路输出与输入

的关系为 $F=\overline{AB}$。

(3) CMOS 或非门

图 3.32 所示的是一个两个输入端的 CMOS 或非门电路，它由两个并联的 NMOS 管 T_{N1}、T_{N2}和两个串联的 PMOS 管 T_{P1}、T_{P2}构成。每个输入端连接到一个 NMOS 管和一个 PMOS 管的栅极。当输入 A、B 均为低电平时，T_{N1}和 T_{N2}截止，T_{P1}和 T_{P2}导通，输出 F 为高电平；只要输入端 A、B 中有一个为高电平，则对应的 T_{N1}、T_{N2}中便至少有一个导通，T_{P1}、T_{P2}中便至少有一个截止，使输出 F 为低电平。因此，该电路实现了或非逻辑功能，电路输出与输入的关系为 $F=\overline{A+B}$。

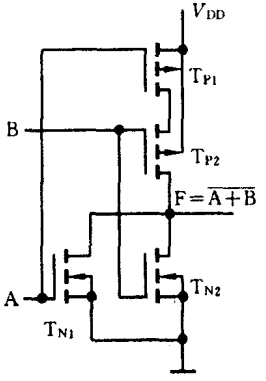


图 3.32　CMOS 或非门电路

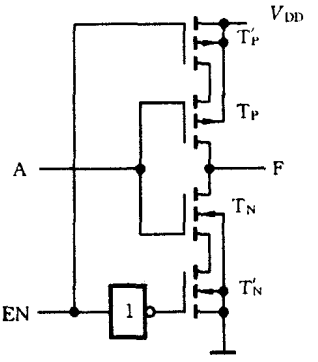


图 3.33　CMOS 三态门

(4) CMOS 三态门

CMOS 三态门是在普通门电路的基础上增加控制电路构成的，其电路结构有不同形式。图 3.33 所示是一个简单的三态非门电路，从电路结构上看，该电路是在 CMOS 反相器的基础上增加了 NMOS 管 T'_N和 PMOS 管 T'_P构成的。当使能控制端 EN 为高电平(EN =1)时，T'_N和 T'_P同时截止，输出 F 呈高阻状态；当使能控制端 EN 为低电平(EN =0)时，T_N和 T_P同时导通，非门正常工作，实现 $F=\overline{A}$ 的功能。由于 EN 为低电平时电路处在正常工作状态，所以称为低电平有效的三态门。

(5) CMOS 传输门

CMOS 传输门的电路结构如图 3.34 (a)所示，它由一个 NMOS 管 T_N和一个 PMOS 管 T_P并接构成，其逻辑符号如图 3.34(b)所示。

图 3.34 (a)中，T_N和 T_P的结构和参数对称，两管的源极连在一起作为传输门的输入端，漏极连在一起作为输出端。T_N的衬底接地，T_P的衬底接电源，两管的栅极分别与一对互补的控制信号 C 和 $\overline{C}$ 相接。

当控制端 $C = V_{DD}$，$\overline{C}= 0V$ 时，输入电压 v_I在 0V～V_{DD}范围内变化，两管中至少有一个导通，输入和输出之间呈低阻状态，相当于开关接通，即输入信号 v_I在

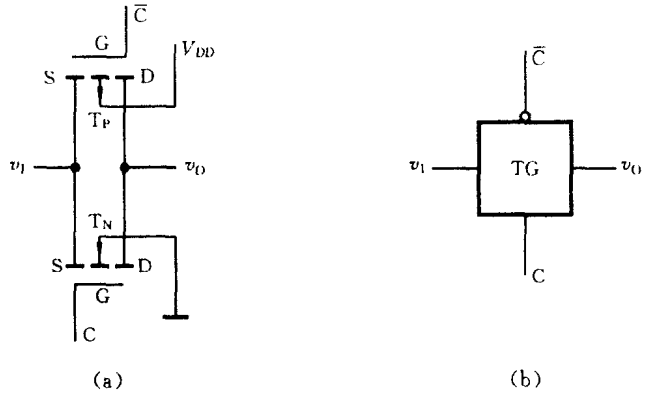


图 3.34 CMOS 传输门

0V～V_{DD}范围内都能通过传输门。

当控制端 C=0V,$\overline{C}=V_{DD}$时,输入信号 v_I在 0V～V_{DD}范围内变化,两管都总是处于截止状态,输入和输出之间呈高阻状态,信号 v_I不能通过,相当于开关断开。

由此可见,变换两个控制端的互补信号,可以使传输门接通或断开,从而决定输入端的信号(0V～V_{DD}之间的任意电平)是否能传送到输出端。由于传输门不仅能传输数字信号,同时也能传输模拟信号,所以在模拟电路中,传输门被用于传输连续变化的模拟电压信号。

此外,由于 MOS 管的结构是对称的,即源极和漏极可以互换使用,因此,传输门的输入端和输出端可以互换使用,即 CMOS 传输门具有双向性,故又称为可控双向开关。

3.3.4 正逻辑和负逻辑

1. 正逻辑与负逻辑的概念

前面介绍各种逻辑门电路时,是约定用高电平表示逻辑 1、低电平表示逻辑 0 来讨论其逻辑功能的。事实上,用电平的高和低表示逻辑值 1 和 0 的关系并不是惟一的。既可以规定用高电平表示逻辑 1、低电平表示逻辑 0,也可以规定用高电平表示逻辑 0,低电平表示逻辑 1。这就引出了正逻辑和负逻辑的概念。

通常,把用高电平表示逻辑 1,低电平表示逻辑 0 的规定称为**正逻辑**。反之,把用高电平表示逻辑 0,低电平表示逻辑 1 的规定称为**负逻辑**。

2. 正逻辑与负逻辑的关系

对于同一电路，正逻辑与负逻辑的规定不涉及逻辑电路本身的结构与性能好坏，但不同的规定可使同一电路具有不同的逻辑功能。

例如，假定某逻辑门电路的输入/输出电平关系如表 3.9 所示。若按正逻辑规定，则可得到表 3.10 所示的真值表，由真值表可知，该电路是一个正逻辑的与门；若按负逻辑规定，则可得到表 3.11 所示的真值表，由真值表可知，该电路是一个负逻辑的或门。即正逻辑与门等价于负逻辑或门。

表 3.9　输入/输出电平关系

输入		输出
A	B	F
L	L	L
L	H	L
H	L	L
H	H	H

表 3.10　正逻辑真值表

输入		输出
A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

表 3.11　负逻辑真值表

输入		输出
A	B	F
1	1	1
1	0	1
0	1	1
0	0	0

上述逻辑关系可以用反演律证明。假定一个正逻辑与门的输出为 F，输入为 A、B 即有

$$F=A\cdot B$$

根据反演律，可得

$$\overline{F}=\overline{A}+\overline{B}$$

这就是说，若将一个逻辑门的输出和所有输入都反相，则正逻辑变为负逻辑。据此，可将正逻辑门转换为负逻辑门。

几种常用逻辑门的正、负逻辑符号变换如图 3.35 所示。

前面讨论各种逻辑门电路时，都是按照正逻辑规定来定义其逻辑功能的。

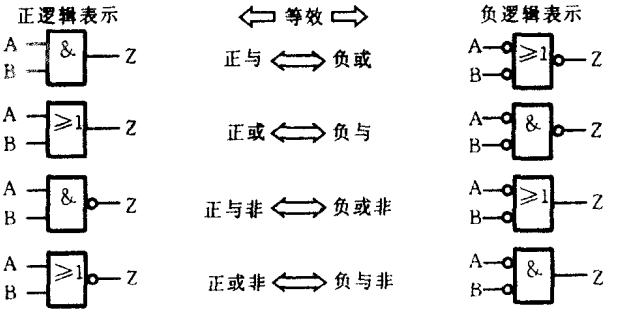
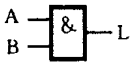
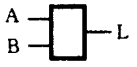
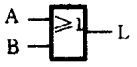
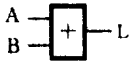
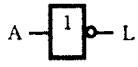
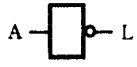
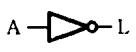
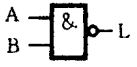
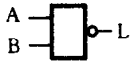
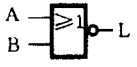
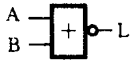
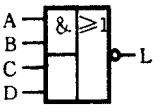
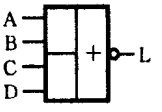
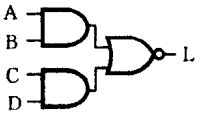
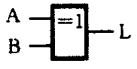
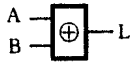
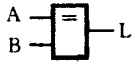
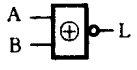


图 3.35　正逻辑门变换成等效负逻辑门

在本教材中，若无特殊说明，则约定按正逻辑讨论问题，所有门电路的符号均按正逻辑表示。表 3.12 列出了各种常用逻辑门的 3 种符号形式以及它们的逻辑表达式。

表 3.12 常用门电路符号及表达式

名称	逻辑功能	新标准符号	沿用符号	国外符号	逻辑表达式
与门	与运算				$L=A\cdot B$
或门	或运算				$L=A+B$
非门	非运算				$L=\overline{A}$
与非门	与非运算				$L=\overline{A\cdot B}$
或非门	或非运算				$L=\overline{A+B}$
与或非门	与或非运算				$L=\overline{AB+CD}$
异或门	异或运算				$L=A\oplus B$ $=A\overline{B}+\overline{A}B$
同或门	同或运算				$L=A\odot B$ $=\overline{A\oplus B}$ $=AB+\overline{A}\overline{B}$

3.4 触 发 器

在数字系统中，为了构造实现各种功能的逻辑电路，除了需要实现逻辑运算的逻辑门之外，还需要有能够保存信息的逻辑器件。触发器是一种具有记忆功能的

电子器件，它具有如下特点：

① 触发器有两个互补的输出端 Q 和 $\overline{Q}$。

② 触发器有两个稳定状态。输出端 $Q=1$、$\overline{Q}=0$ 称为“1”状态；$Q=0$、$\overline{Q}=1$ 称为“0”状态。当输入信号不发生变化时，触发器状态稳定不变。

③ 在一定输入信号作用下，触发器可以从一个稳定状态转移到另一个稳定状态，输入信号撤销后，保持新的状态不变。通常把输入信号作用之前的状态称为“现态”，记作 Q^n 和 $\overline{Q}^n$，而把输入信号作用后的状态称为触发器的“次态”，记作 Q^{n+1} 和 $\overline{Q}^{n+1}$。为了简单起见，一般省略现态的右上标 n，就用 Q 和 $\overline{Q}$ 表示现态。显然，次态是现态和输入的函数。

由上述特点可知，触发器是存储一位二进制信息的理想器件。集成触发器的种类很多，分类方法也各不相同，按触发器的逻辑功能通常将其分为 R-S 触发器、D 触发器、J-K 触发器和 T 触发器 4 种不同类型。不管如何分类，就其结构而言，触发器都是由逻辑门加上适当的反馈线耦合而成。本节从实际应用出发，介绍几种常用集成触发器的内部结构、工作特性和逻辑功能，重点讨论它们的逻辑功能及其描述方法。

3.4.1　基本 R-S 触发器

基本 R-S 触发器是直接复位(Reset)-置位(Set)触发器的简称，由于它既是一种最简单的触发器，又是构成各种其他功能触发器的基本部件，故称为基本 R-S 触发器。

1. 用与非门构成的基本 R-S 触发器

基本 R-S 触发器可由两个与非门交叉耦合构成，其逻辑电路和逻辑符号分别如图 3.36(a)和(b)所示。图中，Q 和 $\overline{Q}$ 为触发器的两个互补输出端；R 和 S 为触发器的两个输入端，其中 R 称为置 0 端或者复位端，S 称为置 1 端或置位端；加在逻辑符号输入端的小圆圈表示低电平或负脉冲有效，即仅当低电平或负脉冲作用于输入端时，触发器状态才能发生变化(通常称为翻转)，有时称这种情况为低电平

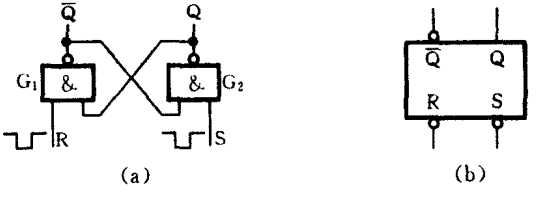


图 3.36　与非门构成的 R-S 触发器

触发或负脉冲触发。

(1) 工作原理

根据与非门的逻辑特性，可分析出 3.36(a) 所示电路的工作原理如下：

① 若 R=1，S=1，则触发器保持原来状态不变。假定触发器原来处于 0 状态，即 Q=0，$\overline{Q}$ =1。由于与非门 G_2 的输出 Q 为 0，反馈到与非门 G_1 的输入端，使 $\overline{Q}$ 保持 1 不变，$\overline{Q}$ 为 1 又反馈到与非门 G_2 的输入端，使 G_2 的两个输入均为 1，从而维持输出 Q 为 0；假定触发器原来处于 1 状态，即 Q=1，$\overline{Q}$ =0，那么，$\overline{Q}$ 为 0 反馈到与非门 G_2 的输入端，使 Q 保持 1 不变，此时与非门 G_1 的两个端入均为 1，所以 $\overline{Q}$ 保持 0。

② 若 R=1，S=0，则触发器置为 1 状态。此时，无论触发器原来处于何状态，因为 S 为 0，必然使与非门 G_2 的输出 Q 为 1，且反馈到与非门 G_1 的输入端；由于门 G_1 的另一个输入 R 也为 1，故使门 G_1 输出 $\overline{Q}$ 为 0，触发器状态为 1 状态。该过程称为触发器置 1。

③ 若 R=0，S=1，则触发器置为 0 状态。与② 的过程类似，不论触发器原来处于 0 状态还是 1 状态，因为 R 为 0，必然使与非门 G_1 的输出 $\overline{Q}$ 为 1，且反馈到与非门 G_2 的输入端，由于门 G_2 的另一个输入 S 也为 1，故使门 G_2 输出 Q 为 0，触发器状态为 0 状态。该过程称为触发器置 0。

④ 不允许出现 R=0，S=0。因为当 R 和 S 端同时为 0 时，将使两个与非门的输出 Q 和 $\overline{Q}$ 均为 1，破坏了触发器两个输出端的状态应该互补的逻辑关系。此外，当这两个输入端的 0 信号被撤消时，触发器的状态将是不确定的，这取决于两个门电路的时间延迟。若 G_1 的时延大于 G_2，则 Q 端先变为 0，使触发器处于 0 状态；反之，若 G_2 的时延大于 G_1，则 $\overline{Q}$ 端先变为 0，从而使触发器处于 1 状态。通常，两个门电路的延迟时间是难以人为控制的，因而在将输入端的 0 信号同时撤去后触发器的状态将难以预测，这是不允许的。因此，规定 R 和 S 不能同时为 0。

(2) 逻辑功能描述

触发器的逻辑功能通常用功能表、状态表、状态图、次态方程和激励表等进行描述。

1) 功能表

根据上述工作原理，可归纳出由与非门构成的 R-S 触发器的逻辑功能，如表 3.13所示，表中“d”表示触发器次态不确定。功能表描述了触发器次态 Q^{n+1} 与现态、输入之间的函数关系，所以又称为次态真值表。

2) 状态表

状态表反映了触发器在输入作用下现态 Q 与次态 Q^{n+1} 之间的转移关系，又称为状态转移表。它详细地给出了触发器次态与现态、输入之间取值关系。由与非

门构成的 R-S 触发器的状态表如表 3.14 所示。

表 3.13　与非门构成的基本 R-S 触发器功能表

R	S	Q^{n+1}	功能说明
0	0	d	不定
0	1	0	置 0
1	0	1	置 1
1	1	Q	不变

表 3.14　与非门构成的基本 R-S 触发器状态表

现　态	次态 Q^{n+1}			
Q	RS=00	RS=01	RS=11	RS=10
0	d	0	0	1
1	d	0	1	1

3）状态图

状态图是一种反映触发器两种状态之间转移关系的有向图，又称为状态转移图。该触发器的状态图如图 3.37 所示。图中，两个圆圈分别代表触发器的两个稳定状态，箭头表示在输入信号作用下状态转移的方向，箭头旁边的标注表示状态转移的条件。

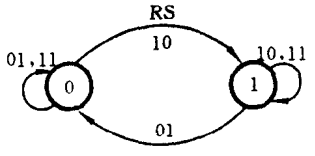


图 3.37　与非门构成的 R-S 触发器的状态图

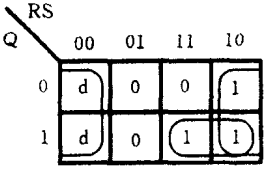


图 3.38　与非门构成的 R-S 触发器的次态卡诺图

4）次态方程

触发器的功能也可以用反映次态 Q^{n+1} 与现态、输入之间关系的逻辑函数表达式进行描述，这种描述触发器功能的逻辑函数表达式称为次态方程。根据表 3.14 所示状态表，可作出描述该触发器次态 Q^{n+1} 与现态 Q 以及输入 R、S 之间函数关系的卡诺图，如图 3.38 所示。

利用 R、S 不允许同时为 0 的约束，化简后可得到该触发器的次态方程为

$$Q^{n+1}=\bar{S}+R\cdot Q$$

因为 R、S 不允许同时为 0，所以输入必须满足约束方程

$$R+S=1$$

5）激励表

触发器的激励表反应了触发器从现态 Q 转移到某种次态 Q^{n+1} 时，对输入信号的要求。它把触发器的现态和次态作为自变量，而把触发器的输入（或激励）作为因变量。激励表可以由功能表导出，与非门构成的基本 R-S 触发器的激励表如表

3.15 所示。

触发器的功能表、状态表、状态图、次态方程和激励表分别从不同角度对触发器的功能进行了描述，它们在时序电路分析和设计中各有用途。例如，在时序电路分析时通常要用到功能表或次态方程，而在时序电路设计时通常要用到激励表或次态方程等。

表 3.15 与非门构成的基本 R-S 触发器激励表

$Q \to Q^{n+1}$		R	S
0	0	d	1
0	1	1	0
1	0	0	1
1	1	1	d

与非门构成的基本 R-S 触发器有一个特点：当输入端 S 连续出现多个置 1 信号，或者输入端 R 连续出现多个清 0 信号时，仅第一个信号使触发器翻转，其工作波形如图 3.39 所示。

由波形图可见，当触发器的同一输入端连续出现多个负脉冲信号时，仅第一个负脉冲信号使触发器发生翻转，后面重复出现的负脉冲信号不起作用。

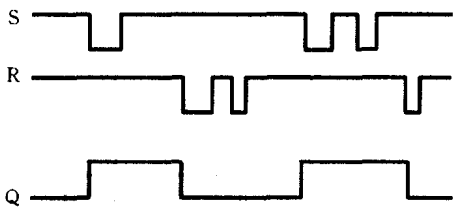


图 3.39 与非门构成的基本 R-S 触发器的时间图

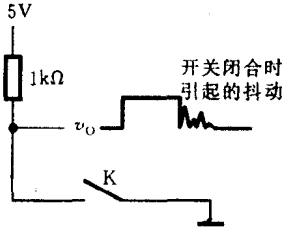


图 3.40 机械开关震动引起的干扰信号

实际应用中，通常利用基本 R-S 触发器的这一特点消除机械开关振动引起的尖脉冲信号。例如，图 3.40 所示的是机械触点开关接通时引起电压波形产生的尖脉冲信号，有时将其称为“毛刺”。显然，这是一种电路中不希望出现的干扰信号。

利用基本 R-S 触发器的上述工作特点，将开关与触发器按照图 3.41(a)所示连接，可以消除开关振动所产生的影响，电压波形如图 3.41(b)所示。设开关原来与 R 接通(R 为 0)，触发器处于 0 状态，当开关由 R 拨向 S 时，经过一个短暂的浮空时间(此时触发器的 R、S 均为 1)，触发器保持 0 状态。当开关与 S 接通时，S 变为 0，同时 S 点的电压 v_S 由于开关振动产生“毛刺”，但由于 R 点的电压 v_R 已为高电平，S 点的电位一旦变低，触发器便翻转为 1 状态，即使 S 点再次出现高电平，也不会再改变触发器状态，所以在触发器 Q 端的电压 v_O 不会出现“毛刺”。类似地，当开关由 S 拨向 R 时，R 点的电压 v_R 会由于开关振动产生“毛刺”，但在触发器 Q 端的电压 v_O 不会出现“毛刺”。

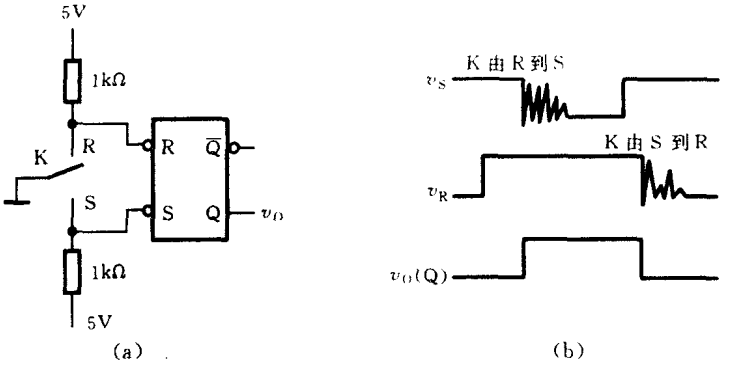


图 3.41　利用基本 R-S 触发器消除机械开关振动引起的干扰

2. 用或非门构成的基本 R-S 触发器

基本 R-S 触发器也可以用两个或非门交叉耦合组成，其逻辑电路和逻辑符号分别如图 3.42(a)和(b)所示。该电路的输入是正脉冲或高电平有效，故逻辑符号的输入端未加小圆圈。

(1) 工作原理

根据或非门的逻辑特性，可分析出 3.42(a) 所示电路的工作原理如下：

① 若 R=0，S=0，则触发器保持原来状态不变；

② 若 R=0，S=1，则触发器置为 1 状态；

③ 若 R=1，S=0，则触发器置为 0 状态；

④ 不允许 R、S 同时为 1，因为当 R 和 S 端同时为 1 时，将破坏触发器正常功能的实现。

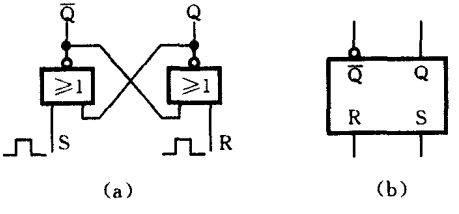


表 3.16　或非门构成的基本 R-S 触发器功能表

R	S	Q^{n+1}	功能说明
0	0	Q	不变
0	1	1	置 1
1	0	0	置 0
1	1	d	不定

图 3.42　或非门构成的 R-S 触发器

(2) 逻辑功能描述

根据电路工作原理，可以得出由或非门构成的基本 R-S 触发器的功能表，如表 3.16 所示。

或非门构成的基本 R-S 触发器的次态方程和约束方程如下：

$$Q^{n+1} = S + \overline{R}Q \qquad \text{（次态方程）}$$

$$R \cdot S = 0 \qquad \text{（约束方程）}$$

读者可自己分析得出该触发器的状态表、状态图和激励表。

基本 R-S 触发器最大的优点是结构简单。它不仅可作为记忆元件独立使用，而且由于它具有直接复位、置位功能，因而被作为各种性能更完善的触发器的基本组成部分。但由于基本 R-S 触发器的输入 R、S 之间存在约束条件，且无法对其状态转换时刻进行统一定时控制，所以它的使用范围受到一定限制。

3.4.2 常用的时钟控制触发器

由上面讨论可知，基本 R-S 触发器的一个特点是触发器状态直接受输入信号 R、S 控制，一旦输入信号变化，触发器的状态便随之发生变化。但实际应用中，往往要求触发器按一定的时间节拍动作，即让输入信号的作用受到时钟脉冲(CP)的控制，为此，在触发器的输入端增加了时钟控制信号，使触发器状态的变化由时钟脉冲和输入信号共同决定。具体说，时钟脉冲确定触发器状态转换的时刻(何时转换)，输入信号确定触发器状态转换的方向(如何转换)。这种具有时钟脉冲控制端的触发器称为“钟控触发器”或者“定时触发器”。加入时钟控制信号后，通常把时钟脉冲(CP)作用前的状态称为“现态”，而把时钟脉冲(CP)作用后的状态称为触发器的“次态”。

1. 简单结构的钟控触发器

简单结构的钟控 R-S 触发器、D 触发器、J-K 触发器和 T 触发器均由 4 个与非门组成。

(1) 钟控 R-S 触发器

图 3.43(a)所示是钟控 R-S 触发器的逻辑电路，其逻辑符号如图 3.43(b)所

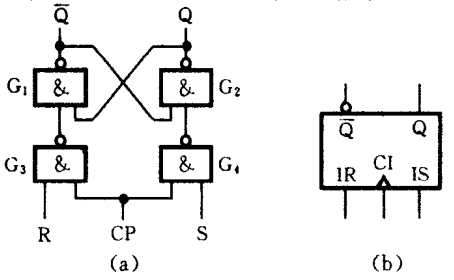


图 3.43 钟控 R-S 触发器

示。该触发器由4个与非门构成，上面的两个与非门 G_1、G_2 构成基本 R-S 触发器；下面的两个与非门 G_3、G_4 组成控制电路，通常称为控制门。

1）工作原理

钟控 R-S 触发器的工作原理如下：

① 当时钟脉冲 CP=0 时，门 G_3、G_4 被封锁。此时，不管 R、S 端的输入为何值，两个控制门的输出均为1，触发器状态保持不变。

② 当时钟脉冲 CP=1 时，控制门 G_3、G_4 被打开，这时输入端 R、S 的值可以通过控制门作用于上面的基本 R-S 触发器。

- 当 R=0，S=0 时，控制门 G_3、G_4 的输出均为1，触发器状态保持不变；
- 当 R=0，S=1 时，控制门 G_3、G_4 的输出分别为1和0，触发器状态置成1状态；
- 当 R=1，S=0 时，控制门 G_3、G_4 的输出分别为0和1，触发器状态置成0状态；
- 当 R=1，S=1 时，控制门 G_3、G_4 的输出均为0，触发器状态不确定，这是不允许的。

由此可见，这种触发器的工作过程受时钟脉冲信号 CP 和输入信号 R、S 的共同作用。当时钟脉冲信号 CP 为低电平(CP=0)时，触发器不接收输入信号，状态保持不变；当时钟脉冲信号 CP 为高电平(CP=1)时，触发器接收输入信号，状态随输入信号发生转移。

2）逻辑功能描述

由上述工作原理可知，当时钟脉冲 CP=1 时，钟控 R-S 触发器的功能表、状态表分别如表3.17和表3.18所示。

表3.17　钟控 R-S 触发器功能表

R	S	Q^{n+1}	功能说明
0	0	Q	不变
0	1	1	置1
1	0	0	置0
1	1	d	不定

表3.18　钟控 R-S 触发器状态表

现　态	次态 Q^{n+1}			
Q	RS=00	RS=01	RS=11	RS=10
0	0	1	d	0
1	1	1	d	0

表中，现态 Q 表示时钟脉冲 CP 作用前的状态，次态 Q^{n+1} 表示时钟脉冲 CP 作用后的状态。d 表示当 RS=11 时，触发器状态不确定。在钟控触发器中，时钟信号 CP 是一种固定的时间基准，通常不作为输入信号列入表中。对触发器功能进行描述时，均只考虑有时钟作用(CP=1)时的情况。

根据表3.18所示状态表，可作出钟控 R-S 触发器的状态图和次态卡诺图，分别如图3.44(a)和图3.44(b)所示。

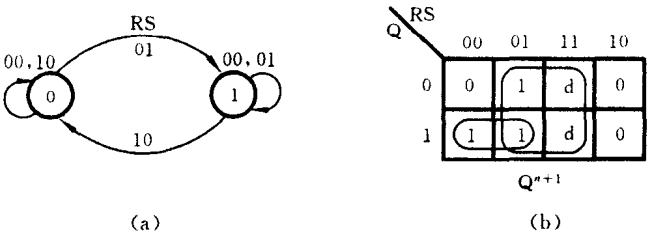


图 3.44 钟控 R-S 触发器的状态图和次态卡诺图

由图 3.44(b)所示次态卡诺图，可得出该触发器的次态方程和约束方程为

$$Q^{n+1} = S + \bar{R}Q \qquad \text{（次态方程）}$$

$$R \cdot S = 0 \qquad \text{（约束方程）}$$

钟控 R-S 触发器的激励表如表 3.19 所示。

该触发器的功能描述形式与用或非门构成的基本 R-S 触发器完全相同，但该触发器的工作过程是受时钟脉冲信号控制的，仅当时钟脉冲 CP=1 时，才能实现上述逻辑功能。此外，钟控 R-S 触发器虽然解决了对触发器工作进行定时控制的问题，而且具有结构简单的优点，但输入信号依然存在约束条件，即 R、S 不能同时为 1。

表 3.19 钟控 R-S 触发器激励表

$Q \to Q^{n+1}$		R	S
0	0	d	0
0	1	0	1
1	0	1	0
1	1	0	d

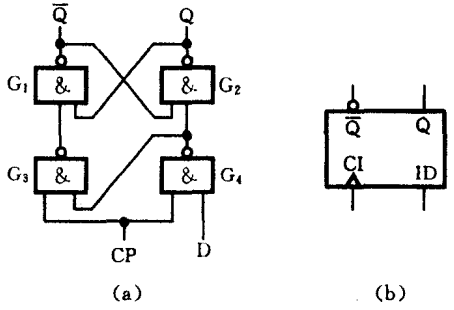


图 3.45 时钟控制 D 触发器

(2) 钟控 D 触发器

钟控 D 触发器只有一个输入端，其逻辑电路和逻辑符号如图 3.45(a)和图 3.45(b)所示。钟控 D 触发器是对钟控 R-S 触发器的控制电路稍加修改后形成的。修改后的控制电路除了实现对触发器工作的定时控制外，另一个作用是在时钟脉冲作用期间(CP=1 时)，将输入信号 D 转换成一对互补信号送至基本 R-S 触发器的两个输入端，使基本 R-S 触发器的两个输入信号只可能为 01 或者 10 两种取值，从而消除了触发器状态不确定的现象。

1) 工作原理

① 当无时钟脉冲作用(CP=0)时，门 G_3、G_4 被封锁。此时，不管 D 端为何值，两个控制门的输出均为 1，触发器状态保持不变。

② 当有时钟脉冲作用(CP=1)时，若 D=0，则门 G_4 输出为 1，门 G_3 输出为 0，触发器状态被置 0；若 D=1，则门 G_4 输出为 0，门 G_3 输出为 1，触发器状态被置 1。

2)逻辑功能描述

由工作原理可知，在 CP=1 时，D 触发器状态的变化仅取决于输入信号 D，而与现态无关。其次态方程为

$$Q^{n+1}=D$$

钟控 D 触发器的功能表、状态表和激励表分别如表 3.20、表 3.21 和表 3.22 所示，状态图如图 3.46 所示。

表 3.20　D 触发器的功能表

D	Q^{n+1}	功能说明
0	0	置 0
1	1	置 1

表 3.21　钟控 D 触发器状态表

现态 Q	次态 Q^{n+1}	
	D=0	D=1
0	0	1
1	0	1

表 3.22　D 触发器的激励表

Q	→	Q^{n+1}	D
0		0	0
0		1	1
1		0	0
1		1	1

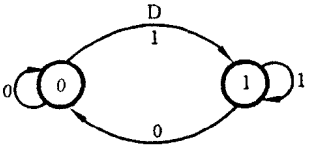
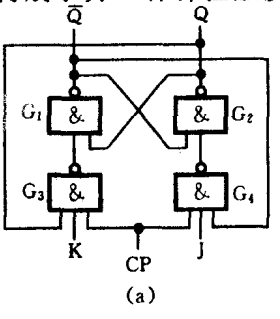


图 3.46　D 触发器的状态图

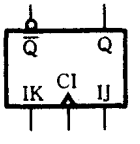
由于钟控 D 触发器在时钟脉冲作用后的次态和输入 D 的值一致，故有时又称为锁存器。

(3) 钟控 J-K 触发器

将钟控 R-S 触发器改进成如图 3.47(a)所示的形式，即增加两条反馈线，将触发器的输出 Q 和 $\overline{Q}$ 交叉反馈到两个控制门的输入端，并把原来的输入端 S 改为 J，R 改为 K，便构成了另一种钟控触发器，称为 J-K 触发器，其逻辑符号如图



(a)



(b)

图 3.47　J-K 触发器的逻辑电路和逻辑符号

3.47(b)所示。J-K 触发器利用触发器两个输出端信号始终互补的特点，有效地解决了在时钟脉冲作用期间两个输入同时为 1 将导致触发器状态不确定的问题。

1) 工作原理

① 在没有时钟脉冲作用(CP=0)时，无论输入端 J 和 K 怎样变化，控制门 G_3、G_4的输出均为 1，触发器保持原来状态不变。

② 在时钟脉冲作用(CP=1)时，可分为 4 种情况。

● 当输入 J=0，K=0 时，不管触发器原来处于何种状态，控制门 G_3和 G_4的输出均为 1，触发器状态保持不变。

● 当输入 J=0，K=1 时，若原来处于 0 状态，则控制门 G_3和 G_4输出均为 1，触发器保持 0 状态不变；若原来处于 1 状态，则门 G_3输出为 0，门 G_4输出为 1，触发器状态置成 0。即输入 JK=01 时，触发器次态一定为 0 状态。

● 当输入 J=1，K=0 时，若原来处于 0 状态，则控制门 G_3输出为 1，门 G_4输出为 0，触发器状态置成 1；若原来处于 1 状态，则门 G_3和 G_4输出均为 1，触发器保持 1 状态不变。即输入 JK=10 时，触发器次态一定为 1 状态。

● 当输入 J=1，K=1 时，若原来处于 0 状态，则门 G_3输出为 1，门 G_4输出为 0，触发器置成 1 状态；若原来处于 1 状态，则门 G_3输出为 0，门 G_4输出为 1，触发器置成 0 状态。即输入 JK=11 时，触发器的次态与现态相反。

2) 逻辑功能描述

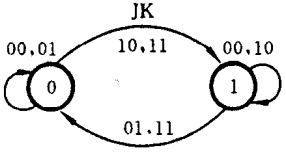
根据上述工作原理，可归纳出 J-K 触发器在时钟脉冲作用下(CP=1)的功能表和状态表分别如表 3.23 和表 3.24 所示，相应的状态图和次态卡诺图分别如图 3.48(a)和图 3.48 (b)所示。

表 3.23 钟控 J-K 触发器功能表

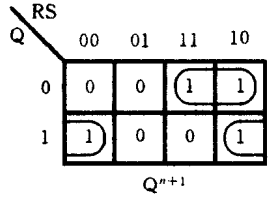
J	K	Q^{n+1}	功能说明
0	0	Q	不变
0	1	0	置 0
1	0	1	置 1
1	1	$\bar{Q}$	翻转

表 3.24 钟控 J-K 触发器状态表

现态	次态 Q^{n+1}			
Q	JK=00	JK=01	JK=11	JK=10
0	0	0	1	1
1	1	0	0	1



(a)



(b)

图 3.48 钟控 J-K 触发器的状态图和次态卡诺图

根据次态卡诺图可得出 J-K 触发器的次态方程为

$$Q^{n+1}=J\overline{Q}+\overline{K}Q$$

J-K 触发器在时钟脉冲作用下(CP=1)的激励表如表 3.25 所示。

表 3.25　J-K 触发器的激励表

$Q \rightarrow Q^{n+1}$		J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

J-K 触发器输入 J、K 的取值没有约束条件，无论 J、K 取何值，在时钟脉冲作用下都有确定的次态，因此，该触发器具有较强的逻辑功能。

(4) 钟控 T 触发器

如果把钟控 J-K 触发器的两个输入端 J 和 K 连接起来，并用符号 T 表示，就构成了钟控 T 触发器。图 3.49(a)所示的是钟控 T 触发器的逻辑电路，其逻辑符号如图 3.49(b)所示。

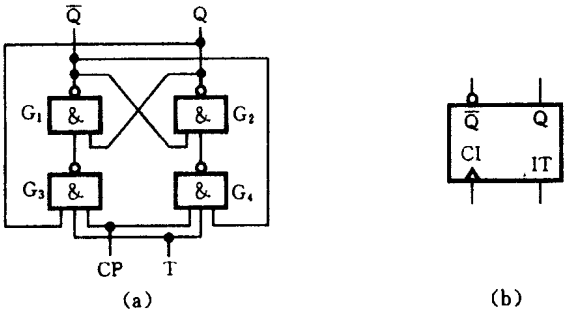


图 3.49　钟控 T 触发器的逻辑电路和逻辑符号

1) 工作原理

钟控 T 触发器的工作原理如下。

① 当无时钟脉冲作用(CP=0)时，门 G_3、G_4 被封锁。此时，不管 T 端为何值，两个控制门的输出均为 1，触发器状态保持不变。

② 当有时钟脉冲作用(CP=1)时，可分为两种情况。

● 当 T=0 时，门 G_3、门 G_4 的输出均为 1，触发器状态保持不变；

● 当 T=1 时，与现态相关。若现态 Q 为 0，则门 G_3 输出为 1，门 G_4 输出为 0，触发器状态被置 1；若现态 Q 为 1，则门 G_3 输出为 0，门 G_4 输出为 1，触发器状态被置 0。

归纳起来，当 T=0 时，在时钟作用下触发器状态保持不变；当 T=1 时，在时钟作用下触发器状态发生翻转。

2) 逻辑功能描述

钟控 T 触发器在时钟脉冲作用下(CP=1)的功能表、状态表和激励表分别如

表 3.26、表 3.27 和表 3.28 所示，相应的状态图如图 3.50 所示。

表 3.26 T 触发器功能表

T	Q^{n+1}	功能说明
0	Q	不变
1	$\overline{Q}$	翻转

表 3.27 T 触发器状态表

现 态	次态 Q^{n+1}	
Q	T=0	T=1
0	0	1
1	1	0

表 3.28 T 触发器激励表

Q →	Q^{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

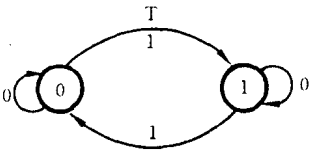


图 3.50 T 触发器状态图

根据 T 触发器的状态表，可直接得出 T 触发器的次态方程：

$$Q^{n+1}=T\overline{Q}+\overline{T}Q=T\oplus Q$$

钟控 T 触发器当 T=1 时，只要有时钟脉冲作用，触发器状态就翻转，或由 1 变为 0 或由 0 变为 1，相当于一位二进制计数器，所以又将 T 触发器称为计数触发器。

上述简单结构钟控触发器的共同特点是，当时钟控制信号为低电平(CP=0)时，触发器保持原来状态不变；当时钟控制信号为高电平(CP=1)时，触发器在输入信号作用下发生状态变化。换而言之，触发器状态转移是被控制在一个约定的时间间隔内，而不是控制在某一时刻进行，触发器的这种钟控方式被称作电位触发方式。

电位触发方式的钟控触发器存在一个共同的问题，就是可能出现“空翻”现象。所谓“空翻”是指在同一个时钟脉冲作用期间触发器状态发生两次或两次以上变化的现象。引起空翻的原因是在时钟脉冲为高电平期间，输入信号的变化直接控制着触发器状态的变化。具体说，当时钟 CP=1 时，如果输入信号发生变化，则触发器状态会跟着发生变化，从而使得一个时钟脉冲作用期间引起多次翻转。“空翻”将造成状态的不确定和系统工作的混乱，这是不允许的。如果要使这种触发器在每个时钟脉冲作用期间仅发生一次翻转，则对时钟信号的控制电平宽度要求极其苛刻。这一不足，使这种触发器的应用受到一定限制。

2. 其他结构的钟控触发器

为了克服简单结构钟控触发器所存在的“空翻”现象，必须对控制电路的结构

进行改进，将触发器的翻转控制在某一时刻完成。为此引出了主从结构、维持阻塞结构等不同类型的钟控触发器。

(1) 主从钟控触发器

主从结构的钟控触发器采用具有存储功能的控制电路，避免了“空翻”现象。下面以主从 R-S 触发器和主从 J-K 触发器为例进行介绍。

1) 主从 R-S 触发器

主从 R-S 触发器由两个简单结构的钟控 R-S 触发器组成，一个称为主触发器，另一个称为从触发器。图 3.51(a)所示是主从 R-S 触发器的逻辑电路，其逻辑符号如图 3.51(b)所示。

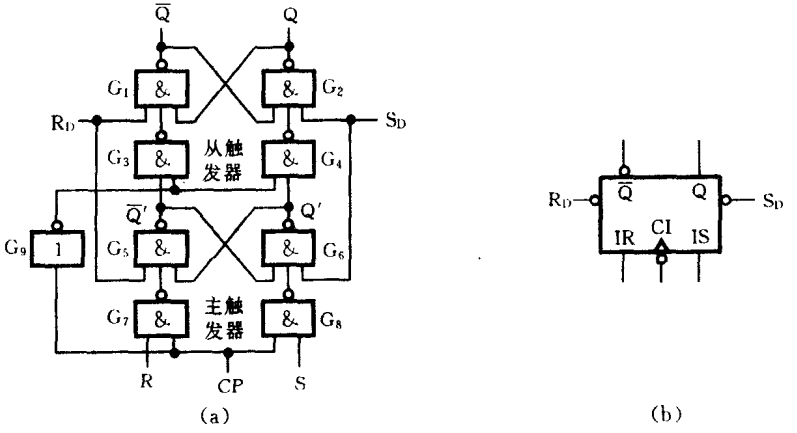


图 3.51　主从 R-S 触发器的逻辑电路和逻辑符号

由图 3.51(a)可知，主、从两个触发器的时钟脉冲是反相的，时钟脉冲 CP 作为主触发器的控制信号，经反相后的$\overline{CP}$作为从触发器的控制信号。输入信号 R、S 送至主触发器输入端，主触发器的状态 Q′和 $\overline{Q}'$作为从触发器的输入，从触发器的输出 Q 和 $\overline{Q}$ 作为整个主从触发器的状态输出。图中的 R_D和 S_D分别为直接清 0 端和直接置 1 端(有时又称为异步清 0 端和异步置 1 端)，低电平有效，平时为高电平。

该触发器的工作原理如下。

当时钟脉冲 CP=1 时，控制门 G_7、G_8被打开，主触发器的状态取决于输入 R、S 的值，逻辑功能与前述简单结构的 R-S 触发器完全相同；而对于从触发器来说，由于此时$\overline{CP}=0$，控制门 G_3、G_4被封锁，故从触发器状态不受主触发器的状态变化的影响，即整个主从触发器状态保持不变。

当时钟脉冲 CP 由 1 变为 0 时，由于 CP=0，控制门 G_7、G_8被封锁，故主触发器的状态不再受输入 R、S 的影响，即主触发器状态保持不变；而对于从触发器来

说，由于此时$\overline{CP}=1$，控制门G_3、G_4被打开，所以主触发器的状态通过控制门作用于从触发器，使从触发器状态与主触发器状态相同（若$Q'=0$、$\overline{Q}'=1$，则$Q=0$、$\overline{Q}=1$；反之若$Q'=1$、$\overline{Q}'=0$，则$Q=1$、$\overline{Q}=0$）。换而言之，当时钟脉冲CP由1变为0时，将主触发器状态转为整个主从触发器状态。

主从R-S触发器的工作波形如图3.52所示。

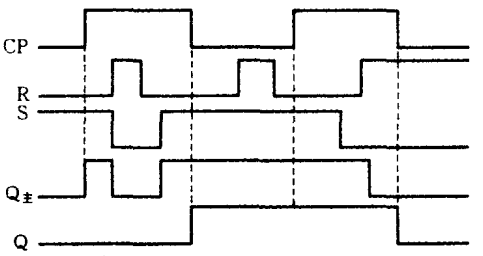


图3.52 主从R-S触发器的工作波形

由上述工作原理可知，主从R-S触发器具有如下特点：

① 触发器状态的变化发生在时钟脉冲CP由1变为0的时刻，因为在CP=0期间主触发器被封锁，其状态不再受输入R、S的影响，因此，不会引起触发器状态发生两次以上翻转，从而克服了“空翻”现象。

② 触发器的状态取决于CP由1变为0时刻主触发器的状态，而主触发器的状态在CP=1期间是随输入R、S变化的，所以触发器的状态实际上取决于CP由1变为0之前输入R、S的值。

③ 主从R-S触发器的逻辑功能与前述简单结构的R-S触发器完全相同。其次态方程和约束方程为

$$Q^{n+1}=S+\overline{R}Q \qquad \text{（次态方程）}$$

$$R\cdot S=0 \qquad \text{（约束方程）}$$

由于主从R-S触发器状态的变化发生在时钟脉冲CP的下降沿（1→0）时刻，故通常称为下降沿触发。图3.51(b)所示逻辑符号中，时钟端的小圆圈表示主从R-S触发器状态的改变是在时钟脉冲的下降沿发生的。

2）主从J-K触发器

主从J-K触发器是对主从R-S触发器稍加修改后形成的，其逻辑电路和逻辑符号分别如图3.53(a)和图3.53 (b)所示。

由图3.53(a)可知，主从J-K触发器通过将输出Q和$\overline{Q}$交叉反馈到两个控制门的输入端，克服了主从R-S触发器两个输入不能同时为1的约束条件。此外，修改后实际上使原主从R-S触发器的$R=KQ$、$S=J\overline{Q}$，将其代入主从R-S触发器的次态方程$Q^{n+1}=S+\overline{R}Q$，即可得到主从J-K触发器的次态方程：

$$\begin{aligned}Q^{n+1} &= J\overline{Q}+\overline{KQ}Q\\ &= J\overline{Q}+(\overline{K}+\overline{Q})Q\\ &= J\overline{Q}+\overline{K}Q\end{aligned}$$

主从J-K触发器的逻辑功能与简单结构的J-K触发器完全相同，但它克服了

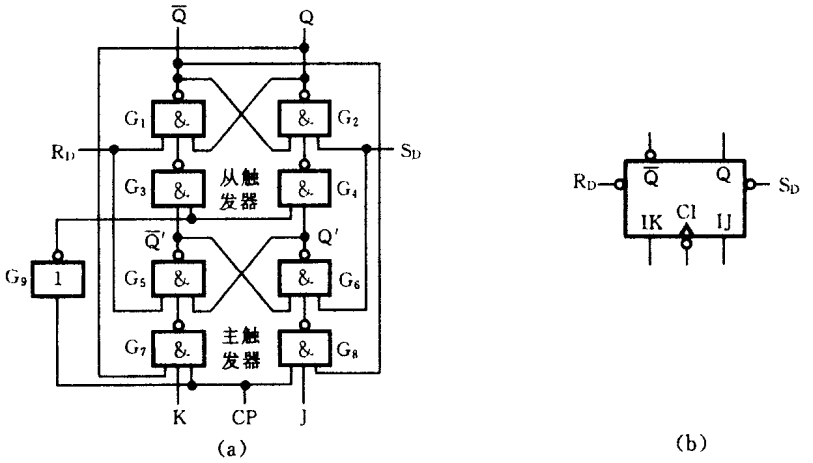


图 3.53　主从 J-K 触发器的逻辑电路和逻辑符号

"空翻"现象。

值得指出的是，主从 J-K 触发器存在"一次翻转"现象。所谓"一次翻转"是指在时钟脉冲作用(CP＝1)期间，主触发器的状态只能根据输入信号的变化改变一次。即主触发器在接收输入信号发生一次翻转后，其状态保持不变，不再受输入 J、K 变化的影响。"一次翻转"与前面所述的"空翻"是两种不同的现象。"一次翻转"现象可能导致触发器的状态转移与触发器的逻辑功能不一致，显然这是不允许的。

主从 J-K 触发器的工作波形如图 3.54所示。

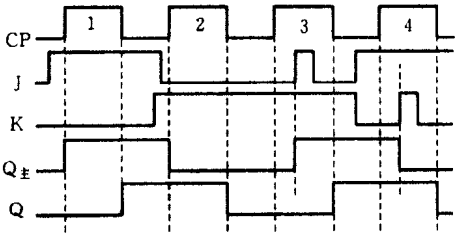


图 3.54　主从 R-S 触发器的工作波形

图 3.54 所示波形图表明了触发器所存在的"一次翻转"现象，工作波形分析如下。

① 在 CP_1 和 CP_2 期间，触发器处于正常工作状态。CP_1＝1 期间，由于 J＝1、K＝0，使主触发器状态为 1，所以在 CP_1 由 1 变为 0 时，触发器翻转为 1 状态；CP_2＝1期间，由于 J＝0、K＝1，使主触发器状态为 0，所以在 CP_2 由 1 变为 0 时，触发器翻转为 0 状态。

② 假定在 CP_3 期间 J 端产生一个正向干扰脉冲，情况如何呢？可结合图 3.53(a)所示主从 J-K 触发器的逻辑图进行分析。干扰脉冲出现前，图 3.53(a)中的主触发器和从触发器都处于 0 状态，即 $Q＝Q'＝0$、$\overline{Q}＝\overline{Q}'＝1$。当干扰脉冲出现(J 由 0→1)时，与非门 G_8 的输入均为 1，输出变为 0，使得主触发器状态 $Q'＝1$、$\overline{Q}'＝0$，即干扰信号的出现使主触发器状态由 0 变为 1。当干扰信号消失时，由于 $\overline{Q}'＝0$

已将与非门 G_6 封锁，G_8 输出的变化不会影响 Q' 的状态，即 J 端干扰信号的消失不能使 Q' 的状态恢复到 0(这就是一次翻转特性)。因此，CP_3 由 1 变为 0 时，使得触发器状态为 $Q=1$。如果 J 端没有正向干扰脉冲出现，根据 $J=0$、$K=1$ 的输入条件，触发器的正常状态应为 $Q=0$。类似地，在图 3.54 所示波形图中，在 CP_4 期间 K 端产生的正向干扰脉冲将使触发器变为 0 状态，而不是正常的 1 状态。

由此可见，因为主从 J-K 触发器存在一次翻转现象，所以当输入端 J、K 出现干扰信号时，可能破坏触发器的正常逻辑功能。为了使主从 J-K 触发器能正常实现预定的逻辑功能，要求它在时钟脉冲作用($CP=1$)期间输入 J、K 不能发生变化，这就降低了其抗干扰能力。这种触发器一般采用窄脉冲作为触发脉冲。

(2) 维持-阻塞钟控触发器

为了既能克服简单结构钟控触发器的“空翻”现象，又能提高触发器的抗干扰能力，引出了边沿触发器。边沿触发器仅仅在时钟脉冲 CP 的上升沿或下降沿时刻响应输入信号，从而大大提高了触发器的抗干扰能力。维持-阻塞触发器是一种广泛使用的边沿触发器。下面以维持-阻塞 D 触发器为例进行介绍。

典型维持阻塞 D 触发器的逻辑电路和逻辑符号分别如图 3.55(a)和图 3.55(b)所示。图中，D 称为数据输入端；R_D 和 S_D 分别称为直接置“0”端和直接置“1”端，它们均为低电平有效，即在不作直接置“0”和置“1”操作时，保持为高电平。

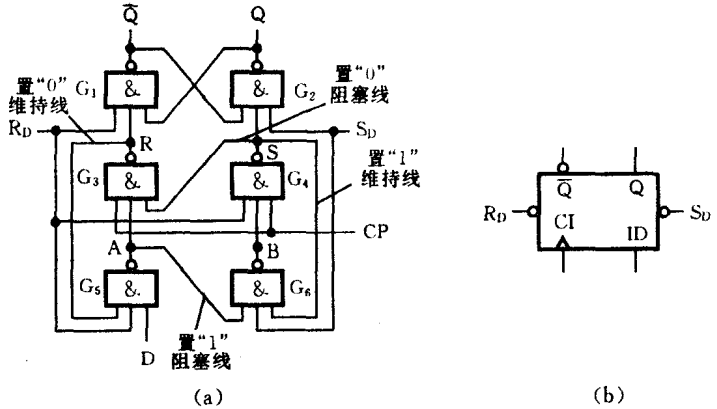


图 3.55 维持阻塞 D 触发器的逻辑电路和逻辑符号

维持阻塞 D 触发器在简单 D 触发器的基础上增加了两个逻辑门 G_5、G_6，并安排了置“0”、置“1”维持线和置“0”、置“1”阻塞线，正是由于这 4 条线的作用，使得该触发器仅在时钟脉冲 CP 由 0 变为 1 的上升沿时刻才根据 D 端的信号发生状态转移，而在其余时间触发器状态均保持不变。下面分 3 种情况对触发器的工作原理进行讨论。

① 时钟脉冲 CP=0，触发器状态保持不变。此时，G_3和 G_4被封锁，G_3的输出 R 和 G_4的输出 S 均为 1，无论 D 的值怎样变化，触发器都保持原来状态不变；此外，由于 R=1 反馈到 G_5的输入端，S=1 反馈到 G_6的输入端，使这两个门打开，故可以接收输入信号 D，使得 G_5的输出 $A=\overline{D}$，G_6的输出 B=D。

② 时钟脉冲 CP 由 0 变为 1 时，使触发器发生状态变化。此时 G_3和 G_4被打开，它们的输出 R、S 由 G_5、G_6的输出 A、B 决定，此时 $R=\overline{A}=\overline{\overline{D}}=D$，$S=\overline{B}=\overline{D}$。若 D=0，则 R=0、S=1，触发器的状态为 0；若 D=1，则 R=1、S=0，触发器的状态为 1，即作用的结果使触发器状态与 D 相同。

③ 触发器被触发后，在时钟脉冲 CP=1 时，不受输入影响，维持原状态不变。此时 G_3和 G_4被打开，它们的输出 R、S 是互补的，即 R=0、S=1 或者 R=1、S=0。

若 R=0、S=1(触发器处于 0 状态)，则此时 R=0 有 3 方面的作用：

● 继续将触发器状态置"0"，即使 $\overline{Q}=1$，Q=0；

● 通过置"0"维持线反馈到 G_5输入，使 G_5的输出 A=1，这样就维持了 R=0，即维持了触发器的置 0 功能；

● 使 G_5的输出 A=1 后，A 点的 1 信号通过置"1"阻塞线，送至 G_6的输入，使 G_6的输出 B=0，G_4的输出 S=1，阻止了触发器置 1。

若 R=1、S=0(触发器处于 1 状态)，则此时 S=0 同样有 3 方面的作用：

● 继续将触发器状态置"1"，即 $\overline{Q}=0$，Q=1；

● 通过置"1"维持线反馈到 G_6输入，使 G_6的输出 B=1，G_4的输出 S=0，维持了触发器的置 1 功能；

● 通过置"0"阻塞线，送至 G_3的输入，保持 R=1，阻止了触发器置 0。

由上述分析可知，由于维持阻塞线路的作用，使触发器在时钟脉冲的上升边沿将 D 输入端的数据可靠地转换成触发器状态，而在上升沿过后的时钟脉冲期间，不论 D 的值如何变化，触发器的状态始终以时钟脉冲上升沿时所采样的值为准。由于是在时钟脉冲的上升边沿采样 D 输入端的数据，所以要求输入 D 在时钟脉冲 CP 由 0 变为 1 之前将数据准备好。

维持阻塞 D 触发器逻辑功能与前述 D 触发器的逻辑功能完全相同。实际中使用的维持阻塞 D 触发器有时具有几个 D 输入端，此时，各输入之间是相与的关系。例如，当有 3 个输入端 D_1、D_2和 D_3时，其次态方程是

$$Q^{n+1}=D_1 D_2 D_3$$

维持阻塞 D 触发器不仅克服了空翻现象，而且由于是边沿触发，抗干扰能力强。因而应用十分广泛。

实际使用的集成触发器还有维持阻塞 R-S 触发器，下降沿触发的 J-K 触发器等不同类型。

由于用上述性能优越的J-K触发器、维持阻塞D触发器可以十分方便地转换成T触发器，所以集成电路厂家很少生产专门的T触发器产品，而一般由其他类型的触发器转换而成。T触发器的类型与用于实现其功能的触发器类型相关，图3.56(a)和(b)所示逻辑符号分别表示用J-K触发器和D触发器构成的T触发器。图3.56(a)表示下降沿触发的T触发器，图3.56(b)表示上升沿触发的T触发器。

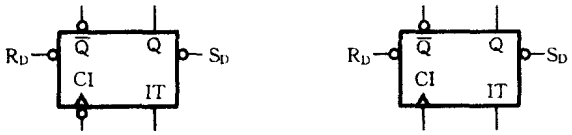


图3.56 T触发器逻辑符号

集成触发器的性能参数通常分为直流参数和开关参数两大类。直流参数包括电源电流，低电平输入电流，高电平输入电流，输出高电平、输出低电平，以及扇出系数等。开关参数有最高时钟频率、时钟信号的延迟时间和直接置0(R_D)或置1(S_D)端的延迟时间等。具体可查阅有关集成电路手册。

*3.4.3 不同类型时钟控制触发器的相互转换

上面介绍了4种不同类型的时钟控制触发器，这些触发器之间可以进行逻辑功能的转换。一旦掌握了转换方法，便可以灵活地运用现有的器件组成各种功能的逻辑电路。由于实际中最常用的现成产品是J-K触发器和D触发器，因此，下面介绍如何把这两种触发器转换成其他类型的触发器。

1. 转换方法

一般来说，在原触发器的输入端加上一定的转换逻辑电路，就可以构成具有新的逻辑功能的触发器，如图3.57所示。因此，转换的关键就是设法求出转换逻辑电路。

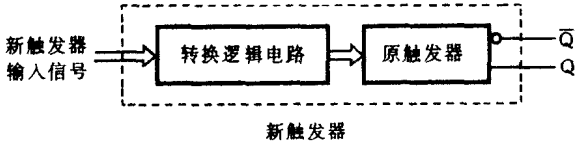


图3.57 触发器转换示意图

由于触发器的逻辑功能有多种描述方法，所以触发器之间的转换也就有各种不同的方法，如直接观察分析法、次态方程联立法、功能表与激励表联立法等。下

面将以次态方程联立法确定各触发器之间的转换逻辑电路。

所谓次态方程联立法，就是将原触发器和新触发器的次态方程联系起来，找出原输入信号与新输入信号及现态之间的函数关系。这种方法要运用逻辑代数的公理、定理和规则进行推演，需要一定的技巧。

2. J-K 触发器转换成 D、T、R-S 触发器

(1) J-K 触发器转换成 D 触发器

实现 J-K 触发器到 D 触发器的转换，要求确定的函数关系是

$$J=f_1(D,Q) \qquad K=f_2(D,Q)$$

已知 J-K 触发器的次态方程为

$$Q^{n+1}=J\overline{Q}+\overline{K}Q \tag{3-1}$$

D 触发器的次态方程为

$$Q^{n+1}=D=D(\overline{Q}+Q)=D\overline{Q}+DQ \tag{3-2}$$

比较方程(3-1)和(3-2)可得

$$J=D \qquad K=\overline{D}$$

由此可画出由 J-K 触发器转换而成的 D 触发器的逻辑电路，如图 3.58 所示。

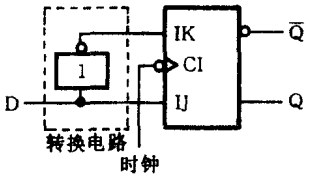


图 3.58　J-K 触发器转换成 D 触发器的逻辑电路

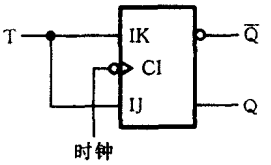


图 3.59　J-K 触发器转换成 T 触发器的逻辑电路

(2) J-K 触发器转换成 T 触发器

J-K 触发器转换成 T 触发器要求确定的函数关系是

$$J=f_1(T,Q) \qquad K=f_2(T,Q)$$

已知 J-K 触发器的次态方程为

$$Q^{n+1}=J\overline{Q}+\overline{K}Q \tag{3-3}$$

T 触发器的次态方程为

$$Q^{n+1}=T\overline{Q}+\overline{T}Q \tag{3-4}$$

比较方程(3-3)和(3-4)，可得

$$J=T \qquad K=T$$

据此可得到由 J-K 触发器转换而成的 T 触发器的逻辑电路，如图 3.59 所示。

(3) J-K 触发器转换成 R-S 触发器

将 J-K 触发器转换成 R-S 触发器要求确定的函数关系是

$$J=f_1(R,S,Q) \qquad K=f_2(R,S,Q)$$

已知 J-K 触发器的次态方程为

$$Q^{n+1}=J\overline{Q}+\overline{K}Q \tag{3-5}$$

R-S 触发器的次态方程为

$$\begin{aligned} Q^{n+1} &= S+\overline{R}Q && \text{(约束条件 RS=0)} \\ &= S(\overline{Q}+Q)+\overline{R}Q \\ &= S\overline{Q}+SQ+\overline{R}Q \\ &= S\overline{Q}+\overline{R}Q+SQ(\overline{R}+R) \\ &= S\overline{Q}+\overline{R}Q+S\overline{R}Q+SRQ \\ &= S\overline{Q}+\overline{R}Q+S\overline{R}Q && \text{(去掉无关项 SRQ)} \\ &= S\overline{Q}+\overline{R}Q && (3\text{-}6) \end{aligned}$$

比较方程(3-5)和(3-6)可以得到

$$J=S \qquad K=R$$

据此可得到由 J-K 触发器转换而成的 R-S 触发器的逻辑电路，如图 3.60 所示。

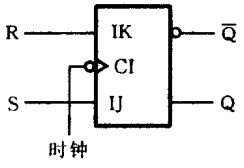


图 3.60 J-K 触发器转换成 R-S 触发器的逻辑电路

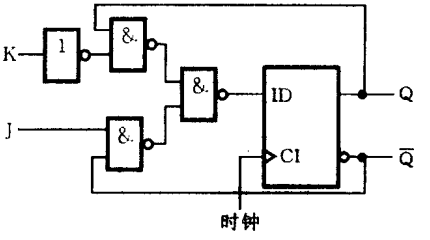


图 3.61 D 触发器转换成 J-K 触发器的逻辑电路

3. D 触发器转换成 J-K、T、R-S 触发器

(1) D 触发器转换成 J-K 触发器

将 D 触发器转换成 J-K 触发器要求确定的函数关系是

$$D=f(J,K,Q)$$

已知 D 触发器的次态方程为

$$Q^{n+1}=D \tag{3-7}$$

J-K 触发器的次态方程为

$$Q^{n+1}=J\overline{Q}+\overline{K}Q \tag{3-8}$$

比较方程(3-7)和(3-8)，可得

$$D=J\overline{Q}+\overline{K}Q$$

据此可得到由 D 触发器转换而成的 J-K 触发器的逻辑电路，如图 3.61 所示。

(2) D 触发器转换成 T 触发器

D 触发器转换成 T 触发器所要求确定的函数关系是

$$D=f(T,Q)$$

根据 D 触发器的次态方程 $Q^{n+1}=D$ 和 T 触发器的次态方程 $Q^{n+1}=T\overline{Q}+\overline{T}Q$，可以得到函数关系式

$$D=T\overline{Q}+\overline{T}Q=T\oplus Q$$

据此可得到由 D 触发器转换而成的 T 触发器的逻辑电路，如图 3.62 所示。

(3) D 触发器转换成 R-S 触发器

D 触发器转换成 R-S 触发器要求确定的函数关系是

$$D=f(R,S,Q)$$

根据 D 触发器的次态方程 $Q^{n+1}=D$ 和 R-S 触发器的次态方程 $Q^{n+1}=S+\overline{R}Q$ 可以得到函数关系式 $D=S+\overline{R}Q=\overline{\overline{S}(R+\overline{Q})}$，据此可得到由 D 触发器转换而成的 R-S 触发器的逻辑电路，如图 3.63 所示。

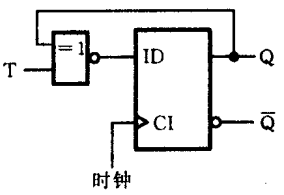


图 3.62　D 触发器转换成 T 触发器的逻辑电路

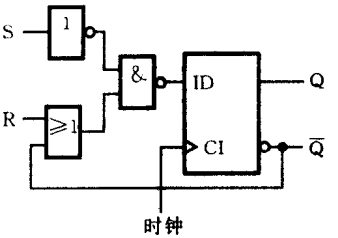


图 3.63　D 触发器转换成 R-S 触发器的逻辑电路

上面讨论了由 J-K 触发器和 D 触发器转换成其他类型触发器的情况。用类似的方法，也可以实现其他触发器之间的转换。但是最有实际意义的转换是把 J-K 触发器转换成 D、T、R-S 触发器以及把 D 触发器转换成 T 触发器。这一方面是因为 J-K 触发器和 D 触发器的应用最多，另一方面是因为转换逻辑电路简单，实际使用方便。

最后要指出的是，转换后的新触发器，其触发方式与原触发器相同。

习　题　三

3.1　根据所采用的半导体器件不同，集成电路可分为哪两大类？各自的主要优缺点是什么？

3.2　简述晶体二极管的静态特性？

3.3 晶体二极管的开关速度主要取决于什么？

3.4 数字电路中，晶体三极管一般工作在什么状态下？

3.5 晶体三极管的开关速度取决于哪些因素？

3.6 TTL 与非门有哪些主要性能参数？

3.7 OC 门和 TS 门的结构与一般 TTL 与非门有何不同？各有何主要应用？

3.8 有两个相同型号的 TTL 与非门，对它们进行测试的结果如下：

(1) 甲的开门电平为 1.4 V，乙的开门电平为 1.5 V；

(2) 甲的关门电平为 1.0 V，乙的关门电平为 0.9 V。

试问在输入相同高电平时，哪个抗干扰能力强？在输入相同低电平时，哪个抗干扰能力强？

3.9 图 3.64(a)所示为三态门组成的总线换向开关电路，其中，A、B 为信号输入端，分别送两个频率不同的信号；EN 为换向控制端，输入信号和控制电平波形如图(b)所示。试画出 Y_1、Y_2 的波形。

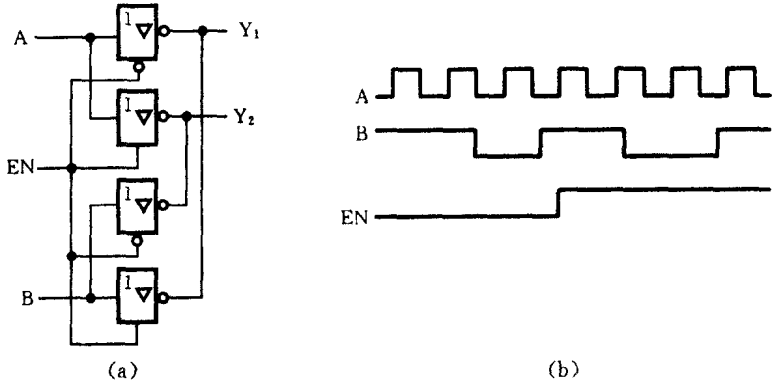


图 3.64 逻辑电路及有关信号波形

3.10 试画出实现如下功能的 CMOS 电路图：

(1) $F=\overline{A \cdot B \cdot C}$

(2) $F=A+B$

(3) $F=\overline{AB+CD}$

3.11 试指出下列 5 种逻辑门中哪几种的输出可以并联使用：

(1) TTL 集电极开路门；

(2) 采用推拉式输出的一般 TTL 与非门；

(3) TTL 三态输出门；

(4) 普通 CMOS 门；

(5) CMOS 三态输出门。

3.12　用与非门构成的基本 R-S 触发器和用或非门构成的基本 R-S 触发器在逻辑功能上有什么区别？

3.13　在图 3.65(a)所示的 D 触发器电路中，若输入端 D 的波形如图 3.65(b)所示，试画出输出端 Q 的波形(设触发器初态为 0)。

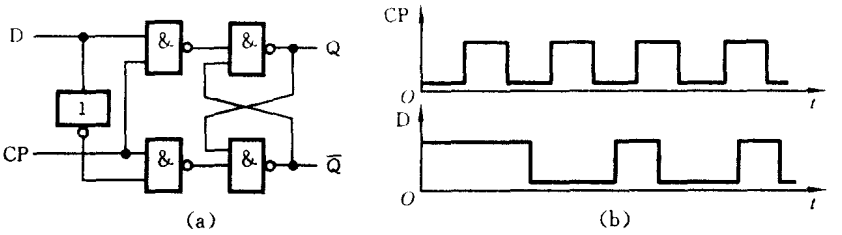


图 3.65　逻辑电路及有关波形

3.14　已知输入信号 A 和 B 的波形如图 3.66(a)所示，试画出图 3.66(b)、(c)中两个触发器 Q 端的输出波形，设触发器初态为 0。

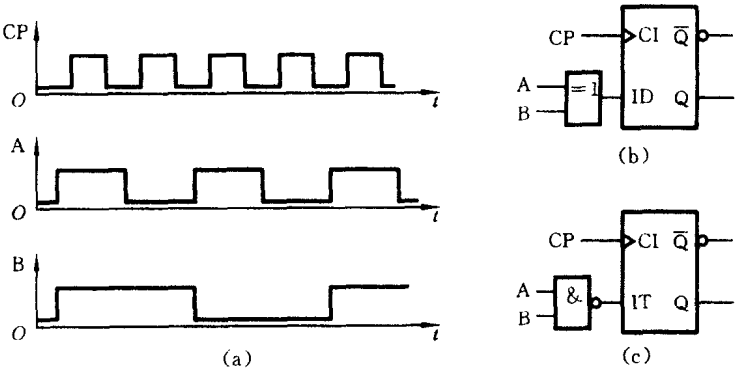


图 3.66　信号波形及电路

3.15　设图 3.67(a)所示电路中的触发器为主从 J-K 触发器，其初始状态 $Q_1=Q_2=0$，输入信号及 CP 端的波形如图 3.67(b)所示，试画出 Q_1、Q_2 的波形图。

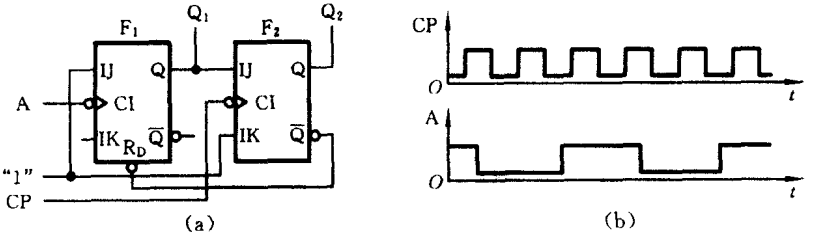


图 3.67　逻辑电路及有关波形



组合逻辑电路

数字系统中的逻辑电路按其是否具有记忆功能分为组合逻辑电路和时序逻辑电路两大类型。

组合逻辑电路是指电路在任何时刻产生的稳定输出值仅仅取决于该时刻各输入值的组合，而与过去的输入值无关。组合逻辑电路的一般结构如图 4.1 所示。



图 4.1　组合电路的一般结构

图中，$X_1, X_2, \cdots, X_n$ 是电路的 n 个输入信号，$F_1, F_2, \cdots, F_m$ 是电路的 m 个输出信号。输出信号是输入信号的函数，表示为

$$F_i = f_i(X_1, X_2, \cdots, X_n) \qquad i = 1, 2, \cdots, m$$

从电路结构看，组合电路具有两个特点：

① 由逻辑门电路组成，不包含任何记忆元件；

② 信号是单向传输的，不存在任何反馈回路。

组合逻辑电路不但能独立完成各种复杂的逻辑功能，而且是时序逻辑电路的组成部分，它在数字系统中的应用十分广泛。

本章主要讨论组合逻辑电路分析和设计的基本方法，在此基础上介绍组合逻辑电路设计中几个常见的实际问题及其处理方法，并对组合逻辑电路中的竞争险象问题作一般讨论。

4.1　组合逻辑电路分析

所谓逻辑电路分析，是指对一个给定的逻辑电路，找出其输出与输入之间的逻

辑关系。通过分析，不仅可以了解给定逻辑电路的功能，同时还能评价其设计方案的优劣，以便吸取优秀的设计思想、改进和完善不合理方案以及更换逻辑电路的某些组件等。由此可见，逻辑电路分析是研究数字系统的一种基本技能。

4.1.1 分析方法概述

组合逻辑电路分析的一般步骤如下。

1. 根据逻辑电路图写出输出函数表达式

为了确保写出的逻辑表达式正确无误，一般是在认清电路中所有逻辑器件和相互连线的基础上，从输入端开始往输出端逐级推导，直至得到所有与输入变量相关的输出函数表达式为止。

2. 化简输出函数表达式

根据给定逻辑电路写出的输出函数表达式不一定是最简表达式，为了简单、清晰地反映输入/输出之间的逻辑关系，应对逻辑表达式进行化简。此外，描述一个电路功能的逻辑表达式是否达到最简，是评价该电路经济技术指标是否良好的依据。

3. 列出输出函数真值表

根据输出函数最简表达式，列出输出函数真值表。真值表详尽地给出了输入/输出取值关系，它通过逻辑值直观地描述了电路的逻辑功能。

4. 功能评述

根据真值表和化简后的函数表达式，概括出对电路逻辑功能的文字描述，并对原电路的设计方案进行评价，必要时提出改进意见和改进方案。

以上分析步骤是就一般情况而言的，实际应用中可根据问题的复杂程度和具体要求对上述步骤进行适当取舍。下面举例说明组合逻辑电路的分析过程。

4.1.2 分析举例

例 4.1 分析图 4.2(a)所示组合逻辑电路。

解 (1) 根据逻辑电路图写出输出函数表达式

根据电路中各逻辑门的功能，从输入端开始逐级写出函数表达式如下：

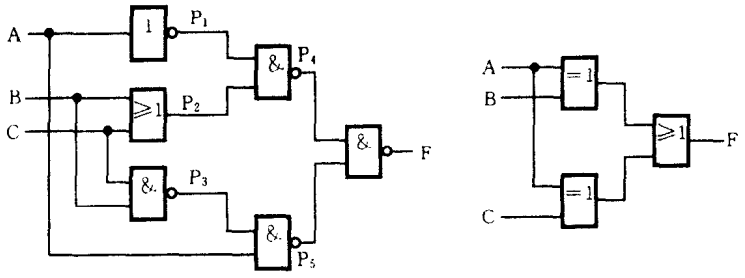


图 4.2　逻辑电路

$$P_1=\overline{A} \qquad P_2=B+C$$

$$P_3=\overline{BC} \qquad P_4=\overline{P_1\cdot P_2}=\overline{\overline{A}(B+C)}$$

$$P_5=\overline{A\cdot P_3}=\overline{A\,\overline{BC}} \qquad F=\overline{P_4\cdot P_5}=\overline{\overline{\overline{A}(B+C)}\cdot A\,\overline{BC}}$$

(2) 化简输出函数表达式

用代数法对输出函数 F 的表达式化简如下：

$$\begin{aligned}F&=\overline{\overline{\overline{A}(B+C)}\cdot A\cdot\overline{BC}}\\&=\overline{A}(B+C)+A\overline{BC}\\&=\overline{A}B+\overline{A}C+A\overline{B}+A\overline{C}\\&=A\oplus B+A\oplus C\end{aligned}$$

(3) 根据化简后的函数表达式列出真值表

该函数的真值表如表 4.1 所示。

表 4.1　真值表

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

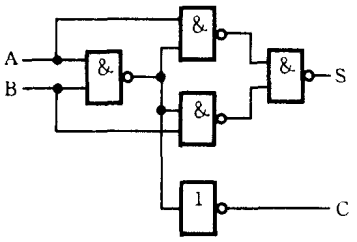
(4) 功能评述

由真值表可知，该电路仅当 A、B、C 取值同为 0 或同为 1 时输出 F 的值为 0，其他情况下输出 F 均为 1。换句话说，当输入取值一致时输出为 0，不一致时输出为 1，可见，该电路具有检查输入信号是否一致的逻辑功能，一旦输出为 1，则表明输入不一致。因此，通常称该电路为“不一致电路”。

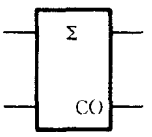
在某些可靠性要求非常高的系统中，往往是几套设备同时工作，一旦运行结果不一致，便由“不一致电路”发出报警信号，通知操作人员排除故障，以确保系统的可靠性。

其次，由分析可知，该电路的设计方案并不是最简的。根据化简后的输出函数表达式可采用异或门和或门画出实现给定功能的逻辑电路图如图 4.2(b)所示。显然，它比原电路简单、清晰。

例 4.2　分析图 4.3(a)所示逻辑电路。



(a)



(b)

表 4.2　真值表

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

图 4.3　逻辑电路及其逻辑符号

解　根据给出的逻辑电路图可写出输出函数表达式

$$S=\overline{\overline{A\overline{B}}\cdot A}\cdot\overline{\overline{A\overline{B}}\cdot B}$$

$$C=\overline{A\overline{B}}$$

用代数化简法对输出函数化简：

$$\begin{aligned}S&=\overline{\overline{A\overline{B}}\cdot A}\cdot\overline{\overline{A\overline{B}}\cdot B}\\&=\overline{A\overline{B}}\cdot A+\overline{A\overline{B}}\cdot B\\&=(\overline{A}+\overline{B})A+(\overline{A}+\overline{B})\cdot B\\&=A\overline{B}+\overline{A}B\end{aligned}$$

$$C=\overline{A\overline{B}}=AB$$

根据简化后的表达式可列出真值表，如表 4.2 所示。

由真值表可以看出，若将 A、B 分别作为一位二进制数，则 S 是 A、B 相加的“和”，而 C 是相加产生的“进位”。该电路通常称作“半加器”，它能实现两个一位二进制数加法运算。半加器已被加工成小规模集成电路器件，其逻辑符号如图 4.3(b)所示。

例 4.3　分析图 4.4 所示组合逻辑电路。已知电路输入 ABCD 为 8421 码，说明该电路功能。

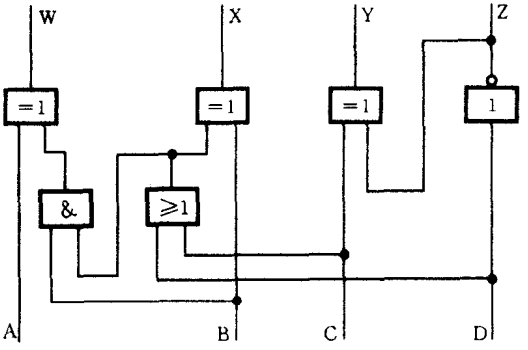


图 4.4　逻辑电路

解 根据图 4.4 所示逻辑电路，可写出输出函数表达式如下：

$$W=A\oplus B\cdot(C+D)$$

$$X=B\oplus(C+D)$$

$$Y=C\oplus\bar{D}$$

$$Z=\bar{D}$$

由于电路输入 ABCD 为 8421 码，所以 ABCD 只允许取值 0000～1001。根据所得输出函数表达式可列出真值表，如表 4.3 所示。

表 4.3 真值表

ABCD	WXYZ	ABCD	WXYZ
0000	0011	0101	1000
0001	0100	0110	1001
0010	0101	0111	1010
0011	0110	1000	1011
0100	0111	1001	1100

由真值表可知，电路输出 WXYZ 是一位十进制数的余 3 码，即该电路是一个将 8421 码转换成余 3 码的代码转换电路。

以上例子说明了组合逻辑电路分析的一般方法。从讨论过程可以看出，通过对电路进行分析，不仅可以找出电路输入、输出之间的关系，确定电路的逻辑功能，同时还能对某些设计不合理的电路进行改进和完善。

4.2 组合逻辑电路设计

根据问题要求完成的逻辑功能，求出在特定条件下实现该功能的逻辑电路，这一过程称为逻辑设计，又叫做逻辑综合。显然，逻辑设计是逻辑分析的逆过程。

4.2.1 设计方法概述

由于实际应用中提出的各种设计要求一般是用文字形式描述的，所以逻辑设计的首要任务是将设计问题转化为逻辑问题，即将文字描述的设计要求抽象为一种逻辑关系。就组合逻辑电路而言，就是抽象出描述问题的逻辑表达式。

组合逻辑电路的设计过程大致如下。

1. 建立给定问题的逻辑描述

对设计要求进行逻辑描述是完成组合电路设计的第一步，也是最重要的一步，

它是确保设计方案正确的前提。这一步的关键是正确理解设计要求，弄清楚与给定问题相关的变量及函数，即电路的输入和输出，建立函数与变量之间的逻辑关系，最终得到描述给定问题的逻辑表达式。求逻辑表达式的常用方法有真值表法和分析法两种，后面将结合实例进行介绍。

2. 求出逻辑函数的最简表达式

基于小规模集成电路的组合逻辑电路设计是以最简方案为目标的，即要求逻辑电路中包含的逻辑门最少且连线最少。因此，要对逻辑表达式进行化简，求出描述设计问题的最简表达式。

3. 选择逻辑门类型并进行逻辑函数变换

根据简化后的逻辑表达式及问题的具体要求，选择合适的逻辑门，并将逻辑表达式变换成与所选逻辑门对应的形式。

4. 画出逻辑电路图

根据变换后的表达式画出逻辑电路图。

以上步骤是就一般情况而言的，根据实际问题的难易程度和设计者的熟练程度，有时可跳过其中的某些步骤。在设计过程中可视具体情况灵活掌握。

4.2.2　设计举例

例 4.4　设计一个 3 变量“多数表决电路”。

解　(1) 建立给定问题的逻辑描述

“多数表决电路”的逻辑功能就是按照少数服从多数的原则执行表决，确定某项决议是否通过。假设用 A、B、C 分别代表参加表决的 3 个逻辑变量，用函数 F 表示表决结果。并约定，逻辑变量取值为 0 表示反对，逻辑变量取值为 1 表示赞成；逻辑函数 F 取值为 0 表示决议被否决，逻辑函数 F 取值为 1 表示决议通过。那么，按照少数服从多数的原则可知，函数和变量的关系是：当 3 个变量 A、B、C 中有 2 个或 2 个以上取值为 1 时，函数 F 的值为 1，其他情况下函数 F 的值为 0。因此，可列出该逻辑函数的真值表，如表 4.4 所示。由真值表可写出函数 F 的最小项表达式为

$$F(A,B,C)=\sum m(3,5,6,7)$$

(2) 求出逻辑函数的最简表达式

作出函数 F 的卡诺图如图 4.5(a)所示，用卡诺图化简后得到函数的最简与-

或表达式为

$$F(A,B,C)=AB+AC+BC$$

表 4.4 真值表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

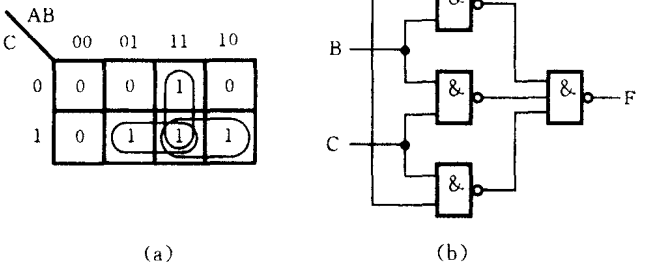


图 4.5 卡诺图及逻辑电路

(3) 选择逻辑门类型并进行逻辑函数变换

假定采用与非门组成实现给定功能的电路，则应将上述表达式变换成“与非-与非”表达式

$$F(A,B,C)=\overline{\overline{AB+AC+BC}}=\overline{\overline{AB}\cdot\overline{AC}\cdot\overline{BC}}$$

(4) 画出逻辑电路图

由函数的“与非-与非”表达式，可画出实现给定功能的逻辑电路，如图 4.5(b)所示。

本例在建立给定问题的逻辑描述时，采用的是“真值表法”，即通过建立真值表得到相应的逻辑表达式。真值表的优点是规整、清晰，缺点是不方便，尤其当变量较多时十分麻烦。因此，针对具体问题通常采用的另一种方法是“分析法”，即通过对设计要求的分析、理解，直接写出逻辑表达式。

例 4.5 设计一个比较两个 3 位二进制数是否相等的数值比较器。

解 (1) 建立给定问题的逻辑描述

设两个 3 位二进制数分别为 $A=a_3a_2a_1$，$B=b_3b_2b_1$，比较结果用 F 表示。当 A=B 时，F 为 1；否则 F 为 0。显然，这是一个有 6 个输入变量、1 个输出函数的组合逻辑电路。

由于二进制数 A 和 B 相等，必须同时满足 $a_3=b_3$、$a_2=b_2$、$a_1=b_1$，而二进制中 $a_i=b_i$ 只有 a_i 和 b_i 同时为 0 或者同时为 1 两种可能，因此，该问题可用逻辑表达式描述如下：

$$F=(\bar{a}_3\bar{b}_3+a_3b_3)(\bar{a}_2\bar{b}_2+a_2b_2)(\bar{a}_1\bar{b}_1+a_1b_1)$$

(2) 求出逻辑函数最简表达式

将上述逻辑函数表达式展开成与-或表达式可以发现，该函数不能简化，表达

式中包含 8 个 6 变量与项。

(3) 选择逻辑门类型并进行逻辑函数变换

假定采用异或门和或非门实现给定功能，可将逻辑表达式作如下变换：

$$F=(\bar{a}_3\bar{b}_3+a_3b_3)(\bar{a}_2\bar{b}_2+a_2b_2)(\bar{a}_1\bar{b}_1+a_1b_1)$$
$$=\overline{(a_3\oplus b_3)}\,\overline{(a_2\oplus b_2)}\,\overline{(a_1\oplus b_1)}$$
$$=\overline{(a_3\oplus b_3)+(a_2\oplus b_2)+(a_1\oplus b_1)}$$

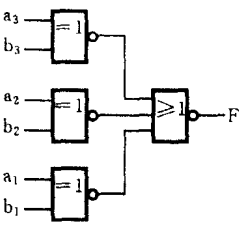


图 4.6　逻辑电路

(4) 画出逻辑电路图

根据变换后的表达式可画出逻辑电路，如图 4.6 所示。

例 4.6　设计一个乘法器，用于产生两个 2 位二进制数相乘的积。

解　因为一个 2 位二进制数表示的十进制数最大为 3，两个 2 位二进制数相乘的积最大为 9，所以相乘的积可用一个 4 位二进制数表示。由此可见，该电路有 4 个输入变量，4 个输出函数。设两个二进制数分别为 A_1A_0 和 B_1B_0，相乘的积为 $M_3M_2M_1M_0$，下面用两种不同的方法抽象出电路的输出函数表达式，并选用合适的逻辑门完成相应电路设计。

方法 I　采用真值表法

(1) 借助真值表写出输出函数表达式

根据设计要求和对问题的假设可列出真值表，如表 4.5 所示。

表 4.5　真值表

A_1	A_0	B_1	B_0	M_3	M_2	M_1	M_0	A_1	A_0	B_1	B_0	M_3	M_2	M_1	M_0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1	0	1	0	0	1	0	0
0	0	1	1	0	0	0	0	1	0	1	1	0	1	1	0
0	1	0	0	0	0	0	0	1	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1	1	1	0	1	0	0	1	1
0	1	1	0	0	0	1	0	1	1	1	0	0	1	1	0
0	1	1	1	0	0	1	1	1	1	1	1	1	0	0	1

根据真值表可写出输出函数表达式如下：

$$M_3(A_1,A_0,B_1,B_0)=m_{15}$$
$$M_2(A_1,A_0,B_1,B_0)=m_{10}+m_{11}+m_{14}$$
$$M_1(A_1,A_0,B_1,B_0)=m_6+m_7+m_9+m_{11}+m_{13}+m_{14}$$
$$M_0(A_1,A_0,B_1,B_0)=m_5+m_7+m_{13}+m_{15}$$

(2) 求出逻辑函数最简表达式

利用卡诺图可求出输出函数的最简与-或表达式为

$$M_3 = A_1 A_0 B_1 B_0$$
$$M_2 = A_1 \overline{A}_0 B_1 + A_1 B_1 \overline{B}_0$$
$$M_1 = A_1 \overline{B}_1 B_0 + A_1 \overline{A}_0 B_0 + \overline{A}_1 A_0 B_1 + A_0 B_1 \overline{B}_0$$
$$M_0 = A_0 B_0$$

(3) 选择逻辑门类型并画出逻辑电路图

假定采用与门和或门实现给定功能，可画出实现给定功能的逻辑电路，如图4.7所示。

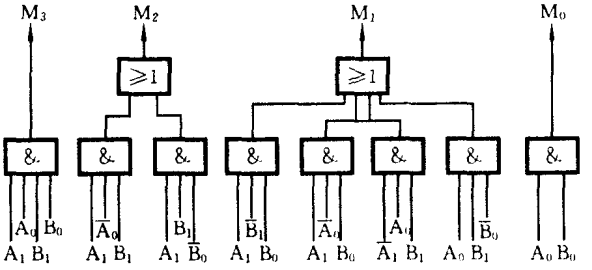


图 4.7 方法 I 的逻辑电路

方法 II 采用分析法

(1) 通过分析写出输出函数表达式

按照二进制乘法运算法则和对问题的假设，可列出乘法计算过程如下：

			A_1	A_0
×(乘)			B_1	B_0
	C_2	C_1	$A_1 \times B_0$	$A_0 \times B_0$
+(加)		$A_1 \times B_1$	$A_0 \times B_1$	
	M_3	M_2	M_1	M_0

上述计算式中，积项 $A_i \times B_j$ $(i,j=0,1)$ 为两个 1 位二进制数相乘，可用两输入与门实现，即 $A_i \times B_j$ 的算术值等于 $A_i \cdot B_j$ 的逻辑值。因为 C_1 为 A_1B_0 和 A_0B_1 相加产生的进位，它等于 A_1B_0 和 A_0B_1 相与，即 $C_1 = A_1B_0A_0B_1$，C_2 为 C_1 和 A_1B_1 相加产生的进位，它等于 C_1 和 A_1B_1 相与，即 $C_2 = C_1 \cdot A_1B_1 = A_1B_0A_0B_1 \cdot A_1B_1 = A_1B_0A_0B_1$；又因为两个 1 位二进制数相加的和等于对其进行异或运算的结果，所以可由分析得到该电路的输出函数表达式如下：

$$M_0 = A_0B_0$$
$$M_1 = A_1B_0 \oplus A_0B_1$$
$$M_2 = C_1 \oplus A_1B_1 = A_1B_0A_0B_1 \oplus A_1B_1$$
$$M_3 = C_2 = A_1B_0A_0B_1$$

假定采用异或门和与门实现该乘法器的逻辑功能，可画出该乘法器的逻辑电路，如图 4.8 所示。

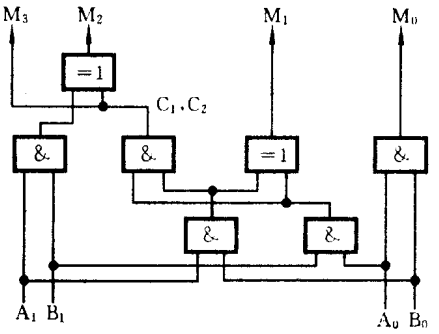


图 4.8　方法Ⅱ的逻辑电路

比较两种方法所得结果可以看出，图 4.7 所示电路中共用了 10 个逻辑门，而图 4.8 所示电路中只用了 7 个逻辑门，显然，后者比前者更简单。为了证明两者逻辑功能的一致性，读者可对方法Ⅱ中的表达式进行展开变换，即可得到方法Ⅰ中的最简与-或表达式。

4.2.3　设计中几个实际问题的处理

上面对组合逻辑电路设计的一般方法进行了讨论，并通过简单例子介绍了设计的全过程。然而，实际提出的设计要求是形形色色的，往往除了问题复杂之外，还存在某些特殊情况需要考虑。为了能在各种特殊情况下设计出最简的逻辑电路，必须针对具体问题作出具体的分析和处理。下面就几个常见问题进行讨论。

1. 包含无关条件的组合逻辑电路设计

前面讨论的设计问题中，对于电路输入变量的任何一种取值组合，都有确定的输出函数值与之对应，换句话说，对于一个具有 n 个输入变量的组合逻辑电路，输出函数与 2^n 种输入取值组合均相关，假如有 m 种取值组合使函数的值为 1，则有 2^n-m 种取值组合使函数的值为 0，该输出函数可以用 m 个最小项之和表示。

但在某些实际问题中，常常由于输入变量之间存在的相互制约或问题的某种特殊限定等，使得输入变量的某些取值组合根本不会出现，或者虽然可能出现，但对在这些输入取值组合下函数的值是为 1 还是为 0 并不关心。通常把这类问题称为包含无关条件的逻辑问题；与这些输入取值组合对应的最小项称为无关最小项，简称为无关项或者任意项；描述这类问题的逻辑函数称为包含无关条件的逻辑函数。

当采用最小项之和表达式描述一个包含无关条件的逻辑问题时，函数表达式中是否包含无关项以及对无关项是令其值为 1 还是为 0，并不影响函数的实际逻辑功能。因此，在化简这类逻辑函数时，利用这种随意性往往可以使逻辑函数得到更好地简化，从而使设计的电路达到更简。

例 4.7 设计一个组合逻辑电路，用于判别以余 3 码表示的 1 位十进制数是否为合数。

解 由题意可知，该电路输入为 1 位十进制数的余 3 码，输出为对其值进行判断的结果。设输入变量为 A、B、C、D，输出函数为 F，当 ABCD 表示的十进制数为合数(4、6、8、9)时，输出 F 为 1，否则 F 为 0。因为按照余 3 码的编码规则，ABCD 的取值组合不允许为 0000、0001、0010、1101、1110、1111，故该问题为包含无关条件的逻辑问题，与上述 6 种取值组合对应的最小项为无关项，即在这些取值组合下输出函数 F 的值可以随意指定为 1 或者为 0，通常记为“d”。据此可建立描述该问题的真值表，如表 4.6 所示。

表 4.6 真值表

A	B	D	D	F	A	B	D	D	F
0	0	0	0	d	1	0	0	0	0
0	0	0	1	d	1	0	0	1	1
0	0	1	0	d	1	0	1	0	0
0	0	1	1	0	1	0	1	1	1
0	1	0	0	0	1	1	0	0	1
0	1	0	1	0	1	1	0	1	d
0	1	1	0	0	1	1	1	0	d
0	1	1	1	1	1	1	1	1	d

根据真值表可写出 F 的逻辑表达式为

$$F(A,B,C,D)=\sum m(7,9,11,12)+\sum d(0,1,2,13,14,15)$$

用卡诺图化简函数 F 时，若不考虑无关项，如图 4.9(a)所示合并卡诺图上的 1 方格，则可得到经化简后的逻辑表达式为

$$F(A,B,C,D)=A\bar{B}D+AB\bar{C}\bar{D}+\bar{A}BCD$$

如果化简时对无关条件加以利用，如图 4.9(b)所示，根据合并的需要将卡诺图中的无关项 d(13,14,15)当成 1 处理，而把 d(0,1,2)当成 0 处理，则可得到经化简后的逻辑表达式为

$$F(A,B,C,D)=AB+AD+BCD$$

显然，后一个表达式比前一个表达式更简单。假定采用与非门构成实现给定逻辑功能的电路，可将 F 的最简表达式变换成“与非-与非”表达式：

$$F(A,B,C,D)=\overline{\overline{AB+AD+BCD}}=\overline{\overline{AB}\cdot\overline{AD}\cdot\overline{BCD}}$$

相应的逻辑电路如图 4.10 所示。

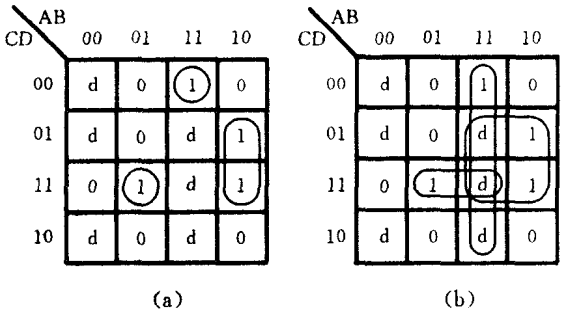


图 4.9　卡诺图

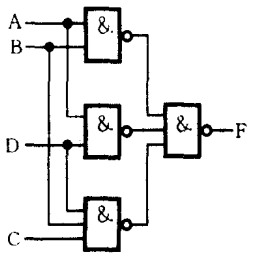


图 4.10　逻辑电路

由此可见，设计包含无关条件的组合逻辑电路时，恰当地利用无关项进行函数化简，通常可使设计出来的电路更简单。

2. 多输出函数的组合逻辑电路设计

实际问题中，大量存在着由同一组输入变量产生多个输出函数的问题，实现这类问题的组合逻辑电路称为多输出函数的组合逻辑电路。

设计多输出函数的组合逻辑电路时，如果只是孤立地求出各输出函数的最简表达式，然后画出相应逻辑电路图并将其拼在一起，通常不能保证逻辑电路整体最简。因为各输出函数之间往往存在相互联系，具有某些共同的部分，因此，应该将它们当作一个整体考虑，而不应该将其截然分开。使这类电路达到最简的关键在于函数化简时找出各输出函数的公用项，以便在逻辑电路中实现对逻辑门的共享，从而使电路整体结构最简。

例 4.8　假定某组合逻辑电路结构框图如图 4.11 所示，试用最少的与非门实现该电路功能。

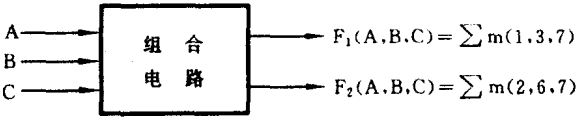


图 4.11　组合逻辑电路结构框图

解　为了求出用与非门构成的最简电路，首先应求出输出函数的最简与-或表达式，并变换成与非-与非表达式，然后画出相应的逻辑电路图。首先，若不考虑两个输出函数之间的共享问题，则可直接作出 F_1、F_2 的卡诺图，如图 4.12(a)、(b)所示。化简后的输出函数表达式为

$$F_1=\overline{A}C+B\overline{C}=\overline{\overline{\overline{A}C+B\overline{C}}}=\overline{\overline{\overline{A}C}\cdot\overline{B\overline{C}}}$$

$$F_2=AB+B\overline{C}=\overline{\overline{AB+B\overline{C}}}=\overline{\overline{AB}\cdot\overline{B\overline{C}}}$$

根据化简后的输出函数表达式可画出相应的逻辑电路，如图 4.12(c)所示。

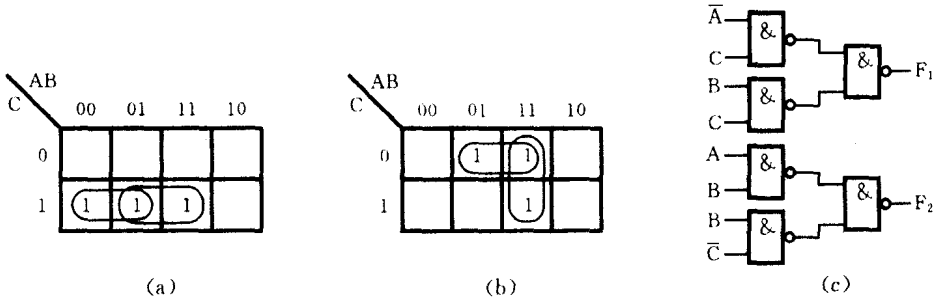


图 4.12 例 4.8 的卡诺图及逻辑电路之一

如果考虑两个输出函数应充分“共享”的问题，则在对 F_1、F_2进行化简时，可按图 4.13(a)、(b)所示卡诺图进行处理，化简后的输出逻辑表达式为

$$F_1=\overline{A}C+ABC=\overline{\overline{A}C+ABC}=\overline{\overline{AC}\cdot\overline{ABC}}$$

$$F_2=ABC+B\overline{C}=\overline{\overline{ABC}+B\overline{C}}=\overline{\overline{ABC}\cdot\overline{B\overline{C}}}$$

根据修改后的表达式，可画出相应的逻辑电路，如图 4.13(c)所示。显然，通过找出两个函数的公共项，使其“共享”同一个逻辑门，从而使电路从整体上得到了进一步简化。

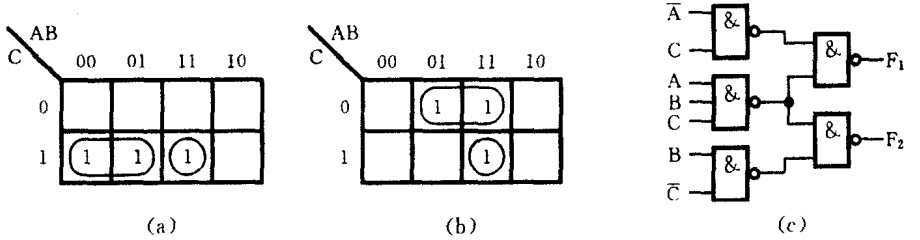


图 4.13 例 4.8 的卡诺图及逻辑电路之二

例 4.9 设计一个全加器。

解 全加器是一个能对两个一位二进制数及来自低位的“进位”进行相加，产生本位“和”及向高位“进位”的逻辑电路。由此可知，该电路有 3 个输入变量，2 个输出函数。设被加数、加数及来自低位的“进位”分别用 A_i、B_i及 C_{i-1}表示，相加产生的“和”及“进位”用 S_i和 C_i表示。根据二进制加法运算法则可列出全加器的真值表，如表 4.7 所示。

表 4.7 全加器真值表

A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

由真值表可写出输出函数表达式：

$$S_i(A_i,B_i,C_{i-1})=\sum m(1,2,4,7)$$

$$C_i(A_i,B_i,C_{i-1})=\sum m(3,5,6,7)$$

假定采用卡诺图化简上述函数，则可作出相应卡诺图，如图 4.14 所示。

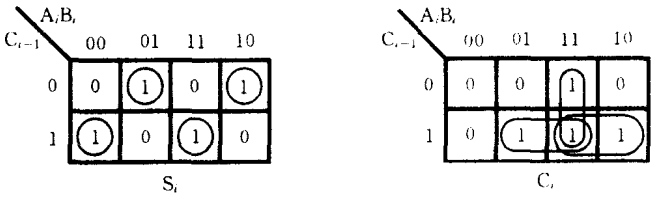


图 4.14　函数 S_i 和 C_i 的卡诺图

经化简后的输出函数表达式为

$$S_i=\overline{A}_iB_i\overline{C}_{i-1}+\overline{A}_i\overline{B}_iC_{i-1}+A_i\overline{B}_i\overline{C}_{i-1}+A_iB_iC_{i-1}$$

$$C_i=A_iB_i+A_iC_{i-1}+B_iC_{i-1}$$

其中，S_i 的标准“与-或”式即最简“与-或”式。当采用异或门和与非门组成实现给定功能的电路时，可对表达式作如下变换：

$$\begin{aligned}S_i&=\overline{A}_iB_i\overline{C}_{i-1}+\overline{A}_i\overline{B}_iC_{i-1}+A_i\overline{B}_i\overline{C}_{i-1}+A_iB_iC_{i-1}\\&=\overline{A}_i(\overline{B}_iC_{i-1}+B_i\overline{C}_{i-1})+A_i(\overline{B}_i\overline{C}_{i-1}+B_iC_{i-1})\\&=\overline{A}_i(B_i\oplus C_{i-1})+A_i(\overline{B_i\oplus C_{i-1}})\\&=A_i\oplus B_i\oplus C_{i-1}\end{aligned}$$

$$\begin{aligned}C_i&=A_iB_i+A_iC_{i-1}+B_iC_{i-1}\\&=\overline{\overline{A_iB_i}\cdot\overline{A_iC_{i-1}}\cdot\overline{B_iC_{i-1}}}\end{aligned}$$

相应的逻辑电路如图 4.15(a)所示。该电路就单个函数而言均已达到最简，但从整体考虑则并非最简。当按多输出函数组合电路进行设计时，可对函数 C_i 作

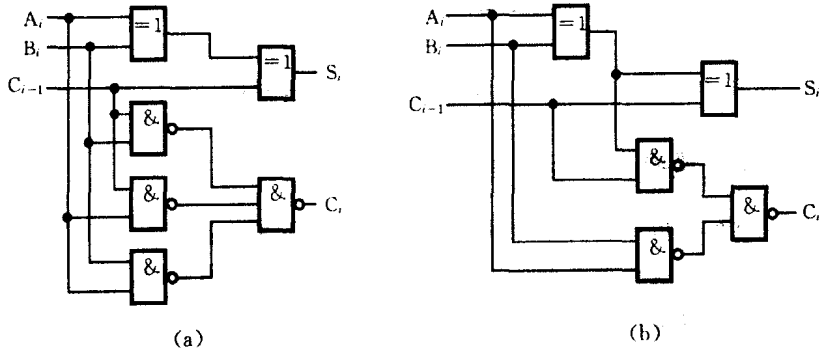


图 4.15　全加器逻辑电路

如下变换：

$$\begin{aligned}C_i&=\overline{A}_iB_iC_{i-1}+A_i\overline{B}_iC_{i-1}+A_iB_i\overline{C}_{i-1}+A_iB_iC_{i-1}\\&=(\overline{A}_iB_i+A_i\overline{B}_i)C_{i-1}+A_iB_i(\overline{C}_{i-1}+C_{i-1})\\&=(A_i\oplus B_i)C_{i-1}+A_iB_i\\&=\overline{\overline{(A_i\oplus B_i)C_{i-1}}\cdot\overline{A_iB_i}}\end{aligned}$$

经变换后，S_i和C_i的逻辑表达式中有公用项$A_i\oplus B_i$，因此，在组成电路时，可令其共享同一异或门，从而使整体得到进一步简化，其逻辑电路如图 4.15(b)所示。

选用第 3 章介绍的集成门电路实现全加器功能时，实现图 4.15(a)所示电路的功能需要 3 种芯片，可用一块异或门芯片 7486、一块 2 输入与非门芯片 7400 和一块 3 输入与非门芯片 7410。而实现图 4.15(b)所示电路的功能则只需要两种芯片，用一块异或门芯片 7486 和一块与非门芯片 7400 即可，图 4.16 给出了一个实现该电路功能的芯片引脚连接图。值得指出的是，由于器件提供的逻辑门数目多于电路所需逻辑门数目，所以在功能实现时对芯片内部逻辑门的选择方案不是惟一的，不同的选择方案对应不同的芯片引脚连接图。

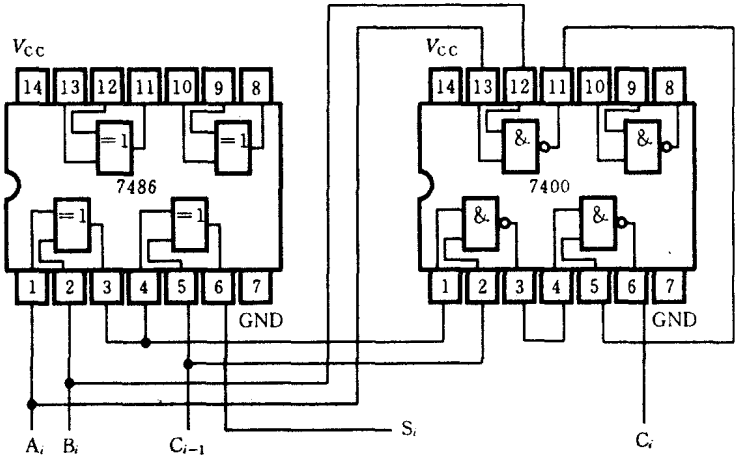


图 4.16 全加器的芯片引脚连接图

3. 无反变量提供的组合逻辑电路设计

在对某些实际问题进行设计的过程中，常常为了减少各部件之间的连线，只给所设计的逻辑电路提供原变量，不提供反变量。设计这类电路时，直截了当的办法是当需要某个反变量时，就用一个非门将相应的原变量转换成反变量，但这样处理往往是不经济的。因此，通常采用适当的方法进行处理，以便在无反变量提供的前提下，使逻辑电路尽可能简单。

例 4.10　输入变量中无反变量时，用与非门实现逻辑函数

$$F(A,B,C,D)=\overline{A}B+B\overline{C}+A\overline{B}C+AC\overline{D}$$

的功能。

解　因为给定函数已经是最简“与-或”表达式，故可直接画出逻辑电路，如图 4.17(a)所示，但这个电路并不是最简的。如果对函数 F 的表达式作如下整理：

$$\begin{aligned}F(A,B,C,D)&=\overline{A}B+B\overline{C}+A\overline{B}C+AC\overline{D}\\&=B(\overline{A}+\overline{C})+AC(\overline{B}+\overline{D})\\&=B\overline{AC}+AC\,\overline{BD}\end{aligned}$$

根据整理后的表达式可画出对应的逻辑电路，如图 4.17(b)所示。显然，图 4.17(b)比图 4.17(a)更合理。

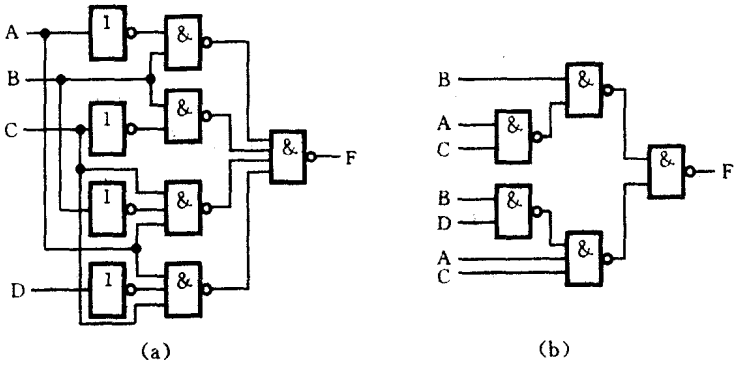


图 4.17　逻辑电路

例 4.11　在输入无反变量提供时，用最少的逻辑门实现逻辑函数 $F(A,B,C)=\overline{A}B+B\overline{C}+A\overline{B}C$ 的功能。

解　给定逻辑函数表达式已为最简与或表达式，直接使用与非门和反相器实现该函数功能的逻辑电路如图 4.18(a)所示，电路中共用了 3 个反相器和 4 个与非门。

图 4.18(a)所示电路是不是最简的呢？不是！为了得到更简单的电路，可对给定逻辑函数表达式作如下变换：

$$\begin{aligned}F(A,B,C)&=\overline{A}B+B\overline{C}+A\overline{B}C\\&=(\overline{A}+\overline{C})B+A\overline{B}C\\&=\overline{AC}B+AC\overline{B}\\&=AC\oplus B\end{aligned}$$

根据变换后的逻辑表达式可画出实现原函数功能的更简逻辑电路，如图 4.18(b)所示。

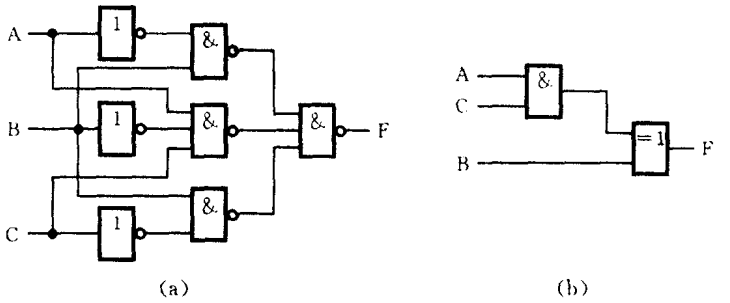


图 4.18 逻辑电路

由上述两个例子可以看出，在输入无反变量提供的场合，即使逻辑函数表达式已为最简，但直接实现时所得到的电路也不一定最简，通常对逻辑表达式作适当变换，可以减少电路中非门的数量，更好地简化电路结构。然而，当描述某种设计要求的表达式被确定下来后，并不是都可以作类似变换的。在实际问题的设计中，电路的复杂度往往和设计过程中某些步骤的处理方案直接相关。

例 4.12 设计一个组合逻辑电路，用来判断献血者与受血者血型是否相容。血型相容规则如表 4.8 所示，表中用"√"表示两者血型相容。

表 4.8 血型相容规则表

受血 / 献血	A	B	AB	O
A	√		√	
B		√	√	
AB			√	
O	√	√	√	√

表 4.9 血型编码(1)

血型	献 WX	受 YZ
A	00	00
B	01	01
AB	10	10
O	11	11

解 根据题意，电路输入变量为献血者血型和受血者血型。血型共 4 种，可用两个变量的 4 种编码进行区分。设变量 WX 表示献血者血型，YZ 表示受血者血型，血型编码如表 4.9 所示。电路输出用 F 表示，当输血者与受血者血型相容时，F 为 1，否则 F 为 0。可根据血型相容规则直接写出输出函数 F 的表达式：

$$
\begin{aligned}
F &= \overline{W}\,\overline{X}(\overline{Y}\,\overline{Z}+Y\overline{Z})+\overline{W}X(\overline{Y}Z+Y\overline{Z})+W\overline{X}Y\overline{Z}+WX(\overline{Y}\,\overline{Z}+\overline{Y}Z+Y\overline{Z}+YZ)\\
&=\overline{W}\,\overline{X}\,\overline{Z}+\overline{W}X\overline{Y}Z+\overline{W}XY\overline{Z}+W\overline{X}Y\overline{Z}+WX\\
&=\overline{W}\,\overline{Z}(\overline{X}+XY)+(\overline{W}\,\overline{Y}Z+W)X+W(\overline{X}Y\overline{Z}+X)\\
&=\overline{W}\,\overline{Z}(\overline{X}+Y)+(W+\overline{Y}Z)X+W(X+Y\overline{Z})\\
&=\overline{W}\,\overline{X}\,\overline{Z}+\overline{W}Y\overline{Z}+WX+X\overline{Y}Z+WX+WY\overline{Z}\\
&=\overline{W}\,\overline{X}\,\overline{Z}+Y\overline{Z}+WX+X\overline{Y}Z
\end{aligned}
$$

由化简后的表达式可知，在无反变量提供的情况下，若通过直接加非门产生反

变量，则组成实现给定功能的电路时需 9 个逻辑门，其中 4 个非门用来产生 4 个输入变量的反变量。用与非门组成实现给定功能的逻辑电路如图 4.19(a)所示。

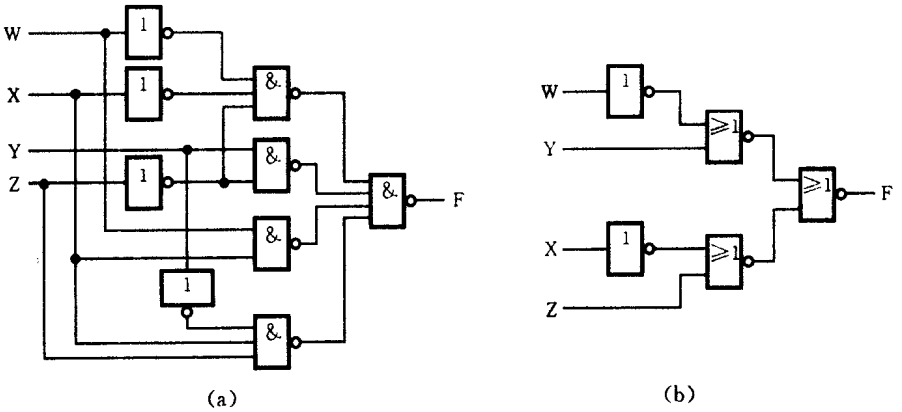


图 4.19　逻辑电路

分析上述设计过程不难发现，对该问题的逻辑描述与血型编码是直接相关的。为了减少逻辑表达式中反变量个数，进一步简化电路结构，可调整血型编码，如表 4.10所示。

表 4.10　血型编码(2)

血　型	献 WX	受 YZ
O	00	00
A	01	01
B	10	10
AB	11	11

根据新的编码方案和血型相容规则，可写出输出函数 F 的表达式：

$$F=\overline{W}\overline{X}(\overline{Y}\,\overline{Z}+\overline{Y}Z+Y\,\overline{Z}+YZ)+\overline{W}X(\overline{Y}Z+YZ)+W\overline{X}(Y\,\overline{Z}+YZ)+WXYZ$$
$$=\overline{W}\overline{X}+\overline{W}XZ+W\overline{X}Y+WXYZ$$
$$=\overline{W}(\overline{X}+XZ)+WY(\overline{X}+XZ)$$
$$=\overline{W}(\overline{X}+Z)+WY(\overline{X}+Z)$$
$$=(\overline{W}+WY)(\overline{X}+Z)$$
$$=(\overline{W}+Y)(\overline{X}+Z)$$

该函数表达式中仅含两个反变量，假定采用或非门实现给定功能，则可将函数表达式变换成“或非-或非”表达式：

$$F=\overline{\overline{(\overline{W}+Y)(\overline{X}+Z)}}=\overline{\overline{\overline{W}+Y+\overline{X}+Z}}$$

逻辑电路如图 4.19(b)所示，电路中只使用了 5 个逻辑门。

关于在无反变量提供时如何使组合电路达到最简的问题，至今尚无一种系统而有效的方法，只能由设计者根据具体问题进行灵活处理。

4.3 组合逻辑电路的险象

前面讨论组合逻辑电路时，只研究了输入和输出稳定状态之间的关系，而没有考虑信号传输中的时延问题，实际上，信号经过任何逻辑门和导线都会产生时间延迟，这就使得当电路所有输入达到稳定状态时，输出并不是立即达到稳定状态。

一般来说，延迟时间对数字系统是一个有害的因素。例如，使得系统操作速度下降，引起电路中信号的波形参数变坏，更严重的是在电路中产生竞争险象的问题。本节将专门针对后一个问题进行讨论。

4.3.1 险象的产生

在实际逻辑电路中，信号经过同一电路中的不同路径所产生的时延一般来说是各不相同的。各路径上延迟时间的长短与信号经过的门的级数有关，与具体逻辑门的时延大小有关，还与导线的长短有关。因此，输入信号经过不同路径到达输出端的时间也就有先有后，这就好像一场赛跑，各运动员到达终点的时间有先有后一样，这种现象称为**竞争现象**。在逻辑电路中，竞争现象是随时随地都可能出现的，我们可以更广义地把竞争现象理解为多个信号到达某一点有时差所引起的现象。

电路中竞争现象的存在，使得输入信号的变化可能引起输出信号出现非预期的错误输出，这一现象称为**险象**。并不是所有的竞争都会产生错误输出。通常，把不产生错误输出的竞争称为**非临界竞争**，而导致错误输出的竞争称为**临界竞争**。

组合电路中的险象是一种瞬态现象，它表现为在输出端产生不应有的尖脉冲，暂时地破坏正常逻辑关系。一旦瞬态过程结束，即可恢复正常逻辑关系。下面举例说明这一现象。

例如，图 4.20(a)所示的是由与非门构成的组合电路，该电路有 3 个输入，1 个输出，输出函数表达式为

$$F=\overline{A\overline{B}\cdot\overline{AC}}=AB+\overline{A}C$$

假设输入变量 B=C=1，将 B、C 的值代入上述函数表达式，得

$$F=A+\overline{A}$$

由互补律可知，无论 A 怎样变化，该函数表达式 F 的值应恒为 1，即当 B=C=1 时，不论 A 是 0 还是 1，输出 F 的值都应保持 1 不变。然而，这是在一种理想状态

下得出的结论。现在要讨论的是当考虑电路存在时间延迟时，该电路的实际输入/输出关系。更具体地说，当 B=C=1 时，A 的变化会使电路引起怎样的输出响应。假定每个门的延迟时间为 t_{pd}，则可用图 4.20(b)所示的时间图来说明。

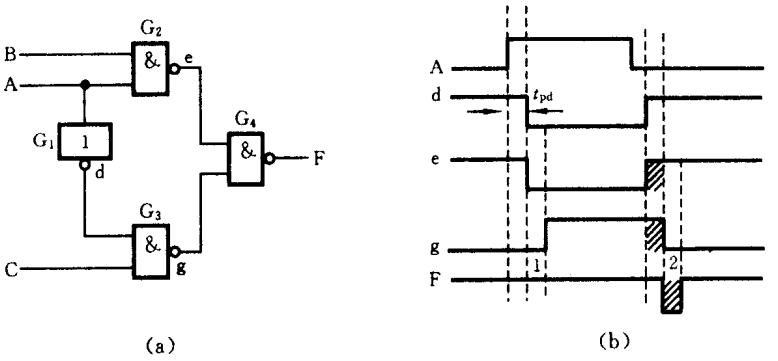


图 4.20　具有险象的逻辑电路及时间图(Ⅰ)

当 A 由低电平变到高电平以后，经过一个 t_{pd}时间，反相器 G_1的输出 d 由高电平变为低电平，同时与非门 G_2的输出 e 也由高电平变为低电平，但要再经过一个 t_{pd}，与非门 G_3的输出 g 才能由低电平变为高电平。最后到达门 G_4输入端的是由同一个 A 信号经不同路径传输而得到的两个信号 e 和 g，e 和 g 的变化方向相反，并具有一个 t_{pd}的时差。显然，这里(图 4.20(b)中 1 处)存在一次竞争。但因门G_4是一个与非门，e 和 g 竞争的结果，使门 G_4的输出保持为高电平，没有出现尖脉冲，即这里没有产生险象，所以，这次竞争是一次非临界竞争。但当 A 由高电平变为低电平时，情况就不一样了。e 和 g 同样要在门 G_4上发生竞争，且 e 和 g 在一个 t_{pd}的时间内同时为高电平，根据门 G_4的与非逻辑特性，输出 F 必然会出现一个负跳变的尖脉冲(如图 4.20(b)中 2 处所示)。也就是说，这次竞争的结果产生了险象，是一次临界竞争。

如果将图 4.20(a)中所示的与非门改为或非门，修改结果如图 4.21(a)所示，则根据修改后的电路可写出输出函数表达式为

$$F=\overline{\overline{A+B}+\overline{\overline{A}+C}}=(A+B)(\overline{A}+C)$$

假设输入变量 B=C=0，将 B、C 的值代入上述函数表达式，可得

$$F=A\cdot\overline{A}$$

由互补律可知，函数 $F=A\cdot\overline{A}$ 的值应恒为 0，即 B=C=0 时，无论 A 怎样变化，输出 F 的值都应保持 0 不变。然而，当考虑电路中存在的时间延迟时，可分析出电路输入/输出关系的时间图如图 4.21(b)所示。

由图 4.21(b)可知，当 A 由低电平变为高电平时，将在输出端产生一个正跳变

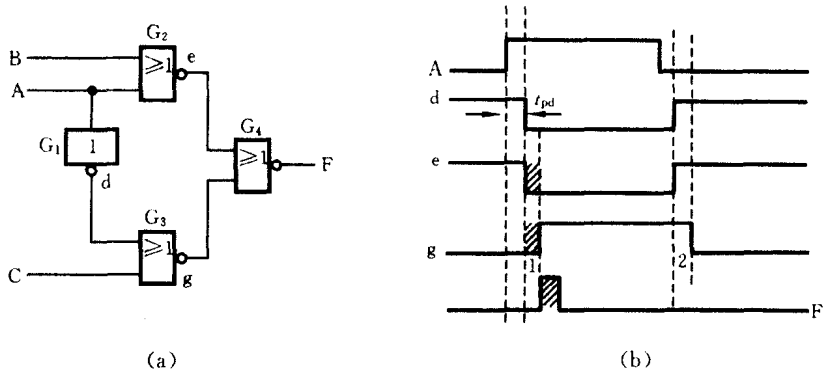


图 4.21　具有险象的逻辑电路及时间图(Ⅱ)

的尖脉冲信号，破坏了 F 的 0 信号输出，即发生了一次临界竞争。

通常按错误输出脉冲信号的极性将组合电路中的险象分为“0”型险象和“1”型险象。若错误输出信号为负脉冲，则称为“0”型险象；若错误输出信号为正脉冲，则称为“1”型险象。

4.3.2　险象的判断

判断一个电路是否有可能产生险象的方法有代数法和卡诺图法。

由前面对竞争和险象的分析可知，当某个变量 X 同时以原变量和反变量的形式出现在函数表达式中，且在一定条件下该函数表达式可简化成 $X+\bar{X}$ 或者 $X\cdot\bar{X}$ 的形式时，则与该函数表达式对应的电路在 X 发生变化时，可能由于竞争而产生险象。

代数法是根据描述电路的函数表达式来判断相应电路是否具有产生险象的条件。具体方法是：首先检查函数表达式中是否存在具备竞争条件的变量，即是否有某个变量 X 同时以原变量和反变量的形式出现在函数表达式中。若有，则消去函数表达式中的其他变量，即将这些变量的各种取值组合依次代入函数表达式中，从而把它们从函数表达式中消去，而仅保留被研究的变量 X，再看函数表达式是否会变为 $X+\bar{X}$ 或者 $X\cdot\bar{X}$ 的形式，若会，则说明对应的逻辑电路可能产生险象。下面举例说明。

例 4.13　已知描述某组合电路的逻辑函数表达式为 $F=\bar{A}\bar{C}+\bar{A}B+AC$，试判断该逻辑电路是否可能产生险象。

解　观察函数表达式可知，变量 A 和 C 均具备竞争条件，所以，应对这两个变量分别进行分析。先考察变量 A，为此将 B 和 C 的各种取值组合分别代入函数表

达式中，可得到如下结果：

$$BC=00 \qquad F=\overline{A}$$
$$BC=01 \qquad F=A$$
$$BC=10 \qquad F=\overline{A}$$
$$BC=11 \qquad F=A+\overline{A}$$

由此可见，当 B=C=1 时，A 的变化可能使电路产生险象。类似地，将 A 和 B 的各种取值组合分别代入函数表达式中，可由代入结果判断出变量 C 发生变化时不会产生险象。

例 4.14　试判断函数表达式 $F=(A+B)(\overline{A}+C)(\overline{B}+C)$ 描述的逻辑电路中是否可能产生险象。

解　从给出的函数表达式可以看出，变量 A 和 B 均具备竞争条件。先考察变量 B，为此将 A 和 C 的各种取值组合分别代入函数表达式中，结果如下：

$$AC=00 \qquad F=B\cdot\overline{B}$$
$$AC=01 \qquad F=B$$
$$AC=10 \qquad F=0$$
$$AC=11 \qquad F=1$$

可见，当 A=C=0 时，B 的变化可能使电路输出产生险象。用同样的方法考察 A，可发现当 B=C=0 时，A 的变化也可能产生险象。

判断险象的另一种方法是卡诺图法。当描述电路的逻辑函数为与-或表达式时，采用卡诺图来判断险象比代数法更为直观、方便。其具体方法是：首先作出函数卡诺图，并画出和函数表达式中各与项对应的卡诺圈。然后观察卡诺图，若发现某两个卡诺圈存在“相切”关系，即两卡诺圈之间存在不被同一卡诺圈包含的相邻最小项，则该电路可能产生险象。下面举列说明。

例 4.15　已知某逻辑电路对应的函数表达式为 $F=\overline{A}D+\overline{A}C+AB\overline{C}$，试判断该电路是否可能产生险象。

解　首先作出给定函数的卡诺图，并画出函数表达式中各与项对应的卡诺圈，如图 4.22 所示。

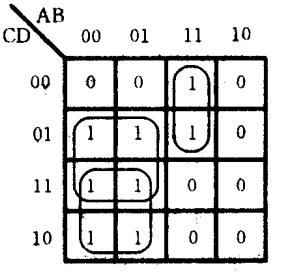


图 4.22　卡诺图

观察该卡诺图可发现，包含最小项 m_1, m_3, m_5, m_7 的卡诺圈和包含最小项 m_{12}, m_{13} 的卡诺圈之间存在相邻最小项 m_5 和 m_{13}，且 m_5 和 m_{13} 不被同一卡诺圈所包含，所以这两个卡诺圈“相切”。这说明相应电路可能产生险象。这一结论可用代数法进行验证，即假定 B=D=1，C=0，代入函数表达式 F 之后可得 $F=A+\overline{A}$，可见相应电路可能由于 A 的变化而产生险象。

4.3.3 险象的消除

为了使一个电路可靠地工作，设计者应当设法消除或避免电路中可能出现的险象。下面介绍几种常用的方法。

1. 用增加冗余项的方法消除险象

增加冗余项的方法是，通过在函数表达式中“或”上多余的与项或者“与”上多余的或项，使原函数不可能在某种条件下化成 $X+\bar{X}$ 或者 $X\cdot\bar{X}$ 的形式，从而消除可能产生的险象。具体冗余项的选择可以采用代数法或者卡诺图法。

例 4.16 用增加冗余项的方法消除图 4.20(a)所示电路中可能产生的险象。

解 图 4.20(a)所示函数表达式为

$$F=AB+\bar{A}C$$

在前面分析过，当 $B=C=1$ 时，输入 A 的变化使电路输出可能产生“0”型险象，即在输出应该为 1 的情况下产生了一个瞬间的 0 信号。解决问题的思路是，如何保证当 $B=C=1$ 时，输出保持为 1。显然，若函数表达式中包含有与项 BC，则可达到这一目的。由逻辑代数的定理 8 可知，若某变量以原变量和反变量的形式出现在与-或表达式的某两个与项中，则由该两项的其余因子组成的第三项是冗余项。因此，BC 是上述函数的一个冗余项，将 BC 加入函数表达式中并不影响原函数的逻辑功能。加入冗余项 BC 后的函数表达式为

$$F=AB+\bar{A}C+BC$$

增加冗余项后的逻辑电路如图 4.23 所示。该电路不再产生险象。

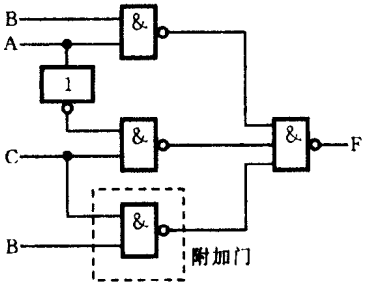


图 4.23 增加冗余项后的逻辑电路

冗余项的选择也可以通过在函数卡诺图上增加多余的卡诺圈来实现。其具体方法是，若卡诺图上某两个卡诺圈“相切”，则用一个多余的卡诺圈将它们之间的相邻最小项圈起来，与多余卡诺圈对应的与项就是要加入函数表达式中的冗余项。

例 4.17 已知描述某组合电路的函数表达式为 $F=\bar{A}C+B\bar{C}D+A\bar{B}\bar{C}$，试用增加冗余项的方法消除该电路中可能产生的险象。

解 首先作出给定函数的卡诺图，如图 4.24 所示。该卡诺图中，包含最小项 m_2,m_3,m_6,m_7 的卡诺圈和包含最小项 m_5,m_{13} 的卡诺圈“相切”，其相邻最小项为

m_7和m_5；包含最小项 m_5，m_{13}的卡诺圈和包含最小项 m_8，m_9的卡诺圈“相切”，其相邻最小项为 m_9和 m_{13}。可见，该电路可能由于竞争而产生险象。为了消除险象，可以在卡诺图上增加两个多余卡诺圈，分别把最小项 m_5，m_7和 m_9，m_{13}圈起来，如图 4.24 中虚线圈所示。由此得到函数表达式

$$F=\overline{A}C+B\overline{C}D+A\overline{B}\overline{C}+\overline{A}BD+A\overline{C}D$$

式中，$\overline{A}BD$ 和 $A\overline{C}D$ 为冗余项。读者可用代数法验证，该函数表达式所对应的逻辑电路不再存在险象。

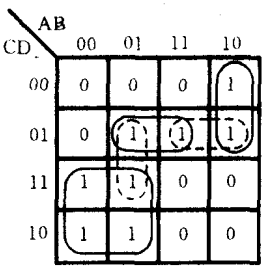


图 4.24　卡诺图

2. 增加惯性延时环节

在实际电路中用来消除险象的另一种方法是在组合电路输出端连接一个惯性延时环节。通常采用 RC 电路作惯性延时环节，如图 4.25(a)所示。由电路知识可知，图中的 RC 电路实际上是一个低通滤波器。由于组合电路的正常输出是一个频率较低的信号，而由竞争引起的险象都是一些频率较高的尖脉冲信号，因此，险象在通过 RC 电路后能基本被滤掉，保留下来的仅仅是一些幅度极小的毛刺，它们不再对电路的可靠性产生影响。图 4.25(b)表明了这种方法的效果。

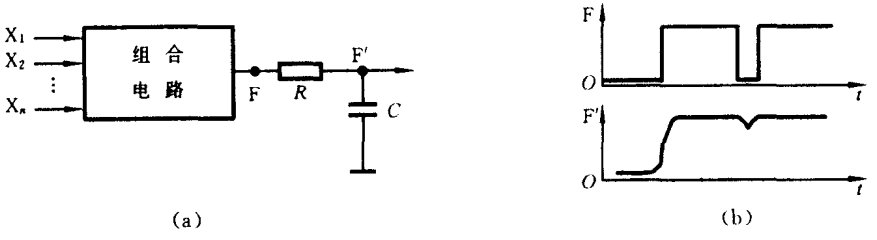


图 4.25　惯性延时环节

要注意的是：采用这种方法时，必须适当选择惯性环节的时间常数($\tau=RC$)，一般要求 τ 大于尖脉冲的宽度，以便能将尖脉冲“削平”；但也不能太大，否则将使正常输出信号产生不允许的畸变。

3. 选通法

上面介绍了用增加冗余项或增加惯性延时环节消除险象的方法。这两种方法的缺点是要增加器件。对于组合电路中的险象除了用上述方法消除外，还可以采用另外一种完全不同的方法，那就是避开险象而不是消除险象。选通法不必增加任何器件，仅仅是利用选通脉冲的作用，从时间上加以控制，以避开险象脉冲。

由于组合电路中的险象总是发生在输入信号发生变化的过程中，且险象总是以尖脉冲的形式输出。因此，只要对输出波形从时间上加以选择和控制，利用选通脉冲选择输出波形的稳定部分，而有意避开可能出现的尖脉冲，便可获得正确的输出。

例如，图 4.26 所示与非门电路的输出函数表达式为

$$F=\overline{\overline{A\cdot 1}\cdot\overline{\overline{A}\cdot 1}}=A+\overline{A}$$

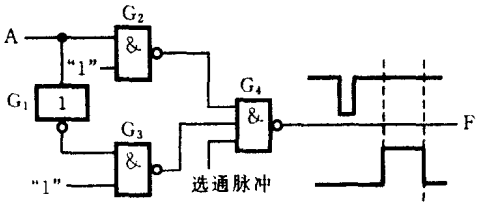


图 4.26 用选通法避开险象原理图

当 A 发生变化时，可能产生"0"型险象。

为了避开险象，可采用选通脉冲对该电路的输出门加以控制。在选通脉冲到来之前，选通控制线为低电平，门 G_4 关闭，电路输出被封锁，使险象脉冲无法输出。当选通脉冲到来后，门 G_4 开启，使电路送出稳定输出信号。

通常把这种在时间上让信号有选择地通过的方法称为选通法。

习 题 四

4.1 分析图 4.27 所示的组合逻辑电路，说明电路功能，并画出其简化逻辑电路图。

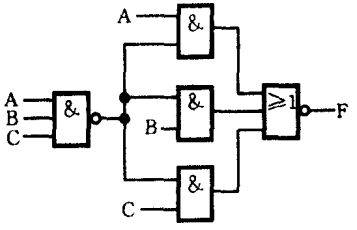


图 4.27 组合逻辑电路

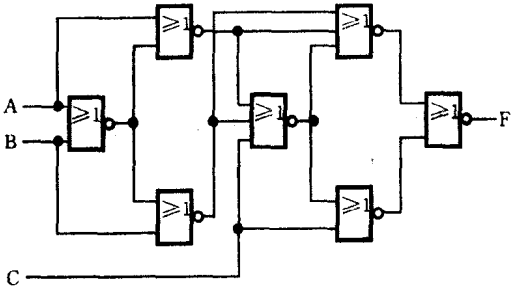


图 4.28 逻辑电路

4.2 分析图 4.28 所示的逻辑电路：

(1) 指出在哪些输入取值下，输出 F 的值为 1。

(2) 改用异或门实现该电路的逻辑功能。

4.3　分析图 4.29 所示组合逻辑电路，列出真值表，说明该电路的逻辑功能。

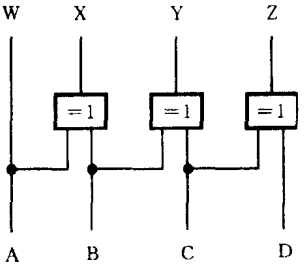


图 4.29　组合逻辑电路

4.4　设计一个组合电路，该电路输入端接收两个 2 位二进制数 $A=A_2A_1$，$B=B_2B_1$。当 $A>B$ 时，输出 $Z=1$，否则 $Z=0$。

4.5　设计一个代码转换电路，将 1 位十进制数的余 3 码转换成 2421 码。

4.6　假定 $X=AB$ 代表一个 2 位二进制数，试设计满足如下要求的逻辑电路（Y 也用二进制数表示）：

(1) $Y=X^2$　　(2) $Y=X^3$

4.7　用与非门设计一个组合电路，该电路输入为 1 位十进制数的 2421 码，当输入的数字为素数时，输出 F 为 1，否则 F 为 0。

4.8　设计一个“四舍五入”电路。该电路输入为 1 位十进制数的 8421 码，当其值大于或等于 5 时，输出 F 的值为 1，否则 F 的值为 0。

4.9　设计一个检测电路，检测 4 位二进制码中 1 的个数是否为偶数。若为偶数个 1，则输出为 1，否则输出为 0。

4.10　设计一个加/减法器，该电路在 M 控制下进行加、减运算。当 $M=0$ 时，实现全加器功能；当 $M=1$ 时，实现全减器功能。

4.11　在输入不提供反变量的情况下，用与非门组成电路实现下列函数：

(1) $F=A\overline{B}+\overline{A}C+B\overline{C}$

(2) $F=A\overline{B}\overline{C}+BC\overline{D}+A\overline{C}\overline{D}+\overline{B}CD$

4.12　下列函数描述的电路是否可能发生竞争？竞争结果是否会产生险象？在什么情况下产生险象？若产生险象，试用增加冗余项的方法消除。

(1) $F_1=AB+A\overline{C}+\overline{C}D$

(2) $F_2=AB+\overline{A}CD+BC$

(3) $F_3=(A+\overline{B})(\overline{A}+\overline{C})$

第5章

同步时序逻辑电路

时序逻辑电路是不同于组合逻辑电路的另一类常用逻辑电路。组合逻辑电路在任何时刻产生的稳定输出信号都仅与该时刻电路的输入信号相关；而时序逻辑电路在任何时刻产生的稳定输出信号不仅与电路该时刻的输入信号有关，而且与电路过去的输入信号有关。

时序逻辑电路按其工作方式的不同，又分为同步时序逻辑电路和异步时序逻辑电路两种类型。本章主要讨论同步时序逻辑电路的分析和设计方法。

5.1 时序逻辑电路概述

5.1.1 时序逻辑电路的结构

由于时序逻辑电路的输出不仅取决于当时的输入，而且还与电路过去的输入有关，因此，电路必须具有记忆功能，以便保存过去的输入信息。时序逻辑电路的一般结构如图 5.1 所示，它由组合电路和存储电路两部分组成，通过反馈回路将两部分连成一个整体。

图中，$x_1,\cdots,x_n$为时序逻辑电路的输入信号，又称为组合电路的外部输入信号；$Z_1,\cdots,Z_m$为时序逻辑电路的输出信号，又称为组合电路的外部输出信号；$y_1,\cdots,y_r$为时序逻辑电路的“状态”，又称为组合电路的内部输入信号；$Y_1,\cdots,Y_r$为时序逻辑电路中的激励信号，又称为组合电路的内部输出信号，激励

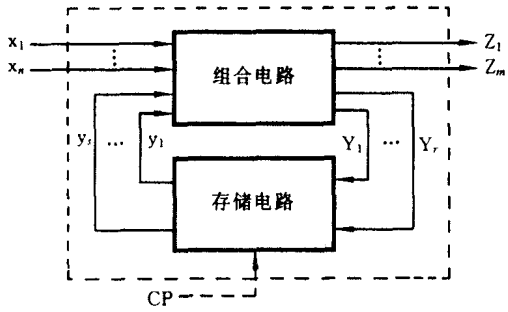


图 5.1 时序逻辑电路的一般结构

信号决定电路下一时刻的状态；CP 为时钟脉冲信号，该信号是否存在取决于时序逻辑电路的类型。

时序逻辑电路的状态 $y_1,\cdots,y_i$ 是存储电路对过去输入信号记忆的结果，它随着外部信号的作用而变化。在对电路功能进行研究时，通常将某一时刻的状态称为“现态”，记 y^n，简记为 y；而把在某一现态下，外部信号发生变化时即将到达的新的状态称为“次态”，记作 y^{n+1}。

从电路结构可知，时序逻辑电路具有如下特征：

① 电路由组合电路和存储电路组成，具有对过去输入进行记忆的功能；

② 电路中包含反馈回路，通过反馈使电路功能与“时序”相关；

③ 电路的输出由电路当时的输入和状态（对过去输入记忆的结果）共同决定。

5.1.2　时序逻辑电路的分类

时序逻辑电路通常可以按照电路的工作方式、电路输出对输入的依从关系或者输入信号的形式进行分类。

1. 按照电路的工作方式分类

按照电路的工作方式，可分为同步时序逻辑电路（简称同步时序电路）和异步时序逻辑电路（简称异步时序电路）两种类型。同步时序电路的存储电路由带有时钟控制端的触发器组成，各触发器的时钟端均与统一的时钟脉冲信号（CP）相连接，各触发器状态的改变受到同一时钟信号的控制。仅当有时钟脉冲到来时，电路状态才可能发生转换，而且每一个时钟脉冲只允许状态改变一次。若时钟脉冲没有到来，则任何输入信号的变化都不可能引起电路状态的改变。因此，时钟脉冲信号对电路状态的变化起着同步的作用。在研究同步时序逻辑电路时，通常不把同步时钟信号作为输入信号处理，而是将它当成一种默认的时间基准。若把某个时钟脉冲到来前电路所处的状态作为现态，则该时钟脉冲作用后电路的状态便称为次态。值得注意的是，前一个脉冲的次态即后一个脉冲的现态。现态与次态之间的相对关系如图 5.2 所示。

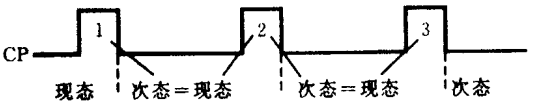


图 5.2　现态与次态之间的相对关系

为了使同步时序逻辑电路稳定、可靠地工作，对时钟脉冲的宽度和频率有一定

要求，脉冲的宽度必须保证触发器可靠翻转，脉冲的频率则必须保证在前一个脉冲引起的电路响应完全结束后，后一个脉冲才能到来。否则，电路状态的变化将发生混乱。

异步时序逻辑电路的存储电路由触发器或延时元件组成，电路中无统一的时钟信号同步，电路输入信号的变化将直接导致电路状态的变化。

2. 按照电路的输出/输入关系分类

根据电路的输出是否与输入直接相关，可以分为Mealy型和Moore型两种不同模型的时序逻辑电路。若时序逻辑电路的输出是电路输入和电路状态的函数，则称为Mealy型时序逻辑电路；若时序逻辑电路的输出仅仅是电路状态的函数，则称为Moore型时序逻辑电路。换而言之，Mealy 型电路是将过去的输入转换成状态后与输出建立联系，当前的输入直接和输出建立联系；而 Moore 型电路则是将全部输入转换成电路状态后再和输出建立联系。两种模型的输出/输入关系如图 5.3 所示。

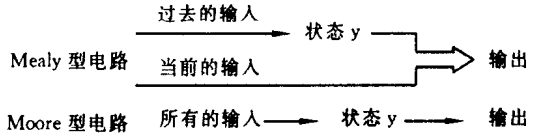


图 5.3 两种模型的输出/输入关系图

若一个时序逻辑电路没有专门的外部输出信号，而是以电路状态作为输出，则可视为 Moore 型电路的特殊情况。无论是同步时序逻辑电路或是异步时序逻辑电路，均有 Mealy 型和 Moore 型两种模型。同步时序逻辑电路中两种模型的结构框图如图 5.4 所示。

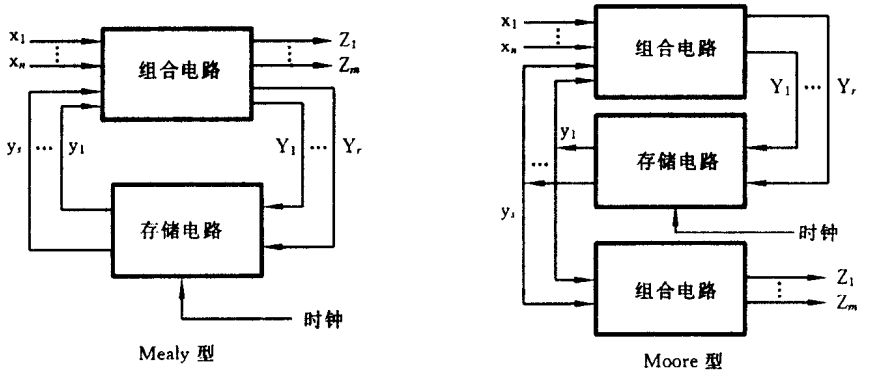


图 5.4 同步时序逻辑电路中两种模型的结构框图

3. 按输入信号的形式进行分类

时序逻辑电路的输入信号可以是脉冲信号也可以是电平信号。根据输入信号形式的不同，时序逻辑电路通常又被分为脉冲型和电平型两种类型。图 5.5 所示为不同输入信号的波形图。

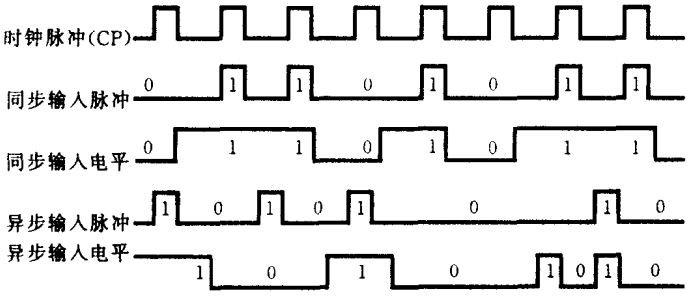


图 5.5　时序逻辑电路的输入信号波形

5.1.3　同步时序逻辑电路的描述方法

要对一种逻辑电路进行研究，必须首先对它进行描述。在研究组合逻辑电路时，为了描述一个电路的功能，除采用逻辑函数表达式外，通常借助真值表、卡诺图进行分析和设计。而研究同步时序电路时，除逻辑函数表达式之外，一般采用状态表、状态图去描述一个电路的逻辑功能。状态表和状态图是同步时序逻辑电路分析和设计的重要工具。此外，必要时还可以通过时间图加以描述。

1. 逻辑函数表达式

要完整地描述同步时序逻辑电路的结构和功能，须用 3 组逻辑函数表达式。

(1) 输出函数表达式

输出函数表达式是反映电路输出 Z 与输入 x 和状态 y 之间关系的表达式。

对于 Mealy 型电路，其函数表达式为

$$Z_i=f_i(x_1,\cdots,x_n,y_1,\cdots,y_r) \qquad i=1,2,\cdots,m$$

对于 Moore 型电路，其函数表达式为

$$Z_i=f_i(y_1,\cdots,y_r) \qquad i=1,2,\cdots,m$$

(2) 激励函数表达式

激励函数又称为控制函数，它反映了存储电路的输入 Y(组合电路内部输出)与电路输入 x 和状态 y 之间的关系。其函数表达式为

$$Y_j=g_j(x_1,\cdots,x_n,y_1,\cdots,y_s) \qquad j=1,2,\cdots,r$$

(3) 次态函数表达式

次态函数用来反映同步时序逻辑电路的次态 y^{n+1} 与激励函数 Y 和电路现态 y 之间的关系，它与触发器类型相关。其函数表达式为

$$y_l^{n+1}=k_l(Y_j,y_l) \qquad j=1,2,\cdots,r \qquad l=1,2,\cdots,s$$

对于任何一个同步时序逻辑电路，一旦上述 3 组函数被确定，则其逻辑功能便被惟一确定。

2. 状态表

由于同步时序逻辑电路的输出不仅取决于当前的输入，同时还取决于过去的输入，即与输出相关的输入并不发生在同一时刻。因此，不可能像组合逻辑电路那样用简单的真值表来描述其逻辑功能。

一种能够完全描述同步时序逻辑电路逻辑功能的表格形式是状态转移表，简称状态表。它是一张反映同步时序逻辑电路输出 Z、次态 y^{n+1} 和电路输入 x、现态 y 之间关系的表格。对于同步时序逻辑电路的两种模型，其状态表的格式略有区别。

Mealy 型同步时序逻辑电路状态表的格式如表 5.1 所示。表格的上方从左到右列出一位输入 x 的全部组合，表格左边从上到下列出电路的全部状态 y，表格的中间列出对应不同输入组合和现态下的次态 y^{n+1} 和输出 Z。

表 5.1 Mealy 型电路状态表格式

现 态	次态/输出		
		输入 x	
y		y^{n+1}/Z	

表 5.2 Moore 型电路状态表格式

现 态	次 态			输 出
		输入 x		
y		y^{n+1}		Z

Moore 型电路状态表的格式如表 5.2 所示。考虑到 Moore 型电路的输出 Z 仅与电路的现态 y 有关，为了清晰起见，将输出单独作为一列，其值完全由现态确定。至于次态 y^{n+1}，依然和 Mealy 型电路状态表中一样，由输入的取值组合和现态共同确定。状态表是同步时序逻辑电路分析和设计中常用的工具，它非常清晰地给出了同步时序电路在不同输入和现态下的次态和输出。

3. 状态图

状态图是一种反映同步时序逻辑电路状态转换规律及相应输入/输出取值关系的有向图。图中用带圆圈的字符表示电路的状态，连接圆圈的有向线段表示状

态的转换关系，箭头的起点表示现态，终点表示次态。Mealy 型电路状态图的形式如图 5.6(a)所示。图中，在有向箭头的旁边标出发生该转换的输入条件以及在该输入和现态下的相应输出。Moore 型电路状态图的形式如图 5.6(b) 所示，除了把电路输出标在圆圈内的状态右下方之外，其他和 Mealy 型电路相同。若某一箭头起止于同一状态，则表明在指定输入下状态保持不变。

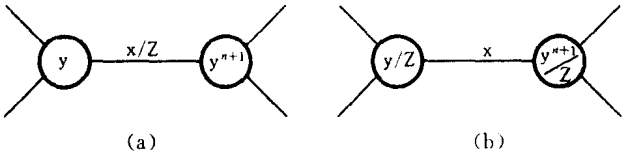


图 5.6 Mealy 型和 Moore 型电路的状态图形式

用状态图描述同步时序逻辑电路的逻辑功能具有直观、形象等优点。它和状态表一样，是同步时序电路分析和设计的重要工具。

4. 时间图

时间图是用波形图的形式来表示输入信号、输出信号和电路状态等的取值在各时刻的对应关系，通常又称为工作波形图。在时间图上，可以把电路状态转换的时刻形象地表示出来，这是前几种方法所不能做到的。关于时间图的绘制，将在后面的分析和设计中结合实例进行介绍。

5.2 同步时序逻辑电路分析

所谓时序逻辑电路分析，就是对一个给定的时序逻辑电路，研究在一系列输入信号作用下，电路将会产生怎样的输出，进而说明该电路的逻辑功能。

同步时序逻辑电路的主要工作特点是，随着时间的推移和外部输入的不断变化，在时钟脉冲作用下电路的状态和输出将发生相应变化。因此，分析的关键是找出电路状态和输出随输入变化而变化的规律，以便确定其逻辑功能。本节将举例介绍两种常用的分析方法。

5.2.1 分析方法和步骤

分析同步时序逻辑电路有两种常用的方法，一种是表格法，另一种是代数法。其分析过程如图 5.7 所示。

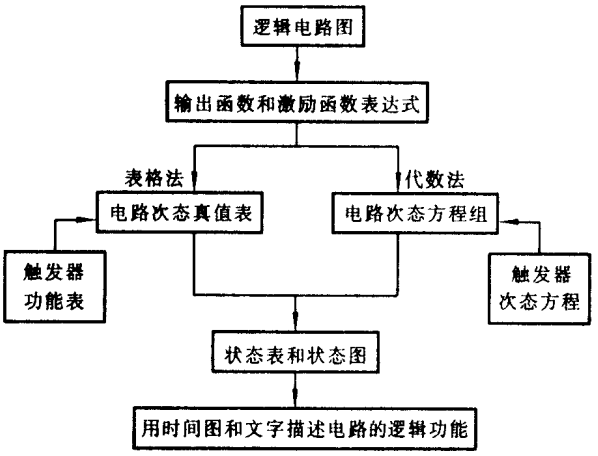


图 5.7 同步时序逻辑电路的分析过程

1. 表格分析法的一般步骤

① 根据给定的同步时序逻辑电路，写出输出函数和激励函数表达式。

② 列出电路次态真值表。根据输入和现态在各种取值下的激励函数值以及触发器的功能表，确定电路的相应次态。它反映了电路在不同时刻和不同输入下的状态转移关系。

③ 根据次态真值表和输出函数表达式，作出给定电路的状态表和状态图。

④ 拟定一典型输入序列画出时间图，并用文字描述电路的逻辑功能。

2. 代数分析法的一般步骤

① 根据给定的同步时序逻辑电路，写出输出函数和激励函数表达式。

② 把激励函数表达式代入触发器的次态方程，导出电路的次态方程组。

③ 根据次态方程组和输出函数表达式，作出给定电路的状态表和状态图。

④ 拟定一典型输入序列画出时间图，并用文字描述电路的逻辑功能。

由分析步骤可知，两种方法大同小异，分析中可视具体问题灵活选用。

5.2.2 分析举例

例 5.1 用表格法分析如图 5.8 所示的同步时序逻辑电路。

解 由图 5.8 可以看出，该电路的存储电路是两个 J-K 触发器，组合电路是一个异或门，电路的输入为 x，电路的状态(即触发器状态)用 y_2、y_1 表示。该电路的

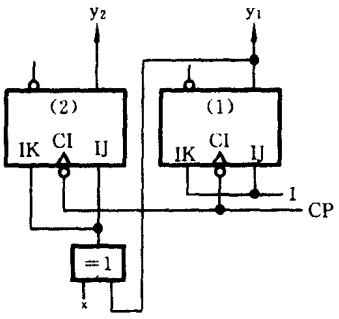


图 5.8　同步时序逻辑电路

表 5.3　次态真值表

输入	现态		激励函数				次态	
x	y_2	y_1	J_2	K_2	J_1	K_1	y_2^{n+1}	y_1^{n+1}
0	0	0	0	0	1	1	0	1
0	0	1	1	1	1	1	1	0
0	1	0	0	0	1	1	1	1
0	1	1	1	1	1	1	0	0
1	0	0	1	1	1	1	1	1
1	0	1	0	0	1	1	0	0
1	1	0	1	1	1	1	0	1
1	1	1	0	0	1	1	1	0

状态变量就是电路的输出，因此，它属于 Moore 型电路的特例。其分析过程如下。

① 写出输出函数表达式和激励函数表达式。该电路的输出即为状态，故只需写出激励函数表达式。由逻辑电路图可知，各触发器的激励函数表达式为

$$J_2=K_2=x\oplus y_1$$

$$J_1=K_1=1$$

② 列出电路次态真值表。次态真值表的填写方法是，首先依次列出电路输入和现态的所有取值组合；然后根据激励函数表达式，填写出每一组输入和现态取值下各激励函数的相应函数值；最后，根据表中的现态和激励函数值以及相应触发器的功能表填出每一组输入和现态取值下的次态。按此方法，可作出该电路的次态真值表如表5.3所示。

③ 作出状态表和状态图。根据表 5.3 所示的次态真值表，可作出该电路的状态表，如表 5.4 所示；状态图如图 5.9 所示。

表 5.4　状态表

现　态		次态　y_2^{n+1}　y_1^{n+1}	
y_2	y_1	x=0	x=1
0	0	0 1	1 1
0	1	1 0	0 0
1	0	1 1	0 1
1	1	0 0	1 0

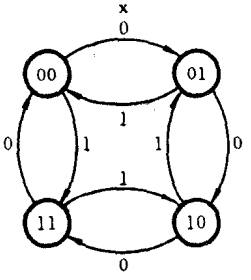
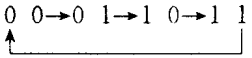


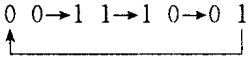
图 5.9　状态图

④ 用时间图和文字描述电路的逻辑功能。由状态图可以看出，图 5.8 所示电路是一个 2 位二进制数可逆计数器。

当电路输入 x=0 时，可逆计数器进行加 1 计数，其计数序列为



当输入 x=1 时，可逆计数器进行减 1 计数，其计数序列为



尽管状态表和状态图已描述了该同步时序电路的逻辑功能，但在时序逻辑电路分析过程中通常利用时间图对电路的工作过程作更深入地描述。时间图反映了时序电路在某一给定初始状态下，对典型输入序列的响应。这种描述虽然有其局限性，但由于能较形象、生动地体现时序电路的工作过程，并可和实验观察的波形相比较，因此是描述时序电路工作特性的一种常用方式。

作一个电路的时间图时，一般先假设电路初始状态，并拟定一典型输入序列；然后作出状态和输出响应序列；最后根据响应序列画出波形图。

设图 5.8 所示电路的初始状态为 $y_2y_1=00$，输入 x 为电平信号，典型输入序列为 111100000，则根据状态表或状态图可作出电路的状态响应序列如下：

CP	1	2	3	4	5	6	7	8	9
x	1	1	1	1	0	0	0	0	0
y_2	0	1	1	0	0	0	1	1	0
y_1	0	1	0	1	0	1	0	1	0
y_2^{n+1}	1	1	0	0	0	1	1	0	0
y_1^{n+1}	1	0	1	0	1	0	1	0	1

根据状态响应序列可作出时间图，如图 5.10 所示。由于现态和次态是针对具体时钟脉冲的作用而言的，前一个时钟脉冲的次态即为后一个时钟脉冲的现态，所以，时间图中可以将现态和次态共用一个波形表示。

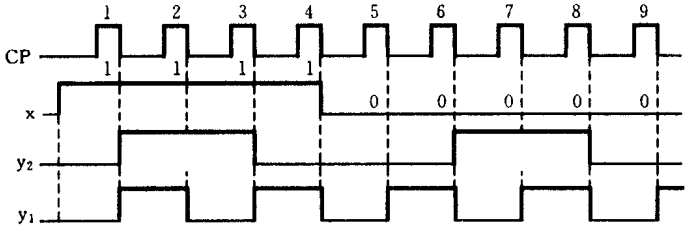


图 5.10 时间图

例 5.2 分析如图 5.11 所示的同步时序逻辑电路。

解 由图 5.11 可以知道，该电路的存储电路由两个 D 触发器构成，组合电路包括一个与门和一个或非门。电路有一个输入 x 和一个输出 Z。输出 Z 和输入 x 及电路状态均有直接联系，因此，属于 Mealy 型。采用表格法分析该电路的过程如下。

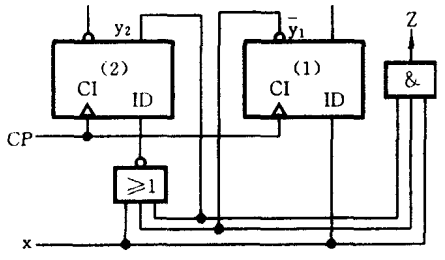


图 5.11　同步时序逻辑电路

表 5.5　次态真值表

输入	现态		激励函数		次态	
x	y_2	y_1	D_2	D_1	y_2^{n+1}	y_1^{n+1}
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	1	0	1
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	0	1	0	1

① 写出输出函数和激励函数的表达式。由逻辑电路图可知，输出函数和激励函数的表达式为

$$Z=xy_2\bar{y}_1 \qquad D_2=\overline{x+y_2+\bar{y}_1}=\bar{x}\bar{y}_2y_1 \qquad D_1=x$$

② 列出电路次态真值表。根据激励函数表达式和 D 触发器的功能表，可作出该电路的次态真值表如表 5.5 所示。

③ 作出状态表和状态图。根据输出函数表达式和表 5.5 所示次态真值表，可作出该电路的状态表，如表 5.6 所示；状态图如图 5.12 所示。

表 5.6　状态表

现态		次态/输出($y_2^{n+1}y_1^{n+1}$/Z)	
y_2	y_1	x=0	x=1
0	0	00/0	01/0
0	1	10/0	01/0
1	1	00/0	01/0
1	0	00/0	01/1

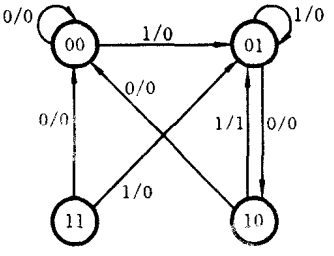


图 5.12　状态图

④ 作出时间图，并说明电路的逻辑功能。设电路初始状态为“00”，输入 x 为电平信号，其输入序列为 010110100。根据状态图可作出电路的状态响应序列和输出响应序列如下：

CP	1	2	3	4	5	6	7	8	9
x	0	1	0	1	1	0	1	0	0
y_2	0	0	0	1	0	0	1	0	1
y_1	0	0	1	0	1	1	0	1	0
y_2^{n+1}	0	0	1	0	0	1	0	1	0
y_1^{n+1}	0	1	0	1	1	0	1	0	0
Z	0	0	0	1	0	0	1	0	0

根据状态、输出对输入的响应序列，作出相应时间图，如图 5.13 所示。

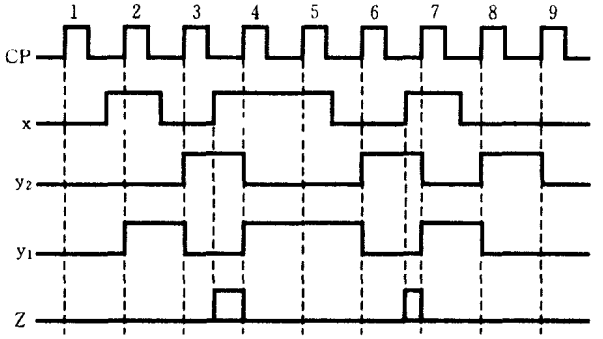


图 5.13 时间图

由时间图可以看出，一旦输入 x 出现信号“101”，输出 Z 便产生一个相应的 1，其他情况下输出 Z 为 0。因此，该电路是一个“101”序列检测器。

上面举例说明了采用表格法分析同步时序逻辑电路的过程，下面再通过实例了解另外一种分析方法，即代数分析法。

例 5.3 试用代数分析法分析图 5.14 所示同步时序逻辑电路，设触发器初始状态为“0”，说明该电路的逻辑功能。

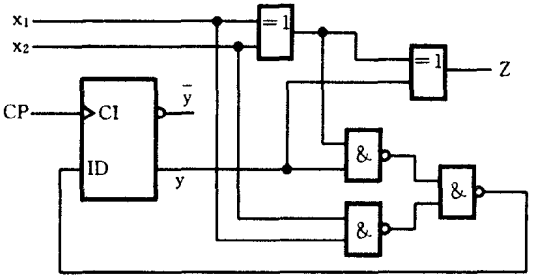


图 5.14 逻辑电路

解 该电路由一个 D 触发器和 5 个逻辑门构成，电路有两个输入端 x_1 和 x_2，一个输出端 Z。输出 Z 与输入和状态均有直接联系，属于 Mealy 型电路。分析该电路的过程如下。

① 写出输出函数和激励函数表达式：

$$Z=x_1\oplus x_2\oplus y \qquad D=x_1x_2+x_1y+x_2y$$

② 把激励函数表达式代入触发器的次态方程，得到电路的次态方程组。

该电路的存储电路只有一个触发器，因此，电路只有一个次态方程。根据 D 触发器的次态方程和电路的激励函数表达式，可导出电路的次态方程如下：

$$y^{n+1}=D=x_1x_2+x_1y+x_2y$$

③ 根据次态方程和输出函数表达式作出状态表和状态图。

根据次态方程和输出函数表达式，可以作出该电路的状态表，如表 5.7 所示；状态图如图 5.15 所示。

表 5.7　状态表

现态 y	次态/输出(y^{n+1}/Z)			
	$x_1 \wedge x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
0	0/0	0/1	1/0	0/1
1	0/1	1/0	1/1	1/0

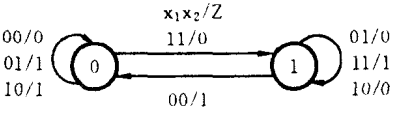


图 5.15　状态图

④ 画出时间图，并说明电路的逻辑功能。

设电路初态为“0”，输入 x_1 为 00110110，输入 x_2 为 01011100，根据状态图可作出电路的输出和状态响应序列如下：

时钟节拍	1	2	3	4	5	6	7	8
输入 x_1	0	0	1	1	0	1	1	0
输入 x_2	0	1	0	1	1	1	0	0
状态 y	0	0	0	0	1	1	1	1
输出 Z	0	1	1	0	0	1	0	1

根据状态响应序列可作出时间图，如图 5.16 所示。

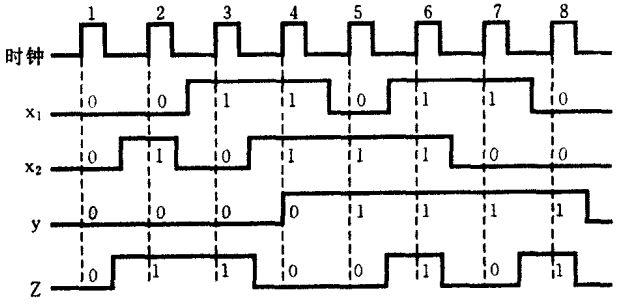


图 5.16　时间图

由时间图可知，该电路是一个串行加法器。电路输入 x_1 和 x_2 按照先低位后高位的顺序串行地输入为被加数和加数。每位相加产生的进位由触发器保存下来参加下一位相加，输出 Z 从低位到高位串行地输出“和”数。

该时间图给出了两个二进制数 $x_1=01101100$ 与 $x_2=00111010$ 相加得到“和”数 $Z=10100110$ 的过程。状态 $y=11110000$ 是由低位到高位依次产生的进位信号。

为了使逻辑功能更清晰，亦可按照左高右低的顺序将输入/输出序列表示如下：

时钟节拍	8	7	6	5	4	3	2	1
输入 x_1	0	1	1	0	1	1	0	0
输入 x_2	0	0	1	1	1	0	1	0
状态 y	1	1	1	1	0	0	0	0
输出 Z	1	0	1	0	0	1	1	0

例 5.4 分析图 5.17 所示同步时序逻辑电路。

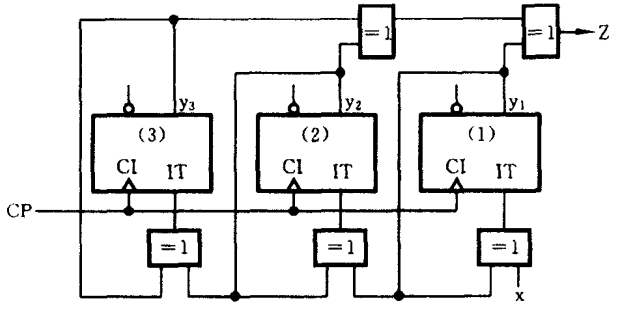


图 5.17 逻辑电路

解 该电路的存储电路部分由 3 个 T 触发器组成，组合电路部分包含 5 个异或门。电路输入为 x，输出为 Z，输出 Z 与输入 x 没有直接关系，故该电路属于 Moore型电路。用代数法分析该电路的过程如下。

① 写出输出函数和激励函数表达式。由逻辑电路图可知，该电路的输出函数和激励函数表达式分别为

$$Z=y_3\oplus y_2\oplus y_1$$

$$T_3=y_3\oplus y_2 \qquad T_2=y_2\oplus y_1 \qquad T_1=y_1\oplus x$$

② 写出电路的次态方程组。根据 T 触发器的次态方程 $Q^{n+1}=Q\oplus T$ 和电路的激励函数表达式，可求出电路的次态方程组如下：

$$y_3^{n+1}=y_3\oplus T_3=y_3\oplus y_3\oplus y_2=y_2$$

$$y_2^{n+1}=y_2\oplus T_2=y_2\oplus y_2\oplus y_1=y_1$$

$$y_1^{n+1}=y_1\oplus T_1=y_1\oplus y_1\oplus x=x$$

③ 作出电路的状态表和状态图。根据次态方程组和输出函数表达式，可作出该电路的状态表，如表 5.8 所示；状态图如图 5.18 所示。

④ 电路功能描述。由状态表和状态图可知，该电路是一个 3 位串行输入移位寄存器。输入 x 与寄存器低位相连，在时钟脉冲作用下，寄存器的内容从低往高左移一位，输入端 x 的信号置入寄存器最低位。输出 Z 用来指示所寄存的 3 位数据中含“1”的个数，当含有奇数个“1”时，输出为 1，否则输出为 0。

表 5.8　状态表

现　态	次态 $y_3^{n+1}y_2^{n+1}y_1^{n+1}$						输出
y_3 y_2 y_1	x=0			x=1			Z
0 0 0	0	0	0	0	0	1	0
0 0 1	0	1	0	0	1	1	1
0 1 0	1	0	0	1	0	1	1
0 1 1	1	1	0	1	1	1	0
1 0 0	0	0	0	0	0	1	1
1 0 1	0	1	0	0	1	1	0
1 1 0	1	0	0	1	0	1	0
1 1 1	1	1	0	1	1	1	1

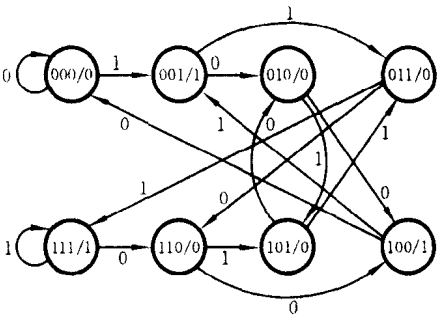


图 5.18　状态图

至此，已介绍了采用两种方法分析同步时序逻辑电路的全过程。值得指出的是，实际分析时根据给定逻辑电路的复杂程度不同，通常可以省去某些步骤。例如，列次态真值表或画时间图等，可视具体情况灵活运用，即应以充分理解电路功能为原则，而不是机械地执行全过程。

5.3　同步时序逻辑电路设计

同步时序逻辑电路设计是根据问题提出的功能要求，求出能正确实现预定功能的逻辑电路，又称为同步时序逻辑电路综合。设计追求的目标是使用尽可能少的触发器和逻辑门实现预定的逻辑功能。

5.3.1　设计的一般步骤

同步时序逻辑电路设计的一般步骤如图 5.19 所示。

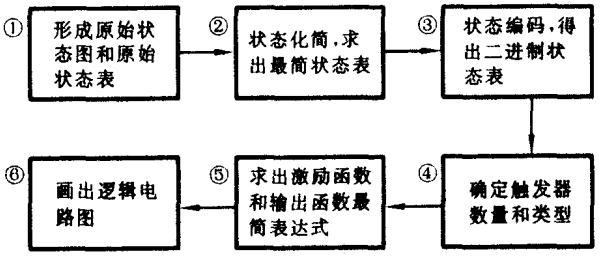


图 5.19　同步时序逻辑电路设计的一般步骤

1. 形成原始状态图和原始状态表

由于状态图和状态表能够直观、清晰、形象地反映同步时序电路的逻辑特性，所以设计的第一步是根据对设计要求的文字描述，抽象出电路的输入/输出及状态之间的关系，进而形成状态图和状态表。由于开始得到的状态图和状态表是对逻辑问题最原始的描述，其中可能包含多余的状态，所以称为原始状态图和原始状态表。

在建立原始状态图和原始状态表时，大部分问题对于所设立的每一个状态，在不同输入取值下都有确定的次态和输出，通常将这类状态图和状态表称为完全确定状态图和状态表，由它们所描述的电路称为**完全确定同步时序逻辑电路**；但实际应用中的某些问题，可能出现对于所设立的某些状态，其次态或输出是不确定的，即存在某些状态，它们在某些输入取值下的次态或输出是随意的。这种状态图和状态表被称为不完全确定状态图和状态表，所描述的电路称为**不完全确定同步时序逻辑电路**。

2. 状态化简

所谓状态化简是指采用某种化简技术从原始状态表中消去多余状态，得到一个既能正确地描述给定的逻辑功能，又能使所包含的状态数目达到最少的状态表，通常称这种状态表为最简状态表或最小化状态表。状态化简的目的是简化电路结构。状态数目的多少直接决定电路中所需触发器数目的多少。为了降低电路的复杂性和电路成本，应尽可能使状态表中包含的状态数达到最少。

3. 状态编码

状态编码是指给最简状态表中用字母或数字表示的状态，指定一个二进制代码，将其转换成二进制状态表，目的是过渡到与电路中触发器的状态对应。状态编码也称状态分配，或者状态赋值。

4. 确定触发器数目和类型

根据二进制状态表中二进制代码的位数确定电路中所需触发器数目，所需触发器数即二进制代码的位数。触发器类型可根据问题的要求确定，当问题中没有具体要求时，可由设计者挑选。

5. 确定激励函数和输出函数表达式

根据二进制状态表和所选触发器的激励表，求出触发器的激励函数表达式和电路的输出函数表达式，并予以化简。激励函数表达式和输出函数表达式的复杂度决定了同步时序电路中组合逻辑部分的复杂度。

6. 画出逻辑电路图

根据触发器数目和类型实现存储电路部分，根据激励函数和输出函数的最简表达式选择合适的逻辑门实现组合逻辑部分，画出完整的逻辑电路图。

以上步骤是就一般设计问题而言的。实际应用中设计者可以根据具体问题灵活掌握。例如，有的问题中对电路的状态数目和状态编码均已给定，因此，可省去状态化简和状态编码两个步骤。而有的设计方案中包含有冗余状态，因而在完成上述步骤后，还必须对这些状态的处理结果加以讨论，以确保电路逻辑功能的可靠实现等。总之，在实际设计过程中不必拘泥于固定的步骤。

此外，对于完全确定同步时序电路和不完全确定同步时序电路，设计过程中的某些步骤处理方法有所不同，下面对两类电路分别进行介绍。

5.3.2 完全确定同步时序逻辑电路设计

同步时序逻辑电路设计过程中最重要的步骤是形成原始状态图和原始状态表、状态化简、状态编码、确定激励函数和输出函数4个步骤。为此，分别对其方法进行专门讨论。

1. 形成原始状态图和原始状态表

原始状态图和原始状态表是对设计要求的最原始的抽象，是构造相应电路的原始依据。如果原始状态图不能正确地反映设计要求，则依此设计出来的电路必然是错误的。因此，建立正确的原始状态图和状态表是同步时序电路设计中最关键的一步。

原始状态图的形成是建立在对设计要求充分理解的基础之上的，设计者必须对给定的问题进行认真、全面地分析，弄清楚电路输出和输入的关系以及状态的转换关系。尽管建立原始状态图没有统一的方法，但一般应考虑如下几个方面的问题。

(1) 确定电路模型

同步时序电路有 Mealy 型和 Moore 型两种模型，具体设计成哪种模型的电路，有的问题由设计要求规定，有的问题可由设计者选择。不同模型对应的电路结构不同，设计时应根据问题中的信号形式、电路所需器件的多少等综合考虑。

(2) 设立初始状态

时序逻辑电路在输入信号开始作用之前的状态称为初始状态。同一个电路用不同状态作为初始状态时，对相同输入序列所产生的状态响应序列和输出响应序列一般是不相同的。因此，在建立原始状态图时，应首先设立初始状态，然后从初始状态出发考虑在各种输入作用下的状态转移和输出响应。

(3) 根据需要记忆的信息增加新的状态

同步时序电路中状态数目的多少取决于需要记忆和区分的信息量。在建立原始状态图时，切忌盲目地设立各种状态，而应该根据问题中要求记忆和区分的信息去考虑设立每一个状态。一般来说，当在某个状态下输入信号作用的结果能用已有的某个状态表示时，应转向相应的已有状态。仅当在某个状态下输入信号作用的结果不能用已有状态表示时，才令其转向新的状态。这样，从初始状态出发，随着输入信号的变化逐步完善，直到每个状态下各种输入取值均已考虑，而无需增加新的状态为止。

(4) 确定各时刻电路的输出

时序逻辑电路的功能是通过输出对输入的响应来体现的。因此，在建立原始状态图时，必须确定各时刻的输出值。在 Moore 型电路中，应指明每种状态下对应的输出；在 Mealy 型电路中则应指明从每一个状态出发，在不同输入作用下的输出值。

在描述逻辑问题的原始状态图和原始状态表中，状态数目不一定能达到最少，这一点无关紧要，因为可以对其再进行状态化简。设计者应把清晰、正确地描述设计要求放在第一位。其次，由于在开始时往往不知道描述一个给定的逻辑问题需多少状态，故在原始状态图和状态表中一般用字母或数字表示状态。

下面举例说明建立原始状态图和原始状态表的方法。

例 5.5 设计一个模 5 可逆计数器，该电路有一个输入 x 和一个输出 Z。输入 x 为加、减控制信号，当 x=0 时，计数器在时钟脉冲作用下进行加 1 计数，输出 Z 为进位输出信号；当 x=1 时，计数器在时钟脉冲作用下进行减 1 计数，输出 Z 为借位输出信号。试建立该计数器的 Mealy 型原始状态图和状态表。

解 该问题已指定电路模型为 Mealy 型，且状态数目以及输入和状态、输出之间的关系也非常清楚，所以状态图的建立很容易。假设计数器的 5 个状态分别用 0、1、2、3、4 表示，其中 0 为初始状态。根据题意可作出原始状态图，如图 5.20 所示；相应的原始状态表如表 5.9 所示。

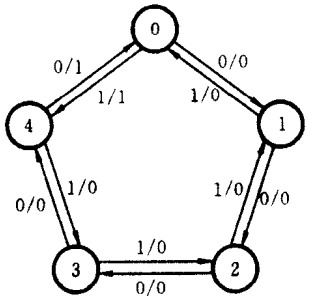


图 5.20 原始状态图

表 5.9 原始状态表

现态	次态/输出 Z	
	x=0	x=1
0	1/0	4/1
1	2/0	0/0
2	3/0	1/0
3	4/0	2/0
4	0/1	3/0

例 5.6　某序列检测器有一个输入端 x 和一个输出端 Z。从输入端 x 输入一串随机的二进制代码，当输入序列中出现 011 时，输出 Z 产生一个 1 输出，平时 Z 输出 0。典型输入、输出序列如下：

输入 x	1	0	1	0	1	1	1	0	0	1	1	0
输出 Z	0	0	0	0	0	1	0	0	0	0	1	0

试作出该序列检测器的原始状态图和原始状态表。

解　假定用 Mealy 型同步时序逻辑电路实现该序列检测器的逻辑功能，则原始状态图的建立过程如下。

设电路的初始状态为 A。当处在初始状态下电路输入为 0 时，输出 Z 为 0，由于输入 0 是序列“011”中的第一个信号，所以应该用一个状态将它记住，假定用状态 B 记住收到了第一个 0，则在状态 A 输入 0 时应转向状态 B；当处在初始状态 A 电路输入为 1 时，输出 Z 为 0，由于输入 1 不是序列“011”的第一个信号，故不需要记住，可令其停留在状态 A。该转换关系如图 5.21(a)所示。

当电路处于状态 B 时，若输入 x 为 0，则它不是序列“011”的第二个信号，但仍可作为序列中的第一个信号，故可令电路输出为 0，停留在状态 B；若输入 x 为 1，则意味着收到了序列“011”的前面两位 01，可用一个新的状态 C 将它记住，故此时电路输出为 0，转向状态 C。部分状态图如图 5.21(b)所示。

当电路处于状态 C 时，若输入 x 为 0，则收到的连续 3 位代码为 010，不是关心的序列 011，但此时输入的 0 依然可以作为序列的第一个信号，故此时应输出 0，转向状态 B；若输入 x 为 1，则表示收到了序列“011”，可用一个新的状态 D 记住，此

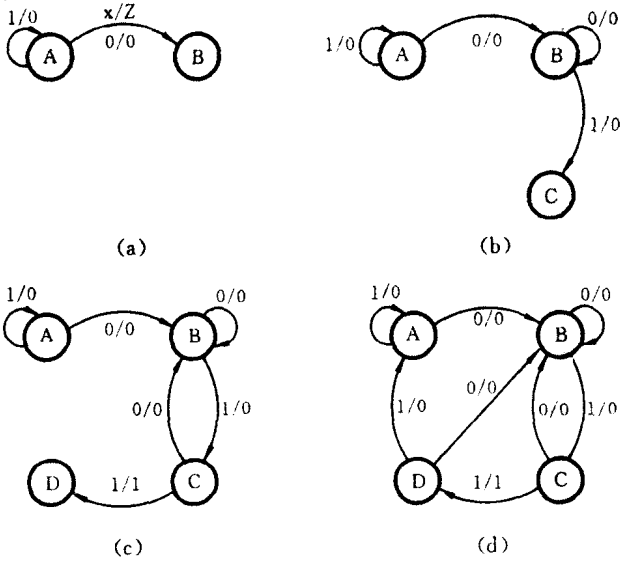


图 5.21　Mealy 型电路的原始状态图

时应输出 1，转向状态 D。部分状态图如图 5.21(c)所示。

当电路处于状态 D 时，若输入 x 为 0，则应输出 0，转向状态 B；若输入 x 为 1，则应输出 0，转向状态 A。至此，得到了该序列检测器完整的 Mealy 型状态图，如图 5.21(d)所示。相应的原始状态表如表5.10所示。

表 5.10 Mealy 型电路的状态表

现 态	次态/输出	
	x=0	x=1
A	B/0	A/0
B	B/0	C/0
C	B/0	D/1
D	B/0	A/0

从上述建立原始状态图的过程可知，实现一个序列检测器的功能所需要的状态数与要识别的序列长度相关，序列越长，需要记忆的代码位数越多，状态数也就越多。实际上在建立序列检测器的原始状态图时，可以先根据序列中要记忆的信息设立好每一个状态，并建立起当输入信号正好按指定序列变化时各状态的相互关系；然后再确定每个状态下输入出现不同取值时的输出和状态转移方向，即可得到一个完整的状态图。

若用 Moore 型同步时序电路实现“011”序列检测器的逻辑功能，则电路输出完全取决于状态，而与输入无直接联系。在作状态图时，应将输出标记在代表各状态的圆圈内。

假定电路初始状态为 A，并用状态 B、C、D 分别表示收到了输入 x 送来的 0、01、011。显然，根据题意，仅当处于状态 D 时电路输出为 1，其他状态下输出均为 0。当从初始状态开始，输入端 x 正好依次输入 0、1、1 时，则状态从 A 转至 B、B 转至 C、C 转至 D。据此可得到部分状态图如图 5.22(a)所示。然后，考虑到 A 状态下输入为 1 时，它不是指定序列中的第一位信号，不必记忆，可令状态停留在 A；B 状态下输入为 0 时，它不是指定序列的第二位，但可作为指定序列的第一位，故可令其停留在 B；C 状态下输入 0 时，它不是指定序列的第三位，但同样可作为第一位，故令其转向状态 B；D 状态下输入 0 时，同样应转向 B，而输入为 1 时，则应令其进入状态 A。完整的 Moore 型原始状态图如图 5.22(b)所示，相应的原始状态表如表 5.11 所示。

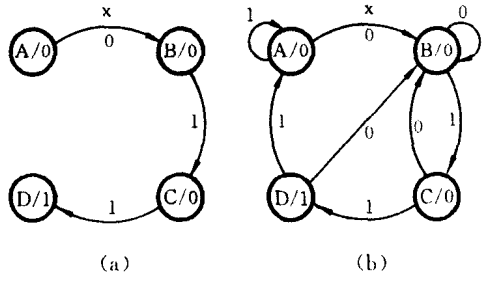


图 5.22 Moore 型电路的原始状态图

表 5.11 Moore 型电路的状态表

现 态	次 态		输 出
	x=0	x=1	Z
A	B	A	0
B	B	C	0
C	B	D	0
D	B	A	1

例 5.7　某同步时序电路用于检测串行输入的 8421 码，其输入的顺序是先低位后高位，当出现非法数字（即输入 1010，1011，1100，1101，1110，1111）时，电路的输出为 1。试作出该时序电路的 Mealy 模型状态图和状态表。

解　根据题意，电路有一个输入和一个输出。设输入 x 用来接收 8421 码，输出 Z 用来指示在输入端是否出现了非法数字。

假定起始状态为 A，当第一位输入代码到来时，有两种可能的情况，即 0 和 1，故需用两个状态 B 和 C 来表示这两种可能；从状态 B 和 C 出发，当 x 输入的第二位代码到来时，又各有两种可能的情况，即前两位代码共有 4 种不同组合，分别用状态 D，E，F，G 表示；从状态 D，E，F，G 出发，当输入 x 的第三位代码到来时，同样，各有两种可能，即前 3 位代码共有 8 种不同的组合，分别用状态 H，I，J，K，L，M，N，P 表示。当 x 输入的第四位代码到来时，一组 8421 码全部被接收。对输入的 8421 码进行判断，若出现非法数字，电路的输出为 1，否则为 0，并返回到起始状态 A。

根据以上分析，可以得到如图 5.23 所示的状态图。

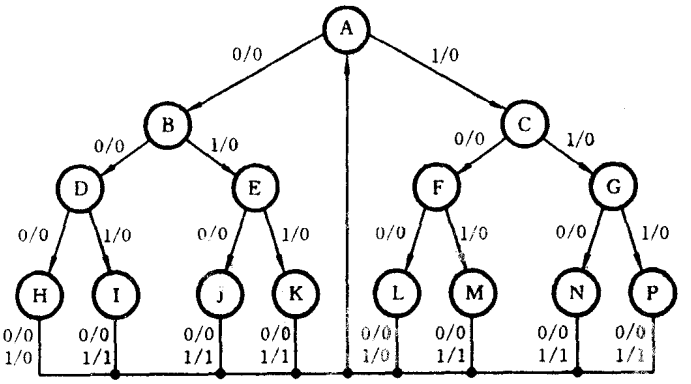


图 5.23　Mealy 型电路的原始状态图

从图 5.23 可看出，由于串行输入是先低位后高位，因此，判断输入的 8421 码是否为非法数字时，应从低位到高位查看各位输入值。待 4 位代码检测完后，则应转向初始状态 A，以便检查下一组代码。

将图 5.23 所示的状态图转换成状态表，如表 5.12 所示。

若将该代码检测器设计成 Moore 型同步时序电路，则电路输出只与状态相关。假定用状态 A 作为初始状态，用状态 B 和 C 表示代码最低位的取值 0 和 1；用状态 D、E、F、G 分别表示代码低二位的 4 种取值组合 00～11；用状态 H、I、J、K、L、M、N、P 分别表示低三位的 8 种取值组合 000～111；用状态 X 表示 4 位代码中的 10 种合法码，用状态 Y 表示 4 位代码中的 6 种非法码。显然，电路仅当处于状

态 Y 时输出为 1，其他状态下均输出 0。原始状态图如图 5.24 所示，原始状态表略。从这个例子可以看出，实现同一逻辑功能的 Moore 型电路比 Mealy 型电路需要的状态数多。

表 5.12 Mealy 型电路的原始状态表

现 态	次态/输出		现 态	次态/输出	
	x=0	x=1		x=0	x=1
A	B/0	C/0	I	A/0	A/1
B	D/0	E/0	J	A/0	A/1
C	F/0	G/0	K	A/0	A/1
D	H/0	I/0	L	A/0	A/0
E	J/0	K/0	M	A/0	A/1
F	L/0	M/0	N	A/0	A/1
G	N/0	P/0	P	A/0	A/1
H	A/0	A/0			

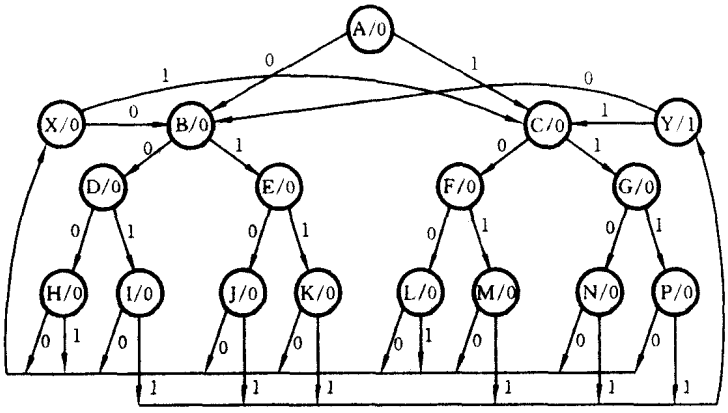


图 5.24 Moore 型电路的原始状态图

2. 状态化简

在建立原始状态图和原始状态表时，主要考虑如何清晰、正确地反映设计要求，而没有刻意追求如何使图、表中包含的状态数目达到最少。因此，在构造的原始状态图和原始状态表中往往存在多余状态。但在设计具体电路时，状态数目的多少将决定电路中所需触发器数目的多少。所以，为了降低电路的复杂性和电路成本，应尽可能地使描述设计要求的状态表中包含的状态数达到最少。

所谓状态化简，就是采用某种化简技术从原始状态表中消去多余状态，得到一

个既能正确地描述给定的逻辑功能，又能使所包含的状态数目达到最少的状态表，通常称这种状态表为最简状态表，又称为最小化状态表。

完全确定状态表的化简是建立在状态“等效”概念基础之上的。在讨论状态化简的具体方法和步骤之前，必须首先介绍化简时涉及的几个概念。

(1) 等效状态和等效类

1) 等效状态

定义　假设状态 S_i 和 S_j 是完全确定状态表中的两个状态，如果对于所有可能的输入序列，分别从 S_i 和 S_j 出发，所得到的输出响应序列完全相同，则状态 S_i 和 S_j 是等效的，记作 (S_i, S_j)，或者称状态 S_i 和 S_j 是“等效对”。

这里所说的所有可能的输入序列，是指输入序列的长度和结构是任意的，它包含无穷多位，且有无穷多种组合。如果企图通过检测所有可能输入序列下的输出来确定两个状态是否等效，显然是不现实的。事实上，由于在形成原始状态表时，对每个状态均考虑了在一位输入各种取值下的次态和输出，因此，从整体上讲，原始状态表已经反映了各状态在任意输入序列下的输出。从而，可以根据原始状态表上所列出的一位输入各种组合下的次态和输出来判断某两个状态是否等效。

判断方法　假定 S_i 和 S_j 是完全确定原始状态表中的两个现态，则 S_i 和 S_j 等效的条件可归纳为在一位输入的各种取值组合下满足如下两条。

第一，输出相同。

第二，次态属于下列情况之一：

a. 次态相同；

b. 次态交错或为各自的现态；

c. 次态循环或为等效对。

这里，情况 b 中所谓的次态交错，是指在某种输入取值下，S_i 的次态为 S_j，而 S_j 的次态为 S_i。而所谓次态为各自的现态，即 S_i 的次态仍为 S_i，S_j 的次态仍为 S_j。情况 c 中的次态循环是指确定两个状态是否等效的关联状态对之间，其依赖关系构成闭环，而两个次态为状态对循环体中的一个状态对。例如，S_1 和 S_2 在某种输入取值下的次态是 S_3 和 S_4，而 S_3 和 S_4 在该种输入取值下的次态又是 S_1 和 S_2，则称这种情况为次态循环。而次态为等效对是指 S_i 和 S_j 的次态已被确认为等效状态。例如，S_1 的次态是 S_3，S_2 的次态是 S_4，尽管 S_3 和 S_4 既不相同，也不交错或循环，但若以 S_3 和 S_4 作为现态，在一位输入的各种取值下，其输出相同且次态相同或交错或循环，即 S_3 和 S_4 等效，那么，S_1 和 S_2 是等效的。

性质　等效状态具有传递性。假若 S_1 和 S_2 等效，S_2 和 S_3 等效，那么，一定有 S_1 和 S_3 等效。记为

$$(S_1,S_2),(S_2,S_3)\rightarrow(S_1,S_3)$$

2) 等效类

所谓等效类是指由若干彼此等效的状态构成的集合。在一个等效类中的任意两个状态都是等效的。根据等效状态的传递性，可以根据状态等效对确定等效类。例如，由(S_1,S_2)和(S_2,S_3)可以推出(S_1,S_3)，进而可知S_1、S_2、S_3属于同一等效类，记为

$$(S_1,S_2),(S_2,S_3)\rightarrow\{S_1,S_2,S_3\}$$

等效类是一个广义的概念，两个状态或多个状态均可以组成一个等效类，甚至一个状态也可以称为等效类，因为任何状态和它的自身必然是等效的。

3) 最大等效类

所谓最大等效类，是指不被任何别的等效类所包含的等效类。这里所指的最大，并不是指包含的状态最多，而是指它的独立性，即使是一个状态，只要它不被包含在别的等效类中，也是最大等效类。换而言之，如果一个等效类不是任何其他等效类的子集，则该等效类称为最大等效类。

完全确定原始状态表的化简过程，就是寻找出表中的所有最大等效类，然后将每个最大等效类中的状态合并为一个新的状态，从而得到最简状态表的过程。最简状态表中的状态数等于原始状态表中最大等效类的个数。

常用的状态化简方法有观察法、输出分类法、隐含表法等，下面介绍最常用的一种方法——隐含表法。

(2) 利用隐含表进行状态化简

用隐含表法进行状态化简的一般步骤如图 5.25 所示。

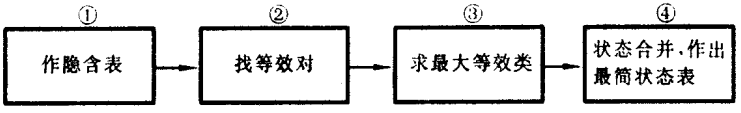


图 5.25 状态化简的一般步骤

① 作隐含表。

隐含表是一个等腰直角三角形阶梯网格，其横向和纵向的网格数相同，等于原始状态表中的状态数 n 减 1。隐含表中的方格是用状态名称来标注的，横向从左到右按原始状态表中的状态顺序依次标上第一个状态至倒数第二个状态的名称，纵向自上到下依次标上第二个状态至最后一个状态的名称。表中每个方格代表一个状态对。

② 寻找等效对。

利用隐含表寻找状态表中的全部等效对一般要进行两轮比较，首先进行顺序比较，然后进行关联比较。

所谓顺序比较是按照隐含表中从上至下、从左至右的顺序，对照原始状态表依次对所有状态对进行逐一检查和比较，并将检查结果以简单明了的方式标注在隐含表中的相应方格内。每个状态对的检查结果有3种情况，一是明确是等效的，在相应方格内填上"√"；二是明确是不等效的，在相应方格内填上"×"；三是与其他状态对相关，有待于进一步检查的，在相应方格内填上相关的状态对。

所谓关联比较是指对那些在顺序比较时尚未确定是否等效的状态对作进一步检查。关联比较时，首先应确定隐含表中待检查的那些次态对是否等效，然后由此确定原状态对是否等效。只要隐含表中某方格内列出的次态对中有一个不等效，则该方格所代表的状态对就不等效，于是在相应方格中增加标志"/"。若方格内的次态对均为等效对，则该方格所代表的状态对等效，该方格不增加任何标志。这种判别有时要反复多次，直到判别出状态对等效或不等效为止。

③ 求出最大等效类。

在找出原始状态表中的所有等效对之后，可利用等效状态的传递性，求出各最大等效类。确定各最大等效类时应注意两点：一是各最大等效类之间不应出现相同状态，因为若两个等效类之间有相同状态，则根据等效的传递性可令其合为一个等效类；二是原始状态表中的每一个状态都必须属于某一个最大等效类，换句话说，由各最大等效类所包含的状态之和必须覆盖原始状态表中的全部状态，否则，化简后的状态表不能描述原始状态表所描述的功能。

④ 作出最简状态表。

根据求出的最大等效类，将每个最大等效类中的全部状态合并为一个状态，即可得到和原始状态表等价的最简状态表。

下面举例说明化简过程。

例5.8　化简表5.13所示原始状态表。

解　表5.13所示为具有7个状态的原始状态表。用隐含表法化简如下。

① 作隐含表。根据画隐含表的规则，可得到与给定状态表对应的隐含表框架如图5.26所示。由于原始状态表中有A～G共7个状态，所以隐含表的横向和纵向各有6个方格。纵向从上到下依次为B～G，横向从左到右依次为A～F。表中每个方格代表一个状态对，如左上角的方格代表状态对A和B，右下角的方格代表状态对F和G。

表 5.13 原始状态表

现 态	次态/输出	
	x=0	x=1
A	C/0	B/1
B	F/0	A/1
C	F/0	G/0
D	D/1	E/0
E	C/0	E/1
F	C/0	G/0
G	C/1	D/0

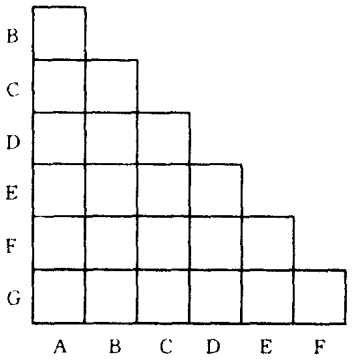


图 5.26 隐含表框架

② 寻找等效对。首先进行顺序比较，根据等效状态的判断标准，依次检查每个状态对，可得到顺序比较结果如图 5.27(a)所示。

例如，状态表中 C 和 F 满足状态等效条件，所以，在隐含表的相应方格内填入“√”；状态 A 和 C 不满足等效条件，故在隐含表的相应方格内填入“×”；状态 A 和 E 虽然满足输出相同这个条件，但它们的次态在 x=1 时分别为 B 和 E，由于当前尚不能确定 B 和 E 是否等效，因此，将 BE 填入相应方格中。

经过顺序比较后，还有 A 和 B，A 和 E，B 和 E 以及 D 和 G 共 4 个状态对尚未确定是否等效，故应接着进行关联比较，比较结果如图 5.27(b)所示。

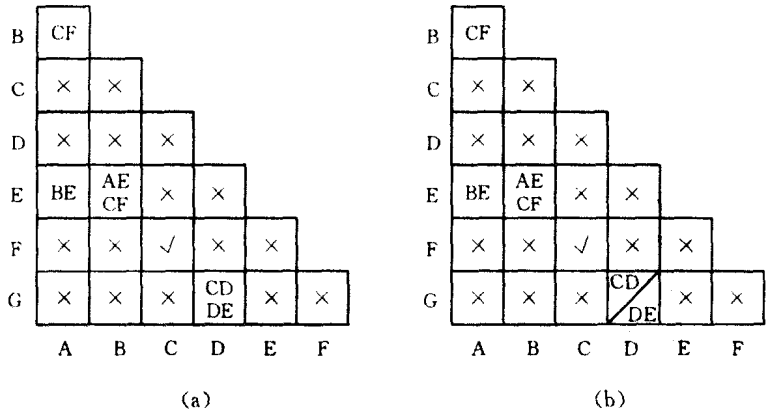
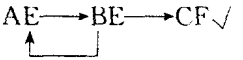


图 5.27 隐含表

例如，在图 5.27(a)所示隐含表中，状态 A、B 对应的方格中次态对为 CF，而状态 C、F 对应的方格标有“√”号，表明状态 C 和 F 等效，由此可判断出状态 A 和 B 等效。检查状态 A、E 的次态对，出现如下关系：



已知状态C和F是等效的，而状态BE又与状态AB构成循环，所以，状态A和E是等效状态对，B和E也是等效状态对。状态D、G对应的方格中含有CD和DE，而状态C、D对应的方格已标以“×”号，这表明状态C和D不等效。因此，可以判断状态D和G不等效，它所对应的方格应增加记号“/”。

由图5.27(b)所示的隐含表可知，原始状态表中的7个状态共有4个等效对：(A,B)，(A,E)，(B,E)，(C,F)。

③ 求出最大等效类。由所得到的4个等效对可知，等效对(A,B)，(A,E)，(B,E)构成一个最大等效类{A,B,E}。等效对(C,F)不包含在任何其他等效类中，所以，它也是一个最大等效类。其次，状态D和G不和任何其他状态等效，故它们各自构成一个最大等效类。由此可见，原始状态表中的7个状态共构成4个最大等效类，分别表示如下：

{A,B,E}，{C,F}，{D}，{G}

④ 作出最小化状态表。将最大等效类{A,B,E}、{C,F}、{D}、{G}分别用新的字母a、b、c、d表示，并对表5.13所示状态表中的状态作相应取代，即可得到化简后的最小化状态表，如表5.14所示。

表5.14　最小化状态表

现　态	次态/输出	
	x=0	x=1
a	b/0	a/1
b	b/0	d/0
c	c/1	a/0
d	b/1	c/0

3. 状态编码

所谓状态编码，是指给最简状态表中用字母或数字表示的状态，指定一个二进制代码，形成二进制状态表。状态编码也称状态分配，或者状态赋值。

在得到最简状态表后，必须将该状态表中的状态用二进制代码表示，以便和电路中触发器的状态对应。一般情况下，采用的状态编码方案不同，所得到的输出函数和激励函数的表达式也不同，从而设计出来的电路的复杂程度也不同。状态编码的任务是：

① 确定二进制代码的位数(即所需触发器个数)；

② 寻找一种最佳的或接近最佳的状态分配方案，以便使所设计的时序电路最简单。

状态编码的长度是由最简状态表中的状态个数来确定的。设最简状态表中的状态数为n，二进制代码的长度为m，则状态数n与二进制代码长度m的关系为

$$2^{m-1}<n\leqslant 2^m$$

根据已知状态数，就可求出状态分配所需的二进制代码的位数。例如，若某状态表的状态数$n=4$，则在进行状态分配时，二进制代码的位数应为$m=2$，或者说

状态变量个数为 2。

在二进制代码的位数确定之后，具体状态与代码之间的对应关系可以有许多种状态分配方案。一般说来，在用 m 位二进制代码的 2^m 种组合来对 n 个状态进行分配时，可能出现的状态分配方案数 K_s 为

$$K_s = A_{2^m}^{n} = \frac{2^m!}{(2^m-n)!}$$

例如，当 $n=4$，$m=2$ 时，有 24 种不同的分配方案。随着状态数目的增加，分配方案的数目急剧增加。如何从众多的分配方案中寻找出一种最佳方案，使所设计的电路最简单，是一件十分困难的事情。而且，分配方案的好坏还与所采用的触发器类型相关，即一种分配方案对某种触发器是最佳的，但对另一种触发器则不一定是最佳的。因此，状态分配的问题是一个比较复杂的问题。尽管做了大量研究工作，但从理论上讲，寻求最佳状态编码的问题还没有完全解决。

在实际工作中，工程技术人员通常按照一定的原则、凭借设计经验去寻找相对最佳的编码方案。一种常用的编码方法称为相邻编码法，这是一种比较直观、简单的方法。相邻编码法的基本思想是：在选择状态编码时，尽可能有利于激励函数和输出函数的化简。

相邻编码法的状态编码原则如下：

① 在相同输入条件下，具有相同次态的现态应尽可能分配相邻的二进制代码；

② 在相邻输入条件下，同一现态的次态应尽可能分配相邻的二进制代码；

③ 输出完全相同的现态应尽可能分配相邻的二进制代码。

一般来说，上述 3 条原则在大多数情况下是有效的。但对于某些状态表常常出现不能同时满足 3 条原则的情况。此时，可按从①至③的优先顺序考虑，即把原则①放在首位。此外，从电路实际工作状态考虑，一般将初始状态分配为“0”状态。下面举例说明相邻法的应用。

例 5.9 对表 5.15 所示的状态表进行状态编码。

表 5.15 状态表

现 态	次态/输出	
	x=0	x=1
A	C/0	B/0
B	A/0	A/1
C	A/1	D/1
D	D/0	C/0

解 在表 5.15 所示的状态表中，共有 4 个状态，即 $n=4$，所以，状态编码的长度应为 $m=2$。也就是说，实现该状态表的功能需要两个触发器。设状态变量用 y_2 和 y_1 表示。

根据相邻法的编码原则，表中 4 个状态的相邻关系如下：

由原则①得到状态 B 和 C 应分配相邻的二进制代码；

由原则②得到状态 B 和 C、A 和 D、C 和 D 应分配相邻的二进制代码；

由原则③得到状态 A 和 D 应分配相邻的二进制代码。

综合原则①至原则③可知，状态分配时要求满足 B 和 C、A 和 D、C 和 D 相邻。

在进行状态分配时，为了使状态之间的相邻关系一目了然，通常用卡诺图作为状态分配的工具。假定将状态 A 分配“0”，即 A 的编码为 $y_2y_1=00$，一种满足上述相邻关系的分配方案如图 5.28 所示。即状态 A、B、C、D 的二进制代码依次为 y_2y_1 的取值 00、01、11、10。

将表 5.15 所示状态表中的状态 A、B、C、D 用各自的编码代替，即可得到该状态表的二进制状态表，如表 5.16 所示。

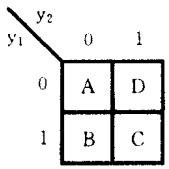


图 5.28　状态分配方案

表 5.16　二进制状态表

现　态		次态 $y_2^{n+1}y_1^{n+1}$/输出	
y_2	y_1	x=0	x=1
0	0	11/0	01/0
0	1	00/0	00/1
1	1	00/1	10/1
1	0	10/0	11/0

最后需要指出的是，通常满足分配原则的方案可以有多种，设计者可从中任选一种。

4. 确定激励函数和输出函数并画出逻辑电路图

在通过状态分配得到二进制状态表之后，设计同步时序电路的下一个步骤是根据二进制状态表和所选定的触发器的激励表或者次态方程，求出触发器的激励函数表达式和电路的输出函数表达式，并予以化简。最后，用适当的逻辑门和所选定的触发器构成实现给定逻辑功能的逻辑电路。

触发器的激励表是触发器从现态转移到某种次态时，对输入条件的要求。各种常用触发器的激励表在第 3 章已进行了介绍。表 5.17～表 5.20 列出了 4 种时钟控制触发器的激励表。

表 5.17　R-S 触发器激励表

Q	→	Q^{n+1}	R	S
0		0	d	0
0		1	0	1
1		0	1	0
1		1	0	d

表 5.18　D 触发器激励表

Q	→	Q^{n+1}	D
0		0	0
0		1	1
1		0	0
1		1	1

表 5.19　J-K 触发器激励表

Q	→	Q^{n+1}	J	K
0		0	0	d
0		1	1	d
1		0	d	1
1		1	d	0

表 5.20　T 触发器激励表

Q	→	Q^{n+1}	T
0		0	0
0		1	1
1		0	1
1		1	0

根据二进制状态表和触发器激励表，求激励函数和输出函数的最简表达式一般分为两步：首先列出激励函数和输出函数真值表；然后画出激励函数和输出函数卡诺图，化简后写出最简表达式。在十分熟练的情况下，也可以直接根据二进制状态表和触发器激励表，画出激励函数和输出函数卡诺图，进行化简后写出最简表达式。下面举例说明。

例 5.10 用 J-K 触发器和适当的逻辑门实现表 5.21 所示二进制状态表的功能。

解 根据给定的二进制状态表和 J-K 触发器的激励表可列出激励函数和输出函数的真值表，如表 5.22 所示。

表 5.21 二进制状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z	
		x=0	x=1
0	0	11/0	01/0
0	1	00/0	00/1
1	1	00/1	10/1
1	0	01/0	11/0

表 5.22 激励函数和输出函数真值表

输入和现态			激励函数				输出函数
x	y_2	y_1	J_2	K_2	J_1	K_1	Z
0	0	0	1	d	1	d	0
0	0	1	0	d	d	1	0
0	1	0	d	1	1	d	0
0	1	1	d	1	d	1	1
1	0	0	0	d	1	d	0
1	0	1	0	d	d	1	1
1	1	0	d	0	1	d	0
1	1	1	d	0	d	1	1

由真值表可作出激励函数和输出函数的卡诺图，如图 5.29 所示。

经化简后得到激励函数和输出函数的最简表达式如下：

$$J_2=\bar{x}\bar{y}_1 \qquad J_1=K_1=1$$

$$K_2=\bar{x} \qquad Z=y_2y_1+xy_1=(y_2+x)y_1$$

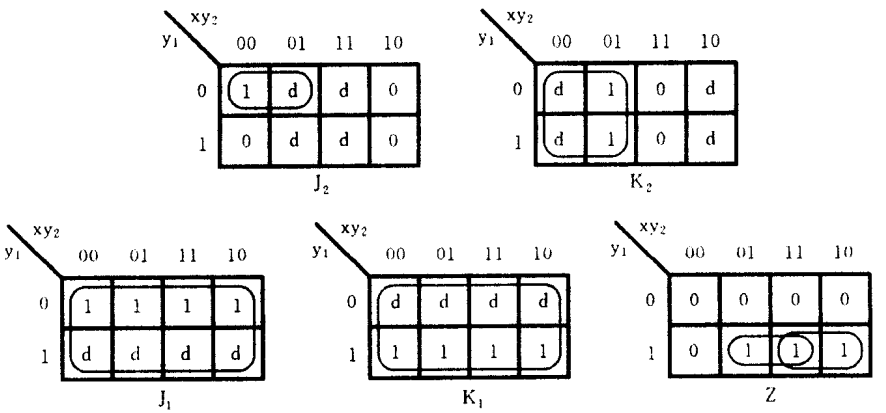


图 5.29 激励函数和输出函数卡诺图

其逻辑电路如图 5.30 所示。

如果选用 D 触发器作为存储元件，实现表 5.21 所示二进制状态表的逻辑功能，那么，根据 D 触发器的激励表(D 与次态相同)和给定二进制状态表，可直接作出激励函数卡诺图，如图 5.31 所示。

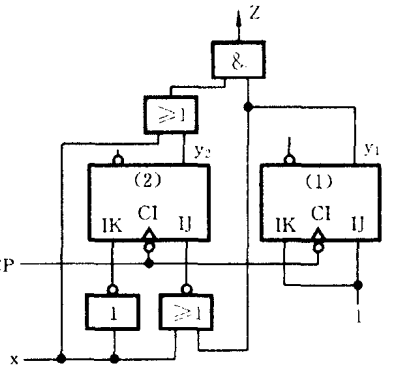


图 5.30　逻辑电路

根据卡诺图进行化简后，得到激励函数的最简表达式为

$$D_2=\bar{x}\,\bar{y}_2\bar{y}_1+xy_2=\overline{x+y_2+y_1}+xy_2$$

$$D_1=\bar{y}_1$$

其逻辑电路如图 5.32 所示。

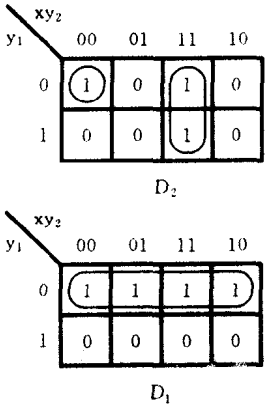


图 5.31　采用 D 触发器的激励函数卡诺图

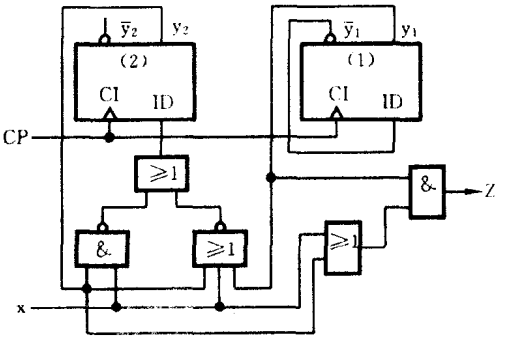


图 5.32　采用 D 触发器的逻辑电路

另外，激励函数表达式也可以根据二进制状态表和触发器的次态方程确定。具体方法是：首先根据二进制状态表作出次态卡诺图，求出次态函数表达式；然后将次态函数表达式与所用触发器的次态方程相比较，确定激励函数表达式。

例如，用 J-K 触发器实现表 5.21 功能时，激励函数表达式亦可利用触发器的次态方程确定。J-K 触发器的次态方程为

$$Q^{n+1}=J\bar{Q}+\bar{K}Q \tag{5-1}$$

根据表 5.21 作出次态卡诺图(次态 y_2^{n+1}、y_1^{n+1} 的卡诺图与图 5.31 中所示的 D_2、D_1 的卡诺图相同)，可求出次态函数表达式为

$$y_2^{n+1}=\bar{x}\bar{y}_2\bar{y}_1+xy_2=\bar{x}\bar{y}_1\cdot\bar{y}_2+\bar{x}\cdot y_2 \tag{5-2}$$

$$y_1^{n+1}=\bar{y}_1=1\cdot\bar{y}_1+\bar{1}\cdot y_1 \tag{5-3}$$

将次态函数表达式(5-2)与 J-K 触发器的次态方程(5-1)相比较，可以得出

$$J_2=\bar{x}\bar{y}_1 \qquad K_2=\bar{x}$$

将次态函数表达式(5-3)与 J-K 触发器的次态方程(5-1)相比较，可以得出

$$J_1=K_1=1$$

由所得结果可知，利用触发器次态方程得到的激励函数表达式与前面利用触发器激励表得到的激励函数表达式完全相同。设计中具体采用哪种方法可由设计者灵活处理。

*5.3.3 不完全确定同步时序逻辑电路设计

由于不完全确定同步时序逻辑电路中存在不确定的次态或输出，因此，这类电路设计过程中的某些步骤相对完全确定同步时序逻辑电路的设计而言显得更加灵活，设计中恰当地利用不确定次态和不确定输出的随意性，通常可使设计方案变得更简单。

1. 形成原始状态图和原始状态表

根据某些设计要求建立的原始状态图和原始状态表中往往存在不确定的次态或输出，即存在某些状态，它们在某些输入取值下的次态或输出是随意的。这种状态图和状态表被称为不完全确定原始状态图和原始状态表。

例 5.11 设计一个用于引爆控制的同步时序电路，该电路有一个输入端 x 和一个输出端 Z。平时输入 x 始终为 0，一旦需要引爆，则从 x 端连续输入 4 个“1”信号(不被“0”间断)，电路收到第四个 1 后，在输出端 Z 产生一个 1 信号点火引爆，该电路连同引爆装置一起被炸毁。试建立该电路的 Mealy 型状态图和状态表。

解 该电路实际上是一个用于特殊场所的“1111”序列检测器。它与一般序列检测器有两点不同。一是输入带有约束条件，即一旦输入出现 1，则一定是不被 0 间断的连续 4 个 1；二是在接收到 4 个 1 信号后，产生的输出 1 信号在点火引爆的同时电路自毁，故此时不再存在次态问题。

设状态 A 表示电路初始状态，状态 B 表示收到了第一个 1 输入，状态 C 表示收到了连续 2 个 1 输入，状态 D 表示收到了连续 3 个 1 输入。根据题意，在 A 状态下当 x 为 1 时，输出为 0 转向状态 B；在 B 状态下当 x 为 1 时，输出为 0 转向状态 C；在 C 状态下当 x 为 1 时，输出为 0 转向状态 D；而在 D 状态下当 x 为 1 时，输出为 1，次态随意(实际上已不存在次态)。其次，在 A 状态下当 x 为 0 时，可令输出为 0，停留在状态 A，而在 B、C、D 这 3 个状态下由于 x 不会为 0，故可令输出和次态作为无关处理。据此，可得到该电路的 Mealy 型原始状态图，如图 5.33 所示；原始状态表如表 5.23 所示。图表中用“d”表示不确定次态或不确定输出。

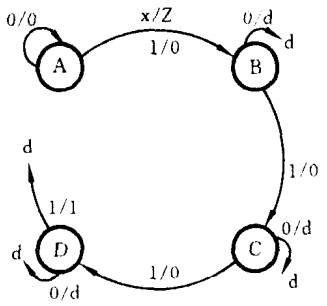


图 5.33　原始状态图

表 5.23　原始状态表

现态	次态/输出	
	x=0	x=1
A	A/0	B/0
B	d/d	C/0
C	d/d	D/0
D	d/d	d/1

2. 不完全确定状态表的化简

化简不完全确定状态表时，对于表中的不确定次态和不确定输出，可以当作无关条件处理，充分利用它们的随意性通常可使设计方案变得更简单。这一点与设计包含无关条件的组合电路是类似的，只不过在处理上要复杂一些。不完全确定状态表的化简是建立在相容状态基础上的。

(1) 相容状态和相容类

1) 相容状态

定义　假定状态 S_i 和 S_j 是不完全确定状态表中的两个状态，如果对于所有的有效输入序列，分别从状态 S_i 和 S_j 出发，所得到的确定输出序列(不确定的位除外)是完全相同的，那么，状态 S_i 和 S_j 是相容的，或者说状态 S_i 和 S_j 是相容对，记作(S_i,S_j)。

这里所谓有效输入序列的含义是：从状态表中的状态 S 出发，如果在某输入序列作用下所得到的状态响应序列除最后一个次态外，其他次态都是确定的(即仅最后一个次态可以确定或者不确定)，那么，这个输入序列对状态 S 是有效的。所有的有效输入序列，是指有效输入序列的长度和结构是任意的。

在不完全确定状态表中，判断两个状态是否相容的依据是表中给出的次态和输出。

判断方法　假定状态 S_i 和 S_j 是不完全确定状态表中的两个现态，那么，状态 S_i 和 S_j 相容的条件，可归纳为在一位输入的各种取值组合下满足如下两条。

第一，输出完全相同，或者其中的一个(或两个)输出不确定。

第二，它们的次态属于下列情况之一：

a. 次态相同；

b. 次态交错或为各自的现态；

c. 次态循环或为相容对；

d. 其中的一个(或两个)为不确定状态。

必须指出,相容状态不具有传递性。即不能由 S_1 和 S_2 相容、S_2 和 S_3 相容,推出 S_1 和 S_3 也相容。这是因为判断两个状态是否相容时,对于不确定的输出和不确定的次态可以随意指定的缘故。

2) 相容类

相容类是由彼此相容的状态构成的集合。处于同一相容类中的所有状态之间都是两两相容的。例如,若有相容对(S_1,S_2)、(S_2,S_3)和(S_1,S_3),则可构成相容类$\{S_1,S_2,S_3\}$。

3) 最大相容类

若一个相容类不是任何其他相容类的子集,则该相容类称为最大相容类。由于相容状态无传递性,所以,同一原始状态表的各最大相容类之间可能存在相同状态,即同一状态可能出现在不同的最大相容类中。

(2) 不完全确定状态表的化简

不完全确定状态表的化简过程与完全确定状态表的化简过程相似,只是某些环节的具体处理稍有不同。一般化简过程如图 5.34 所示。

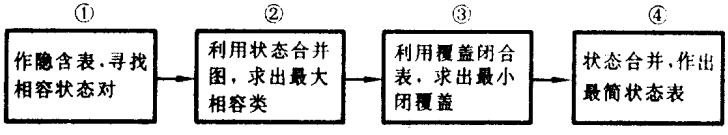


图 5.34 不完全确定状态表的化简过程

① 作隐含表,寻找相容状态对。

利用隐含表寻找相容对的过程与化简完全确定状态表时寻找等效对的过程是相同的,仅仅是状态相容与状态等效的判断法则有所不同而已,即画好隐含表后,首先依次判别每个状态对的相容关系,并将判断结果标注到隐含表中,标注方法与化简完全确定状态表相同。

在顺序比较完成后,可利用已建立的隐含表继续追踪待确定的状态,即进行关联比较。如果与之关联的次态对都是相容的,则原状态对是相容的;只要某方格中填入的次态对中有一对不相容,则该方格所代表的状态对不相容,在该方格中加入标记"/"。逐个检查,直至判断出所有状态对相容或不相容为止,即可列出原始状态表中的全部相容对。

② 利用状态合并图,求出最大相容类。

为了方便地找到最大相容类,通常借助状态合并图。状态合并图直观地反映了不完全确定状态表中状态的相容关系。它将不完全确定状态表的状态以

"点"的形式均匀地绘制在圆周上，然后把所有相容对都用线段连接起来，即用圆周上的点表示状态，点与点之间的连线表示两状态之间的相容关系。如果图中某多边形的所有点之间都两两相连，则该多边形顶点所表示的状态就构成了一个最大相容类。

图 5.35(a)、(b)、(c)所示的状态合并图分别表示包含 3 个、4 个和 5 个状态的最大相容类。

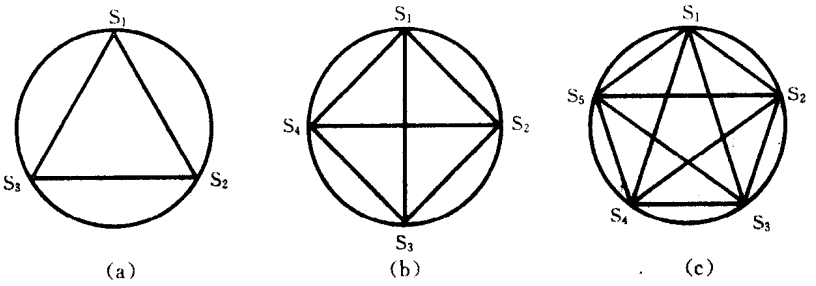


图 5.35 典型状态合并图

③ 利用闭覆盖表，求最小闭覆盖。

这一步与化简完全确定状态表差别较大。要想求出不完全给定状态表的最小化状态表，必须从最大相容类(或相容类)中选出一个相容类的集合，该相容类集合必须满足以下 3 个条件。

a. 覆盖：所选相容类集合应包含原始状态表的全部状态。

b. 最小：所选相容类集合中相容类个数应最少。

c. 闭合：所选相容类集合中的任一相容类，在原始状态表中任一输入条件下产生的次态组合应该属于该集合中的某一个相容类。

同时具备最小、闭合和覆盖 3 个条件的相容类(可以是相容类或者最大相容类)集合，称为**最小闭覆盖**。

不完全确定状态表的化简过程，就是寻找最小闭覆盖的过程。为了方便地求出原始状态表的最小闭覆盖，通常借助覆盖闭合表，简称闭覆盖表。闭覆盖表包括两部分，一部分反映相容类集合对状态的覆盖情况，另一部分反映相容类的闭合关系。闭覆盖表的画法是：在表的左边自上而下列出所选相容类，表的中间覆盖部分列出各相容类对原始状态表中状态的覆盖情况，表的右边闭合部分列出各相容类在一位输入各种取值下的次态组合。必须指出，这里所说的相容类包括最大相容类和它们的子类。

④ 状态合并，作出最简状态表。

选出一个最小闭覆盖之后，将最小闭覆盖中的每个相容类用一个新的状态表

示，并将原始状态表中的相应状态用新的状态取代，即可得到与原始状态表功能相同的最简状态表。下面举例说明不完全确定状态表的化简方法。

例 5.12 化简表 5.24 所示的原始状态表。

表 5.24 原始状态表

现态	次态/输出	
	x=0	x=1
A	d/d	A/d
B	B/0	C/1
C	d/1	D/0
D	B/d	d/d
E	C/1	A/0

解 表 5.24 所示的是一个具有 5 个状态的原始状态表，表中存在不确定的次态和输出，因此，是一个不完全确定状态表。用隐含表法化简该状态表的过程如下。

① 作隐含表，寻找相容状态对。

作出隐含表，并根据相容状态的判断标准对各状态对进行顺序比较和关联比较，比较后的结果如图5.36所示。由隐含表中的标注可知，该状态表中的相容状态对有：(A,B)、(A,C)、(A,D)、(A,E)、(B,D)、(C,D)、(C,E)。

② 作状态合并图，找出最大相容类。

根据相容状态对可作出状态合并图，如图 5.37 所示。从状态合并图得到最大相容类为{A,B,D}、{A,C,D}、{A,C,E}。

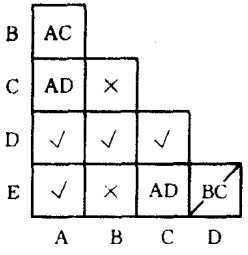


图 5.36 隐含表

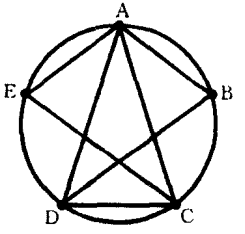


图 5.37 状态合并图

③ 作闭覆盖表，求最小闭覆盖。

假定从最大相容类中选取最小闭覆盖，可作出其闭覆盖表，如表 5.25 所示。由表 5.25 所示闭覆盖表和选择最小闭覆盖的 3 个条件可知，该问题的最小闭覆盖可由最大相容类{A,B,D}和{A,C,E}组成。

表 5.25 闭覆盖表

最大相容类	覆盖					闭合	
	A	B	C	D	E	x=0	x=1
ABD	A	B		D		B	AC
ACD	A		C	D		B	AD
ACE	A		C		E	C	AD

表 5.26 最小化状态表

现态	次态/输出	
	x=0	x=1
a	a/0	b/1
b	b/1	a/0

④ 状态合并，作出最简状态表。

假定最小闭覆盖中的相容类{A,B,D}用状态 a 表示，相容类{A,C,E}用状态 b 表示，并将 5.24 所示原始状态表中的相应状态用新的状态代替，可得到最简状态表，如表 5.26 所示。

在状态合并时注意：若存在确定的次态和不确定的次态，则应根据确定的次态决定合并后的状态；若存在确定的输出和不确定的输出，则应取确定的输出作为合并后的输出。例如，原始状态表中的现态 A、B、D 在输入 x=1 时有确定的次态 A、C 和不确定的次态 d，合并时应根据确定的次态 A、C，合并后用 b 表示；输出有 1 和 d 两种，合并后的状态 b 在 x=1 时的输出应为 1。

值得指出的是，在化简不完全给定状态表时，构成最小闭覆盖的相容类并不要求一定是最大相容类。在某些情况下，仅仅从最大相容类中去选择最小闭覆盖，不一定能求出最简状态表。如果在最大相容类和非最大相容类之间作恰当的选择，反而能使得到的状态表达到最简。

例 5.13　化简表 5.27 所示的原始状态表。

表 5.27　原始状态表

现态	次态/输出	
	x=0	x=1
A	D/d	A/0
B	E/0	A/d
C	D/0	B/0
D	C/d	C/d
E	d/1	B/d

解　用隐含表法化简表 5.27 所示原始状态表的过程如下。

① 作隐含表，寻找相容状态对。

化简表 5.27 的隐含表如图 5.38 所示。由隐含表中的标注可知，该状态表中的相容状态对有：(A,B)、(A,C)、(A,D)、(A,E)、(B,C)、(C,D)、(D,E)。

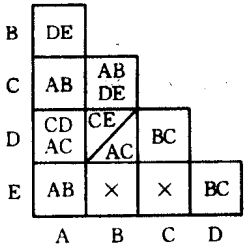


图 5.38　隐含表

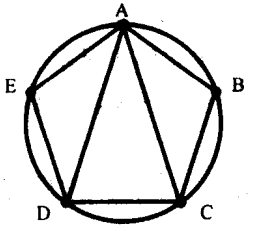


图 5.39　状态合并图

② 作状态合并图，找出最大相容类。

根据相容状态对可作出状态合并图，如图 5.39 所示。从状态合并图得到最大相容类为{A,B,C}、{A,C,D}、{A,D,E}。

③ 作闭覆盖表，求最小闭覆盖。

假定从最大相容类中选取最小闭覆盖，可作出其闭覆盖表，如表 5.28 所示。

由表 5.28 所示闭覆盖表和选择最小闭覆盖的 3 个条件可知，该问题的最小闭覆盖应由最大相容类{A,B,C}、{A,C,D}和{A,D,E}组成。

表 5.28 闭覆盖表

最大相容类	覆盖					闭合	
	A	B	C	D	E	x=0	x=1
ABC	A	B	C			DE	AB
ACD	A		C	D		CD	ABC
ADE	A			D	E	CD	ABC

表 5.29 最小化状态表

现态	次态/输出	
	x=0	x=1
a	c/0	a/0
b	b/0	a/0
c	b/1	a/0

④ 状态合并，作出最简状态表。

假定最小闭覆盖中的相容类{A,B,C}用状态 a 表示，相容类{A,C,D}用状态 b 表示，{A,D,E}用状态 c 表示，并将表 5.27 所示原始状态表中的相应状态用新的状态代替，可得到化简后的状态表，如表 5.29 所示。

然而，表 5.29 所示的状态表并不是最简状态表。如果求最小闭覆盖时选择最大相容类{A,B,C}和非最大相容类{D,E}，可作出闭覆盖表，如表 5.30 所示。由表 5.30 所示闭覆盖表和选择最小闭覆盖的 3 个条件可知，该问题的最小闭覆盖可由最大相容类{A,B,C}和非最大相容类{D,E}组成。假定相容类{A,B,C}用状态 a 表示，相容类{D,E}用状态 b 表示，并将 5.27 所示原始状态表中的相应状态用新的状态代替，可得到最简状态表如表 5.31 所示。显然，表 5.31 所示状态表比表 5.29 所示状态表更简单。

表 5.30 闭覆盖表

最大相容类	覆盖					闭合	
	A	B	C	D	E	x=0	x=1
ABC	A	B	C			DE	AB
DE				D	E	C	BC

表 5.31 最小化状态表

现态	次态/输出	
	x=0	x=1
a	b/0	a/0
b	a/1	a/d

不完全确定同步时序逻辑电路的设计在解决了状态化简问题后，其他步骤与完全确定同步时序逻辑电路的设计相同。

5.3.4 同步时序逻辑电路设计举例

在数字系统中，同步时序电路的应用十分广泛，为了帮助读者熟练掌握其设计方法，下面给出几个设计实例。

例 5.14 用 T 触发器作为存储元件，设计一个 2 位二进制减 1 计数器。电路工作状态受输入信号 x 的控制。当 x=0 时，电路状态不变；当 x=1 时，在时钟脉冲作用下进行减 1 计数。计数器有一个输出 Z，当产生借位时 Z 为 1，其他情况下 Z 为 0。

解 题中对电路的状态数目及状态转换关系均十分清楚，设计过程如下：

① 作出状态图和状态表。根据问题要求，设状态变量用 y_2、y_1 表示，可直接作出计数器的二进制状态图，如图 5.40 所示；二进制状态表如表 5.32 所示。

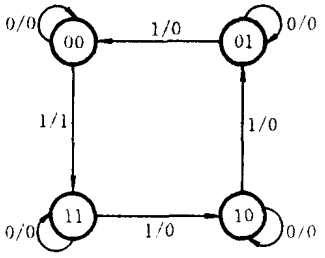


图 5.40　状态图

表 5.32　状态表

现　态		次态 $y_2^{n+1}y_1^{n+1}$/输出 Z	
y_2	y_1	x=0	x=1
0	0	00/0	11/1
0	1	01/0	00/0
1	1	11/0	10/0
1	0	10/0	01/0

② 确定激励函数和输出函数并化简。根据表 5.32 所示的状态表和 T 触发器的激励表，可作出激励函数和输出函数卡诺图，如图 5.41 所示。

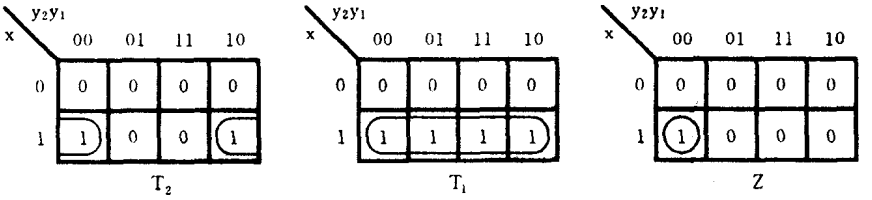


图 5.41　激励函数和输出函数卡诺图

化简后，激励函数和输出函数表达式为

$$T_2 = x\bar{y}_1 \qquad T_1 = x \qquad Z = x\bar{y}_2\bar{y}_1$$

③ 画出逻辑电路图。根据激励函数和输出函数表达式，可画出逻辑电路，如图 5.42 所示。

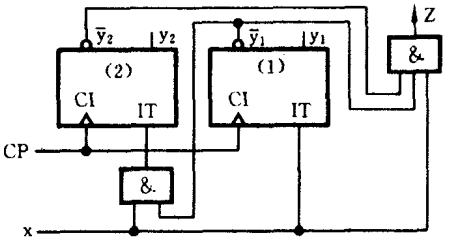


图 5.42　逻辑电路

例 5.15　设计一个两位串行输入、并行输出双向移位寄存器。该寄存器有 x_1 和 x_2 两个输入端，其中 x_2 为控制端，用于控制移位方向，x_1 为数据输入端。当 $x_2=0$ 时，x_1 往寄存器高位串行送数，寄存器中的数据从高位移向低位；当 $x_2=1$ 时，x_1 往寄存器低位串行送数，寄存器中的数据从低位移向高位。寄存器的输出为触发器状态本身。

解　根据题意，设电路的状态变量为 y_2 和 y_1，其中 y_2 为高位，y_1 为低位，因而

可直接画出寄存器的二进制状态图，如图 5.43 所示；状态表如表 5.33 所示。

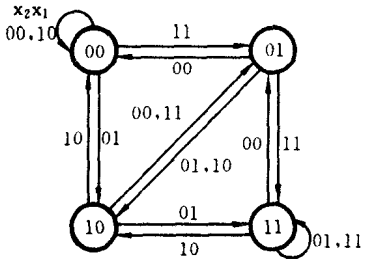


图 5.43 状态图

表 5.33 二进制状态表

现态 y_2	现态 y_1	次态 $y_2^{n+1}y_1^{n+1}$ $x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
0	0	0 0	1 0	0 1	0 0
0	1	0 0	1 0	1 1	1 0
1	1	0 1	1 1	1 1	1 0
1	0	0 1	1 1	0 1	0 0

假定采用 D 触发器作为存储元件，根据 D 触发器的激励表和表5.33所示状态表，可作出激励函数卡诺图，如图 5.44 所示。

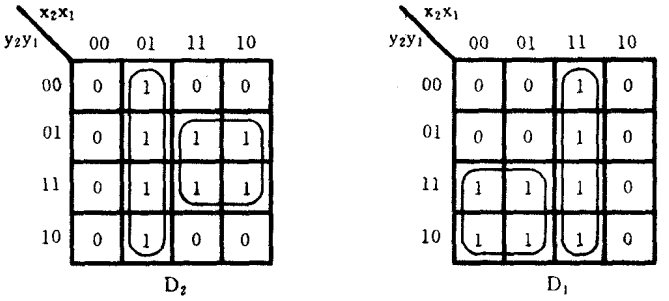


图 5.44 卡诺图

化简后的激励函数表达式为

$$D_2=\bar{x}_2x_1+x_2y_1 \qquad D_1=\bar{x}_2y_2+x_2x_1$$

根据以上函数表达式，可画出该寄存器的逻辑电路，如图 5.45 所示。

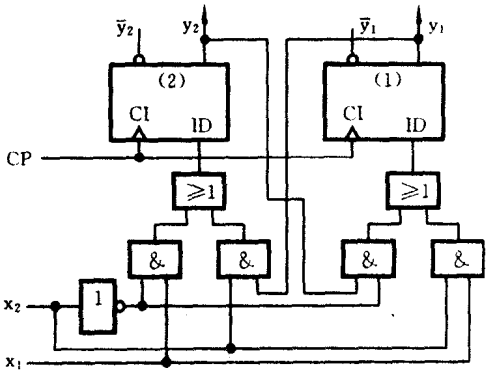


图 5.45 逻辑电路图

例 5.16　用 J-K 触发器作为存储元件，设计一个“101”序列检测器。该电路有一个输入 x 和一个输出 Z，当随机输入信号中出现“101”序列时，输出一个 1 信号。典型输入/输出序列如下。

输入 x　0 0 1 0 1 0 1 1 0 1 0 0

输出 Z　0 0 0 0 1 0 1 0 0 1 0 0

解　假定用 Moore 型同步时序电路实现给定功能，设计过程如下。

① 作出原始状态图和状态表。设初始状态为 A，根据题意可作出原始状态图，如图 5.46 所示；原始状态表如表 5.34 所示。

② 状态化简。根据化简法则可知，表 5.34 已是最小化状态表。

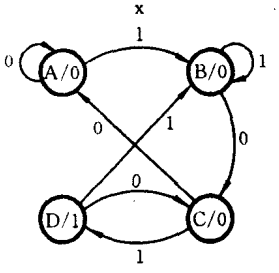


图 5.46　状态图

表 5.34　状态表

现　态	次　态		输　出
	x=0	x=1	Z
A	A	B	0
B	C	B	0
C	A	D	0
D	C	B	1

③ 状态编码。由于最小化状态表中共有 4 个状态，故需 2 位二进制代码表示，即电路中要有 2 个触发器。设状态变量为 y_2、y_1，根据相邻法的编码原则，该例可采用图 5.47 所示的编码方案。相应的二进制状态表如表 5.35 所示。

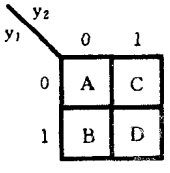


图 5.47　状态编码方案

表 5.35　二进制状态表

现态		次态 $y_2^{n+1}y_1^{n+1}$		输出
y_2	y_1	x=0	x=1	Z
0	0	0 0	0 1	0
0	1	1 0	0 1	0
1	0	0 0	1 1	0
1	1	1 0	0 1	1

④ 确定激励函数和输出函数。根据表 5.35 所示二进制状态表和 J-K 触发器的激励表，可列出激励函数和输出函数真值表，如表 5.36 所示。

表 5.36 激励函数和输出函数真值表

输入	现态		次态		激励函数				输出
x	y_2	y_1	y_2^{n+1}	y_1^{n+1}	J_2	K_2	J_1	K_1	Z
0	0	0	0	0	0	d	0	d	0
0	0	1	1	0	1	d	d	1	0
0	1	0	0	0	d	1	0	d	0
0	1	1	1	0	d	0	d	1	1
1	0	0	0	1	0	d	1	d	0
1	0	1	0	1	0	d	d	0	0
1	1	0	1	1	d	0	1	d	0
1	1	1	0	1	d	1	d	0	1

用卡诺图对表 5.36 中的激励函数和输出函数化简后，可得到其最简表达式如下：

$$J_2=\bar{x}y_1 \qquad K_2=\bar{x}\,\bar{y}_1+xy_1=\bar{x}\oplus y_1$$

$$J_1=x \qquad K_1=\bar{x}$$

$$Z=y_2y_1$$

⑤ 画逻辑电路图。根据输出函数和激励函数表达式，可画出“101”序列检测器的逻辑电路图，如图 5.48 所示。

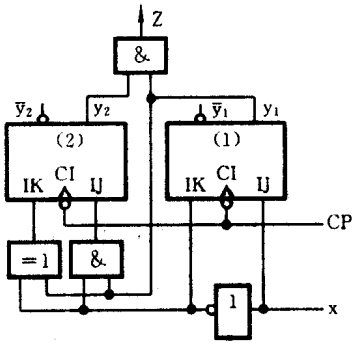


图 5.48 逻辑电路

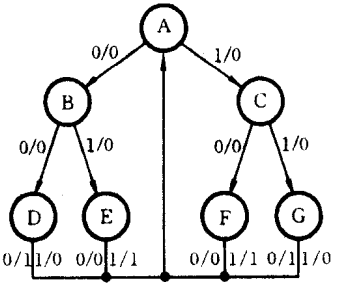


图 5.49 原始状态图

例 5.17 设计一个 3 位二进制码的串行奇偶检测器。该电路从输入端 x 串行输入二进制代码，每 3 位为一组，当 3 位代码中含 1 的个数为偶数时，输出 Z 产生一个 1 输出，平时 Z 输出为 0。

解 代码检测器的特点是输入信号是按位分组的，每组的检测过程相同，即一组检测完后，电路回到初始状态，接着进行下一组的检测。

① 建立原始状态图和原始状态表。根据题意，可作出该电路的原始状态图，如图 5.49 所示；原始状态表如表 5.37 所示。

② 状态化简。用隐含表法化简表 5.37 所示原始状态表，可得到表 5.38 所示最小化状态表。

表 5.37　原始状态表

现　态	次态/输出	
	x=0	x=1
A	B/0	C/0
B	D/0	E/0
C	F/0	G/0
D	A/1	A/0
E	A/0	A/1
F	A/0	A/1
G	A/1	A/0

表 5.38　最小化状态表

现　态	次态/输出	
	x=0	x=1
A	B/0	C/0
B	D/0	E/0
C	E/0	D/0
D	A/1	A/0
E	A/0	A/1

③ 状态编码。由于最小化状态表中有 5 个状态，所以需采用 3 位二进制码表示，即电路要用 3 个触发器。设状态变量用 y_3、y_2、y_1 表示，根据状态编码的 3 条原则，可采用图 5.50 所示状态编码方案。按照该方案，可得到二进制状态表如表 5.39所示。

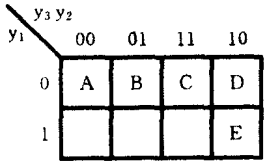


图 5.50　状态分配方案

表 5.39　二进制状态表

y_3	y_2	y_1	$y_3^{n+1}y_2^{n+1}y_1^{n+1}/Z$	
			x=0	x=1
0	0	0	010/0	110/0
0	1	0	100/0	101/0
1	1	0	101/0	100/0
1	0	0	000/1	000/0
1	0	1	000/0	000/1

④ 确定激励函数和输出函数。假定用 D 触发器作为存储元件，根据二进制状态表和 D 触发器激励表，可作出输出函数和激励函数卡诺图，如图 5.51 所示。化简时可将多余状态 001,011,111 作为无关条件处理。

化简后的激励函数和输出函数表达式为

$$D_3=x\bar{y}_3+y_2 \qquad D_2=\bar{y}_3\bar{y}_2=\overline{y_3+y_2}$$

$$D_1=\bar{x}y_3y_2+x\bar{y}_3y_2=(x\oplus y_3)y_2 \qquad Z=xy_1+\bar{x}y_3\bar{y}_2\bar{y}_1$$

⑤ 讨论。通常，当所设计的电路中触发器所能表示的状态数大于有效状态数时，需要对所设计电路的实际工作状态进行讨论。主要讨论两个问题。第一，当电路偶然进入无效状态时，能否在输入信号和时钟脉冲作用下自动进入有效状态，如果能自动进入有效状态，则称为具有自恢复功能，否则，称为“挂起”。第二，电路万一处在无效状态，是否会产生错误输出信号。一旦发现存在“挂起”现象或错误输出现象，就必须对设计的方案进行修改，否则，将影响电路工作的可靠性，甚至破坏正常工作。

实际上，只需要根据确定的激励函数和输出函数表达式作出相应状态表和状态图，便可使需要讨论的两个问题一目了然。至于修改设计方案的问题，则只涉及激励函数和输出函数化简时对无效状态下任意项的处理问题。

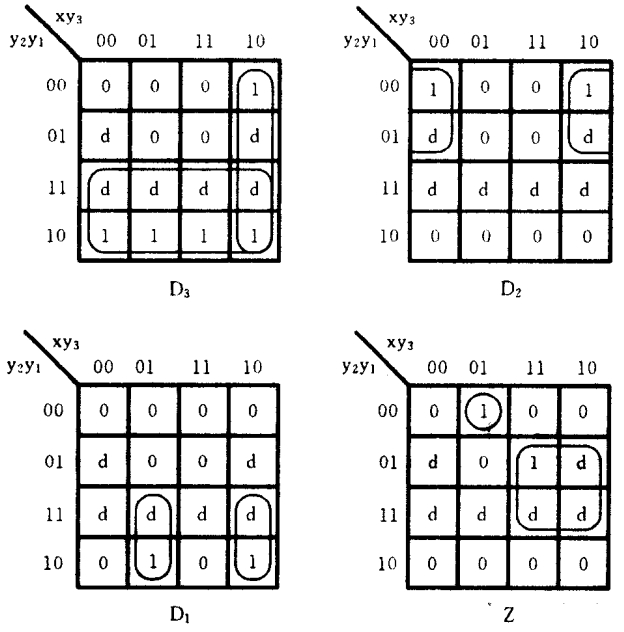


图 5.51 卡诺图

当电路中包含多个无效状态时，往往又将无效状态构成的集合称作状态的无效序列，相应地将正常工作下的状态集合称为状态的有效序列。本例的状态无效序列中包含 001、011 和 111 共 3 个无效状态，它们在化简激励函数和输出函数时被作为无关条件处理，即在这几个状态下，函数的值可根据化简的需要随意指定为 1 或者 0。通常的做法是，在卡诺图上和 1 圈在一起的即指定为 1，否则为 0。因此，只需检查化简函数时的卡诺图，便可知道无效状态下的激励函数和输出函数的取值，并推出相应次态，进而作出与设计方案对应的状态图或状态表，得出讨论结果。

由图 5.51 可知，本例中无效状态下的激励函数、输出函数和电路次态如表 5.40所示。

表 5.40 无效状态检查表

输入	现态			激励函数			次态			输出
x	y_3	y_2	y_1	D_3	D_2	D_1	y_3^{n+1}	y_2^{n+1}	y_1^{n+1}	Z
0	0	0	1	0	1	0	0	1	0	0
0	0	1	1	1	0	0	1	0	0	0
0	1	1	1	1	0	1	1	0	1	0
1	0	0	1	1	1	0	1	1	0	1
1	0	1	1	1	0	1	1	0	1	1
1	1	1	1	1	0	0	1	0	0	1

根据表 5.40 和表 5.39 所示二进制状态表，可作出该设计方案的状态图，如图 5.52 所示。由该状态图可知，此方案的无效状态序列不会产生“挂起”现象，但在无效状态下输入 1 时会产生错误的 1 输出。为此，应将输出函数表达式修改为

$$Z=\bar{x}y_3\bar{y}_2\bar{y}_1+xy_3\bar{y}_2y_1$$
$$=y_3\bar{y}_2(\bar{x}\bar{y}_1+xy_1)=y_3\bar{y}_2\overline{x\oplus y_1}$$
$$=\overline{\bar{y}_3+y_2+(x\oplus y_1)}$$

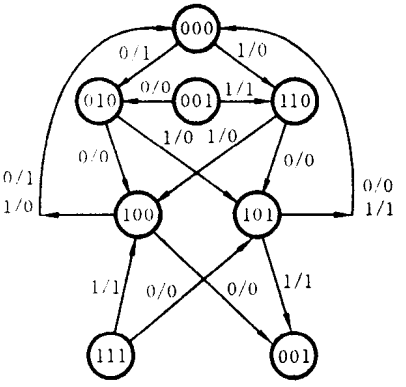


图 5.52　方案检查状态图

⑥ 画逻辑电路图。根据简化后的激励函数表达式和修改后的输出函数表达式，可画出该奇偶检测电路的逻辑电路，如图 5.53 所示。

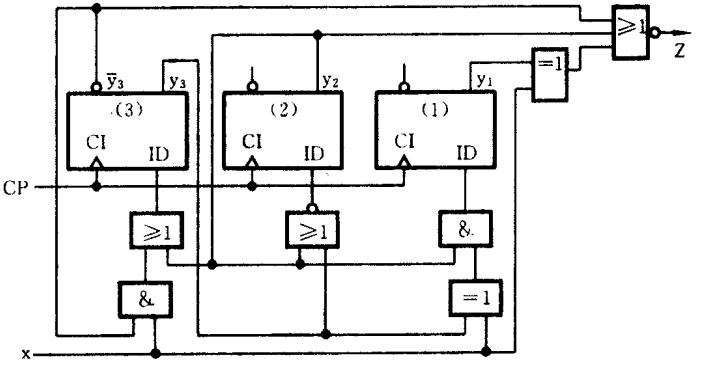


图 5.53　逻辑电路

习　题　五

5.1　简述时序逻辑电路与组合逻辑电路的主要区别。

5.2　作出与表 5.41 所示状态表对应的状态图。

5.3　已知状态图如图 5.54 所示，输入序列为 x＝11010010，设初始状态为 A，求状态和输出响应序列。

5.4　分析图 5.55 所示逻辑电路。假定电路初始状态为“00”，说明该电路逻辑功能。

5.5　分析图 5.56 所示逻辑电路，说明该电路功能。

表 5.41 状态表

现 态	次态 $y_2^{n+1}y_1^{n+1}$/输出 Z			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	B/0	B/0	A/1	B/0
B	B/0	C/1	A/0	D/1
C	C/0	B/0	D/0	A/0
D	A/0	A/1	C/0	C/0

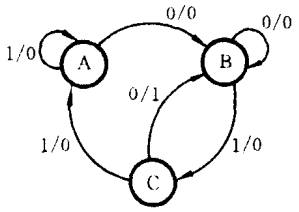


图 5.54 状态图

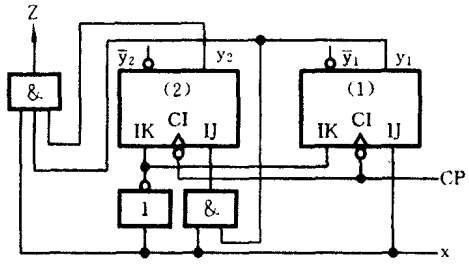


图 5.55 逻辑电路

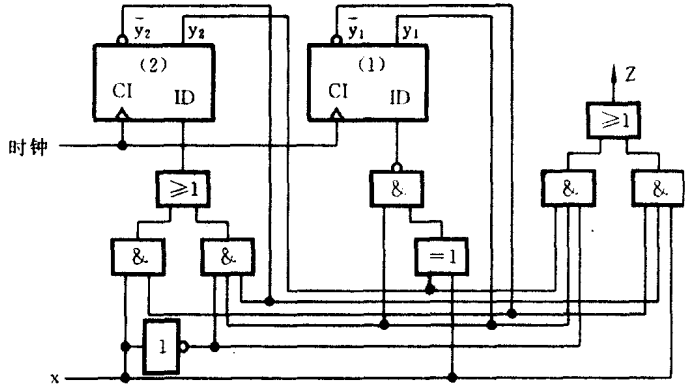


图 5.56 逻辑电路

5.6 分析图 5.57 所示逻辑电路，说明该电路功能。

5.7 作出“0101”序列检测器的 Mealy 型状态图和 Moore 型状态图。典型输入/输出序列如下。

输入 x 1 1 0 1 0 1 0 1 0 0 1 1

输出 Z 0 0 0 0 0 1 0 1 0 0 0 0

5.8 设计一个代码检测器，该电路从输入端 x 串行输入余 3 码（先低位后高位），当出现非法数字时，电路输出 Z 为 1，否则输出为 0。试作出 Mealy 型状态图。

5.9 化简表 5.42 所示原始状态表。

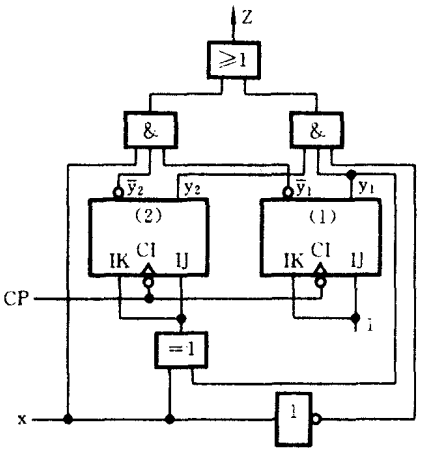


图 5.57　逻辑电路

表 5.42　状态表

现　态	次态/输出	
	x=0	x=1
A	B/0	C/0
B	A/0	F/0
C	F/0	G/0
D	A/0	C/0
E	A/0	A/1
F	C/0	E/0
G	A/0	B/1

表 5.43　状态表

现　态	次态/输出	
	x=0	x=1
A	D/d	C/0
B	A/1	E/d
C	d/d	E/1
D	A/0	C/0
E	B/1	C/d

5.10　化简表 5.43 所示不完全确定原始状态表。

5.11　按照相邻法编码原则对表 5.44 进行状态编码。

表 5.44　状态表

现　态	次态/输出	
	x=0	x=1
A	A/0	B/0
B	C/0	B/0
C	D/1	C/0
D	B/1	A/0

表 5.45　状态表

现　态		次态 $y_2^{(n+1)}y_1^{(n+1)}$/输出 Z	
y_2	y_1	x=0	x=1
0	0	01/0	10/0
0	1	11/0	10/0
1	1	10/1	01/0
1	0	00/1	11/1

5.12　分别用 D、T、J-K 触发器作为同步时序电路的存储元件，实现表 5.45 所示二进制状态表的功能。试写出激励函数和输出函数表达式，比较采用哪种触发器可使电路最简。

5.13　已知某同步时序电路的激励函数和输出函数表达式为

$$D_2=\bar{x}y_2+x\bar{y}_1y_2$$

$$D_1=\bar{x}y_2+\bar{y}_1y_2+xy_1\bar{y}_2$$

$$Z=y_2$$

试求出改用 J-K 触发器作为存储元件的最简电路。

5.14 设计一个能对两个二进制数 $X=x_1x_2\cdots x_n$ 和 $Y=y_1y_2\cdots y_n$ 进行比较的同步时序电路，其中，X、Y 串行地输入到电路的 x、y 输入端。比较从 x_1、y_1 开始，依次进行到 x_n、y_n。电路有两个输出 Z_x 和 Z_y，若比较结果 $X>Y$，则 Z_x 为 1，Z_y 为 0；若 $X<Y$，则 Z_x 为 0，Z_y 为 1；若 $X=Y$，则 Z_x 和 Z_y 都为 1。要求用尽可能少的状态数作出状态图和状态表，并用尽可能少的逻辑门和触发器（采用 J-K 触发器）实现其功能。

5.15 用 T 触发器作为存储元件，设计一个 8421 码十进制加 1 计数器。



异步时序逻辑电路

时序逻辑电路按照工作方式的不同，可以分为同步时序逻辑电路和异步时序逻辑电路两种类型，异步时序逻辑电路又可以进一步分为脉冲异步时序逻辑电路和电平异步时序逻辑电路。前面已对同步时序逻辑电路进行了系统介绍，本章讨论异步时序逻辑电路，并在介绍异步时序逻辑电路特点的基础上，重点讨论两类异步时序逻辑电路的分析与设计方法。

6.1 异步时序逻辑电路的特点与分类

在同步时序逻辑电路中，存储元件采用时钟控制触发器，电路中各触发器的时钟控制端与统一的时钟脉冲(CP)相连接，仅当时钟脉冲作用时，电路状态才能发生变化，改变后的状态一直保持到下一个时钟脉冲到来(在此期间不受外部输入变化的影响)之时。换而言之，由时钟脉冲信号决定电路状态转换时刻并实现“等状态时间”，整个电路在时钟脉冲作用下由一个稳定状态转移到另一个稳定状态。正因为时钟脉冲对电路的控制作用，所以不论输入信号是电平信号还是脉冲信号，对电路引起的状态响应都是相同的。因此，在研究同步时序电路时，没有对输入信号的形式加以区分。此外，该类电路除了对时钟脉冲的宽度和周期有一定要求外，对输入信号的变化过程没有加任何约束。

异步时序逻辑电路的工作特点是：电路中没有统一的时钟脉冲信号同步，电路状态的改变是外部输入信号变化直接作用的结果；在状态转移过程中，各存储元件的状态变化不一定发生在同一时刻，不同状态的维持时间不一定相同，并且可能出现非稳定状态；在研究异步时序逻辑电路时，对输入信号的形式有所区分，无论输入信号是脉冲信号还是电平信号，对其变化过程均有一定约束。

根据电路结构模型和输入信号形式的不同，异步时序逻辑电路可分为**脉冲异步时序逻辑电路**和**电平异步时序逻辑电路**两种类型。

脉冲异步时序逻辑电路的存储电路由触发器组成(可以是时钟控制触发器或

者非时钟控制触发器),电路输入信号为脉冲信号;电平异步时序逻辑电路的存储电路由延迟元件组成(可以是专用的延迟元件或者利用电路本身固有的延迟),通过延迟加反馈实现记忆功能,电路输入信号为电平信号。

根据电路输出是否与输入直接相关,两类异步时序逻辑电路均可分为 Mealy 型和 Moore 型两种不同的模型。

6.2 脉冲异步时序逻辑电路

6.2.1 脉冲异步时序逻辑电路的结构模型

脉冲异步时序逻辑电路的结构模型如图 6.1 所示。

在脉冲异步时序电路中,引起触发器状态变化的脉冲信号是由输入端直接提供的。为了保证电路可靠地工作,输入脉冲信号必须满足如下约束条件:

① 输入脉冲的宽度,必须保证触发器可靠翻转;

② 输入脉冲的间隔,必须保证前一个脉冲引起的电路响应完全结束后,后一个脉冲才能到来;

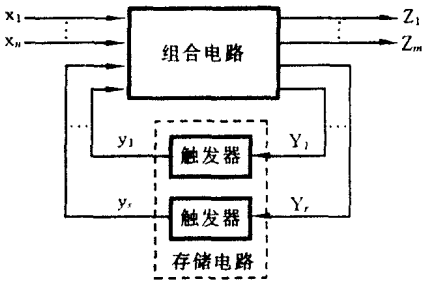


图 6.1 脉冲异步时序电路的结构模型

③ 不允许在两个或两个以上输入端同时出现脉冲。因为客观上两个或两个以上脉冲是不可能准确地"同时"的,在没有时钟脉冲同步的情况下,由不可预知的时间延迟造成的微小时差,可能导致电路产生错误的状态转移。

此外,在脉冲异步时序逻辑电路中,Mealy 型和 Moore 型电路的输出信号会有所不同。对于 Mealy 型电路来说,由于输出不仅是状态变量的函数,而且是输入的函数,所以,输出通常是脉冲信号;而对于 Moore 型电路来说,由于输出仅仅是状态变量的函数,所以输出是电平信号,输出电平的值被定义在两个间隔不定的输入脉冲之间,即由两个输入脉冲之间的状态决定。

6.2.2 脉冲异步时序逻辑电路的分析

脉冲异步时序逻辑电路的分析方法与同步时序逻辑电路大致相同。分析过程中同样采用状态表、状态图、时间图等作为工具,分析步骤如下:

① 写出电路的输出函数和激励函数表达式；

② 列出电路次态真值表或次态方程组；

③ 作出状态表和状态图；

④ 画出时间图并用文字描述电路的逻辑功能。

显然，脉冲异步时序逻辑电路分析步骤与同步时序逻辑电路的完全相同。但是，由于脉冲异步时序逻辑电路没有统一的时钟脉冲以及对输入信号的约束，因此，在具体步骤的实施上是有区别的。其差别主要表现为两点。第一，当存储元件采用时钟控制触发器时，应将触发器的时钟控制端作为激励函数处理。分析时应特别注意触发器时钟端何时有脉冲作用，仅当时钟端有脉冲作用时，才根据触发器的输入确定状态转移方向，否则，触发器状态不变。若采用非时钟控制触发器，则应注意作用到触发器输入端的脉冲信号。第二，由于不允许两个或两个以上输入端同时出现脉冲，加之输入端无脉冲出现时，电路状态不会发生变化，因此，分析时可以排除这些情况，从而使分析过程和使用的图、表得以简化。具体地说，对 n 个输入端的一位输入，只需考虑各自单独出现脉冲的 n 种情况，而不像同步时序逻辑电路中那样需要考虑 2^n 种情况。例如，假定电路有 x_1、x_2 和 x_3 共 3 个输入，并用取值 1 表示有脉冲出现，则一位输入允许的取值只有 000、001、010、100 共 4 种，分析时需要讨论的只有后 3 种情况。下面举例说明脉冲异步时序逻辑电路的分析方法。

例 6.1　分析图 6.2 所示脉冲异步时序逻辑电路，指出该电路功能。

解　该电路由两个 J-K 触发器和一个与门组成，有一个输入端 x 和一个输出端 Z，输出是输入和状态的函数，属于 Mealy 型脉冲异步时序电路。

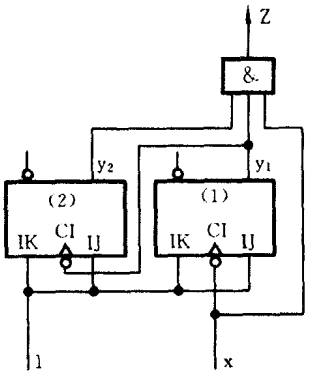


图 6.2　逻辑电路

① 写出输出函数和激励函数表达式。

$$Z = xy_2y_1$$

$$J_2 = K_2 = 1 \qquad C_2 = y_1$$

$$J_1 = K_1 = 1 \qquad C_1 = x$$

② 列出电路次态真值表。由于电路中的两个 J-K 触发器没有统一的时钟脉冲控制，所以，分析电路状态转移时，应特别注意各触发器时钟端何时有脉冲作用。J-K 触发器的状态转移发生在时钟端脉冲负跳变的瞬间，在次态真值表中用“↓”表示。仅当时钟端有“↓”出现时，相应触发器状态才能发生变化，否则状态不变。据此，可列出该电路的次态真值表，如表 6.1 所示。表中，x 为 1 表示输入端有脉冲出现，考虑到输入端无脉冲出现时电路状态不变，故省略了 x 为 0 的情况。

表 6.1 次态真值表

输入	现态		激励函数						次态	
x	y_2	y_1	J_2	K_2	C_2	J_1	K_1	C_1	y_2^{n+1}	y_1^{n+1}
1	0	0	1	1		1	1	↓	0	1
1	0	1	1	1	↓	1	1	↓	1	0
1	1	0	1	1		1	1	↓	1	1
1	1	1	1	1	↓	1	1	↓	0	0

③ 作出状态表和状态图。根据表 6.1 所示次态真值表和输出函数表达式，可作出该电路的状态表如表 6.2 所示，状态图如图 6.3 所示。

表 6.2 状态表

现态		次态 $y_2^{n+1}y_1^{n+1}$/输出 Z
y_2	y_1	x=1
0	0	0 1 / 0
0	1	1 0 / 0
1	0	1 1 / 0
1	1	0 0 / 1

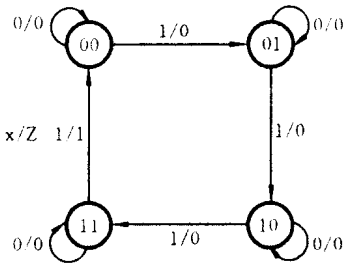


图 6.3 状态图

④ 画出时间图并说明电路逻辑功能。为了进一步描述该电路在输入脉冲作用下的状态和输出变化过程，可根据状态表或状态图画出该电路的时间图，如图 6.4所示。

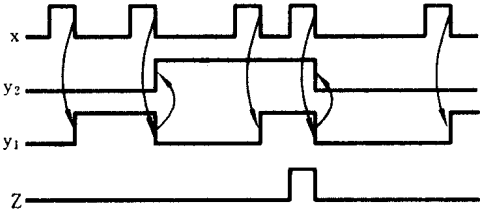


图 6.4 时间图

由状态图和时间图可知，该电路是一个模 4 加 1 计数器，当收到第四个输入脉冲时，电路产生一个进位输出脉冲。

例 6.2 分析图 6.5 所示脉冲异步时序逻辑电路。

解 该电路的存储电路部分由两个与非门构成的基本 R-S 触发器组成。电路有 3 个输入端 x_1、x_2和 x_3，一个输出端 Z，输出 Z 是状态变量的函数，属于 Moore 型脉冲异步时序电路。

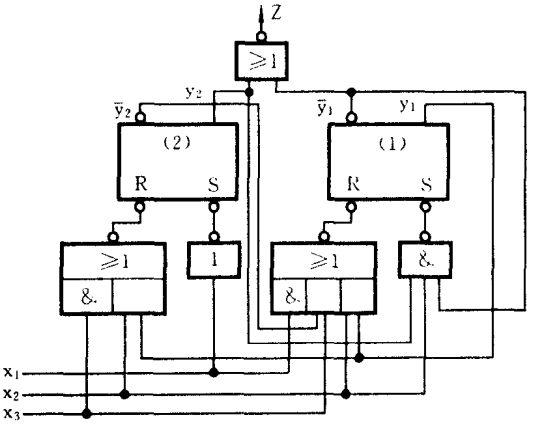


图 6.5　逻辑电路

① 写出输出函数和激励函数表达式。

$$Z=\overline{y_2+\bar{y}_1}=\bar{y}_2y_1 \qquad R_2=\overline{x_3+x_2y_1} \qquad S_2=\bar{x}_1$$

$$R_1=\overline{x_1+x_3\bar{y}_2+x_2y_1} \qquad S_1=\overline{x_2y_2\bar{y}_1}$$

② 列出电路次态真值表。根据激励函数表达式和 R-S 触发器的功能表，可列出电路的次态真值表，如表 6.3 所示。

表 6.3　次态真值表

输　入			现　态		激励函数				次　态	
x_1	x_2	x_3	y_2	y_1	R_2	S_2	R_1	S_1	y_2^{n+1}	y_1^{n+1}
1	0	0	0	0	1	0	0	1	1	0
1	0	0	0	1	1	0	0	1	1	0
1	0	0	1	0	1	0	0	1	1	0
1	0	0	1	1	1	0	0	1	1	0
0	1	0	0	0	1	1	1	1	0	0
0	1	0	0	1	0	1	0	1	0	0
0	1	0	1	0	1	1	1	0	1	1
0	1	0	1	1	0	1	0	1	0	0
0	0	1	0	0	0	1	0	1	0	0
0	0	1	0	1	0	1	0	1	0	0
0	0	1	1	0	0	1	1	1	0	0
0	0	1	1	1	0	1	1	1	0	1

③ 作出状态表和状态图。根据表 6.3 和电路输出函数表达式，可作出该电路的状态表，如表 6.4 所示；状态图如图 6.6 所示。

表 6.4 状态表

现态		次态 $y_2^{n+1}y_1^{n+1}$			输出
y_2	y_1	x_1	x_2	x_3	Z
0	0	10	00	00	0
0	1	10	00	00	1
1	0	10	11	00	0
1	1	10	00	01	0

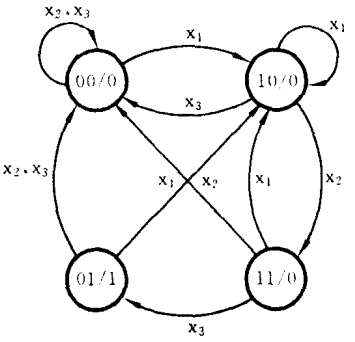


图 6.6 状态图

④ 画出时间图并说明电路功能。假定输入端 x_1、x_2、x_3 出现脉冲的顺序依次为 $x_1-x_2-x_1-x_3-x_1-x_2-x_3-x_1-x_3-x_2$，根据状态表或状态图可作出时间图，如图 6.7 所示。图中，假定电路状态转换发生在输入脉冲作用结束时，因此，转换时刻与脉冲后沿对齐。

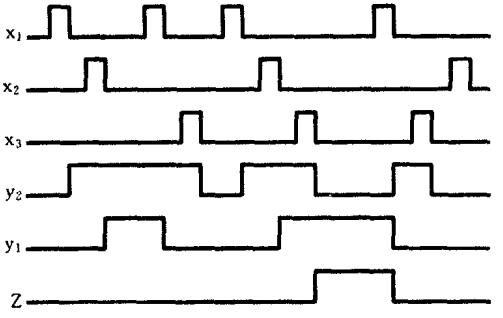


图 6.7 时间图

由状态图和时间图可知，在该电路中，当 3 个输入端按 x_1、x_2、x_3 的顺序依次出现脉冲时，产生一个“1”输出信号，其他情况下输出为“0”。因此，该电路是一个“$x_1-x_2-x_3$”序列检测器。

6.2.3 脉冲异步时序逻辑电路的设计

脉冲异步时序逻辑电路设计的一般过程与同步时序逻辑电路设计大体相同。同样分为形成原始状态图和状态表、状态化简、状态编码、确定激励函数和输出函数、画逻辑电路图等步骤。但由于在脉冲异步时序逻辑电路中没有统一的时钟脉冲信号，以及对输入脉冲信号的约束，所以，在某些步骤的处理细节上有所不同。

在脉冲异步时序逻辑电路设计时，主要应注意如下两点。

① 由于不允许两个或两个以上输入端同时为 1(用 1 表示有脉冲出现),所以,在形成原始状态图和原始状态表时,若有多个输入信号,则只需考虑多个输入信号中仅一个为 1 的情况,从而使问题的描述得以简化。此外,在确定激励函数和输出函数时,可将两个或两个以上输入同时为 1 的情况,作为无关条件处理。无疑,这有利于函数的简化。

② 由于电路中没有统一的时钟脉冲,因此,当存储电路采用带时钟控制端的触发器时,触发器的时钟端是作为激励函数处理的。这就意味着可以通过控制其时钟端输入脉冲的有、无来控制触发器的翻转或不翻转。基于这一思想,在设计脉冲异步时序逻辑电路时,可列出 4 种常用时钟控制触发器的激励表,如表 6.5～表 6.8 所示。

表 6.5　D 触发器激励表

Q	Q^{n+1}	CP	D
0	0	d	0
		0	d
0	1	1	1
1	0	1	0
1	1	d	1
		0	d

表 6.6　J-K 触发器激励表

Q	Q^{n+1}	CP	J	K
0	0	d	0	d
		0	d	d
0	1	1	1	d
1	0	1	d	1
1	1	d	d	0
		0	d	d

表 6.7　T 触发器激励表

Q	Q^{n+1}	CP	T
0	0	d	0
		0	d
0	1	1	1
1	0	1	1
1	1	d	0
		0	d

表 6.8　R-S 触发器激励表

Q	Q^{n+1}	CP	R	S
0	0	d	d	0
		0	d	d
0	1	1	0	1
1	0	1	1	0
1	1	d	0	d
		0	d	d

从表 6.5～表 6.8 可知,在要求触发器状态保持不变时,有两种不同的处理方法。一是令 CP 为 d,输入端取相应值;二是令 CP 为 0,输入端取任意值。例如,当要使 D 触发器维持 0 不变时,可令 CP 为 d,D 为 0;也可令 CP 为 0,D 为 d。显然,这将使激励函数的确定变得更加灵活,究竟选择哪种处理方法,应看怎样更有利于电路简化。一般选 CP 为 0,输入任意,因为这样显得更清晰。

下面,举例说明异步时序逻辑电路设计的方法和步骤。

例 6.3　用 T 触发器作为存储元件,设计一个异步模 8 加 1 计数器,该电路对

输入端 x 出现的脉冲进行计数，当收到第八个脉冲时，输出端 Z 产生一个进位输出脉冲。

解 该电路的状态数目和状态转换关系均非常清楚，故可直接作出二进制状态图和状态表。并由题意可知，电路模型为 Mealy 型。

① 作出状态图和状态表。设电路初始状态为"000"，状态变量用 y_3、y_2、y_1 表示，根据题意可作出二进制状态图，如图 6.8 所示；二进制状态表如表 6.9 所示。

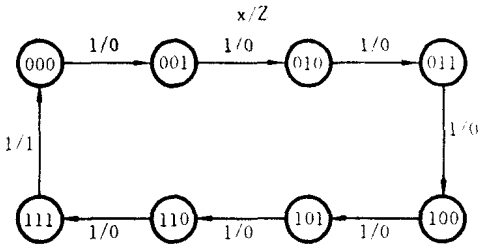


图 6.8 状态图

② 确定激励函数和输出函数。假定状态不变时，令相应触发器的时钟端为 0，输入端 T 任意；而状态需要改变时，令相应触发器的时钟端为 1(有脉冲出现)，T 端为 1。根据表 6.9 所示状态表，可得到 x 为 1 时的激励函数和输出函数真值表，如表 6.10 所示。

表 6.9 状态表

现态 $y_3\ y_2\ y_1$	次态 $y_2^{n+1}y_1^{n+1}y_0^{n+1}$/输出 Z (x=1)
0 0 0	0 0 1 / 0
0 0 1	0 1 0 / 0
0 1 0	0 1 1 / 0
0 1 1	1 0 0 / 0
1 0 0	1 0 1 / 0
1 0 1	1 1 0 / 0
1 1 0	1 1 1 / 0
1 1 1	0 0 0 / 1

表 6.10 激励函数和输出函数真值表

输入脉冲	现态			激励函数						输出
x	y_3	y_2	y_1	C_3	T_3	C_2	T_2	C_1	T_1	Z
1	0	0	0	0	d	0	d	1	1	0
1	0	0	1	0	d	1	1	1	1	0
1	0	1	0	0	d	0	d	1	1	0
1	0	1	1	1	1	1	1	1	1	0
1	1	0	0	0	d	0	d	1	1	0
1	1	0	1	0	d	1	1	1	1	0
1	1	1	0	0	d	0	d	1	1	0
1	1	1	1	1	1	1	1	1	1	1

根据表 6.10，并考虑到 x 为 0(无脉冲输入)时，电路状态不变，令各触发器时钟端为 0，输入端 T 随意，可得到简化后的激励函数和输出函数表达式如下：

$$C_3 = xy_1y_0 \quad T_3 = 1$$

$$C_2 = xy_0 \quad T_2 = 1$$

$$C_1 = x \quad T_1 = 1$$

$$Z = xy_3y_2y_1$$

化简过程略，读者可用卡诺图化简法自行验证。

③ 画出逻辑电路图。根据激励函数和输出函数表达式，可画出实现给定要求的逻辑电路，如图 6.9 所示。

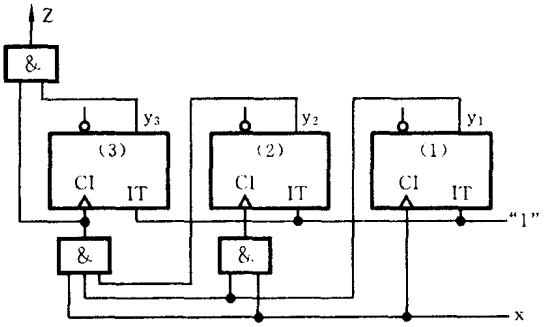


图 6.9　逻辑电路

例 6.4　用 D 触发器作为存储元件，设计一个"$x_1-x_2-x_2$"序列检测器。该电路有两个输入 x_1 和 x_2，一个输出 Z。仅当 x_1 输入一个脉冲后，x_2 连续输入两个脉冲时，输出端 Z 由 0 变为 1，该 1 信号将一直维持到输入端 x_1 或 x_2 再出现脉冲时才由 1 变为 0。其输入/输出时间图如图 6.10 所示。

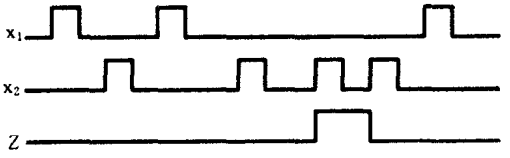


图 6.10　时间图

解　由题意可知，该序列检测器为 Moore 型脉冲异步时序电路。

① 作出原始状态图和原始状态表。设初始状态为 A，根据题意可作出原始状态图，如图 6.11 所示；原始状态表如表 6.11 所示。为了清晰起见，图、表中用 x_1 表

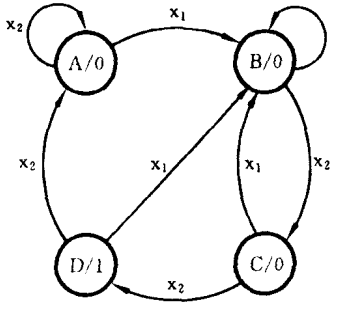


图 6.11　状态图

表 6.11　状态表

现　态	次　态		输　出
	x_1	x_2	Z
A	B	A	0
B	B	C	0
C	B	D	0
D	B	A	1

示 x_1 端有脉冲输入，x_2 表示 x_2 端有脉冲输入。

② 状态化简。用隐含表法检查表 6.11 所示状态表，可知该状态表中的状态均不等效，即已为最简状态表。

③ 状态编码。由于最简状态表中有 4 个状态，故需用两位二进制代码表示。设状态变量用 y_2、y_1 表示，根据相邻编码法的原则，可采用表 6.12 所示编码方案。并由表 6.11、表 6.12 得到二进制状态表，如表 6.13 所示。

表 6.12 编码方案

状态	编码	
	y_2	y_1
A	0	0
B	1	0
C	0	1
D	1	1

表 6.13 二进制状态表

现态		次态 $y_2^{n+1}y_1^{n+1}$		输出
y_2	y_1	$x_1=1$	$x_2=1$	Z
0	0	10	00	0
0	1	10	11	0
1	0	10	01	0
1	1	10	00	1

④ 确定输出函数和激励函数。假定次态与现态相同时，D 端取值随意，时钟端取值为 0；次态与现态不同时，D 端取值与次态相同，时钟端取值为 1（有脉冲出现）。根据表 6.13 所示状态表，可得到激励函数和输出函数真值表，如表 6.14 所示。

表 6.14 激励函数和输出函数真值表

输入		现态		激励函数				输出函数
x_2	x_1	y_2	y_1	C_2	D_2	C_1	D_1	Z
0	1	0	0	1	1	0	d	0
		0	1	1	1	1	0	0
		1	0	0	d	0	d	0
		1	1	0	d	1	0	1
1	0	0	0	0	d	0	d	0
		0	1	1	1	0	d	0
		1	0	1	0	1	1	0
		1	1	1	0	1	0	1

令输入端无脉冲出现时，各触发器时钟端为 0，输入端取任意值“d”，并将两个输入端同时为 1（不允许）作为无关条件处理，可得到激励函数和输出函数卡诺图，如图 6.12 所示。

用卡诺图化简后的激励函数和输出函数如下：

$$C_2=x_1\bar{y}_2+x_2y_1+x_2y_2 \qquad D_2=\bar{y}_2$$

$$C_1=x_1y_1+x_2y_2 \qquad D_1=\bar{y}_1$$

$$Z=y_2y_1$$

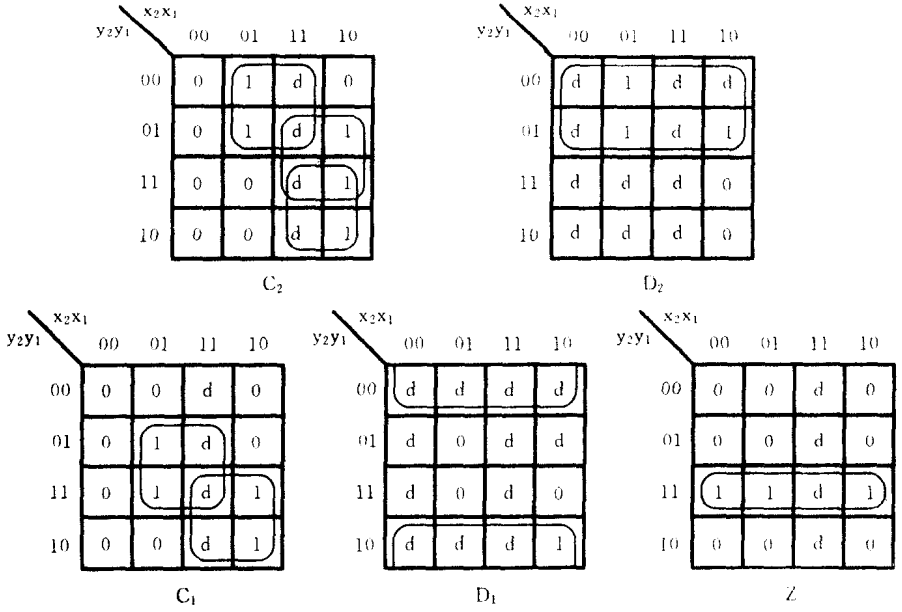


图 6.12　卡诺图

⑤ 画出逻辑电路图。根据激励函数和输出函数表达式，可画出该序列检测器的逻辑电路，如图 6.13 所示。

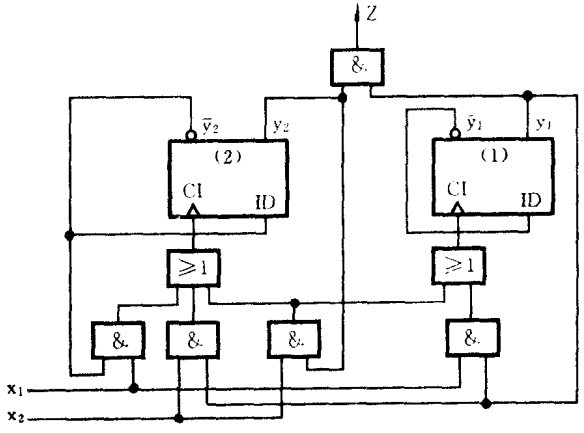


图 6.13　逻辑电路

6.3 电平异步时序逻辑电路

6.3.1 电平异步时序逻辑电路的结构模型与描述方法

前面讨论的脉冲异步时序逻辑电路和同步时序逻辑电路有两个共同的特点。第一，电路状态的转换是在脉冲作用下实现的。在同步时序逻辑电路中尽管输入信号可以是电平信号或者脉冲信号，但电路的状态转换受统一的时钟脉冲控制；脉冲异步时序逻辑电路中没有统一的时钟脉冲，因此，规定输入信号为脉冲信号，即控制电路状态转换的脉冲由电路输入端直接提供。第二，电路对过去输入信号的记忆是由触发器实现的。在同步时序逻辑电路中采用带时钟控制端的触发器；而在脉冲异步时序逻辑电路中既可用带时钟控制端的触发器，也可用非时钟控制触发器。

事实上，脉冲信号只不过是电平信号的一种特殊形式。所谓电平信号是指信号的"0"值和"1"值的持续时间是随意的，它以电位的变化作为信号的变化。而脉冲信号的"1"值仅仅维持一个固定的短暂时刻，它以脉冲信号的有、无标志信号的变化。显然，电平信号在短时间内的两次变化便形成了脉冲。至于电路中的触发器，则不管是哪种类型，都是由逻辑门加反馈回路构成的。将上述两个特点一般化，便可得到时序逻辑电路中更具一般性的另一类电路——电平异步时序逻辑电路。

1. 电平异步时序逻辑电路的结构模型

电平异步时序逻辑电路同样由组合电路和存储电路两部分组成，但存储电路是由反馈回路中的延迟元件构成的。延迟元件一般不用专门插入延迟线，而是利用组合电路本身固有的分布延迟在反馈回路中的"集总"。其一般结构模型如图 6.14 所示。

图中，$x_1, x_2, \cdots, x_n$ 为外部输入信号；$Z_1, Z_2, \cdots, Z_m$ 为外部输出信号；$Y_1, Y_2, \cdots, Y_r$ 为激励状态；$y_1, y_2, \cdots, y_r$ 为二次状态；$\Delta t_1, \Delta t_2, \cdots, \Delta t_r$ 为反馈回路中的时间延迟。电路可用以下方程组描述：

$$Z_i = f_i(x_1, \cdots, x_n, y_1, \cdots, y_r) \qquad i = 1, \cdots, m$$

$$Y_j = g_j(x_1, \cdots, x_n, y_1, \cdots, y_r) \qquad j = 1, \cdots, r$$

$$y_j(t + \Delta t_j) = Y_j(t)$$

由图 6.14 所示的结构模型及相应方程组可知，电路具有如下特点：

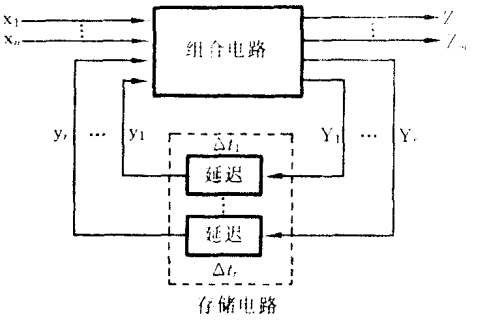


图 6.14　电平异步时序逻辑电路的结构模型

① 电路输出和状态的改变是由输入电位的变化直接引起的，由于电平异步时序逻辑电路可以及时地对输入信号的变化作出响应，所以工作速度较高。

② 电路的二次状态和激励状态仅仅相差一个时间延迟，即二次状态 y 是激励状态 Y 经过 Δt 延迟后的“重现”，因此，y 被命名为二次状态。当输入信号不变时，激励状态与二次状态相同，即 y＝Y，此时电路处于稳定状态。

③ 输入信号的一次变化可能引起二次状态的多次变化。当电路处在稳定状态下输入信号发生变化时，若激励状态 Y 的值与二次状态 y 的值是相同的，则电路处于稳定状态；若激励状态 Y 的值与二次状态 y 的值不同，则变化后的 Y 经过 Δt 延迟后形成新的二次状态 y 反馈到组合电路输入端，这个新的二次状态 y 又会引起输出 Z 和激励状态 Y 的变化，这是一个循环过程，该过程将一直进行到激励状态 Y 等于二次状态 y 为止。在变化过程终止前，电路处于不稳定状态；变化过程结束后，电路进入一个新的稳定状态。这一现象，是电平异步时序电路的一个重要特征。

2. 输入信号的约束

考虑到电平异步时序电路输入信号的变化将直接引起输出和状态的变化，为了保证电路可靠地工作，对输入信号有如下两条约束。

① 不允许两个或两个以上输入信号同时发生变化。因为客观上不可能有准确的“同时”，而微小的时差都可能使最终到达的状态不确定。

② 输入信号变化引起的电路响应必须完全结束后，才允许输入信号再次变化。换句话说，必须使电路进入稳定状态后，才允许输入信号发生变化。

以上两条是使电平异步时序电路能可靠工作的基本条件，通常将满足上述条件的工作方式称为基本工作方式，将按基本工作方式工作的电平异步时序逻辑电路称为基本型电路。

3. 描述方法

由于电平异步时序逻辑电路的组成与同步时序逻辑电路和脉冲异步时序逻辑电路不同，因此，电路的分析和设计方法以及分析和设计中使用的描述工具也不相同。在电平异步时序逻辑电路中，除了逻辑方程外，一般使用流程表和总态图描述一个电路的工作过程和逻辑功能。

流程表是用来反映电路输出信号、激励状态与电路输入信号、二次状态之间关系的一种表格形式。其一般格式如表 6.15 和表 6.16 所示。

表 6.15　Mealy 型流程表格式

二次状态	激励状态/输出		
		输入 x	
y		Y/Z	

表 6.16　Moore 型流程表格式

二次状态	激励状态			输出
		输入 x		
y		Y		Z

在构造流程表时，为了能够明显地区分电路的稳态和非稳态，当表中的激励状态与其对应的二次状态相同时，将激励状态加上圆圈，以表示电路处于稳态，否则，表示电路处于非稳态。其次，为了更好地体现不允许两个或两个以上输入信号同时变化的约束，将一位输入的各种取值按代码相邻的关系排列（类似卡诺图），以表示输入信号只能在相邻位置上发生变化。

例如，图 6.15(a)所示的是一个用与非门构成的基本 R-S 触发器，若对该电路的器件和连线位置稍作变动，将从电路输入到输出的延迟时间集总成反馈回路中的延迟元件，并将延迟前的状态用激励状态 Y 表示，而将经过延迟后的状态用二次状态 y 表示，即可将该触发器变成图 6.15(b)所示电平异步时序逻辑电路的结构模型。

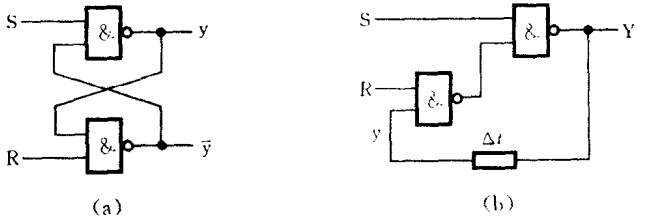


图 6.15　一个简单的电平异步时序逻辑电路

显然，基本 R-S 触发器是一个简单的电平异步时序逻辑电路，该电路的输出即状态，属于 Moore 型电平异步时序逻辑电路的特例。该电平异步时序逻辑电路

的激励方程为

$$Y=\bar{S}+Ry$$

根据激励方程和与非门构成的 R-S 触发器不允许两个输入信号同时为 0 的约束，可作出相应流程表，如表 6.17 所示。

表 6.17　R-S 触发器流程表

二次状态	激励状态 Y				输出
y	RS=00	RS=01	RS=11	RS=10	
0	d	⓪	⓪	1	0
1	d	0	①	①	1

由于电平异步时序逻辑电路在输入信号作用下存在稳态和非稳态，而且在同一种输入信号作用下，可能有一个稳态也可能有多个稳态，因此，为了对电路的工作状态和逻辑功能作出确切的说明，除了流程表和常用的时间图之外，引入了总态和总态图的概念。

总态是指电路输入和二次状态的组合，记作(x,y)。在流程表中，代表某个二次状态的一行和代表某种输入取值的一列的交叉点对应一个总态。当输入信号作相邻变化不引起电路状态变化时，在表内总态只作水平方向的移动。例如，表6.17中，当处在稳定总态(01,0)，输入 RS 由 01 变为 11 时，总态沿水平方向移动，到达稳定总态(11,0)，等待输入信号作新的变化。当输入信号作相邻变化引起状态改变时，总态先作水平移动，进入非稳定总态，然后再作垂直方向的移动，直至进入稳定总态为止。例如，表 6.17 中，当处在稳定总态(11,0)，输入 RS 由 11 变为 10 时，总态先作水平移动，进入非稳定总态(10,0)，由于此时激励状态由 0 变成了 1，紧跟着二次状态将由 0 变为 1，所以总态接着作垂直移动，进入稳定总态(10,1)。

总态图是反映稳定总态之间转移关系及相应输出的一种有向图。一个电平异步时序逻辑电路的逻辑功能，是由该电路在输入作用下稳定状态之间的转移关系以及各时刻的输出来体现的。总态图能够清晰地描述一个电路的逻辑功能。表 6.17 所示流程表对应的总态图如图6.16所示。

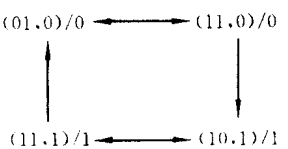


图 6.16　与表 6.17 对应的总态图

6.3.2　电平异步时序逻辑电路的分析

电平异步时序逻辑电路的分析过程比较简单，其一般步骤如下：

① 根据逻辑电路图写出输出函数和激励函数表达式；

② 作出流程表；

③ 作出总态图或时间图；

④ 说明电路逻辑功能。

下面举例说明电平异步时序逻辑电路的分析过程。

例 6.5 分析图 6.17 所示电平异步时序逻辑电路。

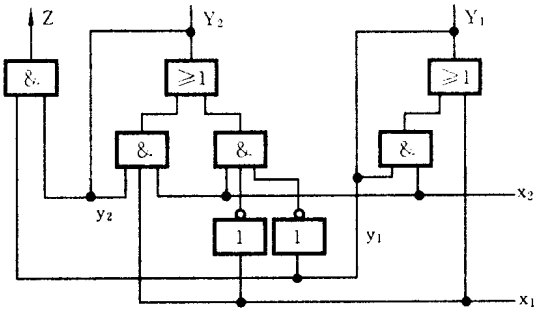


图 6.17 逻辑电路

解 该电路有两个外部输入 x_1、x_2；两条反馈回路，对应的激励状态为 Y_1、Y_2，二次状态为 y_1、y_2；一个外部输出 Z，输出与输入没有直接关系，仅仅是二次状态的函数，所以，该电路为 Moore 模型。

① 写出输出函数和激励函数表达式

$$Z = y_2 y_1$$

$$Y_2 = x_2 x_1 y_2 + x_2 \overline{x_1}\ \overline{y_1}$$

$$Y_1 = x_2 y_1 + x_1$$

② 作出流程表。根据激励函数和输出函数表达式可作出流程表，如表6.18所示。

表 6.18 流程表

二次状态		激励状态 Y_2Y_1				输出
y_2	y_1	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$	Z
0	0	⑥⓪	01	01	10	0
0	1	00	⑥①	⑥①	⑥①	0
1	1	00	01	⑪	01	1
1	0	00	01	11	⑩	0

③ 作出总态图。根据流程表上稳定总态之间的关系，可作出图 6.18 所示总态图。

为了更直观地描述电路功能，还可以作出时间图。假定电路初始总态为

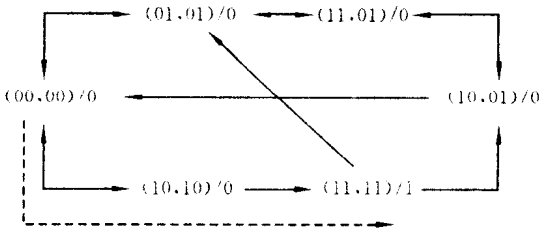


图 6.18　总态图

$(x_2x_1, y_2y_1)=(00,00)$，输入 x_2x_1 的变化序列为 00→10→11→01→00→01→11→10，根据流程表可作出总态和输出响应序列如下：

时 刻 t_i	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7
输入 x_2x_1	00	10	11	01	00	01	11	10
总态(x_2x_1, y_2y_1)	(00,00)	(10,00)* (10,10)	(11,10)* (11,11)	(01,11)* (01,01)	(00,01)* (00,00)	(01,00)* (01,01)	(11,01)	(10,01)
输出 Z	0	0	1	0	0	0	0	0

在总态响应序列中加“*”的表示是非稳定总态。根据以上总态和输出响应序列可作出时间图，如图 6.19 所示。

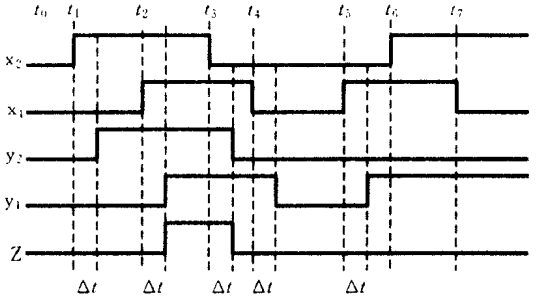


图 6.19　时间图

④ 说明电路功能。从总态图和时间图可以看出，仅当电路收到输入序列“00→10→11”时，才产生一个高电平输出信号，其他情况下均输出低电平。因此，该电路是一个“00→10→11”序列检测器。

6.3.3　电平异步时序逻辑电路的竞争

电平异步时序逻辑电路是利用各反馈回路的时间延迟实现记忆功能的。前面对电路进行分析时，没有对各反馈回路之间时间延迟的长短进行讨论，也就是说，是在假定各回路之间延迟时间相同的情况下对电路的工作过程进行分析的。事实上，各反馈回路的延迟时间往往各不相同。当电路中存在多条反馈回路，而各回路

之间的延时又互不相同时，则可能由于输入信号的变化在反馈回路之间引起竞争。这里的所谓竞争，是指当输入信号变化引起电路中两个或两个以上状态变量发生变化时，由于各反馈回路延迟时间的不同，使状态变量的变化有先有后而导致不同状态响应过程的现象。

根据竞争对电路状态转移产生的影响，可将竞争分为非临界竞争和临界竞争两种类型。若竞争的结果最终能到达预定的稳态，则称为非临界竞争；若竞争的结果可能使电路到达不同的稳态，即状态转移不可预测，则称为临界竞争。

例如，图 6.20 所示为某电平异步时序电路的结构框图，描述该电路的流程表如表 6.19 所示。

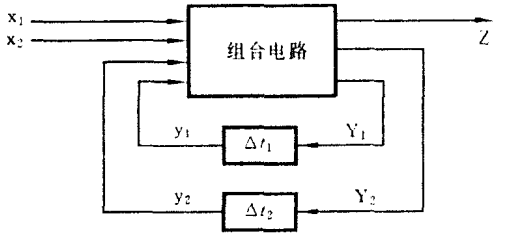


图 6.20 某电平异步时序电路框图

表 6.19 流程表

二次状态		激励状态 Y_2Y_1/输出 Z			
y_2	y_1	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
0	0	(00)/0	(00)/0	01/0	11/0
0	1	00/0	(01)/0	(01)/0	(01)/0
1	1	00/0	00/0	10/0	(11)/0
1	0	00/0	00/0	(10)/1	(10)/0

从表 6.19 可以看出，当电路处于稳定总态(00,00)、输入 x_2x_1 由 00→10 时，电路应经过非稳定总态(10,00)到达稳定总态(10,11)，由于此次输入变化引起激励状态 Y_2Y_1 从 00→11，即两个状态变量均发生变化，所以，当电路中两条反馈回路的延迟时间 Δt_1 和 Δt_2 不相等时，电路中将产生竞争。

此外，当电路处于稳定总态(10,11)、输入 x_2x_1 由 10→00 时，由于激励状态 Y_2Y_1 从 11→00，所以，电路同样可能发生竞争。

下面，按照两条反馈回路延迟时间 Δt_1 和 Δt_2 的大小关系，对上述两处输入信号变化引起的状态响应过程进行分析，讨论所存在的竞争各属于何种类型。

当电路处于稳定总态(00,00)、输入 x_2x_1 由 00→10 时，其状态响应过程如下。

① $\Delta t_2=\Delta t_1$：二次状态 y_2、y_1 将同时响应激励状态 Y_2、Y_1 的变化，即 y_2y_1 由 00→11，总态变化过程为(00,00)→(10,00)→(10,11)，即到达预定的稳定总态

(10,11)。

② $\Delta t_2 < \Delta t_1$：二次状态 y_2 对激励状态 Y_2 的响应先于 y_1 对 Y_1 的响应，即 y_2y_1 将由 00→10，总态变化过程为(00,00)→(10,00)→(10,10)，由于(10,10)是稳定总态，故电路停留在该稳定总态，即电路到达了一个非期望的稳定总态(10,10)。

③ $\Delta t_2 > \Delta t_1$：二次状态 y_2 对激励状态 Y_2 的响应落后于 y_1 对 Y_1 的响应，即 y_2y_1 将由 00→01，总态变化过程为(00,00)→(10,00)→(10,01)，由于(10,01)是稳定总态，故电路停留在该稳态，即电路到达了一个非期望的稳定总态(10,01)。

由此可见，此次输入信号变化，使电路最终到达的稳定状态随电路反馈回路中延迟时间的不同而不同，即状态转移不可预测，所以本次竞争为临界竞争。

当电路处于稳定总态(10,11)、输入 x_2x_1 由 10→00 时，其状态响应过程如下。

① $\Delta t_2 = \Delta t_1$：二次状态 y_2、y_1 将同时响应激励状态 Y_2、Y_1 的变化，即 y_2y_1 由 11→00，总态变化过程为(10,11)→(00,11)→(00,00)，到达预定的稳定总态(00,00)。

② $\Delta t_2 < \Delta t_1$：二次状态 y_2 对激励状态 Y_2 的响应先于 y_1 对 Y_1 的响应，总态变化过程为(10,11)→(00,11)→(00,01)→(00,00)，到达预定的稳定总态(00,00)。

③ $\Delta t_2 > \Delta t_1$：二次状态 y_2 对激励状态 Y_2 的响应落后于 y_1 对 Y_1 的响应，总态变化过程为(10,11)→(00,11)→(00,10)→(00,00)，到达预定的稳定总态(00,00)。

由此可见，无论反馈回路中延迟时间的大小如何，此次输入信号变化引起的竞争最终都能到达预定稳态，所以，本次竞争属于非临界竞争。

从上述分析不难得出用流程表检查电路竞争的**一般法则**：当从某一稳态出发，输入信号发生所允许的变化、引起两个或两个以上激励状态发生变化时，由于反馈回路之间延迟时间的不同会使电路产生竞争。若输入信号变化所到达的列只有一个稳态，则该竞争属于非临界竞争；若输入信号变化所到达的列有两个或两个以上稳态，则该竞争属于临界竞争。

显然，非临界竞争的存在不会影响电路的正确工作，但临界竞争的存在却将导致电路状态转换的不可预测。为了确保电平异步时序电路能可靠地实现预定功能，电路设计时必须避免发生临界竞争。该问题一般可在状态编码时解决，具体方法在 6.3.4 节中介绍。

*6.3.4　电平异步时序逻辑电路的设计

电平异步时序逻辑电路设计的一般步骤如下：

① 根据设计要求，建立原始流程表；

② 化简原始流程表，得到最简流程表；

③ 状态编码，得到二进制流程表；

④ 确定激励状态和输出函数表达式；

⑤ 画出逻辑电路图。

下面对完成上述步骤的具体方法分别进行介绍。

1. 建立原始流程表

原始流程表是按照电平异步时序逻辑电路的描述方法对设计要求的一种最原始的抽象。为了实现从一个逻辑问题的文字描述到流程表的过渡，在建立原始流程表时通常借助时间图或原始总态图，即首先根据题意画出典型输入、输出时间图或作出原始总态图，然后再逐步形成原始流程表。

根据时间图建立原始流程表的过程如下。

(1) 画出典型输入/输出时间图并设立相应状态

画典型输入/输出时间图应注意 3 点：

① 符合题意，即正确体现设计要求；

② 满足电平异步时序逻辑电路不允许两个或两个以上输入信号同时改变的约束条件；

③ 尽可能反映输入信号在各种取值下允许发生的变化。

作出输入/输出时间图后，按输入信号的变化进行时间划分，将每次变化作为一个新的输入，用不同时刻进行区分。由于电平异步时序逻辑电路约定，每次输入信号变化，必须保证电路进入稳定状态后才允许输入信号再次变化，所以，应根据题意设立与各时刻输入/输出对应的稳定状态。

(2) 建立原始流程表

根据时间图和所设立的状态建立原始流程表，一般分为 3 步进行。

① 画出原始流程表，并填入稳定状态和相应输出。由于根据时间图设立状态时，开始并不知道哪些输入取值可用同一状态表示，因此，对不同的输入取值总是设立不同的状态进行区分，这就使得原始流程表中每一行只有一个稳定状态。显然，每设立一个状态，在原始流程表上便占一行，每行对应一个二次状态。根据每个状态设立时的输入值，在表中找到相应列，并在该总态中填入与二次状态相同的激励状态和相应输出，即可得到原始流程表的稳定状态部分。

② 填入非稳定状态并指定非稳定状态下的输出，完善流程表。由于表中每行只有一个稳定状态，所以，在稳态下输入信号发生允许变化时，电路不可能直接进入另一个稳态。假定每次输入信号发生变化时，电路总是经过一个非稳定状态后进入另一个稳定状态，根据时间图中的状态转移关系，可在原始流程表中填入相应

的非稳定状态。

在填写非稳定状态时应注意，由于时间图中不一定反映了所有输入信号变化的情况，所以往往要根据题意作适当的补充和完善。当从某一稳态出发，输入信号发生允许变化所引起的状态转移，不能用时间图中所设立的状态来表示时，则应根据题意补充新的状态，以便无遗漏地反映设计要求。

为了使电路经过非稳定状态时，其输出不产生尖脉冲信号，规定非稳定状态下输出指定的法则为：若转换前后两个稳定状态的输出相同，则指定非稳定状态下的输出与稳态下的输出相同；若转换前后两个稳定状态的输出不同，则可指定非稳定状态下的输出为任意值“d”。

③ 填入无关状态和无关输出。因为不允许两个或两个以上输入信号同时改变，所以，对稳态下输入不允许到达的列，在相应处填入任意状态和任意输出，用“d”表示，即作为无关处理。

至此，可得到一个完整的原始流程表。

例 6.6　某电平异步时序逻辑电路有两个输入端 x_1 和 x_2，一个输出端 Z。输出与输入之间的关系为：只要 $x_1x_2=00$，则 $Z=0$，在此之后当 $x_1x_2=01$ 或 10 时，$Z=1$；只要 $x_1x_2=1$，则 $Z=1$，在此之后当 $x_1x_2=01$ 或 10 时，$Z=0$。作出该电路的原始流程表。

解　根据借助时间图建立原始流程表的方法，形成该电路原始流程表的过程如下。

(1) 画出典型输入/输出时间图并设立相应状态

根据题意，可画出该电路典型输入/输出时间图，如图 6.21 所示。

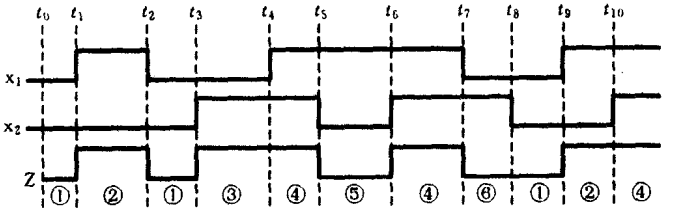


图 6.21　典型输入/输出时间图

图中，假定 t_0 为起始时刻，在该时刻输入 $x_1x_2=00$，输出 Z 为 0，用状态①表示；在 t_1 时刻，输入 x_1x_2 由 00→10，输出 Z 为 1，用状态②表示；在 t_2 时刻，输入 x_1x_2 由 10→00，输出 Z 为 0，因为任何时刻只要 $x_1x_2=00$，则输出 Z 为 0，故与 t_0 时刻相同，仍用状态①表示；t_3 时刻，输入 x_1x_2 由 00→01，输出 Z 为 1，用状态③表示；t_4 时刻，输入 x_1x_2 由 01→11，输出 Z 为 1，用状态④表示；t_5 时刻，输入 x_1x_2 由 11→10，输出 Z 为 0，用状态⑤表示；t_6 时刻，输入 x_1x_2 由 10→11，输出 Z 为 1，因为任何

时刻只要 $x_1x_2=11$，则输出 Z 为 1，故与 t_4 时刻相同，仍用状态④表示；t_7 时刻，输入 x_1x_2 由 11→01，输出 Z 为 0，用状态⑥表示；t_8 时刻，输入 x_1x_2 由 01→00，输出 Z 为 0，与 t_0 时刻相同，用状态①表示；t_9 时刻，输入 x_1x_2 由 00→10，输出 Z 为 1，与 t_1 时刻相同，用状态②表示；t_{10} 时刻，输入 x_1x_2 由 10→11，输出 Z 为 1，与 t_4 时刻相同，用状态④表示。

(2) 建立原始流程表

① 画出原始流程表并填入稳定状态和相应输出。图 6.20 中共设立了 6 个稳定状态，按照每行一个稳定状态，将时间图中设立的各状态和相应的输出填入流程表中与各二次状态、输入取值对应的行、列位置，即可得到表 6.20 所示的部分流程表(Ⅰ)。

表 6.20 部分流程表(Ⅰ)

二次状态 y	激励状态 Y/输出 Z			
	$x_1x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
1	①/0			
2				②/1
3		③/1		
4			④/1	
5				⑤/0
6		⑥/0		

② 填入非稳定状态并指定非稳定状态下的输出，完善流程表。根据时间图中的状态转移关系和非稳定状态下输出指定的法则，在流程表中填入非稳定状态并指定其输出，即可得到表 6.21 所示的部分流程表(Ⅱ)。在图 6.21 所示时间图中，未体现出电路处于稳定状态③、输入 x_1x_2 由 01→00 的情况，此时，可根据题意令其转向状态①；同样，当处于稳态⑤，输入 x_1x_2 由10→00时，可令其转向状态①；当处于稳态⑥，输入 x_1x_2 由 01→11 时，可令其转向状态④。由于时间图中设立的 6 个状态已能反映电路在各种输入取值作用下的状态响应，所以无需补充新的状态。

表 6.21 部分流程表(Ⅱ)

二次状态 y	激励状态 Y/输出 Z			
	$x_1x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
1	①/0	3/d		2/d
2	1/d		4/1	②/1
3	1/d	③/1	4/1	
4		6/d	④/1	5/d
5	1/0		4/d	
6	1/0	⑥/0	4/d	

③ 填入无关状态和无关输出。对表 6.21 中各稳定状态下输入变化不允许到达的列，在相应位置填入无关状态和无关输出“d”，即可得到表 6.22 所示的完整流程表。

表 6.22　完整流程表

二次状态 y	激励状态 Y/输出 Z			
	$x_1x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
1	①/0	3/d	d/d	2/d
2	1/d	d/d	4/1	②/1
3	1/d	③/1	4/1	d/d
4	d/d	6/d	④/1	5/d
5	1/0	d/d	4/d	⑤/0
6	1/0	⑥/0	4/d	d/d

2. 化简原始流程表

在建立原始流程表时，设计者一般将注意力集中在如何正确、清晰地描述给定的设计要求上，并没有刻意追求如何使用最少的状态，因而所得到的流程表往往不是最简的。在进行电平异步时序逻辑电路设计时，流程表中的状态数目决定了电路中反馈回路的数目。显然，状态数目的多少与电路的复杂程度直接相关。为了获得一种经济、合理的设计方案，必须对原始流程表进行化简，求出最简流程表。

原始流程表的化简是建立在状态相容这一概念基础之上的。由于原始流程表中的每一行代表一个稳定状态，因而相容状态的概念被引申为相容行的概念。

对于原始流程表中的某两行，如果每一列确定的输出相同，且确定的激励状态相同、交错、循环、相容或为各自本身，则称这两行为相容行。在检查输出时，对于一个确定而另一个任意，或者两个均任意的情况，都作为相同情况处理。在检查激励状态时，按以下原则确定稳定状态、非稳定状态和任意状态的相容性：

① 稳定状态ⓘ和非稳定状态 i 是相容的；

② 若稳定状态ⓘ和ⓙ是相容的，则稳定状态ⓘ和非稳定状态 j 是相容的；

③ 若稳定状态ⓘ和ⓙ是相容的，则非稳定状态 i 和 j 是相容的；

④ 稳定状态ⓘ和非稳定状态 i 均与任意状态“d”相容；任意状态“d”与任意状态“d”相容。

引入相容行的概念后，原始流程表的化简过程与不完全给定状态表的化简过程类似，同样可用隐含表、合并图和覆盖闭合表作为化简工具，其一般步骤如下：

① 作隐含表，找出相容行；

② 作合并图，求出最大相容行类；

③ 从相容行类中选择一个最小闭覆盖；

④ 作出最简流程表。

对最小闭覆盖中各相容行类中的相容行进行合并时注意，当输出存在确定值和任意值“d”时，合并后取确定值；当激励状态存在稳定状态和非稳定状态时，合并时取稳定状态；当激励状态存在确定状态和任意状态“d”时，合并时取确定状态。

下面举例说明原始流程表的化简过程。

例 6.7 化简表 6.22 所示原始流程表。

解 根据化简原始流程表的方法和步骤，化简过程如下。

① 作隐含表，找相容行。图 6.22 给出了与表 6.22 所示原始流程表对应的隐含表。根据相容行的判断规则，可找出相容行对：(1,2)，(1,3)，(2,3)，(2,6)，(3,5)，(4,5)，(4,6)，(5,6)。

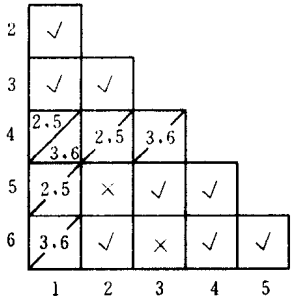


图 6.22 隐含表

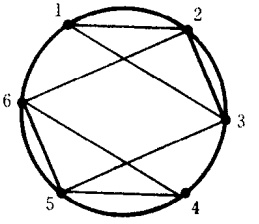


图 6.23 合并图

② 作合并图，求最大相容行类。根据所得出的相容行对，可作出合并图如图 6.23 所示。可见，最大相容行类为(1,2,3)，(4,5,6)，(3,5)，(2,6)。

③ 选择一个最小闭覆盖。显然，选择由两个最大相容行类构成的集合{(1,2,3)，(4,5,6)}，便可满足覆盖、闭合和最小 3 个条件。因此，该集合即为表 6.22 所示原始流程表的最小闭覆盖。

④ 作出最简流程表。将最小闭覆盖中的最大相容类(1,2,3)，(4,5,6)分别用 A、B 代替，即可得到最简流程表，如表 6.23 所示。

表 6.23 最简流程表

二次状态 y	激励状态 Y/输出 Z			
	$x_1x_2=00$	$x_1x_2=01$	$x_1x_2=11$	$x_1x_2=10$
A	Ⓐ/0	Ⓐ/1	B/1	Ⓐ/1
B	A/0	Ⓑ/0	Ⓑ/1	Ⓑ/0

3. 状态编码

状态编码的任务是根据化简后的状态数目确定二进制代码的位数，并选择一种合适的状态分配方案，将每个状态用一个二进制代码表示。在同步时序逻辑电路设计中，选择分配方案时需考虑的主要问题是如何使电路结构最简单。而在电

平异步时序逻辑电路设计中，确定分配方案时应考虑的主要问题是如何避免反馈回路之间的临界竞争，保证电路可靠地实现预定功能。

为了消除临界竞争，在确定状态分配方案时常用以下几种方法。

(1) 相邻状态，相邻分配

由6.3.3节对电平异步时序逻辑电路中竞争现象的分析可知，仅当输入变化引起两个或两个以上状态变量发生变化时，电路中才会产生竞争。换而言之，如果能保证每次状态转移时，仅有一个状态变量变化，则不会产生竞争。据此，可通过"相邻状态，相邻分配"的方法消除竞争。所谓相邻状态，是指稳态下输入取值作相邻变化时，需要直接发生转换的状态。而所谓相邻分配是指分配给相邻状态的代码为相邻代码(仅一位不同)。

为了找出流程表中各状态的相邻关系，通常借助状态相邻图。画状态相邻图的方法是：先将流程表中的每一个状态用一个圆圈表示，然后从流程表中每一个稳态出发，找出输入取值作相邻变化时的下一个稳态，并用有向线段将其连接起来，表示这两个状态为相邻状态。

例6.8　对表6.24所示流程表进行状态编码，作出二进制流程表。

表6.24　流程表

二次状态	激励状态 Y			
y	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ	Ⓐ	B	C
B	A	Ⓑ	Ⓑ	Ⓑ
C	Ⓒ	A	D	Ⓒ
D	C	Ⓓ	Ⓓ	Ⓓ

解　根据"相邻状态，相邻分配"的法则，首先作出表6.24所示流程表的状态相邻图，如图6.24所示。由相邻图可知，A和B、A和C、C和D为相邻状态，状态分配时应令其代码相邻。流程表中共有4个状态，需两位代码，设二次状态用y_2、y_1表示，可选择状态分配方案，如图6.25所示。即：用00表示A，01表示B，10表示C，11表示D。

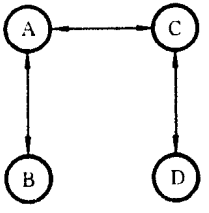


图6.24　状态相邻图

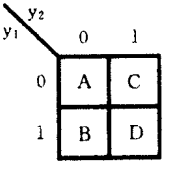


图6.25　状态分配方案

将表 6.24 中的状态用相应二进制编码表示，即可得到表 6.25 所示二进制流程表。由该流程表可知，在任一稳态下输入信号发生允许变化时，均不会引起两个状态变量发生变化，因而从根本上消除了竞争现象。

表 6.25 二进制流程表

二次状态		激励状态 Y_2Y_1			
y_2	y_1	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
0	0	(00)	(00)	01	10
0	1	00	(01)	(01)	(01)
1	1	10	(11)	(11)	(11)
1	0	(10)	00	11	(10)

值得指出的是，并不是所有流程表都能直接用最少的二进制代码位数实现“相邻状态，相邻分配”。设状态数为 n，二进制代码位数为 m，则 n 和 m 的关系为 $2^m \geqslant n > 2^{m-1}$。由于一个 m 位代码最多只有 m 个相邻代码，因此，当相邻图上状态的最大相邻状态数 L 大于 m 时，则不可能用 m 位代码实现相邻分配。通常解决的办法是增加二进制代码位数，实现相邻分配。由于代码位数对应着电路中的反馈回路数，因此，这将增加电路的复杂性。

(2) 增加过渡状态，实现相邻分配

对于某些流程表，尽管相邻图上状态的最大相邻状态数 L 不大于状态分配的最小代码位数 m，但状态之间的相邻关系出现由奇数个状态构成的闭环，因而无法直接实现状态的相邻分配。一种常用的方法是通过增加过渡状态，实现相邻分配，得到一个无竞争的二进制流程表。

例 6.9 对表 6.26 所示流程表进行状态编码，得到二进制流程表。

表 6.26 流程表

二次状态 y	激励状态 Y			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ	B	C	Ⓐ
B	A	Ⓑ	Ⓑ	C
C	A	B	Ⓒ	Ⓒ

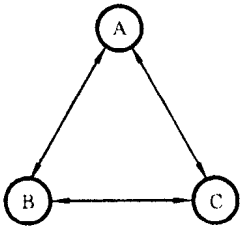


图 6.26 状态相邻图

解 根据表 6.26 所示流程表，可作出状态相邻图，如图 6.26 所示。尽管相邻图上每个状态只有两个相邻状态，但由于 3 个状态之间的相邻关系构成一个闭环，所以用两位代码无论怎样分配均无法满足其相邻关系。如果在状态 A 和 C 之间增加过渡状态 D，将 A→C 改为 A→D→C，C→A 改为 C→D→A，那么，表 6.26 所示流程表可被修改成如表 6.27 所示。修改后的流程表中增加了新的一行，但该行

没有稳定状态，因为状态 D 仅在稳态 A 和 C 发生转换时完成过渡作用。作出表 6.26 和表 6.27 的总态图(略)可以表明，增加过渡状态后的流程表与原流程表描述的逻辑功能相同。表 6.27 所示流程表的状态相邻图如图6.27所示。显然，用两位代码可以方便地满足图 6.27 所示的相邻关系。

表 6.27　增加过渡状态后的流程表

二次状态	激励状态 Y			
y	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ	B	D	Ⓐ
B	A	Ⓑ	Ⓑ	C
C	D	B	Ⓒ	Ⓒ
D	A	d	C	d

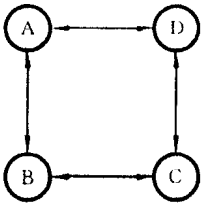


图 6.27　增加过渡状态后的状态相邻图

设二次状态用 y_2、y_1 表示，令 y_2y_1 取值 00 表示 A，01 表示 B，10 表示 D，11 表示 C，即可得到与表 6.27 对应的二进制流程表，如表 6.28 所示，该流程表描述的电路中不存在竞争。

表 6.28　二进制流程表

二次状态	激励状态 Y_2Y_1			
y_2　y_1	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
0　0	⑳	01	10	⑳
0　1	00	㉑	㉑	11
1　1	10	01	⑪	⑪
1　0	00	d	11	d

(3) 允许非临界竞争，避免临界竞争

由于非临界竞争并不影响电路正常工作，所以，在进行状态分配时，只需避免临界竞争。对于有的流程表，虽然无法用最少位数的代码实现无竞争的状态分配，但可以通过将竞争限制在只有一个稳态的列，即允许非临界竞争，从而实现无临界竞争的状态分配。

例 6.10　对表 6.29 所示流程表进行状态编码，得到二进制流程表。

表 6.29　流程表

二次状态	激励状态 Y				输　出
y	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$	Z
A	Ⓐ	C	D	Ⓐ	0
B	A	C	Ⓑ	Ⓑ	0
C	A	Ⓒ	Ⓒ	B	0
D	A	C	Ⓓ	A	1

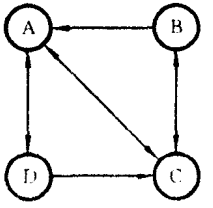


图 6.28　状态相邻图

解 根据表 6.29 所示流程表可作出状态相邻图，如图 6.28 所示。显然，用两位二进制代码无法实现相邻状态相邻分配。解决的方法之一是通过增加过渡状态和增加代码位数实现相邻分配，但这样处理的结果必然增加电路的复杂性。解决该问题的另一种方法是允许非临界竞争，避免临界竞争。观察表6.29不难发现，状态 A 和 C 之间的转换只发生在 $x_2x_1=00$ 和 $x_2x_1=01$ 这两列，而这两列各只有一个稳定状态，这就意味着 A 和 C 发生转换时，即使产生竞争也属于非临界竞争。即在状态分配时 A 和 C 可以不相邻。排除 A 和 C 的相邻关系后，状态编码只需满足 A 和 B、A 和 D、D 和 C、C 和 B 相邻即可。

设二次状态用 y_2、y_1 表示，令 y_2y_1 取值 00 表示 A，01 表示 B，10 表示 D，11 表示 C，将其代入表 6.29，即可得到表 6.30 所示二进制流程表。该流程表描述的电路不会产生临界竞争。

表 6.30 二进制流程表

二次状态	激励状态 Y_2Y_1				输出
y_2 y_1	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$	Z
0 0	⑩	11	10	⑩	0
0 1	00	11	⑪	⑪	0
1 1	00	⑪	⑪	01	0
1 0	00	11	⑩	00	1

4. 确定激励状态和输出函数表达式

二进制流程表给出了激励状态、输出函数与电路输入和二次状态之间的取值关系。根据流程表可作出激励状态、输出函数的卡诺图，化简后即可得到激励状态和输出函数的最简表达式。例如，根据表 6.30 可作出 Y_2、Y_1 和输出 Z 的卡诺图，如图 6.29 所示。

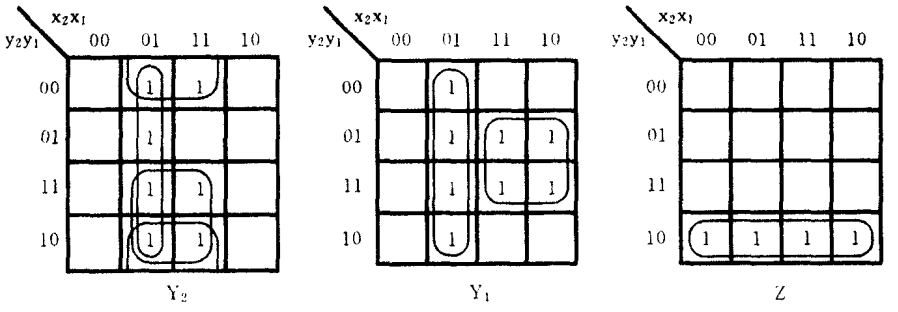


图 6.29 激励状态和输出函数卡诺图

化简后可得到激励状态和输出函数表达式为

$$Y_2 = \bar{x}_2 x_1 + x_1 y_2 + x_1 \bar{y}_1$$

$$Y_1 = \bar{x}_2 x_1 + x_2 y_1$$

$$Z = y_2 \bar{y}_1$$

根据激励状态和输出函数表达式，即可画出相应逻辑电路图(略)。

前面对电平异步时序逻辑电路设计的各主要步骤和方法进行了讨论。为了使读者系统地掌握设计的全过程，下面给出一个设计实例。

例 6.11　用与非门设计一个单脉冲发生器，电路结构框图如图 6.30 所示。

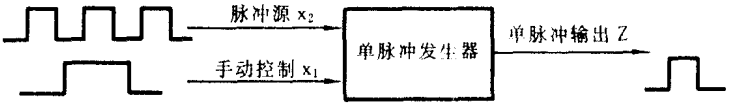


图 6.30　单脉冲发生器的结构框图

该电路有两个输入端 x_1、x_2 和一个输出端 Z。x_2 接时钟脉冲源、x_1 接手动控制按钮。当不按下按钮($x_1=0$)时，x_2 端的脉冲被封锁，输出 Z 为 0，无脉冲输出；当按下按钮并释放(x_1 由 0→1 再由 1→0)之后，输入端 x_2 出现的第一个完整脉冲被送至输出端 Z，即用手启动一次，输出一个完整脉冲。电路规定每启动一次，必须在输出一个完整脉冲后才可再次启动。

解　单脉冲发生器是一种在系统调试、维修、测试中常用的逻辑电路，主要用来控制系统运行于单步工作状态。根据给定要求，设计过程如下。

① 建立原始流程表。根据题意可作出典型输入/输出时间图，如图 6.31 所示。

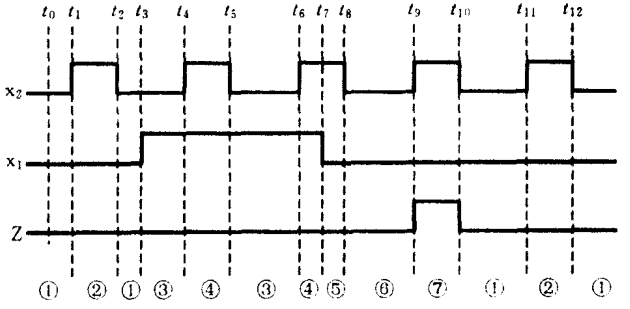


图 6.31　典型时间图

图中，按照输入信号的变化进行时间划分后，根据题意共设立了 7 个不同状态。其中，t_0 时刻 $x_2x_1=00$，启动信号和脉冲信号均未出现，输出 Z 为 0，设用状态①表示；t_1 时刻 $x_2x_1=10$，有脉冲出现但没有启动信号，输出 Z 为 0，设用状态②表示；t_2 时刻与 t_0 时刻相同；t_3 时刻 $x_2x_1=01$，有启动信号但无脉冲信号，输出 Z 为 0，设用状态③表示；t_4 时刻 $x_2x_1=11$，启动信号和脉冲信号同时出现，输出 Z 为 0，设

用状态④表示；t_5时刻与t_3时刻相同；t_6时刻与t_4时刻相同；t_7时刻$x_2x_1=10$，此时x_2端有启动信号结束后的不完整脉冲，输出Z为0，设用状态⑤表示；t_8时刻$x_2x_1=00$，此时启动信号已结束，但第一个完整脉冲尚未出现，输出Z为0，设用状态⑥表示；t_9时刻，出现了启动信号结束后的第一个完整脉冲，输出Z为1，产生一个完整输出脉冲，设用状态⑦表示。此后，电路将重复此过程，实现每启动一次，输出一个完整脉冲的功能。

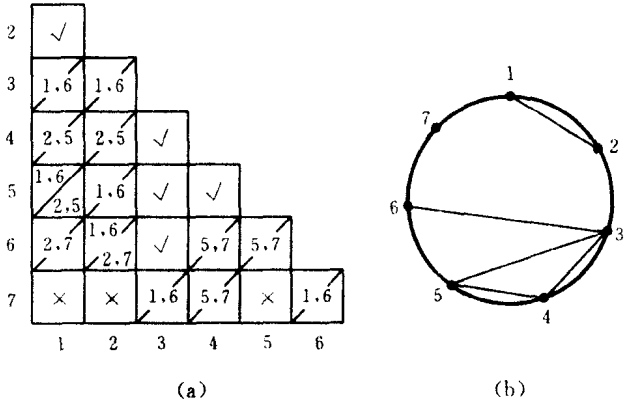
根据时间图中设立的状态可建立原始流程表，如表6.31所示。

表6.31 原始流程表

二次状态	激励状态Y/输出Z			
y	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
1	①/0	3/0	d/d	2/0
2	1/0	d/d	4/0	②/0
3	6/0	③/0	4/0	d/d
4	d/d	3/0	④/0	5/0
5	6/0	d/d	d/d	⑤/0
6	⑥/0	d/d	d/d	7/d
7	1/d	d/d	d/d	⑦/1

由于问题中规定每启动一次，必须输出一个完整脉冲后才能再次启动，所以，处在稳态⑤时输入取值不允许从10→11，处在稳态⑥时输入取值不允许从00→01，处在稳态⑦时输入取值不允许从10→11。因此，在流程表上的相应位置填入任意状态和任意输出"d"。

② 化简流程表。根据相容行的判断法则，可作出与表6.31对应的隐含表，如图6.32(a)所示。由隐含表可得到相容行对(1,2)、(3,4)、(3,5)、(3,6)、(4,5)。据此，可作出状态合并图，如图6.32(b)所示，其最大相容行类为(1,2)、(3,4,5)、



(a) (b)

图6.32 隐含表和状态合并图

(3,6)、(7)。

根据选择最小闭覆盖的条件，可选择相容行类集合{(1,2),(3,4,5),(6),(7)}。令(1,2)用 A 表示，(3,4,5)用 B 表示，(6)用 C 表示，(7)用 D 表示，合并后的最简流程表如表 6.32 所示。

表 6.32　最简流程表

二次状态 y	激励状态 Y/输出 Z			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ/0	B/0	B/0	Ⓐ/0
B	C/0	Ⓑ/0	Ⓑ/0	Ⓑ/0
C	Ⓒ/0	d/d	d/d	D/d
D	A/d	d/d	d/d	Ⓓ/1

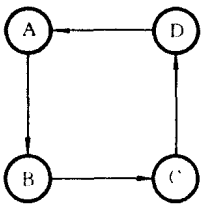


图 6.33　状态相邻图

③ 状态编码。根据表 6.32 可作出状态相邻图，如图 6.33 所示。设二次状态用 y_2、y_1表示，为了满足图 6.33 所示的相邻关系，可令 y_2y_1取值 00 表示 A，01 表示 B，11 表示 C，10 表示 D。将各状态的编码代入表 6.32，即可得到表 6.33 所示二进制流程表。

④ 确定激励状态和输出函数表达式。根据二进制流程表可作出激励状态和输出函数卡诺图，如图 6.34 所示。

用卡诺图化简后，可得到激励状态和输出函数的最简表达式：

表 6.33　二进制流程表

二次状态 y_2 y_1	激励状态 Y_2Y_1/输出 Z			
	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
0 0	(00)/0	01/0	01/0	(00)/0
0 1	11/0	(01)/0	(01)/0	(01)/0
1 1	(11)/0	d/d	d/d	10/d
1 0	00/d	d/d	d/d	(10)/1

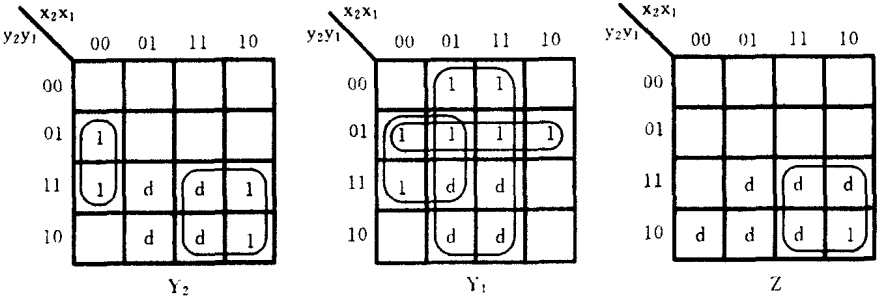


图 6.34　激励函数和输出函数卡诺图

$$Y_2 = x_2y_2 + \bar{x}_2\bar{x}_1y_1 = \overline{\overline{x_2y_2} \cdot \overline{x_2\,x_1y_1}}$$

$$Y_1 = x_1 + \bar{x}_2y_1 + \bar{y}_2y_1 = \overline{\overline{x_1} \cdot \overline{x_2y_2} \cdot y_1}$$

$$Z = x_2y_2 = \overline{\overline{x_2y_2}}$$

⑤ 画出逻辑电路图。根据激励状态和输出函数的与非-与非表达式，可画出用与非门实现给定功能的逻辑电路，如图 6.35 所示。

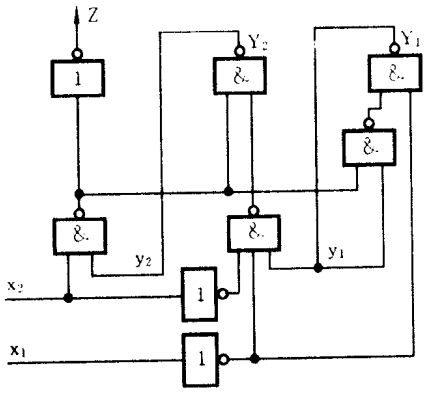


图 6.35　逻辑电路

习　题　六

6.1　分析图 6.36 所示脉冲异步时序逻辑电路。

(1) 作出状态表和状态图；

(2) 说明电路逻辑功能。

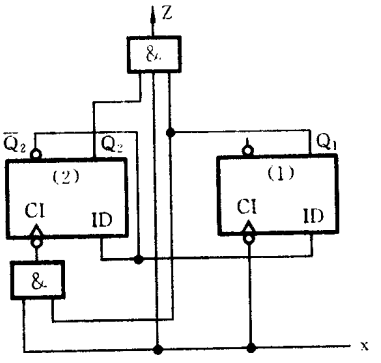


图 6.36　逻辑电路

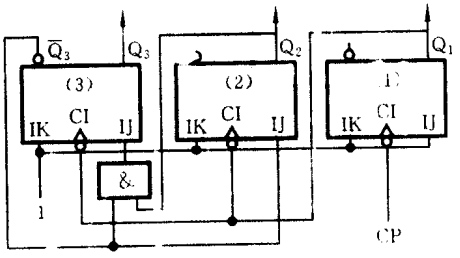


图 6.37　逻辑电路

6.2　分析图 6.37 所示脉冲异步时序逻辑电路。

(1) 作出状态表和时间图；

(2) 说明电路逻辑功能。

6.3　分析图 6.38 所示脉冲异步时序逻辑电路。

(1) 作出状态表和状态图；

(2) 说明电路逻辑功能。

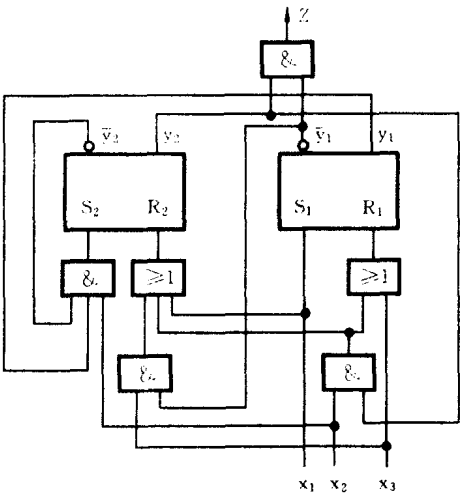


图 6.38　逻辑电路

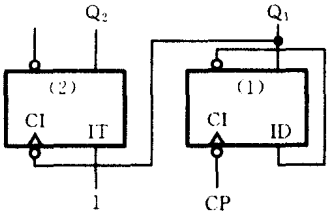


图 6.39　逻辑电路

6.4　分析图 6.39 所示脉冲异步时序逻辑电路，作出时间图并说明该电路逻辑功能。

6.5　用 D 触发器作为存储元件，设计一个脉冲异步时序逻辑电路。该电路在输入端 x 的脉冲作用下，实现 3 位二进制减 1 计数的功能，当电路状态为“000”时，在输入脉冲作用下输出端 Z 产生一个借位脉冲，平时 Z 输出 0。

6.6　用 T 触发器作为存储元件，设计一个脉冲异步时序逻辑电路，该电路有两个输入 x_1 和 x_2，一个输出 Z，当输入序列为“$x_1-x_1-x_2$”时，在输出端 Z 产生一个脉冲，平时 Z 输出为 0。

6.7　试用与非门构成的基本 R-S 触发器设计一个模 4 加 1 计数器。

6.8　分析图 6.40 所示电平异步时序逻辑电路，作出流程表。

6.9　分析图 6.41 所示电平异步时序逻辑电路，作出流程表和总态图，说明该电路的逻辑功能。

6.10　某电平异步时序逻辑电路的流程表如表 6.34 所示。作出输入 x_2x_1 变化序列为 00→01→11→10→11→01→00 时的总态(x_2x_1,y_2y_1)响应序列。

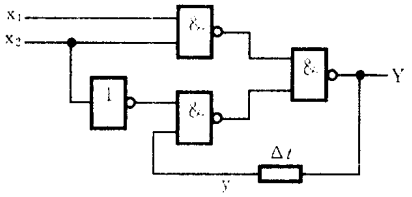


图 6.40　逻辑电路

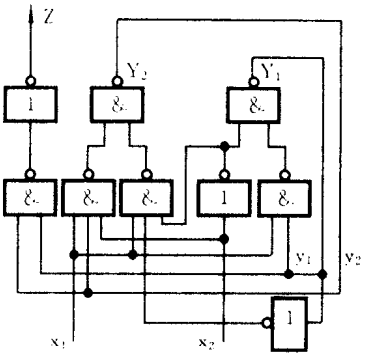


图 6.41　逻辑电路

表 6.34　流程表

二次状态	激励状态 Y_2Y_1/输出 Z			
y_2　y_1	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
0　0	(00)/0	01/0	01/0	10/0
0　1	00/0	(01)/0	(01)/0	11/0
1　1	00/0	01/0	10/0	(11)/0
1　0	00/d	00/1	(10)/1	(10)/1

6.11　某电平异步时序逻辑电路有一个输入 x 和一个输出 Z，每当输入 x 出现一次 0→1→0 的跳变后，当 x 为 1 时输出 Z 为 1，典型输入/输出时间图如图 6.42所示。试建立该电路的原始流程表。

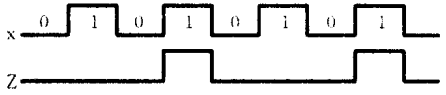


图 6.42　时间图

6.12　简化表 6.35 所示的原始流程表。

表 6.35　原始流程表

二次状态	激励状态/输出(Y/Z)			
y	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
1	①/0	5/d	d/d	2/d
2	1/d	d/d	3/d	②/0
3	d/d	5/d	③/1	4/d
4	1/d	d/d	3/d	④/1
5	1/d	⑤/0	6/d	d/d
6	d/d	5/d	⑥/0	4/d

6.13　图 6.43 为某电平异步时序逻辑电路的结构框图。图中，

$$Y_2 = x_2y_2 + \bar{x}_1y_2 + x_2\bar{x}_1y_1$$

$$Y_1 = x_2x_1 + \bar{x}_2\bar{x}_1y_2 + x_1y_2\bar{y}_1$$

$$Z = y_2y_1$$

试问该电路中是否存在竞争？若存在，请说明竞争类型？

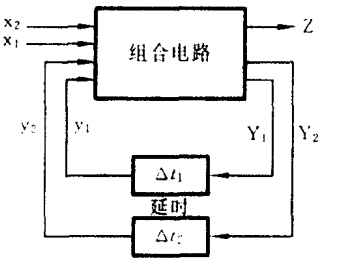


图 6.43　结构框图

6.14　对表 6.36 所示的最简流程表进行无临界竞争的状态编码，并确定激励状态和输出函数表达式。

表 6.36　最简流程表

二次状态	激励状态 Y/输出 Z			
y	$x_2x_1=00$	$x_2x_1=01$	$x_2x_1=11$	$x_2x_1=10$
A	Ⓐ/0	Ⓐ/0	Ⓐ/0	C/d
B	Ⓑ/0	A/0	C/d	Ⓑ/0
C	B/d	A/d	Ⓒ/1	Ⓒ/1

6.15　某电平异步时序逻辑电路有两个输入 x_1、x_2和一个输出 Z。当 $x_2=1$ 时，Z 总为 0；当 $x_2=0$ 时，x_1的第一次从 0→1 的跳变使 Z 变为 1，该 1 输出信号一直保持到 x_2由 0→1，才使 Z 为 0。试用与非门实现该电路功能。



中规模通用集成电路及其应用

集成电路由SSI发展到MSI、LSI和VLSI后，单个芯片的功能大大增强。一般地，在SSI中仅仅是基本器件（如逻辑门或触发器）的集成，在MSI中已是逻辑部件（如译码器、寄存器等）的集成，而在LSI和VLSI中则是一个数字子系统或整个数字系统（如微处理器、单片机）的集成。因此，采用中、大规模集成电路组成数字系统具有体积小、功耗低、可靠性高等优点，且易于设计、调试和维护。

各种中规模通用集成电路本身就是一种完美的逻辑设计作品，使用时只需适当地进行连接，就能实现预定的逻辑功能。另外，由于它们所具有的通用性、灵活性及多功能性，使之除完成基本功能之外，还能以它们为基本器件组成各类逻辑部件和数字系统，有效地实现各种逻辑功能。

本章主要讨论最常用的几种中规模通用集成电路及其应用。

7.1 常用中规模组合逻辑电路

使用最广泛的中规模组合逻辑集成电路有二进制并行加法器、译码器、编码器、多路选择器和多路分配器等。

7.1.1 二进制并行加法器

二进制并行加法器是一种能并行产生两个 n 位二进制数“算术和”的逻辑部件。按其进位方式的不同，可分为串行进位二进制并行加法器和超前进位二进制并行加法器两种类型。

1. 构成思想

(1) 串行进位二进制并行加法器

串行进位二进制并行加法器是由全加器级联构成的，高位的“和”依赖于来自

低位的进位输入。4 位串行进位二进制并行加法器的结构框图如图 7.1 所示。

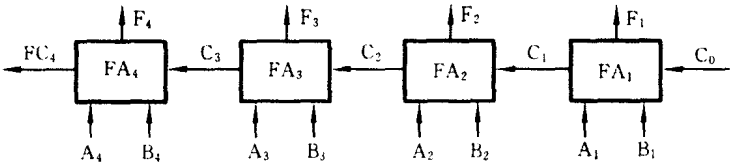


图 7.1　串行进位 4 位二进制并行加法器的结构框图

串行进位二进制并行加法器的特点是：被加数和加数的各位能同时并行到达各位的输入端，而各位全加器的进位输入则是按照由低位向高位逐级串行传递的，各进位形成一个进位链。由于每一位相加的“和”都与本位进位输入有关，所以，最高位必须等到各低位全部相加完成并送来进位信号之后才能产生运算结果。显然，这种加法器运算速度较慢，而且位数越多，速度就越低。

为了提高加法器的运算速度，必须设法减小或去除由于进位信号逐级传送所花费的时间，使各位的进位直接由加数和被加数来决定，而无须依赖低位进位。根据这一思想设计的加法器称为超前进位（又称先行进位）二进制并行加法器。

(2) 超前进位二进制并行加法器

超前进位二进制并行加法器是根据输入信号同时形成各位向高位“进位”的二进制并行加法器。

根据全加器的功能，可写出第 i 位全加器的进位输出函数表达式为

$$\begin{aligned} C_i &= \overline{A}_i B_i C_{i-1} + A_i \overline{B}_i C_{i-1} + A_i B_i \overline{C}_{i-1} + A_i B_i C_{i-1} \\ &= (A_i \oplus B_i) C_{i-1} + A_i B_i \end{aligned}$$

由进位函数表达式可知：当第 i 位的被加数 A_i 和加数 B_i 均为 1 时，有 $A_iB_i=1$，不论低位运算结果如何，本位必然产生进位输出（即 $C_i=1$），据此，定义 $G_i=A_iB_i$ 为**进位产生函数**；当 $A_i \oplus B_i=1$ 时，可使得 $C_i=C_{i-1}$，即当 $A_i \oplus B_i=1$ 时，来自低位的进位输入能传送到本位的进位输出。所以，定义 $P_i=A_i \oplus B_i$ 为**进位传递函数**。将 P_i 和 G_i 代入全加器的“和”及“进位”输出表达式，可得到

$$F_i = A_i \oplus B_i \oplus C_{i-1} = P_i \oplus C_{i-1}$$

$$C_i = P_i C_{i-1} + G_i$$

4 位二进制并行加法器各位的进位输出函数表达式分别为

$$C_1 = P_1 C_0 + G_1$$

$$C_2 = P_2 C_1 + G_2 = P_2 P_1 C_0 + P_2 G_1 + G_2$$

$$C_3 = P_3 C_2 + G_3 = P_3 P_2 P_1 C_0 + P_3 P_2 G_1 + P_3 G_2 + G_3$$

$$C_4 = P_4 C_3 + G_4 = P_4 P_3 P_2 P_1 C_0 + P_4 P_3 P_2 G_1 + P_4 P_3 G_2 + P_4 G_3 + G_4$$

由于 $C_1 \sim C_4$ 是 P_i、G_i 和 C_0 的函数，P_i、G_i 又是 A_i、B_i 的函数，而 A_i、B_i 和 C_0

(一般情况下，C_0在运算前已预置)能同时提供，所以，在输入 A_i、B_i和 C_0之后，可以同时产生 $C_1 \sim C_4$。通常将根据 P_i、G_i和 C_0产生 $C_1 \sim C_4$的逻辑电路称为超前进位发生器。通过超前进位电路，可以有效地提高运算速度，采用超前进位发生器的并行加法器称为超前进位二进制并行加法器，有时又称为先行进位二进制并行加法器或并行进位二进制并行加法器。

2. 典型芯片

常用并行加法器有 4 位超前进位二进制并行加法器 74283，该器件为 16 条引线的芯片，其引脚排列图和逻辑符号分别如图 7.2(a)、(b)所示。图中，A_4、A_3、A_2、A_1和 B_4、B_3、B_2、B_1为两组 4 位二进制加数；F_4、F_3、F_2、F_1为相加产生的 4 位"和"；C_0为最低位的进位输入；FC_4为最高位的进位输出。

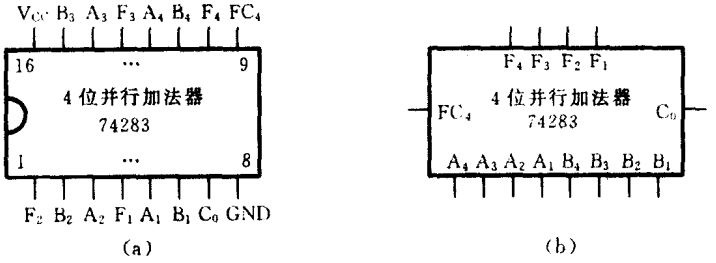


图 7.2 74283 的引脚排列图和逻辑符号

3. 应用举例

二进制并行加法器除实现二进制加法运算外，还可实现代码转换、二进制减法运算、二进制乘法运算、十进制加法运算等功能。下面举例说明。

例 7.1 用 4 位二进制并行加法器设计一个将 8421 码转换成余 3 码的代码转换电路。

解 根据余 3 码的定义可知，余 3 码是由 8421 码加 3 形成的代码。所以，用 4 位二进制并行加法器实现 8421 码到余 3 码的转换，只需从 4 位二进制并行加法器的输入端 A_4、A_3、A_2、A_1输入 8421 码，而从输入端 B_4、B_3、B_2、B_1输入二进制数 0011，进位输入端 C_0加上"0"，便可从输出端 F_4、F_3、F_2、F_1得到与输入 8421 码对应的余 3 码。其逻辑电路如图 7.3 所示。

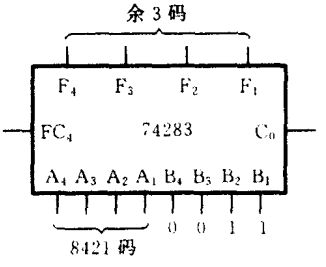


图 7.3 逻辑电路

例 7.2 用 4 位二进制并行加法器设计一个 4 位二进制并行加法/减法器。

解　设 A 和 B 分别为 4 位二进制数，其中 $A=a_4a_3a_2a_1$ 为被加数(或被减数)，$B=b_4b_3b_2b_1$ 为加数(或减数)，$S=s_4s_3s_2s_1$ 为和数(或差数)。并令 M 为功能选择变量，当 M=0 时，执行 A+B；当 M=1 时，执行 A−B。减法采用补码运算。

可用一片 4 位二进制并行加法器和 4 个异或门实现上述逻辑功能。具体可将 4 位二进制数 A 直接加到并行加法器的 A_4、A_3、A_2、A_1 输入端，4 位二进制数 B 通过异或门加到并行加法器的 B_4、B_3、B_2、B_1 输入端。并将功能选择变量 M 作为异或门的另一个输入且同时加到并行加法器的 C_0 进位输入端。使之，当 M=0 时，$C_0=0$，$b_i\oplus M=b_i\oplus 0=b_i$，加法器实现 A+B；当 M=1 时，$C_0=1$，$b_i\oplus M=b_i\oplus 1=\overline{b}_i$，加法器实现 A+$\overline{B}$+1，即 A−B。其逻辑电路如图 7.4 所示。

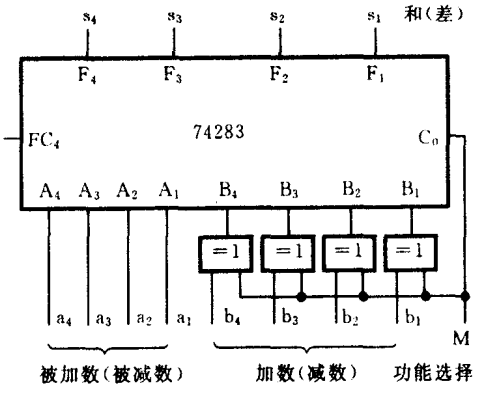


图 7.4　逻辑电路

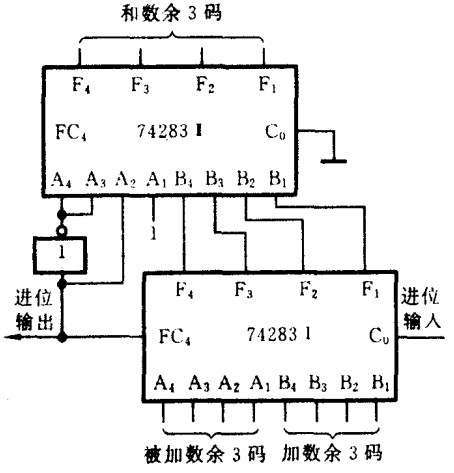


图 7.5　逻辑电路

例 7.3　用 4 位二进制并行加法器设计一个用余 3 码表示的 1 位十进制数加法器。

解　根据余 3 码的特点，两个余 3 码表示的十进制数相加时，需要对相加结果进行修正。修正法则是：若相加结果无进位产生，则“和”需要减 3；若相加结果有进位产生，则“和”需要加 3。据此，可用两片 4 位二进制并行加法器和一个反相器实现给定功能，逻辑电路如图 7.5 所示。其中，片Ⅰ用来对两个 1 位十进制数的余 3 码进行相加，片Ⅱ用来对相加结果进行修正。修正控制函数为片Ⅰ的进位输出 FC_4，当 $FC_4=0$ 时，将片Ⅰ的和输出送至片Ⅱ，并将其加上二进制数 1101(即采用补码实现运算结果减二进制数 0011)；当 $FC_4=1$ 时，将片Ⅰ的和输出送至片Ⅱ，并将其加上二进制数 0011，片Ⅱ的和输出即为两个余 3 码相加的和数。

例 7.4　用 4 位二进制并行加法器实现 4 位二进制数乘法器的逻辑功能。

解　设两个无符号 4 位二进制数 X 和 Y，$X=x_3x_2x_1x_0$，$Y=y_3y_2y_1y_0$，由此可

推出 X 和 Y 的乘积 Z 为一个 8 位二进制数，令 $Z=z_7z_6z_5z_4z_3z_2z_1z_0$。两数相乘求积的过程如下：

	被乘数			x_3	x_2	x_1	x_0
×)	乘数			y_3	y_2	y_1	y_0
				y_0x_3	y_0x_2	y_0x_1	y_0x_0
			y_1x_3	y_1x_2	y_1x_1	y_1x_0	
		y_2x_3	y_2x_2	y_2x_1	y_2x_0		
	y_3x_3	y_3x_2	y_3x_1	y_3x_0			
乘积	z_7	z_6	z_5	z_4	z_3	z_2	z_1	z_0

因为两个 1 位二进制数相乘的法则和逻辑“与”运算法则相同，所以“积”项 $x_iy_j(i,j=0,1,2,3)$ 可用两输入与门实现；而对部分积求和则可用并行加法器实现。由此可知，实现 4 位二进制数乘法运算的逻辑电路可由 16 个两输入与门和 3 个 4 位二进制并行加法器构成。其逻辑电路如图 7.6 所示。

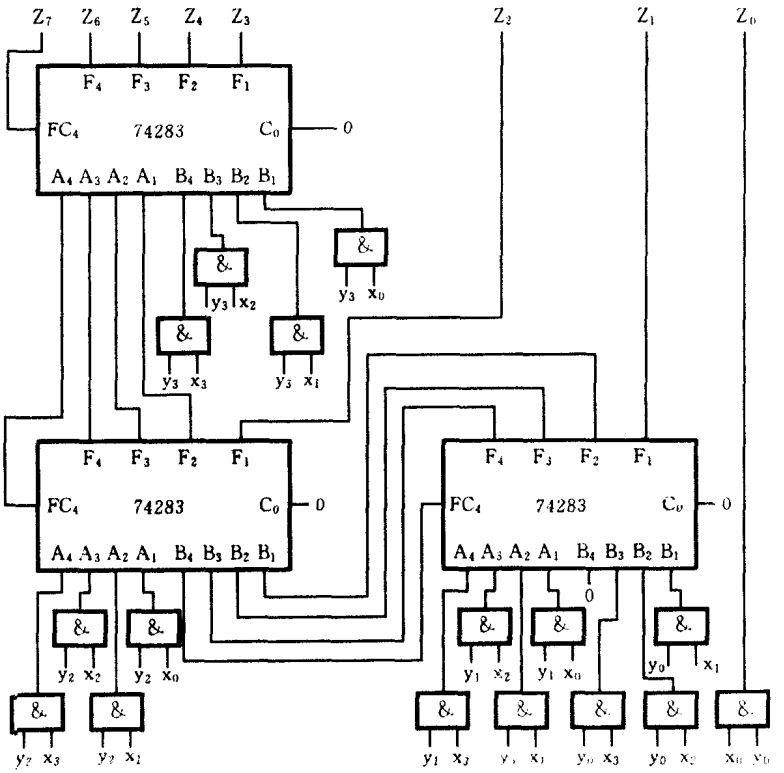


图 7.6 逻辑电路

7.1.2　译码器和编码器

译码器(Decoder)和编码器(Encoder)是数字系统中广泛使用的多输入多输出组合逻辑部件。译码器的功能是对具有特定含义的输入代码进行“翻译”,将其转换成相应的输出信号。编码器的功能恰好与译码器相反,它是对输入信号按一定规律进行编排,使每组输出代码具有特定的含义。

1. 译码器

译码器的种类很多,常见的有二进制译码器、二-十进制译码器和数字显示译码器。

(1) 二进制译码器

二进制译码器是一种能将 n 个输入变量变换成 2^n 个输出函数,且输出函数与输入变量构成的最小项具有对应关系的一种多输出组合逻辑电路。从结构上看,一个二进制译码器一般具有 n 个输入端、2^n 个输出端和一个(或多个)使能输入端。在使能输入端为有效电平时,对应每一组输入代码,仅一个输出端为有效电平,其余输出端为无效电平(与有效电平相反)。输出有效电平可以是高电平(称为高电平译码),也可以是低电平(称为低电平译码)。

常见的 MSI 二进制译码器有 2-4 线(2 输入 4 输出)译码器、3-8 线(3 输入 8 输出)译码器和 4-16 线(4 输入 16 输出)译码器等。下面以 3 输入 8 输出译码器 74138 为例进行介绍。74138 的引脚排列图和逻辑符号分别如图 7.7 (a)、(b)所示。

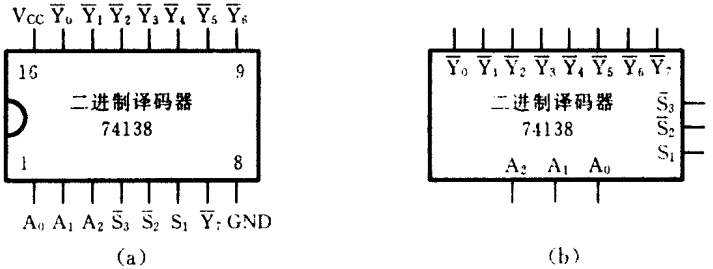


图 7.7　74138 译码器的引脚排列图和逻辑符号

图中,A_2、A_1、A_0 为输入端;$\overline{Y}_0$、$\overline{Y}_1$、$\overline{Y}_2$、$\overline{Y}_3$、$\overline{Y}_4$、$\overline{Y}_5$、$\overline{Y}_6$、$\overline{Y}_7$ 为输出端;S_1、$\overline{S}_2$、$\overline{S}_3$ 为使能端,作用是禁止或选通译码器。该译码器真值表如表 7.1 所示。

表 7.1 74138 译码器真值表

输入					输出							
S_1	S_2+S_3	A_2	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0
0	d	d	d	d	1	1	1	1	1	1	1	1
d	1	d	d	d	1	1	1	1	1	1	1	1

由真值表可知，当 $S_1=1,\overline{S}_2+\overline{S}_3=0$ 时，无论 A_2、A_1 和 A_0 取何值，输出 $\overline{Y}_0$，…，$\overline{Y}_7$ 中有且仅有一个为 0(低电平有效)，其余都是 1。显然，这里的输出 $\overline{Y}_i$ 即输入变量构成的最大项 M_i，亦即最小项之非 $\overline{m}_i$。

二进制译码器在数字系统中的应用非常广泛，它的典型用途是实现存储器的地址译码、控制器中的指令译码等。除此之外，还可用译码器实现各种组合逻辑功能。下面举例说明。

例 7.5 用 3-8 线译码器 74138 和适当的与非门实现全减器的功能。

解 实现对被减数、减数及来自相邻低位的借位进行减法运算而得到差及向相邻高位借位的逻辑电路称为全减器。设它的输入为被减数 A_i、减数 B_i 以及来自低位的借位 G_{i-1}，输出为差数 D_i 和借位 G_i。全减器的真值表如表 7.2 所示。

由表 7.2 可写出差数 D_i 和借位 G_i 的逻辑表达式

$$D_i(A_i,B_i,G_{i-1}) = m_1+m_2+m_4+m_7 = \overline{\overline{m}_1\cdot\overline{m}_2\cdot\overline{m}_4\cdot\overline{m}_7}$$

$$G_i(A_i,B_i,G_{i-1}) = m_1+m_2+m_3+m_7 = \overline{\overline{m}_1\cdot\overline{m}_2\cdot\overline{m}_3\cdot\overline{m}_7}$$

用译码器 74138 和与非门实现全减器功能时，只需将全减器的输入变量 A_i、B_i、G_{i-1} 分别与译码器的输入 A_2、A_1、A_0 相连接，译码器使能输入端 S_1、$\overline{S}_2$、$\overline{S}_3$ 接固定工作电平，便可在译码器输出端得到 3 个变量的 8 个最小项的“非”。根据全减器的输出函数表达式，将相应最小项的“非”送至与非门输入端，便可实现全减器的功能。逻辑电路如图 7.8 所示。

例 7.6 用译码器和与非门实现逻辑函数

$$F(A,B,C,D) = \sum m(2,4,6,8,10,12,14)$$

解 题目给定的逻辑函数有 4 个逻辑变量，显然可采用上例类似的方法用一个 4-16 线的译码器和与非门实现。

表 7.2　全减器真值表

输入			输出	
A_i	B_i	G_{i-1}	D_i	G_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

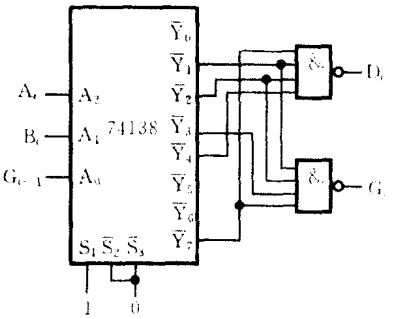


图 7.8　逻辑电路

此外，也可以充分利用译码器的使能输入端，用 3-8 线译码器实现 4 变量逻辑函数。其方法是，用译码器的一个使能端作为变量输入端，将两个 3-8 线译码器扩展成 4-16 线译码器。如图 7.9 所示，用两片 74138 实现给定函数时，可先将给定函数变换为

$$F(A,B,C,D) = \overline{\overline{m}_2\ \overline{m}_4\ \overline{m}_6\ \overline{m}_8\ \overline{m}_{10}\ \overline{m}_{12}\ \overline{m}_{14}}$$

然后将逻辑变量 B、C、D 分别接至片Ⅰ和片Ⅱ的输入端 A_2、A_1、A_0，逻辑变量 A 接至片Ⅰ的使能端 $\overline{S}_2$ 和片Ⅱ的使能端 S_1。这样，当输入变量 A＝0 时，片Ⅰ工作，片Ⅱ禁止，由片Ⅰ产生 $\overline{m}_0 \sim \overline{m}_7$；当 A＝1 时，片Ⅱ工作，片Ⅰ禁止，由片Ⅱ产生 $\overline{m}_8 \sim \overline{m}_{15}$。将译码器输出中与函数相关的项进行与非运算，即可实现给定函数 F 的功能。

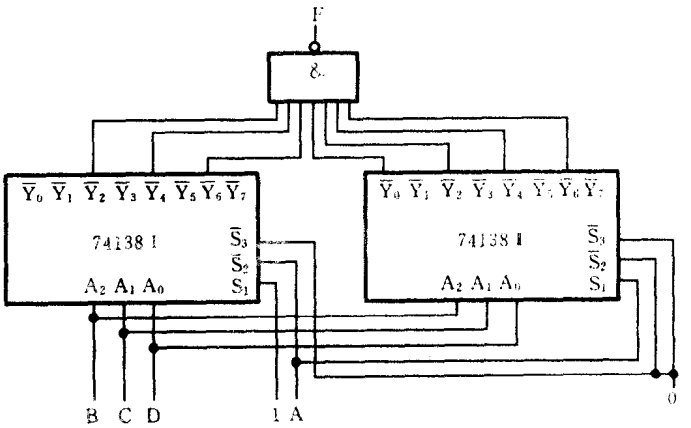


图 7.9　逻辑电路

(2) 二-十进制译码器

二-十进制译码器的功能是将 4 位 BCD 码的 10 组代码翻译成 10 个十进制数字符号对应的 10 个输出信号。图 7.10(a)、(b)所示为二-十进制译码器 74[illegible] 的引脚

排列图和逻辑符号。

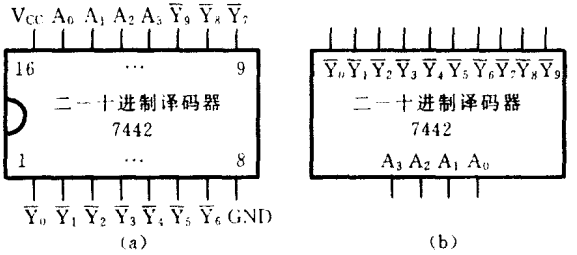


图 7.10 二-十进制译码器 7442 的引脚排列图和逻辑符号

7442 是一个将 8421 码转换成十进制数字的译码器，其输入 $A_3 \sim A_0$ 为 8421 码，输出 $\overline{Y}_0 \sim \overline{Y}_9$ 分别代表十进制数字 0～9。该译码器的真值表如表 7.3所示。

表 7.3 二-十进制译码器 7442 的真值表

输入				输出									
A_3	A_2	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$	$\overline{Y}_8$	$\overline{Y}_9$
0	0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1	1	1
0	1	0	1	1	1	1	1	1	0	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	1
0	1	1	1	1	1	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0
1	0	1	0	1	1	1	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

从真值表可知，该译码器的输出为低电平有效。其次，对于 8421 码中不允许出现的 6 个非法码(1010～1111)，译码器输出端 0～9 均无低电平信号产生，即译码器对这 6 个非法码拒绝翻译。这种译码器的优点是当输入端出现非法码时，电路不会产生错误译码。

(3) 七段显示译码器

在数字系统中，通常需要将数字量直观地显示出来，一方面供人们直接读取处理结果，另一方面用以监视数字系统工作情况。因此，数字显示电路是许多数字设备不可缺少的部分。数字显示译码器是驱动显示器件(如发光数码管、液晶数码管

等)的核心部件。它可以将输入代码转换成相应数字的控制信号,使之在数码管上显示出来。常用的数字显示译码器有七段显示译码器,图 7.11(a)、(b)所示为七段显示译码器 7448 的引脚排列图和逻辑符号。

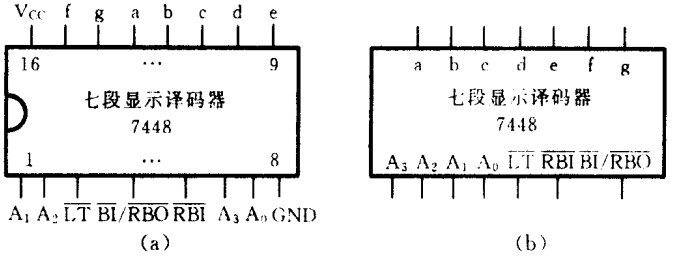


图 7.11　7448 的引脚排列图和逻辑符号

七段显示译码器 7448 的输出为高电平有效,即输出为 1 时,对应字段点亮;输出为 0 时对应字段熄灭。该译码器能够驱动七段显示器显示 16 种字形。输入 A_3、A_2、A_1和 A_0接收 4 位二进制码,输出 a、b、c、d、e、f 和 g 分别驱动七段显示器的 a、b、c、d、e、f 和 g 字段。7448 的功能表如表 7.4 所示。

为了增强器件功能,七段显示译码器 7448 设有 3 个辅助控制信号$\overline{LT}$、$\overline{RBI}$和$\overline{BI/RBO}$。各控制信号功能如下。

表 7.4　7448 的功能表

十进制数或功能	输入						$\overline{BI/RBO}$	输出							说明
	$\overline{LT}$	$\overline{RBI}$	A_3	A_2	A_1	A_0		a	b	c	d	e	f	g	
0	1	1	0	0	0	0	1	1	1	1	1	1	1	0	
1	1	d	0	0	0	1	1	0	1	1	0	0	0	0	
2	1	d	0	0	1	0	1	1	1	0	1	1	0	1	
3	1	d	0	0	1	1	1	1	1	1	1	0	0	1	
4	1	d	0	1	0	0	1	0	1	1	0	0	1	1	
5	1	d	0	1	0	1	1	1	0	1	1	0	1	1	
6	1	d	0	1	1	0	1	0	0	1	1	1	1	1	译
7	1	d	0	1	1	1	1	1	1	1	0	0	0	0	码
8	1	d	1	0	0	0	1	1	1	1	1	1	1	1	显
9	1	d	1	0	0	1	1	1	1	1	0	0	1	1	示
10	1	d	1	0	1	0	1	0	0	0	1	1	0	1	
11	1	d	1	0	1	1	1	0	0	1	1	0	0	1	
12	1	d	1	1	0	0	1	0	1	0	0	0	1	1	
13	1	d	1	1	0	1	1	1	0	0	1	0	1	1	
14	1	d	1	1	1	0	0	0	0	0	1	1	1	1	
15	1	d	1	1	1	1	1	0	0	0	0	0	0	0	
消隐	d	d	d	d	d	d	0	0	0	0	0	0	0	0	熄灭
脉冲消隐	1	0	0	0	0	0	0	0	0	0	0	0	0	0	灭零
灯测试	0	d	d	d	d	d	1	1	1	1	1	1	1	1	测试

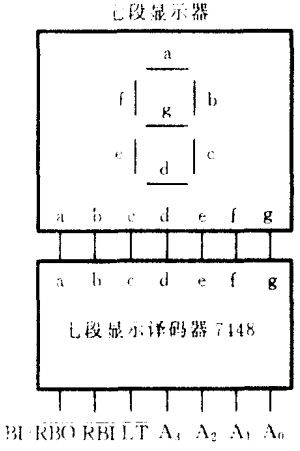
① $\overline{BI}/\overline{RBO}$为熄灭输入端/灭零输出端(低电平有效),该控制端具有双向控制功能,有时作为输入信号,有时作为输出信号。当$\overline{BI}/\overline{RBO}$作为输入使用,且$\overline{BI}=0$时,不管其他输入状态如何,所有各段输出 a～g 均为 0,显示管的七段均熄灭,目的是为了降低系统功耗,在不需要观察时全部熄灭显示器。

当$\overline{BI}/\overline{RBO}$作为输出使用时,受控于$\overline{LT}$和$\overline{RBI}$。当$\overline{LT}=1$,$\overline{RBI}=0$且输入数码$A_3A_2A_1A_0=0000$时,$\overline{RBO}=0$;当$\overline{LT}=0$,或者$\overline{LT}=1$且$\overline{RBI}=1$时,$\overline{RBO}=1$。该端主要用于显示多位数字时,多个译码器之间的连接。

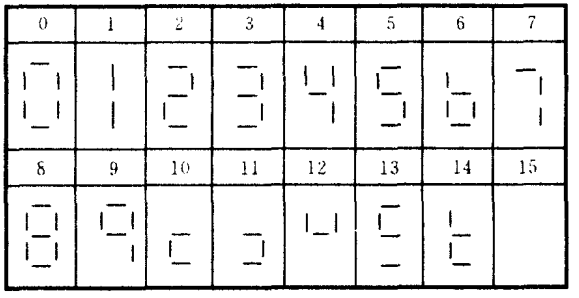
② $\overline{LT}$为灯测试输入端(低电平有效),当$\overline{LT}=0$时,$\overline{BI}/\overline{RBO}$是输出端。当$\overline{LT}=0$且$\overline{BI}=1$时,不管其他输入状态如何,a～g 均输出有效的逻辑 1,显示管的七段均应点亮。该输入端通常用来检查 7448 本身及七段显示器的显示管是否都能正常工作。

③ $\overline{RBI}$为灭零输入端,用来熄灭无意义 0 的显示。当$\overline{LT}=1$,$\overline{RBI}=0$且输入数码$A_3A_2A_1A_0=0000$时,$\overline{BI}/\overline{RBO}$为输出端,且$\overline{RBO}=0$。此时输出 a～g 均为 0,显示管的七段均熄灭,不显示数字 0,故称为"灭零"。利用$\overline{RBI}=0$和$\overline{LT}=1$,可以实现某一位 0 的"消隐",通常用来把有效数字前面的 0 灭掉。输入数码为其他数值时,显示管均能正常显示。

七段译码显示原理如图 7.12(a)所示,图 7.12(b)给出了七段显示笔画与 16 种字形的对应关系。



(a)



(b)

图 7.12 七段显示译码工作原理及笔画与字形关系

2. 编码器

编码器按照被编信号的不同特点和要求,有各种不同的类型,最常见的有

二-十进制编码器(又称为 BCD 码编码器)和优先编码器。

(1) 二-十进制编码器

二-十进制编码器执行的逻辑功能是将十进制的 10 个数字 0～9 分别编成 4 位 BCD 码。这种编码器由 10 个输入端代表 10 个不同数字，4 个输出端代表 BCD 代码。最常见的有 8421 码编码器，图 7.13 所示的是按键式 8421 码编码器的逻辑电路。

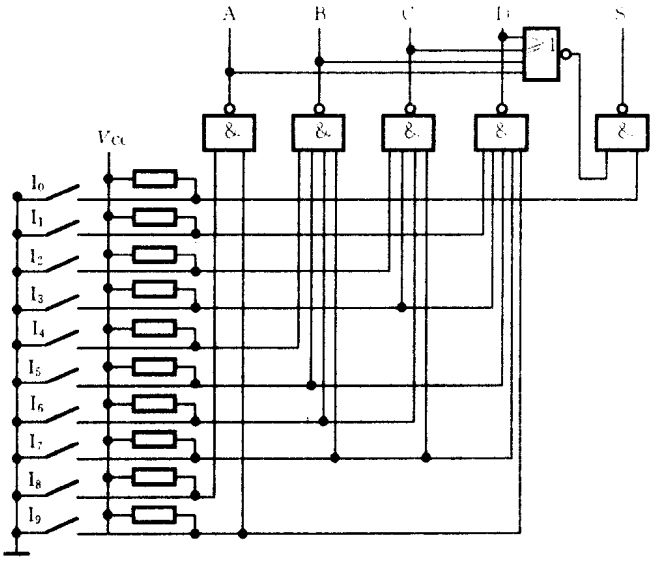


图 7.13　按键式 8421 码编码器

图中，I_0～I_9代表 10 个按键，ABCD 为代码输出端，当按下某一输入键时，在 ABCD 输出相应的 8421 码。图中，S 为使用输出标志，当按下 I_0～I_9中任一个键时，S 为 1，表示输出有效，否则 S 为 0，表示输出无效。设置标志 S 是为了区别按下 I_0键与不按任何键时均有 ABCD＝0000 的两种不同情况。该编码器的真值表如表 7.5 所示。

常用的十进制-BCD 码编码器有中规模集成电路芯片 74147 等，有关详细介绍可查阅集成电路手册。

(2) 优先编码器

优先编码器是数字系统中实现优先权管理的一个重要逻辑部件。它与上述二-十进制编码器的最大区别是，二-十进制编码器的输入信号是互斥的，即任何时候只允许一个输入端为有效信号；而优先编码器的各个输入不是互斥的，它允许多个输入端同时为有效信号。优先编码器的每个输入具有不同的优先级别，当多个输入信号有效时，能识别输入信号的优先级别，并对其中优先级别最高的一个进行

表 7.5　8421 码编码器真值表

输入										输出				
I_9	I_8	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	A	B	C	D	S
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	1	1	0	0	0	0	0	1
1	1	1	1	1	1	1	1	0	1	0	0	0	1	1
1	1	1	1	1	1	1	0	1	1	0	0	1	0	1
1	1	1	1	1	1	0	1	1	1	0	0	1	1	1
1	1	1	1	1	0	1	1	1	1	0	1	0	0	1
1	1	1	1	0	1	1	1	1	1	0	1	0	1	1
1	1	1	0	1	1	1	1	1	1	0	1	1	0	1
1	1	0	1	1	1	1	1	1	1	0	1	1	1	1
1	0	1	1	1	1	1	1	1	1	0	0	0	0	1
0	1	1	1	1	1	1	1	1	1	1	0	0	1	1

编码，产生相应的输出代码。图 7.14(a)、(b)所示的分别为常见 MSI 优先编码器 74148 的引脚排列图和逻辑符号。

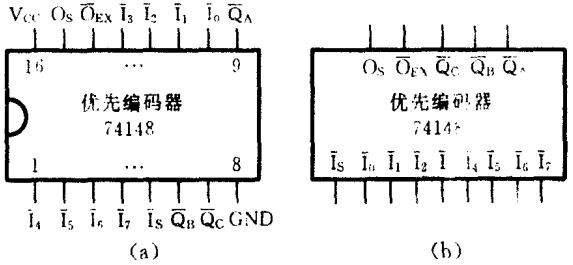


图 7.14　74148 优先编码器引脚排列和逻辑符号

图中，$\overline{I}_0 \sim \overline{I}_7$ 为 8 个输入端，$\overline{Q}_A$、$\overline{Q}_B$ 和 $\overline{Q}_C$ 为 3 位二进制码输出，因此，称它为 8-3 线优先编码器，其真值表如表 7.6 所示。

表 7.6　74148 真值表

输入									输出				
I_S	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	Q_C	Q_B	Q_A	O_{EX}	O_S
1	d	d	d	d	d	d	d	d	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	d	d	d	d	d	d	d	0	0	0	0	0	1
0	d	d	d	d	d	d	0	1	0	0	1	0	1
0	d	d	d	d	d	0	1	1	0	1	0	0	1
0	d	d	d	d	0	1	1	1	0	1	1	0	1
0	d	d	d	0	1	1	1	1	1	0	0	0	1
0	d	d	0	1	1	1	1	1	1	0	1	0	1
0	d	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1

由真值表可知，输入 $\overline{I}_0 \sim \overline{I}_7$ 和输出 $\overline{Q}_A$、$\overline{Q}_B$、$\overline{Q}_C$ 的有效工作电平均为低电平(即

逻辑 0)。在 $\overline{I}_0 \sim \overline{I}_7$ 输入端中，下角标号码越大的优先级越高。例如，$\overline{I}_6$、$\overline{I}_2$、$\overline{I}_3$、$\overline{I}_5$ 和 $\overline{I}_7$ 为 1，$\overline{I}_1$、$\overline{I}_4$ 和 $\overline{I}_6$ 为 0 时，输出按优先级较高的 $\overline{I}_6$ 编码，即 $\overline{Q}_C\overline{Q}_B\overline{Q}_A=001$，而不是按优先级较低的 $\overline{I}_1$ 和 $\overline{I}_4$ 编码。此后，若 $\overline{I}_6$ 变为 1，则按 $\overline{I}_4$ 编码，$\overline{Q}_C\overline{Q}_B\overline{Q}_A=011$。若 $\overline{I}_4$ 也变为 1，输出才按 $\overline{I}_1$ 编码，$\overline{Q}_C\overline{Q}_B\overline{Q}_A=110$。输入 $\overline{I}_S$ 和输出 O_S、$\overline{O}_{EX}$ 在容量扩展时使用。$\overline{I}_S$ 为选通输入端(或称允许输入端)，当 $\overline{I}_S=0$ 时，编码器工作，反之不进行编码工作；O_S 为选通输出端(或称允许输出端)，当选通输入有效(即 $\overline{I}_S=0$)而无信号输入时，O_S 为 0。$\overline{O}_{EX}$ 为工作状态标志，当选通输入无效(即 $\overline{I}_S=1$)，或者虽选通输入有效($\overline{I}_S=0$)但无信号输入(即 $\overline{I}_0 \sim \overline{I}_7$ 均为 1)时，$\overline{O}_{EX}$ 为 1。换而言之，只有当选通输入有效且有信号输入(即 $\overline{I}_0 \sim \overline{I}_7$ 中至少有一个为 0)时，$\overline{O}_{EX}$ 才为 0。$\overline{O}_{EX}$ 通常用来扩展编码器功能。

例 7.7　用优先编码器 74148 设计一个能裁决 16 级不同中断请求的中断优先编码器。

解　设 $\overline{I}_{Z15} \sim \overline{I}_{Z0}$ 为 16 个不同的中断请求信号，下标码越大的优先级别越高，$\overline{Q}_{ZD}$、$\overline{Q}_{ZC}$、$\overline{Q}_{ZB}$ 和 $\overline{Q}_{ZA}$ 为中断请求信号的编码输出，输入和输出均为低电平有效。$\overline{I}_{ZS}$ 为选通输入端，O_{ZS} 为选通输出端，$\overline{Q}_{ZEX}$ 为工作状态标志。根据 74148 的功能，可用两片 74148 实现给定功能，逻辑电路如图 7.15 所示。

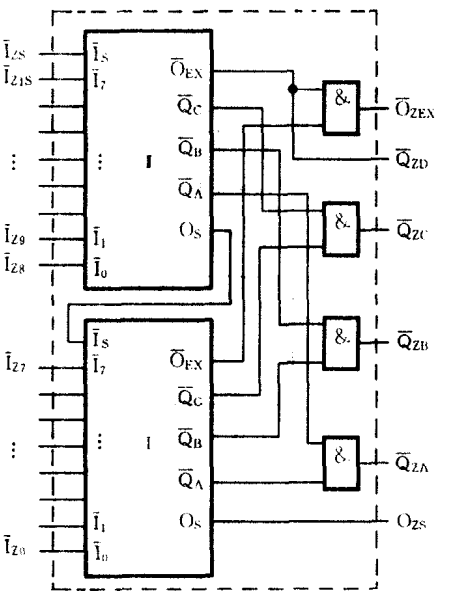


图 7.15　16 级中断优先编码器

图中，中断优先编码器的允许输入端 $\overline{I}_{ZS}$ 接片 II 的 $\overline{I}_S$ 端。当 $\overline{I}_{ZS}$ 为 0 时，片 II 处于工作状态，若 $\overline{I}_{Z15} \sim \overline{I}_{Z8}$ 中有中断请求信号，则其输出 O_S 为 1，$\overline{O}_{EX}$ 为 0，O_S 接到片 I 的 $\overline{I}_S$ 端，使片 I 不工作，其输出均为 1，此时中断优先编码器对高 8 级中断请求信号中优先级最高的中断请求信号进行编码；若 $\overline{I}_{Z15} \sim \overline{I}_{Z8}$ 中无中断请求信号，则片 II 的 $\overline{O}_{EX}$(即 $\overline{Q}_{ZD}$)及 $\overline{Q}_C$、$\overline{Q}_B$、$\overline{Q}_A$ 均为 1，O_S 为 0，使片 I 的 $\overline{I}_S$ 为 0，片 I 处于工作状态，实现对 $I_{Z7} \sim I_{Z0}$ 中优先级最高中断请求信号进行编码。

7.1.3　多路选择器和多路分配器

多路选择器和多路分配器是数字系统中常用的中规模集成电路，其基本功能

是完成对多路数据的选择与分配、在公共传输线上实现多路数据的分时传送。此外，还可完成数据的并串转换、序列信号产生等多种逻辑功能以及实现各种逻辑函数功能。

1. 多路选择器

多路选择器(Multiplexer)又称数据选择器或多路开关，常用 MUX 表示。它是一种多路输入、单路输出的组合逻辑电路，其逻辑功能是从多路输入中选中一路送至输出端，输出对输入的选择受选择控制变量控制。通常，对于一个具有 2^n 路输入和一路输出的 MUX 有 n 个选择控制变量，对应控制变量的每种取值组合选中相应的一路输入送至输出。

(1) 典型芯片

常见的 MSI 多路选择器有双 4 路 MUX 74153、8 路 MUX 74152(无使能控制端)、74151 和 16 路 MUX 74150 等。下面以双 4 路 MUX 74153 为例对其外部特性进行介绍。

双 4 路 MUX 74153 的引脚排列图和逻辑符号如图 7.16 中的(a)和(b)所示，该芯片中有两个 4 路 MUX。其中，G 为使能控制端，低电平有效；$D_0 \sim D_3$ 为数据输入端；A_1、A_0 为选择控制端，两个 MUX 共用；Y 为输出端。

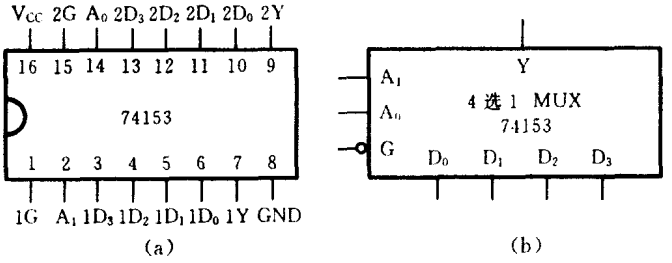


图 7.16 74153 引脚排列图和逻辑符号

4 路 MUX 74153 的功能表如表 7.7 所示。

表 7.7 4 路 MUX 74153 功能表

使能输入	选择输入		数据输入				输出
G	A_1	A_0	D_0	D_1	D_2	D_3	Y
1	d	d	d	d	d	d	0
0	0	0	D_0	d	d	d	D_0
0	0	1	d	D_1	d	d	D_1
0	1	0	d	d	D_2	d	D_2
0	1	1	d	d	d	D_3	D_3

由功能表可知，在工作状态下(G=0)，当 $A_1A_0=00$ 时，$Y=D_0$；当 $A_1A_0=01$ 时，$Y=D_1$；当 $A_1A_0=10$ 时，$Y=D_2$；当 $A_1A_0=11$ 时，$Y=D_3$。即在 A_1A_0 的控制下，依次选中 $D_0\sim D_3$ 端的数据送至输出端。4 路 MUX 的输出函数表达式为

$$Y=\overline{A}_1\overline{A}_0D_0+\overline{A}_1A_0D_1+A_1\overline{A}_0D_2+A_1A_0D_3=\sum_{i=0}^{3}m_iD_i$$

式中，m_i 为选择变量 A_1、A_0 组成的最小项，D_i 为 i 端的输入数据，取值等于 0 或 1。

类似地，可以写出 2^n 路 MUX 的输出表达式为

$$Y=\sum_{i=0}^{2^n-1}m_iD_i$$

式中，m_i 为选择控制变量 A_{n-1}，A_{n-2}，…，A_1，A_0 组成的最小项；D_i 为 2^n 路输入中的第 i 路数据输入，取值 0 或 1。

(2) 应用举例

MUX 除完成对多路数据进行选择的基本功能外，在逻辑设计中常用来实现各种逻辑函数功能。假定用 n 个选择控制变量的 MUX 实现 m 个变量的函数，具体方法可以分为 3 种情况讨论。

① $m=n$(用具有 n 个选择控制变量的 MUX 实现 n 个变量的函数)

实现方法：将函数的 n 个变量依次连接到 MUX 的 n 个选择变量端，并将函数表示成最小项之和的形式。若函数表达式中包含最小项 m_i，则相应 MUX 的 D_i 接 1，否则 D_i 接 0 。

② $m=n+1$(用具有 n 个选择控制变量的 MUX 实现 $n+1$ 个变量的函数)

实现方法：从函数的 $n+1$ 个变量中任选 n 个作为 MUX 的选择控制变量，并根据所选定的选择控制变量将函数变换成 $W=\sum_{i=0}^{2^n-1}m_iD_i$ 的形式，以确定各数据输入 D_i。假定剩余变量为 X，则 D_i 的取值只可能是 0、1、X 或 $\overline{X}$ 四者之一。

③ $m\geqslant n+2$(用具有 n 个选择控制变量的 MUX，实现 $n+1$ 个以上变量的函数)

实现方法与②类似，但此时数据输入为去除选择变量之外剩余变量的函数，因此，一般需要增加适当的逻辑门辅助实现，且所需逻辑门的多少通常与选择控制变量的确定相关。

下面举例说明。

例 7.8　用 MUX 实现以下逻辑函数的功能。

$$F(A,B,C)=\sum m(2,3,5,6)$$

解　给定函数为一个 3 变量函数，可采用 8 路 MUX 或者 4 路 MUX 实现其功能。

方案 1　采用 8 路 MUX 74152 实现

8 路 MUX 的输出表达式为

$$Y = \bar{A}_2\bar{A}_1\bar{A}_0D_0 + \bar{A}_2\bar{A}_1A_0D_1 + \bar{A}_2A_1\bar{A}_0D_2 + \bar{A}_2A_1A_0D_3 + A_2\bar{A}_1\bar{A}_0D_4 + A_2\bar{A}_1A_0D_5 + A_2A_1\bar{A}_0D_6 + A_2A_1A_0D_7$$

给定逻辑函数 F 的表达式为

$$F(A,B,C) = \sum m(2,3,5,6) = \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C + AB\bar{C}$$

比较上述两个表达式可知：要使 Y=F，只需令 8 路 MUX 的 $A_2=A$，$A_1=B$，$A_0=C$ 且 $D_2=D_3=D_5=D_6=1$，而 $D_0=D_1=D_4=D_7=0$ 即可。据此可作出用 8 路 MUX 实现给定函数的逻辑电路，如图 7.17(a)所示。

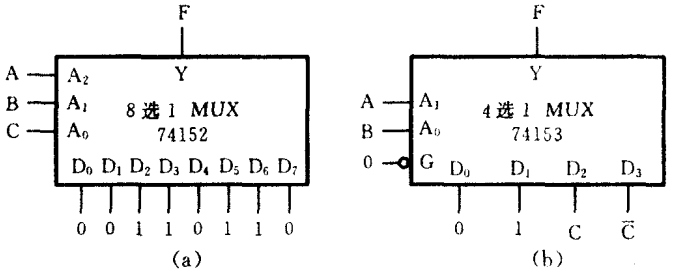


图 7.17　例 7.8 的两种方案

方案 2　采用 4 路 MUX 74153 实现

假定采用 4 路 MUX 74153 实现，由于 4 路 MUX 具有 2 个选择控制变量，所以用来实现 3 变量函数功能时，应该首先从函数的 3 个变量中任选 2 个作为选择控制变量，然后再确定选择器的数据输入。假定选 A、B 与选择控制端 A_1、A_0 相连，则可将函数 F 的表达式表示成如下形式：

$$\begin{aligned} F(A,B,C) &= \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C + AB\bar{C} \\ &= \bar{A}\bar{B}\cdot 0 + \bar{A}B\cdot(\bar{C}+C) + A\bar{B}\cdot C + AB\cdot\bar{C} \\ &= A\bar{B}\cdot 0 + \bar{A}B\cdot 1 + A\bar{B}\cdot C + AB\cdot\bar{C} \end{aligned}$$

显然，要使 4 路 MUX 的输出 Y 与函数 F 相等，只需 $D_0=0$、$D_1=1$、$D_2=C$、$D_3=\bar{C}$。据此，可作出用 4 路 MUX 实现给定函数功能的逻辑电路，如图 7.17(b)所示。类似地，也可以选择 A、C 或 B、C 作为选择控制变量，选择控制变量不同，数据输入也不同。例如，本例若选 A、C 与 A_1、A_0 相连，则应有 $D_0=B$、$D_1=B$、$D_2=B$、$D_3=\bar{B}$。

上述两种方案表明：用具有 n 个选择控制变量的 MUX 实现 n 个变量的函数或 $n+1$ 个变量的函数时，无须任何辅助电路，可由 MUX 直接实现。

当函数的变量数比 MUX 的选择控制变量数多两个以上时，一般需要加适当

的逻辑门辅助实现。

例 7.9　用 4 路 MUX 实现 4 变量逻辑函数的功能，函数表达式为

$$F(A,B,C,D) = \sum m(0,2,3,7,8,9,10,13)$$

解　用 4 路 MUX 实现该函数时，首先应从函数的 4 个变量中选出 2 个作为 MUX 的选择控制变量。原则上讲，这种选择是任意的，但选择合适时可使设计简化。

方案 1　选用变量 A 和 B 作为选择控制变量

假定选用变量 A 和 B 作为选择控制变量与选择控制端 A_1、A_0 相连，首先对函数做如下变换：

$$\begin{aligned}F(A,B,C,D) &= \sum m(0,2,3,7,8,9,10,13)\\ &= \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}CD + \bar{A}BCD + A\bar{B}\bar{C}\bar{D}\\ &\quad + A\bar{B}\bar{C}D + A\bar{B}C\bar{D} + AB\bar{C}D\\ &= \bar{A}\bar{B}(\bar{C}\bar{D} + C\bar{D} + CD) + \bar{A}B \cdot CD\\ &\quad + A\bar{B}(\bar{C}\bar{D} + \bar{C}D + C\bar{D}) + AB \cdot \bar{C}D\\ &= \bar{A}\bar{B}(C + \bar{D}) + \bar{A}B \cdot CD + A\bar{B}(\bar{C} + \bar{D}) + AB \cdot \bar{C}D\end{aligned}$$

根据变换后的逻辑表达式，即可确定各数据输入 D_i 分别为

$$D_0 = C + \bar{D} \qquad D_1 = CD \qquad D_2 = \bar{C} + \bar{D} = +\overline{C\bar{D}} \qquad D_3 = \bar{C}D$$

据此，可得到实现给定函数的逻辑电路，如图 7.18(a)所示。除 4 路 MUX 外，附加了 4 个逻辑门。

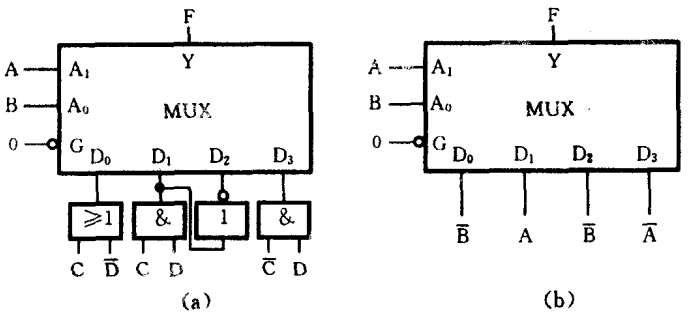


图 7.18　例 7.9 的两种方案

方案 2　选用变量 C 和 D 作为选择控制变量

如果选用变量 C 和 D 作为选择控制变量与选择控制端 A_1、A_0 相连，则可对函数做如下变换：

$$\begin{aligned}F(A,B,C,D) &= \sum m(0,2,3,7,8,9,10,13)\\ &= \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}CD + \bar{A}BCD + A\bar{B}\bar{C}\bar{D}\end{aligned}$$

$$
\begin{aligned}
&+A\bar{B}\bar{C}D+A\bar{B}C\bar{D}+AB\bar{C}D\\
&=\bar{C}\bar{D}(\bar{A}\bar{B}+A\bar{B})+\bar{C}D(A\bar{B}+AB)\\
&+C\bar{D}(\bar{A}\bar{B}+A\bar{B})+CD(\bar{A}\bar{B}+\bar{A}B)\\
&=\bar{C}\bar{D}\cdot\bar{B}+\bar{C}D\cdot A+C\bar{D}\cdot\bar{B}+CD\cdot\bar{A}
\end{aligned}
$$

根据变换后的逻辑表达式，可确定各数据输入 D_i 分别为

$$D_0=\bar{B}\quad D_1=A\quad D_2=\bar{B}\quad D_3=\bar{A}$$

相应逻辑电路如图 7.18(b)所示，在有反变量提供的前提下，无须附加逻辑门。显然，实现给定函数用 C、D 作为选择控制变量更简单。

由上述可见，用 n 个选择控制变量的 MUX 实现 m 个变量($m-n\geqslant 2$)的函数时，MUX 的数据输入 D_i 一般是 2 个或 2 个以上变量的函数。函数 D_i 的复杂程度与选择控制变量的确定相关，只有通过对各种方案的比较，才能从中得到简单经济的方案。

例 7.10 用一片双 4 路 MUX 74153 实现 4 变量多输出函数。函数表达式为

$$\begin{cases}F_1(A,B,C,D)=\sum m(0,1,5,7,10,13,15)\\ F_2(A,B,C,D)=\sum m(8,10,12,13,15)\end{cases}$$

解 假定选取函数变量 A、B 作为 MUX 的选择控制变量 A_1、A_0，则可对函数 F_1 和 F_2 做如下变换。

$$
\begin{aligned}
F_1(A,B,C,D)&=\sum m(0,1,5,7,10,13,15)\\
&=\bar{A}\bar{B}\bar{C}\bar{D}+\bar{A}\bar{B}\bar{C}D+\bar{A}B\bar{C}D+\bar{A}BCD+A\bar{B}C\bar{D}+AB\bar{C}D+ABCD\\
&=\bar{A}\bar{B}(\bar{C}\bar{D}+\bar{C}D)+\bar{A}B(\bar{C}D+CD)+A\bar{B}\cdot C\bar{D}+AB(\bar{C}D+CD)\\
&=\bar{A}\bar{B}\cdot\bar{C}+\bar{A}B\cdot D+A\bar{B}\cdot C\bar{D}+AB\cdot D
\end{aligned}
$$

$$
\begin{aligned}
F_2(A,B,C,D)&=\sum m(8,10,12,13,15)\\
&=A\bar{B}\bar{C}\bar{D}+A\bar{B}C\bar{D}+AB\bar{C}\bar{D}+AB\bar{C}D+ABCD\\
&=\bar{A}\bar{B}\cdot 0+\bar{A}B\cdot 0+A\bar{B}(\bar{C}\bar{D}+C\bar{D})+AB(\bar{C}\bar{D}+\bar{C}D+CD)\\
&=\bar{A}\bar{B}\cdot 0+\bar{A}B\cdot 0+A\bar{B}\cdot\bar{D}+AB\cdot(\bar{C}+D)
\end{aligned}
$$

若令 74153 的 $1Y=F_1$，$2Y=F_2$，则根据上述逻辑表达式可确定 74153 各数据输入端的值依次为

$$1D_0=\bar{C}\quad 1D_1=D\quad 1D_2=C\bar{D}\quad 1D_3=D$$

$$2D_0=0\quad 2D_1=0\quad 2D_2=\bar{D}\quad 2D_3=\bar{C}+D$$

实现函数 F_1 和 F_2 的逻辑电路如图 7.19 所示。

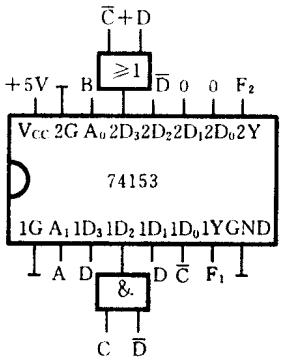


图 7.19　逻辑电路

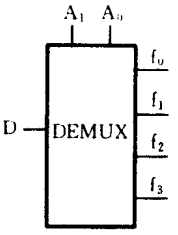


图 7.20　4 路 DEMUX 的逻辑符号

表 7.8　4 路 DEMUX 功能表

A_1	A_0	f_0	f_1	f_2	f_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

2. 多路分配器

多路分配器(Demultiplexer)又称数据分配器，常用 DEMUX 表示。其结构与多路选择器正好相反，它是一种单输入、多输出的逻辑部件，具体从哪一路输出由选择控制变量决定。图 7.20 所示为 4 路 DEMUX 的逻辑符号。图中，D 为数据输入端，A_1、A_0为选择控制输入端，f_0～f_3为数据输出端。其功能表如表 7.8 所示。由功能表可知，4 路 DEMUX 的输出表达式为

$$f_0 = \overline{A}_1\overline{A}_0 \cdot D = m_0 \cdot D$$

$$f_1 = \overline{A}_1 A_0 \cdot D = m_1 \cdot D$$

$$f_2 = A_1\overline{A}_0 \cdot D = m_2 \cdot D$$

$$f_3 = A_1 A_0 \cdot D = m_3 \cdot D$$

式中，m_i(i=0～3)是选择控制变量的 4 个最小项。

由此可见，DEMUX 与译码器十分相似，若将图 7.20 中所示的 D 端接固定的 1，则该电路实现 2-4 线译码器的功能，故 DEMUX 和译码器一般是可以互相替代的，通常归于一类。

DEMUX 常与 MUX 联用，以实现多通道数据分时传送。通常在发送端由 MUX 将各路数据分时送至公共传输线(总线)，接收端再由 DEMUX 将公共线上的数据分配到相应的输出端。图 7.21 所示的是利用一根数据传输线分时传送 8 路数据的示意图，在公共选择控制变量 ABC 的控制下，实现 D_i—f_i的传送(i=0～7)。

以上对几种最常用的 MSI 组合逻辑电路进行了介绍，在逻辑设计时可以灵活使用这些电路实现各种逻辑功能。

例 7.11　用 8 路 MUX 和 3-8 线译码器构造一个 3 位二进制数等值比较器。

解 设比较的两个 3 位二进制数分别为 ABC 和 XYZ，将译码器和 MUX 按图 7.22 所示进行连接，即可实现 ABC 和 XYZ 的等值比较。

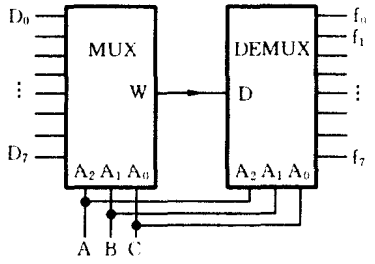


图 7.21 8 路数据传输示意图

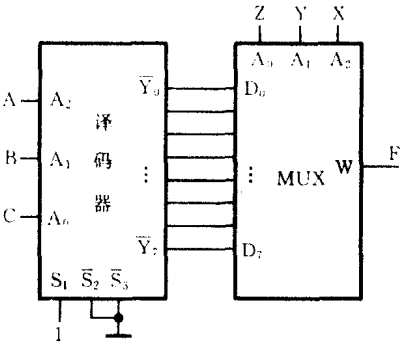


图 7.22 比较器的逻辑电路

从图 7.22 可知，当译码器的使能端$\overline{S}_3$、$\overline{S}_2$接地，S_1接“1”时，电路处于工作状态。若 ABC=XYZ，则 MUX 的输出 F=0，否则 F=1。例如，当 ABC=010 时，译码器输出$\overline{Y}_2=0$，其余均为 1。若 MUX 选择控制变量 XYZ=ABC=010，则选通 D_2送至输出端 F，由于$D_2=\overline{Y}_2=0$，故 F=0；若 XYZ≠010，则 MUX 会选择 D_2之外的其他数据输入送至输出端 F，由于与其余数据输入端相连的译码器输出均为 1，故 F 为 1。用类似方法，采用合适的译码器和 MUX 可构成多位二进制数比较器。

7.2 常用中规模时序逻辑电路

数字系统中最常用的中规模同步时序器件是计数器和寄存器，本节将分别举例介绍其外部特性及在逻辑设计中的应用。要求在掌握外部特性的基础上，能根据需要对器件进行合理选择、灵活使用。

7.2.1 集成计数器

广义地说，计数器是一种能在输入信号作用下依次通过预定状态的时序逻辑电路。计数器中的“数”是用触发器的状态组合来表示的，在计数脉冲作用下使一组触发器的状态依次转换成不同的状态组合来表示数的变化，即可达到计数的目的。计数器在运行时，所经历的状态是周期性的，总是在有限个状态中循环，通常将一次循环所包含的状态总数称为计数器的“模”。

集成计数器的种类很多，通常有不同的分类方法。按其工作方式可分为同

步计数器和异步计数器；按其进位制可分为二进制计数器、十进制计数器和任意进制计数器；按其功能又可分为加法计数器、减法计数器和加/减可逆计数器等。为了满足实际应用的需要，常用的集成计数器一般具有计数、保存、清除、预置等功能。

1. 集成同步计数器

常用的集成同步计数器有 4 位二进制同步加法计数器 74161、单时钟 4 位二进制同步可逆计数器 74191、单时钟十进制同步可逆计数器 74190、双时钟 4 位二进制同步可逆计数器 74193 等。下面以 74193 为例对其外部特性进行介绍。

4 位二进制同步可逆计数器 74193 的引脚排列图及逻辑符号分别如图 7.23 (a)、(b)所示。

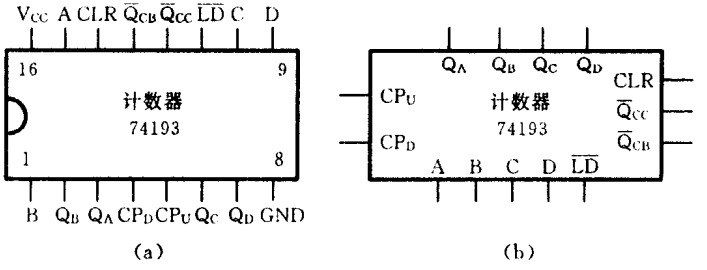


图 7.23　74193 的引脚排列图和逻辑符号

表 7.9 给出了 74193 输入/输出信号的说明。该芯片的功能表如表 7.10 所示。

表 7.9　74193 输入/输出信号的说明

引线名称		说　明
输入端	CLR	清除
	$\overline{LD}$	预置控制
	D,C,B,A	预置初置
	CP_U ↑	累加计数脉冲
	CP_D ↑	累减计数脉冲
输出端	$Q_DQ_CQ_BQ_A$	记数值
	$\overline{Q}_{CC}$	进位输出负脉冲
	$\overline{Q}_{CB}$	借位输出负脉冲

由表 7.10 可知，当 CLR 为高电平时，计数器被清除为“0”；当$\overline{LD}$为低电平时，计数器被预置为 A、B、C、D 端输入的值；当计数脉冲由 CP_U端输入时，计数器进行累加计数；当计数脉冲由 CP_D端输入时，计数器进行累减计数。

表 7.10 74193 的功能表

输入								输出			
CLR	$\overline{LD}$	D	C	B	A	CP_U	CP_D	Q_D	Q_C	Q_B	Q_A
1	d	d	d	d	d	d	d	0	0	0	0
0	0	x_1	x_2	x_3	x_4	d	d	x_1	x_2	x_3	x_4
0	1	d	d	d	d	↑	1	累	加	计	数
0	1	d	d	d	d	1	↑	累	减	计	数

4 位二进制计数器是模为 16 的计数器。在实际应用中，可根据需要用 4 位二进制计数器构成模为任意 R(R 小于 16 或大于 16)的计数器。下面举例介绍用 4 位二进制同步可逆计数器 74193 构成模为任意 R 计数器的方法。

2. 构成模小于 16 的计数器

通过利用计数器的清除、预置等功能，可以很方便地实现模小于 16 的计数器。

例 7.12 用 4 位二进制同步可逆计数器 74193 构成模 10 加法计数器。

解 假设计数器的初始状态为 $Q_3Q_2Q_1Q_0=0000$，其状态变化序列如下：

$$0000\rightarrow0001\rightarrow0010\rightarrow0011\rightarrow0100$$
$$\uparrow \qquad\qquad\qquad\qquad \downarrow$$
$$1001\leftarrow1000\leftarrow0111\leftarrow0110\leftarrow0101$$

根据 74193 的功能表，可用图 7.24 所示逻辑电路实现模 10 加法器的功能。其中，$\overline{LD}$和 CP_D接逻辑 1，CP_U接计数脉冲 CP，74193 工作在累加计数状态。当计数器输出由 1001 变为 1010 时，其中与门输出为 1，该信号接至清除端 CLR，使计数器状态立即变为 0000，当下一个计数脉冲到达时，再由 0000→0001，继续进行加 1 计数。

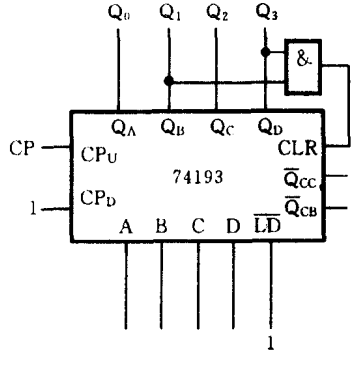


图 7.24 模 10 加法计数器逻辑电路

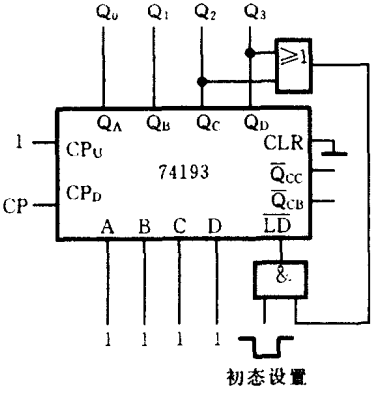


图 7.25 模 12 减法计数器逻辑电路

例 7.13　用 4 位二进制同步可逆计数器 74193 构成模 12 减法计数器。

解　设计数器的初始状态为 $Q_3Q_2Q_1Q_0=1111$，其状态变化序列如下：

$$1111\rightarrow1110\rightarrow1101\rightarrow1100\rightarrow1011\rightarrow1010$$
$$\uparrow\qquad\qquad\qquad\qquad\qquad\qquad\downarrow$$
$$0100\leftarrow0101\leftarrow0110\leftarrow0111\leftarrow1000\leftarrow1001$$

模 12 减法计数器的逻辑电路如上页图 7.25 所示。其中，74193 的 CLR 端接地，CP_U 接逻辑 1，CP_D 接计数脉冲 CP，$\overline{LD}$ 端受初态设置端和计数器状态的控制，当 $\overline{LD}$ 为 1 时 74193 工作在减法计数状态。初态设置端平时为 1，在电路开始工作时通过一个负脉冲信号置入初态“1111”，然后电路在计数脉冲作用下开始减 1 计数。当计数器输出由 0100 变为 0011 时，其中或门输出由 1 变为 0，并经与门送至 $\overline{LD}$ 端，使计数器立即置入 1111，当下一脉冲到来时继续进行减 1 计数。

3. 构成模大于 16 的计数器

利用计数器的进位输出或借位输出脉冲作为计数脉冲，将多个 4 位计数器进行级联，即可构成模大于 16 的计数器。例如，将两片 74193 按图 7.26 所示进行连接，即为一个模为 256 的减法计数器。用类似的方法，也可以构成模为 256 的加法计数器。

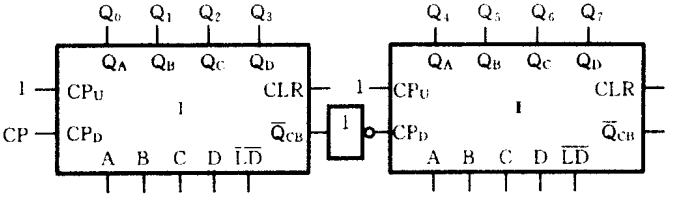


图 7.26　模 256 减法计数器逻辑电路

用 4 位二进制计数器级联后，再恰当地使用预置、清除等功能，便可构成模大于 16 的任意进制计数器。

例 7.14　用两片 4 位二进制同步可逆计数器 74193 构成模 $(147)_{10}$ 的加法计数器。

解　设计数器状态变化序列为 $(0)_{10}\sim(146)_{10}$，当计数器状态由 $(146)_{10}$ 变为 $(147)_{10}$ 时，应该令其进入 $(0)_{10}$。因为 $(147)_{10}=(10010011)_2$，所以根据 74193 的功能可作出模 $(147)_{10}$ 加法计数器的逻辑电路，如图 7.27 所示。图中，片Ⅰ和片Ⅱ的 CP_D 端、$\overline{LD}$ 端均接 1，CLR 端为清除控制端。计数脉冲由片Ⅰ的 CP_U 端输入，片Ⅰ的进位输出脉冲 $\overline{Q}_{CC}$ 经反相后作为片Ⅱ的计数脉冲。工作时先将计数器清零，在计数脉冲到来后，计数器开始加 1 计数，当计数器的状态 $Q_7Q_6Q_5Q_4Q_3Q_2Q_1Q_0=10010011$ 时，产生一个高电平，又将计数器清零，实现了模 147 加法计数。

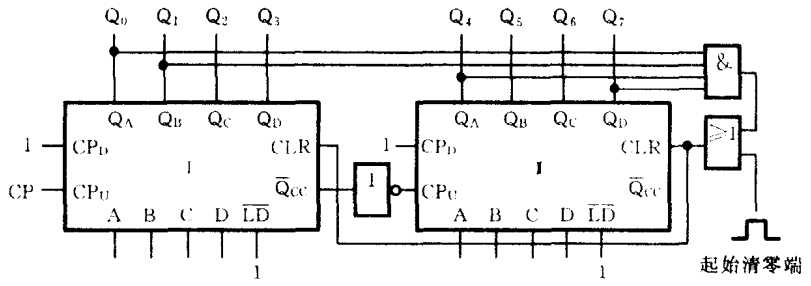


图 7.27　模 147 加法计数器逻辑电路

4. 集成异步计数器

最常用的中规模异步时序逻辑器件有二-五-十进制加法计数器 74290、双时钟 4 位二进制加法计数器 74293 等集成器件。下面以 74290 为例对其外部特性及应用进行介绍。

中规模集成加法计数器 74290 的内部包括 4 个主从 J-K 触发器。触发器 0 组成模 2 计数器，计数脉冲由 CP_A 提供；触发器 1～触发器 3 组成异步模 5 计数器，计数脉冲由 CP_B 提供。该芯片的引脚排列图和逻辑符号如图 7.28 所示，功能表如表 7.11 所示。

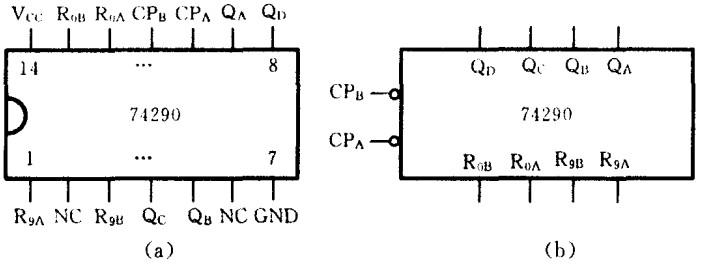


图 7.28　74290 的引脚排列图和逻辑符号

表 7.11　74290 的功能表

输入					输出			
R_{0A}	R_{0B}	R_{9A}	R_{9B}	CP	Q_D	Q_C	Q_B	Q_A
1	1	0	d	d	0	0	0	0
1	1	d	0	d	0	0	0	0
d	d	1	1	d	1	0	0	1
d	0	d	0	↓	计数			
0	d	0	d	↓	计数			
0	d	d	0	↓	计数			
d	0	0	d	↓	计数			

集成异步计数器 74290 共有 6 个输入和 4 个输出。其中，R_{0A}、R_{0B}为清零输入信号，高电平有效；R_{9A}、R_{9B}为置 9(即二进制 1001)输入信号，高电平有效；CP_A、CP_B为计数脉冲信号；Q_D、Q_C、Q_B、Q_A为数据输出信号。由表 7.11 可以归纳出 74290 具有如下功能。

(1) 异步清零功能

当 $R_{9A} \cdot R_{9B}=0$ 且 $R_{0A}=R_{0B}=1$ 时，不需要输入脉冲配合，电路可以实现异步清零操作，使 $Q_DQ_CQ_BQ_A=0000$。

(2) 异步置 9 功能

当 $R_{9A}=R_{9B}=1$ 时，不论 R_{0A}、R_{0B}及输入脉冲为何值，均可实现异步置 9 操作，使 $Q_DQ_CQ_BQ_A=1001$。

(3) 计数功能

当 $R_{9A} \cdot R_{9B}=0$ 且 $R_{0A} \cdot R_{0B}=0$ 时，电路实现如下几种计数功能。

① 模 2 计数器：若将计数脉冲加到 CP_A端，并从 Q_A端输出，则可实现 1 位二进制加法计数(二分频)。

② 模 5 计数器：若将计数脉冲加到 CP_B端，并从 $Q_DQ_CQ_B$端输出，则可实现五进制加法计数，状态转移表如表 7.12 所示。

表 7.12　74290 模 5 计数器状态表

序号	Q_D	Q_C	Q_B
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0

③ 模 10 计数器：用 74290 构成模 10 计数器有两种不同的方法，一种是构成 8421 码十进制计数器，另一种是构成 5421 码十进制计数器。两种方法的连接示意图分别如图 7.29(a)、(b)所示。

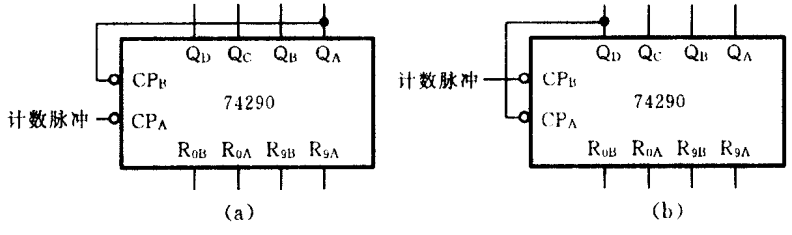


图 7.29　74290 构成的两种模 10 计数器连接示意图

在图 7.29(a)中，计数脉冲加到 CP_A端，并将输出端 Q_A接到 CP_B端。在这种方式下，每来 2 个计数脉冲，模 2 计数器输出 Q_A产生一个负跳变，使模 5 计数器增 1，经过 10 个脉冲作用后，模 5 计数器循环一周，实现 8421 码十进制加法计数。状态转移表如表 7.13 所示。

表 7.13　8421 码模 10 计数器状态表

序号	Q_D	Q_C	Q_B	Q_A
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

表 7.14　5421 码模 10 计数器状态表

序号	Q_A	Q_B	Q_C	Q_D
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	1	0	0	0
6	1	0	0	1
7	1	0	1	0
8	1	0	1	1
9	1	1	0	0

在图 7.29(b)中，计数脉冲加到模 5 计数器的 CP_B 端，并将模 5 计数器的高位输出端 Q_D 接到模 2 计数器的 CP_A 端。在这种方式下，每来 5 个计数脉冲，模 5 计数器输出 Q_D 产生一个负跳变，使模 2 计数器增 1，经过 10 个脉冲作用后，模 2 计数器循环一周，实现 5421 码十进制加法计数。状态转移表如表 7.14 所示。

集成异步计数器 74290 除了完成上述基本功能外，也可用来构成其他计数器。

例 7.15　用集成异步计数器 74290 设计一个模 8 加法计数器。

解　根据设计要求，首先将集成异步计数器 74290 按 8421 码十进制加法计数器连接，并将计数器初始状态设为“0000”。该计数器的状态转移图如图 7.30 所示。

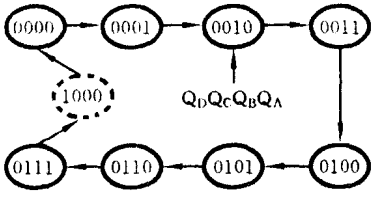


图 7.30　状态转移图

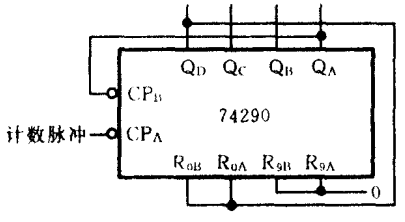


图 7.31　逻辑电路

由计数器的状态转移图可以看出，当电路在第八个计数脉冲作用下 $Q_DQ_CQ_BQ_A$ 由 0111 转为 1000 时，计数器应该回到初始状态 0000，即 Q_D 由 0 变为 1 时，应使计数器清零。根据集成异步计数器 74290 的功能表可知，只需利用 Q_D 作为异步清零信号与 R_{0A}、R_{0B} 相连接即可实现要求的状态转移，逻辑电路如图7.31所示。

7.2.2　集成寄存器

寄存器是数字系统中用来存放数据或运算结果的一种常用逻辑部件。寄存器的主要组成部分是触发器，一个触发器能存储 1 位二进制代码，所以要存放 n 位二进制代码的寄存器应包含 n 个触发器。中规模集成电路寄存器除了具有接收数据、保存数据和传送数据等基本功能外，通常还具有左、右移位，串、并输入，串、并输出以及预置、清零等多种功能，属于多功能寄存器。

1. 典型芯片

中规模集成电路寄存器的种类很多，常用的集成器件有 4 位寄存器 7495、74175、74194，5 位寄存器 7496，8 位寄存器 7491 等。74194 是一种常用的 4 位双向移位寄存器。下面以 74194 为例对其外部特性及进行讨论。

74194 是一种常用的 4 位双向移位寄存器，其管脚排列图和逻辑符号如图 7.32(a)、(b)所示。

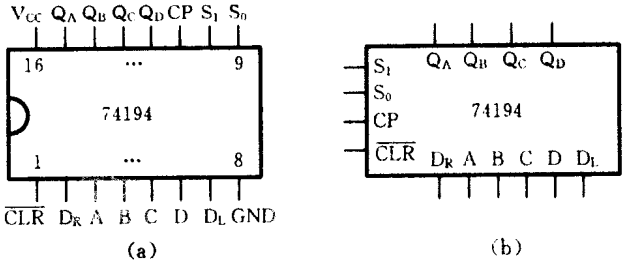


图 7.32　74194 的管脚排列图和逻辑符号

74194 输入/输出引线的说明如表 7.15 所示，该芯片控制端的功能如表 7.16 所示，功能表如表 7.17 所示。

从功能表可知，双向移位寄存器在$\overline{CLR}$、S_1 和 S_0 的控制下可完成数据的并行输入、右移串行输入，左移串行输入、保持和清除等 5 种功能。

表 7.15　74194 引线说明

引线名称		说　明
输入端	$\overline{CLR}$	清除
	D,C,B,A	并行数据输入
	D_R	右移串行数据输入
	D_L	左移串行数据输入
	S_1,S_0	工作方式选择控制
	CP	工作脉冲
输出端	Q_D,Q_C,Q_B,Q_A	寄存器的状态

表 7.16　控制端的功能

控制信号		功　能
S_1	S_0	
0	0	保　持
0	1	右　移
1	0	左　移
1	1	并行输入

表 7.17 74194 的功能表

输入										输出			
$\overline{CLR}$	CP	S_1	S_0	D_R	D_L	D	C	B	A	Q_D	Q_C	Q_B	Q_A
0	d	d	d	d	d	d	d	d	d	0	0	0	0
1	0	d	d	d	d	d	d	d	d	Q_D^n	Q_C^n	Q_B^n	Q_A^n
1	↑	1	1	d	d	x_0	x_1	x_2	x_3	x_0	x_1	x_2	x_3
1	↑	0	1	1	d	d	d	d	d	1	Q_D^n	Q_C^n	Q_B^n
1	↑	0	1	0	d	d	d	d	d	0	Q_D^n	Q_C^n	Q_B^n
1	↑	1	0	d	1	d	d	d	d	Q_C^n	Q_B^n	Q_A^n	1
1	↑	1	0	d	0	d	d	d	d	Q_C^n	Q_B^n	Q_A^n	0
1	d	0	0	d	d	d	d	d	d	Q_D^n	Q_C^n	Q_B^n	Q_A^n

2. 应用举例

寄存器除完成预定功能外，在数字系统中还能用来构成计数器和脉冲序列发生器等逻辑部件。

例 7.16 用 4 位双向移位寄存器 74194 构成一个模 8 计数器。该计数器的状态转移关系如表 7.18 所示。

表 7.18 状态转移表

现态				次态			
Q_D	Q_C	Q_B	Q_A	Q_D^{n+1}	Q_C^{n+1}	Q_B^{n+1}	Q_A^{n+1}
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	1
0	0	1	1	0	1	1	1
0	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
1	1	1	0	1	1	0	0
1	1	0	0	1	0	0	0
1	0	0	0	0	0	0	0

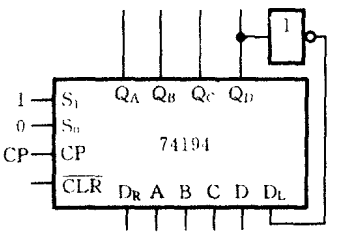


图 7.33 逻辑电路图

解 由 74194 的功能表可知，要满足表 7.18 所示状态转移关系，只需将 74194 预置初始状态“0000”后，将 S_1S_0 接 10，并令 D_L 与 $\overline{Q}_D$ 相连接即可。其逻辑电路如图 7.33 所示。

该电路工作时，首先令 $\overline{CLR}$ 接“0”，将寄存器清 0 后，再令 $\overline{CLR}$ 接“1”。寄存器清 0 后，将 S_1S_0 接 10，寄存器在时钟作用下，循环左移实现给定的状态转移。

例 7.17 用一片 74194 和适当的逻辑门构成产生序列为 01100101 的脉冲序列发生器。

解　序列信号发生器可由移位寄存器和反馈逻辑电路构成，其结构框图如图 7.34 所示。

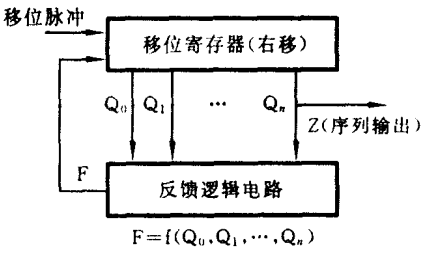


图 7.34　序列发生器结构框图

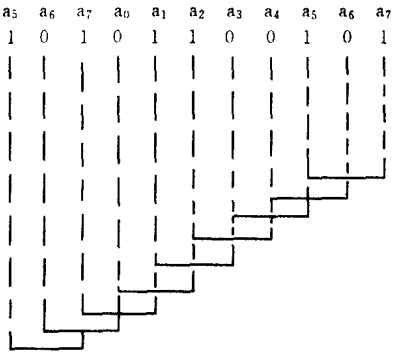


图 7.35　序列与移位寄存器状态的关系

假定序列发生器产生的序列周期为 T_p，移位寄存器的级数(触发器个数)为 n，应满足关系式 $2^n \geqslant T_p$。本例的 $T_p = 8$，故 $n \geqslant 3$，选择 $n=3$。设输出序列 $Z=a_0a_1a_2a_3a_4a_5a_6a_7$，图 7.35 列出了所要产生的序列(以周期 $T_p=8$ 重复，最右边信号先输出)与移位寄存器状态的关系。

图 7.35 中，数码下面的水平线段表示移位寄存器的状态。将 $a_5a_6a_7=101$ 作为寄存器的初始状态，即初始状态为 $Q_AQ_BQ_C=101$，从 Q_C 产生输出，由反馈电路依次形成 a_4、a_3、a_2、a_1、a_0、a_7、a_6、a_5 作为右移串行输入端的输入，这样便可在时钟脉冲作用下，产生规定的输出序列。电路在时钟作用下的状态变化过程及右移输入值如表 7.19 所示。

由表 7.19 可得到反馈函数 F 的逻辑表达式为

$$\begin{aligned} F &= \overline{Q}_A\overline{Q}_BQ_C + Q_A\overline{Q}_B\overline{Q}_C + \overline{Q}_AQ_B\overline{Q}_C + \overline{Q}_AQ_BQ_C \\ &= \overline{Q}_AQ_B + \overline{Q}_AQ_C + Q_A\overline{Q}_B\overline{Q}_C \\ &= \overline{Q}_A(Q_B+Q_C) + Q_A\overline{Q}_B\overline{Q}_C \\ &= \overline{Q}_A(Q_B+Q_C) + Q_A\overline{Q_B+Q_C} \\ &= Q_A \oplus (Q_B+Q_C) \end{aligned}$$

根据上述表达式和 74194 的功能表，可画出该序列发生器的逻辑电路，如图 7.36 所示。

该电路的工作过程为：在 s_1s_0 的控制下，先置寄存器 74194 的初始状态为 $Q_AQ_BQ_C=101$，然后令其工作在右移串行输入方式，即可在时钟脉冲作用下从 Z 端产生所需要的脉冲序列。

表 7.19 电路状态变化表

CP	F(D_R)	Q_A	Q_B	Q_C
0	0	1	0	1
1	0	0	1	0
2	1	0	0	1
3	1	1	0	0
4	0	1	1	0
5	1	0	1	1
6	0	1	0	1
7	1	0	1	0

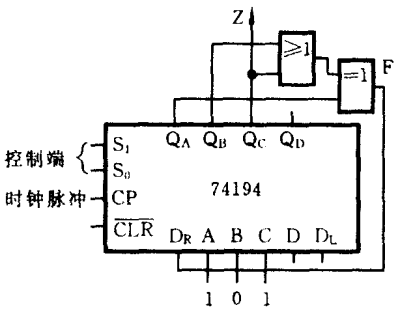


图 7.36 逻辑电路

7.3 常用中规模信号产生与变换电路

数字系统中使用的集成电路除组合电路和时序电路外，还有一种信号产生与变换电路。它们常用于产生各种宽度、幅值的脉冲信号，对信号进行变换、整形以及完成模拟信号与数字信号之间的转换等。本节介绍几种最常用的中规模信号产生与变换集成电路及其应用。

7.3.1 集成定时器 555 及其应用

集成定时器 555 是一种将模拟电路功能与逻辑电路功能巧妙地结合在一起的中规模集成电路。该电路功能灵活、适用范围广，只要在外部配上几个适当的阻容元件，就可以很方便地构成多谐振荡器、施密特触发器和单稳态触发器等电路，完成脉冲信号的产生、定时和整形等功能。因而在控制、定时、检测、仿声、报警等方面有着广泛应用。常用的集成定时器 555 有 5G555（TTL 电路）和 CC7555（CMOS 电路）等。下面以 5G555 为例说明其功能和应用。

1. 5G555 的电路结构与逻辑功能

(1) 电路结构和工作原理

5G555 的电路结构图和管脚排列图分别如图 7.37(a)、(b)所示。

由图 7.37(a)可知，集成定时器 5G555 由电阻分压器、电压比较器、基本 R-S 触发器、放电三极管和输出缓冲器 5 部分组成。定时器的功能主要取决于比较器 C_1 和 C_2，由它们的输出直接控制基本 R-S 触发器的状态和放电三极管 T 的状态，从而决定整个电路的输出状态。

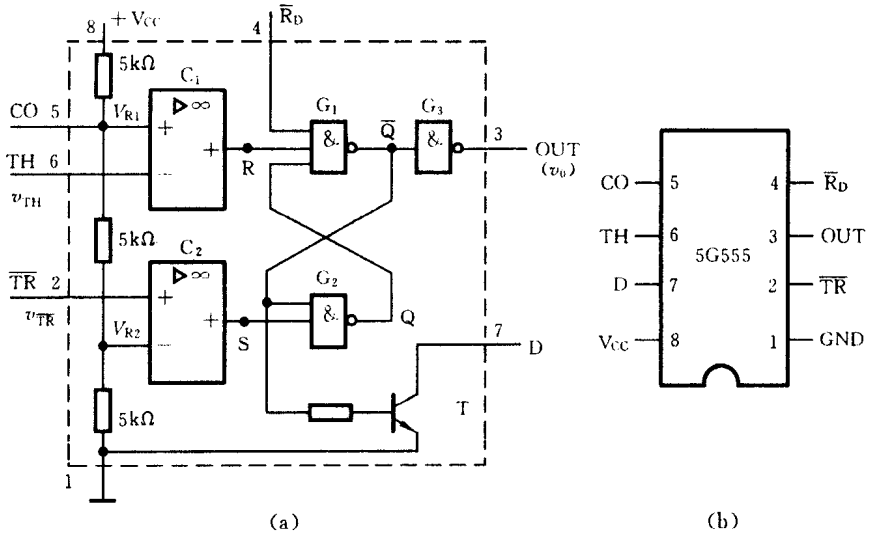


图 7.37　5G555 的电路结构和管脚功能

① 电阻分压器。由 3 个阻值均为 5kΩ 的电阻串联构成分压器，为电压比较器 C_1 和 C_2 提供参考电压。当电压控制端 CO 不外加控制电压 v_{CO} 时，$V_{R1}=\frac{2}{3}V_{CC}$，$V_{R2}=\frac{1}{3}V_{CC}$，当外加控制电压 v_{CO} 时，比较器的参考电压将发生变化，相应电路的阈值、触发电平也将随之改变，并进而影响电路的定时参数。为了防止干扰，当不外加控制电压时，CO 端一般通过一个小电容（如 0.01μF）接地，以旁路高频干扰。

② 电压比较器 C_1 和 C_2。电压比较器 C_1 和 C_2 是两个结构完全相同的理想运算放大器。当运算放大器的同相输入 V_+ 大于反相输入 V_- 时，其输出为高电平 1 信号；而当 V_+ 小于 V_- 时，其输出为低电平 0 信号。

当比较器 C_1 的同相输入端（+端）接参考电压 V_{R1}，反相输入端（−端）与阈值输入端 TH 相连时，其输出 R 端的状态取决于阈值输入信号 v_{TH} 与 V_{R1} 的比较结果。当 $v_{TH}<V_{R1}$ 时，输出 R 为高电平 1；当 $v_{TH}>V_{R1}$ 时，输出 R 为低电平 0。

当比较器 C_2 的同相输入端（+端）与触发输入端 $\overline{TR}$ 相连，反相输入端（−端）接参考电压 V_{R2} 时，其输出端 S 的状态取决于触发输入信号 $v_{\overline{TR}}$ 与 V_{R2} 的比较结果。当 $v_{\overline{TR}}<V_{R2}$ 时，输出 S 为低电平 0；当 $v_{\overline{TR}}>V_{R2}$ 时，输出 S 为高电平 1。

③ 基本 R-S 触发器。两个与非门 G_1 和 G_2 构成了低电平触发的基本 R-S 触发器。触发器输入信号 R、S 为比较器 C_1、C_2 的输出，触发器 Q 端状态为电路输出端 OUT 的状态，触发器 $\overline{Q}$ 端状态控制放电三极管 T 的导通与截止。当外部复位

信号 $\overline{R}_D$ 为 0(低电平)时,可使 $v_o=0$,定时器输出直接复位。

④ 放电三极管 T。构成泄放电路,T 的集电极即输出端 D。如果将 D 端经过一个外接电阻接至电源,即可组成一个反相器。当 $Q=0$ ($\overline{Q}=1$)时,T 导通,D 端输出为低电平 0;当 $Q=1$($\overline{Q}=0$)时,T 截止,D 端输出为高电平 1。可见,D 端的逻辑状态与输出端 OUT 的状态相同。

⑤ 输出缓冲器。输出缓冲器 G_3 的作用是提高负载能力,并隔离负载对定时器的影响。

(2) 电路功能

根据电路结构和工作原理,可归纳出 5G555 的功能表,如表 7.20 所示。

表 7.20 5G555 的功能表

输入			比较器输出		输出	
v_{TH}	v_{TR}	$\overline{R}_D$	$R(C_1)$	$S(C_2)$	OUT	放电三极管 T
d	d	0	d	d	0	导通
$<V_{R1}$	$<V_{R2}$	1	1	0	1	截止
$<V_{R1}$	$>V_{R2}$	1	1	1	不变	不变
$>V_{R1}$	$>V_{R2}$	1	0	1	0	导通

当 CO 端不外接控制电压时,5G555 的功能表如表 7.21 所示。

表 7.21 5G555 不外接控制电压时的功能表

输入			比较器输出		输出	
v_{TH}	v_{TR}	$\overline{R}_D$	$R(C_1)$	$S(C_2)$	OUT	放电三极管 T
d	d	0	d	d	0	导通
$<\frac{2}{3}V_{CC}$	$<\frac{1}{3}V_{CC}$	1	1	0	1	截止
$<\frac{2}{3}V_{CC}$	$>\frac{1}{3}V_{CC}$	1	1	1	不变	不变
$>\frac{2}{3}V_{CC}$	$>\frac{1}{3}V_{CC}$	1	0	1	0	导通

2. 5G555 的应用

由于 5G555 具有电源范围宽、定时精度高、使用方法灵活、带负载能力强等特点,所以应用非常广泛。下面介绍它在脉冲信号产生、定时与整形方面的基本应用。

(1) 用 5G555 构成多谐振荡器

多谐振荡器又称矩形波发生器,它有两个暂稳态,电路一旦起振,两个暂稳态就交替变化,输出矩形脉冲信号。

① 电路构成及工作原理。用 5G555 构成的多谐振荡器如图 7.38(a)所示，图 7.38(b)所示为电路的工作波形图。

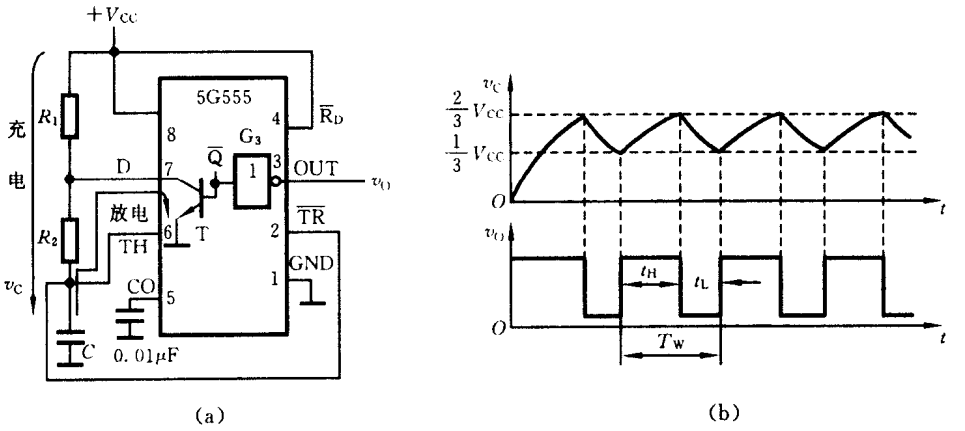


图 7.38　用 5G555 构成的多谐振荡器电路及其工作波形图

从图 7.38(a)可知，电路由 5G555 外加两个电阻和一个电容组成。5G555 的 D 端(即放电三极管 T 的集电极)经 R_1 接至电源 V_{CC}，构成一个反相器。电阻 R_2 和电容 C 构成积分电路。积分电路的电容电压 v_C 作为电路输入接至输入端 TH 和 $\overline{TR}$。

电路的工作原理如下：

接通电源 V_{CC} 的瞬间，电容 C 上的电压 v_C 不能突变，故 TH 端的电压小于 $\frac{2}{3}V_{CC}$，$\overline{TR}$ 端的电压小于 $\frac{1}{3}V_{CC}$，输出端 OUT 的状态为 1，此时 $\overline{Q}=0$，放电三极管 T 截止，电源 V_{CC} 经过 R_1、R_2 对电容 C 充电，v_C 逐渐上升，电路处在第一个暂稳态。

当电容上的电压 v_C 逐渐升高到 $\frac{2}{3}V_{CC}$ 时，由于 TH 端和 $\overline{TR}$ 端的电压为 $\frac{2}{3}V_{CC}$，使输出端 OUT 的状态变为 0，此时 $\overline{Q}=1$，放电三极管 T 导通，电容 C 经 R_2 和 T 放电，v_C 逐渐下降，电路处在第二个暂稳态。

当电容 C 上的电压 v_C 下降到 $\frac{1}{3}V_{CC}$ 时，使输出 OUT 又从低电平 0 变为高电平 1，放电三极管 T 截止，电源 V_{CC} 再经 R_1、R_2 向 C 充电，电路返回到第一个暂稳态。

如此周而复始地在两个暂稳态之间交替变换，便产生了如图 7.39(b)所示的矩形脉冲信号输出。

② 输出脉冲信号参数的计算。输出高电平的持续时间 t_H 是电容电压 v_C 从 $\frac{1}{3}V_{CC}$ 上升到 $\frac{2}{3}V_{CC}$ 所需的时间，它与充电回路的时间常数 $(R_1+R_2)C$ 相关，近似计算公式为

$$t_H \approx 0.7(R_1+R_2)C$$

输出低电平的持续时间 t_L 是电容电压 v_C 从 $\frac{2}{3}V_{CC}$ 下降到 $\frac{1}{3}V_{CC}$ 所需的时间，它与放电回路的时间常数 R_2C 相关，近似计算公式为

$$t_L \approx 0.7R_2C$$

矩形波振荡周期 T_W 的近似计算公式为

$$T_W \approx t_H + t_L \approx 0.7(R_1+2R_2)C$$

矩形波振荡频率 f 的近似计算公式为

$$f = \frac{1}{T_W} \approx \frac{1}{0.7(R_1+2R_2)C} \approx \frac{1.43}{(R_1+2R_2)C}$$

矩形波的占空比 Q 的近似计算公式为

$$Q = \frac{t_H}{T_W} \approx \frac{0.7(R_1+R_2)C}{0.7(R_1+2R_2)C} \approx \frac{R_1+R_2}{R_1+2R_2}$$

③ 占空比可调的多谐振荡器。在图 7.38(a)所示电路中，一旦选定电阻 R_1 和 R_2，输出信号的占空比 Q 便固定下来。如果对该电路稍加改进，在原有基础上增加一个可调电阻 R_W，并利用二极管的单向导电性，用 D_1、D_2 两个二极管将充电回路和放电回路隔离开，便可构成占空比可调的多谐振荡器。改进后的电路如图 7.39 所示，调节电阻 R_W 的阻值就可改变输出矩形波的占空比 Q。

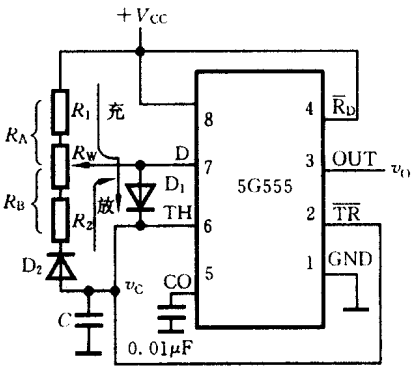


图 7.39 占空比可调的多谐振荡器

在图 7.39 所示振荡器中，R_W 分成可变的两部分，靠近 R_1 一侧的部分和 R_1 一起构成 R_A，靠近 R_2 一侧的部分和 R_2 一起构成 R_B。电源 V_{CC} 通过 R_A、D_1 向电容 C 充电；电容 C 通过 D_2、R_B 及 5G555 内部的放电三极管 T 放电。充、放电回路的时间常数决定输出信号高、低电平的持续时间，它们分别为

$$t_H \approx 0.7R_AC \qquad t_L \approx 0.7R_BC$$

占空比 Q 为

$$Q=\frac{t_H}{t_H+t_L}\approx\frac{0.7R_AC}{0.7(R_A+R_B)C}\approx\frac{R_A}{R_A+R_B}$$

调节可变电阻 R_W，便可改变 R_A 和 R_B 的阻值，进而改变输出矩形波的占空比。

(2) 用 5G555 构成施密特触发器

施密特触发器是一种特殊的双稳态时序电路，与一般的双稳态触发器相比，它具有两个特点。第一，施密特触发器属于电平触发，对于缓慢变化的信号同样适用。只要输入信号电平达到相应的触发电平，输出信号就会发生突变，从一个稳态翻转到另一个稳态，并且稳态的维持依赖于外加触发输入信号。第二，对于正向和负向增长的输入信号，电路有不同的阈值电平。这一特性称为滞后特性或回差特性。

图 7.40(a)、(b)分别给出了一种常用施密特触发器的逻辑符号和电压传输特性，该器件实际上是一个具有滞后特性的反相器。

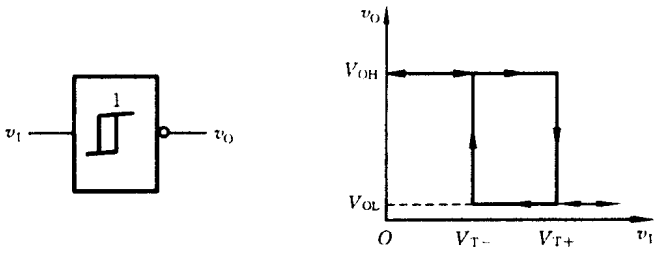


图 7.40　施密特触发器

图中，v_I 上升时的阈值电压 V_{T+} 称为**正向阈值电平或上限触发电平**；下降时的阈值电压 V_{T-} 称为**负向阈值电平或下限触发电平**。它们之间的差值称为回差电压(或滞后电压)，用 ΔV_T 表示。即

$$\Delta V_T=V_{T+}-V_{T-}$$

下面介绍用 5G555 构成的施密特触发器及其应用。

1) 工作原理

用 5G555 构成的施密特触发器原理图及其传输特性分别如图 7.41(a)、(b)所示。按图 7.41(a)所示，将 5G555 的 TH 端和 $\overline{TR}$ 端连接在一起作为信号输入端，OUT 作为输出端，便构成了一个施密特反相器。该电路工作原理如下：

当 v_I 从 0 开始逐渐升高时，若 $v_I<\frac{1}{3}V_{CC}$，则 $v_{TH}=v_{\overline{TR}}<\frac{1}{3}V_{CC}$，电路输出 v_O 为高电平 V_{OH}；若 v_I 处于 $\frac{1}{3}V_{CC}<v_I<\frac{2}{3}V_{CC}$，则 $v_{TH}<\frac{2}{3}V_{CC}$，而 $v_{\overline{TR}}>\frac{1}{3}V_{CC}$，电路输出保持高电平不变；若 v_I 上升到 $v_I\geqslant\frac{2}{3}V_{CC}$ 时，则 $v_{TH}=v_{\overline{TR}}\geqslant\frac{2}{3}V_{CC}$，电路输出

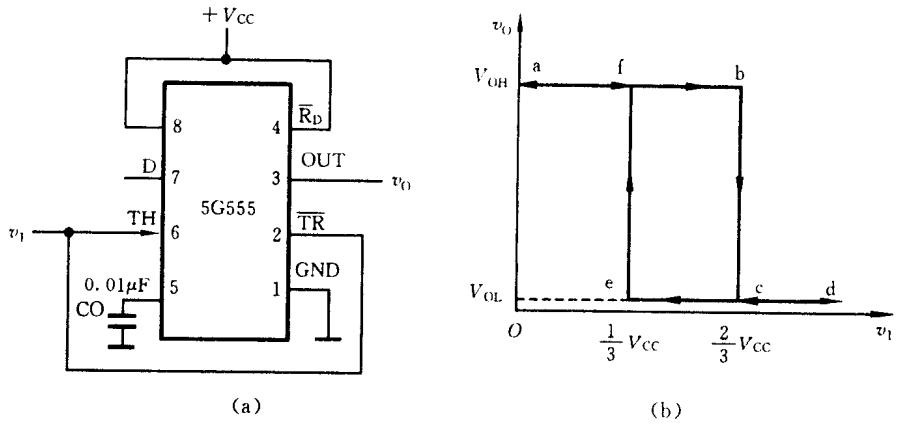


图 7.41　5G555 构成的施密特触发器

变为低电平 V_{OL}。可见，电路正向阈值电压 $V_{T+}=\frac{2}{3}V_{CC}$。传输特性如图 7.41(b)中的 a→b→c→d。

当 v_I从高于$\frac{2}{3}V_{CC}$逐渐下降时，若 v_I处于$\frac{1}{3}V_{CC}<v_I<\frac{2}{3}V_{CC}$，则因为 $v_{TH}<\frac{2}{3}V_{CC}$，$v_{\overline{TR}}>\frac{1}{3}V_{CC}$，电路输出保持低电平不变；若 v_I下降到 $v_I\leqslant\frac{1}{3}V_{CC}$，则由于 $v_{TH}=v_{\overline{TR}}\leqslant\frac{1}{3}V_{CC}$，故电路输出变为高电平 V_{OH}。可见，电路的负向阈值电压$V_{T-}=\frac{1}{3}V_{CC}$。传输特性如图 7.41(b)中的 d→e→f→a。

由以上分析可知，该电路的回差电压

$$\Delta V_T=V_{T+}-V_{T-}=\frac{1}{3}V_{CC}$$

如果将图 7.41(a)中的电压控制端 CO(5 脚)接控制电压 v_{CO}，则随着参考电压的改变，将使 V_{T+}、V_{T-}和 ΔV_T发生相应变化。

2) 典型应用

施密特触发器的应用广泛，其典型应用有波形变换、脉冲整形、幅值鉴别等。

波形变换：施密特触发器能将正弦波、三角波或任意形状的模拟信号波形变换成矩形波。图 7.42(a)所示的是将正弦波变换成矩形波。

脉冲整形：经传输后的矩形脉冲往往由于干扰及传输线路的分布电容等因素而使信号发生畸变，出现前、后沿变坏或信号电平波形上叠加脉动干扰波等现象。用施密特触发器，选择适当的回差电压 ΔV_T，即可对输入信号整形后输出。图 7.42(b)所示的就是将干扰后的不规则波形，经整形后变成规则波形。

幅值鉴别：施密特触发器能在一系列幅值各异的脉冲信号中鉴别出幅值大于 V_{T+} 的脉冲，并产生对应的输出信号。如图 7.42(c)所示，输入信号经鉴幅后，仅幅值大于 V_{T+} 的脉冲会产生相应输出信号。

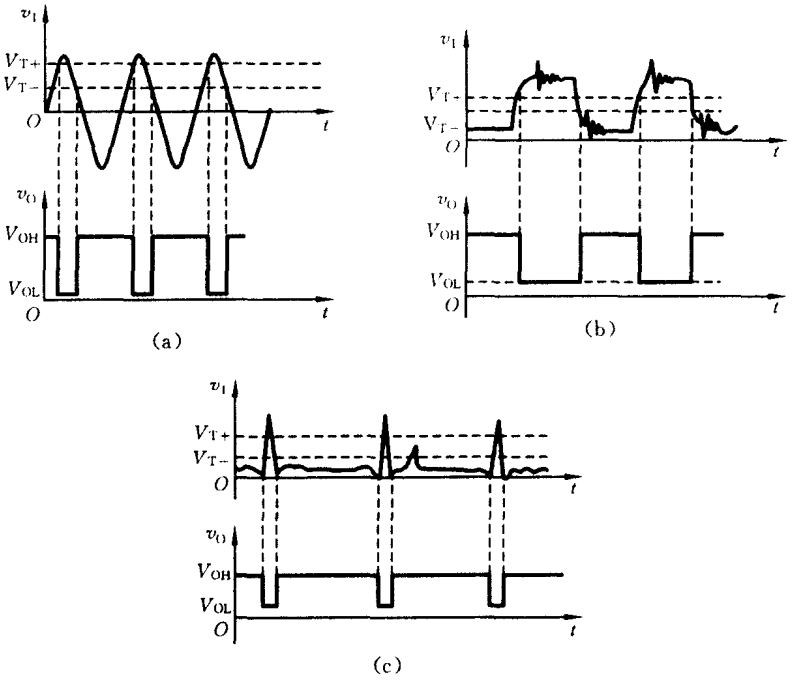


图 7.42　施密特触发器的典型应用

(3) 用 5G555 构成单稳态触发器

单稳态触发器只有一个稳态。在外来触发脉冲的作用下，电路由稳态翻转到暂稳态，暂稳态是一个不能长久保持的状态，维持一段时间后，电路会自动返回到稳态。暂稳态持续时间的长短取决于电路本身的参数。

单稳态触发器的种类很多，有不同型号的集成单稳态触发器供选用。下面介绍用 5G555 构成的单稳态触发器。

1) 工作原理

用 5G555 构成的单稳态触发器电路原理图和工作波形图分别如图 7.43(a)、(b)所示。在图 7.43(a)所示电路中，R 和 C 是定时元件，触发信号 v_I 从 $\overline{TR}$ 端输入，输出信号 v_O 从 OUT 端输出，电压控制端 CO 经过一个 0.01 μF 的电容接地。D 端(即放电三极管 T 的集电极)和输入端 TH 相连接，一端经电阻 R 接至电源，另一端经电容 C 接地。电路工作原理如下。

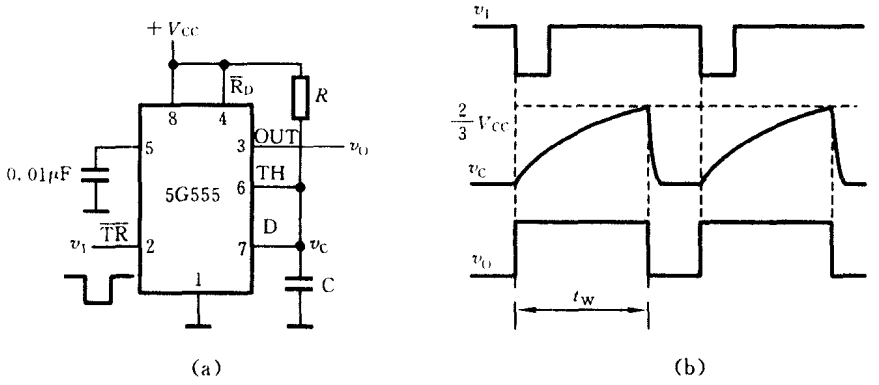


图 7.43　5G555 构成的单稳态触发器

稳态：当未加触发脉冲时，输入 v_I 保持高电平（$>\frac{1}{3}V_{CC}$），即 $\overline{TR}$ 端的电压大于 $\frac{1}{3}V_{CC}$。设刚接通电源时输出 v_O 为高电平，放电三极管截止，电源经 R 向电容 C 充电。开始时 v_C 很小，即 $v_{TH}<\frac{2}{3}V_{CC}$，v_O 维持高电平，当 v_C 逐渐上升到 $v_C\geqslant\frac{2}{3}V_{CC}$ 时，由于 $v_{\overline{TR}}>\frac{1}{3}V_{CC}$，$v_{TH}>\frac{2}{3}V_{CC}$，使输出 v_O 变为低电平。这时放电三极管 T 导通，电容 C 通过 T 迅速放电，v_C 下降，直至 $v_C\approx0$。由于此时 $v_{TH}<\frac{2}{3}V_{CC}$，$v_{\overline{TR}}>\frac{1}{3}V_{CC}$，所以输出 v_O 保持低电平不变，即输出稳定在 0 状态，可见，稳态时 $v_O=0$，T 导通。

暂稳态：当从 v_I 输入一个触发脉冲时，v_I 从 1 到 0 的跳变，使 $v_{\overline{TR}}<\frac{1}{3}V_{CC}$，此时 TH 端仍为低（$<\frac{2}{3}V_{CC}$），故输出 v_O 由 0 变为 1，电路进入暂稳态：$v_O=1$，T 管截止，电源经 R 向 C 充电。

在暂稳态期间，v_I 端的触发脉冲撤消，使 v_I 变为 1，即 $v_{\overline{TR}}>\frac{1}{3}V_{CC}$，且随着电源对 C 的充电，$v_C$ 按指数规律上升，待 v_C 上升到 $v_C\geqslant\frac{2}{3}V_{CC}$ 时，v_O 由 1 变为 0，暂稳态结束。此时 T 管导通，电容 C 迅速放电直至 $v_C\approx0$，电路自动返回到稳态。

至此，单稳态电路在触发信号作用下完成了一个从稳态到暂稳态，然后自动返回稳态的工作周期。以后重复上述过程，工作波形如图 7.43(b)所示。

2) 脉冲宽度的计算和调整

由图 7.43(b)可知，单稳态电路的暂稳态持续时间也就是输出脉冲宽度 t_W，是

由充电回路的时间常数 RC 决定的，近似计算公式为

$$t_w \approx 1.1RC$$

由此可见，调节定时元件 R、C 的参数，即可改变输出脉冲的宽度 t_w。在工程应用中，通常 R 的取值范围是几百欧姆到几兆欧姆，C 的取值范围是几百皮法到几百微法，对应的脉冲宽度可以从几微秒到几分钟。

单稳态触发器在数字系统中通常用于脉冲的整形、定时和延迟等。

7.3.2　集成 D/A 转换器

数字系统只能处理数字信号。但在工业过程控制、智能化仪器仪表和数字通信等领域，数字系统处理的对象往往是模拟信号。例如，图 7.44 是一个数字控制系统输入/输出信号关系示意图。在生产过程控制中对温度、压力、流量等物理量进行控制时，经过传感器获取的电信号都是模拟信号。这些模拟信号必须变换成数字信号才能由数字系统加工、运算。另一方面，数字系统输出的数字信号，有时又必须变换成模拟信号才能去控制执行单元，通过执行单元对被控对象进行调节。因此，在实际应用中，必须解决模拟信号与数字信号之间的转换问题。

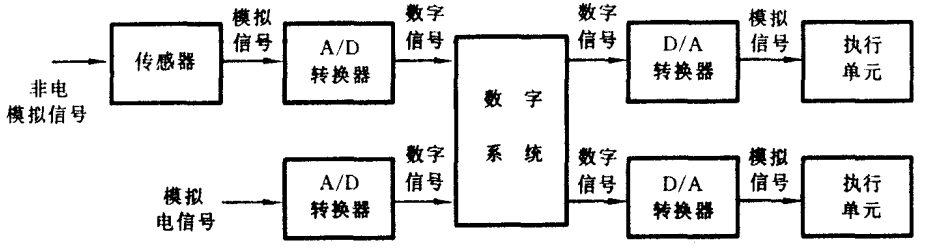


图 7.44　数字控制系统信号关系示意图

把数字信号转换成模拟信号的器件称为数/模转换器，简称 D/A 转换器或 DAC(Digital to Analog Converter)；把模拟信号转换成数字信号的器件称为模/数转换器，简称 A/D 转换器或 ADC(Analog to Digital Converter)。

下面首先介绍 D/A 转换器的基本概念以及常用集成 D/A 芯片的结构与特性。

1. D/A 转换的基本原理

(1) 转换原理

众所周知，数字量是由数字字符按位组合形成的一组代码，每位字符有一定的“权”。将数字量转换成模拟量的基本思想是：首先把数字量的每一位代码按其权

的大小转换成相应的模拟量，然后将代表各位数字量的模拟量相加，便可得到与数字量对应的模拟量。

D/A 转换器主要由数字寄存器、模拟电子开关、解码网络、求和电路和基准电压源组成，图 7.45 所示的是一个 n 位 D/A 转换器的一般结构框图。图中，数字寄存器用于存放 n 位数字量，寄存器输出的每位数码分别控制对应位的模拟电子开关，使之在解码网络中获得与该位数码权值对应的模拟量送至求和电路，求和电路将各位权值对应的模拟量相加，便可得到与 n 位数字量对应的模拟量。

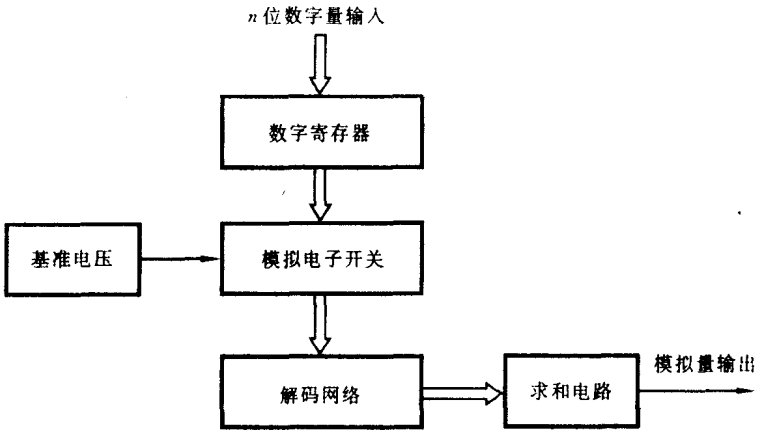


图 7.45 n 位 D/A 转换器的结构框图

(2) 转换特性

D/A 转换器输入数字量与输出模拟量之间的对应关系称为转换特性。理想的 D/A 转换器应使输出模拟量与输入数字量成正比。设输入数字量 $D=D_{n-1}\cdots D_1D_0$，输出模拟量用 A 表示，则输出与输入之间的关系为

$$A = K \cdot D = K \cdot \sum_{i=0}^{n-1} D_i 2^i$$

式中，K 为转换比例系数。

图 7.46(a) 是一个 4 位 D/A 转换器的示意框图，其转换特性曲线如图 7.46(b)所示。图中，设输出模拟量的满刻度值为 A_m，则当数字量为 0001，即只有最低有效位(LSB)为 1，其余各位为 0 时，电路输出最小模拟量 $A_{LSB}=\frac{1}{15}A_m$。推广到一般情况，n 位输入的 D/A 转换器所能转换输出的最小模拟量 $A_{LSB}=\frac{1}{2^n-1}A_m$。

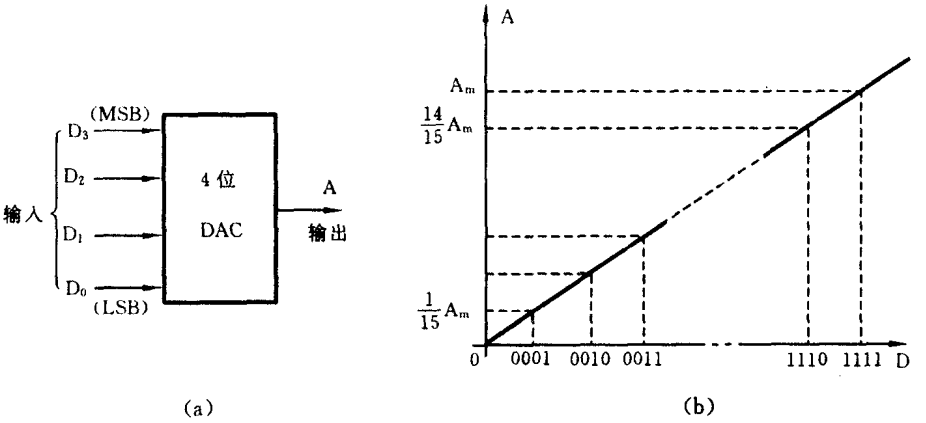


图 7.46　4 位 D/A 转换器框图和转换特性曲线

2. D/A 转换器的类型和参数

(1) D/A 转换器的类型

目前，集成 D/A 转换器有许多类型和不同的分类方法。从电路结构来看，各类集成 D/A 转换器至少都包括电阻解码网络和电子开关两个基本组成部分。

根据电阻解码网络结构的不同，D/A 转换器可分成权电阻网络 D/A 转换器、*R*-2*R* 正梯形电阻网络 D/A 转换器和 *R*-2*R* 倒梯形电阻网络 D/A 转换器等类型。

根据电子开关的不同，又可分成 CMOS 电子开关 D/A 转换器和双极型电子开关 D/A 转换器。通常双极型电子开关比 CMOS 电子开关的开关速度高。

根据输出模拟信号的类型，D/A 转换器可分为电流型和电压型两种。常用的 D/A 转换器大部分是电流型，当需要将模拟电流转换成模拟电压时，通常在输出端外加运算放大器。

随着集成电路技术的发展，D/A 转换器在电路结构、性能等方面都有很大变化。从只能实现数字量到模拟电流转换的 D/A 转换器，发展到能与微处理器完全兼容、具有输入数据锁存功能的 D/A 转换器，进一步又出现了带有参考电压源和输出放大器的 D/A 转换器等，大大提高了 D/A 转换器综合性能。

(2) 主要参数

衡量 D/A 转换器性能的主要参数有分辨率、非线性误差、绝对精度和建立时间。

① 分辨率。分辨率是指最小模拟量输出与最大模拟量输出之比。一个 n 位 D/A 转换器的分辨率为

$$分辨率 = \frac{A_{LSB}}{A_m} = \frac{A_m/2^n-1}{A_m} = \frac{1}{2^n-1}$$

由于该参数决定于数字量的位数，所以有时也用输入数字量的位数表示，如 8 位、10 位等。显然，位数越多越能反映出模拟量输出的细微变化，分辨率越高。

② 非线性误差。具有理想转换特性的 D/A 转换器，每两个相邻数字量对应的模拟量之差都为 A_{LSB}。在满刻度范围内偏离理想转换特性的最大值，称为非线性误差。

③ 绝对精度。绝对精度是指在输入端加对应满刻度数字量时，输出的实际值与理想值之差。一般该值应低于 A_{LSB}。

④ 建立时间。建立时间是指从送入数字信号起，到输出模拟量达到稳定值为止所需要的时间。建立时间反映了电路的转换速度。

在选用 D/A 转换器时，除考虑以上主要参数外，还应了解电源电压、输出方式、输入逻辑电平等。

3. 典型芯片

常用的 D/A 转换器有 8 位、10 位、12 位、16 位等种类，每种又有不同的型号。下面以集成 D/A 转换器 DAC0832 为例，对 D/A 转换器的结构与功能予以介绍。

DAC0832 是采用 CMOS 工艺制作的 8 位 D/A 转换器，采用 20 引脚双列直插式封装，该芯片与微处理器完全兼容。

(1) 引脚功能

DAC0832 的引脚排列图如图 7.47 所示。

DAC0832 共有 20 条引脚，功能如下。

$D_7 \sim D_0$：数字信号输入端，D_7 为最高位，D_0 为最低位。

$\overline{CS}$、ILE、$\overline{WR_1}$、$\overline{WR_2}$、$\overline{XFER}$：控制信号输入端。

V_R：参考电压输入端，电压值可在 $+10V \sim -10V$ 范围内选择。

V_{CC}：电源电压输入端，电压值可在 $+5V \sim +15V$ 范围内选择，最佳工作状态为 $+15V$。

I_{OUT1}、I_{OUT2}：电流输出端，因芯片内部不包含运算放大器，所以，I_{OUT1} 和 I_{OUT2} 应分别和外接运算放大器的反相输入端和同相输入端相连接。

R_{fb}：反馈电阻引出端，因 R_{fb} 与 I_{OUT1} 间有内部反馈电阻，故运算放大器的输出端可直接接到 R_{fb} 端。

AGND：模拟信号接地端。

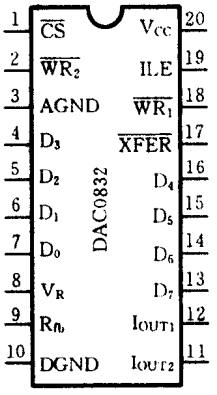


图 7.47 DAC0832 的引脚排列图

DGND:数字信号接地端。

(2) 电路结构与工作方式

DAC0832 内部包括两个 8 位数据缓冲寄存器,1 个由 T 型电阻网络和电子开关构成的 8 位 D/A 转换器和 3 个控制逻辑门。两个 8 位寄存器均带有使能控制端 EN,当 EN=1(高电平)时,寄存器输出跟随输入数据变化;当 EN=0(低电平)时,输入数据被锁存到寄存器中,寄存器输出不再受输入数据变化的影响。图7.48 所示的为 DAC0832 的内部结构图。

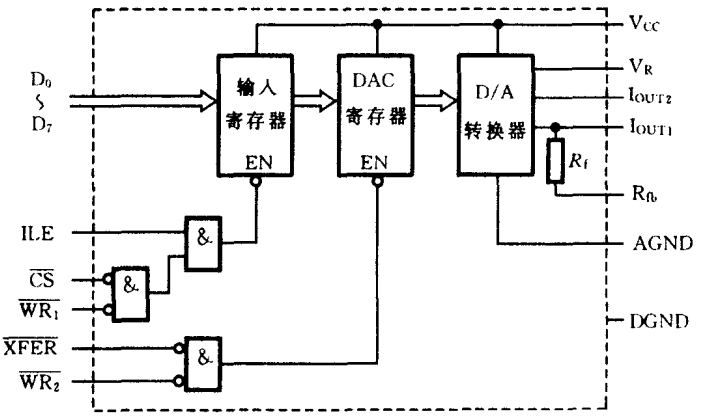


图 7.48　DAC0832 的内部结构图

该芯片有 5 个控制信号:片选($\overline{CS}$),写入 1($\overline{WR_1}$),写入 2($\overline{WR_2}$),允许输入锁存(ILE)和传递控制($\overline{XFER}$)。除 ILE 是高电平有效外,其余都是低电平有效。5 个控制信号分成两组,第一组由$\overline{CS}$、ILE 和$\overline{WR_1}$组合,控制输入寄存器,实现输入数据的第一级缓冲;第二组由$\overline{XFER}$和$\overline{WR_2}$组合,控制 DAC 寄存器,实现数据的第二级缓冲。D/A 转换器产生的模拟量输出由 DAC 寄存器输出的数字量决定。具体功能实现时对控制信号的要求如表 7.22 所示。

表 7.22　DAC0832 芯片对控制信号的要求

功　能	控制条件					说　明
	$\overline{CS}$	ILE	$\overline{WR_1}$	$\overline{XFER}$	$\overline{WR_2}$	
数据 D_7~D_0锁存到输入寄存器	0	1	⊓			$\overline{WR_1}$=0 接收数据 $\overline{WR_1}$=1 锁定
数据由输入寄存器转存到 DAC 寄存器				0	⊓	$\overline{WR_2}$=0 接收数据 $\overline{WR_2}$=1 锁定
从输出端取模拟量						不受控制,随时可取

通过对控制信号输入端作不同的连接，可使 DAC0832 工作在 3 种不同工作方式。

① 双缓冲方式。输入数字量进行两级缓冲。首先在$\overline{CS}$、ILE 和$\overline{WR_1}$控制下，将输入数据锁存到输入寄存器，然后在$\overline{XFER}$和$\overline{WR_2}$控制下将输入寄存器中的数据锁存到 DAC 寄存器。当数据从输入寄存器转存到 DAC 寄存器后，在 D/A 转换器进行数模转换的同时，输入寄存器可以接收新的数据而不影响模拟量输出。

② 单缓冲方式。输入数字量只进行一级缓冲。具体实现时可令两个寄存器中的一个处于受控状态，另一个处于直通状态。例如，将$\overline{CS}$、ILE 和$\overline{WR_1}$接相应控制信号，而将$\overline{XFER}$和$\overline{WR_2}$接地，这时输入寄存器在控制信号作用下实现对输入数据的锁存，而 DAC 寄存器处在直通状态，即输出随输入变化而变化。显然，此时输入寄存器的输出直接施加到了 D/A 转换器的输入端。同样，也可令输入寄存器处在直通状态，而 DAC 寄存器处于受控状态，从而实现了对输入数据的一级缓冲。

③ 直通方式。输入数字量不进行缓冲，直接作用到 D/A 转换器上。此时可令两个寄存器均处于直通状态，即除 ILE 接高电平 1 外，其余 4 个控制信号均接低电平 0。

DAC0832 在应用方面具有较大的灵活性，图 7.49 是用 DAC0832 构成单缓冲 D/A 转换器的典型接线图。

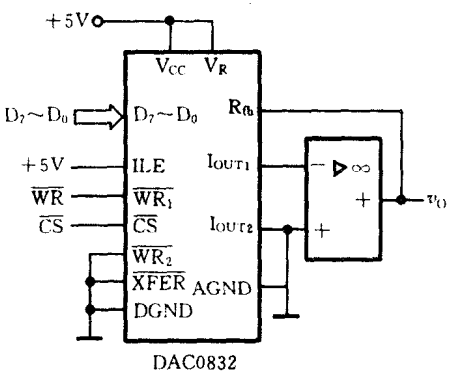


图 7.49 单缓冲方式 D/A 转换器接线图

7.3.3 集成 A/D 转换器

因为模拟信号在时间上是连续的，所以在将模拟信号转换成数字信号时，必须在选定的一系列时间点上对输入的模拟信号进行采样，然后将这些采样值转换成数字量输出。通常 A/D 转换的过程包括采样、保持和量化、编码两大步骤。

所谓采样是指周期地获取模拟信号的瞬时值，从而得到一系列时间上离散的脉冲采样值。而保持是指在两次采样之间将前一次采样值保存下来，使其在量化编码期间不发生变化。采样保持电路一般由采样模拟开关、保持电容和运算放大器等几个部分组成。

经采样保持得到的信号值依然是模拟量，而不是数字量。任何一个数字量的大小，都是以某个最小数字量单位的整数倍来表示的，用数字量表示采样保持电路输出的模拟电压也是如此，即必须把这个电压转化为最小数值单位的整数倍，这个

转化过程称为量化。所取的最小数量单位叫做量化单位，其大小等于数字量的最低有效位所代表的模拟电压大小，记作 V_{LSB}。

把量化的结果用代码（如二进制数码、BCD 码等）表示出来，称为编码。

A/D 转换过程中的量化和编码是由 A/D 转换器实现的。下面对集成 A/D 转换器的常见类型、主要参数和典型芯片予以介绍。

1. A/D 转换器的类型

A/D 转换器的类型很多，根据转换方法的不同，最常用的 A/D 转换器有如下几种类型。

(1) 并行比较型 A/D 转换器

并行比较型 A/D 转换器由电阻分压器、电压比较器、数码寄存器及编码器 4 个部分组成。由于转换是并行的，所以，这种 A/D 转换器最大的特点是转换速度快，其转换时间只受电路传输延迟时间的限制，能达到低于 20ns。其缺点是随着输出二进制位数的增加，器件数目按几何级数增加。一个 n 位的转换器，需要 2^n-1 个比较器。例如，$n=8$ 时，需要 $2^8-1=255$ 个比较器。因此，制造高分辨率的集成并行 A/D 转换器受到一定限制。显然，这种类型的 A/D 转换器适用于要求转换速度高、但分辨率较低的场合。

(2) 逐次比较型 A/D 转换器

逐次比较型 A/D 转换器是集成 ADC 芯片中使用最广泛的一种类型。它由电压比较器、逻辑控制器、D/A 转换器及数码寄存器组成。逐次比较型 A/D 转换器的特点是转换速度较快，且输出代码的位数多，精度高。

(3) 双积分型 A/D 转换器

双积分型 A/D 转换器是一种间接 A/D 转换器。其工作原理是把输入的模拟电压转换成一个与之成正比的时间宽度信号，然后在这个时间宽度里对固定频率的时钟脉冲进行计数，其结果就是正比于输入模拟信号的数字量输出。它由积分器、检零比较器、时钟控制门和计数器等几部分组成。双积分型 A/D 转换器的优点是精度高、抗干扰能力强，缺点是速度较慢。在对速度要求不高的数字化仪表中得到广泛使用。

2. A/D 转换器的主要技术参数

(1) 分辨率

分辨率是指输出数字量变化一个最小单位（最低位的变化），对应输入模拟量需要变化的量。输出位数越多，分辨率越高。通常以输出二进制码的位数表示分辨率。

(2) 相对精度

相对精度是指实际转换值偏离理想特性的误差。通常以数字量最低位所代表的模拟输入值来衡量，如相对精度不超过$\pm\frac{1}{2}$LSB。

(3) 转换时间

转换时间是指 A/D 转换器从接到转换命令起到输出稳定的数字量为止所需要的时间。它反映了 A/D 转换器的转换速度。

此外，还有输入电压范围、功率损耗等。

3. 集成 A/D 转换器典型芯片

常用的集成 A/D 转换器有 8 位、10 位、12 位、16 位等，每种又可分为不同的型号。下面以 ADC0809 为例，介绍集成 A/D 转换器的内部结构与外部特性。

(1) ADC0809 的电路结构

ADC0809 是采用 CMOS 工艺制成的逐次比较型 A/D 转换器，采用 28 引脚双列直插式封装。其分辨率为 8 位；转换时间为 100μs；相对精度为±1LSB；采用单电源供电；电源电压为+5V；功耗为 15mW。该芯片的原理框图和管脚排列图分别如图 7.50(a)和(b)所示。

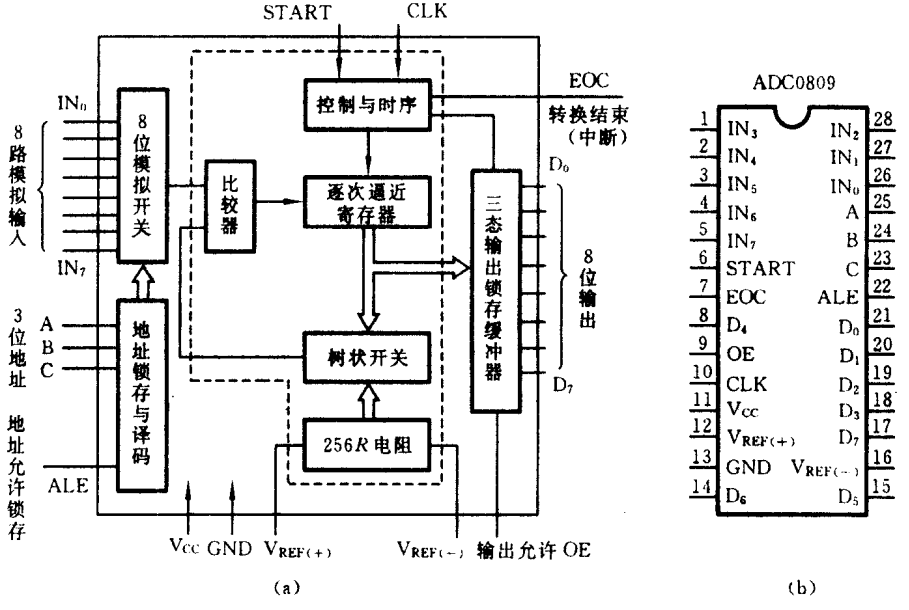


图 7.50 ADC0809 原理框图和管脚排列图

由图 7.50(a)可知，该芯片内部由 8 位模拟开关、地址锁存与译码器、比较器、电阻网络、树状电子开关、逐次逼近寄存器、控制与时序电路、三态输出锁存器等组成。虚线框中为芯片核心部分。各部分功能大致如下。

地址锁存与译码器控制 8 位模拟开关，实现对 8 路模拟信号的选择。8 个模拟输入端可接收 8 路模拟信号，但某一时刻只能选择其中的一路进行转换。树状开关与 256R 电阻网络一起构成 D/A 转换电路，产生与逐次逼近寄存器中二进制数字量对应的反馈模拟电压，送至比较器，与输入模拟电压进行比较。比较器的输出结果和控制与时序电路的输出一起控制逐次逼近寄存器中的数据从高位至低位变化，依次确定各位的值，直至最低位被确定为止。在转换完成后，转换结果送到三态输出缓冲器。当输出允许信号 OE 有效时，选通输出缓冲器，输出转换结果。

(2) 引脚功能

ADC0809 共有 28 个引脚，各引脚功能如下。

$IN_0 \sim IN_7$：8 路模拟电压输入端。

A，B，C：模拟输入通道的地址选择线。当 CBA＝000 时，选中 IN_0；CBA＝001 时，选中 IN_1……依此类推，当 CBA＝111 时，选中 IN_7。

ALE：地址锁存允许信号输入端。该端接高电平时有效，仅当该信号有效时，才能将地址信号锁存，经译码后选中一个通道。

START：启动转换脉冲输入端。该端所加信号的上升沿将所有内部寄存器清 0，下降沿开始进行模数转换。

CLK：时钟脉冲输入端。

$D_7 \sim D_0$：数据输出端，D_7 为高位。

OE：输出允许端，高电平有效。该端为高电平时，打开三态输出缓冲器，输出转换结果。

$V_{REF(+)}$ 和 $V_{REF(-)}$：参考电压正端和负端。

ADC0809 可直接与微机系统相连接。

有关 A/D 转换器和 D/A 转换器的应用，将在“微机接口技术”课程中作详细介绍。

习　题　七

7.1　用 4 位二进制并行加法器设计一个实现 8421 码对 9 求补的逻辑电路。

7.2　用两个 4 位二进制并行加法器实现 2 位十进制数 8421 码到二进制码的转换。

7.3 用 4 位二进制并行加法器设计一个用 8421 码表示的 1 位十进制加法器。

7.4 用一片 3-8 线译码器和必要的逻辑门实现下列逻辑函数表达式。

$$F_1 = \overline{A}\,\overline{C} + AB\overline{C}$$

$$F_2 = \overline{A} + B$$

$$F_3 = AB + \overline{A}\,\overline{B}$$

7.5 用一片 4-16 线译码器和适当的逻辑门设计一个 1 位十进制数 2421 码的奇偶位产生电路(假定采用奇检验)。

7.6 当优先编码器 74LS148 的 I_S 接 0,输入 $I_7I_6I_5I_4I_3I_2I_1I_0=11010001$ 时,输出状态为何值?

7.7 试用 4 路数据选择器实现余 3 码到 8421 码的转换。

7.8 当 4 路选择器的选择控制变量 A_1、A_0 接变量 A、B,数据输入端 D_0、D_1、D_2、D_3 依次接 $\overline{C}$、0、0、C 时,电路实现何功能?

7.9 用 4 位二进制同步可逆计数器 74193 和必要的逻辑门实现模 12 加法计数器。

7.10 用 74194 双向移位寄存器和必要的逻辑门设计一个 00011101 序列信号发生器。

7.11 在图 7.38(a)所示电路中,若取 $R_1=2R_2$,请问输出矩形波的占空比为多少?

7.12 分析图 7.51 所示由定时器 5G555 构成的多谐振荡器。

(1) 计算其振荡周期;

(2) 若要产生占空比为 50% 的方波,R_1 和 R_2 的取值关系如何?

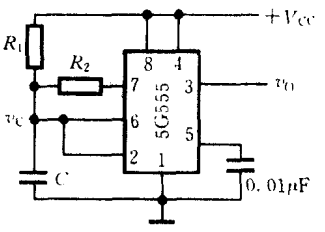


图 7.51 多谐振荡器

7.13 将 5G555 定时器按图 7.52(a)所示连接,输入波形如图 7.52(b)所示。请画出定时器输出波形,并说明该电路相当于什么器件。

7.14 D/A 转换器有哪些主要参数?通常用什么参数来衡量转换精度?

7.15 DAC1210 是 12 位 D/A 芯片,请问其分辨率为多少?(用百分数表示)

7.16 DAC0832 由哪几部分组成?可以构成哪几种工作方式?每种方式如何控制?

7.17 常见集成 A/D 转换器按转换方法的不同可分成哪几种类型?各有何特点?

7.18 ADC0809 如何实现对 8 路模拟量输入的选择?当它与微机连接时是否要外加三态缓冲器?

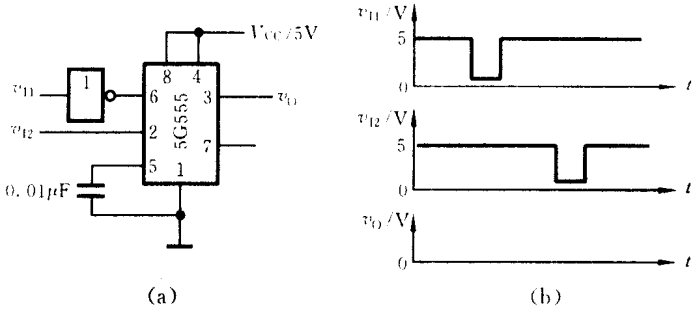


图 7.52　5G555 连线图和有关波形图



可编程逻辑器件

目前，在数字系统设计中广泛使用的可编程逻辑器件(Programmable Logic Device，简称 PLD)属于 LSI 中的半用户定制电路。由于 PLD 具有性能优越、设计简单、功能变更灵活等特点，因而在不同应用领域中受到广泛重视，是构成数字系统的理想器件。

本章将对几种常用的可编程逻辑器件进行介绍。

8.1 PLD 概述

8.1.1 PLD 的发展

PLD 是 20 世纪 70 年代开始发展起来的一种新型大规模集成电路。一片 PLD 所容纳的逻辑门可达数百、数千甚至更多，其逻辑功能可由用户编程指定。特别适宜于构造小批量生产的系统，或在系统开发研制过程中使用。

20 世纪 70 年代初期推出的第一种 PLD 是可编程只读存储器(PROM)。PROM 由一个与阵列和一个或阵列组成，与阵列是固定的，或阵列是可编程的。20 世纪 70 年代中期出现了可编程逻辑阵列(PLA)，PLA 同样由一个与阵列和一个或阵列组成，但其与阵列和或阵列都是可编程的。

20 世纪 70 年代末期，出现了可编程阵列逻辑(PAL)。PAL 器件的与阵列是可编程的，而或阵列是固定的，它有多种输出和反馈结构，因而给逻辑设计带来了很大的灵活性。但 PAL 器件一般采用熔丝工艺，一旦编程后便不能改写。此外，型号太多，设计者要根据需要选择不同输出结构的器件，这些无疑给用户带来了不便。

20 世纪 80 年代中期，通用阵列逻辑(GAL)器件问世。GAL 器件是在 PAL 器件基础上发展起来的新一代器件。它采用了高速电可擦 CMOS 工艺，可以反复擦除和改写。特别是在结构上采用了“输出逻辑宏单元”电路，使一种型号的 GAL

器件可以对几十种 PAL 器件做到全兼容。因而，给逻辑设计带来了更大的灵活性。

20 世纪 90 年代，又产生了在系统编程(ISP)器件。在系统编程是指用户具有在自己设计的目标系统中或线路板上为重构逻辑而对逻辑器件进行编程或反复改写的能力。ISP 器件为用户提供了传统的 PLD 技术无法达到的灵活性，带来了极大的时间效益和经济效益，使可编程逻辑技术产生了实质性飞跃。

PLD 的发展和应用，不仅简化了数字系统设计过程、降低了系统的体积和成本、提高了系统的可靠性和保密性，而且使用户从被动地选用厂商提供的通用芯片发展到主动地投入到对芯片的设计和使用，从根本上改变了系统设计方法，使各种逻辑功能的实现变得灵活、方便。

8.1.2　PLD 的基本结构

PLD 的基本结构如图 8.1 所示，它由一个与阵列和一个或阵列组成，每个输出都是输入的与或函数。阵列中输入线和输出线通过逻辑元件相连接。这些元件是接通还是断开，可由厂家根据器件的结构特征决定或由用户根据要求编程决定。PLD 与阵列的输入为外部输入原变量及在阵列中经过反相后的反变量。它们按一定的规律连接到各个与门的输入端，并在各与门的输出端产生某些输入变量的与项作为或阵列的输入，这些与项按一定的要求连接到相应或门的输入端，在每个或门的输出端产生输入变量的与或函数表达式。

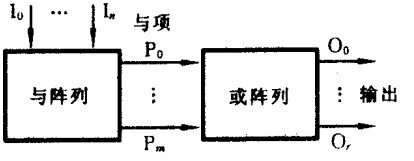


图 8.1　PLD 的基本结构

各种不同的 PLD 是在上述基本结构的基础上，附加一些其他逻辑元件，如输入缓冲器、输出寄存器、内部反馈、输出宏单元等而构成的。

8.1.3　PLD 的电路表示法

对于 PLD 器件，用逻辑电路的一般表示法很难描述其内部电路，这给 PLD 的生产和应用带来诸多不便。为了在芯片的内部配置和逻辑图之间建立一一对应关系，把逻辑图和真值表结合在一起构成一种紧凑而易于识读的形式，对描述 PLD

基本结构的有关逻辑符号和规则作出了某些约定。现对这些约定作简单介绍。

组成 PLD 的基本器件是与门和或门。图 8.2 给出了 3 输入与门的两种表示法。传统表示法(图 8.2(a))中与门的 3 个输入 A、B、C 在 PLD 表示法(图 8.2(b))中称为 3 个输入项,而输出 D 称为与项。同样,或门也采用类似方法表示。

图 8.3 所示为 PLD 的典型输入缓冲器。它的两个输出 B、C 是其输入 A 的原和反,如图中真值表所示,$B=A$,$C=\overline{A}$。

图 8.4(a)给出了 PLD 阵列交叉点上的 3 种连接方式。实点"·"表示硬线连接,也就是固定连接;"×"表示可编程连接;没有"×"也没有"·"的表示两线不连接。如图 8.4(b)中所示的输出 $F=A\cdot C$。

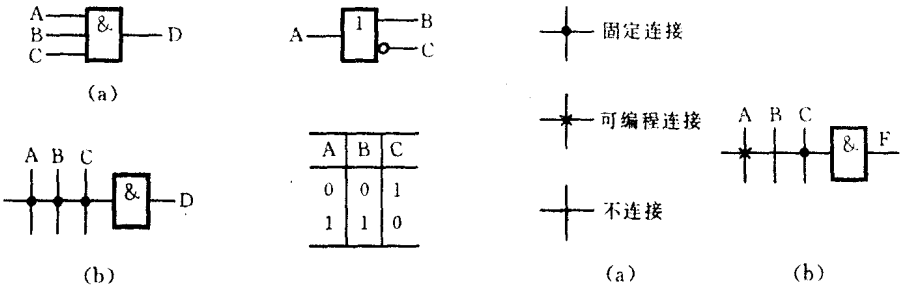
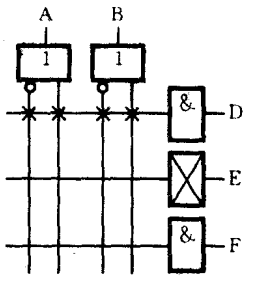


图 8.2 与门表示法　图 8.3 PLD 输入缓冲器　图 8.4 PLD 连接方式的表示

图 8.5 列出了与门不执行任何功能的连接表示法。图中,输出为 D 的与门连接了所有的输入项,其输出方程为

$$D=A\cdot\overline{A}\cdot B\cdot\overline{B}=0$$

它表示输入缓冲器的互补输出全部连到同一与项时,该与项的输出总为逻辑"0",这种状态称为与门的缺省(Default)状态。为了方便起见,用标有"×"标记的与门输出来表示所有输入缓冲器输出全部连到某一与项的情况,如图中输出 E。相反,图中输出 F 表示无任何输入项与其相连,因此,该与项总是处于"浮动"的逻辑"1"。



A	B	D	E	F
0	0	0	0	1
0	1	0	0	1
1	0	0	0	1
1	1	0	0	1

图 8.5 PLD 与门的缺省状态表示

8.2　常用的可编程逻辑器件

根据 PLD 中阵列和输出结构的不同，目前常用的 PLD 有 4 种主要类型：

① 可编程只读存储器 PROM；

② 可编程逻辑阵列 PLA；

③ 可编程阵列逻辑 PAL；

④ 通用阵列逻辑 GAL。

下面分别介绍这 4 种器件，重点讨论其逻辑结构以及在逻辑设计中的应用。

8.2.1　可编程只读存储器

1. 半导体存储器的分类

存储器(Memory)是数字计算机和其他数字系统中存放信息的重要部件。随着大规模集成电路的发展，半导体存储器因其具有集成度高、速度快、功耗小、价格低等优点而被广泛应用于各种数字系统中。

半导体存储器按功能可分为随机存取存储器 RAM(Random Access Memory)和只读存储器 ROM(Read Only Memory)两大类。

(1) 随机存取存储器 RAM

RAM 是一种既可读又可写的存储器，故又称为读/写存储器。

根据制造工艺的不同，RAM 又可分为双极型和 MOS 型两种类型。双极型 RAM 工作速度高，但成本高、功耗大、集成度低，主要用作高速小容量存储器。MOS 型 RAM 功耗小、集成度高、成本低，但速度比双极型 RAM 低。MOS 型 RAM 又可进一步分为静态 RAM(SRAM)和动态 RAM(DRAM)两种，相比之下 DRAM 的集成度更高。MOS 型 RAM 适宜于构造大容量存储器。

RAM 的优点是读/写方便，使用灵活；缺点是一旦断电，所存储的信息便会丢失，它属于易失性存储器。

(2) 只读存储器 ROM

只读存储器 ROM 是一种在正常工作时只能读出、不能写入的存储器。通常用来存放那些固定不变的信息。工作时，将一个给定的地址码加到 ROM 的地址码输入端，便可在它的输出端得到一个事先存入的固定数据。若把地址码视为自变量，输出数据视为因变量，则 ROM 相当于一个组合逻辑电路。

只读存储器存入数据的过程通常称为编程。根据编程方法的不同，可分为掩膜编程 ROM 和用户可编程 ROM 两类。掩膜编程 ROM 中存放的内容是由生产厂家在芯片制造时利用掩膜技术写入的。一旦制成，内容便已固定，用户无法进行改变，通常又称为固定 ROM。这类 ROM 的优点是可靠性高，集成度高，批量生产时价格便宜；缺点是用户不能重写或改写，使用不灵活。可编程 ROM 中存放的内容是由用户根据自己的需要在编程设备上写入的。最大的优点是使用灵活方便，特别适宜于用来实现各种逻辑功能，属于常用的可编程逻辑器件。

只读存储器 ROM 属于非易失性存储器，即使切断电源，ROM 中存放的信息也不会丢失，因而在数字系统中获得广泛应用。

RAM 和 ROM 是计算机和其他数字系统中不可缺少的重要组成部分，通常用来存放各种程序和数据。有关它们的内部电路及工作原理在计算机组成原理、微机原理等有关书籍中将进行详细讨论。下面仅从逻辑设计的角度出发，对可编程 ROM 的结构、类型及应用予以介绍。

2. 可编程 ROM 的结构与类型

(1) 可编程 ROM 的逻辑结构

可编程 ROM 的结构框图如图 8.6(a)所示，它主要由地址译码器和存储体两大部分组成。图中，$A_0 \sim A_{n-1}$ 为地址输入线；$W_0 \sim W_{2^n-1}$ 为地址译码输出线，一般称为字线；$D_0 \sim D_{m-1}$ 为数据输出线，一般称为位线。地址译码器根据输入地址码译出相应字线，使之有选择地去驱动相应存储单元，并通过输出端 $D_0 \sim D_{m-1}$ 读出该单元中存放的 m 位代码。通常，一个 n 位地址输入和 m 位数据输出的可编程 ROM，其存储容量表示为 $2^n \times m$(位)，意味着存储体中有 $2^n \times m$ 个存储元，每个存储元的状态代表一位二进制代码。图 8.6(b)给出了存储体的结构示意图。

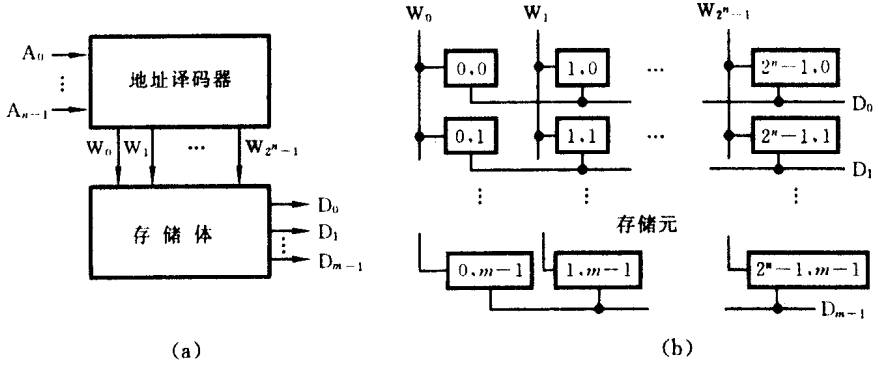


图 8.6 可编程 ROM 的结构框图和存储体的结构示意图

从逻辑器件的角度理解，可编程 ROM 的基本结构是由一个固定连接的与阵列和一个可编程连接的或阵列所组成的组合电路。图 8.7(a)给出了一个 8×3(8 与门×3 或门)可编程 ROM 的逻辑结构。图中上半部分的与阵列构成一个 3 变量全译码器，下半部分的或阵列是由 3 个或门组成的一个或门网络。8 个与门用来产生 3 变量的 8 个最小项，3 个或门用来将相应的最小项相“或”构成 3 个指定的逻辑函数。

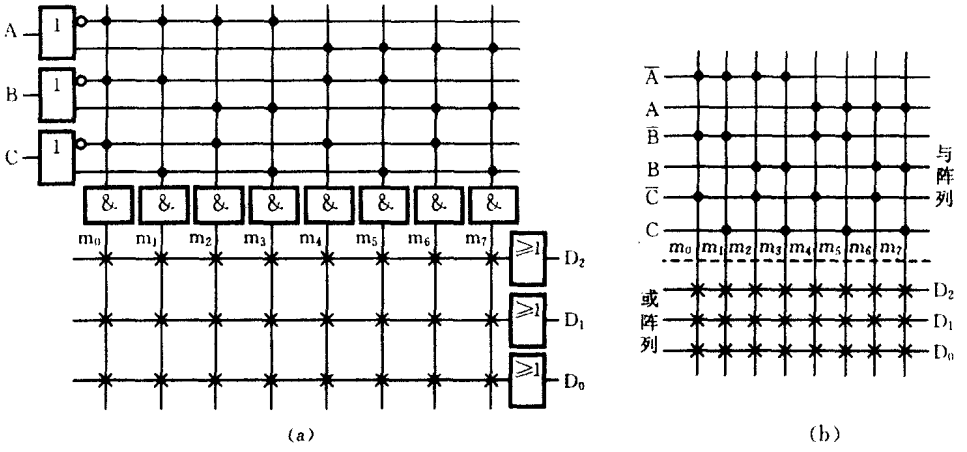


图 8.7　8×3 可编程 ROM 的逻辑结构和阵列逻辑图

为了可编程 ROM 设计的方便，常将图 8.7(a)所示的逻辑结构图进一步简化为图 8.7(b)所示的形式，该形式的图称为阵列逻辑图，简称阵列图。画阵列图时，将可编程 ROM 中的每个与门和或门都简化成一根线。图 8.7(b)中虚线上面 6 根水平线分别表示输入线 $\overline{A}$、A、$\overline{B}$、B、$\overline{C}$ 和 C。与阵列的 8 根垂直线代表 8 个与门，或阵列中标有 D_2、D_1、D_0的 3 根水平线表示 3 个或门。

(2) 可编程 ROM 的类型

根据存储体中存储元电路构造的不同，可编程 ROM 有不同的类型。常用的有如下 3 种。

1) 一次编程的 ROM(PROM)

PROM 产品在出厂时，所有存储元均被加工成同一状态“0”(或“1”)，用户可根据需要通过编程将某些存储元的状态改变成另一状态“1”(或“0”)。但这种编程只能进行一次，一旦编程完毕，其内容便不能再改变。

例如，双极型 PROM 有两种常用的电路结构，一种是熔丝烧断型 PROM，另一种是 PN 结击穿型 PROM。熔丝型 PROM 在每个存储元位置都制作了一个带熔丝的三极管，三极管的射极通过一根易熔金属丝接到相应的位线上，如图8.8(a)

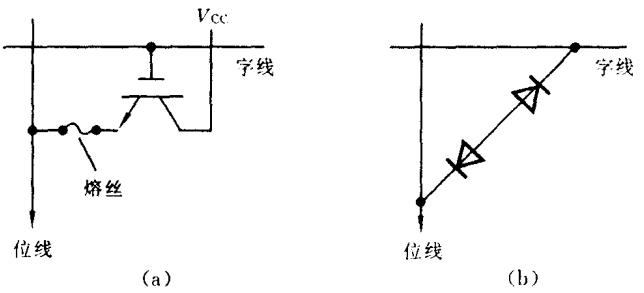


图 8.8 熔丝型和击穿型 PROM 存储元电路

所示。用户对 PROM 编程是逐字逐位进行的。根据需要写入的信息，按字线和位线选择某个存储元管子，通过施加规定宽度和幅度的脉冲电流，将该三极管的熔丝熔断，使该存储元的状态被改变成与原状态相反的状态。熔丝一旦熔断，便不可再恢复，因此，编程是一次性的。

结击穿型 PROM 的所有存储元在出厂时的原始状态均为一对背靠背连接的二极管，如图 8.8(b)所示。正常工作时，处于原始状态的存储元二极管不导通。用户在编程时，可根据需要对选中的存储元施加足够大的电流，将其反向二极管击穿，仅剩下一个正向连接的二极管，从而使其正常工作时处于导通状态。由二极管的导通与截止标志存储元中的信息是"1"或"0"。显然，这种编程同样是一次性的。

PROM 与固定 ROM 相比，增加了使用的灵活性。但因其可靠性较差，加之只能一次性编程，故目前很少使用。

2) 可抹可编程 ROM(EPROM)

EPROM 不仅可由用户编程存放指定的信息，而且可由用户通过专用的紫外线灯照射芯片上的受光窗口，将原存储内容抹去，再写入新的内容。这一特性是由 EPROM 中存储元的电路结构决定的。

EPROM 的存储元采用浮栅雪崩注入 MOS 电路，简称 FAMOS 管。FAMOS 管的栅极全部被二氧化硅绝缘层包着，没有引出线，如悬浮状，所以称作"浮栅"。FAMOS 有 P 沟道和 N 沟道两种结构，P 沟道 FAMOS 管的结构示意图和符号分别如图 8.9(a)和(b)所示。图 8.9(c)所示的是用 P 沟道 FAMOS 作存储元的电路，一个存储元由一个普通 MOS 管和一个 FAMOS 管串接而成。产品在出厂时，所有 FAMOS 的浮栅均不带电，FAMOS 管处于截止状态。写入信息时，在对应单元的漏极加上足够高的负压，使漏极与衬底之间的 PN 结击穿，雪崩击穿产生的高能电子穿过 SiO_2 层堆积在浮置栅上，使 FAMOS 管导通。对应 FAMOS 管的截止或导通，可使位线上输出的电平为高或低，以表示该存储元存放的信息是"1"或"0"。由于浮栅被绝缘的二氧化硅包着，编程时堆积的电子没有放电回路，故电荷

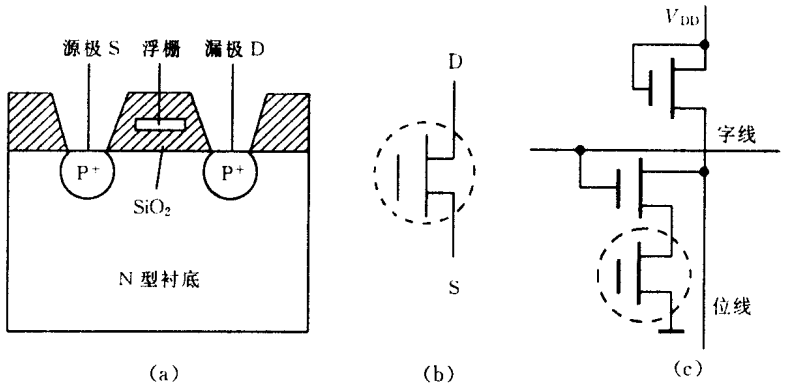


图 8.9　EPROM 存储元管结构、符号和电路

不会消失，信息能够长期保存。如果用紫外线照射 FAMOS 管，则浮栅上积累的电子将形成光电流而泄放，从而导电沟道消失，管子又恢复截止状态。为了使编程后能进行擦除和重写，在芯片的封装外壳装有透明的石英窗口。对编程好的 EPROM 要用不透光的胶纸将受光窗口封住，以免信息丢失。

3）电可抹可编程 ROM(EEPROM)

EPROM 虽然具有可反复编程的优点而被广泛使用，但 EPROM 只能整体擦除，不能一个存储单元一个存储单元地独立擦除，而且擦除操作比较麻烦。而 EEPROM 克服了 EPROM 的这一不足。

EEPROM 的结构与 EPROM 相似，但 EEPROM 在浮栅上增加了一个遂道二极管，编程时可使电荷通过它流向浮栅，而擦除时可使电荷通过它流走，即擦除和编程均用电完成。这种器件不仅工作电流小、擦除速度快，而且允许改写的次数大大高于 EPROM，一般允许改写次数可达 1000 次左右。

目前，EPROM 和 EEPROM 的应用均十分广泛。

3. PROM 应用举例

由于 PROM 是由一个固定连接的与阵列和一个可编程连接的或阵列组成，所以，用户只要改变或阵列上连接点的数量和位置，就可以在输出端排列出输入变量的任何一种最小项的组合，实现不同的逻辑函数。因此，采用 PROM 进行逻辑设计时，只需首先根据逻辑要求列出真值表，然后把真值表的输入作为 PROM 的输入，把要实现的逻辑函数用对 PROM 或阵列进行编程的代码来代替，画出相应的阵列图。

例 8.1　用 PROM 设计一个代码转换电路，将 4 位二进制码转换为 Gray 码。

解 设4位二进制码为B_3、B_2、B_1、B_0，4位Gray码为G_3、G_2、G_1、G_0，其对应关系如表8.1所示。将4位二进制码作为PROM的输入，Gray码作为PROM的输出，可选容量为$2^4\times4$的PROM实现给定功能。根据表8.1，可直接画出该转换电路的PROM阵列图，如图8.10所示。

表8.1 4位二进制码与Gray码的转换真值表

二进制码				Gray码				二进制码				Gray码			
B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0	B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0
0	0	0	0	0	0	0	0	1	0	0	0	1	1	0	0
0	0	0	1	0	0	0	1	1	0	0	1	1	1	0	1
0	0	1	0	0	0	1	1	1	0	1	0	1	1	1	1
0	0	1	1	0	0	1	0	1	0	1	1	1	1	1	0
0	1	0	0	0	1	1	0	1	1	0	0	1	0	1	0
0	1	0	1	0	1	1	1	1	1	0	1	1	0	1	1
0	1	1	0	0	1	0	1	1	1	1	0	1	0	0	1
0	1	1	1	0	1	0	0	1	1	1	1	1	0	0	0

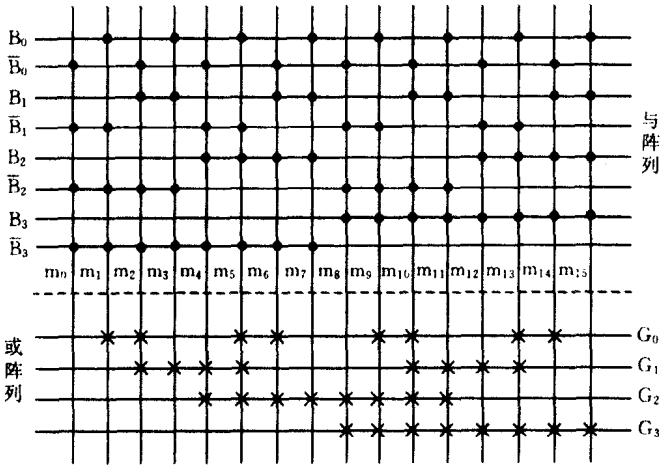


图8.10 PROM阵列图

从图8.10可以看出，PROM的与阵列产生了输入变量($B_3\sim B_0$)的全部最小项($m_0\sim m_{15}$)。PROM的或阵列根据真值表进行编程，实现有关最小项的或运算，产生4个输出函数($G_3\sim G_0$)。因此，用PROM实现逻辑函数时，主要是或阵列的编程问题。用阵列图表示时，就是交叉点是否连接的问题，图中标"×"处代表"1"，否则代表"0"。

例8.2 用PROM设计一个Π发生器，其输入为4位二进制码，输出为8421

码。该电路串行地产生常数 π，若取小数点后 15 位数字，则 π = 3.141592653589793，其逻辑框图如图 8.11 所示。

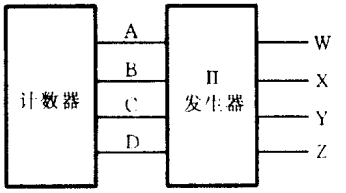


图 8.11　逻辑框图

解　根据题意，可用一个 4 位同步计数器控制 PROM 的地址输入端，使其地址码按 4 位二进制码递增的顺序进行周期性地变化，以便对所有存储单元逐个进行访问。Π 发生器的输入为与同步计数器输出状态对应的 4 位二进制码，而输出则为 8421 码。输入/输出的关系可用表 8.2 所示的真值表来表示。

根据表 8.2 可画出 Π 发生器的 PROM 阵列图，如图 8.12 所示。

表 8.2　Π 发生器的真值表

输入				输出				π
A	B	C	D	W	X	Y	Z	
0	0	0	0	0	0	1	1	3
0	0	0	1	0	0	0	1	1
0	0	1	0	0	1	0	0	4
0	0	1	1	0	0	0	1	1
0	1	0	0	0	1	0	1	5
0	1	0	1	1	0	0	1	9
0	1	1	0	0	0	1	0	2
0	1	1	1	0	1	1	0	6
1	0	0	0	0	1	0	1	5
1	0	0	1	0	0	1	1	3
1	0	1	0	0	1	0	1	5
1	0	1	1	1	0	0	0	8
1	1	0	0	1	0	0	1	9
1	1	0	1	0	1	1	1	7
1	1	1	0	1	0	0	1	9
1	1	1	1	0	0	1	1	3

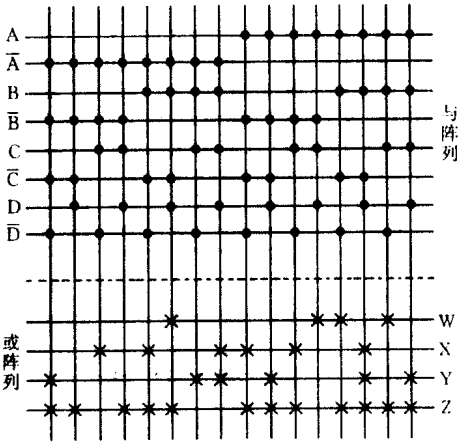


图 8.12　PROM 阵列图

8.2.2　可编程逻辑阵列 PLA

从前面的介绍可知，PROM 的地址译码器采用全译码方式，n 个地址码可选中 2^n 个不同的存储单元；而且，地址码与存储单元有一一对应的关系。因此，即使有多个存储单元所存放的内容完全相同也必须重复存放，无法节省这些单元。从实现逻辑函数的角度看，PROM 的与阵列固定地产生 n 个输入变量的全部最小项。而对于大多数逻辑函数而言，并不需要使用全部最小项，有许多最小项是无用的，

尤其对于包含约束条件的逻辑函数，许多最小项是不可能出现的。因此，PROM的与阵列未能获得充分利用而造成硬件浪费，使得芯片面积的利用率不高。为了解决此问题，在PROM的基础上出现了一种与阵列和或阵列均可编程的逻辑器件，即可编程逻辑阵列PLA(Programmable Logic Array)。

1. PLA的逻辑结构

PLA的逻辑结构与PROM类似，也是由一个与阵列和一个或阵列构成。所不同的是，它的与阵列和或阵列一样是可编程的。而且，n个输入变量的与阵列不再是产生2^n个与项，而是有p个与门就提供p个与项，每个与项与哪些变量相关可由编程决定。或阵列通过编程可选择需要的与项相或、形成与-或函数式。由PLA实现的与-或函数式一般是最简与-或表达式。图8.13(a)给出了一个具有3个输入变量、可提供6个与项、产生3个输出函数的PLA逻辑结构图。其相应阵列图如图8.13(b)所示。

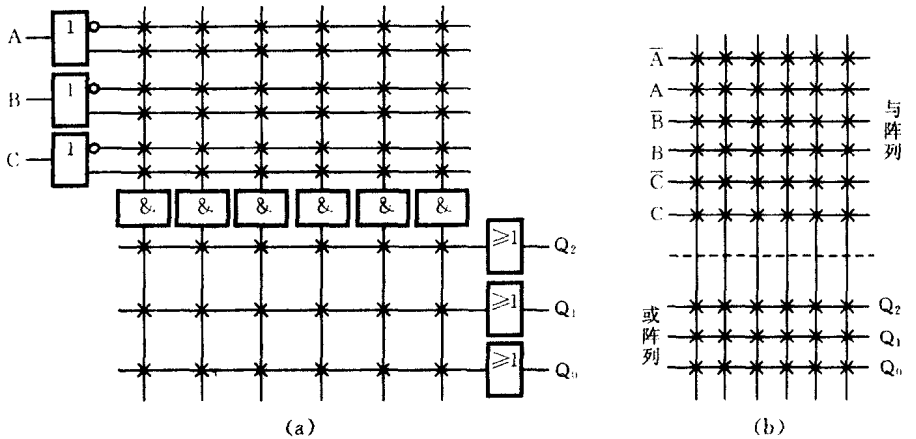


图8.13 PLA逻辑结构图和阵列图

PLA的存储容量不仅与输入变量个数和输出端个数有关，而且还和它的与项数(即与门数)有关，其存储容量用输入变量数(n)、与项数(p)、输出端数(m)来表示。如图8.13所示的PLA的容量为3—6—3。目前常见的PLA器件的容量有16—48—8和14—96—8等。

2. PLA应用举例

采用PLA进行逻辑设计，可以十分有效地实现各种逻辑功能。相对PROM而言，PLA结构更简单、使用更灵活。用PLA设计组合逻辑电路时，一般首先将

给定问题的逻辑函数按多输出逻辑函数的化简方法简化成最简与-或表达式，然后，根据最简表达式中的不同与项以及各函数式的与项之和分别构成与阵列和或阵列，并画出阵列逻辑图。

例 8.3　用 PLA 设计一个代码转换电路，将一位十进制数的 8421 码转换成余 3 码。

解　设 A、B、C、D 表示 8421 码的各位，W、X、Y、Z 表示余 3 码的各位，可列出转换电路的真值表，如表 8.3 所示。

表 8.3　8421 码—余 3 码转换电路真值表

A	B	C	D	W	X	Y	Z	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	1	1	1	0	0	0	1	0	1	1
0	0	0	1	0	1	0	0	1	0	0	1	1	1	0	0
0	0	1	0	0	1	0	1	1	0	1	0	d	d	d	d
0	0	1	1	0	1	1	0	1	0	1	1	d	d	d	d
0	1	0	0	0	1	1	1	1	1	0	0	d	d	d	d
0	1	0	1	1	0	0	0	1	1	0	1	d	d	d	d
0	1	1	0	1	0	0	1	1	1	1	0	d	d	d	d
0	1	1	1	1	0	1	0	1	1	1	1	d	d	d	d

根据表 8.3 写出函数表达式，并按照多输出函数化简法则用卡诺图进行化简，得到最简与-或表达式如下：

$$W = A + BC + BD$$

$$X = \bar{B}C + \bar{B}D + B\bar{C}\bar{D}$$

$$Y = CD + \bar{C}\bar{D}$$

$$Z = \bar{D}$$

由此可见，全部输出函数只包含 9 个不同与项，所以，该代码转换电路可用一个容量为 4—9—4 的 PLA 实现，其阵列图如图 8.14 所示。

PLA 不仅可以用来实现任意组合逻辑电路的功能，而且可以实现各种时序逻辑电路的功能，实现后种功能的 PLA 称之为“时序 PLA”。由于时序逻辑电路的输出不仅取决于当时的输入，而且取决于反映过去输入的内部状态；所以时序 PLA 与组合 PLA 的区别在于：除了具有与阵列和或阵列之外，还具有一个用于存储以前状态的触发器网络。这个网络中包含若干触发器，它的输入接受或阵列输出及时钟脉冲、复位信号的控制，其输出反馈到与阵列，用来和现有输入一起产生与项输出。时序 PLA 的结构框图如图 8.15 所示。图中，与阵列的输入有两部分，即外部输入变量 $x_1, x_2, \cdots, x_n$ 和状态变量 $y_1, y_2, \cdots, y_r$；或阵列的输出也有两部分，即激励状态 $Y_1, Y_2, \cdots, Y_r$ 和外部输出 $Z_1, Z_2, \cdots, Z_m$。

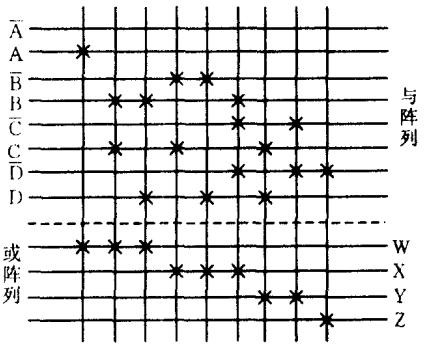


图 8.14 阵列图

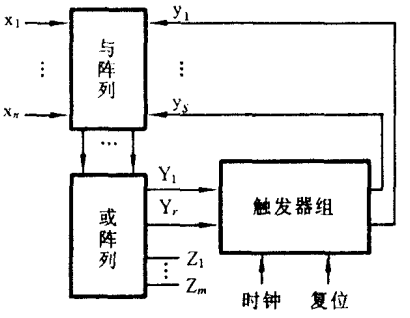


图 8.15 时序 PLA 结构图

例 8.4 用 PLA 设计一 8421 码加 1 计数器，并用七段显示器显示计数状态。

解 该设计包含两部分，一是按照同步时序电路的设计方法设计一个 8421 码加 1 计数器；另一是按照组合电路设计方法设计一个将 8421 码转换成七段显示码的代码转换电路。首先，作出 8421 码加 1 计数器的状态表，如表 8.4 所示。

表 8.4 8421 码加 1 计数器的状态表

Q_4	Q_3	Q_2	Q_1	Q_4^{n+1}	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0

表 8.5 8421 码转换为七段显示码的真值表

8421 码				七段显示码						
Q_4	Q_3	Q_2	Q_1	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	0	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	1	1

假定采用 J-K 触发器作为存储元件，根据表 8.4 和 J-K 触发器的激励表，可得到触发器的激励函数表达式：

$$J_4 = Q_3Q_2Q_1 \qquad K_4 = Q_1$$

$$J_3 = K_3 = Q_2Q_1$$

$$J_2 = K_2 = \overline{Q}_4Q_1 \qquad J_1 = K_1 = 1$$

激励函数共包含 4 个不同与项：$Q_3Q_2Q_1$，Q_1，Q_2Q_1，$\overline{Q}_4Q_1$。

8421 码转换为七段显示码的真值表如表 8.5 所示。

根据表 8.5 作出 a、b、c、d、e、f、g 的卡诺图，按多输出函数进行化简后，可得到代码转换电路的输出函数最简与-或式：

$$a = Q_4 + Q_2Q_1 + \overline{Q}_3\overline{Q}_1 + Q_3\overline{Q}_2Q_1$$

$$b = \overline{Q}_3 + Q_2Q_1 + \overline{Q}_2\overline{Q}_1$$

$$c = Q_1 + \overline{Q}_2\overline{Q}_1 + Q_3\overline{Q}_1$$

$$d = \overline{Q}_3\overline{Q}_1 + Q_3\overline{Q}_2Q_1 + Q_2\overline{Q}_1 + \overline{Q}_3Q_2$$

$$e = \overline{Q}_3\overline{Q}_1 + Q_2\overline{Q}_1$$

$$f = Q_4 + Q_3\overline{Q}_1 + \overline{Q}_2\overline{Q}_1 + Q_3\overline{Q}_2Q_1$$

$$g = Q_4 + Q_3\overline{Q}_1 + \overline{Q}_3Q_2 + Q_3\overline{Q}_2Q_1$$

上述表达式在触发器激励函数表达式的基础上增加了 8 个不同的与项：Q_4，$\overline{Q}_3\overline{Q}_1$，$Q_3\overline{Q}_2Q_1$，$\overline{Q}_3$，$\overline{Q}_2\overline{Q}_1$，$Q_3\overline{Q}_1$，$Q_2\overline{Q}_1$，$\overline{Q}_3Q_2$。

根据激励函数和输出函数表达式，可画出用 PLA 实现给定功能的阵列逻辑图，如图 8.16 所示。

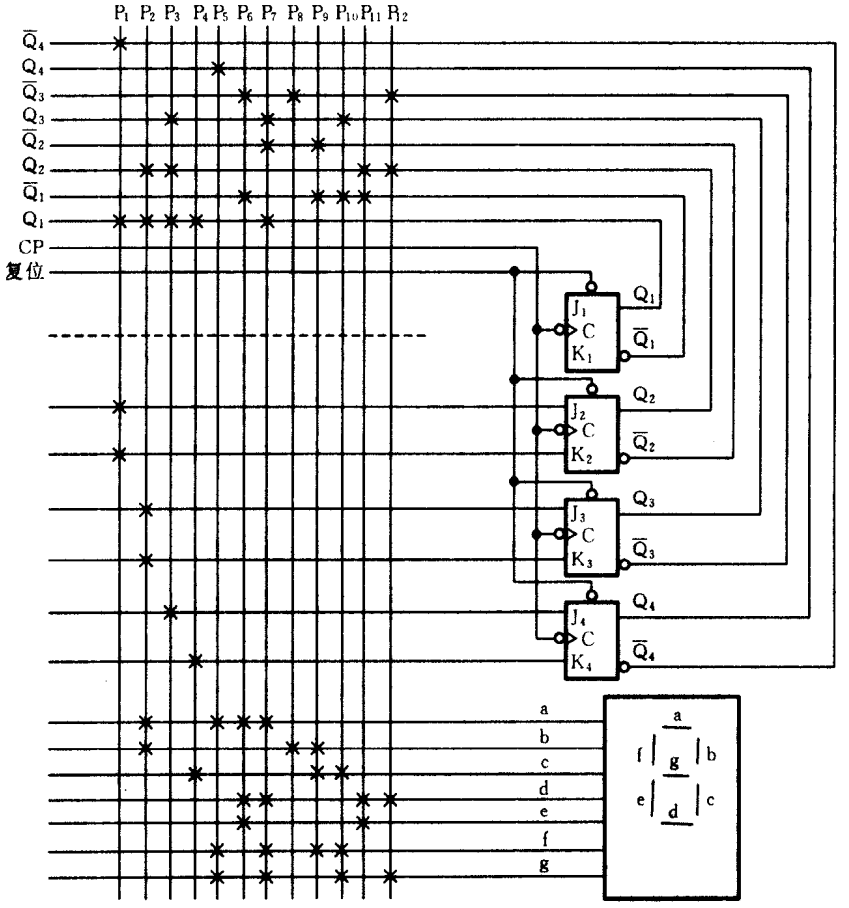


图 8.16　阵列逻辑图

*8.2.3 可编程阵列逻辑 PAL

PAL(Programmable Array Logic)是在 PROM 和 PLA 的基础上发展起来的一种可编程逻辑器件。它相对于 PROM 而言，使用更灵活，且易于完成多种逻辑功能，同时又比 PLA 工艺简单，易于编程和实现。

1. PAL 的逻辑结构

PAL 由一个可编程的与阵列和一个固定连接的或阵列组成。图 8.17(a)给出了一个 3 输入 3 输出 PAL 的逻辑结构图，通常将其表示成图 8.17(b)所示的形式。

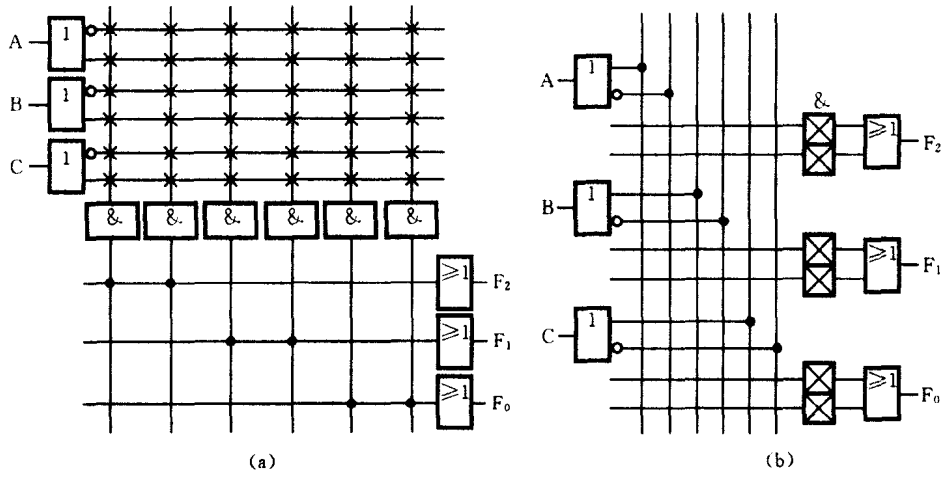


图 8.17 PAL 逻辑结构图的两种表示形式

PAL 每个输出包含的与项数目是由固定连接的或阵列提供的。在典型逻辑设计中，一般函数约包含 3～4 个与项，而现有 PAL 器件可为每个输出提供 8 个左右与项，因此，使用这种器件能很好地完成各种常用电路的设计。

PAL 器件的结构(包括输入数、输出数以及与项数)是由生产厂家固定的。从 PAL 问世至今，大约已生产出几十种不同的产品，按其输出和反馈结构，大致可将其分为 5 种基本类型。

(1) 专用输出的基本门阵列结构

图 8.18 表示这种结构类型的 1 个输入、1 个输出、4 个与项的局部电路。图中，输出部分采用或非门，为低电平有效器件。若输出部分采用或门结构，则为高电平有效器件。有的器件输出部分采用互补输出，则称为互补输出器件。这种结

构类型适用于实现组合逻辑函数。常见产品有 PAL10H8(10 个输入,8 个输出,输出高电平有效),PAL12L6(12 个输入,6 个输出,输出低电平有效)等。

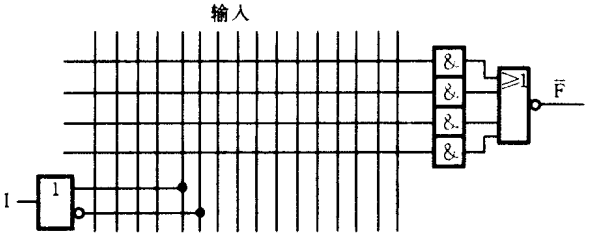


图 8.18　PAL 专用输出结构的局部电路

(2) 带反馈的可编程 I/O 结构

PAL 器件的特点之一是可编程输入/输出,即允许与项直接控制 PAL 的输出驱动器,同时该输出端又可作为一个输入反馈到 PAL 与阵列。如图 8.19 所示,图中最上面一个与门所对应的与项作为输出三态缓冲器的选通控制。编程时如果该与门的所有输入项全接通,即此与项为"0",则三态缓冲器处于高阻状态,对应的 I/O 引脚作为输入使用,这时右边一个互补输出缓冲器作为输入缓冲器用。相反地,若最上面与门的所有输入项都断开,即此与项为"1",则三态缓冲器为工作状态,对应 I/O 引脚作为输出使用。根据这一特性,可通过编程指定某些 I/O 端方向,从而改变器件输入/输出线数目的比例。该类 PAL 器件常见产品有 PAL16L8(10 个输入,8 个输出,6 个反馈输入)以及 PAL20L10(12 个输入,10 个输出,8 个反馈输入)。这种结构通常又称为异步可编程 I/O 结构。

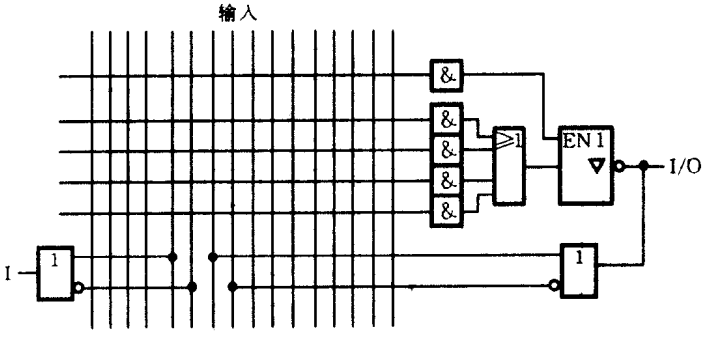


图 8.19　PAL 带反馈的可编程 I/O 结构的局部电路

(3) 带反馈的寄存器输出结构

PAL 的另一种结构是带有反馈的寄存器输出结构。如图 8.20 所示,图中由或门产生的具有 8 个与项的与-或输出,在系统时钟 CLK(公共的)作用下存入到 D 触发器中,触发器的输出通过带有公共选通(OE)的三态缓冲器送到输出端,此输

出 $\overline{Q}$ 是低电平有效。D 触发器的 $\overline{Q}$ 输出通过一个互补输出缓冲器反馈回与阵列，这种反馈功能使 PAL 构成了典型的时序网络结构，从而能实现时序逻辑电路功能，例如，加减计数、移位、转移等操作。该类器件的典型产品有 PAL16R8(8 个输入、8 个寄存器输出、8 个反馈输入、1 个公共时钟和 1 个公共选通)。

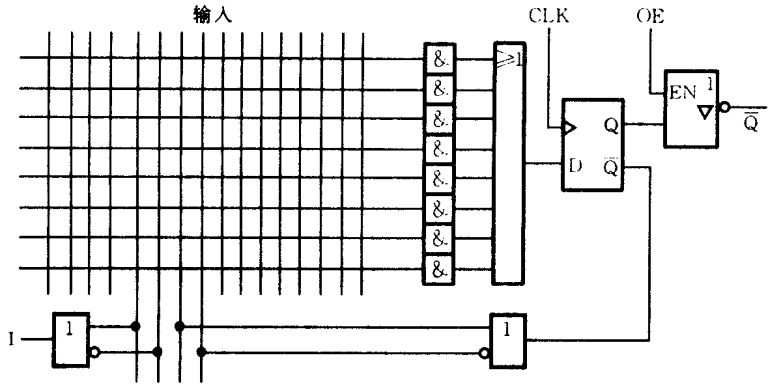


图 8.20 PAL 带反馈的寄存器输出结构的局部电路

(4) 加"异或"、带反馈的寄存器输出结构

这种结构是在带反馈的寄存器输出结构的基础上增加了一个异或门，图 8.21 给出了该类 PAL 的一个局部电路。图中，在 D 触发器的 D 端引入一个异或门，使 D 端的极性可通过编程设置，这实际上是允许把输出端置为高电位有效或者低电位有效。该类电路的典型产品有 PAL16RP8(8 个输入，8 个寄存器输出，8 个反馈输入)。

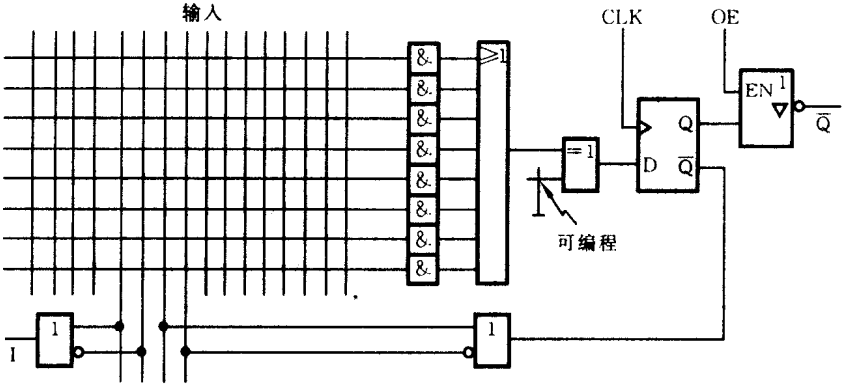


图 8.21 PAL 加"异或"、带反馈的寄存器输出结构局部电路

(5) 算术选通反馈结构

算术 PAL 是在综合前几种 PAL 结构特点的基础上，增加了反馈选通电路，使之能实现多种算术运算功能。图 8.22 给出了这种结构的一个局部电路。图中，D

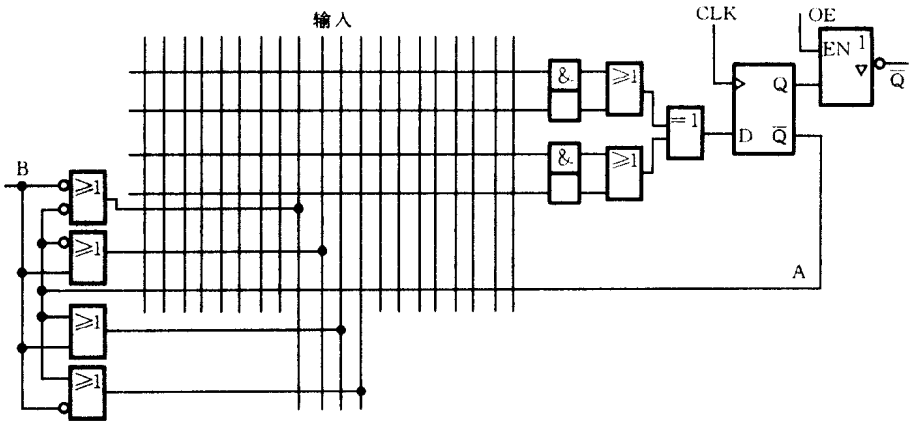


图 8.22　PAL 算术选通反馈结构的局部电路

触发器输入端的异或门允许先前操作产生的“进位”跟 PAL 矩阵中产生的两变量“和”进行异或。反馈选通电路中的 4 个或门接收反馈项 A 和输入项 B，输出 $(A+B)$、$(A+\overline{B})$、$(\overline{A}+B)$、$(\overline{A}+\overline{B})$，以此 4 个项作为逻辑变量送与阵列进行编程，可以获得对 A、B 进行二元逻辑操作的 16 种可能的逻辑组合。图 8.23(a)、(b)分别给出了反馈选通电路的卡诺图和算术功能的编程。算术 PAL 的典型产品有 PAL16A4(8 个输入、4 个寄存器输出、4 个可编程I/O输出、4 个反馈输入、4 个算术选通反馈输入)。

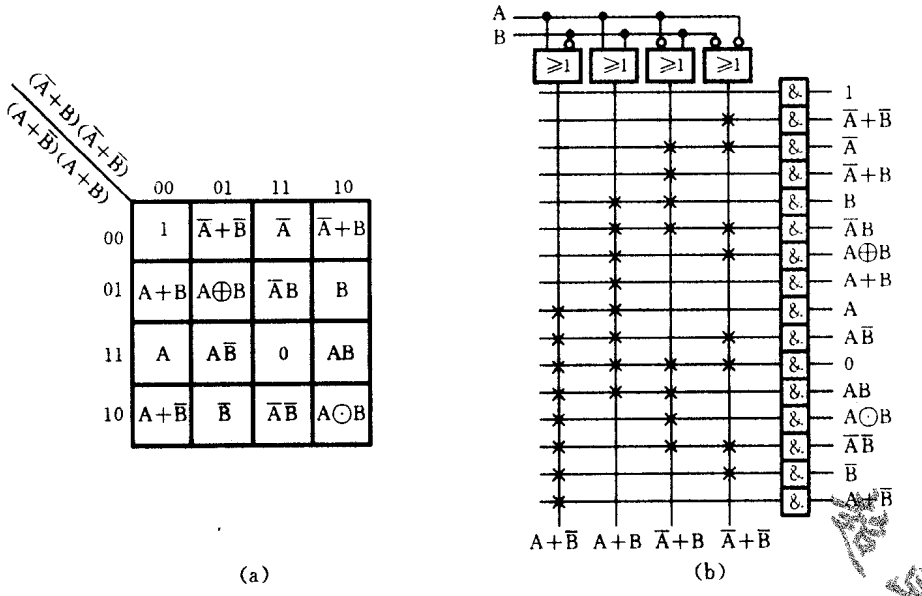


图 8.23　反馈选通电路的卡诺图和算术功能的编程实现

2. PAL 应用举例

由于 PAL 提供了多种灵活的内部结构，因此，可以很方便地用来实现各种组合逻辑和时序逻辑功能。用 PAL 进行逻辑设计时，一般首先按常规设计方法对要求实现的功能进行正确的逻辑描述（如作出真值表、状态图等），写出相应的函数表达式，然后根据具体要求（如输入数、输出数、寄存器数以及与项数等）选择合适的器件，并按函数表达式进行编程。编程既可采用手工方式，也可采用专门的编程软件实现，具体方法可查阅有关资料。这里仅举例介绍 PAL 在逻辑设计中的应用。

例 8.5 采用 PAL16RP8 构成交通灯信号控制器。

图 8.24(a)表示一个单行道交叉路口的交通管理方案。图中，1、2 为单行道路；SEN_1 和 SEN_2 为探测器。当车辆要求通行时，相应探测器给出逻辑“1”信号；否则为逻辑“0”信号。在交通控制器中，探测器是输入装置，交通信号灯是输出装置，信号灯有红(R)、绿(G)和黄(Y)三色。控制器的框图如图8.24(b)所示。它是一个同步时序电路，CP 为时钟信号，INIT 为初始化信号，用来置系统于初始状态。

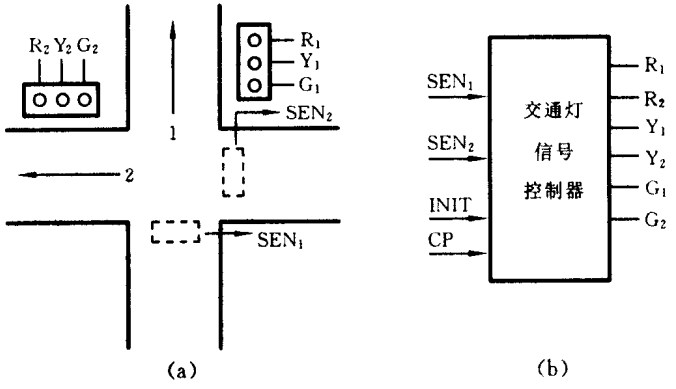


图 8.24 交通灯控制器管理方案和框图

解 设控制器的初始状态为 S_0，可作出该电路的状态图如图 8.25 所示。

利用初始化信号 INIT 置系统于初始状态 S_0，仅当 1 路有车辆要求通过且 2 路无车辆要求通过时，电路停在 S_0 状态；否则，根据另外 3 种不同情况分别转向 S_1 状态或 S_2 状态。S_3 状态为交通转换通行的过渡状态。仅当 2 路有车辆要求通过且 1 路无车辆要求通过时，电路停在 S_4 状态；否则根据另外 3 种不同情况分别转向 S_5 状态或 S_6 状态。S_7 为交通转换通行的过渡状态。凡遇到初始化信号 INIT，电路都回到 S_0 状态（状态图中略）。

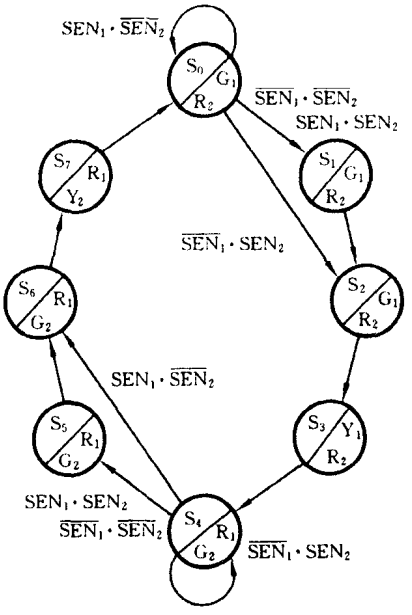


图 8.25　状态图

表 8.6　状态表

现态	次态					输出
	$I_1=1$	$I_2=1$	$I_3=1$	$I_4=1$	$I_0=INIT=1$	
S_0	S_1	S_2	S_0	S_1	S_0	G_1R_2
S_1	S_2	S_2	S_2	S_2	S_0	G_1R_2
S_2	S_3	S_3	S_3	S_3	S_0	G_1R_2
S_3	S_4	S_4	S_4	S_4	S_0	Y_1R_2
S_4	S_5	S_4	S_6	S_5	S_0	R_1G_2
S_5	S_6	S_6	S_6	S_6	S_0	R_1G_2
S_6	S_7	S_7	S_7	S_7	S_0	R_1G_2
S_7	S_0	S_0	S_0	S_0	S_0	R_1Y_2

根据状态图可作出相应状态表，如表 8.6 所示。表中，

$$I_1=\overline{SEN_1}\cdot\overline{SEN_2}\cdot\overline{INIT};$$

$$I_2=\overline{SEN_1}\cdot SEN_2\cdot\overline{INIT}$$

$$I_3=SEN_1\cdot\overline{SEN_2}\cdot\overline{INIT};$$

$$I_4=SEN_1\cdot SEN_2\cdot\overline{INIT};$$

$$I_0=INIT$$

电路共有 8 个状态，需 3 个触发器，设状态变量为 Q_2、Q_1、Q_0，状态分配方案如下。

状态：	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7
$Q_2Q_1Q_0$：	000	001	010	011	100	101	110	111

将其代入表 8.6，可得到二进制状态表，如表 8.7 所示。

根据二进制状态表及对具体问题的分析，可得到激励函数和输出函数表达式：

$$D_2=Q_2^{n+1}=\bar{I}_0\cdot\bar{Q}_2Q_1Q_0+\bar{I}_0\cdot Q_2\bar{Q}_1\bar{Q}_0+\bar{I}_0\cdot Q_2\bar{Q}_1Q_0+\bar{I}_0\cdot Q_2Q_1\bar{Q}_0$$

$$D_1=Q_1^{n+1}=I_2\cdot\bar{Q}_2\bar{Q}_1\bar{Q}_0+I_3\cdot Q_2\bar{Q}_1\bar{Q}_0+\bar{I}_0\cdot\bar{Q}_2\bar{Q}_1Q_0+\bar{I}_0\cdot\bar{Q}_2Q_1\bar{Q}_0$$
$$+\bar{I}_0\cdot Q_2\bar{Q}_1Q_0+\bar{I}_0\cdot Q_2Q_1\bar{Q}_0$$

$$D_0=Q_0^{n+1}=I_1\cdot\bar{Q}_2\bar{Q}_1\bar{Q}_0+I_1\cdot Q_2\bar{Q}_1\bar{Q}_0+I_4\cdot\bar{Q}_2\bar{Q}_1\bar{Q}_0+I_4\cdot Q_2\bar{Q}_1\bar{Q}_0$$
$$+\bar{I}_0\cdot\bar{Q}_2Q_1\bar{Q}_0+\bar{I}_0\cdot Q_2Q_1\bar{Q}_0$$

$R_1=Q_2$

$Y_1=\overline{Q}_2Q_1Q_0$

$G_1=\overline{Q}_2\overline{Q}_1\overline{Q}_0+\overline{Q}_2\overline{Q}_1Q_0+\overline{Q}_2Q_1\overline{Q}_0$

$R_2=\overline{Q}_2$(即 $\overline{R}_2=Q_2$)

$Y_2=Q_2Q_1Q_0$

$G_2=Q_2\overline{Q}_1\overline{Q}_0+Q_2\overline{Q}_1Q_0+Q_2Q_1\overline{Q}_0$

表 8.7 二进制状态表

现态			次态 $Q_2^{n+1}Q_1^{n+1}Q_0^{n+1}$					输出					
Q_2	Q_1	Q_0	$I_1=1$	$I_2=1$	$I_3=1$	$I_4=1$	$I_0=1$	R_1	Y_1	G_1	R_2	Y_2	G_2
0	0	0	001	010	000	001	000	0	0	1	1	0	0
0	0	1	010	010	010	010	000	0	0	1	1	0	0
0	1	0	011	011	011	011	000	0	0	1	1	0	0
0	1	1	100	100	100	100	000	0	1	0	1	0	0
1	0	0	101	100	110	101	000	1	0	0	0	0	1
1	0	1	110	110	110	110	000	1	0	0	0	0	1
1	1	0	111	111	111	111	000	1	0	0	0	0	1
1	1	1	000	000	000	000	000	1	0	0	0	1	0

PAL16RP8 共有 20 根引脚，根据器件功能和交通灯控制器的输入/输出设置，可按图 8.26 所示的方案对 PAL16RP8 进行引脚分配。

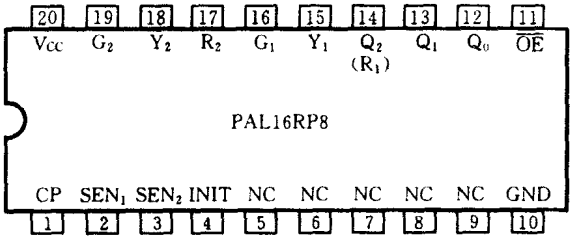


图 8.26 PAL 引脚分配图

依照上述函数表达式和芯片的引脚分配方案，可画出用 PAL16RP8 实现交通灯控制器功能的阵列逻辑图，如图 8.27 所示。由此例可见，利用一片 PAL 可实现较复杂的逻辑功能。通常，一片等效 200～500 个门的 PAL 可代替 4～10 片 14 引脚的中小规模集成电路。

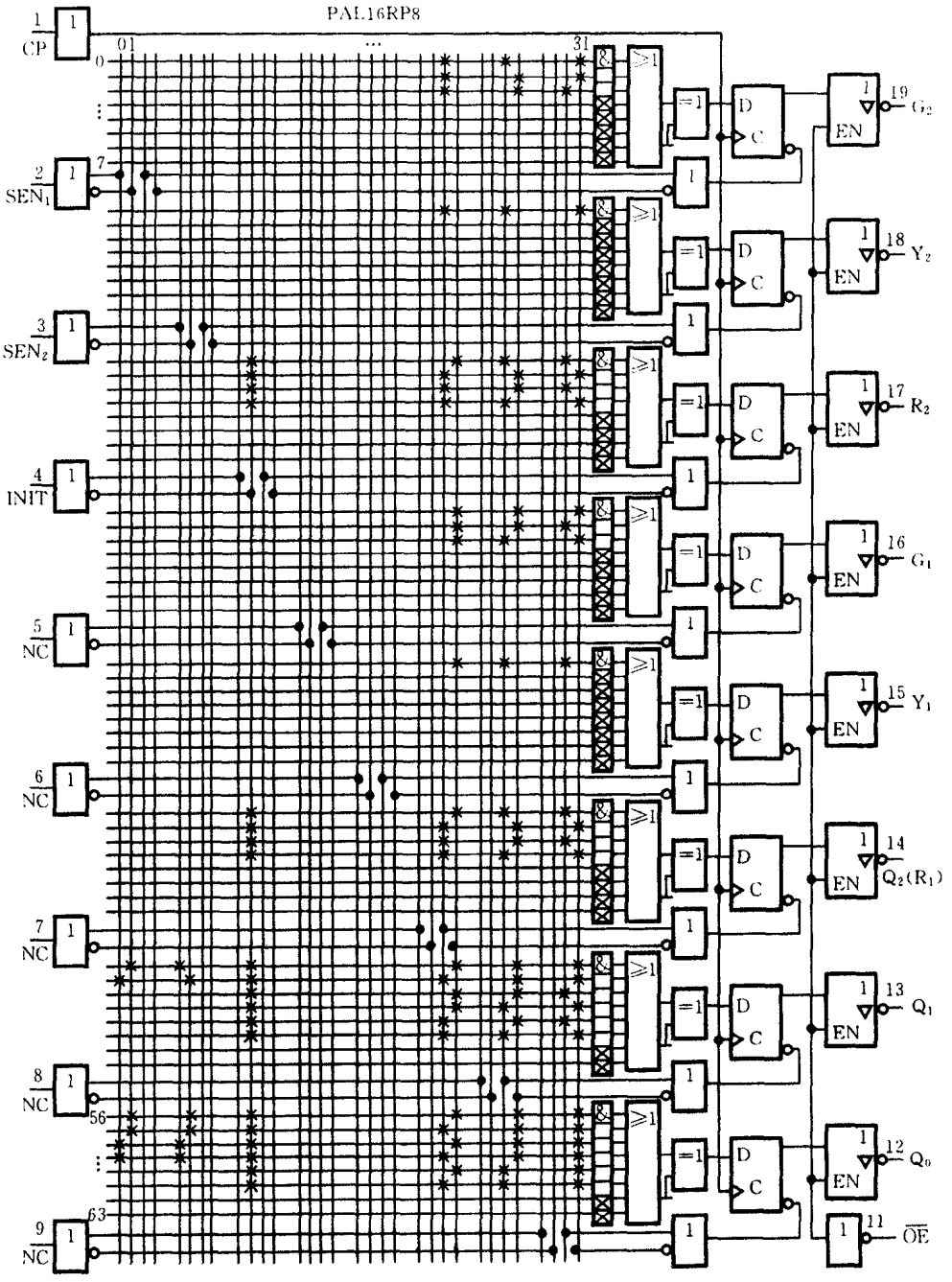


图 8.27　交通灯控制器阵列逻辑图

8.2.4 通用阵列逻辑 GAL

PAL 器件的发展和应用，给逻辑设计提供了很大的灵活性，但这种灵活性仍有一定的局限性。例如，PAL 一般采用的是熔丝式工艺，编程后不能再改写，使用户应用感到不方便；PAL 输出结构的变更不够灵活，致使对不同输出结构的需求应选用不同型号的 PAL 器件等。GAL(Generic Array Logic)器件是 1985 年由美国 LATTICE 公司开发并商品化的一种新的 PLD 器件。它是在 PAL 器件的基础上综合了 E^2PROM 和 CMOS 技术发展起来的一种新型技术。GAL 器件具有 PAL 器件所没有的可擦除、可重新编程及其结构可组态的特点。这些特点形成了器件的可测试性和高可靠性，且具有更大的灵活性。

GAL 器件按门阵列的可编程结构可分为两大类：一类是与 PAL 基本结构相似的普通型 GAL 器件，其与阵列是可编程的，或阵列是固定连接的，如 20 引脚的 GAL16V8；另一类是与 PLA 器件相类似的新一代 GAL 器件，其与阵列和或阵列都是可编程的，如 24 引脚的 GAL39V8 便属于这一类器件。

1. GAL 的基本逻辑结构

普通型 GAL 的基本结构与 PAL 相类似，都是由一个可编程的与阵列去驱动一个固定连接的或阵列，所不同的是输出部件结构不同。GAL 在每一个输出端都集成有一个输出逻辑宏单元 OLMC(Output Logic Macro Cell)，允许使用者定义每个输出的结构和功能。现以 GAL16V8 为例介绍 GAL 的基本逻辑结构和原理。

GAL16V8 芯片是具有 8 个固定输入引脚、最多可达 16 个输入引脚，8 个输出引脚，输出可编程的普通型 GAL。其逻辑结构如图 8.28 所示。由图可见，它由 8 个输入缓冲器、8 个反馈输入缓冲器、8 个输出逻辑宏单元 OLMC，8 个输出三态缓冲器、与阵列和系统时钟、输出选通信号等组成。其中，与阵列包含 32 列和 64 行，32 列表示 8 个输入的原变量和反变量及 8 个输出反馈信号的原变量和反变量；64 行表示与阵列可产生 64 个与项，对应 8 个输出，每个输出包括 8 个与项。

(1) 输出逻辑宏单元 OLMC

OLMC 的逻辑结构示于图 8.29。它由一个 8 输入或门、极性选择异或门、D 触发器、4 个多路选择器等组成。

或门的每个输入对应一个来自与阵列的与项，输出形成与或函数表达式。

异或门控制输出信号的极性选择，当异或门的控制变量 XOR(n)(其中 n 为 OLMC 输出引脚号)为“0”时，异或门的输出与输入相同，当 XOR(n)为“1”时，异或门的输出与输入相反。极性选择还可以用来简化实现某一功能所需要的与项

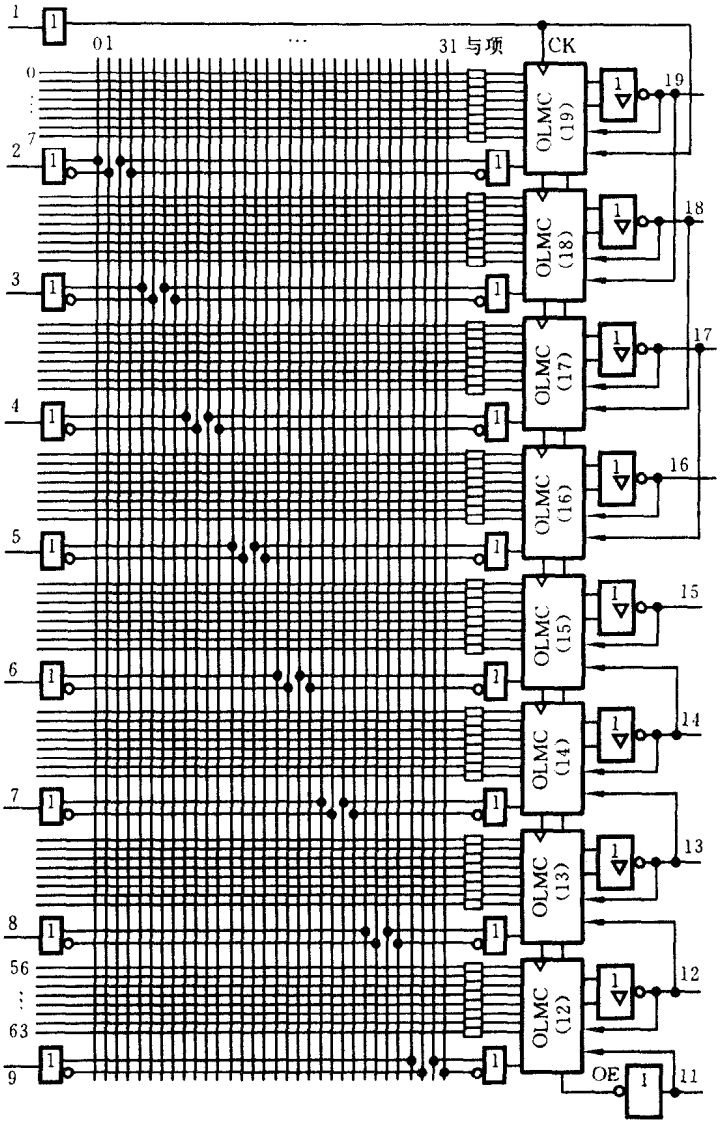


图 8.28　GAL16V8 的逻辑结构

数。GAL 的输出一般只能实现不大于 8 个与项的函数，如果采用异或门，则可以把大于 8 项而每项只包含一个变量的函数简化为一个与项。例如：

$$D=A+B+C+E+F+G+H+I+J \qquad (9\text{ 项})$$

利用德·摩根定律可变换为

$$\overline{D}=\overline{A}\cdot\overline{B}\cdot\overline{C}\cdot\overline{E}\cdot\overline{F}\cdot\overline{G}\cdot\overline{H}\cdot\overline{I}\cdot\overline{J} \qquad (1\text{ 项})$$

要得到 D，只需由或门产生 $\overline{D}$，然后令异或门控制变量 XOR(n)为“1”，对其求反即可。

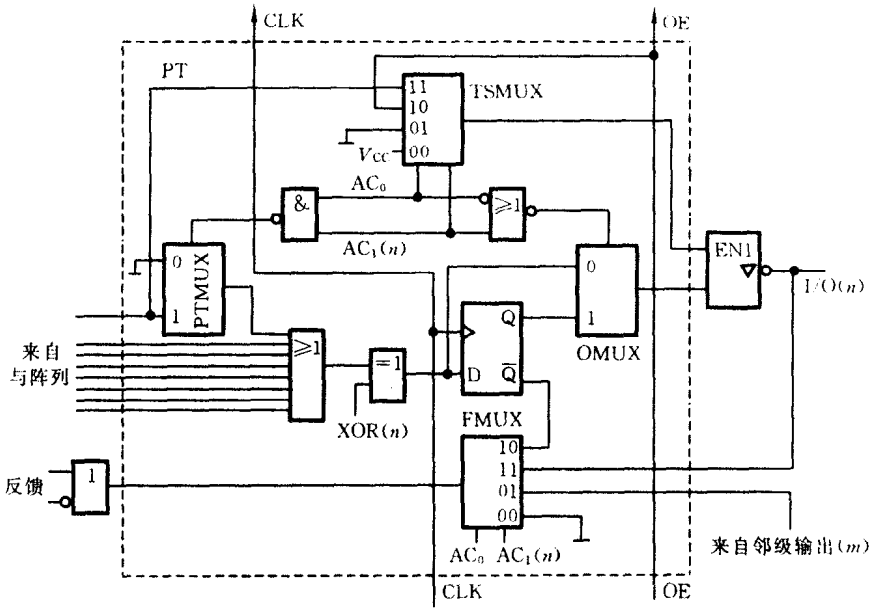


图 8.29 OLMC 的逻辑结构

D 触发器对输出状态起寄存作用，使 GAL 适应于时序逻辑电路。

4 个多路选择器的功能如下。

① 与项选择多路选择器 PTMUX 用于控制第一个与项。来自与阵列的 8 个与项当中有 7 个直接作为或门的输入，另一个作为 PTMUX 的输入，PTMUX 的另一输入接“地”。在 AC_0 和 $AC_1(n)$ 控制下 $(\overline{AC_0 \cdot AC_1(n)})$，PTMUX 选择该与项或者“地”作为或门的输入。

② 输出选择多路选择器 OMUX 用于选择输出信号来自组合逻辑还是时序逻辑。由异或门输出的所需极性的与或逻辑结果，在送至 OMUX 一个输入端的同时通过时钟信号 CLK 送入 D 触发器中，触发器的 Q 输出送至 OMUX 的另一输入端。OMUX 在 AC_0 和 $AC_1(n)$ 的控制下，由 $(\overline{\overline{AC_0}+AC_1(n)})$ 选择组合型或寄存器型结果作为输出。

③ 输出允许控制选择多路选择器 TSMUX 用于选择输出三态缓冲器的选通信号。在 AC_0 和 $AC_1(n)$ 的控制下，TSMUX 选择 V_{CC}、“地”、OE 或者一个与项(PT)作为允许输出的控制信号。

④ 反馈选择多路选择器 FMUX 用于控制反馈信号的来源。在 AC_0 和 $AC_1(n)$ 的控制下，FMUX 选择“地”、相邻位的输出、本位的输出或者触发器的输出 $\overline{Q}$ 作为反馈信号，送回与阵列作为输入信号。

由 OLMC 的各部分功能可知，只要恰当地给出各控制信号的值，就能形成 OLMC 的不同组态。可以说，在适应不同要求方面，OLMC 给设计者提供了最大的灵活性。具体各控制信号的值是由 GAL 结构控制字中的相应可编程位决定的。

(2) 结构控制字

GAL16V8 由一个 82 位的结构控制字控制着器件的各种功能组合状态。该控制字中各位的功能如图 8.30 所示。图中，XOR(n)和 $AC_1(n)$字段下面的数字分别对应器件的输出引脚号，指相应引脚号对应的宏单元。

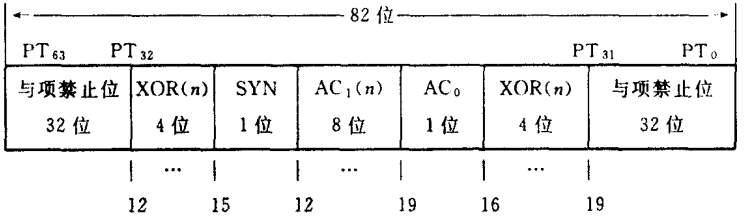


图 8.30　GAL16V8 结构控制字

结构控制字中各位的功能如下。

同步位 SYN：它的值确定器件是具有寄存器输出能力或是纯粹的组合输出。SYN＝0 时，GAL 器件有寄存器输出能力；SYN＝1 时，GAL 为一个纯粹组合逻辑器件。此处，为了保证与 PAL 型器件结构完全兼容，在图 8.28 所示逻辑图的最外层两个宏单元 OLMC(12)和 OLMC(19)中，用$\overline{SYN}$代替 AC_0，SYN 代替 $AC_1(n)$作为多路选择器 FMUX 的选择控制端。

结构控制位 AC_0：该位对于 8 个 OLMC 是公共的，它与 $AC_1(n)$配合控制各个 OLMC(n)中的多路选择器。

结构控制位 AC_1：它共有 8 位，每个 OLMC(n)有单独的 $AC_1(n)$。

极性控制位 XOR(n)：它通过 OLMC(n)中的异或门控制逻辑操作结果的输出极性。XOR(n)＝0时，输出信号 O(n)低电平有效；XOR(n)＝1 时，输出信号 O(n)高电平有效。

与项(PT)禁止位：共 64 位，分别控制与阵列的 64 行(PT_0～PT_{63})，以便屏蔽某些不用的与项。

通过编程结构控制字中的 SYN、AC_0和 $AC_1(n)$，输出逻辑宏单元 OLMC(n)可以组成以下 5 种组态。

① 专用输入方式(SYN · AC_0 · $AC_1(n)$＝101)。在这种方式中，CLK 和 OE 均不起作用，输出三态缓冲器控制开关接地，输出被禁止，此时的 I/O 只作为普通数据输入端。此方式又称为纯输入方式，如图 8.31(a)所示。

② 专用组合型输出方式(SYN · AC_0 · $AC_1(n)$＝100)。在该方式中，CLK 和

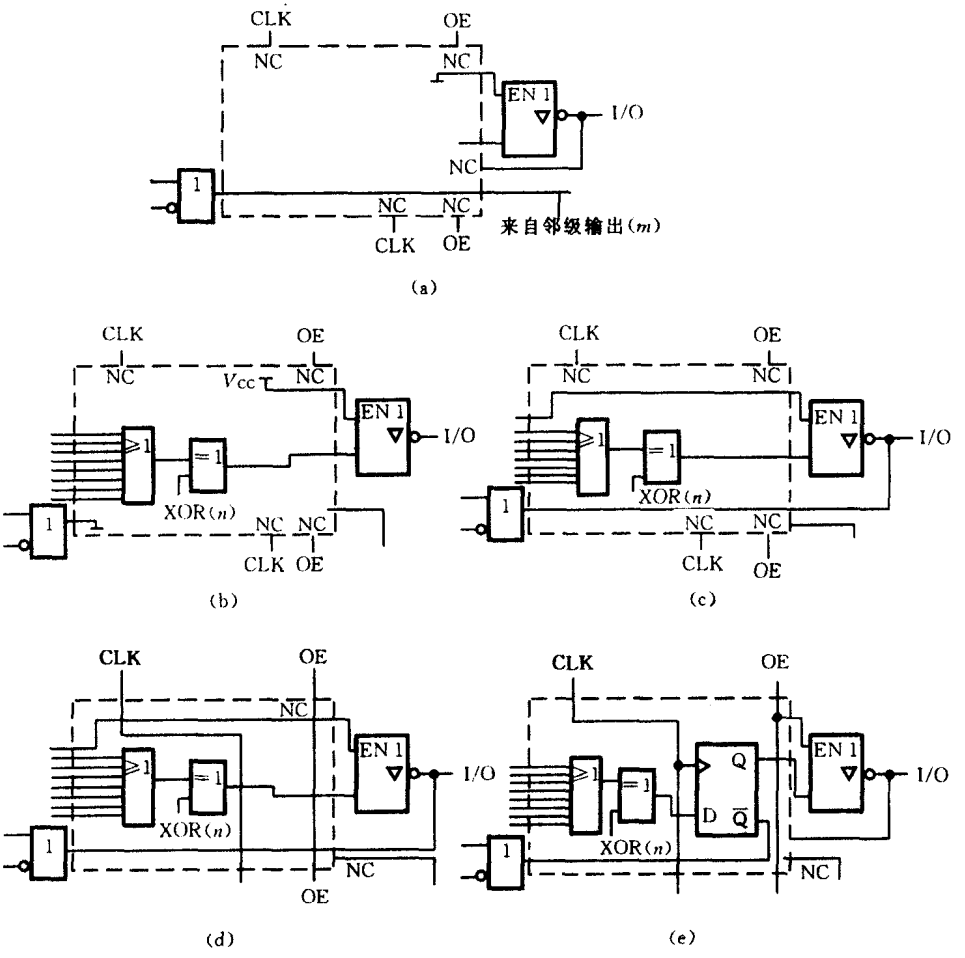


图 8.31 OLMC 的 5 种组态结构方式

OE 均不起作用，输出三态缓冲器控制开关接 V_{CC}，处于工作状态。又称它为纯组合逻辑输出方式，如图 8.31(b)所示。

③ 组合型输出方式($SYN \cdot AC_0 \cdot AC_1(n)=111$)。在该方式中，CLK 和 OE 均不起作用，芯片的输出是组合型的，但输出三态缓冲器受来自与阵列的与项控制。又称它为三态控制的纯组合逻辑输出方式，如图 8.31(c)所示。

④ 寄存器型器件中的组合逻辑输出方式($SYN \cdot AC_0 \cdot AC_1(n)=011$)。在此方式中，CLK 和 OE 均有效，所有输出中至少有一个寄存器型的输出；但被组态的这个 OLMC(n)本身的 CLK 悬空，仍为组合逻辑。此方式又被称为组合逻辑、时序逻辑混合方式，如图 8.31(d)所示。

⑤ 寄存器型输出方式(SYN · AC_0 · $AC_1(n)$=010)。在此方式中,CLK 和 OE 均有效,输出端全部为寄存器输出方式。又称它为三态控制的纯时序逻辑输出方式,如图 8.31(e)所示。

应该说明,上述 OLMC 组态的实现是由开发软件和硬件完成的,而且对用户是完全透明的。开发软件将选择与配制控制字的所有位,并自动检查各引线的用法。

(3) 行地址布局

GAL 器件的可编程阵列包括与阵列、结构控制字、保密位及整体擦除位等。对其进行编程时是由行地址进行映射的。GAL16V8 的行地址布局如图 8.32 所示。

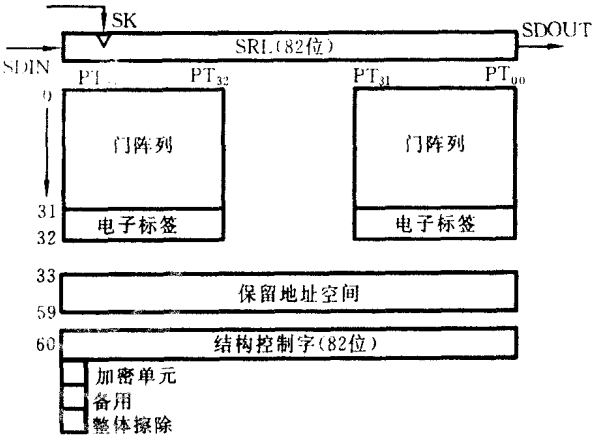


图 8.32　GAL16V8 行地址布局示意图

图中,82 位的 SRL 用于将各列(位)编程点图案或数据串行输入编程或读出测试编程结果串行输出。当对 GAL16V8 进行编程实现某个设计方案时,可供用户使用的行地址总共有 36 个,它们分别是:

① 行地址 0～31:这是用户用来编程制定逻辑模式的阵列。32 个行地址对应逻辑结构图上与阵列的 32 个输入,每个行地址可写入 64 位数据,对应于 64 个与项。

② 行地址 32:这是器件的电子标签(ES)。它提供 64 位供用户定义,例如,用来识别用户身份的代码、版本号、编目管理等,这个标签中的数据与下述保密单元的状态无关,用户始终可以使用。

③ 行地址 60:这是 82 位的结构控制字,用于设计所需用途的器件。

④ 行地址 61:这是仅 1 位的保密位,用于防止复制阵列的逻辑点阵,该位一旦被编程,存取阵列的电路就不能工作,从而防止了对 32 位的与阵列再次编程或读出。该单元只能在整体擦除时和阵列一起被擦除,所以,一旦保密位被编程,就绝对不能检查阵列的原始配置,仅电子标签除外。

⑤ 行地址 63：仅含 1 位，用于器件整体擦除。在器件编程期间访问该行，就执行清除功能，整个与阵列、结构控制字、电子标签以及保密单元统统被擦除，使编程的器件恢复到未使用的状态。

2. GAL 器件的开发支持和编程

可编程逻辑器件的应用大大简化了逻辑设计过程。GAL 器件的编程是在相应开发软件和硬件的支持下完成的，除得到现有的大部分开发软件和标准编程器的支持外，新的高级开发软件的相继出现，使 GAL 的设计应用愈来愈方便。GAL 器件能得到广泛地应用，除了器件自身具有的优良特性之外，还在很大程度上取决于它的开发环境。

在 GAL 开发工具中硬件包括计算机和编程器，软件有 GALLAB、CUPL、ABEL等。GALLAB 是基于已经简化的与或表达式的汇编型编码软件。CUPL 和ABEL属于编译型的高级开发软件，它具有自动化逻辑化简功能，设计说明书文件可采用高级的逻辑描述方法(高级逻辑方程、真值表和状态转换图)，并可对器件进行仿真测试，修改逻辑设计，是通用性很强的开发工具。下面仅就实用、易学的简易开发工具 GALLAB 给予介绍。

(1) GALLAB 开发软件

GALLAB 软件工具包含两个相关文件，GALLAB. EXE 和 FM. EXE。GALLAB. EXE 文件是直接与编程接口的联机软件，它具有手工编辑熔丝图、文件存盘、装入文件、读/写 GAL、校验等功能。由此看出，GALLAB. EXE 具有相当完整的 GAL 芯片开发功能，但用它来设计 GAL 的“熔丝图”靠手工编辑，是一件繁琐而容易出错的工作。FM. EXE 则弥补了 GALLAB. EXE 的不足。

使用 FM(Fast Map)软件时，在 DOS 提示符下键入 FM 及以. PLD为扩展名的 GAL 设计说明书文件名后，屏幕上将出现如下所示的 FM 菜单：

•CREAT DOCUMENT FILE	产生文档文件和芯片引脚分配图
•CREAT FUSE FILE	产生可供阅读的熔丝图
•CREAT JEDEC FILE	产生以 JED 为结尾的可写入的文件
•GET A NEW SOURCE FILE	产生新的源程序
•EXIT FROM FAST MAP	退出 Fast Map

调用 FM. EXE，键入菜单所提示的选择项，FM 将生成所需要的相应文件同时存盘。生成的文件包括：

① 扩展名为. LST 的文本文件，包括源文件和 GAL 引脚配置图；

② 扩展名为. PLT 的芯片熔丝图文件；

③ 扩展名为. JED 的标准装载文件。在 JEDEC 标准文件中包含了门阵列中

所有编程节点的编程信息，可以直接传送给编程器，对 GAL 进行编程。

键入菜单提示中的后面两个选择项，FM 将要求放弃现在的源文件汇编，输入新的源文件名并重新显示菜单，或者从 FM 退回到系统环境。

(2) GAL 的编程

生成上述各文件后，便可利用 GAL 编程器对 GAL 芯片进行编程。

GAL 编程器只需要一台 PC 机，以及一片包含有 HELP. DATA，FM. EXE，LL. EXE 软件的 FM 软盘。

编程系统在建立和运行前，将编程器通过串行口 RS232C 与主机相连，并把 GAL 芯片插入编程器的插座内。

在硬件编程时，调用 LL. EXE 文件建立 GAL 编程系统，选择编程 GAL 芯片型号，执行 DOWN LOAD 功能，将磁盘中的文件内容(由 FM 生成的芯片熔丝图文件)装入缓冲区，建立 JEDEC 文件，屏幕上将显示出熔丝熔断状态窗口图。如不再进行编辑，则按下[F6]写入键，把编辑好的熔丝图写入编程器插座上的 GAL 芯片，同时，将写入结果与所编辑的数据进行核对，一旦发现错误，就在屏幕右上角显示出错总数。编程后若要加密，可利用[F5]写加密位键，设置加密位。

概括地说，软件 FM. EXE 是将用户用编程语言编写的芯片逻辑功能描述处理成可写入 GAL 中的编程模式。LL. EXE 软件是把编程后正确无误的熔丝熔断状态图写入 GAL 芯片，完成编程。由于编程系统具有较强的人机对话功能，因而简单、易学，使用也很方便。

3. GAL 应用举例

GAL 作为一种通用的 PLD 器件，其设计过程遵循典型的 PLD 设计流程，一般步骤如下。

第一步：分析设计要求，确定描述给定功能的逻辑表达式。这是关键的一步，设计者可综合运用逻辑设计技术，完成有关逻辑设计。

第二步：根据逻辑设计结果，选择 GAL 器件并对器件进行引脚分配。在器件选择时，一般首先应满足输入/输出端数目的要求，其次考虑速度、功率等因素。器件确定后，根据要求给各引脚赋予变量名。

第三步：编写 GAL 设计说明书。在调用 FM 软件前，首先要根据 FM 的语法格式和逻辑设计结果用文本编辑程序 EDLIN 或 WORDSTAR 编写设计说明书(又称为 GAL 源程序)。说明书中通常包括器件型号、实现的功能、设计者姓名和设计日期、电子标签、GAL 引脚表以及逻辑表达式等。

第四步：调用 FM. EXE，建立文本文件(LST 文件)，熔丝图文件(PLT 文件)，标准装载文件(JED 文件)。

第五步：硬件编程。包括连接编程器，调用 LL.EXE 文件，建立 GAL 编程系统，编程后写保密位等。

例 8.6 采用 GAL16V8 设计一个 6 位移位寄存器。要求该寄存器具有左/右移位，串行/并行数据输入，串行/并行数据输出及高阻输出等功能。

解 根据设计要求，画出寄存器的逻辑框图，如图8.33所示。其中，$\overline{OE}$为使能端，S_1、S_0为工作方式选择端，LI 为左移串行输入端，RI 为右移串行输入端，CLK 为时钟端，$d_0 \sim d_5$为数据输入端，$Q_0 \sim Q_5$为寄存器输出端。该寄存器的功能表如表 8.8 所示。

表 8.8 6 位移位寄存器功能表

输入												输出						说明
$\overline{OE}$	S_1	S_0	RI	LI	d_0	d_1	d_2	d_3	d_4	d_5		Q_0	Q_1	Q_2	Q_3	Q_4	Q_5	
1	×	×	×	×	×	×	×	×	×	×		Z	Z	Z	Z	Z	Z	高阻输出
0	0	0	×	×	×	×	×	×	×	×		Q_0	Q_1	Q_2	Q_3	Q_4	Q_5	保 持
0	0	1	×	D	×	×	×	×	×	×		Q_1	Q_2	Q_3	Q_4	Q_5	D	左 移
0	1	0	D	×	×	×	×	×	×	×		D	Q_0	Q_1	Q_2	Q_3	Q_4	右 移
0	1	1	×	×	d_0	d_1	d_2	d_3	d_4	d_5		d_0	d_1	d_2	d_3	d_4	d_5	并行输入

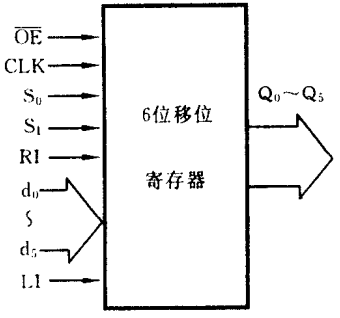


图 8.33 6 位移位寄存器的逻辑框图

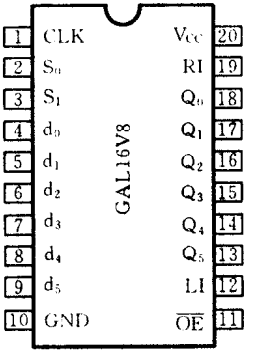


图 8.34 GAL16V8 引脚分配图

根据功能表可写出各触发器的激励函数表达式：

$$D_0 = Q_0^{n+1} = \bar{S}_1\bar{S}_0Q_0 + \bar{S}_1S_0 \cdot Q_1 + S_1\bar{S}_0 \cdot RI + S_1S_0d_0$$

$$D_1 = Q_1^{n+1} = \bar{S}_1\bar{S}_0Q_1 + \bar{S}_1S_0Q_2 + S_1\bar{S}_0Q_0 + S_1S_0d_1$$

$$D_2 = Q_2^{n+1} = \bar{S}_1\bar{S}_0Q_2 + \bar{S}_1S_0Q_3 + S_1\bar{S}_0Q_1 + S_1S_0d_2$$

$$D_3 = Q_3^{n+1} = \bar{S}_1\bar{S}_0Q_3 + \bar{S}_1S_0Q_4 + S_1\bar{S}_0Q_2 + S_1S_0d_3$$

$$D_4 = Q_4^{n+1} = \bar{S}_1\bar{S}_0Q_4 + \bar{S}_1S_0Q_5 + S_1\bar{S}_0Q_3 + S_1S_0d_4$$

$$D_5 = Q_5^{n+1} = \bar{S}_1\bar{S}_0Q_5 + \bar{S}_1S_0 \cdot LI + S_1\bar{S}_0Q_4 + S_1S_0d_5$$

采用 GAL16V8 实现给定功能时，芯片引脚分配如图 8.34 所示。

按照 FM 软件要求，用文本编辑程序产生 GAL 设计说明书(以. PLD 为扩展名的一个输入文件)，然后启动 FM 汇编程序，生成 6 位移位寄存器的 GAL 输入文件，如图 8.35 所示。

```
GAL16V8
6 BIT SHIFT REGISTER
SHIFREG6
CLK    S0    S1    d0    d1    d2    d3    d4    d5    GND:PIN NAME
/OE    LI    Q5    Q4    Q3    Q2    Q1    Q0    RI    VCC
       Q0    :=/S0 * /S1 * Q0+S0 * /S1 * Q1+/S0 * S1 * RI+S0 * S1 * d0
       Q1    :=/S0 * /S1 * Q1+S0 * /S1 * Q2+/S0 * S1 * Q0+S0 * S1 * d1
       Q2    :=/S0 * /S1 * Q2+S0 * /S1 * Q3+/S0 * S1 * Q1+S0 * S1 * d2
       Q3    :=/S0 * /S1 * Q3+S0 * /S1 * Q4+/S0 * S1 * Q2+S0 * S1 * d3
       Q4    :=/S0 * /S1 * Q4+S0 * /S1 * Q5+/S0 * S1 * Q3+S0 * S1 * d4
       Q5    :=/S0 * /S1 * Q5+S0 * /S1 * LI+/S0 * S1 * Q4+S0 * S1 * d5
DESCRIPTION
```

图 8.35　6 位移位寄存器输入文件

经编程系统编程后，形成图 8.36 和图 8.37 所示的 GAL16V8 熔丝图，然后，完成硬件编程即可。与熔丝图对应的实际阵列逻辑图如图 8.38 所示。

```
GAL16V8
6 BIT SHIFT REGISTER
SHIFREG6
  Array Input Pin      1  1  1  1  1  1  1  1
                      29 38 47 56 65 74 83 92
Polarity Fuse X
AC1 Fuse X
      Output Pin 19 Row 0    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 19 Row 1    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 19 Row 2    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 19 Row 3    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 19 Row 4    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 19 Row 5    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 19 Row 6    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 19 Row 7    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
Polarity Fuse X
AC1 Fuse X
      Output Pin 18 Row 0    _ x _ _ _ x x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 18 Row 1    x _ _ _ _ x _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 18 Row 2    _ x x _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 18 Row 3    x _ _ _ x _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 18 Row 4    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 18 Row 5    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 18 Row 6    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 18 Row 7    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
Polarity Fuse X
AC1 Fuse X
      Output Pin 17 Row 0    _ x _ _ _ x _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 17 Row 1    x _ _ _ _ x _ _ _ _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 17 Row 2    _ x _ _ x _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 17 Row 3    x _ _ _ x _ _ _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 17 Row 4    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 17 Row 5    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 17 Row 6    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 17 Row 7    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
Polarity Fuse X
AC1 Fuse X
      Output Pin 16 Row 0    _ x _ _ _ x _ _ _ _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 16 Row 1    x _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 16 Row 2    _ x _ _ x _ _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 16 Row 3    x _ _ _ x _ _ _ _ _ _ _ _ _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ _ _
      Output Pin 16 Row 4    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 16 Row 5    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 16 Row 6    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
      Output Pin 16 Row 7    x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
```

图 8.36　6 位移位寄存器的熔丝图(GAL 16V8)之一

```
GAL16V8
6 BIT SHIFT REGISTER
SUN HANFANG 5 1989 BSERC
SHIFERG6
  Array Input Pin      1  1  1  1  1  1  1  1
                       2 9 3 8 4 7 5 6 6 5 7 4 8 3 9 2
Polarity Fuse X
AC1 Fuse X
    Output Pin 15 Row 0     -x---x----------x----------
    Output Pin 15 Row 1     x----x----------x----------
    Output Pin 15 Row 2     -x--x----------x----------
    Output Pin 15 Row 3     x---x----------x----------
    Output Pin 15 Row 4     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 15 Row 5     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 15 Row 6     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 15 Row 7     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Polarity Fuse X
AC1 Fuse X
    Output Pin 14 Row 0     -x---x----------x----------
    Output Pin 14 Row 1     x----x----------x----------
    Output Pin 14 Row 2     -x--x----------x----------
    Output Pin 14 Row 3     x---x----------x----------
    Output Pin 14 Row 4     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 14 Row 5     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 14 Row 6     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 14 Row 7     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Polarity Fuse X
AC1 Fuse X
    Output Pin 13 Row 0     -x---x----------x----------
    Output Pin 13 Row 1     x----x----------x----------
    Output Pin 13 Row 2     -x--x----------x----------
    Output Pin 13 Row 3     x---x----------x----------
    Output Pin 13 Row 4     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 13 Row 5     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 13 Row 6     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 13 Row 7     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Polarity Fuse X
AC1 Fuse X
    Output Pin 12 Row 0     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 12 Row 1     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 12 Row 2     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 12 Row 3     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 12 Row 4     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 12 Row 5     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 12 Row 6     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    Output Pin 12 Row 7     xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
SYN Fuse X
AC1 Fuse
```

图 8.37　6 位移位寄存器的熔丝图(GAL 16V8)之二

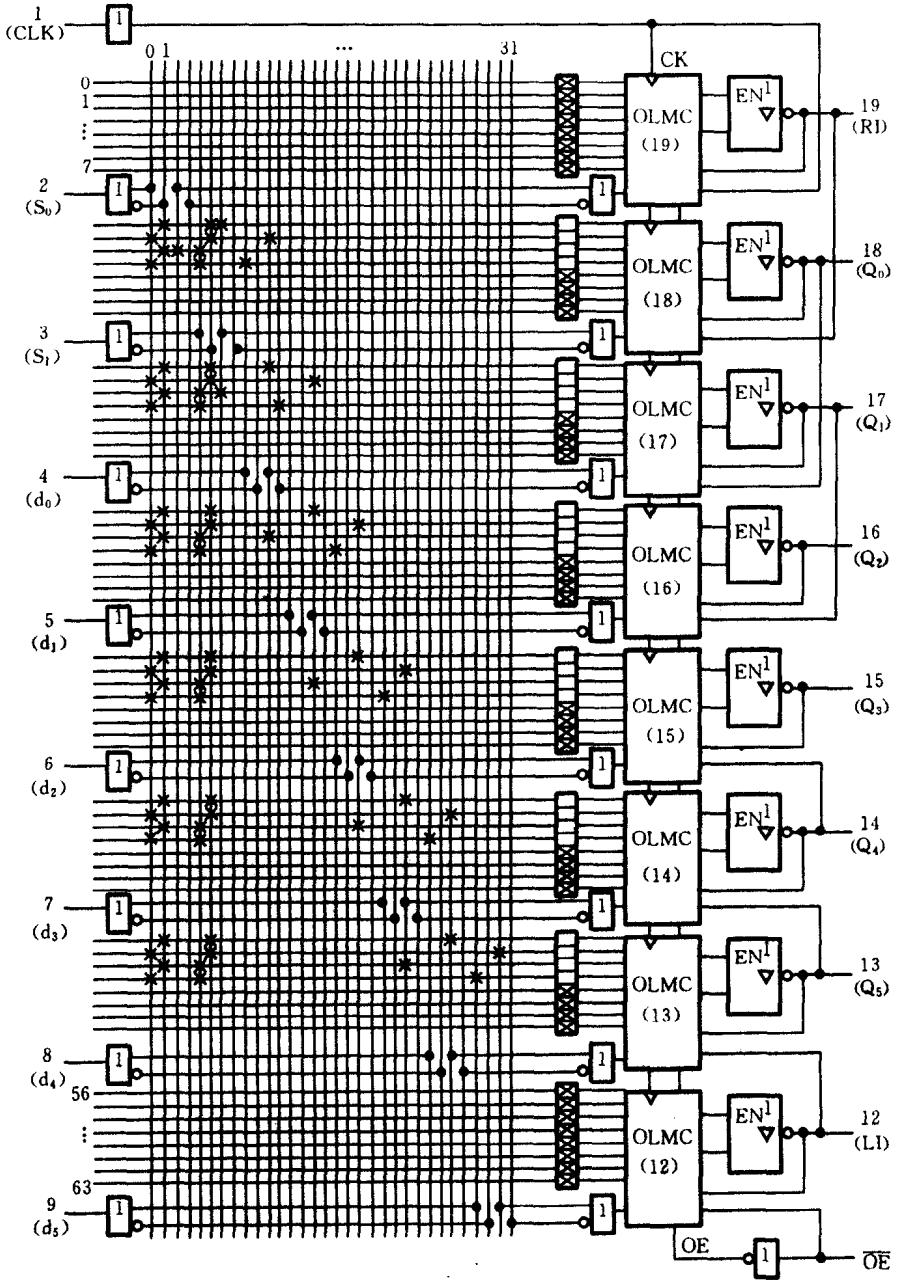


图 8.38　6 位移位寄存器阵列逻辑图(GAL16V8)

8.3 在系统编程技术简介

在系统编程(In-System Programmable,简称 ISP)技术是 20 世纪 90 年代发展起来的一种 PLD 技术。所谓在系统编程,是指可以在用户自己设计的目标系统上、为实现预定逻辑功能而对逻辑器件进行编程或改写。ISP 器件的出现,使数字系统设计更加灵活、方便,为用户带来了显著的经济效益和时间效益。可以说,ISP 技术是 PLD 设计技术发展中的一次重要变革。

8.3.1 ISP 技术的主要特点

使用常规的 PLD 进行设计是先编程后装配,而采用 ISP 技术的 PLD 则是先装配后编程,并且在成为产品之后仍可根据需要反复编程。ISP 技术的应用,对数字系统硬件设计方法、设计环境、系统调试周期、测试与维护、系统的升级以及器件的充分利用等均产生了重要影响。使数字系统设计的面貌焕然一新。归纳起来,有如下主要特点。

1. 全面实现了硬件设计与修改的软件化

ISP 技术,使硬件设计变得和软件设计一样方便。设计时可由用户按编程方法构建各种逻辑功能,并且对器件实现的逻辑功能可以像软件一样随时进行修改和重构。这不仅实现了数字系统中硬件逻辑功能的软件化,而且实现了硬件设计和修改方法的软件化。从根本上改变了传统的硬件设计方法与步骤,成功地实现了硬、软件技术的有机结合,形成了一种全新的硬件设计方法。

2. 简化了设计与调试过程

由于采用了 ISP 技术,所以在用器件实现预定功能时,省去了利用专门的编程设备对器件进行单独编程的环节,从而简化了设计过程。并且,利用 ISP 技术进行功能修改时,可以在不从系统中取下器件的情况下直接对芯片进行重新编程,故方案调整验证十分方便,能够及时处理那些设计过程中无法预料的逻辑变动,因此,可大大缩短系统的设计与调试周期。

3. 容易实现系统硬件的现场升级

采用常规逻辑设计技术构造的系统,要想对安装在应用现场的系统进行硬件

升级一般是非常困难的，往往要付出很高的代价。但采用ISP技术设计的系统，则可利用系统本身的资源和ISP软件，通过新的器件组态程序，由微处理器I/O端口产生ISP控制信号及数据，立即实现硬件现场升级。

4. 可降低系统成本，提高系统可靠性

ISP技术不仅使逻辑设计技术产生了变革，而且推动了生产制造技术的发展。利用ISP技术可以实现多功能硬件设计，即将具有一种或几种功能的硬件设计成可以实现多种系统级功能的硬件，从而大大减少在同一系统中使用不同部件的数目，使系统成本显著下降；由于ISP器件支持为系统测试而进行的功能重构，因此，可以在不浪费电路板资源或电路板面积的情况下进行电路板级的测试，从而提高电路板级的可测试性，使系统可靠性得以改善；此外，利用ISP技术还可以简化标准PLD制造流程，降低生产成本等。

5. 器件制造工艺先进

由于ISP逻辑器件采用E^2CMOS工艺制造，因此，不仅具有集成度高、可靠性高、速度快、功耗低、可反复改写等优点，而且有100%的参数可测试性及100%的编程正确率。编程或擦除次数可达1000次以上，编程内容20年不丢失。ISP器件还具有加密功能，用来防止对片内编程模式的非法复制。

8.3.2 ISP逻辑器件

1. ISP逻辑器件的类型

目前，市场上提供的ISP逻辑器件可分为ispLSI、ispGAL和ispGDS三种类型。

(1) ispLSI逻辑器件

ispLSI(在系统编程大规模集成)逻辑器件具有集成度高、速度快、可靠性好、灵活方便等优点，能满足在高性能系统中实现各种复杂逻辑功能的需要，被广泛应用于数据处理、图形处理、空间技术、军事装备及通信、自动控制等领域。

ISP技术是美国Lattice公司于1991年率先推出的，该公司将ISP技术应用到高密度可编程逻辑器件(HDPLD)中，形成了ispLSI系列高密度在系统可编程逻辑器件。目前，该公司生产的ispLSI器件有4个系列：

① 基本系列ispLSI1000，适用于高速编码、总线管理等；

② 高速系列ispLSI2000，该系列I/O端口数较多，适用于高速计数、定时等场合，并可用作高速RISC/CISC微处理器的接口；

③ 高密系列 ispLSI3000，该系列是集成密度最高的系列，能实现非常复杂的逻辑功能，适用于数字信号处理、图形处理、数据压缩以及数据加密、解密等；

④ 模块化系列 ispLSI6000，该系列带有存储器和寄存器/计数器，适用于数据处理、数据通信等。

(2) ispGAL 器件

ispGAL 系列器件，是把 ISP 技术引入到标准的低密度系列可编程逻辑器件中形成的 ISP 器件。例如，ispGAL22V10 就是把流行的 GAL22V10 与 ISP 技术相结合形成的产品，在功能和结构上与 GAL22V10 完全相同。ispGAL22V10 的传输时延低于7.5ns，系统速度高达 111MHz，不仅适合于高速图形处理和高速总线管理，而且由于它的每个输出单元平均能容纳 12 个乘积项，最多的单元可达 16 个乘积项，因而更适用于状态控制、数据处理、通信工程、测量仪器等。此外，用它还可以非常容易地实现诸如地址译码器之类的基本逻辑功能。ispGAL22V10 的 4 个在系统编程控制信号 SDI(串行数据输入)，MODE(方式选择)，SDO(串行数据输出)和 SCLK(串行时钟)巧妙地利用了 GAL22V10 的 4 个空脚，从而使两种器件的引脚相互兼容。ispGAL22V10 的在系统编程电源为＋5V，无需外接编程高压电源。在系统编程次数可达 1 万次以上。

(3) ispGDS 器件

ispGDS(在系统可编程数字开关)是 ISP 技术与开关矩阵相结合的产物。它标志着 ISP 技术已从系统逻辑领域扩展到系统互连领域。ispGDS 器件能提供的一种独特功能是，在不拨动机械开关或不改变系统硬件的情况下，快速地改变或重构印制电路板的连接关系。ispGDS 系列的器件非常适合于重构目标系统的连接关系，它使系统硬件可以通过软件控制进行重构而无需人工干预。此外，由于这种器件的传输时延短，所以，还非常适用于高性能的信号分配与布线。ispGDS系列产品具有多种矩形尺寸和封装形式，使用十分方便。

2. ispLSI 逻辑器件的结构

ISP 逻辑器件中最早问世的、最具代表性的是 ispLSI 逻辑器件。ispLSI 系列器件是基于与、或阵列结构的复杂 PLD 产品。芯片由若干巨块组成，巨块之间通过全局布线区 GRP 连接起来，每个巨块包括若干个通用逻辑块 GLB、输出布线区 ORP、若干输入/输出引脚和专用输入引脚等。下面以 ispLSI1000 系列的 ispLSI1016 为例，对 ispLSI 的结构予以介绍。ispLSI1016 是采用 E^2CMOS 工艺制造、PLCC(塑料有引脚芯片载体)封装的器件。芯片共有 44 个引脚，其中 38 个 I/O 引脚；集成密度为 2000 等效门，每片含 64 个触发器和 32 个锁存器；最高工作频率为 110MHz。其功能框图和引脚排列图分别如图 8.39(a)和(b)所示。

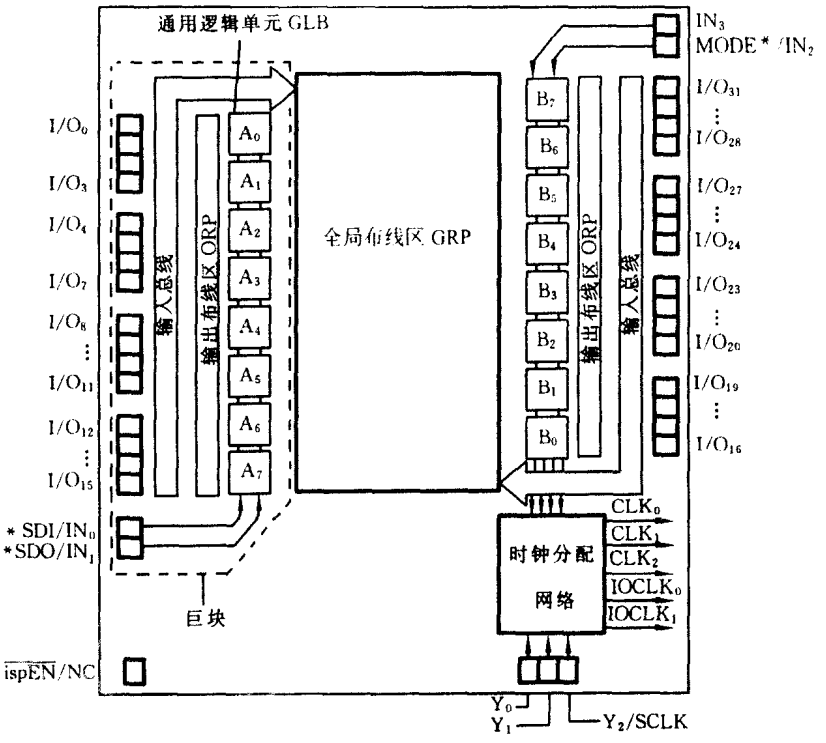


图 8.39　ispLSI1016 的功能框图和引脚排列图

从图 8.39(a)可知,该器件由两个巨块、一个全局布线区和一个时钟分配网络构成。其中每个巨块包括:8 个通用逻辑单元 GLB,16 个 I/O 单元,两只专用输入引脚(IN_0,IN_1或IN_2,IN_3),一个输出布线区 ORP 及 16 位输入(通过 I/O 单元以后)总线。图中,$A_0 \sim A_7$及$B_0 \sim B_7$是 16 个相同的通用逻辑块 GLB,器件型号 ispLSI1016中的"16"表示器件含有 16 个 GLB。下面对各主要功能块进行简单介绍。

(1) 全局布线区 GRP

全局布线区 GRP 位于两个巨块之间。除了经过各个 I/O 单元的输入信号由 16 位输入总线送至 GRP 之外,各 GLB 的输出在送往输出布线区的同时也送往 GRP。GRP 用许许多多的 E^2CMOS 单元实现上述信号和各个 GLB 输入之间的灵活互连,将所有片内逻辑联系在一起,供设计者使用,设计者可以根据需要方便地实现各种复杂的逻辑功能。

(2) 通用逻辑块 GLB

通用逻辑块 GLB 是 ispLSI 器件最基本的逻辑单元,它由与阵列、乘积项共享阵列、输出逻辑宏单元和控制逻辑电路组成。其逻辑结构如图 8.40 所示。

由图可知,ispLSI1016 的与阵列有 18 个输入,其中 16 个来自全局布线区 GRP,2 个为专用输入(被同一巨块内的 8 个 GLB 共用)。18 个输入经过输入缓冲器后形成 18 个输入信号的原信号和非信号,送至 20 个与门的输入端,其中,含有 $18 \times 2 \times 20$ 个 E^2CMOS 单元,这些构成了 GLB 中的可编程与阵列,通过编程可形成 20 个与项(又称乘积项 PT)。

20 个与项 $PT_0 \sim PT_{19}$被分组送到 4 个或门的输入端,对 4 个或门的输出进行可编程"线或"后,再送到输出逻辑宏单元中的可重构触发器。该部分被称为乘积项共享阵列。

输出宏单元中有 4 个可重构触发器,由 4 个数据选择器 MUX 分别选择 GLB 的 4 个输出 $O_3 \sim O_0$为组合输出或者寄存器输出。组合电路可有"与或"和"异或"两种方式,触发器也可组态为 D、T、JK 等形式。

输出宏单元中 4 个触发器的时钟是连在一起的,因此,同一 GLB 中的触发器必须同步工作。但是通过控制逻辑可使触发器的时钟是全局时钟 $CLK_0 \sim CLK_3$,也可以是片内与项产生的与项时钟 PT CLK,且时钟的极性也是可选的。从而使不同 GLB 中的触发器可以使用不同的时钟。同样,4 个触发器的复位信号是相连的,但复位信号是可选的,这就使得同一 GLB 中的 4 个触发器只能同时复位,而各 GLB 中的触发器则可以不同时复位。

(3) 输出布线区 ORP

输出布线区 ORP 是介于 GLB 和输入/输出单元 IOC 之间的可编程互连阵列,其可编程连接关系如图 8.41 所示。

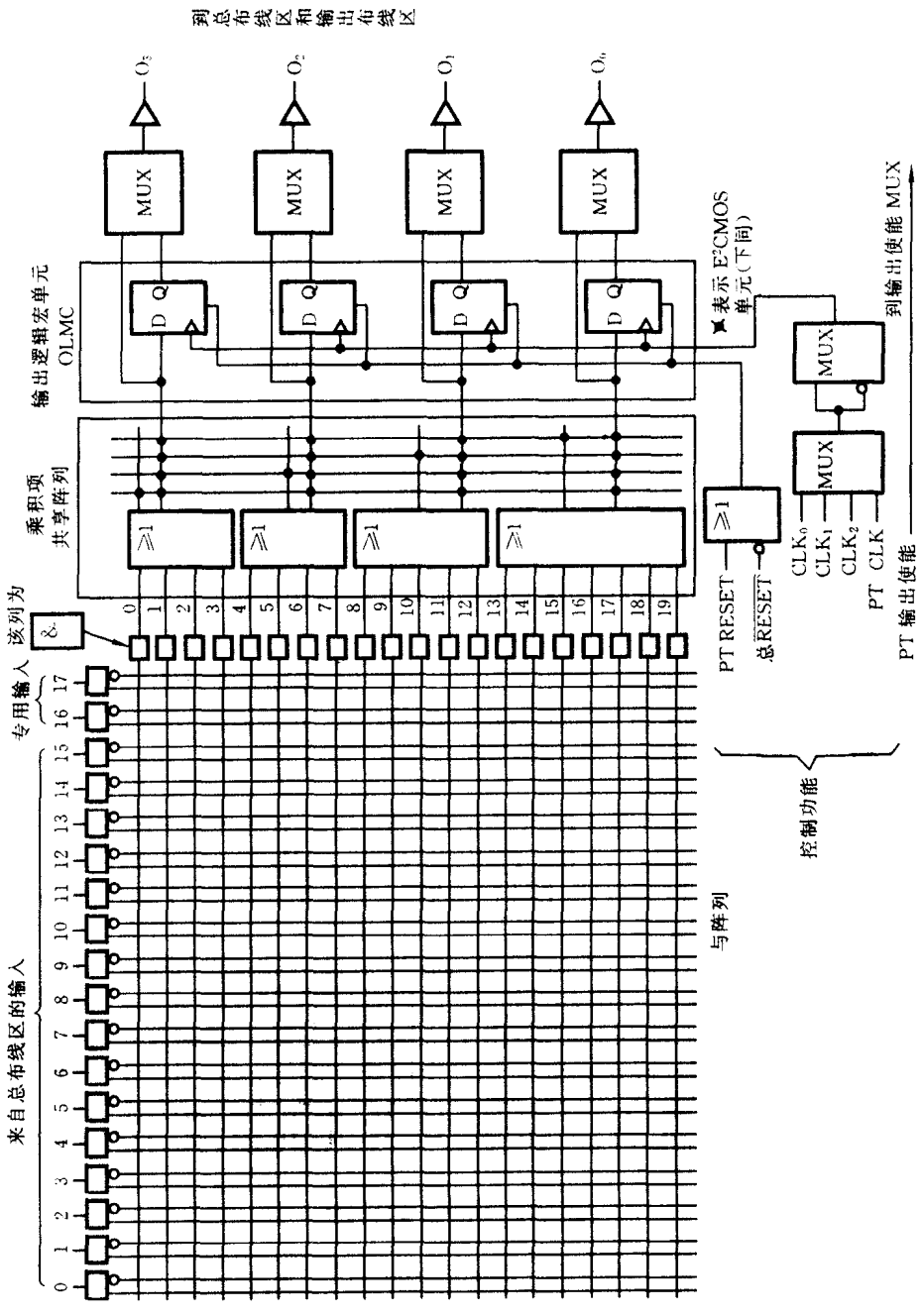


图 8.40　GLB 的逻辑结构图

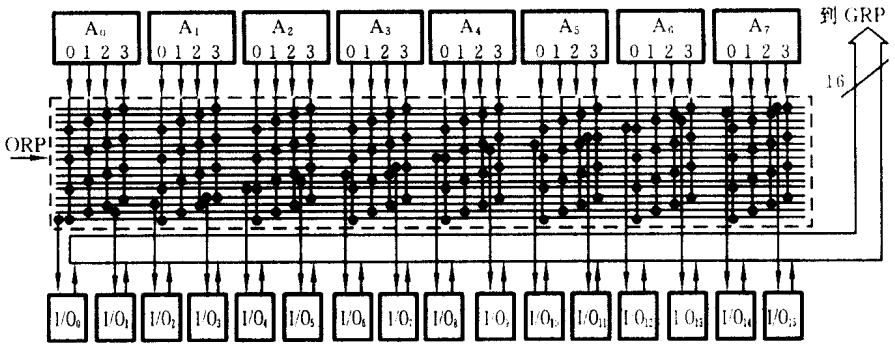


图 8.41 ORP 的可编程连接

阵列的输入是 8 个 GLB 的 32 个输出端，阵列有 16 个输出端，分别与相应侧的 16 个 IOC 相连。通过对 ORP 编程，可以将任何一个 GLB 的输出灵活地送到 16 个 IOC 中的任何一个(每个 GLB 有 4 个输出，每个输出通往 4 个 IOC 中的一个)。显然，这种结构的最大特点是 GLB 与 IOC 之间没有一一对应的关系，因而可将对 GLB 的编程和对外部引脚的连接分开进行，从而可以在不改变外部引脚排列的情况下，修改芯片内部的逻辑设计。

为减少传输延迟，提高工作速度，每个 GLB 的 4 个输出中有两个输出可跨过 ORP 直接通向固定的 IOC，这种方式称为旁路连接。其旁路连接方式如图8.42所示。

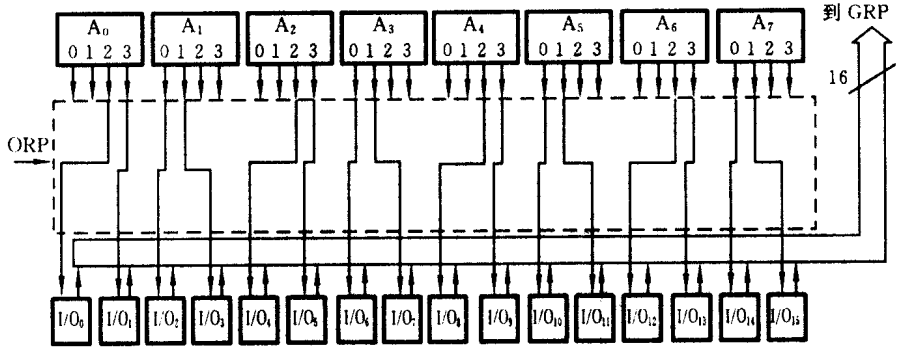


图 8.42 ORP 的旁路连接

在 ORP 旁边还有 16 条通向 GRP 的总线。IOC 可以使用它，GLB 的输出也可以通过 ORP 使用它。

(4) 输入/输出单元 IOC

输入/输出单元 IOC 用于将输入信号、输出信号或输入/输出双向信号与具体

的 I/O 管脚相连接形成输入、输出、三态输出的双向 I/O 口，具体由控制输出三态缓冲器使能端的 MUX 来选择。IOC 工作于输入状态时，包括有输入缓冲、锁存输入及寄存器输入；IOC 工作于输出状时，包括有输出缓冲、反向输出缓冲及三态输出缓冲；IOC 工作于双向状态时，则有双向 I/O 及带有寄存器的双向I/O。各种 I/O组态与 GLB 组态相组合，可构成几十种电路方式。

(5) 巨块输出使能控制电路

巨块是 GLB、ORP 及 IOC 等的总称。巨块内 8 个 GLB 中的与项 PT_{19}都能用作输出使能控制。8 个 PT_{19}接到一个 8 选 1 输出使能数据选择器 OEMUX 的数据输入端，通过对 3 个选择输入的“熔丝”值编程，可选择其中一个 PT_{19}作为巨块内 16 个 IOC 的公共输出使能控制信号。该方法的优点是避免了每个需要三态输出的 GLB 皆要产生 OE 信号，从而有可能正好利用某个不用作逻辑项的 PT_{19}作为巨块的公共 OE，而让其他 7 个 PT_{19}作逻辑项使用。

(6) 时钟分配网络

ispLSI1016 的时钟分配网络如图 8.43 所示。GLB B0 的 4 个输出 $O_0 \sim O_3$能分别作为 CLK_1，CLK_2，$IOCLK_0$及 $IOCLK_1$时钟，这 4 个时钟是用户定义的内部时钟，CLK_0是外部时钟。

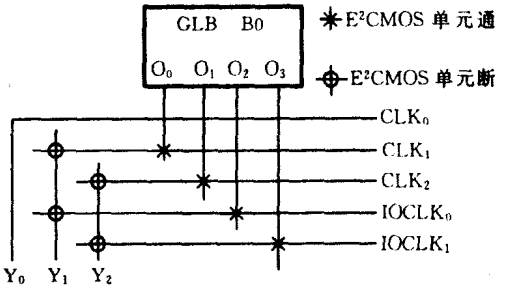


图 8.43　时钟分配网络

8.3.3　ISP 器件的开发软件与设计流程

1. ISP 器件的开发软件

目前，常用的 ISP 器件开发软件有 PDS 软件，Synario 软件，ISP Synario System软件等。

(1) PDS 软件

PDS 是设计工具软件，它向用户提供基于 PC 机的设计输入与器件之间的映

射关系。该软件可以单独使用，也可以和 ABEL、ViewLogic、Synario 等工具软件配合作用。

利用 PDS 进行设计时可以采用逻辑描述方式或宏方式，为了简化设计过程，应尽量使用宏方式。逻辑描述方式是最基本的也是最低一级（门、触发器级）的方式。而宏（MACRO）是一组预先编好，存放在库中的逻辑方程，每个宏器件代表一个逻辑模块，在设计中可作为逻辑器件调用。宏可分为标准宏和用户宏，它们存于不同的库中。PDS 有 3 种库：系统库、设计库和用户库。在系统库中存放了诸如计数器、寄存器、译码器、选择器和算术运算电路等各种标准宏 200 多个。设计库是建于某个设计文件中的临时库，它所保存的宏是该设计中用到的宏。用户宏是用户自建的宏，它们是一些使用者在设计中需要多次用到的非标准电路。用户宏存于用户库中，设计时再调入设计库，从而达到节省时间、简化设计过程的目的。

PDS 的使用环境：硬件要求 486 以上机型，内存大于 4MB；软件要求 Windows 3.1 和 MS-DOS3.1。软件一经启动，就按其程序安排自动安装。

(2) SYNARIO 软件

SYNARIO 是美国 Lattice 公司和 Data I/O 公司合作开发的一种运行于 PC 机 Windows 环境下的通用电子设计工具软件。该软件继承和发扬了 PLD 器件开发软件 ABEL 的特点。它有一个包括各种常用逻辑器件和模块的较完善的宏库。设计中能进行逻辑图输入和 ABEL 硬件描述语言输入，并包括功能模拟显示和波形显示。此外，该软件还具有将多个 ABEL 设计文件编译成高密度 PLD 设计的能力，从而开拓出一条将多个低密度 PLD 设计升级成为高密度 PLD 设计的捷径。

(3) ISP Synario System 软件

ISP Synario System 软件是一个基于 SYNARIO 的完整的在系统编程设计系统，它具有设计输入、编译、逻辑模拟等功能，支持 ispLSI 器件、ispGAL 器件、ispGDS 器件以及全系列 GAL 器件。该软件系统包括了 SYNARIO 的全部功能，同样有一个较为完善的宏库，库中的宏是用 ABEL 语言编写的。调用宏通常采用逻辑图输入方式，即将这些宏做成电路符号，然后像逻辑元件那样画在逻辑图中，并绘出它们之间的连线以及各个输入/输出缓冲电路。此外，ISP Synario System 采用混合输入方式，允许在同一器件的设计中同时采用逻辑图、逻辑方程、真值表和状态图输入，从而使设计输入十分方便。

2. 设计流程

利用 ISP 器件开发软件进行设计一般可以分为逻辑设计规划、设计输入、设计检验、布局布线、逻辑功能仿真、熔丝图生成和下载编程等步骤。下面以 ispLSI 器

件为例简单说明。

(1) 逻辑设计规划

逻辑设计规划的目的是选择合适的 ispLSI 器件实现预定功能。进行设计规划时，首先定义 I/O 端口，以便考虑器件的 I/O 单元是否够用。然后进行任务划分，即将要求完成的设计任务分配到各个通用逻辑块 GLB 中。最后统计出所需要的 GLB 数目和 I/O 单元数目，作为器件型号选择的依据。

确定了 I/O 单元数和 GLB 数之后，便可选择合适的 ispLSI 器件。当所需的 GLB 和 I/O 单元数量不超过芯片所能提供资源的 80%时，用上述软件布线和布局是非常方便的。

如果使用 ISP Synario System 软件，每个宏或方程所占用的 GLB 是由软件自动布放的。若使用 PDS 软件则需设计者自行安排，即在编程前将所设计电路中的各个宏和方程合理地分配到各 GLB 单元。

(2) 设计输入

所谓设计输入就是将设计者所设计的电路，按照开发软件要求的某种形式表达出来，并输入计算机中。ISP Synario System 软件有逻辑图和硬件描述语言(HDL)两种输入方式。其中，语言输入又可分为逻辑方程、真值表和状态图。而 PDS 软件只有语言输入方式。两种开发软件中所用的语言都基于优秀的通用编程语言 ABEL，两者存在很多相似之处，但编写方式不同。ISP Synario System 是用一个或多个文件输入，源文件格式与 ABEL 源文件格式相同；而 PDS 源文件的格式没有统一，它是在其设计管理窗口上按 GLB 和 IOC 单元逐个地编写输入的。

使用 ABEL-HDL 对逻辑电路进行描述时，应正确使用逻辑变量、逻辑常量、运算符、赋值符和器件的描述方法。描述一个逻辑电路的方式有逻辑方程、真值表和状态图，为了简化真值表和状态图，在电路描述时可用数组表示。

(3) 设计检验

在完成设计输入后，应对输入的文件进行检验。设计检验一般包括如下内容。

① 语法检验：列出语法错误。

② 设计规划检验：要求设计资源不超过 GLB 或 IOC 的限制，如输入/输出数等。

③ 逻辑最小化：简化所有逻辑方程或用户自建的宏，使耗费的资源达到最少。

④ 逻辑适配：确定最小化的逻辑能否与所安排的 GLB 或 IOC 适配，然后将逻辑映射到相应的单元中并提供输入到熔丝图的程序。此外，对用于布局布线的全局网表进行修正。

⑤ 全局设计规划检验：保证总的设计资源不超出器件资源或规划的限制，并查出网络中是否有漏连或信号是否有双重来源等错误。

检验结束后，设计者应根据检验结果中报告的问题修改设计方案。

(4) 布局布线

在设计检验通过后，便可进行布局布线。布局布线工作是由软件自动完成的，它能以最优方式对逻辑元件布局和准确地实现元件间的互连。

布线后生成一个布线报告，提供关于单元输入、GLB 平均输入和输出数、扇出数和复用信号，以及设计中 GLB 和 IOC 的使用情况等信息。

(5) 逻辑模拟

逻辑模拟又称为逻辑功能仿真，它包括功能模拟和定时模拟。

PDS 中没有模拟程序，需要用其他软件处理。ISP Synario System 有自己的模拟程序，其模拟过程为：按软件要求建立模拟测试向量源文件、输入测试向量、编译测试向量文件、测试向量模拟和波形图显示。

(6) 熔丝图生成

熔丝图生成是设计过程的重要目标，生成 JEDEC 文件(熔丝图)。在生成熔丝图过程中，凡设计中未使用的 I/O 端，皆被自动接上有源上拉电阻，这样处理有利于降低功耗和减少噪声。

(7) 下载编程

下载编程就是将设计生成的熔丝图文件 JEDEC 装入 ispLSI 器件中。为使编程迅速和对多片 ISP 器件共用一条下载电缆编程，可使用 ispCODE 软件和 ISP 菊花链下载软件 IDCD。

实际编程时，开发软件在收到设计者发出的 DOWNLOAD(下载)命令后，首先检查编程电缆是否接上，计算机与器件之间通信是否正常，然后执行编程操作，并对编程结果进行检验。检验通过后，器件便可在系统中运行。

以上对常用的 ISP 器件开发软件及设计流程进行了简单介绍。对有关 ISP 器件的编程原理和开发软件的使用等请参考相关书籍。

习 题 八

8.1 可编程逻辑器件有哪些主要特点？

8.2 常用 PLD 器件有哪几种主要类型？

8.3 用只读存储器(ROM)设计一个 3 位二进制平方器，指出实现该平方器需要的 ROM 容量。

8.4 用可编程序逻辑阵列(PLA)实现 4 位二进制码到 Gray 码的转换。

8.5 用 PLA 和 D 触发器设计一个三进制可逆计数器。当 x=1 时，实现加 1

计数；当 x＝0 时，实现减 1 计数。当计数中有进位或借位产生时，电路输出 Z 为 1，否则 Z 为 0。

8.6 可编程阵列逻辑(PAL)和可编程逻辑阵列(PLA)在结构上的主要区别是什么？

8.7 PAL 可分为哪几种基本类型？实现组合逻辑函数时通常采用哪种类型的 PAL？

8.8 用 PAL 实现 4 位二进制并行加法器的功能。

8.9 用 GAL16V8 实现基本 R-S 触发器和 D、T、J-K 触发器的基本功能。

8.10 简述用 GAL 进行逻辑设计的一般流程。

8.11 简述 ISP 技术的主要特点。

8.12 ISP 逻辑器件有哪几种常用类型？各适宜于哪些场合？

8.13 ispLSI 器件一般包括哪些主要部分？

8.14 ispLSI 器件中的 GLB 由哪几部分组成？各部分的主要作用是什么？

8.15 ispLSI 器件中的输出布线区 ORP 有何主要特点？有哪两种连接方式？

8.16 简述采用 ISP 技术的设计流程。



第9章 综合应用举例

前面分别对有关数字系统逻辑设计的基本知识、基本理论、常用器件，以及逻辑电路分析与设计的基本方法进行了系统的介绍。本章将结合实际应用需求，介绍6个逻辑设计实例。通过实例的设计，以进一步提高综合运用所学知识解决实际问题的能力。

9.1 简单运算器设计

9.1.1 设计要求

设计一个能实现两种算术运算和两种逻辑运算的4位运算器。参加运算的4位二进制代码分别存放在4个寄存器A、B、C、D中，要求在选择变量控制下完成如下4种基本运算：

① 实现A加B，显示运算结果并将结果送寄存器A；

② 实现A减B，显示运算结果并将结果送寄存器B；

③ 实现A与C，显示运算结果并将结果送寄存器C；

④ 实现A异或D，显示运算结果并将结果送寄存器D。

9.1.2 功能描述

根据设计要求，为了区分4种不同的运算，需设置2个运算控制变量。设运算控制变量为S_1和S_0，可列出运算器的功能，如表9.1所示。

根据功能描述可得出运算器的结构框图，如图9.1所示。整个电路可由传输控制电路、运算电路、显示电路3部分组成。

表 9.1　运算器的功能

S_1S_0	功能	说明
0 0	A+B→A	A 加 B,结果送至 A
0 1	A−B→B	A 减 B,结果送至 B
1 0	A · C →C	A 与 C,结果送至 C
1 1	A⊕D →D	A 异或 D,结果送至 D

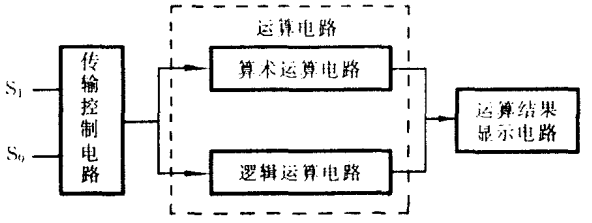


图 9.1　运算器的结构框图

9.1.3　电路设计

1. 运算电路

运算电路的功能是实现指定的两种算术运算和两种逻辑运算。假定选择 4 位并行加法器实现加、减运算,由第 7 章有关内容可知,用 1 片 74LS283 和 4 个异或门可构成相应电路。两种逻辑运算可由 4 个与门和 4 个异或门实现。据此,可画出运算电路的逻辑电路,如图 9.2 所示。

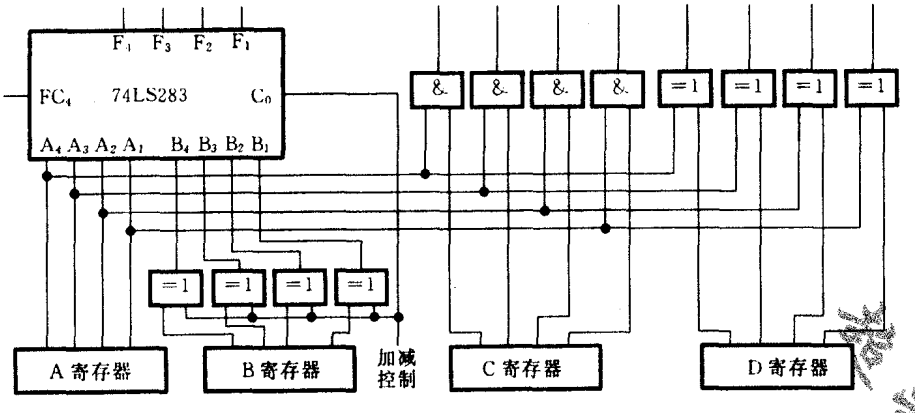


图 9.2　运算电路的逻辑电路

2. 传输控制电路

传输控制电路的功能是根据运算控制变量的取值，从运算电路输出的 4 种不同运算结果中选出一种结果，一方面送至输出端显示，另一方面送至目的寄存器保存。

① 运算结果的选择可采用 2 片双 4 路数据选择器 74LS153 实现，用运算控制变量 S_1 和 S_0 作为数据选择器 74LS153 的选择变量，即可实现对 4 种不同运算结果的选择。

② 假定 4 个寄存器 A、B、C、D 均采用 4 位双向移位寄存器 74LS194，则传输控制电路应实现对寄存器工作方式和工作脉冲的控制，即在运算控制变量 S_1 和 S_0 的不同取值下，令相应目的寄存器处在并行数据输入方式($S_1S_0=11$)且工作脉冲(CP)有效。此外，还需形成加、减运算的控制信号。为了区分 4 个不同的目的寄存器，可用 2-4 线二进制译码器 74LS139 对控制变量 S_1 和 S_0 进行译码后形成相应的传送控制信号。传输控制电路的逻辑电路如图 9.3 所示。

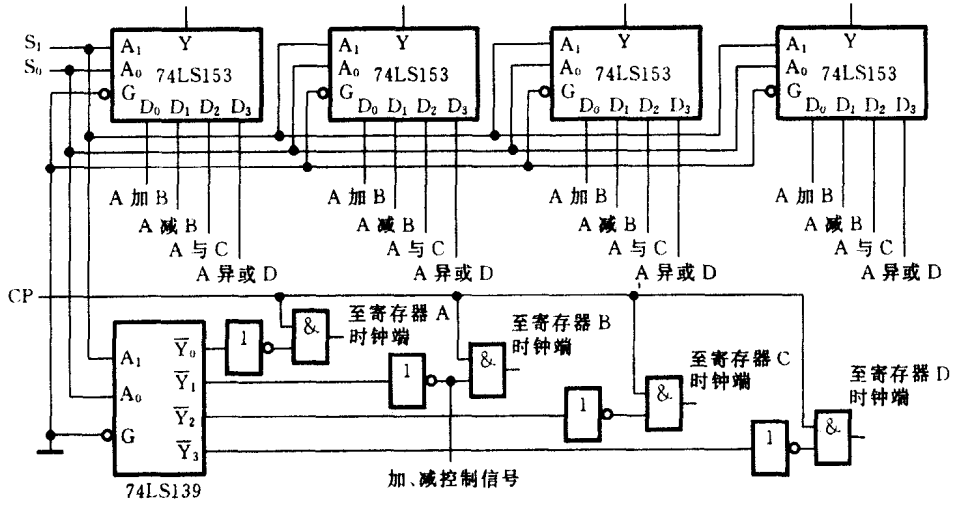


图 9.3 传输控制电路的逻辑电路

3. 运算结果显示电路

假定采用发光二极管(BT-204)显示运算结果，则显示电路可由 4 个发光二极管、4 个电阻和 4 个反相器组成。将图 9.3 所示传输控制电路中的 4 个 4 路数据选择器输出接至 4 个反相器的输入，当 4 路数据选择器输出为高电平时，反相器的输出为低电平，相应发光二极管被点亮。

该运算电路的完整逻辑电路如图 9.4 所示。

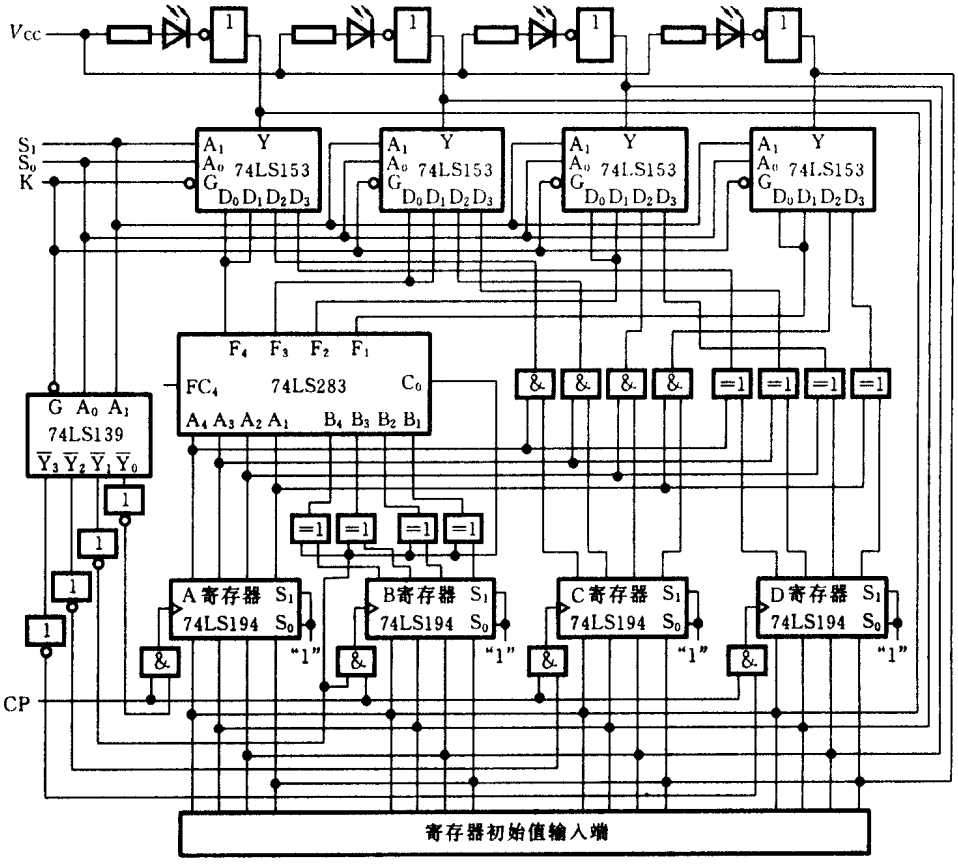


图 9.4　运算电路的完整逻辑电路

9.2　时序信号发生器设计

9.2.1　设计要求

时序信号是使计算机能够准确、迅速、有条不紊地工作的时间基准。CPU 每取出并执行一条指令所需要的时间通常叫做一个指令周期，一个指令周期一般由若干个 CPU 周期(通常定义为从内存中读取一个指令字的最短时间，又称为机器周期)组成。时序信号的最简单体制是节拍电位－节拍脉冲二级体制。一个节拍电位表示一个 CPU 周期的时间，在一个节拍电位中又包含若干个节拍脉冲，节拍

脉冲表示较小的时间单位。时序信号发生器的功能就是产生一系列的节拍电位和节拍脉冲，它一般由时钟脉冲源、时序信号产生电路、启停控制电路等部分组成。

要求设计一个用于实验系统的简单时序信号发生器，具体功能如下：

① 由时钟脉冲源提供频率稳定的方波信号作为系统的主频信号（即时序发生器的输入信号），要求系统的主频信号可以在 2MHz、1MHz、500kHz、250kHz 等 4 种不同频率之间进行选择。

② 规定一个 CPU 周期由 4 个时钟周期组成，即要求在一个 CPU 周期中产生 4 个等间隔的节拍脉冲。

③ 为了保证系统可靠地启动和停止，必须对时序信号进行有效的控制。此外，由于启动信号和停止信号都是随机产生的，考虑到节拍脉冲的完整性，所以要求时序信号发生器启动时从第一个节拍脉冲的前沿开始工作，停止时在第四个节拍脉冲的后沿关闭。

9.2.2 功能描述

根据设计要求可知，时序信号发生器由时钟脉冲源、时序信号产生电路、启停控制电路 3 部分组成，其结构框图如图 9.5 所示。

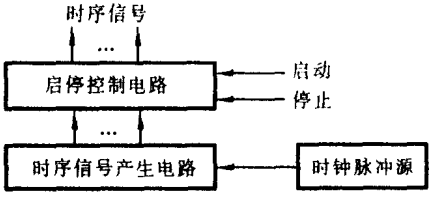


图 9.5 时序信号发生器结构框图

假定节拍脉冲信号用 T_1、T_2、T_3、T_4 表示，可画出时序信号发生器产生的波形，如图 9.6 所示。

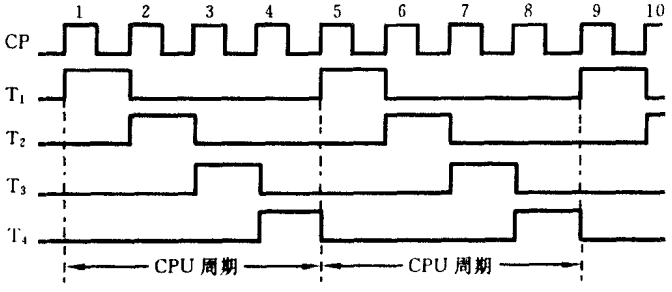


图 9.6 时序信号的波形

9.2.3　电路设计

1. 时钟电路(时钟源)

由于要求时序发生器的输入信号能在 2MHz、1MHz、500kHz、250kHz 这 4 种不同频率的方波信号之间进行选择，所以时钟电路应由信号源和分频电路两部分组成。为保证频率的稳定度和精度，信号源可选用石英晶体振荡器，而分频电路可用计数器实现。假定信号源用特定频率为 4MHz 的石英晶体，分频电路采用 4 位二进制同步计数器 74LS161，可设计出时钟电路，如图 9.7 所示。图中，用信号源产生的 4MHz 的方波信号作为计数器 74LS161 的时钟脉冲，计数器输出 Q_0、Q_1、Q_2、Q_3 可分别提供 2MHz、1MHz、500kHz、250kHz 这 4 种不同频率的方波信号。

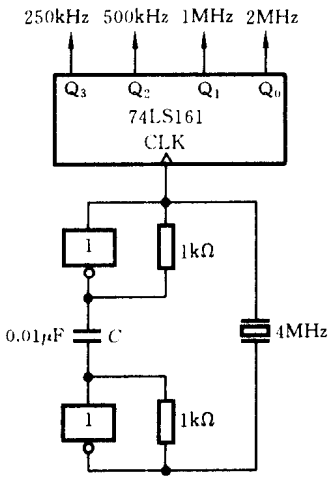


图 9.7　时钟电路

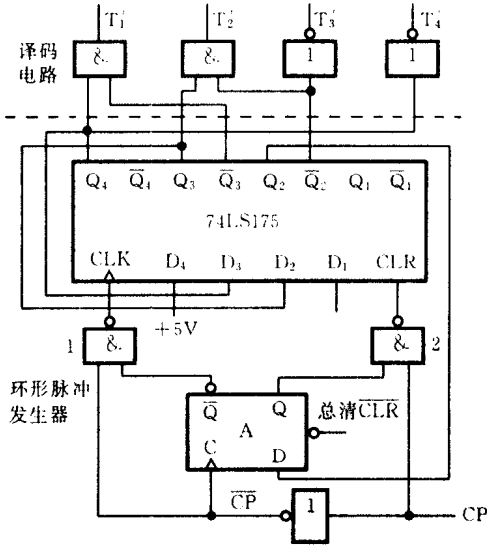


图 9.8　时序信号产生电路

2. 时序信号产生电路

时序信号产生电路一般由一个环形脉冲发生器和一个译码逻辑电路组成。环形脉冲发生器通常采用循环移位寄存器的形式，其作用是产生一组间隔和宽度相等或不相等的脉冲序列；译码逻辑电路的作用是通过对环形脉冲发生器的输出信号进行译码，产生所需要的节拍脉冲。假定采用 4D 触发器 74LS175 构成循环移位寄存器，一个满足本设计要求的时序信号产生电路如图 9.8 所示。

图中，虚线下方为环形脉冲发生器逻辑电路。该电路的工作过程如下。

当系统发出总清信号($\overline{CLR}$)后，触发器 A 置为“1”状态，与非门 2 被打开，系统时钟(CP)的第一个正脉冲通过与非门 2 使 74LS175 清“0”。经过半个时钟周期的延迟(CP 第一个正脉冲的下降沿，即$\overline{CP}$第一个脉冲的上升沿)，触发器 A 由“1”状态翻转为“0”状态，与非门 1 被打开，再经过半个时钟周期的延迟，CP 的第二个正脉冲作为 74LS175 的移位脉冲信号(注意：当触发器 A 为“0”状态时，与非门 1 的输出为 CP)，使 74LS175 的 $Q_4Q_3Q_2$变为“100”状态。此后，CP 的第三个、第四个正脉冲连续通过门 1 形成移位脉冲信号，使 $Q_4Q_3Q_2$相继变为“110”、“111”状态。当Q_2变为“1”状态时，触发器 A 的输入端 D 为“1”，因而在 CP 第四个正脉冲的下降沿又将触发器 A 置为“1”状态，与非门 2 再次被打开，CP 的第五个正脉冲通过与非门 2 使 74LS175 再次清“0”，开始下一轮循环。

图 9.8 中，虚线上方的电路为节拍脉冲译码电路。根据对节拍脉冲信号的设计要求和环形脉冲发生器 $Q_4Q_3Q_2$产生的输出信号，节拍脉冲译码电路的输出 T_1'、T_2'、T_3'、T_4'可表示为

$$T_1'=Q_4\cdot\overline{Q}_3$$
$$T_2'=Q_3\cdot\overline{Q}_2$$
$$T_3'=Q_2$$
$$T_4'=\overline{Q}_4$$

时序信号产生电路的波形如图 9.9 所示。

3. 启停控制电路

系统接通电源并发出总清信号($\overline{CLR}$)后，时序信号产生电路会自动产生节拍脉冲信号 $T_1'\sim T_4'$。但是，仅在启动系统运行的情况下，才允许时序发生器发出系统工作所需的节拍脉冲 $T_1\sim T_4$。为此，需要由启停控制电路控制 $T_1'\sim T_4'$的发送。启停控制电路的核心是一个运行标志触发器，当运行标志触发器为“1”时，允许节拍脉冲 $T_1'\sim T_4'$发出，当运行标志触发器为“0”时，节拍脉冲 $T_1'\sim T_4'$被封锁。根据设计要求可设计出启停控制电路，如图 9.10 所示。

由于启动和停止信号都是随机发生的，因此，为了保证输出节拍脉冲信号的完整性，采用了“维持-阻塞型”电路。图中，在运行标志触发器 B 的下方加上了一个 R-S 触发器，并且用环形脉冲发生器 74LS175 的 Q_4作为运行标志触发器 B 的时钟信号，从而保证了时序信号发生器启动时从第一个节拍脉冲的前沿开始工作，停止时在第四个节拍脉冲的后沿终止工作。

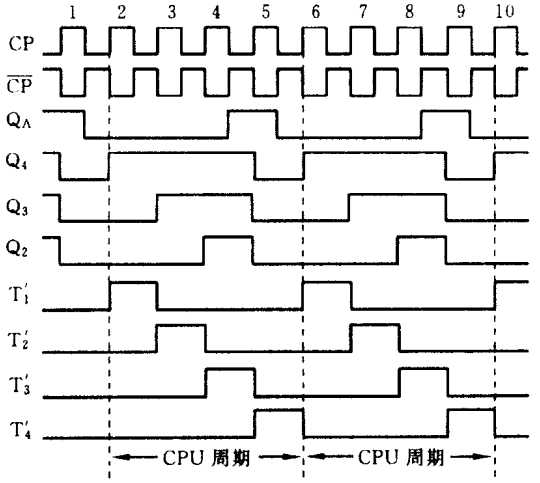


图 9.9　时序信号产生电路的波形

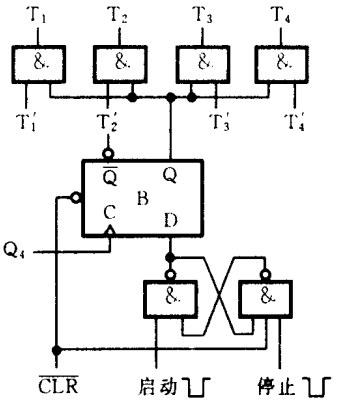


图 9.10　启停控制电路

综合上述设计过程，可得到一个满足设计要求的时序信号发生器完整电路，如图9.11所示。

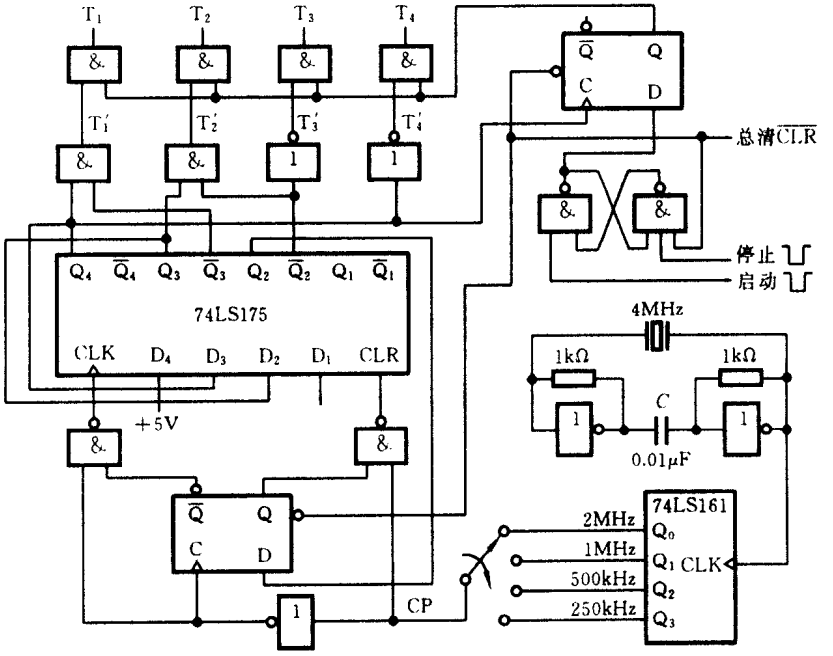


图 9.11　时序信号发生器的完整电路

9.3 地址译码电路设计

9.3.1 设计要求

用 GAL 设计一个微机接口实验台的 I/O 地址译码电路。该实验台允许 CPU 访问 256 个端口，其端口地址范围为 300H～3FFH。所有端口被分为 16 个段，每个段包含 4 个接口芯片，每个芯片内部具有 4 个端口。该 I/O 地址译码电路的输入/输出关系如图 9.12 所示，电路输入端接受 10 个地址信号 A_9～A_0，3 个控制信号 AEN(当 AEN=0 时，处于非 DMA 操作，此时译码有效)、$\overline{IOR}$(读端口)和$\overline{IOW}$(写端口)；输出端产生要求的寻址信号，以及传输控制电路 74LS245 的通\断控制信号 $\overline{E}$(仅当端口地址处在 300H～3FFH 范围内时，才使 $\overline{E}$ 有效，打开 74LS245)和传输方向控制信号 DIR(取值与 IOR 相同，当 DIR=0 时，数据从 B 端传到 A 端，当 DIR=1 时，数据从 A 端传到 B 端)。该实验台要求通过 DIP 开关实现段选。

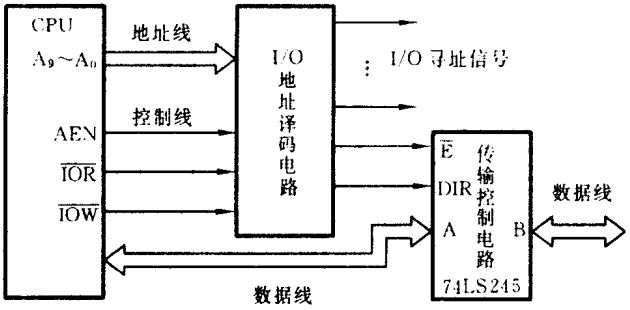


图 9.12 I/O 地址译码电路的输入/输出关系

9.3.2 功能描述

1. 地址线分配

根据给定端口地址范围可知，地址线 $A_9=A_8=1$，A_7～A_0可在 00H～FFH 范围内变化。一种满足寻址要求的地址分配方案是：令地址线 A_7～A_4进行组合实现段选，A_3和 A_2进行组合实现段内片选，并由 A_1和 A_0进行组合实现片内端口寻址(不参加译码，令其直接与接口芯片相连接)。10 根地址线分配如表 9.2 所示。

表 9.2　译码电路地址线分配表

地　址　线	$00A_9A_8$	$A_7A_6A_5A_4$	A_3A_2	A_1A_0
十六进制码	3	0～F	0～3	0～3
作　　用		段选	片选	片内端口寻址

2. 功能描述

根据设计要求和地址分配方案，可得到 I/O 地址译码电路的框图，如图 9.13 所示。该电路由两部分组成，第一部分通过 DIP 开关实现段选，当 A_7～A_4的取值与 DIP 设置的段地址相同时，段地址译码电路的输出 F 为 1；第二部分接收地址信号 A_9、A_8、A_3、A_2，控制信号 AEN、$\overline{IOR}$、$\overline{IOW}$和段地址译码电路的输出 F，输出片选信号$\overline{CS_3}$、$\overline{CS_2}$、$\overline{CS_1}$、$\overline{CS_0}$，以及传输控制电路的通\断控制信号 $\overline{E}$ 和传输方向控制信号 DIR。

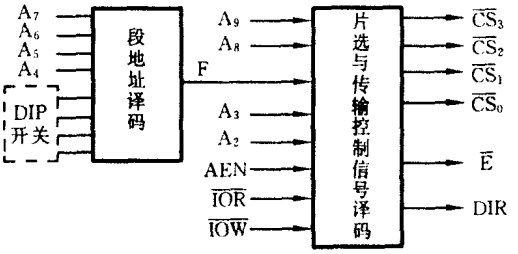


图 9.13　I/O 地址译码电路的框图

9.3.3　电路设计

1. 段地址译码电路

因为要求通过 DIP 开关实现段选，所以段地址译码电路可用 4 个开关、4 个异或门和一个与门实现，逻辑电路如图 9.14 所示。

图中，$F=(A_7\oplus K_3)(A_6\oplus K_2)(A_5\oplus K_1)(A_4\oplus K_0)$，F=1 为有效信号。F 是否为 1，取决于开关 K_3～K_0的状态和地址 A_7～A_4的值。例如，当开关 K_3～K_0全部断开时，A_7～A_4必须全部为 0 才能使 F 为 1。可见，开关 K_3～K_0的 16 种状态组合和地址 A_7～A_4的 16 种取值组合一一对应。

2. 片选与传输控制信号译码电路

由于要求用 GAL 设计 I/O 地址译码电路，所以，在解决了段地址译码后，实际上只需用 GAL 完成片选信号与传输控制信号的译码。由图 9.13 可知，GAL 应

接收地址信号 A_9、A_8、A_3、A_2，段地址译码电路输出信号 F 和控制信号 AEN、$\overline{IOR}$、$\overline{IOW}$共 8 个输入信号，产生片选信号$\overline{CS_3}$、$\overline{CS_2}$、$\overline{CS_1}$、$\overline{CS_0}$，传输控制信号 $\overline{E}$ 和 DIR 共 6 个输出信号。因此，可以选择 GAL16V8 完成片选信号与传输控制信号的译码，并可对 GAL16V8 的引脚按图 9.15 所示进行分配。

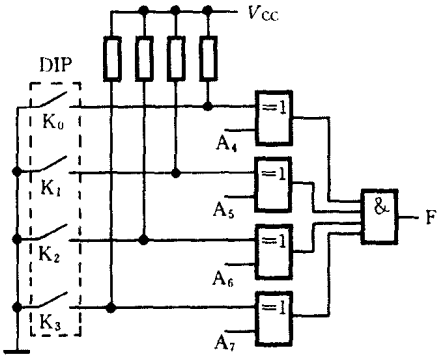


图 9.14 段地址译码电路的逻辑电路

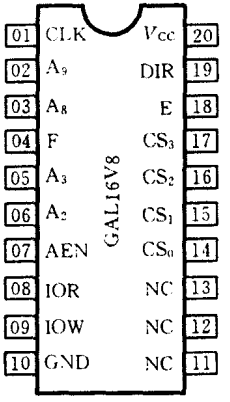


图 9.15 GAL16V8 的引脚分配

假定采用汇编型编程软件 FM 编程，根据 FM 软件要求和片选与传输控制信号译码电路的功能，可利用文本编辑程序编写出 GAL 设计说明书（名为 IODECODER. PLD 的输入源文件），如图 9.16 所示。

```
GAL 16V8
IOADDRESS DECODER
OYXM 08/11/23
IODECODER
NC  A9 A8 F A3 A2 ANE IOR IOW GND
VCC DIR E CS3 CS2 CS1 CS0 NC NC NC
DIR=IOR
/E = A9 * A8 * F * /AEN * /IOR+ A9 * A8 * F * /AEN * /IOW
/CS3 = A9 * A8 * F * /AEN * A3 * A2 * /IOR+A9 * A8 * F * /AEN * A3 * A2 * /IOW
/CS2 = A9 * A8 * F * /AEN * A3 * /A2 * /IOR+A9 * A8 * F * /AEN * A3 * /A2 * /IOW
/CS1 = A9 * A8 * F * /AEN * /A3 * A2 * /IOR+A9 * A8 * F * /AEN * /A3 * A2 * /IOW
/CS0 = A9 * A8 * F * /AEN * /A3 * /A2 * /IOR+A9 * A8 * F * /AEN * /A3 * /A2 * /IOW
DESCRIPTION
```

图 9.16 输入源文件 IODECODER. PLD

利用 FM 对 IODECODER. PLD 进行汇编后，自动生成的熔丝状态分布图文件 IODECODER. PLT，如图 9.17 所示；与熔丝状态分布图对应的 GAL 阵列逻辑图如图 9.18 所示。

```
Polarity Fuse x
AC1 Fuse x
        Output pin 19 Row 0     _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ x _ _ _ _ _ _ _
        Output pin 19 Row 1     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 19 Row 2     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 19 Row 3     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 19 Row 4     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 19 Row 5     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 19 Row 6     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 19 Row 7     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x

Polarity Fuse x
AC1 Fuse x
        Output pin 18 Row 0     x _ _ _ x _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ x _ _ _ x _ _ _ _ _ _
        Output pin 18 Row 1     x _ _ _ x _ _ _ x _ _ _ _ _ _ _ _ _ _ _ _ x _ _ _ _ _ _ _ _ x _ _
        Output pin 18 Row 2     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 18 Row 3     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 18 Row 4     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 18 Row 5     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 18 Row 6     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 18 Row 7     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x

Polarity Fuse x
AC1 Fuse x
        Output pin 17 Row 0     x _ _ _ x _ _ _ x _ _ _ x _ _ _ x _ _ _ _ x _ _ _ x _ _ _ _ _ _
        Output pin 17 Row 1     x _ _ _ x _ _ _ x _ _ _ x _ _ _ x _ _ _ _ x _ _ _ _ _ _ _ _ x _ _
        Output pin 17 Row 2     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 17 Row 3     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 17 Row 4     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 17 Row 5     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 17 Row 6     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 17 Row 7     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x

Polarity Fuse x
AC1 Fuse x
        Output pin 16 Row 0     x _ _ _ x _ _ _ x _ _ _ x _ _ _ x _ _ _ x _ _ _ x _ _ _ _ _ _
        Output pin 16 Row 1     x _ _ _ x _ _ _ x _ _ _ x _ _ _ x _ _ _ x _ _ _ _ _ _ _ _ x _ _
        Output pin 16 Row 2     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 16 Row 3     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 16 Row 4     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 16 Row 5     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 16 Row 6     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 16 Row 7     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x

Polarity Fuse x
AC1 Fuse x
        Output pin 15 Row 0     x _ _ _ x _ _ _ x _ _ _ _ x _ _ x _ _ _ _ x _ _ _ x _ _ _ _ _ _
        Output pin 15 Row 1     x _ _ _ x _ _ _ x _ _ _ _ x _ _ x _ _ _ _ x _ _ _ _ _ _ _ _ x _ _
        Output pin 15 Row 2     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 15 Row 3     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 15 Row 4     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 15 Row 5     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 15 Row 6     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 15 Row 7     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x

Polarity Fuse x
AC1 Fuse x
        Output pin 14 Row 0     x _ _ _ x _ _ _ x _ _ _ _ x _ _ _ x _ _ _ x _ _ _ x _ _ _ _ _ _
        Output pin 14 Row 1     x _ _ _ x _ _ _ x _ _ _ _ x _ _ _ x _ _ _ x _ _ _ _ _ _ _ _ x _ _
        Output pin 14 Row 2     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 14 Row 3     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 14 Row 4     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 14 Row 5     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 14 Row 6     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
        Output pin 14 Row 7     x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x x
SYN Fuse _
AC0 Fuse x
```

图 9.17　熔丝状态分布图文件 IODECODER.PLT

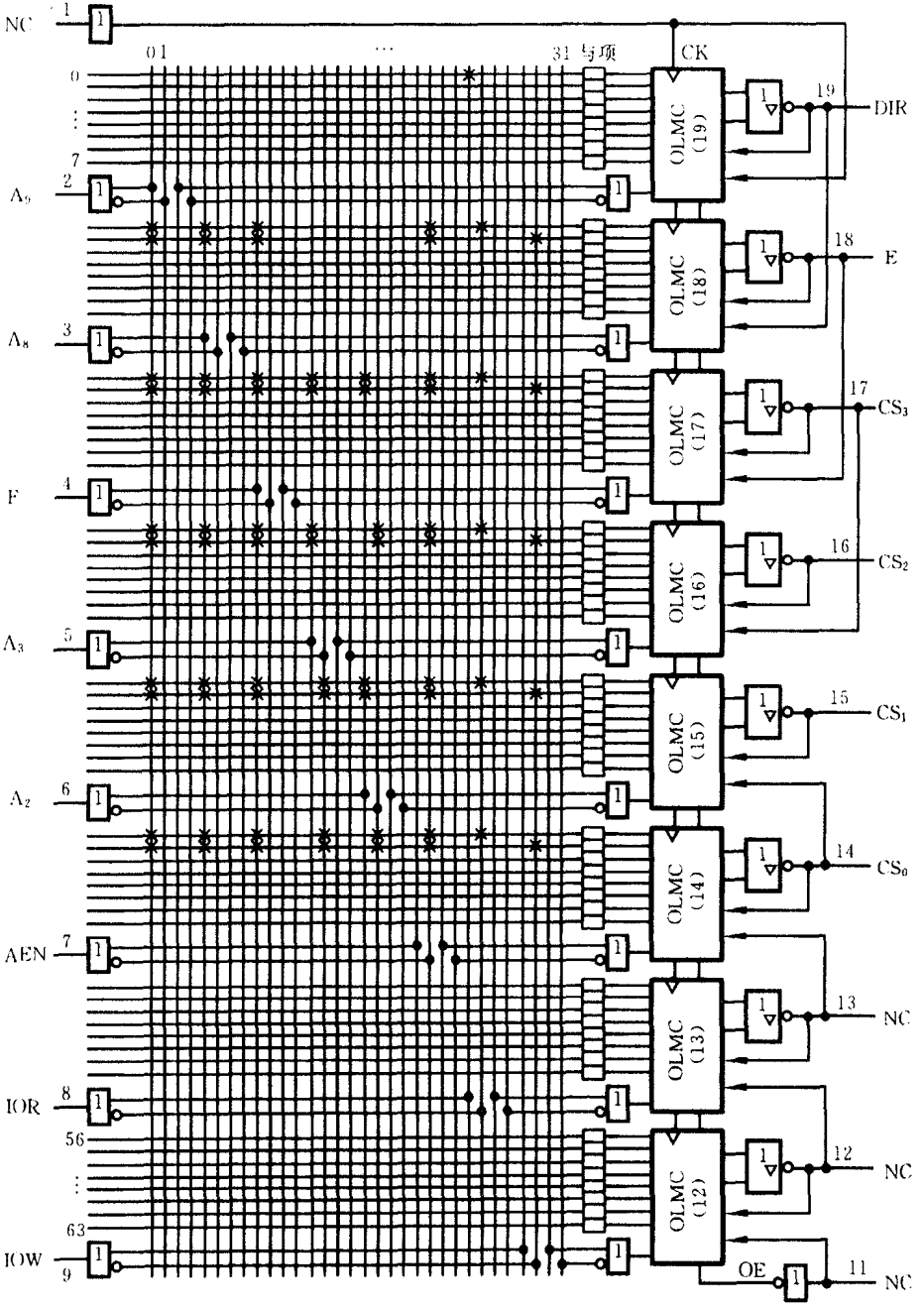


图 9.18　IODECODER 的 GAL 阵列逻辑图

9.4　弹道计时器设计

9.4.1　设计要求

设计一个用来测量手枪子弹等发射物速度的便携式电池供电计时器，这种计时器可用来测定子弹或其他发射物的速度。竞赛射手通常用这种设备来测定装备的性能。

基本操作要求是：射手在两个分别产生起始测量脉冲和终止测量脉冲的光敏传感器上方射出一个发射物，两个光传感器（本例中假定为阴影传感器）分开放置，两者之间的距离已知。发射物在两个传感器之间的飞行时间直接与发射物的速度成正比。如图 9.19 所示，当子弹等发射物从上方经过起始传感器时产生 ST 信号，经过终止传感器时产生 SP 信号。传感器之间的距离 S 是固定的。通过测量子弹等发射物经过传感器之间的时间 t 即可计算出子弹的速度 V，$V=S/t$。

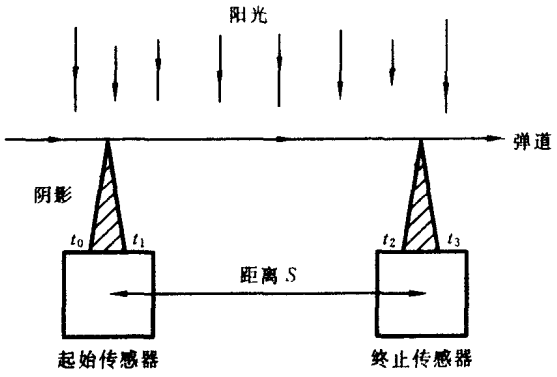


图 9.19　子弹等发射物经过起始、终止传感器的示意图

9.4.2　功能描述

1. 功能分析

由题意可知，弹道计时器的主要功能是测量出子弹等发射物穿过起始传感器和终止传感器之间的距离所需要的时间，并将该时间显示出来。因此，该计时器需要由方波信号发生器、控制电路、计数器和译码显示器等几个部分组成。控制电路收到起始传感器产生的信号 ST 后，在一定频率脉冲作用下启动计数器开始计数，收

到终止传感器产生的信号 SP 后令计数器停止计数。这样，计数器统计的脉冲数便直接对应子弹等发射物穿过起始传感器和终止传感器之间的距离所需要的时间。

2. 功能描述

根据设计要求，可构造出弹道计时器的结构框图，如图 9.20 所示。

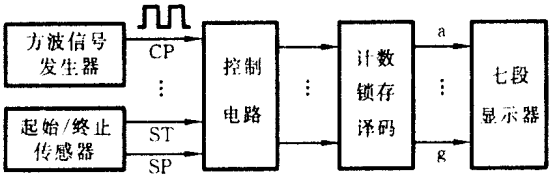


图 9.20 弹道计时器的结构框图

该弹道计时器的工作原理是：由方波信号发生器产生稳定的高频脉冲信号，作为计时基准；控制电路接收方波信号发生器产生的脉冲信号以及来自传感器的起始信号 ST 和终止信号 SP，输出计数、锁存、译码等所需要的控制信号；最后将计数器统计的脉冲数送显示器显示。

9.4.3 电路设计

由图 9.20 可知，整个电路由方波信号发生器，起始/终止传感器，控制电路，计数、锁存、译码电路和显示电路几部分组成。

1. 方波信号发生器设计

由于子弹等发射物的速度很快，因此，要求方波信号发生器能提供稳定的高频脉冲信号作为计数脉冲。为了保证频率的稳定度和精度，可选用石英晶体振荡器。图 9.21 所示为一个频率为 2MHz 方波信号发生器，它经触发器二分频后产生频率为 1MHz 方波信号作为计数脉冲，即为计数器提供周期为 1μs 的计数脉冲。

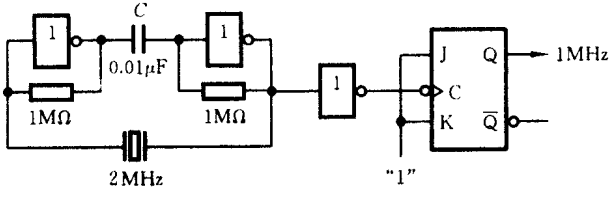


图 9.21 方波信号发生器

2. 控制电路设计

控制电路的功能是接收方波信号发生器产生的脉冲信号以及来自传感器的起始信号 ST 和终止信号 SP，输出计数、锁存、译码电路所需要的控制信号。假定计数、锁存、译码电路采用中规模集成电路 74C925，则控制电路应为 74C925 提供计数器复位信号 CRST、计数使能信号 CEN、计数器锁存信号 CLTCH。

控制电路何时发出复位信号 CRST？何时发出计数使能信号 CEN 和计数器锁存信号 CLTCH？显然取决于来自传感器的起始信号 ST 和终止信号 SP。由图 9.19 给出的示意图可知，在 t_0 时刻起始传感器将产生信号 ST，该信号在 t_1 时刻消失；在 t_2 时刻终止传感器将产生信号 SP，该信号在 t_3 时刻消失。因此，设计控制电路时可考虑在收到起始传感器发出的信号 ST 时，控制电路发出计数器复位信号 CRST；在起始信号 ST 消失时，控制电路发出计数器使能信号 CEN，该使能信号 CEN 一直维持到终止传感器产生的信号 SP 消失为止；当终止传感器产生的信号 SP 在 t_3 时刻消失时，在停止计数的同时，发出计数器锁存信号 CLTCH。此时，计数器统计的脉冲数便直接对应子弹等发射物穿过起始传感器和终止传感器之间的距离 S 所需要的时间。

根据上述分析，可作出控制器的状态图，如图 9.22(a)所示。图中，假定 A 为初始状态，B 状态对应图 9.19 中的 t_0 至 t_1 时刻，C 状态对应图 9.19 中的 t_1 至 t_2 时刻，D 状态对应图 9.19 中的 t_2 至 t_3 时刻。4 个状态需要 2 个触发器，设状态变量为 Q_2Q_1，并令 Q_2Q_1 取值 00 表示 A，01 表示 B，11 表示 C，10 表示 D，可得到控制器的二进制状态图，如图 9.22(b)所示。

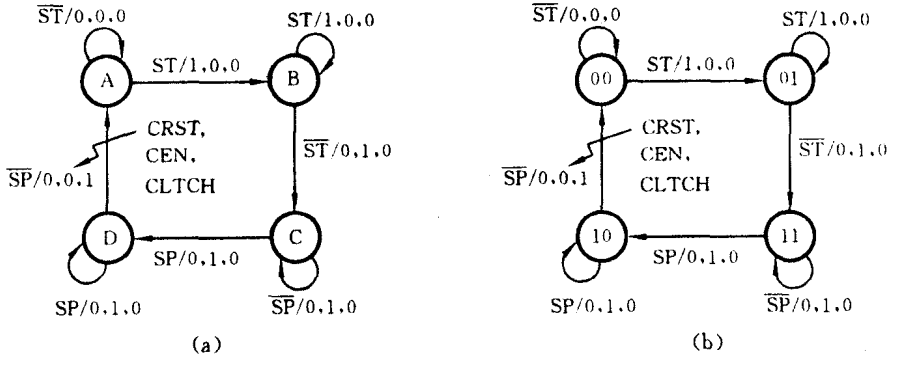


图 9.22　控制电路的状态图

假定采用 J-K 触发器作为控制电路的存储元件，可作出控制电路的激励函数、输出函数简化真值表，如表 9.3 所示。

表 9.3　控制电路的激励函数、输出函数简化真值表

现　态	输　入	次　态	激励函数	输出函数
Q_2Q_1	ST　SP	$Q_2^{n+1}Q_1^{n+1}$	$J_2\ K_2\ J_1\ K_1$	CRST　CEN　CLTCH
0 0	0　0	0　0	0　d　0　d	0　0　0
	1　0	0　1	0　d　1　d	1　0　0
0 1	1　0	0　1	0　d　d　0	1　0　0
	0　0	1　1	1　d　d　0	0　1　0
1 1	0　0	1　1	d　0　d　0	0　1　0
	0　1	1　0	d　0　d　1	0　1　0
1 0	0　1	1　0	d　0　0　d	0　1　0
	0　0	0　0	d　1　0　d	0　0　1

确定激励函数和输出函数表达式时，可以将每种状态下不会出现的输入取值作为无关条件考虑，充分利用无关最小项进行函数化简，可得到激励函数和输出函数的最简表达式如下：

$$J_2=\overline{ST}\cdot Q_1 \qquad K_2=\overline{SP}\cdot\overline{Q}_1 \qquad J_1=ST \qquad K_1=SP$$

$$CRST=ST \qquad CEN=Q_1\overline{ST}+Q_2SP \qquad CLTCH=Q_2\overline{Q}_1\overline{SP}$$

根据激励函数和输出函数表达式，可画出控制电路的逻辑电路，如图 9.23 所示。

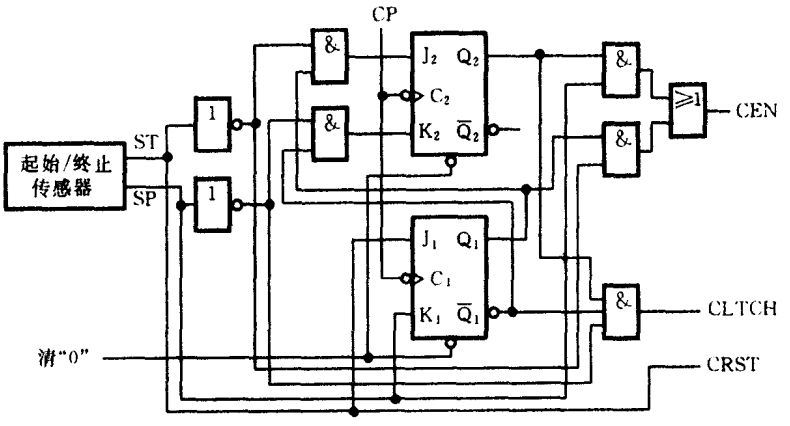


图 9.23　控制电路的逻辑电路

3. 计数、锁存、译码电路和显示电路

假定选用中规模集成电路 74C925 完成计数、锁存、译码功能，可将计数结果直接送七段显示器显示。

弹道计时器的完整电路如图 9.24 所示。

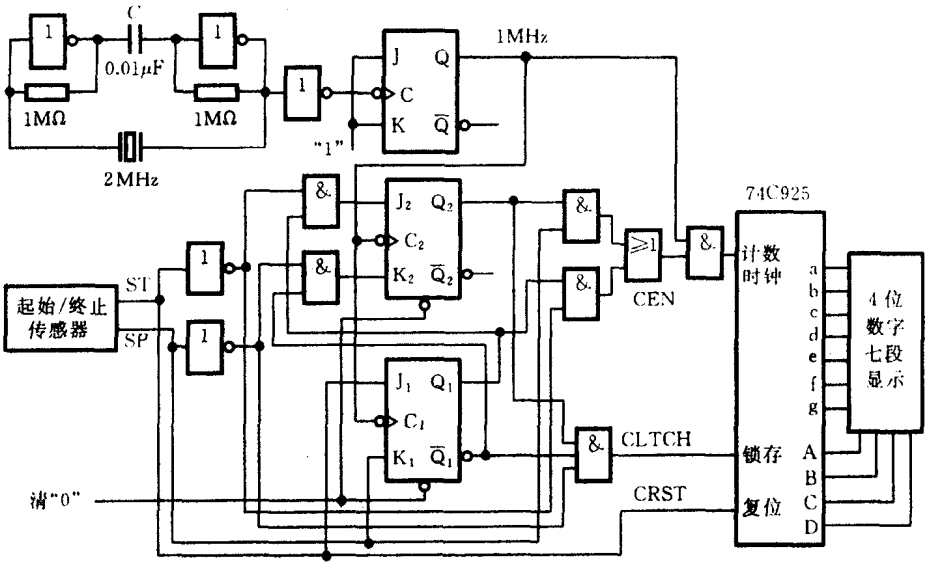


图 9.24　弹道计时器的完整电路

9.5　汽车尾灯控制器设计

9.5.1　设计要求

设计一个汽车尾灯控制器，实现对汽车尾灯显示状态的控制。在汽车尾部左右两侧各有 3 个指示灯(假定采用发光二极管模拟)，根据汽车运行情况，指示灯具有 4 种不同的显示模式：① 汽车正向行驶时，左右两侧的指示灯全部处于熄灭状态；② 汽车右转弯行驶时，右侧的 3 个指示灯按右循环顺序点亮；③ 汽车左转弯行驶时，左侧的 3 个指示灯按左循环顺序点亮；④ 汽车临时刹车时，左右两侧的指示灯同时处于闪烁状态。

9.5.2　功能描述

1. 汽车尾灯显示状态与汽车运行状态的关系

为了区分汽车尾灯的 4 种不同的显示模式，需设置 2 个状态控制变量。假定用开关 K_1 和 K_0 进行显示模式控制，可列出汽车尾灯显示状态与汽车运行状态的关系，如表 9.4 所示。

表 9.4 汽车尾灯显示状态与汽车运行状态的关系

控制变量 K_1 K_0	汽车运行状态	左侧的 3 个指示灯 D_{L1} D_{L2} D_{L3}	右侧的 3 个指示灯 D_{R1} D_{R2} D_{R3}
0 0	正向行驶	熄灭状态	熄灭状态
0 1	右转弯行驶	熄灭状态	按 $D_{R1}D_{R2}D_{R3}$ 顺序循环点亮
1 0	左转弯行驶	按 $D_{L3}D_{L2}D_{L1}$ 顺序循环点亮	熄灭状态
1 1	临时刹车	左右两侧的指示灯在时钟脉冲 CP 作用下同时闪烁	

2. 汽车尾灯控制器功能描述

在汽车左、右转弯行驶时，由于 3 个指示灯被循环顺序点亮，所以可用一个三进制计数器的状态控制译码器电路顺序输出高电平，按要求顺序点亮 3 个指示灯。设三进制计数器的状态用 Q_1 和 Q_0 表示，可得出描述指示灯 D_{L3}、D_{L2}、D_{L1}、D_{R3}、D_{R2}、D_{R1} 与开关控制变量 K_1、K_0，计数器的状态 Q_1、Q_0 以及时钟脉冲 CP 之间关系的功能表，如表 9.5 所示(表中指示灯的状态“1”表示点亮，“0”表示熄灭)。

表 9.5 汽车尾灯控制器功能表

控制变量 K_1 K_0	计数器状态 Q_1 Q_0	汽车尾灯 $D_{L1}D_{L2}D_{L3}$	汽车尾灯 $D_{R1}D_{R2}D_{R3}$
0 0	d d	0 0 0	0 0 0
0 1	0 0	0 0 0	1 0 0
	0 1	0 0 0	0 1 0
	1 0	0 0 0	0 0 1
1 0	0 0	0 0 1	0 0 0
	0 1	0 1 0	0 0 0
	1 0	1 0 0	0 0 0
1 1	d d	CP CP CP	CP CP CP

根据以上设计分析与功能描述，可得出汽车尾灯控制器的结构框图，如图9.25所示。整个电路可由模式控制电路、三进制计数器、译码与显示驱动电路、尾灯状

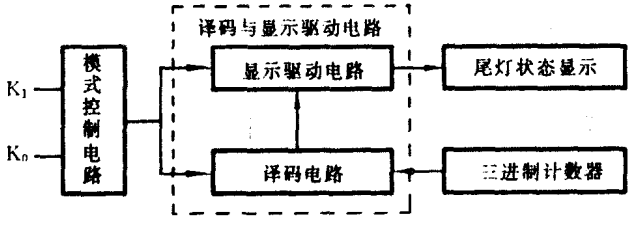


图 9.25 汽车尾灯控制器的结构框图

态显示4部分组成。

9.5.3 电路设计

1. 模式控制电路

设译码与显示驱动电路的使能控制信号为G和F，G与译码器74LS138的使能输入端G_1相连接，F与显示驱动电路中与非门的一个输入端相连接。由总体逻辑功能可知，G和F与开关控制变量K_1、K_0，以及时钟脉冲CP之间的关系如表9.6所示。

表9.6 使能控制信号与模式控制变量、时钟脉冲的关系

模式控制 K_1 K_0	时钟脉冲 CP	使能控制信号 G F	电路工作状态
0 0	d	0 1	汽车正向行驶(此时译码器不工作，译码器输出全部为高，显示驱动电路中的与非门输出均为低，反相器输出均为高，尾灯全部熄灭)
0 1	d	1 1	汽车右转弯行驶(此时译码器在计数器控制下工作，显示驱动电路中的与非门输出取决于译码器输出，右侧尾灯D_{R1} $D_{R2}D_{R3}$在译码器输出作用下顺序循环点亮)
1 0	d	1 1	汽车左转弯行驶(此时译码器在计数器控制下工作，显示驱动电路中的与非门输出取决于译码器输出，左侧尾灯D_{L1} $D_{L2}D_{L3}$在译码器输出作用下顺序循环点亮)
1 1	CP	0 CP	汽车临时刹车(此时译码器不工作，译码器输出全部为高，时钟脉冲CP通过显示驱动电路中的与非门作用到反相器输出端，使左右两侧的指示灯在时钟脉冲CP作用下同时闪烁)

根据表9.6所示关系，可求出使能控制信号G和F的逻辑表达式为

$$G = \overline{K}_1K_0 + K_1\overline{K}_0 = K_1 \oplus K_0$$

$$\begin{aligned} F &= \overline{K}_1\overline{K}_0 + \overline{K}_1K_0 + K_1\overline{K}_0 + K_1K_0CP \\ &= \overline{K}_1 + \overline{K}_0 + K_1K_0CP \\ &= \overline{K_1K_0} + K_1K_0CP \\ &= \overline{K_1K_0} + CP \\ &= \overline{K_1K_0\overline{CP}} \end{aligned}$$

根据G和F的逻辑表达式，可画出模式控制电路，如图9.26所示。

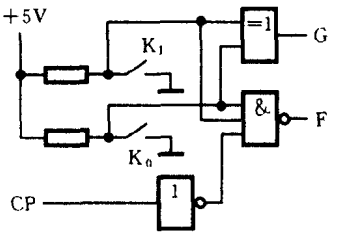


图9.26 模式控制电路

2. 三进制计数器

三进制计数器的状态表如表 9.7 所示。假定采用 J-K 触发器作为存储元件，则可设计出逻辑电路，如图 9.27 所示。

表 9.7 三进制计数器的状态表

现态 Q_1 Q_0	次态 $Q_1^{n+1}Q_0^{n+1}$
0 0	0 1
0 1	1 0
1 0	0 0
1 1	d d

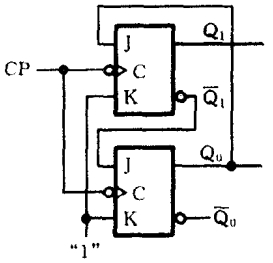


图 9.27 三进制计数器的逻辑电路

3. 译码与显示驱动电路

译码与显示驱动电路的功能是：在模式控制电路输出和三进制计数器状态的作用下，提供 6 个尾灯控制信号，当译码驱动电路输出的控制信号为低电平时，相应指示灯点亮。因此，译码与显示驱动电路可用 3-8 线译码器 74LS138、6 个与非门和 6 个反相器构成，逻辑电路如图 9.28 中的(Ⅰ)所示。图中，译码器 74LS138 的输入端 A_2、A_1、A_0 分别接 K_1、Q_1、Q_0。当图中 $G=F=1$、$K_1=0$ 时，对应计数器

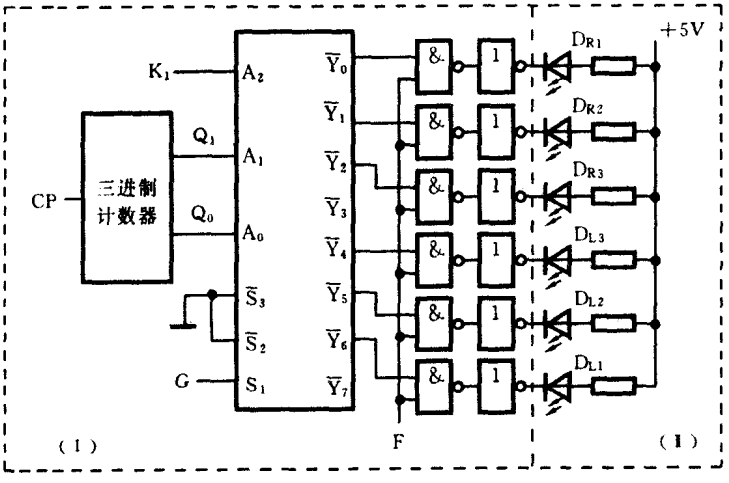


图 9.28 译码及尾灯状态显示、驱动电路

状态 Q_1Q_0 为 00、01、10，译码器输出 $\overline{Y}_0$ $\overline{Y}_1$ $\overline{Y}_2$ 依次为 0，使得与指示灯 D_{R1} D_{R2} D_{R3} 对应的反相器输出依次为低电平，从而使指示灯 $D_{R1}\rightarrow D_{R2}\rightarrow D_{R3}$ 依次顺序点亮，示意汽车右转弯；当图中 G=F=1、K_1=1 时，对应计数器状态 Q_1Q_0 为 00、01、10，译码器输出 $\overline{Y}_4$ $\overline{Y}_5$ $\overline{Y}_6$ 依次为 0，使与指示灯 D_{L3} D_{L2} D_{L1} 对应的反相器输出依次为低电平，从而使指示灯 $D_{L3}\rightarrow D_{L2}\rightarrow D_{L1}$ 依次顺序点亮，示意汽车左转弯；当图中 G=0，F=1 时，译码器输出为全 1，使所有指示灯对应的反相器输出全部为高电平，指示灯全部熄灭；当图中 G=0，F=CP 时，所有指示灯随 CP 的频率闪烁。实现了 4 种不同模式下的尾灯状态显示。

4. 尾灯状态显示电路

尾灯状态显示电路可由 6 个发光二极管和 6 个电阻组成，逻辑电路如图 9.28 中的(Ⅱ)所示。图中，当 6 个反相器的输出为低电平时，相应发光二极管被点亮。

在完成各个局部电路设计后，可得到汽车尾灯控制器的完整逻辑电路，如图 9.29 所示。

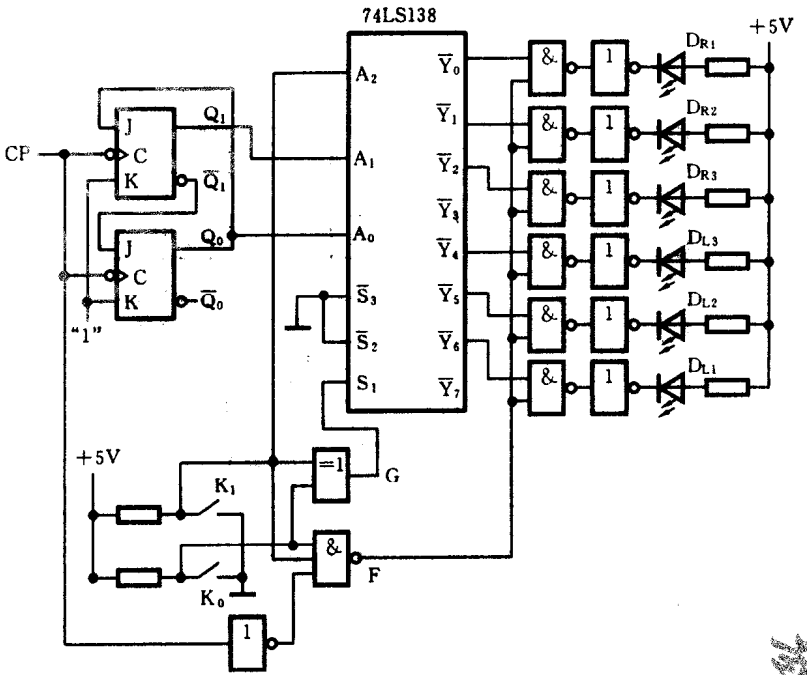


图 9.29　汽车尾灯控制器的逻辑电路

9.6 数字钟设计

9.6.1 设计要求

设计一个数字钟，电路具体要求如下：

① 信号发生器产生稳定的脉冲信号，作为数字钟的计时基准；

② 具有“时”、“分”、“秒”的十进制数字显示；

③ 小时计时以一昼夜为一个周期(即 24 进制)，分和秒计时为 60 进制；

④ 具有校时功能，可在任何时候将其调至标准时间或者指定时间。

9.6.2 功能描述

根据设计要求，可构造出数字钟电路的结构框图，如图 9.30 所示。

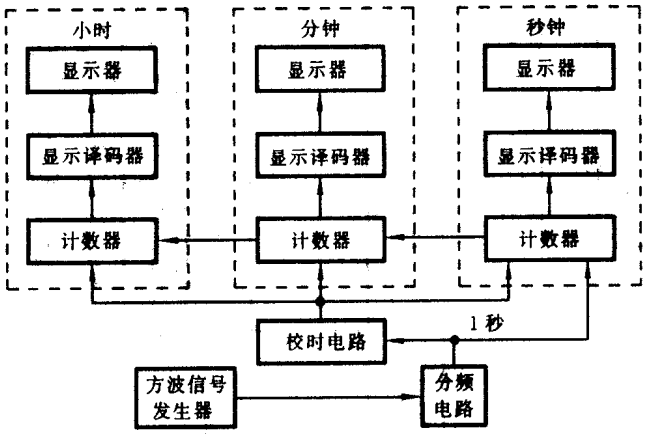


图 9.30 数字钟电路的结构框图

该数字钟电路的工作原理是：由方波信号发生器产生稳定的高频脉冲信号，经分频电路输出标准的秒脉冲信号，作为秒计时脉冲。秒计数器计满 60 后向分计数器产生进位脉冲，分计数器计满 60 后向时计数器产生进位脉冲，时计数器按照模 24 的规律计数。计数器的输出经显示译码器译码后送显示器显示。当电路计时出现误差时，可以由校时电路分别对“时”、“分”、“秒”进行校准。

9.6.3　电路设计

由数字钟电路的结构框图可知，整个电路由方波信号发生器、分频电路、计数显示电路和校时电路几部分组成。设计时可以首先分别完成各部分电路设计，然后再将各部分组装成一个整体。

1. 方波信号发生器设计

方波信号发生器是数字钟的一个重要组成部分，其稳定度及频率精度决定了数字钟计时的准确度。一般来说，方波信号发生器的频率越高，计时精度越高。方波信号发生器通常选用石英晶体构成的振荡器或者定时器 555 与 RC 组成的多谐振荡器。假定选用 555 构成的多谐振荡器，产生频率 $f=1\text{kHz}$ 的方波信号，则可设计出相应电路，如图 9.31 所示。

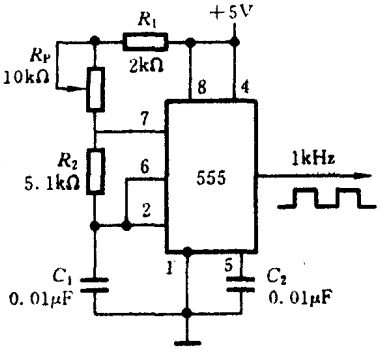


图 9.31　方波信号发生器

2. 分频电路设计

分频电路的功能是对方波信号发生器产生的方波信号进行分频处理，一方面形成计时所需的标准秒脉冲信号(1Hz 的方波信号)，另一方面提供数字钟功能扩充时所需的信号，如仿电视台正点报时用的 1kHz 的高音频信号和 500Hz 的低音频信号等。如图 9.32 所示，选用 3 片中规模集成电路 74LS90(十进制计数器)可以构成分频电路产生所需信号。图中，每片为 1/10 分频，第 1 片的 Q_A 端输出频率为 500Hz 的方波信号，Q_D 端输出频率为 100Hz 的方波信号；第 2 片的 Q_D 端输出频率为 10Hz 的方波信号；第 3 片的 Q_D 端输出频率为 1Hz 的方波信号。

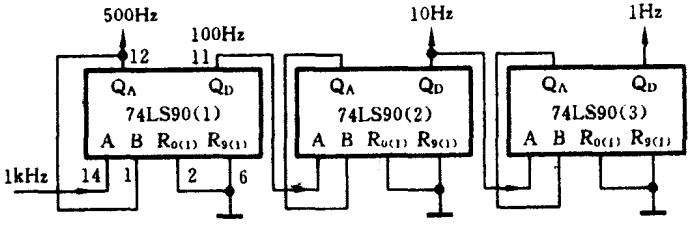


图 9.32　分频电路

3. 时、分、秒计数器设计

数字钟的“分”和“秒”计数器均为模 60 的计数器，其计数规律为 00→01→…→58→59→00… 可选用 74LS90 作为“分”和“秒”的个位和十位计数器，其中，十位计数器将 74LS90 连接成模 6 计数器。

数字钟的“时”计数器为模 24 的计数器，其计数规律为 00→01→…→22→23→00… 即当数字钟运行到 23 时 59 分 59 秒时，在下一个秒脉冲作用下，数字钟显示 00 时 00 分 00 秒。可选用 2 片 74LS90，将其连接成模 24 计数器作为“时”计数器。

根据分析可构造出时、分、秒计数器的逻辑电路，如图 9.33 所示。

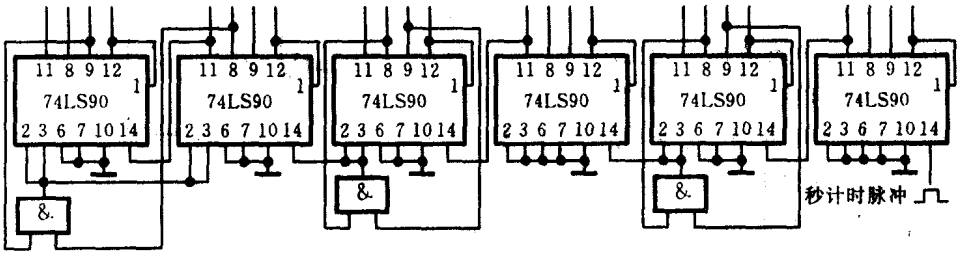


图 9.33 时、分、秒计数器的逻辑电路

4. 校时电路设计

当数字钟计时出现误差时，必须对时间进行校正，通常称为“校时”。校时是数字钟应该具备的基本功能，一般要求能对时、分、秒分别进行校对。

校时电路的设计要求是，在进行“时”校正时，不影响“分”和“秒”的正常计数；在进行“分”校正时，不影响“时”和“秒”的正常计数；在进行“秒”校正时，不影响“时”和“分”的正常计数。为此，可设置 K_3、K_2、K_1 3 个开关分别作为时、分、秒的校时控制开关。此外，根据时间误差的大小，可考虑两种校时方法：一种方法是采用单脉冲进行手动校时，拨动校时开关后，每按一次单脉冲按钮相应计数器增 1；另一种方法是利用秒计时脉冲进行自动校时，拨动校时开关后，在秒计时脉冲作用下相应计数器自动递增，直至增加到希望的值后再将校时开关拨回初始状态。设计时可设置开关 K_0 区分两种不同的校时方法。校时开关的功能如表 9.8 所示。

表 9.8 校时开关功能表

K_3	K_2	K_1	K_0	功　能
0	0	0	d	正常计数
0	0	1	0	秒校正(手动)
0	0	1	1	秒校正(自动)
0	1	0	0	分校正(手动)
0	1	0	1	分校正(自动)
1	0	0	0	时校正(手动)
1	0	0	1	时校正(自动)

根据功能表，可写出数字钟“时”、“分”、“秒”计数器个位计数脉冲信号的表达

式如下：

$$F_{时}=\overline{K}_3 \cdot P_{分十位进位脉冲}+K_3\overline{K}_0 \cdot P_{单脉冲}+K_3K_0 \cdot P_{秒计数脉冲}$$

$$F_{分}=\overline{K}_2 \cdot P_{秒十位进位脉冲}+K_2\overline{K}_0 \cdot P_{单脉冲}+K_2K_0 \cdot P_{秒计数脉冲}$$

$$F_{秒}=\overline{K}_1 \cdot P_{秒计数脉冲}+K_1\overline{K}_0 \cdot P_{单脉冲}+K_1K_0 \cdot P_{秒计数脉冲}$$

$$=(\overline{K}_1+K_0) \cdot P_{秒计数脉冲}+K_1\overline{K}_0 \cdot P_{单脉冲}$$

据此可设计出数字钟校时电路，如图 9.34 所示。

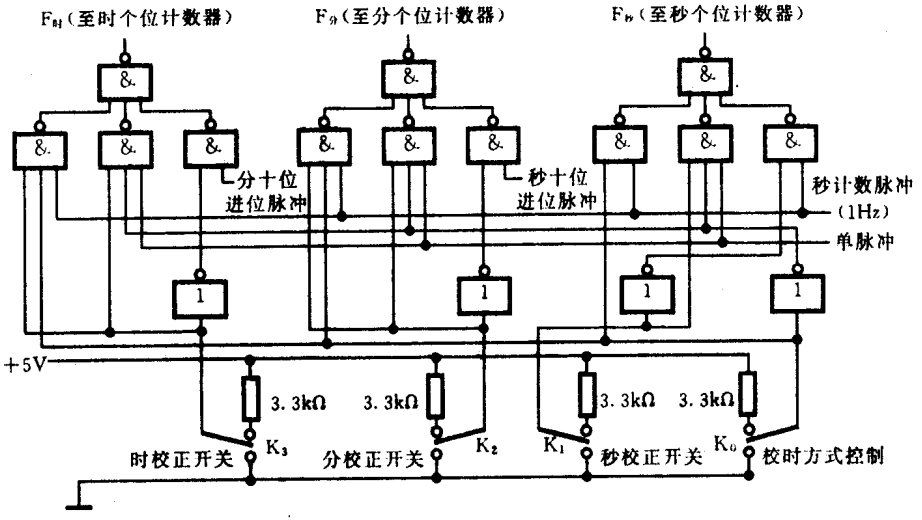


图 9.34　数字钟校时电路

5. 译码显示电路设计

译码显示电路的功能是将时、分、秒计数器输出的 4 位二进制码进行翻译后显示出相应的十进制数字。通常译码器与显示器是配套使用的，假定选择共阴极发光二极管数码显示器 BS202，则可选择译码驱动器 74LS48 与之配套使用。

完成各部分电路设计之后，按照图 9.30 所示总体框图中的信号流向，将各部分组装成一个整体，即可得到满足设计要求的数字钟电路，如图 9.35 所示。

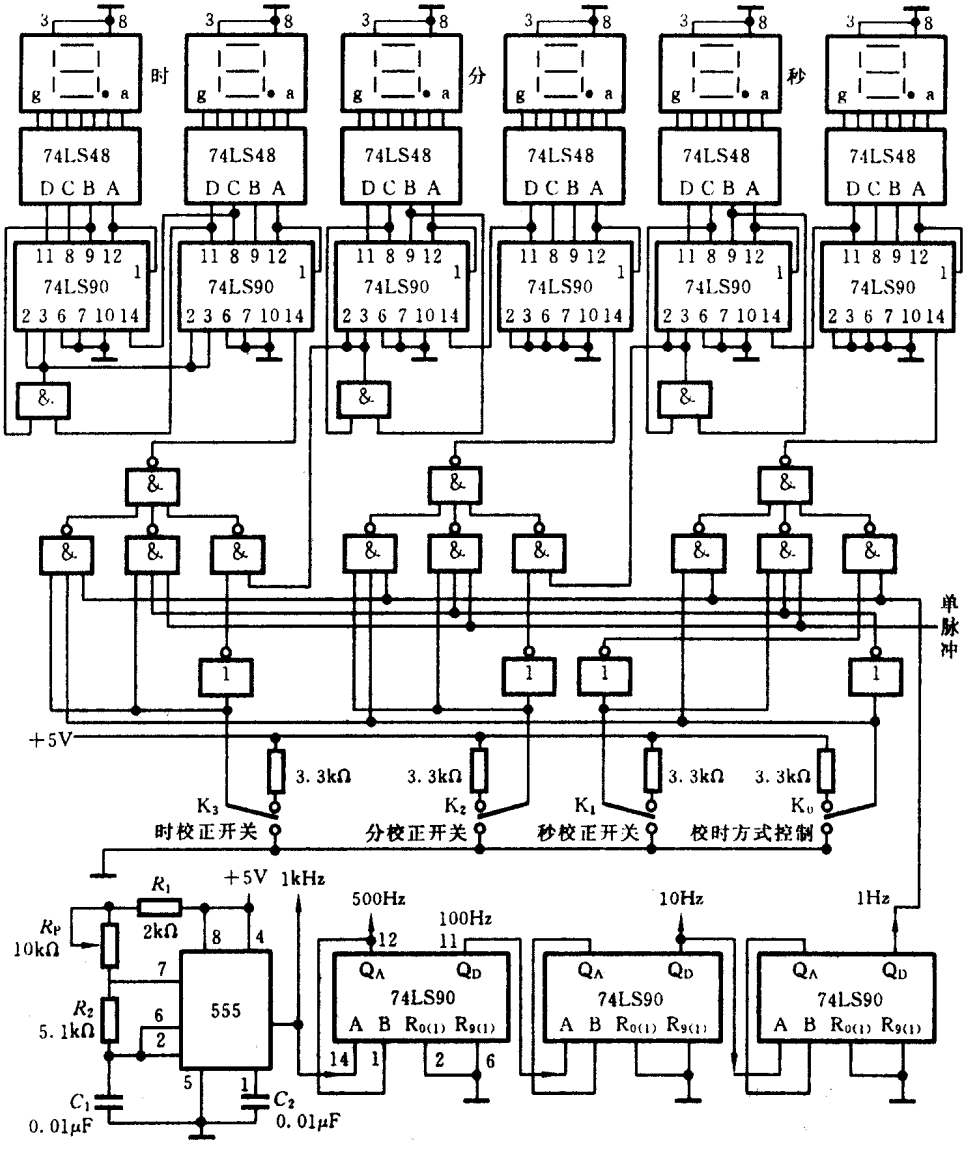

时
分
秒
74LS48
D C B A
11 8 9 12
74LS90
2 3 6 7 10 14
1
&
单脉冲
+5V
3.3kΩ
K3
时校正开关
K2
分校正开关
K1
秒校正开关
K0
校时方式控制
R1
2kΩ
RP
10kΩ
R2
5.1kΩ
555
C1
0.01μF
C2
0.01μF
+5V 1kHz
500Hz
100Hz
10Hz
1Hz
QA
QD
A B R0(1) R9(1)

图 9.35　数字钟电路

习 题 九

1. 设计一个数字密码锁电路，具体要求如下。

(1) 电路输入端接收用8421码表示的4位密码。当密码正确时，输出F为1，锁打开；否则，输出F为0，锁关闭。

(2) 8421码表示的4位密码由8个逻辑开关分时输入，先输入高2位，后输入低2位。

(3) 当输入密码错误时，亮红灯报警。

2. 设计一个红外线数据发送与接收电路，具体要求如下。

(1) 发送端首先预置并显示发送数据(两位十进制数的8421码)，然后串行发送出8位数据。

(2) 接收端串行接收8位数据并存入一个8位寄存器，然后用七段数码显示器显示接收数据的8421码。

(3) 比较发送与接收数据，确保数据可靠传送。

3. 设计一个洗衣机工作过程控制电路。假定洗衣机启动后，首先进入注水状态，一旦水的注入量达到要求，便产生一个工作过程启动信号，令其进入工作状态。工作过程控制电路的具体要求如下。

(1) 洗衣机的工作过程为：待机5秒→正转55秒→待机5秒→反转55秒。要求分别用指示灯和七段译码显示器显示洗衣机所处工作状态和相应的时间。

(2) 能够自行设定洗衣机的循环次数(范围：0～15)，重复执行上述工作过程。

(3) 能够处理紧急事件，当发生紧急情况时立即进入待机状态，直至紧急情况排除。

(4) 工作过程循环次数结束后，发出报警信号。

参考文献

1. 欧阳星明主编. 数字逻辑(第一版). 武汉:华中科技大学出版社,2000

2. 王毓银主编. 数字电路逻辑设计. 北京:高等教育出版社,1999

3. 欧阳星明主编. 数字系统逻辑设计. 北京:电子工业出版社,2004

4. 康华光主编. 电子技术基础(数字部分)(第四版). 北京:高等教育出版社,2000

5. 白中英主编. 数字逻辑与数字系统(第三版). 北京:科学出版社,2002

6. 李大友主编. 数字电路逻辑设计. 北京:清华大学出版社,1997

7. 王树堃,徐惠民编. 数字电路与逻辑设计. 北京:人民邮电出版社,1995

8. 欧阳星明主编. 数字逻辑学习与解题指南. 武汉:华中科技大学出版社,2000

9. 毛法尧,欧阳星明,任宏萍编. 数字逻辑. 武汉:华中理工大学出版社,1996

10. (美)韦克利(Wakerly,J. F.)著. 数字设计原理与实践. 林生等译. 北京:机械工业出版社,2003

11. Victor P Nelson, H Troy Nagle, Bill D Carroll, J David Irwin. Digital Logic Circuit Analysis & Design. Prentice-Hall, 1995

12. John M Yarbrough. Digital Logic: Applications and Design. PWS, 1997

13. Milos D Ercegovac, Tomas—Lang, Jaime H Moreno. Introduction to Digital System. John Wiley Son, 1998

14. Kenneth J,Breeding. Digital Design Fundamentals. 2nd ed. New Jersey: Prentice-Hall International Inc,1992

15. Wilkinson, Barry. Digital System Design. 2nd ed. New York: Prentice-Hall Inc,1992

16. Douglas A Pucknell. Fundamentals of Digital Logic Design: With VLSI Circuit Application. New York: Prentice Hall of Australia Pty Ltd, 1990

《数字逻辑》教学资源简介

主教材《数字逻辑》(第二版)

教辅教材《数字逻辑学习与解题指南》

本书是与《数字逻辑》主教材配套的学习指导用书。编写本书的目的是:更好地启发读者的逻辑思维能力;帮助读者加深对主教材内容的理解,提高分析问题和解决问题的能力。对应于主教材的各章分别按重点与难点、例题精选及自测练习三部分进行编写。全书共编入例题和自测练习题约500题,其中精选例题89题,所有自测练习题均附有解答。通过引例的分析、求解和归纳,总结了各类问题的解题规律、方法和技巧。

本书可供高等院校计算机、电信、自控等信息类专业的本、专科师生作为教学参考书,也是"数字逻辑"课程自学者的良好辅导教材。

互动型数字逻辑虚拟实验平台(IVDLEP)

IVDLEP是为"数字逻辑"课程实验教学而研制的软件系统,设计者用全新的创意和一流的技术营造了一个可供学生自由发挥的实验空间。

该系统不仅提供了实验所需的各种虚拟物理设备、器件和仪器,实现了对物理器件的自主装配、可视化操作、功能仿真、实验调试等功能,而且将互动的可视化操作贯穿于整个实验过程,能激发学生的学习兴趣和创作灵感,很好地满足了该课程实验教学的需要。在实验过程中,学生可以根据各自的创意构思出充分体现个性的设计方案,十分有利于学生创新能力和综合素质的培养。

多媒体CAI课件(教师版)

该课件采用PowerPoint和Flash制作,是供教师讲授"数字逻辑"课程使用的,涵盖了主教材《数字逻辑》的全部内容。课件以画面美观、文字简洁、图表清晰、重点突出、难点透彻、生动活泼为目标,实现了对文字、动画、图形、表格、公式等的有机结合;使用者可以根据不同的教学对象和学时数,对内容进行灵活取舍和修改,以满足不同的教学需求。

多媒体网络课件

该课件是供远程教育使用的，采用 FrontPage 和 Flash 制作，以网页阅读的形式进行教学，除涵盖了主教材《数字逻辑》的全部内容外，还提供了课程简介、知识要点、习题解答、实验演示、自评测试和参考书目等内容。本课件具有画面美观、导航清晰、链接正确、跳转灵活，取材丰富、重点突出、生动活泼的特点，十分有利于网络学院的学生自学。

多媒体视频课件

该课件采用最新的网络流媒体传播技术开发，采用教师视频讲解和板书展示相结合的网页形式进行教学，很好地模拟了课堂教学实况。用户可通过交互方式进行内容选择、进度控制等，十分适合远程教育使用。此外，为了满足教学的需要，该课件还被制作成光盘，供在单机环境下教学和自学使用。

数字逻辑测试题库

该测试题库中存放了各类数字逻辑试题，使用者不仅可以按照题例、内容、难度、分值等要求自动组卷、查阅答案，而且可以根据需要添加各类试题。既可供教师教学使用，又可供学生自测练习。

如果您需要上述教学资源，请与华中科技大学电子音像出版社联系：

联系电话：027-87556191

027-87548431

E-mail：hust-ep@hust-ep.com

hust-ep@163.com